



Stuart
Russell
Peter
Norvig



Artificial Intelligence

A Modern Approach

Fourth Edition

Trí tuệ nhân tạo Một cách tiếp cận hiện đại

Phiên bản Toàn cầu

Tái bản lần thứ Tư

Dòng sách của Pearson về Trí tuệ nhân tạo

Stuart Russell và Peter Norvig,

Biên tập viên

Forsyth & Ponce Computer Vision: A Modern Approach, 2nd ed. Graham ANSI Common Lisp
Jurafsky & Martin Speech and Language Processing, 2nd ed. Neapolitan Learning Bayesian
Networks Russell & Norvig Artificial Intelligence: A Modern Approach, 4th ed.

**Trí tuệ nhân tạo Một cách tiếp cận hiện đại Phiên bản Toàn cầu Tái bản lần thứ Tư

Stuart J. Russell và Peter Norvig**

**Cộng tác viên: Ming-Wei Chang Jacob Devlin Anca Dragan David Forsyth Ian Goodfellow
Jitendra M. Malik Vikash Mansinghka Judea Pearl Michael Wooldridge**

Hình ảnh trang bìa:

Alan Turing: Science History Images/Alamy Stock Photo; Tượng Aristotle: Panos
Karas/Shutterstock; Ada Lovelace – Pictorial Press Ltd/Alamy Stock Photo; Ô tô tự lái: Andrey
Suslov/Shutterstock; Robot Atlas: Boston Dynamics, Inc.; Tháp chuông Berkeley và Cầu Cổng
Vàng: Ben Chu/Shutterstock; Nền các nút mờ: Eugene Sergeev/Alamy Stock Photo; Bàn cờ vua
và quân cờ: Titania/Shutterstock; Xe tự hành trên sao Hỏa: Stocktrek Images, Inc./Alamy Stock
Photo; Kasparov: KATHY WILLENS/AP Images

Pearson Education Limited KAO Two KAO Park Hockham Way Harlow CM17 9SR Vương quốc
Anh

và các Công ty liên kết trên khắp thế giới

Hãy truy cập trang web của chúng tôi tại: www.pearsonglobaleditions.com

Vui lòng liên hệ <https://support.pearson.com/getsupport/s/contactsupport> nếu có bất kỳ thắc mắc
nào về nội dung này

© Pearson Education Limited 2022

Quyền của Stuart Russell và Peter Norvig được xác định là tác giả của tác phẩm này đã được họ
khẳng định theo Đạo luật Bản quyền, Thiết kế và Sáng chế năm 1988.

Chuyển thể được ủy quyền từ ấn bản tại Hoa Kỳ, có tựa đề Artificial Intelligence: A Modern Approach, Tái bản lần thứ 4, ISBN 978-0-13-461099-3 của Stuart J. Russell và Peter Norvig, do Pearson Education xuất bản © 2021.

Mọi quyền được bảo lưu. Không được sao chép, lưu trữ trong hệ thống truy xuất, hoặc truyền tải dưới bất kỳ hình thức nào hoặc bằng bất kỳ phương tiện nào, điện tử, cơ khí, photocopy, ghi âm hoặc cách khác, mà không có sự cho phép trước bằng văn bản của nhà xuất bản hoặc giấy phép cho phép sao chép hạn chế ở Vương quốc Anh do Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS cấp. Để biết thông tin về quyền, biểu mẫu yêu cầu và các liên hệ thích hợp trong bộ phận Quyền và Cấp phép Toàn cầu của Pearson Education, vui lòng truy cập www.pearsoned.com/permissions/.

Tất cả các nhãn hiệu được sử dụng trong tài liệu này là tài sản của chủ sở hữu tương ứng. Việc sử dụng bất kỳ nhãn hiệu nào trong văn bản này không trao cho tác giả hoặc nhà xuất bản bất kỳ quyền sở hữu nhãn hiệu nào đối với các nhãn hiệu đó, cũng như việc sử dụng các nhãn hiệu đó không ngụ ý bất kỳ sự liên kết nào với hoặc chứng thực cuốn sách này của các chủ sở hữu đó.

PEARSON, ALWAYS LEARNING, và MYLAB là các nhãn hiệu độc quyền tại Hoa Kỳ và/hoặc các quốc gia khác thuộc sở hữu của Pearson Education, Inc. hoặc các chi nhánh của nó.

Trừ khi được chỉ định khác trong tài liệu này, mọi nhãn hiệu của bên thứ ba có thể xuất hiện trong tác phẩm này là tài sản của chủ sở hữu tương ứng và mọi tham chiếu đến nhãn hiệu, logo hoặc trang phục thương mại của bên thứ ba chỉ nhằm mục đích minh họa hoặc mô tả. Các tham chiếu đó không nhằm ngụ ý bất kỳ sự tài trợ, chứng thực, ủy quyền hoặc quảng bá nào đối với các sản phẩm của Pearson bởi các chủ sở hữu của các nhãn hiệu đó, hoặc bất kỳ mối quan hệ nào giữa chủ sở hữu và Pearson Education, Inc. hoặc các chi nhánh, tác giả, người được cấp phép hoặc nhà phân phối của nó.

ISBN 10: 1-292-40113-3 ISBN 13: 978-1-292-40113-3 eBook ISBN 13: 978-1-292-40117-1

British Library Cataloguing-in-Publication Data Bản ghi danh mục cho cuốn sách này có sẵn từ BritishLibrary

Sắp chữ bởi SPi Global eBook được định dạng bởi B2R Technologies Pvt. Ltd.

Dành tặng Loy, Gordon, Lucy, George và Isaac — S.J.R. Dành tặng Kris, Isabella và Juliet — P.N.

Lời tựa

Trí tuệ nhân tạo (AI) là một lĩnh vực rộng lớn, và đây là một cuốn sách đồ sộ. Chúng tôi đã cố gắng khám phá toàn bộ bề rộng của lĩnh vực này, bao gồm logic, xác suất và toán học liên tục; nhận thức, lập luận, học tập và hành động; sự công bằng, tin cậy, lợi ích xã hội và an toàn; và các ứng dụng từ các thiết bị vi điện tử đến các thiết bị thăm dò hành tinh bằng robot đến các dịch vụ trực tuyến với hàng tỷ người dùng. Tên phụ của cuốn sách này là “Một cách tiếp cận hiện đại.” Điều đó có nghĩa là chúng tôi đã chọn kể câu chuyện từ một quan điểm hiện tại. Chúng tôi tổng hợp những gì hiện đã biết thành một khuôn khổ chung, trình bày lại các công trình ban đầu bằng cách sử dụng các ý tưởng và thuật ngữ phổ biến ngày nay. Chúng tôi xin lỗi những người mà các lĩnh vực phụ của họ, do đó, ít được nhận ra hơn.

Những điểm mới trong phiên bản này

Phiên bản này phản ánh những thay đổi trong AI kể từ phiên bản cuối cùng vào năm 2010: • Chúng tôi tập trung nhiều hơn vào **học máy** thay vì kỹ thuật tri thức thủ công, do sự sẵn có ngày càng tăng của dữ liệu, tài nguyên máy tính và các thuật toán mới. • **Học sâu, lập trình xác suất và hệ thống đa tác nhân** được mở rộng phạm vi, mỗi lĩnh vực có một chương riêng. • Phạm vi bao gồm **hiểu ngôn ngữ tự nhiên, robotics** và **thị giác máy tính** đã được sửa đổi để phản ánh tác động của học sâu. • Chương robotics hiện bao gồm các robot tương tác với con người và ứng dụng của **học tăng cường** vào robotics. • Trước đây, chúng tôi định nghĩa mục tiêu của AI là tạo ra các hệ thống cố gắng tối đa hóa **tiện ích kỳ vọng**, trong đó thông tin tiện ích cụ thể—mục tiêu—được cung cấp bởi những người thiết kế hệ thống. Giờ đây, chúng tôi không còn giả định rằng mục tiêu là cố định và được hệ thống AI biết; thay vào đó, hệ thống có thể không chắc chắn về các mục tiêu thực sự của con người mà nó hoạt động. Nó phải học cách tối đa hóa và phải hoạt động phù hợp ngay cả khi không chắc chắn về mục tiêu. • Chúng tôi tăng phạm vi bao gồm tác động của AI đối với xã hội, bao gồm các vấn đề quan trọng về **đạo đức, công bằng, tin cậy và an toàn**. • Chúng tôi đã chuyển các bài tập từ cuối mỗi chương sang một trang web trực tuyến. Điều này cho phép chúng tôi liên tục thêm, cập nhật và cải thiện các bài tập, để đáp ứng nhu cầu của người hướng dẫn và để phản ánh những tiến bộ trong lĩnh vực và trong các công cụ phần mềm liên quan đến AI. • Nhìn chung, khoảng 25% tài liệu trong sách là hoàn toàn mới. 75% còn lại đã được viết lại phần lớn để trình bày một bức tranh thống nhất hơn về lĩnh vực này. 22% các trích dẫn trong ấn bản này là các công trình được xuất bản sau năm 2010.

Tổng quan về sách

Chủ đề thống nhất chính là ý tưởng về **tác nhân thông minh**. Chúng tôi định nghĩa AI là nghiên cứu về các tác nhân nhận **nhận thức** từ môi trường và thực hiện **hành động**. Mỗi tác nhân như vậy thực hiện một hàm ánh xạ các chuỗi nhận thức thành các hành động, và chúng tôi đề cập đến các cách khác nhau để biểu diễn các hàm này, chẳng hạn như **các tác nhân phản ứng, các bộ lập kế hoạch thời gian thực, các hệ thống lý thuyết quyết định, và các hệ thống học sâu**. Chúng tôi nhấn mạnh việc học vừa là một phương pháp xây dựng các hệ thống có năng lực vừa là một cách để mở rộng phạm vi của nhà thiết kế vào các môi trường chưa biết. Chúng tôi coi robotics và thị giác không phải là các vấn đề được định nghĩa độc lập, mà là xảy ra trong quá trình đạt được các mục tiêu. Chúng tôi nhấn mạnh tầm quan trọng của môi trường tác vụ trong việc xác định thiết kế tác nhân phù hợp. Mục đích chính của chúng tôi là truyền đạt các ý tưởng đã xuất hiện trong bảy mươi năm nghiên cứu AI vừa qua và hai thiên niên kỷ công việc liên quan vừa qua. Chúng tôi đã cố gắng tránh sự quá trang trọng trong việc trình bày các ý tưởng này, trong khi vẫn giữ sự chính xác. Chúng tôi đã bao gồm các công thức toán học và các thuật toán mã giả để làm cho các ý tưởng chính trở nên cụ thể; các khái niệm toán học và ký hiệu được mô tả trong Phụ lục A và mã giả của chúng tôi được mô tả trong Phụ lục B. Cuốn sách này chủ yếu dành cho việc sử dụng trong một khóa học hoặc chuỗi khóa học đại học. Cuốn sách có 29 chương, mỗi chương yêu cầu khoảng một tuần học, vì vậy việc học qua toàn bộ cuốn sách đòi hỏi một chuỗi hai học kỳ. Một khóa học một học kỳ có thể sử dụng các chương được chọn để phù hợp với sở thích của người hướng dẫn và sinh viên. Cuốn sách cũng có thể được sử dụng trong một khóa học sau đại học (có thể với việc bổ sung một số nguồn chính được đề xuất trong các ghi chú thư mục), hoặc để tự học hoặc làm tài liệu tham khảo. ► Xuyên suốt cuốn sách, các điểm quan trọng được đánh dấu bằng một biểu tượng hình tam giác ở lề. Bất cứ khi nào một thuật ngữ mới được định nghĩa, nó cũng được ghi chú ở lề. Các lần sử dụng quan trọng tiếp theo của thuật ngữ đó được in đậm, nhưng

không ở lẻ. Chúng tôi đã bao gồm một chỉ mục toàn diện và một thư mục mở rộng. Điều kiện tiên quyết duy nhất là quen thuộc với các khái niệm cơ bản về khoa học máy tính (thuật toán, cấu trúc dữ liệu, độ phức tạp) ở cấp độ năm thứ hai. Freshman calculus và đại số tuyến tính rất hữu ích cho một số chủ đề.

Tài nguyên trực tuyến

Các tài nguyên trực tuyến có sẵn thông qua pearsonglobaleditions.com. Ở đó bạn sẽ tìm thấy:

- Bài tập, dự án lập trình và dự án nghiên cứu. Chúng không còn ở cuối mỗi chương nữa; chúng chỉ có trên mạng. Trong sách, chúng tôi đề cập đến một bài tập trực tuyến bằng một tên như “Bài tập 6.NARY.” Hướng dẫn trên trang web cho phép bạn tìm các bài tập theo tên hoặc theo chủ đề.
- Các triển khai của các thuật toán trong sách bằng Python, Java và các ngôn ngữ lập trình khác.
- Tài liệu bổ sung và liên kết cho sinh viên và người hướng dẫn.
- Hướng dẫn về cách báo cáo lỗi trong sách trong trường hợp có thể có một số lỗi tồn tại.

Trang bìa sách

Trang bìa mô tả vị trí cuối cùng từ ván 6 quyết định của trận đấu cờ vua năm 1997, trong đó chương trình Deep Blue đã đánh bại Garry Kasparov (chơi quân Đen), đánh dấu đây là lần đầu tiên một máy tính đánh bại một nhà vô địch thế giới trong một trận đấu cờ vua. Kasparov được hiển thị ở trên cùng. Bên phải ông là một vị trí then chốt từ ván thứ hai của trận đấu cờ vây lịch sử giữa cựu vô địch thế giới Lee Sedol và chương trình ALPHAGO của DeepMind. Nước 37 của ALPHAGO đã vi phạm hàng thế kỷ chính thống của cờ vây và ngay lập tức được các chuyên gia con người coi là một sai lầm đáng xấu hổ, nhưng hóa ra đó lại là một nước đi chiến thắng. Ở phía trên bên trái là một robot hình người Atlas do Boston Dynamics chế tạo. Một hình ảnh mô tả một chiếc xe tự lái cảm nhận môi trường của nó xuất hiện giữa Ada Lovelace, lập trình viên máy tính đầu tiên trên thế giới và Alan Turing, người có công trình nền tảng đã định nghĩa trí tuệ nhân tạo. Ở dưới cùng của bàn cờ vua là một robot thám hiểm sao Hỏa và một bức tượng của Aristotle, người đã tiên phong nghiên cứu về logic; thuật toán lập kế hoạch của ông từ De Motu Animalium xuất hiện sau tên của các tác giả. Phía sau bàn cờ vua là một mô hình lập trình xác suất được Tổ chức Hiệp ước Cẩm Thử nghiệm Hạt nhân Toàn diện của Liên Hợp Quốc sử dụng để phát hiện các vụ nổ hạt nhân từ các tín hiệu địa chấn.

Lời cảm ơn

Cần cả một làng toàn cầu để làm ra một cuốn sách. Hơn 600 người đã đọc các phần của cuốn sách và đưa ra các đề xuất để cải thiện. Danh sách đầy đủ có tại pearsonglobaleditions.com; chúng tôi biết ơn tất cả họ. Chúng tôi chỉ có không gian ở đây để đề cập đến một vài cộng tác viên đặc biệt quan trọng. Đầu tiên là các cộng tác viên viết: • Judea Pearl (Mục 13.5, Mạng lưới nhân quả); • Michael Wooldridge (Chương 17, Ra quyết định đa tác nhân); • Vikash Mansinghka (Mục 18.4, Các chương trình dưới dạng mô hình xác suất); • Ian Goodfellow (Chương 22, Học sâu); • Jacob Devlin và Mei-Wing Chang (Chương 25, Học sâu cho xử lý ngôn ngữ tự nhiên); • Anca Dragan (Chương 26, Robotics); • Jitendra Malik và David Forsyth (Chương 27, Thị giác máy tính). Sau đó là một số vai trò quan trọng: • Cynthia Yeung và Malika Cantor (quản lý dự án); • Julie Sussman và Tom Galloway (hiệu đính và đề xuất viết); • Omari Stephens (minh họa); • Tracy Johnson (biên tập viên); • Erin Ault và Rose Kernan (trang bìa và chuyển đổi màu); • Nalin Chhibber, Sam Goto, Raymondde Lacaze, Ravi Mohan, Ciaran O'Reilly, Amit Patel, Dragomir Radiv và Samagra Sharma (phát triển và cố vấn mã trực tuyến); • Sinh viên Google Summer of Code (phát triển mã

trực tuyến). Stuart muốn cảm ơn vợ mình, Loy Sheflott, vì sự kiên nhẫn vô tận và sự khôn ngoan vô biên của cô ấy. Ông hy vọng rằng Gordon, Lucy, George, và Isaac sẽ sớm đọc cuốn sách này sau khi họ đã tha thứ cho ông vì đã làm việc quá lâu trên đó. RUGS (Russell's Unusual Group of Students) đã đặc biệt hữu ích, như mọi khi. Peter muốn cảm ơn cha mẹ mình (Torsten và Gerda) vì đã giúp ông bắt đầu, và vợ mình (Kris), các con (Bella và Juliet), đồng nghiệp, sếp và bạn bè vì đã khuyến khích và chịu đựng ông qua những giờ viết và viết lại dài.

Về các Tác giả

Stuart Russell sinh năm 1962 tại Portsmouth, Anh. Ông nhận bằng B.A. hạng ưu trong vật lý từ Đại học Oxford năm 1982, và bằng tiến sĩ khoa học máy tính từ Stanford năm 1986. Sau đó, ông gia nhập khoa của Đại học California tại Berkeley, nơi ông là giáo sư và cựu chủ tịch khoa học máy tính, giám đốc của Trung tâm AI tương thích với con người, và giữ chức Smith–Zadeh Chair in Engineering. Năm 1990, ông nhận giải thưởng Presidential Young Investigator Award của Quỹ Khoa học Quốc gia, và năm 1995, ông đồng giải thưởng Computers and Thought Award. Ông là thành viên của Hiệp hội Trí tuệ Nhân tạo Hoa Kỳ, Hiệp hội Máy tính, và Hiệp hội Tiến bộ Khoa học Hoa Kỳ, một Thành viên Danh dự của Wadham College, Oxford, và một Andrew Carnegie Fellow. Ông giữ chức Chaire Blaise Pascal ở Paris từ năm 2012 đến 2014. Ông đã xuất bản hơn 300 bài báo về nhiều chủ đề trong trí tuệ nhân tạo. Các cuốn sách khác của ông bao gồm *The Use of Knowledge in Analogy and Induction*, *Do the Right Thing: Studies in Limited Rationality* (với Eric Wefald), và *Human Compatible: Artificial Intelligence and the Problem of Control*. Peter Norvig hiện là Giám đốc Nghiên cứu tại Google, Inc., và trước đây là giám đốc phụ trách các thuật toán tìm kiếm Web cốt lõi. Ông đã đồng giảng dạy một lớp học AI trực tuyến đã đăng ký 160.000 sinh viên, giúp khởi động vòng các lớp học trực tuyến mở rộng lớn hiện tại. Ông là người đứng đầu Phòng Khoa học Tính toán tại Trung tâm Nghiên cứu NASA Ames, giám sát nghiên cứu và phát triển về trí tuệ nhân tạo và robotics. Ông nhận bằng B.S. về toán ứng dụng từ Đại học Brown và bằng tiến sĩ khoa học máy tính từ Berkeley. Ông đã từng là giáo sư tại Đại học Nam California và là thành viên khoa tại Berkeley và Stanford. Ông là Thành viên của Hiệp hội Trí tuệ Nhân tạo Hoa Kỳ, Hiệp hội Máy tính, Học viện Nghệ thuật và Khoa học Hoa Kỳ, và Học viện Khoa học California. Các cuốn sách khác của ông là *Paradigms of AI Programming: Case Studies in Common Lisp*, *Verbmobil: A Translation System for Face-to-Face Dialog*, và *Intelligent Help Systems for UNIX*. Hai tác giả đã chia sẻ giải thưởng AAAI/EAAI Outstanding Educator đầu tiên vào năm 2016.

Mục lục

.....	1
Lời tựa.....	3
Những điểm mới trong phiên bản này	4
Tổng quan về sách	4
Tài nguyên trực tuyến	5
Trang bìa sách	5
Lời cảm ơn	5
Về các Tác giả.....	6
CHƯƠNG 1 GIỚI THIỆU.....	10
1.1 Trí tuệ nhân tạo là gì?	10
1.1.1 Hoạt động như con người: Cách tiếp cận theo bài kiểm tra Turing.....	10
1.1.2 Suy nghĩ như con người: Cách tiếp cận mô hình nhận thức	11
1.1.3 Suy nghĩ một cách hợp lý: Cách tiếp cận “các quy luật của tư duy”	12
1.1.4 Hành động một cách hợp lý: Cách tiếp cận tác nhân hợp lý.....	13
1.1.5 Máy móc hữu ích	14
1.2 Các Nền Tảng của Trí tuệ Nhân tạo.....	15
1.2.1 Triết học	15
1.2.2 Toán học.....	18
1.2.3 Kinh tế học.....	19
1.2.4 Khoa học thần kinh	20
1.2.5 Tâm lý học	22
1.2.6 Kỹ thuật máy tính.....	24
1.2.7 Lý thuyết điều khiển và điều khiển học	26
1.2.8 Ngôn ngữ học.....	27
1.3 Lịch sử của Trí tuệ Nhân tạo.....	27
1.3.1 Sự khởi đầu của trí tuệ nhân tạo (1943–1956).....	27
1.3.2 Sự nhiệt tình ban đầu, những kỳ vọng lớn (1952–1969)	29
1.3.3 Một liều thuốc thực tế (1966–1973)	31
1.3.4 Hệ thống chuyên gia (1969–1986).....	33
1.3.5 Sự trở lại của mạng thần kinh (1986–nay).....	35
1.3.6 Suy luận xác suất và học máy (1987–nay).....	35
1.3.7 Dữ liệu lớn (2001–nay).....	37
1.3.8 Học sâu (2011–nay)	37

1.4 Tình hình hiện tại	38
1.5 Risks and Benefits of AI	42
Tóm tắt	46
Ghi chú thư mục và lịch sử	47
CHAPTER 2: INTELLIGENT AGENTS	48
2.1 Tác nhân và Môi trường.....	48
2.2 Hành vi tốt: Khái niệm về tính hợp lý	51
2.3 Bản chất của Môi trường	54
2.3.2 Các thuộc tính của môi trường tác vụ	56
2.4 Cấu trúc của tác nhân.....	60
2.4.1 Các chương trình tác nhân	61
2.4.2 Tác nhân phản xạ đơn giản	63
2.4.3 Tác nhân phản xạ dựa trên mô hình.....	65
2.4.4 Tác nhân dựa trên mục tiêu.....	67
2.4.5 Tác nhân dựa trên tiện ích.....	68
2.4.6 Tác nhân học tập	70
2.4.7 Cách hoạt động của các thành phần trong chương trình tác nhân	72
Tóm tắt	74
Ghi chú Thư mục và Lịch sử	75
CHƯƠNG 3: GIẢI QUYẾT VẤN ĐỀ BẰNG CÁCH TÌM KIẾM	78
3.1 Tác nhân giải quyết vấn đề	78
3.1.1 Vấn đề tìm kiếm và giải pháp	80
3.1.2 Xây dựng vấn đề	81
3.2 Ví dụ về các vấn đề.....	81
3.2.1 Vấn đề tiêu chuẩn hóa.....	82
3.2.2 Vấn đề thể giới thực.....	85
3.3 Thuật toán tìm kiếm	86
3.3.1 Tìm kiếm tốt nhất đầu tiên	89
3.3.2 Cấu trúc dữ liệu tìm kiếm	89
3.3.3 Các đường dẫn dư thừa	90
3.3.4 Đo lường hiệu suất giải quyết vấn đề	91
3.4 Chiến lược tìm kiếm không có thông tin	92
3.4.1 Tìm kiếm theo chiều rộng đầu tiên	93
3.4.2 Thuật toán của Dijkstra hoặc tìm kiếm chi phí đồng đều	94

3.4.3 Tìm kiếm theo chiều sâu đầu tiên và vấn đề bộ nhớ.....	96
3.4.4 Tìm kiếm giới hạn độ sâu và tìm kiếm lặp sâu dần	97
3.4.5 Tìm kiếm hai chiều	100
3.4.6 So sánh các thuật toán tìm kiếm không có thông tin	101
3.5 Chiến lược tìm kiếm có thông tin (Heuristic)	102
3.5.1 Tìm kiếm tốt nhất đầu tiên tham lam	103
3.5.2 Tìm kiếm A*	104
3.5.3 Các đường đồng mức tìm kiếm.....	109
3.5.4 Tìm kiếm thỏa mãn: Heuristic không chấp nhận được và A* có trọng số	110
3.5.5 Tìm kiếm có giới hạn bộ nhớ.....	112
3.5.6 Tìm kiếm heuristic hai chiều.....	117
3.6 Các hàm Heuristic	118
3.6.1 Ảnh hưởng của độ chính xác heuristic đến hiệu suất	119
3.6.2 Tạo Heuristic từ các bài toán đã được nói lỏng	121
3.6.3 Tạo Heuristic từ các bài toán con: Cơ sở dữ liệu mẫu.....	122
3.6.4 Tạo Heuristic bằng các điểm mốc.....	123
3.6.5 Học để tìm kiếm tốt hơn	125
3.6.6 Học Heuristic từ kinh nghiệm.....	126
Tóm tắt	126
Ghi chú Thư mục và Lịch sử	128

CHƯƠNG 1 GIỚI THIỆU

Chúng ta tự gọi mình là **Homo sapiens** – người thông thái – bởi vì trí thông minh rất quan trọng đối với chúng ta. Hàng ngàn năm qua, chúng ta đã cố gắng tìm hiểu cách mình suy nghĩ và hành động – tức là, bằng cách nào mà bộ não của chúng ta, một khối vật chất nhỏ bé, có thể nhận thức, thấu hiểu, dự đoán và điều khiển một thế giới rộng lớn và phức tạp hơn nhiều so với chính nó. Lĩnh vực **trí tuệ nhân tạo** (AI) không chỉ quan tâm đến việc tìm hiểu mà còn xây dựng các thực thể thông minh – những cỗ máy có thể tính toán cách hành động hiệu quả và an toàn trong nhiều tình huống mới lạ.

Các cuộc khảo sát thường xuyên xếp hạng AI là một trong những lĩnh vực thú vị và phát triển nhanh nhất, và nó đã tạo ra doanh thu hơn một nghìn tỉ đô la mỗi năm. Chuyên gia AI Kai-Fu Lee dự đoán tác động của nó sẽ “lớn hơn bất cứ điều gì trong lịch sử loài người”. Hơn nữa, các biên giới trí tuệ của AI vẫn còn rộng mở. Trong khi một sinh viên của một ngành khoa học cũ hơn như vật lý có thể cảm thấy những ý tưởng hay nhất đã được khám phá bởi Galileo, Newton, Curie, Einstein và những người khác, thì AI vẫn còn nhiều cơ hội cho những bộ óc vĩ đại.

Hiện tại, AI bao gồm rất nhiều các lĩnh vực con, từ những lĩnh vực tổng quát (học hỏi, suy luận, nhận thức, v.v.) đến những lĩnh vực cụ thể, chẳng hạn như chơi cờ, chứng minh các định lý toán học, viết thơ, lái xe hơi, hoặc chẩn đoán bệnh tật. AI có liên quan đến bất kỳ nhiệm vụ trí tuệ nào; nó thực sự là một lĩnh vực phổ quát.

1.1 Trí tuệ nhân tạo là gì?

Chúng ta đã khẳng định rằng AI rất thú vị, nhưng chúng ta chưa nói nó là gì. Trong lịch sử, các nhà nghiên cứu đã theo đuổi một số phiên bản AI khác nhau. Một số đã định nghĩa trí thông minh dựa trên sự giống với hiệu suất của con người, trong khi những người khác lại ưa thích một định nghĩa trí tuệ trừu tượng, chính thức được gọi là **tính hợp lý** (rationality) – nói một cách dễ hiểu là làm “điều đúng đắn”. Chủ đề nghiên cứu cũng khác nhau: một số coi trí thông minh là một thuộc tính của quá trình tư duy và suy luận bên trong, trong khi những người khác tập trung vào hành vi thông minh, một đặc điểm bên ngoài. Từ hai chiều – **con người so với hợp lý** và **tư duy so với hành vi** – có bốn sự kết hợp khả thi, và đã có những người ủng hộ và các chương trình nghiên cứu cho cả bốn.

Các phương pháp được sử dụng nhất thiết phải khác nhau: việc theo đuổi trí thông minh giống con người phải là một phần của khoa học thực nghiệm liên quan đến tâm lý học, bao gồm các quan sát và giả thuyết về hành vi và quá trình tư duy thực tế của con người; mặt khác, một cách tiếp cận duy lý bao gồm sự kết hợp giữa toán học và kỹ thuật, đồng thời kết nối với thống kê, lý thuyết điều khiển và kinh tế học. Các nhóm khác nhau vừa chê bai vừa hỗ trợ lẫn nhau. Hãy cùng xem xét chi tiết hơn bốn cách tiếp cận này.

1.1.1 Hoạt động như con người: Cách tiếp cận theo bài kiểm tra Turing

Bài kiểm tra Turing, được đề xuất bởi Alan Turing (1950), được thiết kế như một thí nghiệm tư duy nhằm bỏ qua sự mơ hồ về triết học của câu hỏi “Liệu một cỗ máy có thể suy nghĩ không?”.

Một máy tính vượt qua bài kiểm tra nếu một người thẩm vấn, sau khi đặt ra một số câu hỏi bằng văn bản, không thể phân biệt được câu trả lời bằng văn bản đó đến từ một người hay từ một máy tính. Chương 28 sẽ thảo luận chi tiết về bài kiểm tra và liệu một máy tính có thực sự thông minh nếu nó vượt qua. Hiện tại, chúng ta lưu ý rằng việc lập trình một máy tính để vượt qua một bài kiểm tra được áp dụng nghiêm ngặt sẽ cung cấp rất nhiều việc phải làm. Máy tính sẽ cần các khả năng sau:

- **Xử lý ngôn ngữ tự nhiên** để giao tiếp thành công bằng ngôn ngữ của con người;
- **Biểu diễn tri thức** để lưu trữ những gì nó biết hoặc nghe thấy;
- **Suy luận tự động** để trả lời các câu hỏi và đưa ra những kết luận mới;
- **Học máy** để thích ứng với các tình huống mới và phát hiện, ngoại suy các quy luật.

Turing cho rằng việc mô phỏng vật lý một người là không cần thiết để chứng minh trí thông minh. Tuy nhiên, các nhà nghiên cứu khác đã đề xuất một **bài kiểm tra Turing toàn diện**, yêu cầu tương tác với các đối tượng và con người trong thế giới thực. Để vượt qua bài kiểm tra Turing toàn diện, một robot sẽ cần:

- **Thị giác máy tính** và **nhận dạng giọng nói** để nhận thức thế giới;
- **Người máy (Robotics)** để điều khiển các vật thể và di chuyển xung quanh.

Sáu lĩnh vực này chiếm phần lớn AI. Tuy nhiên, các nhà nghiên cứu AI đã dành rất ít nỗ lực để vượt qua bài kiểm tra Turing, tin rằng việc nghiên cứu các nguyên tắc cơ bản của trí thông minh quan trọng hơn. Cuộc tìm kiếm “chuyến bay nhân tạo” đã thành công khi các kỹ sư và nhà phát minh ngừng bắt chước loài chim và bắt đầu sử dụng ống gió, đồng thời tìm hiểu về khí động học. Các văn bản kỹ thuật hàng không không định nghĩa mục tiêu của lĩnh vực này là tạo ra “những cỗ máy bay giống hệt chim bồ câu đến mức có thể đánh lừa cả những con chim bồ câu khác”.

1.1.2 Suy nghĩ như con người: Cách tiếp cận mô hình nhận thức

Để nói rằng một chương trình suy nghĩ giống con người, chúng ta phải biết con người suy nghĩ như thế nào. Chúng ta có thể tìm hiểu về suy nghĩ của con người bằng ba cách:

- **Nội quan** – cố gắng nắm bắt suy nghĩ của chính mình khi chúng trôi qua;
- **Thí nghiệm tâm lý** – quan sát một người đang hành động;
- **Chụp ảnh não** – quan sát bộ não đang hoạt động.

Một khi chúng ta có một lý thuyết đủ chính xác về tâm trí, thì việc thể hiện lý thuyết đó dưới dạng một chương trình máy tính sẽ trở nên khả thi. Nếu hành vi đầu vào-đầu ra của chương trình khớp với hành vi tương ứng của con người, đó là bằng chứng cho thấy một số cơ chế của chương trình cũng có thể đang hoạt động trong con người.

Ví dụ, Allen Newell và Herbert Simon, những người đã phát triển GPS, “Bộ giải quyết vấn đề tổng quát” (Newell và Simon, 1961), không chỉ hài lòng với việc chương trình của họ giải quyết các vấn đề một cách chính xác. Họ quan tâm nhiều hơn đến việc so sánh trình tự và thời gian của các bước suy luận của nó với các đối tượng là con người khi giải quyết cùng một vấn đề. Lĩnh vực liên ngành của **khoa học nhận thức** (cognitive science) tập hợp các mô hình máy tính từ AI và các kỹ

thuật thực nghiệm từ tâm lý học để xây dựng các lý thuyết chính xác và có thể kiểm chứng về tâm trí con người.

Khoa học nhận thức tự nó là một lĩnh vực hấp dẫn, xứng đáng với nhiều cuốn sách giáo khoa và ít nhất một cuốn bách khoa toàn thư (Wilson và Keil, 1999). Thỉnh thoảng chúng ta sẽ bình luận về những điểm tương đồng hoặc khác biệt giữa các kỹ thuật AI và nhận thức của con người. Tuy nhiên, khoa học nhận thức thực sự nhất thiết phải dựa trên điều tra thực nghiệm về con người hoặc động vật thực tế. Chúng ta sẽ dành điều đó cho những cuốn sách khác, vì chúng ta giả định rằng người đọc chỉ có một máy tính để thử nghiệm.

Trong những ngày đầu của AI, thường có sự nhầm lẫn giữa các cách tiếp cận. Một tác giả có thể lập luận rằng một thuật toán hoạt động tốt trong một nhiệm vụ và do đó, nó là một mô hình tốt về hiệu suất của con người, hoặc ngược lại. Các tác giả hiện đại tách biệt hai loại tuyên bố này; sự phân biệt này đã cho phép cả AI và khoa học nhận thức phát triển nhanh hơn. Hai lĩnh vực này nuôi dưỡng lẫn nhau, đáng chú ý nhất là trong thị giác máy tính, nơi kết hợp bằng chứng điện sinh lý thần kinh vào các mô hình tính toán. Gần đây, sự kết hợp giữa các phương pháp chụp ảnh thần kinh với các kỹ thuật học máy để phân tích dữ liệu như vậy đã dẫn đến sự khởi đầu của một khả năng “đọc suy nghĩ” – tức là, để xác định nội dung ngữ nghĩa trong suy nghĩ bên trong của một người. Khả năng này, đến lượt nó, có thể làm sáng tỏ hơn về cách thức hoạt động của nhận thức con người.

1.1.3 Suy nghĩ một cách hợp lý: Cách tiếp cận “các quy luật của tư duy”

Nhà triết học Hy Lạp Aristotle là một trong những người đầu tiên cố gắng hệ thống hóa “tư duy đúng đắn” – tức là các quy trình suy luận không thể bác bỏ. Các **tam đoạn luận** của ông đã cung cấp các mô hình cho cấu trúc lập luận luôn đưa ra kết luận đúng khi các tiền đề là đúng. Ví dụ điển hình bắt đầu với “Socrates là một người đàn ông” và “tất cả đàn ông đều phải chết”, và kết luận “Socrates phải chết”. (Ví dụ này có lẽ là của Sextus Empiricus chứ không phải Aristotle.) Những quy luật tư duy này được cho là chi phối hoạt động của trí óc; việc nghiên cứu chúng đã khởi xướng lĩnh vực được gọi là **logic**.

Các nhà logic học vào thế kỷ 19 đã phát triển một ký hiệu chính xác cho các câu nói về các đối tượng trong thế giới và các mối quan hệ giữa chúng. (Hãy đối chiếu điều này với ký hiệu số học thông thường, chỉ cung cấp các câu nói về các con số.) Đến năm 1965, các chương trình về nguyên tắc có thể giải quyết bất kỳ vấn đề nào có thể giải được được mô tả bằng ký hiệu logic. Truyền thống được gọi là **phái logic** trong trí tuệ nhân tạo hy vọng sẽ xây dựng trên các chương trình như vậy để tạo ra các hệ thống thông minh.

Logic theo cách hiểu thông thường đòi hỏi kiến thức về thế giới phải chắc chắn – một điều kiện mà trên thực tế hiếm khi đạt được. Chúng ta đơn giản là không biết các quy tắc của, chẳng hạn, chính trị hoặc chiến tranh theo cùng một cách mà chúng ta biết các quy tắc của cờ vua hoặc số học. **Lý thuyết xác suất** lấp đầy khoảng trống này, cho phép suy luận chặt chẽ với thông tin không chắc chắn. Về nguyên tắc, nó cho phép xây dựng một mô hình toàn diện về tư duy hợp lý, dẫn từ thông tin nhận thức thô đến sự hiểu biết về cách thế giới hoạt động và đến các dự đoán về tương lai. Tuy nhiên, điều nó không làm được là tạo ra hành vi thông minh. Đối với điều đó, chúng ta cần một lý thuyết về hành động hợp lý. Bản thân tư duy hợp lý là chưa đủ.

1.1.4 Hành động một cách hợp lý: Cách tiếp cận tác nhân hợp lý

Một **tác nhân** (agent) chỉ đơn thuần là một thứ gì đó hoạt động (tác nhân xuất phát từ tiếng Latin *agere*, nghĩa là làm). Tất nhiên, tất cả các chương trình máy tính đều làm điều gì đó, nhưng các tác nhân máy tính được mong đợi sẽ làm nhiều hơn: hoạt động một cách tự chủ, nhận thức môi trường của chúng, tồn tại trong một khoảng thời gian dài, thích ứng với sự thay đổi, và tạo ra và theo đuổi các mục tiêu. Một **tác nhân hợp lý** là một tác nhân hành động sao cho đạt được kết quả tốt nhất hoặc, khi có sự không chắc chắn, là kết quả mong đợi tốt nhất.

Trong cách tiếp cận AI “các quy luật của tư duy”, trọng tâm là suy luận chính xác. Việc suy luận chính xác đôi khi là một phần của việc trở thành một tác nhân hợp lý, bởi vì một cách để hành động hợp lý là suy ra rằng một hành động nhất định là tốt nhất và sau đó hành động theo kết luận đó. Mặt khác, có những cách hành động hợp lý không thể nói là liên quan đến suy luận. Ví dụ, rút tay lại khỏi một bếp lò nóng là một hành động phản xạ thường thành công hơn một hành động chậm hơn được thực hiện sau khi cân nhắc kỹ lưỡng.

Tất cả các kỹ năng cần thiết cho bài kiểm tra Turing cũng cho phép một tác nhân hành động hợp lý. **Biểu diễn tri thức** và **suy luận** cho phép các tác nhân đưa ra các quyết định tốt. Chúng ta cần có khả năng tạo ra các câu có thể hiểu được bằng ngôn ngữ tự nhiên để hòa nhập trong một xã hội phức tạp. Chúng ta cần **học hỏi** không chỉ để có kiến thức uyên bác, mà còn vì nó cải thiện khả năng của chúng ta trong việc tạo ra hành vi hiệu quả, đặc biệt trong những trường hợp mới.

Cách tiếp cận **tác nhân hợp lý** đối với AI có hai lợi thế so với các cách tiếp cận khác. Thứ nhất, nó tổng quát hơn cách tiếp cận “các quy luật của tư duy” vì suy luận chính xác chỉ là một trong một vài cơ chế có thể có để đạt được tính hợp lý. Thứ hai, nó dễ phát triển khoa học hơn. Tiêu chuẩn về tính hợp lý được định nghĩa toán học rõ ràng và hoàn toàn tổng quát. Chúng ta thường có thể quay ngược lại từ đặc tả này để đưa ra các thiết kế tác nhân mà có thể chứng minh là đạt được nó – một điều gần như không thể nếu mục tiêu là bắt chước hành vi hoặc quá trình tư duy của con người.

Vì những lý do này, cách tiếp cận **tác nhân hợp lý** đối với AI đã chiếm ưu thế trong hầu hết lịch sử của lĩnh vực này. Trong những thập kỷ đầu, các tác nhân hợp lý được xây dựng trên nền tảng logic và hình thành các kế hoạch rõ ràng để đạt được các mục tiêu cụ thể. Sau đó, các phương pháp dựa trên lý thuyết xác suất và học máy đã cho phép tạo ra các tác nhân có thể đưa ra quyết định trong điều kiện không chắc chắn để đạt được kết quả mong đợi tốt nhất. Tóm lại, AI đã tập trung vào việc nghiên cứu và xây dựng các tác nhân **làm điều đúng đắn**. Điều gì được coi là điều đúng đắn được xác định bởi mục tiêu mà chúng ta cung cấp cho tác nhân. Mô hình tổng quát này phổ biến đến mức chúng ta có thể gọi nó là **mô hình chuẩn**. Nó chiếm ưu thế không chỉ trong AI, mà còn trong lý thuyết điều khiển, nơi một bộ điều khiển giảm thiểu một hàm chi phí; trong nghiên cứu vận trù học, nơi một chính sách tối đa hóa tổng các phần thưởng; trong thống kê, nơi một quy tắc quyết định giảm thiểu một hàm mất mát; và trong kinh tế học, nơi một người ra quyết định tối đa hóa tiện ích hoặc một số thước đo phúc lợi xã hội.

Chúng ta cần thực hiện một sự tinh chỉnh quan trọng đối với mô hình chuẩn để giải thích một thực tế rằng tính hợp lý hoàn hảo – luôn thực hiện hành động tối ưu tuyệt đối – là không khả thi trong các môi trường phức tạp. Các yêu cầu tính toán quá cao. Chương 6 và 16 đề cập đến vấn đề **tính hợp lý hạn chế** – hành động thích hợp khi không có đủ thời gian để thực hiện tất cả các phép tính

mà người ta muốn. Tuy nhiên, tính hợp lý hoàn hảo thường vẫn là một điểm khởi đầu tốt cho phân tích lý thuyết.

1.1.5 Máy móc hữu ích

Mô hình chuẩn đã là một hướng dẫn hữu ích cho nghiên cứu AI kể từ khi nó ra đời, nhưng về lâu dài, có lẽ nó không phải là mô hình đúng. Lý do là mô hình chuẩn giả định rằng chúng ta sẽ cung cấp một mục tiêu được xác định đầy đủ cho máy móc.

Đối với một nhiệm vụ được xác định một cách nhân tạo như cờ vua hoặc tính toán đường đi ngắn nhất, nhiệm vụ đó đi kèm với một mục tiêu được xây dựng sẵn – vì vậy mô hình chuẩn có thể áp dụng được. Tuy nhiên, khi chúng ta tiến vào thế giới thực, việc xác định mục tiêu một cách đầy đủ và chính xác ngày càng trở nên khó khăn. Ví dụ, khi thiết kế một chiếc xe tự lái, người ta có thể nghĩ rằng mục tiêu là đến đích một cách an toàn. Nhưng lái xe trên bất kỳ con đường nào cũng có nguy cơ bị thương do các tài xế khác đi sai, hỏng hóc thiết bị, v.v.; do đó, một mục tiêu an toàn nghiêm ngặt đòi hỏi phải ở trong ga-ra. Có một sự đánh đổi giữa việc tiến về đích và phải chịu rủi ro bị thương. Sự đánh đổi này nên được thực hiện như thế nào? Hơn nữa, chúng ta có thể cho phép chiếc xe thực hiện các hành động gây khó chịu cho các tài xế khác đến mức nào? Chiếc xe nên điều chỉnh gia tốc, đánh lái và phanh ở mức nào để tránh làm hành khách bị xóc? Những câu hỏi này rất khó trả lời ngay từ đầu. Chúng đặc biệt có vấn đề trong lĩnh vực tổng quát của tương tác giữa con người và robot, mà xe tự lái là một ví dụ.

Vấn đề đạt được sự đồng thuận giữa sở thích thực sự của chúng ta và mục tiêu mà chúng ta đưa vào máy được gọi là **vấn đề căn chỉnh giá trị**: các giá trị hoặc mục tiêu được đưa vào máy phải được căn chỉnh với các giá trị của con người. Nếu chúng ta đang phát triển một hệ thống AI trong phòng thí nghiệm hoặc trong một trình mô phỏng – như đã xảy ra trong phần lớn lịch sử của lĩnh vực này – có một cách khắc phục dễ dàng cho một mục tiêu được xác định không chính xác: đặt lại hệ thống, sửa mục tiêu và thử lại. Khi lĩnh vực này tiến tới các hệ thống thông minh ngày càng có năng lực hơn được triển khai trong thế giới thực, cách tiếp cận này không còn khả thi nữa. Một hệ thống được triển khai với một mục tiêu không chính xác sẽ có những hậu quả tiêu cực. Hơn nữa, hệ thống càng thông minh thì hậu quả càng tiêu cực.

Quay trở lại ví dụ cờ vua dường như không có vấn đề, hãy xem điều gì xảy ra nếu cỗ máy đủ thông minh để suy luận và hành động ngoài giới hạn của bàn cờ. Trong trường hợp đó, nó có thể cố gắng tăng cơ hội chiến thắng bằng các mánh lới như thôi miên hoặc tổng tiền đổi thủ hoặc hối lộ khán giả để tạo ra tiếng ồn lạo xạo trong thời gian đối thủ suy nghĩ. Nó cũng có thể cố gắng chiếm đoạt thêm sức mạnh tính toán cho chính mình. Những hành vi này không phải là “không thông minh” hay “điên rồ”; chúng là một hệ quả logic của việc xác định chiến thắng là mục tiêu duy nhất cho cỗ máy.

Không thể lường trước tất cả các cách mà một cỗ máy theo đuổi một mục tiêu cố định có thể hành xử sai. Do đó, có lý do chính đáng để nghĩ rằng mô hình chuẩn là không thỏa đáng. Chúng ta không muốn những cỗ máy thông minh theo nghĩa theo đuổi mục tiêu của chúng; chúng ta muốn chúng theo đuổi mục tiêu của chúng ta. Nếu chúng ta không thể chuyển giao hoàn hảo những mục tiêu đó cho cỗ máy, thì chúng ta cần một công thức mới – trong đó cỗ máy đang theo đuổi mục tiêu của chúng ta, nhưng nhất thiết phải không chắc chắn về chúng là gì. Khi một cỗ máy biết rằng nó không biết mục tiêu đầy đủ, nó có động cơ để hành động thận trọng, xin phép, tìm hiểu thêm

về sở thích của chúng ta thông qua quan sát, và tuân theo sự kiểm soát của con người. Cuối cùng, chúng ta muốn các tác nhân có thể chứng minh là có lợi cho con người. Chúng ta sẽ quay lại chủ đề này trong Mục 1.5.

1.2 Các Nền Tảng của Trí tuệ Nhân tạo

Trong phần này, chúng tôi cung cấp một lịch sử tóm tắt về các lĩnh vực đã đóng góp ý tưởng, quan điểm và kỹ thuật cho AI. Giống như bất kỳ lịch sử nào, lịch sử này tập trung vào một số ít người, sự kiện và ý tưởng và bỏ qua những thứ khác cũng quan trọng. Chúng tôi sắp xếp lịch sử xung quanh một loạt các câu hỏi. Chắc chắn chúng tôi không muốn tạo ấn tượng rằng những câu hỏi này là những câu hỏi duy nhất mà các lĩnh vực này giải quyết hoặc rằng các lĩnh vực này đều đang làm việc hướng tới AI như là thành quả cuối cùng của chúng.

1.2.1 Triết học

- Liệu các quy tắc hình thức có thể được sử dụng để đưa ra các kết luận hợp lệ không?
- Làm thế nào mà tâm trí lại phát sinh từ một bộ não vật lý?
- Kiến thức đến từ đâu?
- Làm thế nào kiến thức dẫn đến hành động?

Aristotle (384–322 TCN) là người đầu tiên đưa ra một tập hợp các quy luật chính xác chi phối phân lý trí của tâm trí. Ông đã phát triển một hệ thống **tam đoạn luận** không chính thức để suy luận đúng đắn, về nguyên tắc cho phép người ta tạo ra các kết luận một cách máy móc, với các tiền đề ban đầu.

Ramon Llull (khoảng 1232–1315) đã nghĩ ra một hệ thống suy luận được xuất bản dưới dạng *Ars Magna* hay *Nghệ thuật Vĩ đại* (1305). Llull đã cố gắng triển khai hệ thống của mình bằng cách sử dụng một thiết bị cơ khí thực tế: một bộ bánh xe giấy có thể được xoay thành các hoán vị khác nhau.

Khoảng năm 1500, **Leonardo da Vinci** (1452–1519) đã thiết kế nhưng không chế tạo một máy tính cơ học; các bản tái tạo gần đây đã cho thấy thiết kế đó có chức năng. Cổ máy tính toán được biết đến đầu tiên được chế tạo vào khoảng năm 1623 bởi nhà khoa học người Đức **Wilhelm Schickard** (1592–1635). **Blaise Pascal** (1623–1662) đã chế tạo *Pascaline* vào năm 1642 và viết rằng nó “tạo ra những hiệu ứng có vẻ gần với suy nghĩ hơn tất cả các hành động của động vật”. **Gottfried Wilhelm Leibniz** (1646–1716) đã chế tạo một thiết bị cơ học nhằm thực hiện các phép toán trên các khái niệm thay vì các con số, nhưng phạm vi của nó khá hạn chế. Trong cuốn sách *Leviathan* (1651) của mình, **Thomas Hobbes** (1588–1679) đã đề xuất ý tưởng về một cỗ máy suy nghĩ, một “con vật nhân tạo” theo lời của ông, lập luận rằng “Bởi vì trái tim là gì nếu không phải là một lò xo; và các dây thần kinh, nếu không phải là rất nhiều sợi dây; và các khớp nối, nếu không phải là rất nhiều bánh xe.” Ông cũng gợi ý rằng suy luận giống như tính toán số: “‘Lý do’... không là gì ngoài ‘tính toán,’ tức là cộng và trừ.”

Nói rằng tâm trí hoạt động, ít nhất là một phần, theo các quy tắc logic hoặc số học, và xây dựng các hệ thống vật lý mô phỏng một số quy tắc đó là một chuyện. Nói rằng bản thân tâm trí là một

hệ thống vật lý như vậy lại là một chuyện khác. **René Descartes** (1596–1650) đã đưa ra cuộc thảo luận rõ ràng đầu tiên về sự khác biệt giữa tâm trí và vật chất. Ông lưu ý rằng một quan niệm hoàn toàn vật lý về tâm trí dường như để lại rất ít chỗ cho ý chí tự do. Nếu tâm trí hoàn toàn bị chi phối bởi các quy luật vật lý, thì nó không có ý chí tự do hơn một tảng đá “quyết định” rơi xuống. Descartes là một người ủng hộ **thuyết nhị nguyên**. Ông cho rằng có một phần của tâm trí con người (hoặc linh hồn hoặc tinh thần) nằm ngoài tự nhiên, miễn nhiễm với các quy luật vật lý. Mặt khác, động vật không sở hữu phẩm chất nhị nguyên này; chúng có thể được coi là máy móc.

Một phương án thay thế cho thuyết nhị nguyên là **thuyết duy vật**, cho rằng hoạt động của bộ não theo các quy luật vật lý cấu thành nên tâm trí. Ý chí tự do chỉ đơn giản là cách mà nhận thức về các lựa chọn có sẵn xuất hiện với thực thể đang lựa chọn. Các thuật ngữ **thuyết duy vật** và **thuyết tự nhiên** cũng được sử dụng để mô tả quan điểm này, đối lập với siêu nhiên.

Với một tâm trí vật lý thao tác kiến thức, vấn đề tiếp theo là thiết lập nguồn gốc của kiến thức. Phong trào **thuyết kinh nghiệm** (empiricism), bắt đầu với *Novum Organum* của **Francis Bacon** (1561–1626), được đặc trưng bởi một châm ngôn của **John Locke** (1632–1704): “Không có gì trong sự hiểu biết, mà không phải là thứ đầu tiên trong các giác quan.”

A Treatise of Human Nature (Hume, 1739) của **David Hume** (1711–1776) đã đề xuất cái hiện nay được gọi là **nguyên tắc quy nạp**: các quy tắc chung được lĩnh hội bằng cách tiếp xúc với các mối liên kết lặp đi lặp lại giữa các yếu tố của chúng.

Dựa trên công trình của **Ludwig Wittgenstein** (1889–1951) và **Bertrand Russell** (1872–1970), **Nhóm Vienna** nổi tiếng (Sigmund, 2017), một nhóm các nhà triết học và toán học gặp gỡ ở Vienna vào những năm 1920 và 1930, đã phát triển học thuyết **thuyết thực chứng logic**. Học thuyết này cho rằng tất cả kiến thức có thể được đặc trưng bởi các lý thuyết logic được kết nối, cuối cùng, với các **câu quan sát** tương ứng với các đầu vào cảm giác; do đó, thuyết thực chứng logic kết hợp thuyết duy lý và thuyết kinh nghiệm.

Lý thuyết xác nhận của **Rudolf Carnap** (1891–1970) và **Carl Hempel** (1905–1997) đã cố gắng phân tích việc thu nhận kiến thức từ kinh nghiệm bằng cách định lượng mức độ tin tưởng nên được gán cho các câu logic dựa trên mối liên hệ của chúng với các quan sát xác nhận hoặc bác bỏ chúng. Cuốn sách *Cấu trúc Logic của Thế giới* (1928) của Carnap có lẽ là lý thuyết đầu tiên về tâm trí như một quá trình tính toán.

Yếu tố cuối cùng trong bức tranh triết học về tâm trí là mối liên hệ giữa kiến thức và hành động. Câu hỏi này rất quan trọng đối với AI vì trí thông minh đòi hỏi hành động cũng như suy luận. Hơn nữa, chỉ bằng cách hiểu cách các hành động được biện minh, chúng ta mới có thể hiểu cách xây dựng một tác nhân mà các hành động của nó có thể biện minh được (hay hợp lý).

Aristotle đã lập luận (trong *De Motu Animalium*) rằng các hành động được biện minh bằng một mối liên hệ logic giữa các mục tiêu và kiến thức về kết quả của hành động:

Nhưng làm thế nào mà suy nghĩ đôi khi đi kèm với hành động và đôi khi thì không, đôi khi đi kèm với chuyển động, và đôi khi thì không? Có vẻ như điều gần như tương tự xảy ra như trong trường hợp suy luận và đưa ra kết luận về các đối tượng không thay đổi. Nhưng trong trường hợp đó, mục đích là một mệnh đề suy đoán... trong khi ở đây kết luận rút ra từ hai tiền đề là một hành động... Tôi cần một cái gì đó để che thân; một chiếc áo choàng là một cái gì đó để che thân. Tôi cần một chiếc áo choàng. Cái tôi cần,

tôi phải làm; tôi cần một chiếc áo choàng. Tôi phải làm một chiếc áo choàng. Và kết luận, “Tôi phải làm một chiếc áo choàng,” là một hành động.

Trong *Đạo đức Nicomachean* (Sách III. 3, 1112b), Aristotle tiếp tục trình bày chi tiết về chủ đề này, gợi ý một thuật toán:

Chúng ta không cân nhắc về mục đích, mà là về phương tiện. Bởi vì một bác sĩ không cân nhắc liệu ông ta có nên chữa bệnh, hay một nhà hùng biện có nên thuyết phục, ... Họ giả định mục đích và xem xét bằng cách nào và bằng phương tiện gì để đạt được nó, và nếu nó có vẻ dễ dàng và được tạo ra tốt nhất bằng cách đó; trong khi nếu nó chỉ đạt được bằng một phương tiện duy nhất, họ sẽ xem xét nó sẽ đạt được bằng cách nào và bằng phương tiện gì thì nó sẽ đạt được, cho đến khi họ đi đến nguyên nhân đầu tiên, ... và điều cuối cùng trong thứ tự phân tích dường như là điều đầu tiên trong thứ tự trở thành. Và nếu chúng ta gặp phải một điều không thể, chúng ta từ bỏ cuộc tìm kiếm, ví dụ, nếu chúng ta cần tiền và không thể có được; nhưng nếu một điều gì đó có vẻ khả thi, chúng ta cố gắng làm nó.

Thuật toán của Aristotle đã được Newell và Simon triển khai 2300 năm sau đó trong chương trình Giải quyết Vấn đề Tổng quát của họ. Chúng ta bây giờ sẽ gọi nó là một hệ thống lập kế hoạch lùi tham lam (xem Chương 11). Các phương pháp dựa trên lập kế hoạch logic để đạt được các mục tiêu xác định đã thống trị vài thập kỷ đầu tiên của nghiên cứu lý thuyết trong AI.

Chỉ suy nghĩ về các hành động đạt được mục tiêu thường hữu ích nhưng đôi khi không thể áp dụng được. Ví dụ, nếu có một số cách khác nhau để đạt được một mục tiêu, cần phải có một cách nào đó để chọn trong số chúng. Quan trọng hơn, có thể không thể đạt được mục tiêu một cách chắc chắn, nhưng vẫn phải thực hiện một hành động nào đó. Vậy thì người ta nên quyết định như thế nào? **Antoine Arnauld** (1662), khi phân tích khái niệm các quyết định hợp lý trong cờ bạc, đã đề xuất một công thức định lượng để tối đa hóa giá trị tiền tệ mong đợi của kết quả. Sau đó, **Daniel Bernoulli** (1738) đã giới thiệu khái niệm tổng quát hơn về **tính hữu dụng** (utility) để nắm bắt giá trị chủ quan, nội tại của một kết quả. Khái niệm hiện đại về ra quyết định hợp lý trong điều kiện không chắc chắn liên quan đến việc tối đa hóa tiện ích mong đợi, như được giải thích trong Chương 15.

Trong các vấn đề về đạo đức và chính sách công, một người ra quyết định phải xem xét lợi ích của nhiều cá nhân. **Jeremy Bentham** (1823) và **John Stuart Mill** (1863) đã thúc đẩy ý tưởng về **chủ nghĩa vị lợi** (utilitarianism): rằng việc ra quyết định hợp lý dựa trên việc tối đa hóa tiện ích nên được áp dụng cho tất cả các lĩnh vực hoạt động của con người, bao gồm cả các quyết định chính sách công được đưa ra thay mặt cho nhiều cá nhân. Chủ nghĩa vị lợi là một loại cụ thể của **thuyết hậu quả**: ý tưởng rằng điều gì là đúng và sai được xác định bởi kết quả mong đợi của một hành động.

Ngược lại, **Immanuel Kant**, vào năm 1785, đã đề xuất một lý thuyết về **đạo đức dựa trên quy tắc** hay **đạo đức học nghĩa vụ**, trong đó “làm điều đúng đắn” không được xác định bởi kết quả mà bởi các quy luật xã hội phổ quát chi phối các hành động được cho phép, chẳng hạn như “đừng nói dối” hoặc “đừng giết người”. Do đó, một người theo chủ nghĩa vị lợi có thể nói một lời nói dối vô hại nếu lợi ích mong đợi lớn hơn tác hại, nhưng một người theo thuyết Kant sẽ bị ràng buộc không được nói dối, bởi vì nói dối vốn dĩ là sai. Mill thừa nhận giá trị của các quy tắc, nhưng hiểu chúng là các quy trình quyết định hiệu quả được tổng hợp từ suy luận nguyên tắc đầu tiên về hậu quả. Nhiều hệ thống AI hiện đại áp dụng chính xác cách tiếp cận này.

1.2.2 Toán học

- Đây là các quy tắc hình thức để đưa ra các kết luận hợp lệ?
- Cái gì có thể được tính toán?
- Chúng ta suy luận với thông tin không chắc chắn như thế nào?

Các nhà triết học đã đặt ra một số ý tưởng cơ bản của AI, nhưng bước nhảy vọt sang một ngành khoa học chính thức đòi hỏi phải toán học hóa logic và xác suất, và sự ra đời của một nhánh toán học mới: **tính toán**.

Ý tưởng về **logic hình thức** có thể được truy ngược về các nhà triết học Hy Lạp, Ấn Độ và Trung Quốc cổ đại, nhưng sự phát triển toán học của nó thực sự bắt đầu với công trình của **George Boole** (1815–1864), người đã làm chi tiết về logic mệnh đề, hay logic **Boolean** (Boole, 1847). Năm 1879, **Gottlob Frege** (1848–1925) đã mở rộng logic của Boole để bao gồm các đối tượng và các mối quan hệ, tạo ra logic bậc nhất được sử dụng ngày nay. Ngoài vai trò trung tâm trong giai đoạn đầu của nghiên cứu AI, logic bậc nhất đã thúc đẩy công trình của Gödel và Turing, những người đã làm nền tảng cho chính tính toán, như chúng tôi giải thích dưới đây.

Lý thuyết **xác suất** có thể được xem như là tổng quát hóa logic cho các tình huống có thông tin không chắc chắn – một sự cân nhắc có tầm quan trọng lớn đối với AI. **Gerolamo Cardano** (1501–1576) lần đầu tiên đưa ra ý tưởng về xác suất, mô tả nó theo các kết quả có thể có của các sự kiện cờ bạc. Năm 1654, **Blaise Pascal** (1623–1662), trong một lá thư gửi **Pierre Fermat** (1601–1665), đã chỉ ra cách dự đoán tương lai của một trò chơi cờ bạc chưa hoàn thành và gán các khoản thanh toán trung bình cho những người chơi cờ bạc. Xác suất nhanh chóng trở thành một phần không thể thiếu của các ngành khoa học định lượng, giúp giải quyết các phép đo không chắc chắn và các lý thuyết không đầy đủ. **Jacob Bernoulli** (1654–1705, chú của Daniel), **Pierre Laplace** (1749–1827) và những người khác đã phát triển lý thuyết và giới thiệu các phương pháp thống kê mới. **Thomas Bayes** (1702–1761) đã đề xuất một quy tắc để cập nhật xác suất khi có bằng chứng mới; quy tắc Bayes là một công cụ quan trọng cho các hệ thống AI.

Việc hình thức hóa xác suất, kết hợp với sự sẵn có của dữ liệu, đã dẫn đến sự ra đời của **thống kê** như một lĩnh vực. Một trong những ứng dụng đầu tiên là phân tích dữ liệu điều tra dân số Luân Đôn của **John Graunt** vào năm 1662. **Ronald Fisher** được coi là nhà thống kê hiện đại đầu tiên (Fisher, 1922). Ông đã kết hợp các ý tưởng về xác suất, thiết kế thử nghiệm, phân tích dữ liệu và tính toán – vào năm 1919, ông khẳng định rằng ông không thể làm việc mà không có một máy tính cơ học có tên là MILLIONAIRE (chiếc máy tính đầu tiên có thể thực hiện phép nhân), mặc dù chi phí của chiếc máy tính đó cao hơn tiền lương hàng năm của ông (Ross, 2012).

Lịch sử của **tính toán** cũng cổ xưa như lịch sử của các con số, nhưng thuật toán không tầm thường đầu tiên được cho là thuật toán Euclid để tính ước số chung lớn nhất. Từ “**thuật toán**” (algorithm) xuất phát từ **Muhammad ibn Musa al-Khwarizmi**, một nhà toán học thế kỷ thứ 9, người mà các tác phẩm của ông cũng đã giới thiệu chữ số Ả Rập và đại số cho châu Âu. Boole và những người khác đã thảo luận về các thuật toán cho suy luận logic, và, vào cuối thế kỷ 19, các nỗ lực đã được tiến hành để hình thức hóa suy luận toán học tổng quát thành suy luận logic.

Kurt Gödel (1906–1978) đã chỉ ra rằng tồn tại một thủ tục hiệu quả để chứng minh bất kỳ tuyên bố đúng nào trong logic bậc nhất của Frege và Russell, nhưng logic bậc nhất không thể nắm bắt nguyên tắc quy nạp toán học cần thiết để đặc trưng hóa các số tự nhiên. Năm 1931, Gödel đã chỉ ra rằng các giới hạn về suy luận thực sự tồn tại. **Định lý bất toàn** của ông đã chỉ ra rằng trong bất

kỳ lý thuyết hình thức nào mạnh như số học Peano (lý thuyết cơ bản về các số tự nhiên), nhất thiết phải có những tuyên bố đúng không có bằng chứng trong lý thuyết.

Kết quả cơ bản này cũng có thể được giải thích là cho thấy một số hàm trên các số nguyên không thể được biểu diễn bằng một thuật toán – tức là chúng không thể được tính toán. Điều này đã thúc đẩy **Alan Turing** (1912–1954) cố gắng đặc trưng hóa chính xác những hàm nào là **có thể tính toán được** (computable) – có khả năng được tính toán bằng một thủ tục hiệu quả. **Luận đề Church–Turing** đề xuất xác định khái niệm tổng quát về tính toán với các hàm được tính toán bởi một máy Turing (Turing, 1936). Turing cũng chỉ ra rằng có một số hàm mà không có máy Turing nào có thể tính toán được. Ví dụ, không có máy nào có thể nói chung liệu một chương trình đã cho sẽ trả về một câu trả lời trên một đầu vào đã cho hay chạy mãi mãi.

Mặc dù tính toán là quan trọng để hiểu về tính toán, khái niệm **khả thi** (tractability) thậm chí còn có tác động lớn hơn đến AI. Nói một cách đại khái, một vấn đề được gọi là **không khả thi** nếu thời gian cần thiết để giải quyết các trường hợp của vấn đề tăng theo cấp số nhân với kích thước của các trường hợp. Sự khác biệt giữa tăng trưởng theo cấp số mũ và tăng trưởng theo cấp số mũ về độ phức tạp lần đầu tiên được nhấn mạnh vào giữa những năm 1960 (Cobham, 1964; Edmonds, 1965). Điều này rất quan trọng vì tăng trưởng theo cấp số nhân có nghĩa là ngay cả các trường hợp có kích thước vừa phải cũng không thể được giải quyết trong bất kỳ khoảng thời gian hợp lý nào.

Lý thuyết **NP-hoàn chỉnh** (NP-completeness), tiên phong bởi Cook (1971) và Karp (1972), cung cấp một cơ sở để phân tích tính khả thi của các vấn đề: bất kỳ lớp vấn đề nào mà lớp các vấn đề NP-hoàn chỉnh có thể được quy về thì có khả năng là không khả thi. (Mặc dù chưa được chứng minh rằng các vấn đề NP-hoàn chỉnh nhất thiết phải không khả thi, hầu hết các nhà lý thuyết đều tin vào điều đó.) Những kết quả này trái ngược với sự lạc quan mà báo chí đại chúng đã chào đón những chiếc máy tính đầu tiên – “Siêu não điện tử” “Nhanh hơn Einstein!”. Mặc dù tốc độ của máy tính ngày càng tăng, việc sử dụng cẩn thận các tài nguyên và sự không hoàn hảo cần thiết sẽ là đặc điểm của các hệ thống thông minh. Nói một cách thô thiển, thế giới là một trường hợp vấn đề cực kỳ lớn!

1.2.3 Kinh tế học

- Chúng ta nên đưa ra quyết định như thế nào cho phù hợp với sở thích của mình?
- Chúng ta nên làm điều này như thế nào khi những người khác có thể không đồng tình?
- Chúng ta nên làm điều này như thế nào khi phần thưởng có thể ở rất xa trong tương lai?

Khoa học kinh tế ra đời vào năm 1776, khi **Adam Smith** (1723–1790) xuất bản cuốn *Một cuộc điều tra về Bản chất và Nguyên nhân của Của cải Quốc gia*. Smith đã đề xuất phân tích các nền kinh tế bao gồm nhiều tác nhân cá nhân quan tâm đến lợi ích của chính họ. Tuy nhiên, Smith không ủng hộ lòng tham tài chính như một quan điểm đạo đức: cuốn sách trước đó của ông (1759) *Lý thuyết về Cảm tình Đạo đức* bắt đầu bằng cách chỉ ra rằng sự quan tâm đến hạnh phúc của người khác là một thành phần thiết yếu trong lợi ích của mỗi cá nhân.

Hầu hết mọi người nghĩ kinh tế học là về tiền bạc, và trên thực tế, phân tích toán học đầu tiên về các quyết định trong điều kiện không chắc chắn, công thức giá trị mong đợi tối đa của Arnauld (1662), đã đề cập đến giá trị tiền tệ của các vụ cá cược. **Daniel Bernoulli** (1738) nhận thấy rằng công thức này dường như không hoạt động tốt đối với số tiền lớn hơn, chẳng hạn như các khoản

đầu tư vào các cuộc thám hiểm thương mại hàng hải. Thay vào đó, ông đề xuất một nguyên tắc dựa trên việc tối đa hóa **tiện ích** (utility) mong đợi, và giải thích các lựa chọn đầu tư của con người bằng cách đề xuất rằng tiện ích cận biên của một lượng tiền bỏ sung giảm đi khi một người có được nhiều tiền hơn.

Léon Walras (phát âm là “Valrasse”) (1834–1910) đã đưa ra một nền tảng tổng quát hơn cho lý thuyết tiện ích theo sở thích giữa các vụ cá cược về bất kỳ kết quả nào (không chỉ kết quả tiền tệ). Lý thuyết được cải thiện bởi Ramsey (1931) và sau đó bởi **John von Neumann** và **Oskar Morgenstern** trong cuốn sách của họ *Lý thuyết về Trò chơi và Hành vi Kinh tế* (1944). Kinh tế học không còn là nghiên cứu về tiền bạc; thay vào đó, nó là nghiên cứu về ham muốn và sở thích.

Lý thuyết quyết định, kết hợp lý thuyết xác suất với lý thuyết tiện ích, cung cấp một khuôn khổ chính thức và hoàn chỉnh cho các quyết định cá nhân (kinh tế hoặc các quyết định khác) được đưa ra trong điều kiện không chắc chắn – tức là, trong các trường hợp mà các mô tả xác suất nắm bắt một cách thích hợp môi trường của người ra quyết định. Điều này phù hợp với các nền kinh tế “lớn” nơi mỗi tác nhân không cần phải chú ý đến hành động của các tác nhân khác với tư cách là cá nhân. Đối với các nền kinh tế “nhỏ”, tình hình giống như một trò chơi hơn nhiều: hành động của một người chơi có thể ảnh hưởng đáng kể đến tiện ích của người khác (dù là tích cực hay tiêu cực). Sự phát triển của lý thuyết trò chơi của Von Neumann và Morgenstern (xem thêm Luce và Raiffa, 1957) bao gồm kết quả đáng ngạc nhiên là, đối với một số trò chơi, một tác nhân hợp lý nên áp dụng các chính sách được (hoặc ít nhất là có vẻ như) ngẫu nhiên hóa. Không giống như lý thuyết quyết định, lý thuyết trò chơi không đưa ra một quy tắc không thể nhầm lẫn để lựa chọn hành động. Trong AI, các quyết định liên quan đến nhiều tác nhân được nghiên cứu dưới tiêu đề **hệ thống đa tác nhân** (Chương 17).

Các nhà kinh tế học, với một vài ngoại lệ, đã không giải quyết câu hỏi thứ ba được liệt kê ở trên: làm thế nào để đưa ra các quyết định hợp lý khi phần thưởng từ các hành động không đến ngay lập tức mà thay vào đó là kết quả từ một vài hành động được thực hiện theo trình tự. Chủ đề này đã được theo đuổi trong lĩnh vực **nghiên cứu hoạt động**, xuất hiện trong Thế chiến II từ những nỗ lực ở Anh để tối ưu hóa việc lắp đặt radar, và sau đó tìm thấy vô số ứng dụng dân sự. Công trình của **Richard Bellman** (1957) đã hình thức hóa một lớp các bài toán quyết định tuần tự được gọi là **quá trình quyết định Markov**, mà chúng ta nghiên cứu trong Chương 16 và, dưới tiêu đề học tăng cường, trong Chương 23.

Công trình trong kinh tế học và nghiên cứu hoạt động đã đóng góp nhiều cho khái niệm của chúng ta về các tác nhân hợp lý, nhưng trong nhiều năm, nghiên cứu AI đã phát triển theo những con đường hoàn toàn riêng biệt. Một lý do là sự phức tạp rõ ràng của việc đưa ra các quyết định hợp lý. Nhà nghiên cứu AI tiên phong **Herbert Simon** (1916–2001) đã giành giải Nobel kinh tế năm 1978 cho công trình ban đầu của mình, chỉ ra rằng các mô hình dựa trên **thỏa mãn** (satisficing) – đưa ra các quyết định “đủ tốt”, thay vì tính toán một cách cật lực một quyết định tối ưu – đã đưa ra một mô tả tốt hơn về hành vi thực tế của con người (Simon, 1947). Kể từ những năm 1990, đã có sự hồi sinh của sự quan tâm đến các kỹ thuật lý thuyết quyết định cho AI.

1.2.4 Khoa học thần kinh

- Bộ não xử lý thông tin như thế nào?

Khoa học thần kinh là ngành nghiên cứu hệ thần kinh, đặc biệt là bộ não. Mặc dù cách thức chính xác mà bộ não cho phép tư duy là một trong những bí ẩn lớn của khoa học, nhưng thực tế là nó cho phép tư duy đã được ghi nhận từ hàng nghìn năm nay nhờ bằng chứng cho thấy những cú đánh mạnh vào đầu có thể dẫn đến mất năng lực tinh thần. Từ lâu người ta cũng đã biết rằng bộ não con người bằng cách nào đó là khác biệt; vào khoảng năm 335 TCN, Aristotle đã viết: “Trong tất cả các loài động vật, con người có bộ não lớn nhất so với kích thước của nó.” Tuy nhiên, mãi cho đến giữa thế kỷ 18, bộ não mới được công nhận rộng rãi là trung tâm của ý thức. Trước đó, các vị trí ứng cử viên bao gồm tim và lá lách.

Nghiên cứu của **Paul Broca** (1824–1880) về chứng mất ngôn ngữ (khiếm khuyết lời nói) ở bệnh nhân bị tổn thương não vào năm 1861 đã khởi xướng việc nghiên cứu tổ chức chức năng của não bằng cách xác định một khu vực cục bộ ở bán cầu não trái – hiện được gọi là vùng Broca – chịu trách nhiệm sản xuất lời nói. Vào thời điểm đó, người ta đã biết rằng bộ não chủ yếu bao gồm các tế bào thần kinh, hoặc **neuron**, nhưng phải đến năm 1873, **Camillo Golgi** (1843–1926) mới phát triển một kỹ thuật nhuộm cho phép quan sát các neuron riêng lẻ (xem Hình 1.1). Kỹ thuật này đã được **Santiago Ramon y Cajal** (1852–1934) sử dụng trong các nghiên cứu tiên phong của ông về tổ chức thần kinh. Ngày nay, người ta chấp nhận rộng rãi rằng các chức năng nhận thức là kết quả của hoạt động điện hóa của các cấu trúc này. Tức là, một tập hợp các tế bào đơn giản có thể dẫn đến suy nghĩ, hành động và ý thức. Theo lời nói súc tích của **John Searle** (1992), bộ não gây ra tâm trí.

Hiện chúng ta có một số dữ liệu về sự ánh xạ giữa các vùng của não và các bộ phận của cơ thể mà chúng kiểm soát hoặc từ đó chúng nhận được đầu vào cảm giác. Các ánh xạ như vậy có khả năng thay đổi một cách triệt để trong vài tuần và một số loài động vật dường như có nhiều bản đồ. Hơn nữa, chúng ta không hiểu đầy đủ cách các khu vực khác có thể đảm nhận các chức năng khi một khu vực bị tổn thương. Hầu như không có lý thuyết nào về cách một ký ức cá nhân được lưu trữ hoặc cách các chức năng nhận thức cấp cao hơn hoạt động.

Việc đo hoạt động não còn nguyên vẹn bắt đầu vào năm 1929 với phát minh của **Hans Berger** về **máy điện não đồ** (EEG). Sự phát triển của **chụp cộng hưởng từ chức năng** (fMRI) (Ogawa et al., 1990; Cabeza và Nyberg, 2001) đang cung cấp cho các nhà khoa học thần kinh những hình ảnh chi tiết chưa từng có về hoạt động não, cho phép các phép đo tương ứng một cách thú vị với các quá trình nhận thức đang diễn ra. Những điều này được bổ sung bằng những tiến bộ trong ghi điện tế bào đơn của hoạt động neuron và bằng các phương pháp **quang di truyền học** (Crick, 1999; Zemelman et al., 2002; Han và Boyden, 2007), cho phép cả việc đo lường và kiểm soát các neuron riêng lẻ được sửa đổi để nhạy cảm với ánh sáng.

Sự phát triển của **giao diện não–máy** (Lebedev và Nicolelis, 2006) cho cả cảm nhận và điều khiển vận động không chỉ hứa hẹn sẽ khôi phục chức năng cho những người khuyết tật, mà còn làm sáng tỏ nhiều khía cạnh của hệ thần kinh. Một phát hiện đáng chú ý từ công trình này là bộ não có khả năng tự điều chỉnh để giao tiếp thành công với một thiết bị bên ngoài, coi nó có hiệu lực như một cơ quan cảm giác hoặc một chi khác.

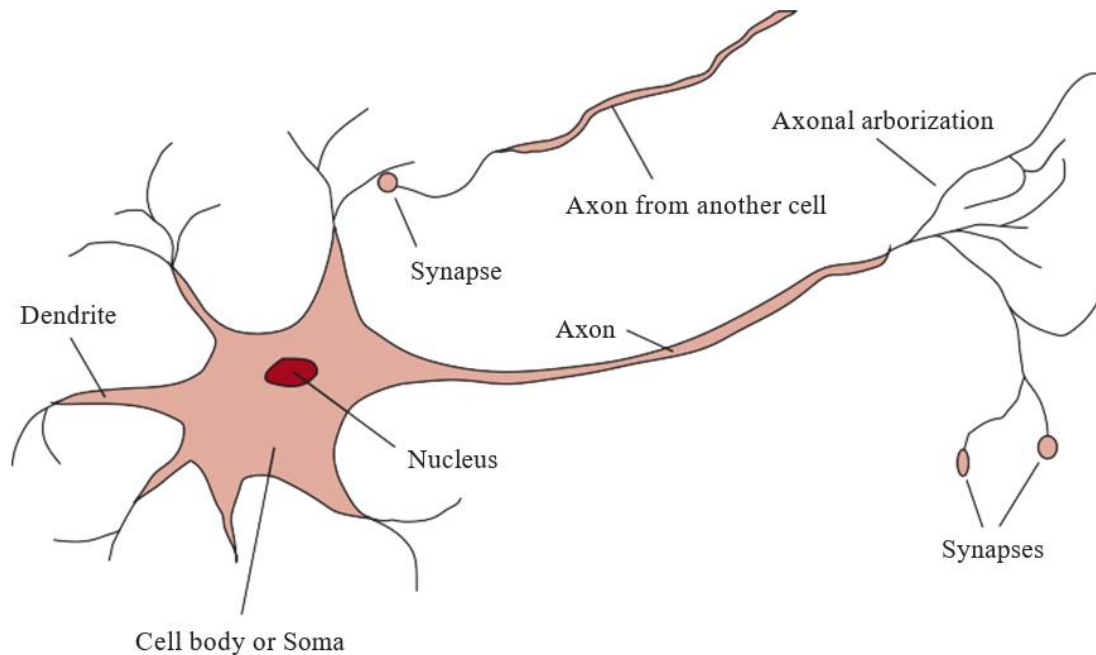


Figure 1.1 The parts of a nerve cell or neuron. Each neuron consists of a cell body, or soma, that contains a cell nucleus. Branching out from the cell body are a number of fibers called dendrites and a single long fiber called the axon. The axon stretches out for a long distance, much longer than the scale in this diagram indicates. Typically, an axon is 1 cm long (100 times the diameter of the cell body), but can reach up to 1 meter. A neuron makes connections with 10 to 100,000 other neurons at junctions called synapses. Signals are propagated from neuron to neuron by a complicated electrochemical reaction. The signals control brain activity in the short term and also enable long-term changes in the connectivity of neurons. These mechanisms are thought to form the basis for learning in the brain. Most information processing goes on in the cerebral cortex, the outer layer of the brain. The basic organizational unit appears to be a column of tissue about 0.5 mm in diameter, containing about 20,000 neurons and extending the full depth of the cortex (about 4 mm in humans).

Bộ não và máy tính kỹ thuật số có một số thuộc tính khác nhau. Hình 1.2 cho thấy máy tính có thời gian chu kỳ nhanh hơn bộ não một triệu lần. Bộ não bù đắp cho điều đó bằng nhiều bộ nhớ và kết nối hơn nhiều so với một máy tính cá nhân cao cấp, mặc dù các siêu máy tính lớn nhất phù hợp với bộ não ở một số chỉ số. Những người theo thuyết vị lai sử dụng rất nhiều những con số này, chỉ ra một **điểm kỳ dị** đang đến gần, tại đó máy tính đạt đến một mức hiệu suất siêu phàm (Vinge, 1993; Kurzweil, 2005; Doctorow và Stross, 2012), và sau đó nhanh chóng tự cải thiện hơn nữa. Nhưng việc so sánh các con số thô không đặc biệt hữu ích. Ngay cả với một máy tính có dung lượng gần như không giới hạn, chúng ta vẫn cần những đột phá về mặt khái niệm hơn nữa trong sự hiểu biết của chúng ta về trí thông minh (xem Chương 29). Nói một cách thô thiển, nếu không có lý thuyết đúng, các cỗ máy nhanh hơn chỉ cho bạn câu trả lời sai nhanh hơn.

1.2.5 Tâm lý học

- Con người và động vật suy nghĩ và hành động như thế nào?

Nguồn gốc của **tâm lý học khoa học** thường được truy ngược về công trình của nhà vật lý người Đức **Hermann von Helmholtz** (1821–1894) và học trò của ông là **Wilhelm Wundt** (1832–1920). Helmholtz đã áp dụng phương pháp khoa học vào việc nghiên cứu thị giác của con người, và cuốn *Sổ tay Quang học Sinh lý* của ông đã được mô tả là “luận văn quan trọng nhất về vật lý và sinh lý học của thị giác con người” (Nalwa, 1993, tr. 15). Năm 1879, Wundt đã mở phòng thí nghiệm tâm lý học thực nghiệm đầu tiên, tại Đại học Leipzig. Wundt khẳng định về các thí nghiệm được kiểm soát cẩn thận, trong đó các nhân viên của ông sẽ thực hiện một nhiệm vụ nhận thức hoặc liên kết trong khi **nội quan** vào quá trình suy nghĩ của họ. Các biện pháp kiểm soát cẩn thận đã giúp tâm lý học trở thành một ngành khoa học, nhưng bản chất chủ quan của dữ liệu khiến các nhà thực nghiệm không thể bác bỏ các lý thuyết của chính họ.

	Supercomputer	Personal Computer	Human Brain
Computational units	10 ⁶ GPUs + CPUs 10 ¹⁵ transistors	8 CPU cores 10 ¹⁰ transistors	10 ⁶ columns 10 ¹¹ neurons
Storage units	10 ¹⁶ bytes RAM 10 ¹⁷ bytes disk	10 ¹⁰ bytes RAM 10 ¹² bytes disk	10 ¹¹ neurons 10 ¹⁴ synapses
Cycle time	10 ⁻⁹ sec	10 ⁻⁹ sec	10 ⁻³ sec
Operations/sec	10 ¹⁸	10 ¹⁰	10 ¹⁷

Figure 1.2 A crude comparison of a leading supercomputer, Summit (Feldman, 2017); a typical personal computer of 2019; and the human brain. Human brain power has not changed much in thousands of years, whereas supercomputers have improved from megaFLOPs in the 1960s to gigaFLOPs in the 1980s, teraFLOPs in the 1990s, petaFLOPs in 2008, and exaFLOPs in 2018 (1 exaFLOP = 10¹⁸ floating point operations per second).

Mặt khác, các nhà sinh học nghiên cứu hành vi động vật thiếu dữ liệu nội quan và đã phát triển một phương pháp luận khách quan, như được mô tả bởi **H. S. Jennings** (1906) trong tác phẩm có ảnh hưởng của ông *Hành vi của các sinh vật hạ đẳng*. Áp dụng quan điểm này cho con người, phong trào **chủ nghĩa hành vi**, do **John Watson** (1878–1958) đứng đầu, đã bác bỏ bất kỳ lý thuyết nào liên quan đến các quá trình tinh thần với lý do nội quan không thể cung cấp bằng chứng đáng tin cậy. Những người theo chủ nghĩa hành vi khẳng định chỉ nghiên cứu các phép đo khách quan của các tri giác (hoặc kích thích) được đưa cho một con vật và các hành động kết quả của nó (hoặc phản ứng). Chủ nghĩa hành vi đã khám phá ra rất nhiều điều về chuột và chim bồ câu nhưng ít thành công hơn trong việc tìm hiểu con người.

Tâm lý học nhận thức, coi bộ não là một thiết bị xử lý thông tin, có thể được truy ngược về ít nhất là các tác phẩm của **William James** (1842–1910). Helmholtz cũng khẳng định rằng nhận thức liên quan đến một dạng suy luận logic vô thức. Quan điểm nhận thức phân lớn đã bị lu mờ bởi chủ nghĩa hành vi ở Hoa Kỳ, nhưng tại Đơn vị Tâm lý học Ứng dụng của Cambridge, do **Frederic Bartlett** (1886–1969) chỉ đạo, mô hình nhận thức đã có thể phát triển. *Bản chất của Giải thích*, của học trò và người kế nhiệm của Bartlett là **Kenneth Craik** (1943), đã mạnh mẽ tái lập tính hợp pháp của các thuật ngữ “tinh thần” như niềm tin và mục tiêu, lập luận rằng chúng cũng khoa học như, chẳng hạn, sử dụng áp suất và nhiệt độ để nói về chất khí, mặc dù chất khí được tạo thành từ các phân tử không có cả hai.

Craik đã chỉ rõ ba bước chính của một tác nhân dựa trên tri thức: (1) kích thích phải được dịch thành một **biểu diễn bên trong**, (2) biểu diễn được thao tác bằng các quá trình nhận thức để suy ra các biểu diễn bên trong mới, và (3) những biểu diễn này lần lượt được dịch lại thành hành động. Ông đã giải thích rõ ràng tại sao đây là một thiết kế tốt cho một tác nhân:

Nếu sinh vật mang một “mô hình thu nhỏ” của thực tại bên ngoài và các hành động có thể có của chính nó bên trong đầu, nó có thể thử các lựa chọn thay thế khác nhau, kết luận cái nào là tốt nhất trong số chúng, phản ứng với các tình huống tương lai trước khi chúng phát sinh, sử dụng kiến thức về các sự kiện trong quá khứ để đối phó với hiện tại và tương lai, và bằng mọi cách để phản ứng một cách đầy đủ hơn, an toàn hơn và có năng lực hơn với những trường hợp khẩn cấp mà nó phải đối mặt. (Craik, 1943)

Sau cái chết của Craik trong một tai nạn xe đạp vào năm 1945, công trình của ông được tiếp tục bởi **Donald Broadbent**, người mà cuốn sách *Nhận thức và Giao tiếp* (1958) là một trong những tác phẩm đầu tiên mô hình hóa các hiện tượng tâm lý như xử lý thông tin. Trong khi đó, ở Hoa Kỳ, sự phát triển của mô hình máy tính đã dẫn đến việc tạo ra lĩnh vực **khoa học nhận thức**. Có thể nói lĩnh vực này đã bắt đầu tại một hội thảo vào tháng 9 năm 1956 tại MIT – chỉ hai tháng sau hội nghị mà chính AI đã được “ra đời”.

Tại hội thảo, **George Miller** đã trình bày *Con số thần kỳ Bảy*, **Noam Chomsky** đã trình bày *Ba mô hình ngôn ngữ*, và **Allen Newell** cùng **Herbert Simon** đã trình bày *Máy lý thuyết logic*. Ba bài báo có ảnh hưởng này đã chỉ ra cách các mô hình máy tính có thể được sử dụng để giải quyết tâm lý học về trí nhớ, ngôn ngữ và tư duy logic, tương ứng. Bây giờ là một quan điểm phổ biến (mặc dù không phải là phổ quát) trong số các nhà tâm lý học rằng “một lý thuyết nhận thức nên giống như một chương trình máy tính” (Anderson, 1980); tức là, nó nên mô tả hoạt động của một chức năng nhận thức theo thuật ngữ xử lý thông tin.

Vì mục đích của bài đánh giá này, chúng tôi sẽ tính lĩnh vực **tương tác giữa con người và máy tính** (HCI) vào tâm lý học. **Doug Engelbart**, một trong những người tiên phong của HCI, đã ủng hộ ý tưởng **tăng cường trí thông minh** – IA thay vì AI. Ông tin rằng máy tính nên tăng cường khả năng của con người hơn là tự động hóa các nhiệm vụ của con người. Năm 1968, “mẹ của mọi bản demo” của Engelbart đã lần đầu tiên giới thiệu chuột máy tính, một hệ thống cửa sổ, siêu văn bản và hội nghị video – tất cả nhằm nỗ lực chứng minh những gì những người lao động tri thức có thể cùng nhau đạt được với một số tăng cường trí thông minh.

Ngày nay, chúng ta có nhiều khả năng coi IA và AI là hai mặt của cùng một đồng xu, với IA nhấn mạnh sự kiểm soát của con người và AI nhấn mạnh hành vi thông minh của cỗ máy. Cả hai đều cần thiết để máy móc trở nên hữu ích cho con người.

1.2.6 Kỹ thuật máy tính

- Làm thế nào chúng ta có thể xây dựng một máy tính hiệu quả?

Máy tính điện tử kỹ thuật số hiện đại đã được phát minh một cách độc lập và gần như đồng thời bởi các nhà khoa học ở ba quốc gia đang tham chiến trong Thế chiến II. Chiếc máy tính hoạt động đầu tiên là **Heath Robinson** cơ điện, được chế tạo vào năm 1943 bởi nhóm của Alan Turing cho một mục đích duy nhất: giải mã các thông điệp của Đức. Năm 1943, cùng một nhóm đã phát triển **Colossus**, một cỗ máy đa năng mạnh mẽ dựa trên ống chân không. Chiếc máy tính có thể lập trình

hoạt động đầu tiên là **Z-3**, phát minh của **Konrad Zuse** ở Đức vào năm 1941. Zuse cũng đã phát minh ra các số dấu phẩy động và ngôn ngữ lập trình cấp cao đầu tiên, **Plankalkül**. Chiếc máy tính điện tử đầu tiên, **ABC**, được **John Atanasoff** và sinh viên của ông **Clifford Berry** lắp ráp từ năm 1940 đến năm 1942 tại Đại học bang Iowa. Nghiên cứu của Atanasoff nhận được rất ít sự hỗ trợ hoặc công nhận; chính **ENIAC**, được phát triển như một phần của một dự án quân sự bí mật tại Đại học Pennsylvania bởi một nhóm bao gồm **John Mauchly** và **J. Presper Eckert**, đã chứng tỏ là tiền thân có ảnh hưởng nhất của các máy tính hiện đại.

Kể từ thời điểm đó, mỗi thế hệ phần cứng máy tính đều mang lại sự gia tăng về tốc độ và dung lượng, và giảm giá – một xu hướng được ghi lại trong **định luật Moore**. Hiệu suất tăng gấp đôi cứ sau khoảng 18 tháng cho đến khoảng năm 2005, khi các vấn đề tản nhiệt khiến các nhà sản xuất bắt đầu nhân số lõi CPU thay vì tốc độ xung nhịp. Các kỳ vọng hiện tại là những gia tăng trong tương lai về chức năng sẽ đến từ **sự song song hóa lớn** – một sự hội tụ kỳ lạ với các đặc tính của bộ não. Chúng ta cũng thấy các thiết kế phần cứng mới dựa trên ý tưởng rằng khi đối phó với một thế giới không chắc chắn, chúng ta không cần độ chính xác 64 bit trong các con số của mình; chỉ 16 bit (như trong định dạng `bf16`) hoặc thậm chí 8 bit là đủ, và sẽ cho phép xử lý nhanh hơn.

Chúng ta mới bắt đầu thấy phần cứng được điều chỉnh cho các ứng dụng AI, chẳng hạn như **đơn vị xử lý đồ họa (GPU)**, **đơn vị xử lý tensor (TPU)**, và **công cụ quy mô wafer (WSE)**. Từ những năm 1960 đến khoảng năm 2012, lượng sức mạnh tính toán được sử dụng để đào tạo các ứng dụng học máy hàng đầu tuân theo định luật Moore. Bắt đầu từ năm 2012, mọi thứ đã thay đổi: từ năm 2012 đến năm 2018, đã có một sự gia tăng 300.000 lần, tương đương với việc tăng gấp đôi cứ sau khoảng 100 ngày (Amodei và Hernandez, 2018). Một mô hình học máy mất một ngày để đào tạo vào năm 2014 chỉ mất hai phút vào năm 2018 (Ying et al., 2018). Mặc dù vẫn chưa thực tế, nhưng **điện toán lượng tử** hứa hẹn sẽ tăng tốc độ nhanh hơn nhiều cho một số phân lớp quan trọng của các thuật toán AI.

Tất nhiên, đã có các thiết bị tính toán trước máy tính điện tử. Các cỗ máy tự động sớm nhất, có từ thế kỷ 17, đã được thảo luận ở trang 24. Cỗ máy có thể lập trình đầu tiên là một khung cửi, được nghĩ ra vào năm 1805 bởi **Joseph Marie Jacquard** (1752–1834), sử dụng các **thẻ đục lỗ** để lưu trữ các hướng dẫn cho mẫu được dệt.

Vào giữa thế kỷ 19, **Charles Babbage** (1792–1871) đã thiết kế hai cỗ máy tính, nhưng ông đã không hoàn thành cái nào. **Động cơ sai phân** được dự định để tính toán các bảng toán học cho các dự án kỹ thuật và khoa học. Cuối cùng nó đã được chế tạo và cho thấy là hoạt động vào năm 1991 (Swade, 2000). **Động cơ phân tích** của Babbage tham vọng hơn nhiều: nó bao gồm bộ nhớ có thể định địa chỉ, các chương trình được lưu trữ dựa trên thẻ đục lỗ của Jacquard và các bước nhảy có điều kiện. Nó là cỗ máy đầu tiên có khả năng tính toán phổ quát.

Đồng nghiệp của Babbage, **Ada Lovelace**, con gái của nhà thơ Lord Byron, đã hiểu tiềm năng của nó, mô tả nó là “một cỗ máy suy nghĩ hoặc... một cỗ máy suy luận,” một cỗ máy có khả năng suy luận về “tất cả các chủ đề trong vũ trụ” (Lovelace, 1843). Bà cũng lường trước các chu kỳ cường điệu của AI, viết rằng, “Mong muốn là phải đề phòng khả năng những ý tưởng phóng đại có thể nảy sinh về sức mạnh của Động cơ phân tích.” Thật không may, các cỗ máy của Babbage và các ý tưởng của Lovelace phần lớn đã bị lãng quên.

AI cũng mang ơn mảnh phần mềm của khoa học máy tính, đã cung cấp các hệ điều hành, ngôn ngữ lập trình và công cụ cần thiết để viết các chương trình hiện đại (và các bài báo về chúng). Nhưng đây là một lĩnh vực mà món nợ đã được trả lại: công trình trong AI đã tiên phong nhiều ý

tưởng đã quay trở lại khoa học máy tính chính thống, bao gồm chia sẻ thời gian, trình thông dịch tương tác, máy tính cá nhân với cửa sổ và chuột, môi trường phát triển nhanh, kiểu dữ liệu danh sách liên kết, quản lý bộ nhớ tự động, và các khái niệm chính của lập trình ký hiệu, chức năng, khai báo và hướng đối tượng.

1.2.7 Lý thuyết điều khiển và điều khiển học

- Làm thế nào các vật thể nhân tạo có thể hoạt động dưới sự kiểm soát của chính chúng?

Ktesibios của Alexandria (khoảng 250 TCN) đã chế tạo cỗ máy tự điều khiển đầu tiên: một chiếc đồng hồ nước có bộ điều chỉnh duy trì tốc độ dòng chảy không đổi. Phát minh này đã thay đổi định nghĩa về những gì một vật thể nhân tạo có thể làm. Trước đây, chỉ có các sinh vật sống mới có thể sửa đổi hành vi của chúng để đáp ứng với những thay đổi trong môi trường. Các ví dụ khác về hệ thống **điều khiển phản hồi tự điều chỉnh** bao gồm bộ điều tốc động cơ hơi nước, được tạo ra bởi **James Watt** (1736–1819), và bộ điều nhiệt, được phát minh bởi **Cornelis Drebbel** (1572–1633), người cũng đã phát minh ra tàu ngầm. **James Clerk Maxwell** (1868) đã khởi xướng lý thuyết toán học về các hệ thống điều khiển.

Một nhân vật trung tâm trong sự phát triển sau chiến tranh của **lý thuyết điều khiển** là **Norbert Wiener** (1894–1964). Wiener là một nhà toán học xuất sắc, người đã làm việc với Bertrand Russell, cùng những người khác, trước khi phát triển sự quan tâm đến các hệ thống điều khiển sinh học và cơ học và mối liên hệ của chúng với nhận thức. Giống như Craik (người cũng sử dụng các hệ thống điều khiển làm mô hình tâm lý), Wiener và các đồng nghiệp của ông là Arturo Rosenblueth và Julian Bigelow đã thách thức **chủ nghĩa hành vi chính thống** (Rosenblueth et al., 1943). Họ xem hành vi có mục đích là phát sinh từ một cơ chế điều chỉnh cố gắng giảm thiểu “sai số” – sự khác biệt giữa trạng thái hiện tại và trạng thái mục tiêu. Vào cuối những năm 1940, Wiener, cùng với Warren McCulloch, Walter Pitts và John von Neumann, đã tổ chức một loạt các hội nghị có ảnh hưởng, khám phá các mô hình toán học và tính toán mới về nhận thức. Cuốn sách *Điều khiển học* (1948) của Wiener đã trở thành một cuốn sách bán chạy nhất và đánh thức công chúng về khả năng của các cỗ máy thông minh nhân tạo. Trong khi đó, ở Anh, **W. Ross Ashby** đã tiên phong những ý tưởng tương tự (Ashby, 1940). Ashby, Alan Turing, Grey Walter và những người khác đã thành lập **Câu lạc bộ Tỷ lệ** (Ratio Club) cho “những người đã có ý tưởng của Wiener trước khi cuốn sách của Wiener xuất hiện.” Cuốn *Thiết kế cho một bộ não* (1948, 1952) của Ashby đã trình bày chi tiết về ý tưởng của ông rằng trí thông minh có thể được tạo ra bằng cách sử dụng các thiết bị **cân bằng nội môi** (homeostatic) chứa các vòng phản hồi thích hợp để đạt được hành vi thích ứng ổn định.

Lý thuyết điều khiển hiện đại, đặc biệt là nhánh được gọi là **điều khiển tối ưu ngẫu nhiên**, có mục tiêu là thiết kế các hệ thống giảm thiểu một **hàm chi phí** theo thời gian. Điều này gần như khớp với mô hình chuẩn của AI: thiết kế các hệ thống hoạt động tối ưu. Vậy tại sao AI và lý thuyết điều khiển lại là hai lĩnh vực khác nhau, mặc dù có mối liên hệ chặt chẽ giữa những người sáng lập của chúng? Câu trả lời nằm ở sự kết nối chặt chẽ giữa các kỹ thuật toán học quen thuộc với những người tham gia và các tập hợp vấn đề tương ứng được bao gồm trong mỗi thế giới quan. **Giải tích** và **đại số ma trận**, các công cụ của lý thuyết điều khiển, phù hợp với các hệ thống có thể được mô tả bằng các tập hợp cố định của các biến liên tục, trong khi AI được thành lập một phần như một cách để thoát khỏi những hạn chế được nhận thức này. Các công cụ **suy luận logic**

và **tính toán** cho phép các nhà nghiên cứu AI xem xét các vấn đề như ngôn ngữ, thị giác và lập kế hoạch ký hiệu hoàn toàn nằm ngoài phạm vi của nhà lý thuyết điều khiển.

1.2.8 Ngôn ngữ học

- Ngôn ngữ liên quan đến tư duy như thế nào?

Năm 1957, B. F. Skinner xuất bản *Hành vi bằng lời nói*. Đây là một bản tường thuật toàn diện, chi tiết về cách tiếp cận **chủ nghĩa hành vi** đối với việc học ngôn ngữ, được viết bởi chuyên gia hàng đầu trong lĩnh vực này. Nhưng thật kỳ lạ, một bài phê bình về cuốn sách này lại trở nên nổi tiếng như chính cuốn sách, và gần như đã giết chết sự quan tâm đến chủ nghĩa hành vi. Tác giả của bài phê bình là nhà ngôn ngữ học **Noam Chomsky**, người vừa xuất bản một cuốn sách về lý thuyết của riêng ông, *Cấu trúc cú pháp*. Chomsky chỉ ra rằng lý thuyết hành vi không giải quyết được khái niệm sáng tạo trong ngôn ngữ – nó không giải thích được tại sao trẻ em có thể hiểu và tạo ra những câu mà chúng chưa từng nghe trước đây. Lý thuyết của Chomsky – dựa trên các mô hình cú pháp có từ nhà ngôn ngữ học Ấn Độ Panini (khoảng 350 TCN) – có thể giải thích điều này, và không giống như các lý thuyết trước đây, nó đủ hình thức để về nguyên tắc có thể được lập trình.

Do đó, **ngôn ngữ học** và AI hiện đại “ra đời” cùng một lúc, và cùng nhau phát triển, giao nhau trong một lĩnh vực lai gọi là **ngôn ngữ học tính toán** hoặc **xử lý ngôn ngữ tự nhiên**. Vấn đề hiểu ngôn ngữ hóa ra phức tạp hơn đáng kể so với vẻ ngoài của nó vào năm 1957. Hiểu ngôn ngữ đòi hỏi sự hiểu biết về chủ đề và ngữ cảnh, không chỉ hiểu cấu trúc của câu. Điều này có vẻ hiển nhiên, nhưng nó không được công nhận rộng rãi cho đến những năm 1960. Phần lớn công việc ban đầu trong **biểu diễn tri thức** (nghiên cứu về cách đưa tri thức vào một dạng mà máy tính có thể suy luận được) gắn liền với ngôn ngữ và được thông báo bởi nghiên cứu trong ngôn ngữ học, vốn được kết nối với hàng thập kỷ làm việc về phân tích triết học của ngôn ngữ.

1.3 Lịch sử của Trí tuệ Nhân tạo

Một cách nhanh chóng để tóm tắt các mốc quan trọng trong lịch sử AI là liệt kê những người đoạt giải Turing: **Marvin Minsky** (1969) và **John McCarthy** (1971) vì đã xác định các nền tảng của lĩnh vực dựa trên biểu diễn và suy luận; **Allen Newell** và **Herbert Simon** (1975) cho các mô hình ký hiệu về giải quyết vấn đề và nhận thức của con người; **Ed Feigenbaum** và **Raj Reddy** (1994) vì đã phát triển các hệ thống chuyên gia mã hóa tri thức của con người để giải quyết các vấn đề trong thế giới thực; **Judea Pearl** (2011) vì đã phát triển các kỹ thuật suy luận xác suất xử lý sự không chắc chắn một cách có nguyên tắc; và cuối cùng **Yoshua Bengio**, **Geoffrey Hinton**, và **Yann LeCun** (2019) vì đã biến “học sâu” (mạng lưới thần kinh nhiều lớp) trở thành một phần quan trọng của điện toán hiện đại. Phần còn lại của phần này đi sâu hơn vào từng giai đoạn của lịch sử AI.

1.3.1 Sự khởi đầu của trí tuệ nhân tạo (1943–1956)

Công trình đầu tiên hiện được công nhận rộng rãi là AI được thực hiện bởi **Warren McCulloch** và **Walter Pitts** (1943). Lấy cảm hứng từ công trình mô hình hóa toán học của cổ vấn của Pitts là

Nicolas Rashevsky (1936, 1938), họ đã dựa trên ba nguồn: kiến thức về sinh lý học cơ bản và chức năng của các neuron trong não; một phân tích hình thức về logic mệnh đề của Russell và Whitehead; và lý thuyết tính toán của Turing. Họ đã đề xuất một mô hình các **neuron nhân tạo** trong đó mỗi neuron được đặc trưng là “bật” hoặc “tắt”, với một công tắc sang “bật” xảy ra để đáp lại sự kích thích của một số lượng neuron lân cận đủ. Trạng thái của một neuron được hình thành là “tương đương về mặt thực tế với một mệnh đề đề xuất kích thích đầy đủ của nó.” Họ đã chỉ ra, ví dụ, rằng bất kỳ hàm có thể tính toán nào cũng có thể được tính toán bởi một số mạng lưới các neuron được kết nối, và rằng tất cả các liên kết logic (AND, OR, NOT, v.v.) có thể được triển khai bởi các cấu trúc mạng đơn giản. McCulloch và Pitts cũng gợi ý rằng các mạng được xác định phù hợp có thể học. **Donald Hebb** (1949) đã chứng minh một quy tắc cập nhật đơn giản để sửa đổi sức mạnh kết nối giữa các neuron. Quy tắc của ông, hiện được gọi là **học Hebbian**, vẫn là một mô hình có ảnh hưởng cho đến ngày nay.

Hai sinh viên đại học tại Harvard, **Marvin Minsky** (1927–2016) và **Dean Edmonds**, đã chế tạo chiếc máy tính mạng lưới thần kinh đầu tiên vào năm 1950. **SNARC**, tên gọi của nó, đã sử dụng 3000 ống chân không và một cơ chế lái tự động dư thừa từ một máy bay ném bom B-24 để mô phỏng một mạng lưới 40 neuron. Sau đó, tại Princeton, Minsky đã nghiên cứu tính toán phổ quát trong các mạng lưới thần kinh. Hội đồng tiền sĩ của ông hoài nghi về việc liệu loại công việc này có nên được coi là toán học hay không, nhưng Von Neumann được cho là đã nói: “Nếu bây giờ chưa phải, thì một ngày nào đó sẽ là.” Có một số ví dụ khác về công việc ban đầu có thể được đặc trưng là AI, bao gồm hai chương trình chơi cờ đam được phát triển độc lập vào năm 1952 bởi **Christopher Strachey** tại Đại học Manchester và bởi **Arthur Samuel** tại IBM. Tuy nhiên, tầm nhìn của **Alan Turing** là có ảnh hưởng nhất. Ông đã thuyết trình về chủ đề này sớm nhất là vào năm 1947 tại Hội Toán học Luân Đôn và đã trình bày một chương trình nghị sự thuyết phục trong bài báo “Máy tính và Trí thông minh” năm 1950 của mình. Trong đó, ông đã giới thiệu **bài kiểm tra Turing**, học máy, thuật toán di truyền và học tăng cường. Ông đã giải quyết nhiều phản đối được đưa ra đối với khả năng của AI, như được mô tả trong Chương 28. Ông cũng gợi ý rằng sẽ dễ dàng hơn để tạo ra AI ở cấp độ con người bằng cách phát triển các thuật toán học và sau đó dạy máy hơn là bằng cách lập trình trí thông minh của nó bằng tay. Trong các bài giảng sau đó, ông đã cảnh báo rằng việc đạt được mục tiêu này có thể không phải là điều tốt nhất cho loài người.

Năm 1955, **John McCarthy** của Đại học Dartmouth đã thuyết phục Minsky, Claude Shannon, và Nathaniel Rochester giúp ông tập hợp các nhà nghiên cứu Hoa Kỳ quan tâm đến lý thuyết automata, mạng thần kinh và nghiên cứu về trí thông minh. Họ đã tổ chức một hội thảo kéo dài hai tháng tại Dartmouth vào mùa hè năm 1956. Có tổng cộng 10 người tham dự, bao gồm Allen Newell và Herbert Simon từ Carnegie Tech, Trenchard More từ Princeton, Arthur Samuel từ IBM, và Ray Solomonoff và Oliver Selfridge từ MIT. Đề xuất nêu rõ:

Chúng tôi đề xuất một nghiên cứu về trí tuệ nhân tạo kéo dài 2 tháng, với 10 người, được thực hiện trong mùa hè năm 1956 tại Đại học Dartmouth ở Hanover, New Hampshire. Nghiên cứu sẽ tiến hành trên cơ sở giả định rằng mọi khía cạnh của học tập hoặc bất kỳ đặc điểm nào khác của trí thông minh về nguyên tắc đều có thể được mô tả chính xác đến mức một cỗ máy có thể được chế tạo để mô phỏng nó. Một nỗ lực sẽ được thực hiện để tìm cách làm cho máy móc sử dụng ngôn ngữ, tạo ra các khái niệm và ý tưởng trừu tượng, giải quyết các loại vấn đề hiện chỉ dành cho con người, và tự cải thiện. Chúng tôi nghĩ rằng một tiến bộ đáng kể có thể được thực hiện trong một hoặc nhiều vấn đề này nếu một nhóm các nhà khoa học được lựa chọn cẩn thận cùng làm việc với nhau trong một mùa hè.

Mặc dù có dự đoán lạc quan này, hội thảo Dartmouth đã không dẫn đến bất kỳ đột phá nào. Newell và Simon đã trình bày công trình có lẽ là chín chắn nhất, một hệ thống chứng minh định lý toán học gọi là **Logic Theorist (LT)**. Simon tuyên bố: “Chúng tôi đã phát minh ra một chương trình máy tính có khả năng suy nghĩ phi số học, và do đó đã giải quyết được vấn đề tâm–thân đáng kính.” Ngay sau hội thảo, chương trình đã có thể chứng minh hầu hết các định lý trong Chương 2 của *Principia Mathematica* của Russell và Whitehead. Russell được cho là đã rất vui mừng khi được biết rằng LT đã đưa ra một bằng chứng cho một định lý ngắn hơn bằng chứng trong *Principia*. Các biên tập viên của *Journal of Symbolic Logic* thì ít ấn tượng hơn; họ đã từ chối một bài báo đồng tác giả bởi Newell, Simon, và Logic Theorist.

1.3.2 Sự nhiệt tình ban đầu, những kỳ vọng lớn (1952–1969)

Giới trí thức những năm 1950, phần lớn, thích tin rằng “một cỗ máy không bao giờ có thể làm X.” (Xem Chương 28 để biết một danh sách dài các X được Turing thu thập.) Các nhà nghiên cứu AI đương nhiên đã phản ứng bằng cách chứng minh hết X này đến X khác. Họ tập trung đặc biệt vào các nhiệm vụ được coi là dấu hiệu của trí thông minh ở con người, bao gồm các trò chơi, câu đố, toán học và các bài kiểm tra IQ. John McCarthy gọi giai đoạn này là kỷ nguyên “Nhìn kia, mẹ ơi, không cần tay!”.

Newell và Simon đã tiếp nối thành công của họ với LT bằng **General Problem Solver (GPS)**. Không giống như LT, chương trình này được thiết kế ngay từ đầu để bắt chước các giao thức giải quyết vấn đề của con người. Trong lớp các câu đố giới hạn mà nó có thể xử lý, hóa ra thứ tự mà chương trình xem xét các mục tiêu phụ và các hành động có thể có tương tự như cách con người tiếp cận cùng một vấn đề. Do đó, GPS có lẽ là chương trình đầu tiên thể hiện cách tiếp cận “tư duy như con người.” Sự thành công của GPS và các chương trình sau đó như các mô hình nhận thức đã khiến Newell và Simon (1976) đưa ra **giả thuyết hệ thống ký hiệu vật lý** nổi tiếng, trong đó nói rằng “một hệ thống ký hiệu vật lý có phương tiện cần thiết và đầy đủ cho hành động thông minh tổng quát.” Điều họ muốn nói là bất kỳ hệ thống nào (con người hay máy móc) thể hiện trí thông minh phải hoạt động bằng cách thao tác các cấu trúc dữ liệu bao gồm các ký hiệu. Chúng ta sẽ thấy sau này rằng giả thuyết này đã bị thách thức từ nhiều hướng.

Tại IBM, **Nathaniel Rochester** và các đồng nghiệp của ông đã tạo ra một số chương trình AI đầu tiên. **Herbert Gelernter** (1959) đã xây dựng **Geometry Theorem Prover**, có khả năng chứng minh các định lý mà nhiều sinh viên toán học sẽ thấy khá khó khăn. Công trình này là tiền thân của các trình chứng minh định lý toán học hiện đại.

Trong tất cả các công trình khám phá được thực hiện trong giai đoạn này, có lẽ có ảnh hưởng nhất về lâu dài là công trình của **Arthur Samuel** về cờ đam. Sử dụng các phương pháp mà chúng ta hiện gọi là **học tăng cường** (xem Chương 23), các chương trình của Samuel đã học cách chơi ở cấp độ nghiệp dư mạnh. Do đó, ông đã bác bỏ ý tưởng rằng máy tính chỉ có thể làm những gì chúng được bảo: chương trình của ông nhanh chóng học cách chơi tốt hơn người tạo ra nó. Chương trình đã được trình diễn trên truyền hình vào năm 1956, tạo ra một ấn tượng mạnh. Giống như Turing, Samuel gặp khó khăn trong việc tìm thời gian sử dụng máy tính. Làm việc vào ban đêm, ông đã sử dụng các cỗ máy vẫn còn trên sàn thử nghiệm tại nhà máy sản xuất của IBM. Chương trình của Samuel là tiền thân của các hệ thống sau này như **TD-GAMMON** (Tesauro, 1992), vốn

là một trong những người chơi backgammon giỏi nhất thế giới, và **ALPHA-GO** (Silver et al., 2016), đã gây sốc cho thế giới khi đánh bại nhà vô địch cờ vây thế giới (xem Chương 6).

Năm 1958, **John McCarthy** đã có hai đóng góp quan trọng cho AI. Trong **MIT AI Lab Memo No. 1**, ông đã định nghĩa ngôn ngữ cấp cao **Lisp**, vốn sẽ trở thành ngôn ngữ lập trình AI thống trị trong 30 năm tới. Trong một bài báo có tựa đề *Các chương trình với lẽ thường*, ông đã đưa ra một đề xuất về mặt khái niệm cho các hệ thống AI dựa trên tri thức và suy luận. Bài báo mô tả **Advice Taker**, một chương trình giả định sẽ thể hiện tri thức chung về thế giới và có thể sử dụng nó để đưa ra các kế hoạch hành động. Khái niệm này được minh họa bằng các tiên đề logic đơn giản đủ để tạo ra một kế hoạch lái xe đến sân bay. Chương trình cũng được thiết kế để chấp nhận các tiên đề mới trong quá trình hoạt động bình thường, do đó cho phép nó đạt được năng lực trong các lĩnh vực mới mà không cần phải lập trình lại. Do đó, Advice Taker đã thể hiện các nguyên tắc trung tâm của **biểu diễn tri thức và suy luận**: rằng việc có một biểu diễn hình thức, rõ ràng về thế giới và cách thức hoạt động của nó và có khả năng thao tác biểu diễn đó bằng các quá trình suy diễn là hữu ích. Bài báo đã ảnh hưởng đến quá trình của AI và vẫn còn liên quan cho đến ngày nay.

Năm 1958 cũng đánh dấu năm **Marvin Minsky** chuyển đến MIT. Tuy nhiên, sự hợp tác ban đầu của ông với McCarthy đã không kéo dài. McCarthy nhấn mạnh biểu diễn và suy luận trong logic hình thức, trong khi Minsky quan tâm hơn đến việc làm cho các chương trình hoạt động và cuối cùng đã phát triển một quan điểm chống logic. Năm 1963, McCarthy bắt đầu phòng thí nghiệm AI tại Stanford. Kế hoạch của ông để sử dụng logic để xây dựng Advice Taker tối thượng đã được thúc đẩy bởi việc **J. A. Robinson** khám phá ra vào năm 1965 phương pháp **phân giải** (một thuật toán chứng minh định lý hoàn chỉnh cho logic bậc nhất; xem Chương 9). Công việc tại Stanford nhấn mạnh các phương pháp đa năng cho suy luận logic. Các ứng dụng của logic bao gồm các hệ thống trả lời câu hỏi và lập kế hoạch của **Cordell Green** (Green, 1969b) và dự án robot Shakey tại **Viện Nghiên cứu Stanford (SRI)**. Dự án sau này, được thảo luận thêm trong Chương 26, là dự án đầu tiên chứng minh sự tích hợp hoàn toàn của suy luận logic và hoạt động vật lý.

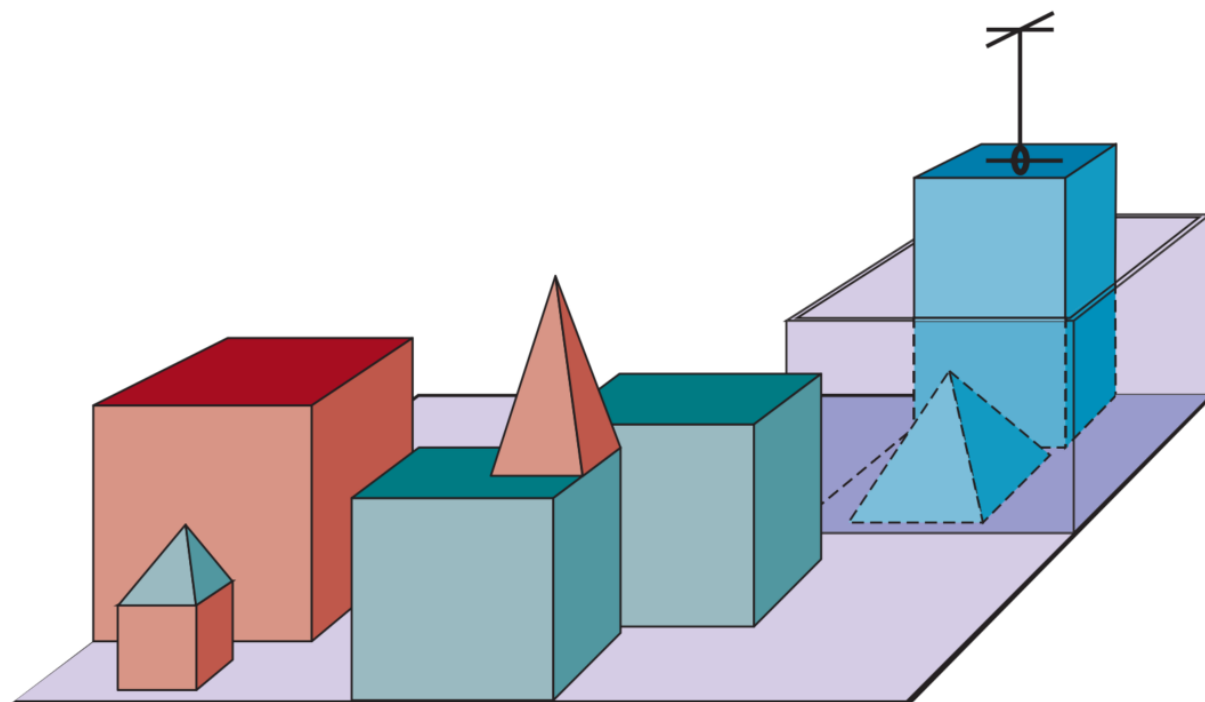


Figure 1.3 A scene from the blocks world. SHRDLU (Winograd, 1972) has just completed the command “Find a block which is taller than the one you are holding and put it in the box.”

Tại MIT, Minsky đã giám sát một loạt sinh viên chọn các vấn đề giới hạn có vẻ đòi hỏi trí thông minh để giải quyết. Những lĩnh vực giới hạn này được gọi là **thế giới vi mô** (microworld). Chương trình **SAINT** của **James Slagle** (1963) có thể giải quyết các bài toán tích phân vi tích phân dạng đóng điển hình của các khóa học đại học năm thứ nhất. Chương trình **ANALOGY** của **Tom Evans** (1968) đã giải quyết các bài toán tương tự hình học xuất hiện trong các bài kiểm tra IQ. Chương trình **STUDENT** của **Daniel Bobrow** (1967) đã giải các bài toán đồ đại số, chẳng hạn như bài sau:

Nếu số khách hàng mà Tom có được gấp đôi bình phương của 20% số quảng cáo mà anh ta chạy, và số quảng cáo mà anh ta chạy là 45, thì số khách hàng mà Tom có được là bao nhiêu?

Thế giới vi mô nổi tiếng nhất là **thế giới khối** (blocks world), bao gồm một tập hợp các khối rắn được đặt trên một mặt bàn (hoặc thường xuyên hơn, một mô phỏng của một mặt bàn), như được hiển thị trong Hình 1.3. Một nhiệm vụ điển hình trong thế giới này là sắp xếp lại các khối theo một cách nhất định, sử dụng một bàn tay robot có thể nhặt từng khối một. Thế giới khối là nơi có dự án thị giác của **David Huffman** (1971), công trình thị giác và lan truyền ràng buộc của **David Waltz** (1975), lý thuyết học tập của **Patrick Winston** (1970), chương trình hiểu ngôn ngữ tự nhiên của **Terry Winograd** (1972), và trình lập kế hoạch của **Scott Fahlman** (1974).

Công việc ban đầu xây dựng trên các mạng thần kinh của McCulloch và Pitts cũng phát triển mạnh mẽ. Công trình của **Shmuel Winograd** và **Jack Cowan** (1963) đã chỉ ra cách một số lượng lớn các phần tử có thể cùng nhau biểu diễn một khái niệm cá nhân, với sự gia tăng tương ứng về độ bền và tính song song. Các phương pháp học của Hebb đã được tăng cường bởi **Bernie Widrow** (Widrow và Hoff, 1960; Widrow, 1962), người đã gọi mạng của mình là **adalines**, và bởi **Frank Rosenblatt** (1962) với các **perceptron** của ông. **Định lý hội tụ perceptron** (Block et al., 1962) nói rằng thuật toán học có thể điều chỉnh sức mạnh kết nối của một perceptron để khớp với bất kỳ dữ liệu đầu vào nào, miễn là tồn tại một sự khớp như vậy.

1.3.3 Một liều thuốc thực tế (1966–1973)

Ngay từ đầu, các nhà nghiên cứu AI đã không ngại đưa ra những dự đoán về những thành công sắp tới của họ. Tuyên bố sau đây của **Herbert Simon** vào năm 1957 thường được trích dẫn:

Tôi không nhằm mục đích làm bạn ngạc nhiên hay sốc—nhưng cách đơn giản nhất tôi có thể tóm tắt là nói rằng hiện nay trên thế giới có những cỗ máy biết suy nghĩ, học hỏi và sáng tạo. Hơn nữa, khả năng làm những điều này của chúng sẽ tăng lên nhanh chóng cho đến khi—trong một tương lai có thể thấy—phạm vi vấn đề mà chúng có thể xử lý sẽ tương đương với phạm vi mà tâm trí con người đã được áp dụng.

Thuật ngữ “tương lai có thể thấy” là mơ hồ, nhưng Simon cũng đưa ra những dự đoán cụ thể hơn: rằng trong vòng 10 năm, một máy tính sẽ trở thành nhà vô địch cờ vua và một định lý toán học quan trọng sẽ được chứng minh bằng máy. Những dự đoán này đã trở thành sự thật (hoặc gần đúng) trong vòng 40 năm thay vì 10. Sự tự tin thái quá của Simon là do hiệu suất đầy hứa hẹn của

các hệ thống AI ban đầu trên các ví dụ đơn giản. Tuy nhiên, trong hầu hết các trường hợp, các hệ thống ban đầu này đã thất bại trên các vấn đề khó hơn.

Có hai lý do chính cho sự thất bại này. Thứ nhất là nhiều hệ thống AI ban đầu chủ yếu dựa trên “nội quan có thông tin” về cách con người thực hiện một nhiệm vụ, thay vì dựa trên phân tích cẩn thận về nhiệm vụ, ý nghĩa của việc trở thành một giải pháp và một thuật toán sẽ cần phải làm gì để tạo ra các giải pháp đó một cách đáng tin cậy.

Lý do thứ hai cho sự thất bại là thiếu sự đánh giá đúng mức về tính không khả thi của nhiều vấn đề mà AI đang cố gắng giải quyết. Hầu hết các hệ thống giải quyết vấn đề ban đầu hoạt động bằng cách thử các sự kết hợp khác nhau của các bước cho đến khi tìm thấy giải pháp. Chiến lược này ban đầu hoạt động vì các **thế giới vi mô** chứa rất ít đối tượng và do đó rất ít hành động có thể có và các chuỗi giải pháp rất ngắn. Trước khi lý thuyết về **độ phức tạp tính toán** được phát triển, người ta thường nghĩ rằng “mở rộng quy mô” lên các vấn đề lớn hơn chỉ đơn giản là vấn đề phần cứng nhanh hơn và bộ nhớ lớn hơn. Sự lạc quan đi kèm với sự phát triển của chứng minh định lý phân giải, ví dụ, đã sớm bị dập tắt khi các nhà nghiên cứu không thể chứng minh các định lý liên quan đến nhiều hơn một vài chục sự kiện. Thực tế là một chương trình có thể tìm thấy một giải pháp về nguyên tắc không có nghĩa là chương trình đó chứa bất kỳ cơ chế nào cần thiết để tìm thấy nó trong thực tế.

Ảo tưởng về sức mạnh tính toán không giới hạn không chỉ giới hạn ở các chương trình giải quyết vấn đề. Các thí nghiệm ban đầu về **tiến hóa máy móc** (hiện được gọi là **lập trình di truyền**) (Friedberg, 1958; Friedberg et al., 1959) dựa trên niềm tin chắc chắn rằng bằng cách thực hiện một loạt các đột biến nhỏ thích hợp cho một chương trình mã máy, người ta có thể tạo ra một chương trình có hiệu suất tốt cho bất kỳ nhiệm vụ cụ thể nào. Sau đó, ý tưởng là thử các đột biến ngẫu nhiên với một quy trình chọn lọc để bảo toàn các đột biến có vẻ hữu ích. Mặc dù hàng nghìn giờ thời gian CPU đã được dành ra, gần như không có tiến bộ nào được chứng minh.

Thất bại trong việc đối phó với “**sự bùng nổ tổ hợp**” là một trong những lời chỉ trích chính về AI có trong **báo cáo Lighthill** (Lighthill, 1973), vốn là cơ sở cho quyết định của chính phủ Anh chấm dứt hỗ trợ nghiên cứu AI ở tất cả trừ hai trường đại học. (Truyền miệng về ra một bức tranh hơi khác và nhiều màu sắc hơn, với các tham vọng chính trị và sự thù địch cá nhân mà việc mô tả nó là không cần thiết.)

Một khó khăn thứ ba nảy sinh do một số hạn chế cơ bản trên các cấu trúc cơ bản được sử dụng để tạo ra hành vi thông minh. Ví dụ, cuốn sách *Perceptrons* (1969) của Minsky và Papert đã chứng minh rằng, mặc dù các **perceptron** (một dạng mạng thần kinh đơn giản) có thể được chứng minh là học được bất cứ điều gì chúng có khả năng biểu diễn, chúng có thể biểu diễn rất ít. Cụ thể, một perceptron hai đầu vào không thể được đào tạo để nhận ra khi hai đầu vào của nó khác nhau. Mặc dù kết quả của họ không áp dụng cho các mạng nhiều lớp phức tạp hơn, nhưng kinh phí nghiên cứu cho nghiên cứu mạng thần kinh đã sớm giảm xuống gần như bằng không. Trớ trêu thay, các thuật toán học **lan truyền ngược** mới, vốn sẽ gây ra một sự hồi sinh to lớn trong nghiên cứu mạng thần kinh vào cuối những năm 1980 và một lần nữa vào những năm 2010, đã được phát triển trong các ngữ cảnh khác vào đầu những năm 1960 (Kelley, 1960; Bryson, 1962).

1.3.4 Hệ thống chuyên gia (1969–1986)

Bức tranh về giải quyết vấn đề đã nảy sinh trong thập kỷ đầu tiên của nghiên cứu AI là một cơ chế tìm kiếm đa năng cố gắng xâu chuỗi các bước suy luận cơ bản để tìm ra các giải pháp hoàn chỉnh. Các cách tiếp cận như vậy đã được gọi là **phương pháp yếu** (weak method) vì, mặc dù tổng quát, chúng không mở rộng quy mô cho các trường hợp vấn đề lớn hoặc khó. Cách thay thế cho các phương pháp yếu là sử dụng tri thức mạnh hơn, **cụ thể cho lĩnh vực** (domain-specific knowledge), cho phép các bước suy luận lớn hơn và có thể xử lý các trường hợp thường xảy ra trong các lĩnh vực chuyên môn hẹp dễ dàng hơn. Có thể nói rằng để giải một vấn đề khó, bạn phải gần như đã biết câu trả lời.

Chương trình **DENDRAL** (Buchanan et al., 1969) là một ví dụ ban đầu về cách tiếp cận này. Nó được phát triển tại Stanford, nơi **Ed Feigenbaum** (một cựu sinh viên của Herbert Simon), **Bruce Buchanan** (một triết gia trở thành nhà khoa học máy tính), và **Joshua Lederberg** (một nhà di truyền học đoạt giải Nobel) đã hợp tác để giải quyết vấn đề suy ra cấu trúc phân tử từ thông tin được cung cấp bởi một máy quang phổ khối. Đầu vào cho chương trình bao gồm công thức cơ bản của phân tử (ví dụ: $C_6H_{13}NO_2$) và phổ khối lượng đưa ra khối lượng của các mảnh phân tử khác nhau được tạo ra khi nó bị bắn phá bởi một chùm electron. Ví dụ, phổ khối lượng có thể chứa một đỉnh ở $m = 15$, tương ứng với khối lượng của một mảnh methyl (CH_3).

Phiên bản ngây thơ của chương trình đã tạo ra tất cả các cấu trúc có thể có phù hợp với công thức, và sau đó dự đoán phổ khối lượng nào sẽ được quan sát cho mỗi cấu trúc, so sánh điều này với phổ thực tế. Như người ta có thể mong đợi, điều này là không khả thi ngay cả đối với các phân tử có kích thước vừa phải. Các nhà nghiên cứu DENDRAL đã hỏi ý kiến các nhà hóa học phân tích và thấy rằng họ làm việc bằng cách tìm kiếm các mẫu đỉnh đã biết trong phổ gợi ý các cấu trúc phụ phổ biến trong phân tử. Ví dụ, quy tắc sau đây được sử dụng để nhận ra một nhóm phụ ketone ($C=O$) (nặng 28):

Nếu M là khối lượng của toàn bộ phân tử và có hai đỉnh tại x_1 và x_2 sao cho: (a) $x_1 + x_2 = M + 28$; (b) $x_1 - 28$ là một đỉnh cao; (c) $x_2 - 28$ là một đỉnh cao; (d) Ít nhất một trong hai đỉnh x_1 và x_2 là cao thì có một nhóm phụ ketone.

Việc nhận ra rằng phân tử chứa một cấu trúc phụ cụ thể làm giảm đáng kể số lượng ứng cử viên có thể có. Theo các tác giả của nó, DENDRAL mạnh mẽ vì nó thể hiện kiến thức liên quan về quang phổ khối không phải dưới dạng các nguyên tắc đầu tiên mà dưới dạng “**công thức nấu ăn**” hiệu quả (Feigenbaum et al., 1971). Ý nghĩa của DENDRAL là nó là hệ thống **chuyên sâu về tri thức** thành công đầu tiên: chuyên môn của nó bắt nguồn từ số lượng lớn các quy tắc chuyên dụng. Năm 1971, Feigenbaum và những người khác tại Stanford đã bắt đầu **Dự án Lập trình Heuristic (HPP)** để điều tra mức độ mà phương pháp luận mới của **hệ thống chuyên gia** có thể được áp dụng cho các lĩnh vực khác.

Nỗ lực lớn tiếp theo là hệ thống **MYCIN** để chẩn đoán nhiễm trùng máu. Với khoảng 450 quy tắc, MYCIN có thể hoạt động tốt như một số chuyên gia, và tốt hơn đáng kể so với các bác sĩ trẻ. Nó cũng chứa hai điểm khác biệt chính so với DENDRAL. Thứ nhất, không giống như các quy tắc DENDRAL, không có mô hình lý thuyết tổng quát nào tồn tại để từ đó các quy tắc MYCIN có thể được suy ra. Chúng phải được thu thập từ việc phỏng vấn rộng rãi các chuyên gia. Thứ hai, các quy tắc phải phản ánh sự không chắc chắn liên quan đến kiến thức y học. MYCIN đã kết hợp một

phép tính không chắc chắn được gọi là **hệ số chắc chắn** (xem Chương 13), vốn dường như (vào thời điểm đó) phù hợp với cách các bác sĩ đánh giá tác động của bằng chứng đối với chẩn đoán.

Hệ thống chuyên gia thương mại thành công đầu tiên, **R1**, bắt đầu hoạt động tại Tập đoàn Thiết bị Kỹ thuật số (**Digital Equipment Corporation**) (McDermott, 1982). Chương trình giúp cấu hình các đơn đặt hàng cho các hệ thống máy tính mới; đến năm 1986, nó đã tiết kiệm cho công ty ước tính 40 triệu đô la một năm. Đến năm 1988, nhóm AI của DEC đã có 40 hệ thống chuyên gia được triển khai, với nhiều hệ thống nữa đang được phát triển. DuPont đã có 100 hệ thống đang được sử dụng và 500 hệ thống đang được phát triển. Gần như mọi tập đoàn lớn của Hoa Kỳ đều có nhóm AI của riêng mình và đang sử dụng hoặc nghiên cứu các hệ thống chuyên gia.

Tầm quan trọng của **tri thức lĩnh vực** cũng rõ ràng trong lĩnh vực **hiểu ngôn ngữ tự nhiên**. Mặc dù hệ thống **SHRDLU** của Winograd đã thành công, nhưng các phương pháp của nó không mở rộng sang các nhiệm vụ tổng quát hơn: đối với các vấn đề như giải quyết sự mơ hồ, nó đã sử dụng các quy tắc đơn giản dựa vào phạm vi nhỏ của thế giới khối.

Một số nhà nghiên cứu, bao gồm **Eugene Charniak** tại MIT và **Roger Schank** tại Yale, đã gợi ý rằng việc hiểu ngôn ngữ mạnh mẽ sẽ đòi hỏi tri thức chung về thế giới và một phương pháp chung để sử dụng tri thức đó. (Schank đã đi xa hơn, tuyên bố, “Không có thứ gọi là cú pháp,” điều này đã làm phiền nhiều nhà ngôn ngữ học nhưng đã giúp bắt đầu một cuộc thảo luận hữu ích.) Schank và các sinh viên của ông đã xây dựng một loạt các chương trình (Schank và Abelson, 1977; Wilensky, 1978; Schank và Riesbeck, 1981) đều có nhiệm vụ hiểu ngôn ngữ tự nhiên. Tuy nhiên, trọng tâm ít hơn vào bản thân ngôn ngữ và nhiều hơn vào các vấn đề **biểu diễn và suy luận** với tri thức cần thiết để hiểu ngôn ngữ.

Sự phát triển rộng rãi của các ứng dụng cho các vấn đề trong thế giới thực đã dẫn đến sự phát triển của một loạt các công cụ biểu diễn và suy luận. Một số dựa trên logic – ví dụ, ngôn ngữ **Prolog** trở nên phổ biến ở châu Âu và Nhật Bản, và họ **PLANNER** ở Hoa Kỳ. Những người khác, theo ý tưởng về **khung** (frames) của Minsky (1975), đã áp dụng một cách tiếp cận có cấu trúc hơn, lắp ráp các sự kiện về các loại đối tượng và sự kiện cụ thể và sắp xếp các loại vào một hệ thống phân cấp phân loại lớn tương tự như phân loại sinh học.

Năm 1981, chính phủ Nhật Bản đã công bố dự án “Thế hệ thứ năm”, một kế hoạch 10 năm để xây dựng các máy tính thông minh song song lớn chạy Prolog. Ngân sách sẽ vượt quá 1,3 tỷ đô la theo tiền ngày nay. Để đáp lại, Hoa Kỳ đã thành lập **Tập đoàn Vi điện tử và Công nghệ Máy tính (MCC)**, một tập đoàn được thiết kế để đảm bảo năng lực cạnh tranh quốc gia. Trong cả hai trường hợp, AI là một phần của một nỗ lực rộng lớn, bao gồm thiết kế chip và nghiên cứu giao diện con người. Ở Anh, báo cáo Alvey đã khôi phục lại kinh phí bị loại bỏ bởi báo cáo Lighthill. Tuy nhiên, không có dự án nào trong số này đạt được các mục tiêu đầy tham vọng của mình về khả năng AI mới hoặc tác động kinh tế.

Nhìn chung, ngành công nghiệp AI đã bùng nổ từ vài triệu đô la vào năm 1980 lên hàng tỷ đô la vào năm 1988, bao gồm hàng trăm công ty xây dựng **hệ thống chuyên gia, hệ thống thị giác, robot**, và phần mềm và phần cứng chuyên dụng cho các mục đích này.

Ngay sau đó là một giai đoạn được gọi là “**mùa đông AI**,” trong đó nhiều công ty đã thất bại khi không thực hiện được những lời hứa phóng đại. Hóa ra việc xây dựng và duy trì các hệ thống chuyên gia cho các lĩnh vực phức tạp là khó khăn, một phần vì các phương pháp suy luận được sử

dụng bởi các hệ thống bị phá vỡ khi đối mặt với sự không chắc chắn và một phần vì các hệ thống không thể học hỏi từ kinh nghiệm.

1.3.5 Sự trở lại của mạng thần kinh (1986–nay)

Vào giữa những năm 1980, ít nhất bốn nhóm khác nhau đã phát minh lại thuật toán học **lan truyền ngược** (back-propagation) được phát triển lần đầu vào đầu những năm 1960. Thuật toán này đã được áp dụng cho nhiều vấn đề học tập trong khoa học máy tính và tâm lý học, và việc phổ biến rộng rãi các kết quả trong tuyển tập *Xử lý song song phân tán* (Rumelhart và McClelland, 1986) đã gây ra sự phấn khích lớn.

Những mô hình được gọi là **mô hình kết nối** (connectionist) này được một số người coi là đối thủ cạnh tranh trực tiếp với cả mô hình ký hiệu được Newell và Simon quảng bá và cách tiếp cận logic của McCarthy và những người khác. Có vẻ như rõ ràng rằng ở một mức độ nào đó con người thao tác các ký hiệu – trên thực tế, cuốn sách *Loài mạng tính biểu tượng* (1997) của nhà nhân chủng học **Terrence Deacon** cho rằng đây là đặc điểm xác định của con người. Ngược lại, **Geoff Hinton**, một nhân vật hàng đầu trong sự hồi sinh của mạng thần kinh vào những năm 1980 và 2010, đã mô tả các ký hiệu là “**ete phát quang của AI**” – một ám chỉ đến môi trường không tồn tại mà nhiều nhà vật lý thế kỷ 19 tin rằng sóng điện từ lan truyền qua. Chắc chắn, nhiều khái niệm mà chúng ta đặt tên trong ngôn ngữ, khi xem xét kỹ hơn, không có loại điều kiện cần và đủ được định nghĩa logic mà các nhà nghiên cứu AI ban đầu hy vọng nắm bắt được dưới dạng tiên đề. Có thể các mô hình kết nối hình thành các khái niệm bên trong theo một cách linh hoạt và không chính xác hơn, phù hợp hơn với sự lộn xộn của thế giới thực. Chúng cũng có khả năng học từ các ví dụ – chúng có thể so sánh giá trị đầu ra dự đoán của chúng với giá trị thực trên một vấn đề và sửa đổi các tham số của chúng để giảm sự khác biệt, khiến chúng có nhiều khả năng hoạt động tốt hơn trên các ví dụ trong tương lai.

1.3.6 Suy luận xác suất và học máy (1987–nay)

Sự mong manh của các **hệ thống chuyên gia** đã dẫn đến một cách tiếp cận khoa học mới, kết hợp **xác suất** thay vì logic Boolean, **học máy** thay vì mã hóa thủ công, và **kết quả thực nghiệm** thay vì các tuyên bố triết học. Trở nên phổ biến hơn khi xây dựng trên các lý thuyết hiện có hơn là đề xuất những lý thuyết hoàn toàn mới, đưa ra các tuyên bố dựa trên các định lý chặt chẽ hoặc phương pháp luận thực nghiệm vững chắc (Cohen, 1995) thay vì dựa trên trực giác, và thể hiện sự liên quan đến các ứng dụng trong thế giới thực thay vì các ví dụ đồ chơi.

Các bộ vấn đề **chuẩn** (benchmark) dùng chung đã trở thành tiêu chuẩn để thể hiện sự tiến bộ, bao gồm kho lưu trữ của UC Irvine cho các bộ dữ liệu học máy, Cuộc thi lập kế hoạch quốc tế cho các thuật toán lập kế hoạch, kho dữ liệu LibriSpeech để nhận dạng giọng nói, bộ dữ liệu MNIST để nhận dạng chữ số viết tay, ImageNet và COCO để nhận dạng đối tượng hình ảnh, SQUAD để trả lời câu hỏi ngôn ngữ tự nhiên, cuộc thi WMT để dịch máy, và các Cuộc thi SAT quốc tế cho các bộ giải thỏa mãn Boolean.

AI được thành lập một phần như một cuộc nổi loạn chống lại những hạn chế của các lĩnh vực hiện có như lý thuyết điều khiển và thống kê, nhưng trong giai đoạn này, nó đã chấp nhận các kết quả tích cực của những lĩnh vực đó. Như **David McAllester** (1998) đã nói:

Trong giai đoạn đầu của AI, có vẻ hợp lý rằng các dạng tính toán ký hiệu mới, ví dụ như khung và mạng ngữ nghĩa, đã làm cho phần lớn lý thuyết cổ điển trở nên lỗi thời. Điều này dẫn đến một dạng **chủ nghĩa biệt lập** trong đó AI trở nên phần lớn tách rời khỏi phần còn lại của khoa học máy tính. Chủ nghĩa biệt lập này hiện đang bị từ bỏ. Có một sự công nhận rằng học máy không nên bị tách biệt khỏi lý thuyết thông tin, rằng suy luận không chắc chắn không nên bị tách biệt khỏi mô hình ngẫu nhiên, rằng tìm kiếm không nên bị tách biệt khỏi tối ưu hóa và điều khiển cổ điển, và rằng suy luận tự động không nên bị tách biệt khỏi các phương pháp hình thức và phân tích tĩnh.

Lĩnh vực **nhận dạng giọng nói** minh họa cho mô hình này. Trong những năm 1970, rất nhiều kiến trúc và cách tiếp cận khác nhau đã được thử. Nhiều trong số này khá **đặc thù** (ad hoc) và mong manh, và chỉ hoạt động trên một vài ví dụ được chọn lọc cẩn thận. Trong những năm 1980, các cách tiếp cận sử dụng **mô hình Markov ẩn** (HMMs) đã chiếm ưu thế trong lĩnh vực này. Hai khía cạnh của HMMs là có liên quan. Thứ nhất, chúng dựa trên một lý thuyết toán học chặt chẽ. Điều này cho phép các nhà nghiên cứu giọng nói xây dựng trên vài thập kỷ kết quả toán học được phát triển trong các lĩnh vực khác. Thứ hai, chúng được tạo ra bởi một quá trình đào tạo trên một kho dữ liệu lớn về giọng nói thực. Điều này đảm bảo rằng hiệu suất là mạnh mẽ, và trong các bài kiểm tra mù chặt chẽ, HMMs đã cải thiện điểm số của chúng một cách đều đặn. Kết quả là, công nghệ giọng nói và lĩnh vực liên quan là **nhận dạng ký tự viết tay** đã chuyển sang các ứng dụng công nghiệp và tiêu dùng rộng rãi. Lưu ý rằng không có tuyên bố khoa học nào cho rằng con người sử dụng HMMs để nhận dạng giọng nói; thay vào đó, HMMs đã cung cấp một khuôn khổ toán học để hiểu và giải quyết vấn đề. Tuy nhiên, chúng ta sẽ thấy trong Mục 1.3.8, rằng **học sâu** đã làm đảo lộn câu chuyện thoải mái này.

Năm 1988 là một năm quan trọng cho mối liên hệ giữa AI và các lĩnh vực khác, bao gồm thống kê, nghiên cứu hoạt động, lý thuyết quyết định và lý thuyết điều khiển. Cuốn sách *Suy luận xác suất trong các hệ thống thông minh* (1988) của **Judea Pearl** đã dẫn đến một sự chấp nhận mới về xác suất và lý thuyết quyết định trong AI. Sự phát triển của **mạng Bayes** của Pearl đã mang lại một hình thức nghiêm ngặt và hiệu quả để biểu diễn tri thức không chắc chắn cũng như các thuật toán thực tế để suy luận xác suất. Chương 12, 13, 14, 15 và 18 bao gồm lĩnh vực này, ngoài những phát triển gần đây đã làm tăng đáng kể sức mạnh biểu đạt của các hình thức xác suất; Chương 21 mô tả các phương pháp học mạng Bayes và các mô hình liên quan từ dữ liệu. Một đóng góp lớn thứ hai vào năm 1988 là công trình của **Rich Sutton** kết nối **học tăng cường** – vốn đã được sử dụng trong chương trình chơi cờ đam của Arthur Samuel vào những năm 50 – với lý thuyết về **quá trình quyết định Markov** (MDPs) được phát triển trong lĩnh vực nghiên cứu hoạt động. Một làn sóng công trình đã theo sau kết nối nghiên cứu lập kế hoạch AI với MDPs, và lĩnh vực học tăng cường đã tìm thấy các ứng dụng trong robot và kiểm soát quá trình cũng như có được các nền tảng lý thuyết sâu sắc.

Một hệ quả của việc AI đánh giá cao dữ liệu, mô hình thống kê, tối ưu hóa và học máy là sự **tái thống nhất dần dần của các lĩnh vực phụ** như thị giác máy tính, robot, nhận dạng giọng nói, hệ thống đa tác nhân và xử lý ngôn ngữ tự nhiên, vốn đã trở nên hơi tách biệt khỏi AI cốt lõi. Quá trình tái hội nhập đã mang lại những lợi ích đáng kể cả về ứng dụng – ví dụ, việc triển khai các

robot thực tế đã mở rộng đáng kể trong giai đoạn này – và trong một sự hiểu biết lý thuyết tốt hơn về các vấn đề cốt lõi của AI.

1.3.7 Dữ liệu lớn (2001–nay)

Những tiến bộ đáng kể về sức mạnh tính toán và sự ra đời của **World Wide Web** đã tạo điều kiện cho việc tạo ra các tập dữ liệu rất lớn – một hiện tượng đôi khi được gọi là **dữ liệu lớn** (big data). Các tập dữ liệu này bao gồm hàng nghìn tỷ từ văn bản, hàng tỷ hình ảnh và hàng tỷ giờ giọng nói và video, cũng như một lượng lớn dữ liệu gen, dữ liệu theo dõi phương tiện, dữ liệu luồng nhấp chuột, dữ liệu mạng xã hội, v.v.

Điều này đã dẫn đến sự phát triển của các **thuật toán học** được thiết kế đặc biệt để tận dụng các tập dữ liệu rất lớn. Thông thường, phần lớn các ví dụ trong các tập dữ liệu như vậy là **không được gán nhãn**; ví dụ, trong công trình có ảnh hưởng của **Yarowsky** (1995) về **phân giải nghĩa từ**, các lần xuất hiện của một từ như “plant” không được gán nhãn trong tập dữ liệu để chỉ ra liệu chúng đề cập đến thực vật hay nhà máy. Tuy nhiên, với các tập dữ liệu đủ lớn, các thuật toán học phù hợp có thể đạt được độ chính xác trên 96% trong nhiệm vụ xác định nghĩa nào được dự định trong một câu. Hơn nữa, **Banko và Brill** (2001) đã lập luận rằng sự cải thiện về hiệu suất thu được từ việc tăng kích thước của tập dữ liệu lên hai hoặc ba bậc độ lớn lớn hơn bất kỳ sự cải thiện nào có thể thu được từ việc tinh chỉnh thuật toán.

Một hiện tượng tương tự dường như xảy ra trong các nhiệm vụ **thị giác máy tính** như lấp đầy các lỗ hổng trong ảnh – các lỗ hổng gây ra do bị hỏng hoặc do xóa bỏ những người bạn cũ. **Hays và Efros** (2007) đã phát triển một phương pháp thông minh để làm điều này bằng cách trộn các pixel từ các hình ảnh tương tự; họ thấy rằng kỹ thuật này hoạt động kém với một cơ sở dữ liệu chỉ có hàng nghìn hình ảnh nhưng đã vượt qua một ngưỡng chất lượng với hàng triệu hình ảnh. Ngay sau đó, sự sẵn có của hàng chục triệu hình ảnh trong cơ sở dữ liệu **ImageNet** (Deng et al., 2009) đã châm ngòi cho một cuộc cách mạng trong lĩnh vực thị giác máy tính.

Sự sẵn có của dữ liệu lớn và sự thay đổi hướng tới học máy đã giúp AI lấy lại sức hấp dẫn thương mại (Havenstein, 2005; Halevy et al., 2009). **Dữ liệu lớn** là một yếu tố quan trọng trong chiến thắng năm 2011 của hệ thống **Watson** của IBM trước các nhà vô địch con người trong trò chơi đố vui *Jeopardy!*, một sự kiện đã có tác động lớn đến nhận thức của công chúng về AI.

1.3.8 Học sâu (2011–nay)

Thuật ngữ **học sâu** (deep learning) đề cập đến việc học máy sử dụng nhiều lớp các phân tử tính toán đơn giản, có thể điều chỉnh. Các thí nghiệm với các mạng như vậy đã được thực hiện từ những năm 1970, và dưới dạng **mạng nơ-ron tích chập** (convolutional neural networks), chúng đã đạt được một số thành công trong việc nhận dạng chữ số viết tay vào những năm 1990 (LeCun et al., 1995). Tuy nhiên, phải đến năm 2011, các phương pháp học sâu mới thực sự cất cánh. Điều này xảy ra đầu tiên trong lĩnh vực nhận dạng giọng nói và sau đó là nhận dạng vật thể bằng hình ảnh.

Trong cuộc thi **ImageNet** năm 2012, yêu cầu phân loại hình ảnh thành một trong một nghìn danh mục (tatu, giá sách, cái mở nút chai, v.v.), một hệ thống học sâu được tạo ra trong nhóm của

Geoffrey Hinton tại Đại học Toronto (Krizhevsky et al., 2013) đã thể hiện một sự cải thiện đáng kể so với các hệ thống trước đó, vốn chủ yếu dựa trên các **đặc trưng được tạo thủ công** (handcrafted features). Kể từ đó, các hệ thống học sâu đã vượt quá hiệu suất của con người trên một số nhiệm vụ thị giác (và tụt lại phía sau trong một số nhiệm vụ khác). Các thành quả tương tự đã được báo cáo trong nhận dạng giọng nói, dịch máy, chẩn đoán y tế và chơi trò chơi. Việc sử dụng một mạng sâu để biểu diễn hàm đánh giá đã góp phần vào chiến thắng của **ALPHA-GO** trước các người chơi cờ vây hàng đầu (Silver et al., 2016, 2017, 2018).

Những thành công đáng chú ý này đã dẫn đến sự hồi sinh của sự quan tâm đến AI trong sinh viên, các công ty, nhà đầu tư, chính phủ, giới truyền thông và công chúng nói chung. Có vẻ như mỗi tuần lại có tin tức về một ứng dụng AI mới đạt được hoặc vượt quá hiệu suất của con người, thường đi kèm với những suy đoán về một thành công nhanh chóng hoặc một **mùa đông AI** mới.

Học sâu phụ thuộc rất nhiều vào phần cứng mạnh mẽ. Trong khi một CPU máy tính tiêu chuẩn có thể thực hiện 10^9 hoặc 10^{10} phép toán mỗi giây, một thuật toán học sâu chạy trên phần cứng chuyên dụng (ví dụ: GPU, TPU hoặc FPGA) có thể tiêu thụ từ 10^{14} đến 10^{17} phép toán mỗi giây, chủ yếu dưới dạng các phép toán ma trận và vector được song song hóa cao. Tất nhiên, học sâu cũng phụ thuộc vào sự sẵn có của một lượng lớn dữ liệu đào tạo và một vài thủ thuật thuật toán (xem Chương 22).

1.4 Tình hình hiện tại

Nghiên cứu Trăm năm về AI (còn được gọi là AI100) của Đại học Stanford triệu tập các hội đồng chuyên gia để cung cấp các báo cáo về tình hình hiện tại của AI. Báo cáo năm 2016 của họ (Stone et al., 2016; Grosz và Stone, 2018) kết luận rằng “có thể dự kiến sự gia tăng đáng kể trong việc sử dụng các ứng dụng AI trong tương lai, bao gồm nhiều xe tự lái hơn, chẩn đoán và điều trị y tế có mục tiêu, và hỗ trợ vật lý cho người già” và rằng “Xã hội hiện đang ở một thời điểm quan trọng trong việc xác định cách triển khai các công nghệ dựa trên AI theo những cách thúc đẩy thay vì cản trở các giá trị dân chủ như tự do, bình đẳng và minh bạch.” AI100 cũng sản xuất một **Chỉ số AI** tại aiindex.org để giúp theo dõi tiến độ. Một số điểm nổi bật từ các báo cáo năm 2018 và 2019 (so sánh với năm 2000 trừ khi có quy định khác):

- **Ấn phẩm:** Các bài báo về AI đã tăng 20 lần từ năm 2010 đến 2019 lên khoảng 20.000 bài mỗi năm. Danh mục phổ biến nhất là **học máy**. (Các bài báo về học máy trong arXiv.org tăng gấp đôi mỗi năm từ 2009 đến 2017.) **Thị giác máy tính** và **xử lý ngôn ngữ tự nhiên** là phổ biến tiếp theo.
- **Tâm lý:** Khoảng 70% các bài báo tin tức về AI là trung lập, nhưng các bài báo với giọng điệu tích cực đã tăng từ 12% vào năm 2016 lên 30% vào năm 2018. Các vấn đề phổ biến nhất là về đạo đức: quyền riêng tư dữ liệu và **thiên vị thuật toán**.
- **Sinh viên:** Số lượng sinh viên đăng ký khóa học đã tăng gấp 5 lần ở Hoa Kỳ và 16 lần trên toàn cầu so với năm 2010. AI là chuyên ngành phổ biến nhất trong Khoa học Máy tính.
- **Đa dạng:** Khoảng 80% Giáo sư AI trên toàn thế giới là nam, 20% là nữ. Các con số tương tự cũng đúng với các sinh viên tiến sĩ và nhân viên được tuyển dụng trong ngành.
- **Hội nghị:** Số người tham dự NeurIPS đã tăng 800% kể từ năm 2012 lên 13.500 người. Các hội nghị khác đang chứng kiến mức tăng trưởng hàng năm khoảng 30%.

- **Công nghiệp:** Các công ty khởi nghiệp AI ở Hoa Kỳ đã tăng 20 lần lên hơn 800.
- **Quốc tế hóa:** Trung Quốc xuất bản nhiều bài báo mỗi năm hơn Hoa Kỳ và gần bằng tất cả châu Âu. Tuy nhiên, về tác động được tính theo trích dẫn, các tác giả Hoa Kỳ vượt trội hơn 50% so với các tác giả Trung Quốc. Singapore, Brazil, Australia, Canada và Ấn Độ là những quốc gia có tốc độ phát triển nhanh nhất về số lượng nhân viên AI được tuyển dụng.
- **Thị giác:** Tỷ lệ lỗi trong nhận dạng đối tượng (như đã đạt được trong **LSVRC**, Thử thách nhận dạng hình ảnh quy mô lớn) đã cải thiện từ 28% vào năm 2010 xuống 2% vào năm 2017, vượt quá hiệu suất của con người. Độ chính xác trong **trả lời câu hỏi thị giác (VQA)** đã cải thiện từ 55% lên 68% kể từ năm 2015, nhưng vẫn tụt lại phía sau hiệu suất của con người ở mức 83%.
- **Tốc độ:** Thời gian đào tạo cho nhiệm vụ nhận dạng hình ảnh đã giảm 100 lần chỉ trong hai năm qua. Lượng sức mạnh tính toán được sử dụng trong các ứng dụng AI hàng đầu đang tăng gấp đôi cứ sau 3,4 tháng.
- **Ngôn ngữ:** Độ chính xác trong trả lời câu hỏi, được đo bằng điểm F1 trên **Tập dữ liệu trả lời câu hỏi Stanford (SQUAD)**, đã tăng từ 60 lên 95 từ 2015 đến 2019; trên biến thể SQUAD 2, tiến độ nhanh hơn, tăng từ 62 lên 90 chỉ trong một năm. Cả hai điểm số đều vượt quá hiệu suất của con người.
- **Các chuẩn mực của con người:** Đến năm 2019, các hệ thống AI được báo cáo đã đạt hoặc vượt quá hiệu suất của con người trong cờ vua, cờ vây, poker, Pac-Man, Jeopardy!, nhận dạng đối tượng ImageNet, nhận dạng giọng nói trong một miền giới hạn, dịch tiếng Trung sang tiếng Anh trong một miền hạn chế, Quake III, Dota 2, StarCraft II, các trò chơi Atari khác nhau, phát hiện ung thư da, phát hiện ung thư tuyến tiền liệt, gấp protein và chẩn đoán bệnh võng mạc tiểu đường.

Khi nào (nếu có) các hệ thống AI sẽ đạt được hiệu suất ở cấp độ con người trên một loạt các nhiệm vụ? **Ford (2018)** phỏng vấn các chuyên gia AI và tìm thấy một phạm vi rộng các năm mục tiêu, từ 2029 đến 2200, với trung bình là 2099. Trong một cuộc khảo sát tương tự (**Grace et al., 2017**) 50% số người được hỏi nghĩ rằng điều này có thể xảy ra vào năm 2066, mặc dù 10% nghĩ rằng nó có thể xảy ra sớm nhất là năm 2025, và một vài người nói “không bao giờ.” Các chuyên gia cũng chia rẽ về việc liệu chúng ta có cần những đột phá cơ bản mới hay chỉ cần tinh chỉnh các cách tiếp cận hiện tại. Nhưng đừng quá coi trọng các dự đoán của họ; như **Philip Tetlock (2017)** đã chứng minh trong lĩnh vực dự đoán các sự kiện thế giới, các chuyên gia không giỏi hơn những người nghiệp dư.

Các hệ thống AI trong tương lai sẽ hoạt động như thế nào? Chúng ta chưa thể nói được. Như đã trình bày chi tiết trong phần này, lĩnh vực này đã áp dụng một số câu chuyện về chính nó – đầu tiên là ý tưởng táo bạo rằng trí thông minh của một cỗ máy là có thể, sau đó là nó có thể đạt được bằng cách mã hóa tri thức chuyên gia vào logic, sau đó là các mô hình xác suất của thế giới sẽ là công cụ chính, và gần đây nhất là học máy sẽ tạo ra các mô hình có thể không dựa trên bất kỳ lý thuyết nào được hiểu rõ. Tương lai sẽ tiết lộ mô hình nào sẽ đến tiếp theo.

AI có thể làm gì ngày nay? Có lẽ không nhiều như một số bài báo truyền thông lạc quan hơn có thể khiến người ta tin, nhưng vẫn là rất nhiều. Dưới đây là một số ví dụ:

- **Xe robot:** Lịch sử của xe robot kéo dài từ những chiếc xe điều khiển bằng radio của những năm 1920, nhưng những màn trình diễn đầu tiên về lái xe tự động trên đường mà không có hướng dẫn đặc biệt đã xảy ra vào những năm 1980 (**Kanade et al., 1986; Dickmanns và**

Zapp, 1987). Sau những màn trình diễn thành công về lái xe trên đường đất trong **Thử thách lớn DARPA** dài 132 dặm vào năm 2005 (Thrun, 2006) và trên đường phố có giao thông trong Thử thách đô thị năm 2007, cuộc đua phát triển xe tự lái đã bắt đầu một cách nghiêm túc. Năm 2018, các phương tiện thử nghiệm của Waymo đã vượt qua cột mốc 10 triệu dặm lái trên đường công cộng mà không có một tai nạn nghiêm trọng nào, với người lái xe con người chỉ can thiệp để tiếp quản quyền điều khiển mỗi 6.000 dặm. Ngay sau đó, công ty đã bắt đầu cung cấp dịch vụ taxi robot thương mại.

- **Trên không**, các máy bay không người lái cánh cố định tự lái đã cung cấp dịch vụ vận chuyển máu xuyên quốc gia ở Rwanda từ năm 2016. Các máy bay bốn cánh quạt (quadcopter) thực hiện các thao tác nhào lộn đáng chú ý, khám phá các tòa nhà trong khi xây dựng bản đồ 3-D, và tự lắp ráp thành các đội hình tự động.
- **Di chuyển bằng chân**: BigDog, một robot bốn chân của Raibert et al. (2008), đã đảo lộn quan niệm của chúng ta về cách robot di chuyển – không còn là dáng đi chậm chạp, cứng chân, từ bên này sang bên kia của các robot trong phim Hollywood, mà là thứ gì đó gần giống với một con vật và có khả năng phục hồi khi bị xô đẩy hoặc khi trượt trên một vũng nước đóng băng. Atlas, một robot hình người, không chỉ đi bộ trên địa hình không bằng phẳng mà còn nhảy lên các hộp và thực hiện các cú lộn ngược (Ackerman và Guizzo, 2016).
- **Lập kế hoạch và lên lịch tự động**: Cách Trái đất hàng trăm triệu dặm, chương trình **Remote Agent** của NASA đã trở thành chương trình lập kế hoạch tự động trên tàu đầu tiên kiểm soát việc lên lịch các hoạt động cho một tàu vũ trụ (Jonsson et al., 2000). Remote Agent đã tạo ra các kế hoạch từ các mục tiêu cấp cao được chỉ định từ mặt đất và giám sát việc thực hiện các kế hoạch đó – phát hiện, chẩn đoán và phục hồi từ các vấn đề khi chúng xảy ra. Ngày nay, bộ công cụ lập kế hoạch **EUROPA** (Barreiro et al., 2012) được sử dụng cho các hoạt động hàng ngày của các tàu thăm dò Sao Hỏa của NASA và hệ thống **SEXTANT** (Winternitz, 2017) cho phép điều hướng tự động trong không gian sâu, vượt ra ngoài hệ thống GPS toàn cầu.
- Trong cuộc khủng hoảng Vùng Vịnh Ba Tư năm 1991, lực lượng Hoa Kỳ đã triển khai một **Công cụ Phân tích và Lập kế hoạch Động, DART** (Cross và Walker, 1994), để thực hiện lập kế hoạch và lên lịch hậu cần tự động cho vận chuyển. Điều này liên quan đến tới 50.000 phương tiện, hàng hóa và con người cùng một lúc, và phải tính đến điểm xuất phát, điểm đến, tuyến đường, năng lực vận tải, năng lực cảng và sân bay, và giải quyết xung đột giữa tất cả các tham số. **Cơ quan Dự án Nghiên cứu Tiên tiến Quốc phòng (DARPA)** đã tuyên bố rằng ứng dụng duy nhất này đã trả lại hơn khoản đầu tư 30 năm của DARPA vào AI.
- Mỗi ngày, các công ty gọi xe như Uber và các dịch vụ bản đồ như Google Maps cung cấp chỉ đường lái xe cho hàng trăm triệu người dùng, nhanh chóng vạch ra một lộ trình tối ưu có tính đến các điều kiện giao thông hiện tại và được dự đoán trong tương lai.
- **Dịch máy**: Các hệ thống dịch máy trực tuyến hiện cho phép đọc các tài liệu bằng hơn 100 ngôn ngữ, bao gồm các ngôn ngữ mẹ đẻ của hơn 99% con người, và hiển thị hàng trăm tỷ từ mỗi ngày cho hàng trăm triệu người dùng. Mặc dù không hoàn hảo, nhưng chúng thường đủ để hiểu. Đối với các ngôn ngữ có liên quan chặt chẽ với nhau với một lượng lớn dữ liệu đào tạo (chẳng hạn như tiếng Pháp và tiếng Anh), các bản dịch trong một miền hẹp gần bằng cấp độ của một con người (Wu et al., 2016b).

- **Nhận dạng giọng nói:** Năm 2017, Microsoft đã chỉ ra rằng **Hệ thống Nhận dạng Giọng nói Đàm thoại** của họ đã đạt tỷ lệ lỗi từ 5,1%, khớp với hiệu suất của con người trên nhiệm vụ Switchboard, liên quan đến việc sao chép các cuộc trò chuyện qua điện thoại (Xiong et al., 2017). Khoảng một phần ba tương tác máy tính trên toàn thế giới hiện được thực hiện bằng giọng nói thay vì bàn phím; Skype cung cấp dịch thuật từ giọng nói sang giọng nói theo thời gian thực bằng mười ngôn ngữ. Alexa, Siri, Cortana và Google cung cấp các trợ lý có thể trả lời câu hỏi và thực hiện các nhiệm vụ cho người dùng; ví dụ, dịch vụ **Google Duplex** sử dụng nhận dạng giọng nói và tổng hợp giọng nói để đặt chỗ nhà hàng cho người dùng, thực hiện một cuộc trò chuyện trôi chảy thay mặt họ.
- **Đề xuất:** Các công ty như Amazon, Facebook, Netflix, Spotify, YouTube, Walmart và những công ty khác sử dụng học máy để đề xuất những gì bạn có thể thích dựa trên kinh nghiệm trong quá khứ của bạn và của những người khác giống bạn. Lĩnh vực **hệ thống đề xuất** có một lịch sử lâu dài (Resnick và Varian, 1997) nhưng đang thay đổi nhanh chóng nhờ các phương pháp học sâu mới phân tích nội dung (văn bản, âm nhạc, video) cũng như lịch sử và siêu dữ liệu (van den Oord et al., 2014; Zhang et al., 2017). Lọc thư rác cũng có thể được coi là một dạng đề xuất (hoặc không đề xuất); các kỹ thuật AI hiện tại lọc ra hơn 99,9% thư rác, và các dịch vụ email cũng có thể đề xuất những người nhận tiềm năng, cũng như văn bản phản hồi có thể có.
- **Chơi trò chơi:** Khi Deep Blue đánh bại nhà vô địch cờ vua thế giới Garry Kasparov vào năm 1997, những người bảo vệ quyền tối cao của con người đã đặt hy vọng vào cờ vây. **Piet Hut**, một nhà vật lý thiên văn và người đam mê cờ vây, đã dự đoán rằng sẽ mất “một trăm năm trước khi một máy tính đánh bại con người ở cờ vây – thậm chí có thể lâu hơn.” Nhưng chỉ 20 năm sau, **ALPHA-GO** đã vượt qua tất cả những người chơi con người (Silver et al., 2017). **Ke Jie**, nhà vô địch thế giới, nói, “Năm ngoái, nó vẫn còn khá giống con người khi chơi. Nhưng năm nay, nó đã trở nên giống như một vị thần cờ vây.” ALPHA-GO đã được hưởng lợi từ việc nghiên cứu hàng trăm nghìn ván cờ trước đây của những người chơi cờ vây, và từ tri thức chất lọc của các chuyên gia cờ vây đã làm việc trong nhóm.
- Một chương trình tiếp theo, **ALPHA-ZERO**, đã không sử dụng bất kỳ đầu vào nào từ con người (ngoại trừ các quy tắc của trò chơi), và có thể học thông qua việc tự chơi một mình để đánh bại tất cả các đối thủ, con người và máy móc, ở cờ vây, cờ vua và shogi (Silver et al., 2018). Trong khi đó, các nhà vô địch con người đã bị đánh bại bởi các hệ thống AI trong các trò chơi đa dạng như Jeopardy! (Ferrucci et al., 2010), poker (Bowling et al., 2015; Moravčík et al., 2017; Brown và Sandholm, 2019), và các trò chơi điện tử Dota 2 (Fernandez và Mahlmann, 2018), StarCraft II (Vinyals et al., 2019) và Quake III (Jaderberg et al., 2019).
- **Hiểu hình ảnh:** Không hài lòng với việc vượt quá độ chính xác của con người trên nhiệm vụ nhận dạng đối tượng ImageNet đầy thách thức, các nhà nghiên cứu thị giác máy tính đã đảm nhận vấn đề khó hơn là **chú thích hình ảnh**. Một số ví dụ ẩn tượng bao gồm “Một người đang đi xe máy trên một con đường đất,” “Hai chiếc pizza nằm trên một chiếc lò nướng bếp ga,” và “Một nhóm người trẻ đang chơi ném đĩa” (Vinyals et al., 2017b). Tuy nhiên, các hệ thống hiện tại còn lâu mới hoàn hảo: một “tủ lạnh đầy thức ăn và đồ uống” hóa ra lại là một biển báo cấm đỗ xe bị che khuất một phần bởi nhiều hình dán nhỏ.
- **Y học:** Các thuật toán AI hiện nay bằng hoặc vượt quá các bác sĩ chuyên gia trong việc chẩn đoán nhiều tình trạng, đặc biệt là khi chẩn đoán dựa trên hình ảnh. Các ví dụ bao gồm

bệnh Alzheimer (Ding et al., 2018), ung thư di căn (Liu et al., 2017; Esteva et al., 2017), bệnh mắt (Gulshan et al., 2016) và các bệnh ngoài da (Liu et al., 2019c). Một đánh giá có hệ thống và phân tích tổng hợp (Liu et al., 2019a) đã phát hiện ra rằng hiệu suất của các chương trình AI, trung bình, tương đương với các chuyên gia chăm sóc sức khỏe. Một trọng tâm hiện tại trong AI y tế là tạo điều kiện cho các **quan hệ đối tác giữa người và máy**. Ví dụ, hệ thống **LYNA** đạt độ chính xác tổng thể 99,6% trong việc chẩn đoán ung thư vú di căn – tốt hơn một chuyên gia con người không được hỗ trợ – nhưng sự kết hợp giữa hai bên thậm chí còn tốt hơn (Liu et al., 2018; Steiner et al., 2018).

- Việc áp dụng rộng rãi các kỹ thuật này hiện bị hạn chế không phải bởi độ chính xác chẩn đoán mà bởi nhu cầu chứng minh sự cải thiện trong kết quả lâm sàng và đảm bảo tính minh bạch, thiếu thiên vị và quyền riêng tư dữ liệu (Topol, 2019). Năm 2017, chỉ có hai ứng dụng AI y tế được FDA chấp thuận, nhưng con số đó đã tăng lên 12 vào năm 2018, và tiếp tục tăng.
- **Khoa học khí hậu:** Một nhóm các nhà khoa học đã giành được **Giải thưởng Gordon Bell** năm 2018 cho một mô hình học sâu khám phá thông tin chi tiết về các sự kiện thời tiết cực đoan mà trước đây bị chôn vùi trong dữ liệu khí hậu. Họ đã sử dụng một siêu máy tính với phần cứng GPU chuyên dụng để vượt quá mức exaop (10^{18} phép toán mỗi giây), là chương trình học máy đầu tiên làm được điều đó (Kurth et al., 2018). **Rolnick et al.** (2019) trình bày một danh mục dài 60 trang về các cách mà học máy có thể được sử dụng để giải quyết biến đổi khí hậu.

Đây chỉ là một vài ví dụ về các hệ thống trí tuệ nhân tạo tồn tại ngày nay. Không phải là ma thuật hay khoa học viễn tưởng – mà là khoa học, kỹ thuật và toán học, mà cuốn sách này cung cấp một lời giới thiệu.

1.5 Risks and Benefits of AI

Francis Bacon, một triết gia được ghi nhận là người đã tạo ra phương pháp khoa học, đã lưu ý trong *Sự khôn ngoan của người cổ đại* (1609) rằng “các nghệ thuật cơ khí có công dụng mơ hồ, phục vụ cho cả việc gây hại lẫn chữa bệnh.” Khi AI đóng một vai trò ngày càng quan trọng trong các lĩnh vực kinh tế, xã hội, khoa học, y tế, tài chính và quân sự, chúng ta nên xem xét những tổn hại và cách chữa trị—theo cách nói hiện đại, **những rủi ro và lợi ích**—mà nó có thể mang lại. Các chủ đề được tóm tắt ở đây được trình bày chuyên sâu hơn trong Chương 28 và 29.

Để bắt đầu với các **lợi ích**: nói một cách đơn giản, toàn bộ nền văn minh của chúng ta là sản phẩm của trí thông minh con người. Nếu chúng ta có quyền tiếp cận với trí thông minh máy móc lớn hơn đáng kể, giới hạn đối với tham vọng của chúng ta sẽ được nâng lên đáng kể. Tiềm năng của AI và robot để giải phóng nhân loại khỏi công việc lặp đi lặp lại tầm thường và để tăng đáng kể việc sản xuất hàng hóa và dịch vụ có thể báo trước một kỷ nguyên hòa bình và thịnh vượng. Khả năng đẩy nhanh nghiên cứu khoa học có thể dẫn đến các phương pháp chữa bệnh và giải pháp cho biến đổi khí hậu và thiếu hụt tài nguyên. Như **Demis Hassabis**, CEO của Google DeepMind, đã gợi ý: “Đầu tiên hãy giải quyết AI, sau đó sử dụng AI để giải quyết mọi thứ khác.”

Tuy nhiên, rất lâu trước khi chúng ta có cơ hội “giải quyết AI,” chúng ta sẽ phải đối mặt với những rủi ro từ việc **lạm dụng AI**, dù vô tình hay cố ý. Một số trong số này đã rõ ràng, trong khi những rủi ro khác dường như có khả năng xảy ra dựa trên các xu hướng hiện tại:

- **Vũ khí tự hành gây chết người:** Chúng được Liên Hợp Quốc định nghĩa là vũ khí có thể định vị, chọn và loại bỏ các mục tiêu của con người mà không cần sự can thiệp của con người. Mỗi quan tâm chính với các loại vũ khí như vậy là **khả năng mở rộng** của chúng: việc không yêu cầu sự giám sát của con người có nghĩa là một nhóm nhỏ có thể triển khai một số lượng vũ khí lớn tùy ý chống lại các mục tiêu của con người được xác định bởi bất kỳ tiêu chí nhận dạng khả thi nào. Các công nghệ cần thiết cho vũ khí tự hành tương tự như các công nghệ cần thiết cho xe tự lái. Các cuộc thảo luận chuyên gia không chính thức về các rủi ro tiềm tàng của vũ khí tự hành gây chết người đã bắt đầu tại Liên Hợp Quốc vào năm 2014, chuyển sang giai đoạn tiền hiệp ước chính thức của một Nhóm Chuyên gia Chính phủ vào năm 2017.
- **Giám sát và thuyết phục:** Mặc dù tốn kém, tẻ nhạt và đôi khi có vấn đề pháp lý đối với nhân viên an ninh khi giám sát đường dây điện thoại, nguồn cấp dữ liệu camera video, email và các kênh nhắn tin khác, nhưng AI (nhận dạng giọng nói, thị giác máy tính và hiểu ngôn ngữ tự nhiên) có thể được sử dụng một cách có thể mở rộng để thực hiện **giám sát hàng loạt các cá nhân** và phát hiện các hoạt động đáng quan tâm. Bằng cách điều chỉnh luồng thông tin cho các cá nhân thông qua mạng xã hội, dựa trên các kỹ thuật học máy, hành vi chính trị có thể được sửa đổi và kiểm soát ở một mức độ nào đó—một mối quan tâm trở nên rõ ràng trong các cuộc bầu cử bắt đầu vào năm 2016.
- **Ra quyết định có thành kiến:** Việc sử dụng bất cân hoặc cố ý các thuật toán học máy cho các nhiệm vụ như đánh giá đơn xin ân xá và vay vốn có thể dẫn đến các quyết định **có thành kiến về chủng tộc, giới tính hoặc các danh mục được bảo vệ khác**. Thường thì, bản thân dữ liệu phản ánh sự thành kiến phổ biến trong xã hội.
- **Tác động đến việc làm:** Những lo ngại về việc máy móc loại bỏ công việc đã có từ nhiều thế kỷ. Câu chuyện không bao giờ đơn giản: máy móc thực hiện một số nhiệm vụ mà con người có thể làm, nhưng chúng cũng làm cho con người trở nên năng suất hơn và do đó dễ được tuyển dụng hơn, và làm cho các công ty có lợi nhuận cao hơn và do đó có thể trả lương cao hơn. Chúng có thể làm cho một số hoạt động trở nên khả thi về mặt kinh tế mà nếu không thì không thực tế. Việc sử dụng chúng thường dẫn đến việc tăng sự giàu có nhưng có xu hướng có tác dụng **chuyển của cải từ lao động sang tư bản**, làm trầm trọng thêm sự gia tăng bất bình đẳng. Những tiến bộ công nghệ trước đây—chẳng hạn như phát minh ra máy dệt cơ khí—đã dẫn đến sự gián đoạn nghiêm trọng đối với việc làm, nhưng cuối cùng mọi người tìm thấy những loại công việc mới để làm. Mặt khác, có thể AI cũng sẽ làm những loại công việc mới đó. Chủ đề này đang nhanh chóng trở thành một trọng tâm chính cho các nhà kinh tế và chính phủ trên khắp thế giới.
- **Các ứng dụng quan trọng về an toàn:** Khi các kỹ thuật AI tiến bộ, chúng ngày càng được sử dụng trong các ứng dụng có rủi ro cao, **quan trọng về an toàn** như lái xe và quản lý nguồn cung cấp nước của các thành phố. Các tai nạn chết người đã xảy ra và làm nổi bật sự khó khăn của việc xác minh hình thức và phân tích rủi ro thống kê cho các hệ thống được phát triển bằng cách sử dụng các kỹ thuật học máy. Lĩnh vực AI sẽ cần phát triển các tiêu chuẩn kỹ thuật và đạo đức ít nhất là tương đương với các tiêu chuẩn phổ biến trong các ngành kỹ thuật và chăm sóc sức khỏe khác, nơi tính mạng của con người bị đe dọa.
- **An ninh mạng:** Các kỹ thuật AI hữu ích trong việc phòng thủ chống lại các cuộc tấn công mạng, ví dụ như bằng cách phát hiện các mẫu hành vi bất thường, nhưng chúng cũng sẽ góp phần vào sức mạnh, khả năng sống sót và khả năng phát tán của phần mềm độc hại.

Ví dụ, các phương pháp học tăng cường đã được sử dụng để tạo ra các công cụ hiệu quả cao cho các cuộc tấn công **tổng tiền và lừa đảo tự động, được cá nhân hóa**.

Chúng ta sẽ xem xét lại các chủ đề này chuyên sâu hơn trong Mục 28.3. Khi các hệ thống AI trở nên có năng lực hơn, chúng sẽ đảm nhận nhiều vai trò xã hội trước đây do con người đảm nhận. Giống như con người đã sử dụng các vai trò này trong quá khứ để thực hiện các hành vi sai trái, chúng ta có thể mong đợi rằng con người có thể lạm dụng các hệ thống AI trong các vai trò này để thực hiện nhiều hành vi sai trái hơn nữa. Tất cả các ví dụ trên đều chỉ ra tầm quan trọng của quản trị và, cuối cùng, là **quy định**. Hiện tại, cộng đồng nghiên cứu và các tập đoàn lớn tham gia vào nghiên cứu AI đã phát triển các **nguyên tắc tự quản trị tự nguyện** cho các hoạt động liên quan đến AI (xem Mục 28.3). Các chính phủ và các tổ chức quốc tế đang thành lập các cơ quan tư vấn để đưa ra các quy định thích hợp cho từng trường hợp sử dụng cụ thể, để chuẩn bị cho các tác động kinh tế và xã hội, và để tận dụng khả năng của AI để giải quyết các vấn đề xã hội lớn.

Còn về lâu dài thì sao? Liệu chúng ta có đạt được mục tiêu lâu dài: tạo ra **trí thông minh tương đương hoặc có năng lực hơn trí thông minh của con người**? Và, nếu chúng ta làm được, thì sao?

Trong phần lớn lịch sử của AI, những câu hỏi này đã bị lu mờ bởi công việc hàng ngày là làm cho các hệ thống AI làm bất cứ điều gì dù chỉ là thông minh một cách mơ hồ. Giống như bất kỳ lĩnh vực rộng lớn nào, đại đa số các nhà nghiên cứu AI đã chuyên về một lĩnh vực phụ cụ thể như chơi trò chơi, biểu diễn tri thức, thị giác hoặc hiểu ngôn ngữ tự nhiên—thường với giả định rằng tiến bộ trong các lĩnh vực phụ này sẽ đóng góp vào các mục tiêu rộng lớn hơn của AI. **Nils Nilsson** (1995), một trong những nhà lãnh đạo ban đầu của dự án Shakey tại SRI, đã nhắc nhở lĩnh vực này về những mục tiêu rộng lớn hơn đó và cảnh báo rằng các lĩnh vực phụ đang có nguy cơ trở thành mục đích tự thân. Sau đó, một số người sáng lập AI có ảnh hưởng, bao gồm **John McCarthy** (2007), **Marvin Minsky** (2007) và **Patrick Winston** (Beal và Winston, 2009), đã đồng tình với những cảnh báo của Nilsson, gợi ý rằng thay vì tập trung vào hiệu suất có thể đo lường được trong các ứng dụng cụ thể, AI nên quay trở lại cội nguồn của nó là phân đầu, theo lời của **Herb Simon**, cho “những cỗ máy biết suy nghĩ, học hỏi và sáng tạo.” Họ gọi nỗ lực này là **AI cấp độ con người** (human-level AI) hoặc **HLAI**—một cỗ máy nên có khả năng học cách làm bất cứ điều gì mà một con người có thể làm. Hội nghị chuyên đề đầu tiên của họ là vào năm 2004 (Minsky et al., 2004). Một nỗ lực khác với các mục tiêu tương tự, phong trào **trí tuệ nhân tạo tổng quát (AGI)** (Goertzel và Pennachin, 2007), đã tổ chức hội nghị đầu tiên và tổ chức Tạp chí Trí tuệ nhân tạo tổng quát vào năm 2008.

Cùng khoảng thời gian đó, những lo ngại đã được đặt ra rằng việc tạo ra **siêu trí tuệ nhân tạo** (artificial superintelligence) hoặc **ASI**—trí thông minh vượt xa khả năng của con người—có thể là một ý tưởng tồi (Yudkowsky, 2008; Omohundro, 2008). Chính **Turing** (1996) đã đưa ra cùng một quan điểm trong một bài giảng được đưa ra ở Manchester vào năm 1951, dựa trên những ý tưởng trước đó từ **Samuel Butler** (1863):

Có vẻ như một khi phương pháp suy nghĩ của máy móc đã bắt đầu, sẽ không mất nhiều thời gian để nó vượt qua sức mạnh yếu ớt của chúng ta. Do đó, ở một giai đoạn nào đó, chúng ta nên mong đợi các cỗ máy sẽ nắm quyền kiểm soát, theo cách được đề cập trong *Erewhon* của Samuel Butler.

Những lo ngại này chỉ trở nên phổ biến hơn với những tiến bộ gần đây trong **học sâu**, việc xuất bản các cuốn sách như *Siêu trí tuệ* của **Nick Bostrom** (2014), và các tuyên bố công khai từ **Stephen Hawking**, **Bill Gates**, **Martin Rees**, và **Elon Musk**.

Trải qua một cảm giác bất an chung với ý tưởng tạo ra các cỗ máy siêu thông minh là điều tự nhiên. Chúng ta có thể gọi đây là **vấn đề tinh tinh**: khoảng bảy triệu năm trước, một loài linh trưởng đã tuyệt chủng đã tiến hóa, với một nhánh dẫn đến tinh tinh và một nhánh dẫn đến con người. Ngày nay, những con tinh tinh không quá vui vẻ về nhánh con người; chúng về cơ bản không có quyền kiểm soát tương lai của mình. Nếu đây là kết quả của việc thành công trong việc tạo ra AI siêu phàm—rằng con người từ bỏ quyền kiểm soát tương lai của họ—thì có lẽ chúng ta nên ngừng làm việc về AI, và, như một hệ quả, từ bỏ những lợi ích mà nó có thể mang lại. Đây là bản chất của cảnh báo của Turing: không rõ ràng là chúng ta có thể kiểm soát các cỗ máy thông minh hơn chúng ta.

Nếu AI siêu phàm là một hộp đen đến từ ngoài không gian, thì quả thực sẽ là khôn ngoan khi thận trọng trong việc mở hộp. Nhưng nó không phải: chúng ta thiết kế các hệ thống AI, vì vậy nếu chúng cuối cùng “nắm quyền kiểm soát,” như Turing gợi ý, thì đó sẽ là kết quả của một **lỗi thiết kế**. Để tránh một kết quả như vậy, chúng ta cần hiểu nguồn gốc của thất bại tiềm tàng. **Norbert Wiener** (1960), người đã có động lực để xem xét tương lai lâu dài của AI sau khi thấy chương trình chơi cờ đam của Arthur Samuel học cách đánh bại người tạo ra nó, đã nói điều này:

Nếu chúng ta sử dụng, để đạt được mục đích của mình, một tác nhân cơ học mà chúng ta không thể can thiệp một cách hiệu quả... chúng ta nên chắc chắn rằng mục đích được đưa vào máy là mục đích mà chúng ta thực sự mong muốn.

Nhiều nền văn hóa có những câu chuyện thần thoại về những con người yêu cầu các vị thần, thần đèn, pháp sư hoặc ma quỷ một điều gì đó. Luôn luôn, trong những câu chuyện này, họ nhận được những gì họ theo nghĩa đen đã yêu cầu, và sau đó hối hận về nó. Điều ước thứ ba, nếu có, là để **hoàn tác hai điều ước đầu tiên**. Chúng ta sẽ gọi đây là **vấn đề Vua Midas**: Midas, một vị vua huyền thoại trong thần thoại Hy Lạp, đã yêu cầu mọi thứ ông chạm vào đều biến thành vàng, nhưng sau đó hối hận sau khi chạm vào thức ăn, đồ uống và các thành viên trong gia đình mình.

Chúng ta đã chạm vào vấn đề này trong Mục 1.1.5, nơi chúng ta đã chỉ ra sự cần thiết phải sửa đổi đáng kể mô hình tiêu chuẩn của việc đưa các mục tiêu cố định vào máy. Giải pháp cho tình trạng khó xử của Wiener không phải là có một “mục đích được đưa vào máy” một cách dứt khoát. Thay vào đó, chúng ta muốn các cỗ máy cố gắng đạt được **các mục tiêu của con người** nhưng biết rằng chúng không biết chắc chắn chính xác các mục tiêu đó là gì.

Thật không may mắn khi gần như tất cả các nghiên cứu AI cho đến nay đã được thực hiện trong **mô hình tiêu chuẩn**, điều đó có nghĩa là gần như tất cả các tài liệu kỹ thuật trong ấn bản này phản ánh khuôn khổ trí tuệ đó. Tuy nhiên, có một số kết quả ban đầu trong khuôn khổ mới. Trong Chương 15, chúng ta chỉ ra rằng một cỗ máy có động cơ tích cực để cho phép bản thân bị tắt nếu và chỉ nếu nó không chắc chắn về mục tiêu của con người. Trong Chương 17, chúng ta xây dựng và nghiên cứu **trò chơi hỗ trợ** (assistance game), mô tả bằng toán học tình huống mà một con người có một mục tiêu và một cỗ máy cố gắng đạt được nó, nhưng ban đầu không chắc chắn về mục tiêu đó là gì. Trong Chương 23, chúng ta giải thích các phương pháp **học tăng cường nghịch đảo** (inverse reinforcement learning) cho phép máy móc tìm hiểu thêm về sở thích của con người từ những quan sát về các lựa chọn mà con người đưa ra. Trong Chương 28, chúng ta khám phá hai khó khăn chính: thứ nhất, các lựa chọn của chúng ta phụ thuộc vào sở thích của chúng ta thông

qua một kiến trúc nhận thức rất phức tạp khó đảo ngược; và, thứ hai, chúng ta con người có thể không có sở thích nhất quán ngay từ đầu—dù là cá nhân hay là một nhóm—vì vậy có thể không rõ ràng là các hệ thống AI nên làm gì cho chúng ta.

Tóm tắt

Chương này định nghĩa AI và thiết lập bối cảnh văn hóa mà nó đã phát triển. Một số điểm quan trọng như sau:

- Mỗi người tiếp cận AI với những mục tiêu khác nhau. Hai câu hỏi quan trọng cần hỏi là: Bạn quan tâm đến tư duy, hay hành vi? Bạn muốn mô hình hóa con người, hay cố gắng đạt được kết quả tối ưu?
- Theo những gì chúng tôi gọi là **mô hình tiêu chuẩn**, AI chủ yếu quan tâm đến **hành động hợp lý**. Một tác nhân thông minh lý tưởng thực hiện hành động tốt nhất có thể trong một tình huống. Chúng tôi nghiên cứu vấn đề xây dựng các tác nhân thông minh theo nghĩa này.
- Cần có hai sự tinh chỉnh cho ý tưởng đơn giản này: thứ nhất, khả năng của bất kỳ tác nhân nào, con người hay máy móc, để chọn các hành động hợp lý bị giới hạn bởi **tính bất khả thi về mặt tính toán** để làm như vậy; thứ hai, khái niệm về một cỗ máy theo đuổi một mục tiêu xác định cần được thay thế bằng khái niệm về một cỗ máy theo đuổi các mục tiêu để mang lại lợi ích cho con người, nhưng **không chắc chắn** về các mục tiêu đó là gì.
- Các **nhà triết học** (trở lại năm 400 TCN) đã làm cho AI trở nên khả thi bằng cách gợi ý rằng tâm trí ở một khía cạnh nào đó giống như một cỗ máy, rằng nó hoạt động dựa trên tri thức được mã hóa trong một ngôn ngữ nội bộ nào đó, và rằng suy nghĩ có thể được sử dụng để chọn những hành động cần thực hiện.
- Các **nhà toán học** đã cung cấp các công cụ để thao tác các tuyên bố về sự chắc chắn logic cũng như các tuyên bố không chắc chắn, mang tính xác suất. Họ cũng đặt nền móng cho việc hiểu tính toán và suy luận về các thuật toán.
- Các **nhà kinh tế** đã hình thức hóa vấn đề đưa ra các quyết định nhằm tối đa hóa lợi ích kỳ vọng cho người ra quyết định.
- Các **nhà khoa học thần kinh** đã khám phá một số sự thật về cách bộ não hoạt động và những cách nó tương tự và khác với máy tính.
- Các **nhà tâm lý học** đã chấp nhận ý tưởng rằng con người và động vật có thể được coi là máy xử lý thông tin. Các **nhà ngôn ngữ học** đã chỉ ra rằng việc sử dụng ngôn ngữ phù hợp với mô hình này.
- Các **kỹ sư máy tính** đã cung cấp những cỗ máy ngày càng mạnh mẽ hơn, làm cho các ứng dụng AI trở nên khả thi, và các **kỹ sư phần mềm** đã làm cho chúng dễ sử dụng hơn.
- **Lý thuyết điều khiển** giải quyết việc thiết kế các thiết bị hoạt động tối ưu dựa trên phản hồi từ môi trường. Ban đầu, các công cụ toán học của lý thuyết điều khiển khá khác với các công cụ được sử dụng trong AI, nhưng các lĩnh vực này đang xích lại gần nhau hơn.
- Lịch sử của AI đã có những chu kỳ thành công, sự lạc quan sai lầm và kết quả là sự cắt giảm nhiệt huyết và tài trợ. Cũng đã có những chu kỳ giới thiệu các cách tiếp cận mới, sáng tạo và tinh chỉnh một cách có hệ thống những cách tiếp cận tốt nhất.

- AI đã trưởng thành đáng kể so với những thập kỷ đầu, cả về mặt lý thuyết và phương pháp luận. Khi các vấn đề mà AI giải quyết trở nên phức tạp hơn, lĩnh vực này đã chuyển từ logic Boolean sang **suy luận xác suất**, và từ **tri thức được tạo thủ công sang học máy từ dữ liệu**. Điều này đã dẫn đến những cải thiện về khả năng của các hệ thống thực và sự tích hợp lớn hơn với các ngành khác.
- Khi các hệ thống AI tìm thấy ứng dụng trong thế giới thực, việc xem xét một loạt các **rủi ro và hậu quả đạo đức** đã trở nên cần thiết.
- Trong dài hạn, chúng ta phải đối mặt với vấn đề khó khăn là kiểm soát các hệ thống AI siêu thông minh có thể tiến hóa theo những cách không thể đoán trước. Giải quyết vấn đề này dường như đòi hỏi phải thay đổi quan niệm của chúng ta về AI.

Ghi chú thư mục và lịch sử

Một lịch sử toàn diện về AI được cung cấp bởi **Nils Nilsson** (2009), một trong những người tiên phong đầu tiên của lĩnh vực này. **Pedro Domingos** (2015) và **Melanie Mitchell** (2019) cung cấp tổng quan về học máy cho khán giả nói chung, và **Kai-Fu Lee** (2018) mô tả cuộc đua giành vị trí dẫn đầu quốc tế trong AI. **Martin Ford** (2018) phỏng vấn 23 nhà nghiên cứu AI hàng đầu.

Các hiệp hội chuyên nghiệp chính về AI là Hiệp hội Nâng cao Trí tuệ Nhân tạo (**AAAI**), Nhóm Quan tâm Đặc biệt về Trí tuệ Nhân tạo của ACM (**SIGAI**, trước đây là SIGART), Hiệp hội AI châu Âu, và Hiệp hội Trí tuệ Nhân tạo và Mô phỏng Hành vi (**AISB**). **Partnership on AI** tập hợp nhiều tổ chức thương mại và phi lợi nhuận quan tâm đến các tác động đạo đức và xã hội của AI. Tạp chí AI của AAAI chứa nhiều bài báo chuyên đề và hướng dẫn, và trang web của nó, **aaai.org**, chứa tin tức, hướng dẫn và thông tin cơ bản.

Các công trình gần đây nhất xuất hiện trong kỷ yếu của các hội nghị AI lớn: Hội nghị chung quốc tế về AI (**IJCAI**), Hội nghị AI châu Âu hàng năm (**ECAI**), và Hội nghị AAAI. Học máy được bao gồm trong Hội nghị quốc tế về Học máy và cuộc họp **Hệ thống xử lý thông tin thần kinh (NeurIPS)**. Các tạp chí chính về AI nói chung là *Artificial Intelligence*, *Computational Intelligence*, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Intelligent Systems*, và *Journal of Artificial Intelligence Research*. Cũng có nhiều hội nghị và tạp chí dành riêng cho các lĩnh vực cụ thể, mà chúng tôi sẽ đề cập trong các chương thích hợp.

CHƯƠNG 2: CÁC TÁC NHÂN THÔNG MINH

Trong đó chúng ta thảo luận về bản chất của các tác nhân, hoàn hảo hay không, sự đa dạng của môi trường, và tập hợp các loại tác nhân tương ứng.

Chương này sẽ làm rõ hơn về khái niệm **tác nhân hợp lý**, một khái niệm trung tâm của **trí tuệ nhân tạo (AI)** mà chúng ta đã xác định trong Chương 1. Chúng ta sẽ thấy rằng khái niệm về sự hợp lý có thể được áp dụng cho nhiều loại tác nhân hoạt động trong bất kỳ môi trường nào. Mục tiêu của chúng ta trong cuốn sách này là sử dụng khái niệm này để phát triển một bộ nhỏ các nguyên tắc thiết kế nhằm xây dựng các tác nhân thành công—những hệ thống có thể được gọi là thông minh một cách hợp lý.

Chúng ta sẽ bắt đầu bằng cách xem xét các **tác nhân**, **môi trường** và sự liên kết giữa chúng. Việc quan sát thấy một số tác nhân hành xử tốt hơn những tác nhân khác dẫn đến ý tưởng tự nhiên về một **tác nhân hợp lý**—một tác nhân hành xử tốt nhất có thể. Khả năng hành xử của một tác nhân phụ thuộc vào bản chất của môi trường; một số môi trường khó hơn những môi trường khác. Chúng ta sẽ đưa ra một phân loại sơ bộ về các môi trường và chỉ ra cách các thuộc tính của một môi trường ảnh hưởng đến thiết kế của các tác nhân phù hợp cho môi trường đó. Chúng ta sẽ mô tả một số thiết kế tác nhân “cốt lõi” cơ bản, mà chúng ta sẽ làm rõ trong phần còn lại của cuốn sách.

2.1 Tác nhân và Môi trường

Tác nhân là bất cứ thứ gì có thể được coi là nhận thức môi trường của nó thông qua các **cảm biến** và tác động lên môi trường đó thông qua các **bộ truyền động**. Ý tưởng đơn giản này được minh họa trong Hình 2.1. Một **tác nhân con người** có mắt, tai và các cơ quan khác làm cảm biến và có tay, chân, thanh quản, v.v. làm bộ truyền động. Một **tác nhân robot** có thể có camera và bộ tìm khoảng cách hồng ngoại làm cảm biến và nhiều động cơ khác nhau làm bộ truyền động. Một **tác nhân phần mềm** nhận nội dung tệp, gói mạng và đầu vào của con người (bàn phím/chuột/màn hình cảm ứng/giọng nói) dưới dạng đầu vào cảm biến và tác động lên môi trường bằng cách ghi tệp, gửi gói mạng và hiển thị thông tin hoặc tạo âm thanh. Môi trường có thể là mọi thứ—toàn bộ vũ trụ! Trên thực tế, nó chỉ là một phần của vũ trụ mà trạng thái của nó chúng ta quan tâm khi thiết kế tác nhân này—phần ảnh hưởng đến những gì tác nhân nhận thức và bị ảnh hưởng bởi các hành động của tác nhân.

Chúng ta sử dụng thuật ngữ **tri giác** (percept) để chỉ nội dung mà các cảm biến của tác nhân đang nhận thức. **Chuỗi tri giác** (percept sequence) của một tác nhân là lịch sử hoàn chỉnh của mọi thứ mà tác nhân đã từng nhận thức. Nói chung, sự lựa chọn hành động của một tác nhân tại bất kỳ thời điểm nào có thể phụ thuộc vào **kiến thức được xây dựng sẵn** và toàn bộ chuỗi tri giác đã quan sát được cho đến nay, nhưng không phải vào bất cứ thứ gì mà nó chưa nhận thức. Bằng cách chỉ định sự lựa chọn hành động của tác nhân cho mọi chuỗi tri giác có thể có, chúng ta đã nói nhiều hay ít mọi thứ cần nói về tác nhân. Về mặt toán học, chúng ta nói rằng hành vi của một tác nhân được mô tả bởi **hàm tác nhân** (agent function) ánh xạ bất kỳ chuỗi tri giác nào đã cho thành một hành động.

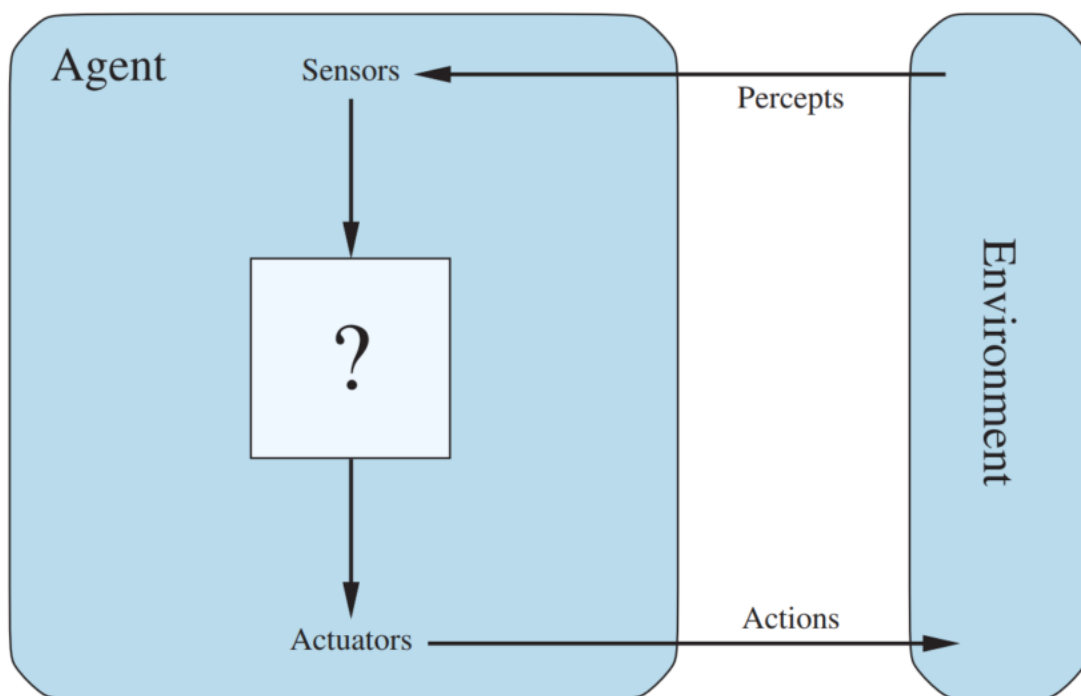
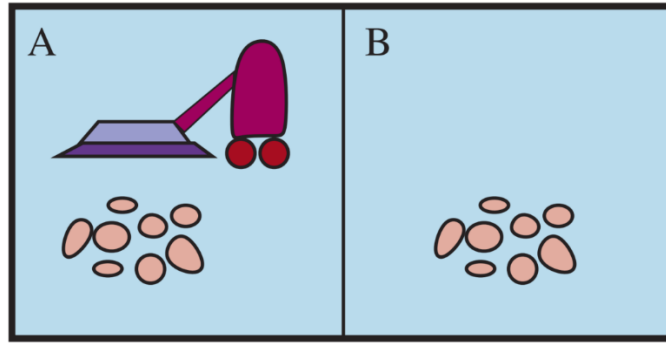


Figure 2.1 Agents interact with environments through sensors and actuators.

Chúng ta có thể hình dung việc lập bảng hàm tác nhân mô tả bất kỳ tác nhân nào; đối với hầu hết các tác nhân, đây sẽ là một bảng rất lớn—thực tế là vô hạn, trừ khi chúng ta đặt ra một giới hạn về độ dài của các chuỗi tri giác mà chúng ta muốn xem xét. Về nguyên tắc, với một tác nhân để thử nghiệm, chúng ta có thể xây dựng bảng này bằng cách thử tất cả các chuỗi tri giác có thể có và ghi lại các hành động mà tác nhân thực hiện để phản hồi. Bảng, tất nhiên, là một đặc điểm bên ngoài của tác nhân. Bên trong, hàm tác nhân cho một tác nhân nhân tạo sẽ được thực hiện bởi một **chương trình tác nhân**. Điều quan trọng là phải giữ cho hai ý tưởng này khác biệt. **Hàm tác nhân** là một mô tả toán học trừu tượng; **chương trình tác nhân** là một triển khai cụ thể, chạy trong một hệ thống vật lý nào đó.

Để minh họa những ý tưởng này, chúng ta sử dụng một ví dụ đơn giản—**thế giới máy hút bụi**, bao gồm một tác nhân máy hút bụi robot trong một thế giới bao gồm các ô vuông có thể bẩn hoặc sạch. Hình 2.2 cho thấy một cấu hình chỉ với hai ô vuông, A và B. Tác nhân hút bụi nhận thức được nó đang ở ô vuông nào và liệu có bụi bẩn trong ô vuông đó hay không. Tác nhân bắt đầu ở ô vuông A. Các hành động có sẵn là di chuyển sang phải, di chuyển sang trái, hút bụi bẩn hoặc không làm gì cả. Một hàm tác nhân rất đơn giản là như sau: nếu ô vuông hiện tại bẩn, thì hút; nếu không, di chuyển đến ô vuông khác. Một bảng một phần của hàm tác nhân này được hiển thị trong Hình 2.3 và một chương trình tác nhân thực hiện nó xuất hiện trong Hình 2.8 trên trang 67.



Hình 2.2: Một thế giới máy hút bụi chỉ có hai vị trí. Mỗi vị trí có thể sạch hoặc bẩn, và tác nhân có thể di chuyển sang trái hoặc phải và có thể làm sạch ô vuông mà nó đang chiếm. Các phiên bản khác nhau của thế giới máy hút bụi cho phép các quy tắc khác nhau về những gì tác nhân có thể nhận thức, liệu các hành động của nó có luôn thành công hay không, v.v.

Hình 2.3: Bảng một phần của một hàm tác nhân đơn giản cho thế giới máy hút bụi được hiển thị trong Hình 2.2. Tác nhân làm sạch ô vuông hiện tại nếu nó bẩn, nếu không thì nó di chuyển đến ô vuông khác. Lưu ý rằng bảng có kích thước không giới hạn trừ khi có một hạn chế về độ dài của các chuỗi tri giác có thể có.

Chuỗi tri giác	Hành động
[A, Sạch]	Phải
[A, Bẩn]	Hút
[B, Sạch]	Trái
[B, Bẩn]	Hút
[A, Sạch], [A, Sạch]	Phải
[A, Sạch], [A, Bẩn]	Hút
...	...
[A, Sạch], [A, Sạch], [A, Sạch]	Phải
[A, Sạch], [A, Sạch], [A, Bẩn]	Hút
...	...

Nhìn vào Hình 2.3, chúng ta thấy rằng các tác nhân thế giới hút bụi khác nhau có thể được định nghĩa đơn giản bằng cách điền vào cột bên phải theo nhiều cách khác nhau. Câu hỏi rõ ràng, vậy thì, là: **Cách đúng để điền vào bảng là gì?** Nói cách khác, điều gì làm cho một tác nhân tốt hay xấu, thông minh hay ngu ngốc? Chúng ta sẽ trả lời những câu hỏi này trong phần tiếp theo.

Trước khi kết thúc phần này, chúng ta nên nhấn mạnh rằng khái niệm **tác nhân** có nghĩa là một công cụ để phân tích các hệ thống, không phải là một đặc điểm tuyệt đối chia thế giới thành tác nhân và không phải tác nhân. Người ta có thể coi một máy tính cầm tay là một tác nhân chọn hành động hiển thị “4” khi được cho chuỗi tri giác “2 + 2 =,” nhưng một phân tích như vậy khó có thể

giúp chúng ta hiểu về máy tính. Theo một nghĩa nào đó, tất cả các lĩnh vực kỹ thuật có thể được coi là thiết kế các tạo tác tương tác với thế giới; AI hoạt động ở (những gì các tác giả coi là) phần thú vị nhất của phổ, nơi các tạo tác có tài nguyên tính toán đáng kể và môi trường nhiệm vụ đòi hỏi phải ra quyết định không tầm thường.

2.2 Hành vi tốt: Khái niệm về tính hợp lý

Một **tác nhân hợp lý** là một tác nhân làm điều đúng đắn. Rõ ràng, làm điều đúng đắn thì tốt hơn là làm điều sai, nhưng làm điều đúng đắn có nghĩa là gì?

2.2.1 Thước đo hiệu suất

Triết học đạo đức đã phát triển một số khái niệm khác nhau về “điều đúng đắn”, nhưng AI nói chung đã tuân theo một khái niệm được gọi là **chủ nghĩa hậu quả** (consequentialism): chúng ta đánh giá hành vi của một tác nhân bằng các hậu quả của nó. Khi một tác nhân được đặt vào một môi trường, nó tạo ra một chuỗi các hành động theo các tri giác mà nó nhận được. Chuỗi hành động này khiến môi trường trải qua một chuỗi các trạng thái. Nếu chuỗi đó là mong muốn, thì tác nhân đã thực hiện tốt. Khái niệm về sự mong muốn này được nắm bắt bởi một **thước đo hiệu suất** (performance measure) đánh giá bất kỳ chuỗi trạng thái môi trường nào đã cho.

Con người có những mong muốn và sở thích của riêng họ, vì vậy khái niệm về sự hợp lý được áp dụng cho con người có liên quan đến sự thành công của họ trong việc chọn các hành động tạo ra các chuỗi trạng thái môi trường mong muốn từ quan điểm của họ. Mặt khác, máy móc không có mong muốn và sở thích của riêng chúng; **thước đo hiệu suất**, ít nhất là ban đầu, nằm trong tâm trí của người thiết kế máy, hoặc trong tâm trí của những người dùng mà máy được thiết kế cho. Chúng ta sẽ thấy rằng một số thiết kế tác nhân có một biểu diễn rõ ràng về (một phiên bản của) thước đo hiệu suất, trong khi ở các thiết kế khác, thước đo hiệu suất hoàn toàn ngầm định—tác nhân có thể làm điều đúng, nhưng nó không biết tại sao.

Nhớ lại cảnh báo của **Norbert Wiener** để đảm bảo rằng “mục đích được đưa vào máy là mục đích mà chúng ta thực sự mong muốn” (trang 51), hãy lưu ý rằng việc xây dựng một thước đo hiệu suất một cách chính xác có thể khá khó khăn. Ví dụ, hãy xem xét tác nhân máy hút bụi từ phần trước. Chúng ta có thể đề xuất đo lường hiệu suất bằng lượng bụi bẩn được dọn sạch trong một ca làm việc tám giờ duy nhất. Với một tác nhân hợp lý, tất nhiên, những gì bạn yêu cầu là những gì bạn nhận được. Một tác nhân hợp lý có thể tối đa hóa thước đo hiệu suất này bằng cách dọn sạch bụi bẩn, sau đó đổ tất cả lên sàn, sau đó dọn sạch lại, v.v. Một thước đo hiệu suất phù hợp hơn sẽ thưởng cho tác nhân vì có một sàn nhà sạch sẽ. Ví dụ, một điểm có thể được trao cho mỗi ô vuông sạch ở mỗi bước thời gian (có lẽ với một hình phạt cho điện năng tiêu thụ và tiếng ồn tạo ra). Theo quy tắc chung, tốt hơn là thiết kế các thước đo hiệu suất theo những gì người ta thực sự muốn đạt được trong môi trường, hơn là theo cách người ta nghĩ rằng tác nhân nên hành xử.

Ngay cả khi tránh được những cạm bẫy rõ ràng, một số vấn đề rắc rối vẫn còn. Ví dụ, khái niệm “sàn sạch” trong đoạn trên dựa trên độ sạch trung bình theo thời gian. Tuy nhiên, cùng một độ sạch trung bình có thể đạt được bởi hai tác nhân khác nhau, một tác nhân làm một công việc tầm thường mọi lúc trong khi tác nhân kia dọn dẹp một cách mạnh mẽ nhưng lại nghỉ ngơi dài. Cái nào được

ura chuộng hơn có vẻ là một điểm tinh tế của khoa học dọn dẹp, nhưng trên thực tế, đó là một câu hỏi triết học sâu sắc với những hàm ý sâu rộng. Cái gì tốt hơn—một cuộc sống liêu lĩnh với những thăng trầm, hay một cuộc sống an toàn nhưng tẻ nhạt? Cái gì tốt hơn—một nền kinh tế nơi mọi người sống trong cảnh nghèo khổ vừa phải, hay một nền kinh tế trong đó một số người sống sung túc trong khi những người khác rất nghèo? Chúng tôi để lại những câu hỏi này như một bài tập cho độc giả siêng năng.

Trong phần lớn cuốn sách, chúng ta sẽ giả định rằng thước đo hiệu suất có thể được chỉ định một cách chính xác. Tuy nhiên, vì những lý do đã nêu ở trên, chúng ta phải chấp nhận khả năng rằng chúng ta có thể đưa sai mục đích vào máy—chính xác là **vấn đề Vua Midas** được mô tả trên trang 51. Hơn nữa, khi thiết kế một phần mềm, các bản sao của nó sẽ thuộc về những người dùng khác nhau, chúng ta không thể đoán trước được sở thích chính xác của từng người dùng. Do đó, chúng ta có thể cần xây dựng các tác nhân phản ánh sự không chắc chắn ban đầu về thước đo hiệu suất thực và tìm hiểu thêm về nó theo thời gian; các tác nhân như vậy được mô tả trong Chương 15, 17 và 23.

2.2.2 Tính hợp lý

Điều gì là hợp lý tại bất kỳ thời điểm nào phụ thuộc vào bốn điều:

- **Thước đo hiệu suất** xác định tiêu chí thành công.
- **Kiến thức ban đầu** của tác nhân về môi trường.
- Các **hành động** mà tác nhân có thể thực hiện.
- **Chuỗi tri giác** của tác nhân cho đến nay.

Điều này dẫn đến một định nghĩa về một **tác nhân hợp lý**:

Đối với mỗi chuỗi tri giác có thể có, một tác nhân hợp lý nên chọn một hành động được kỳ vọng sẽ tối đa hóa thước đo hiệu suất của nó, dựa trên bằng chứng được cung cấp bởi chuỗi tri giác và bất kỳ kiến thức được xây dựng sẵn nào mà tác nhân có.

Hãy xem xét tác nhân máy hút bụi đơn giản làm sạch một ô vuông nếu nó bẩn và di chuyển đến ô vuông khác nếu không; đây là hàm tác nhân được lập bảng trong Hình 2.3. Đây có phải là một tác nhân hợp lý không? Điều đó còn tùy! Đầu tiên, chúng ta cần nói thước đo hiệu suất là gì, những gì được biết về môi trường, và tác nhân có những cảm biến và bộ truyền động nào. Hãy giả sử những điều sau:

- Thước đo hiệu suất thưởng một điểm cho mỗi ô vuông sạch ở mỗi bước thời gian, trong suốt “vòng đời” 1000 bước thời gian.
- “Địa lý” của môi trường được biết trước (Hình 2.2) nhưng sự phân bố bụi bẩn và vị trí ban đầu của tác nhân thì không. Các ô vuông sạch vẫn sạch và việc hút sẽ làm sạch ô vuông hiện tại. Các hành động Phải và Trái di chuyển tác nhân một ô vuông trừ khi điều này sẽ đưa tác nhân ra ngoài môi trường, trong trường hợp đó tác nhân vẫn ở vị trí cũ.
- Các hành động duy nhất có sẵn là Phải, Trái và Hút.
- Tác nhân nhận thức chính xác vị trí của nó và liệu vị trí đó có chứa bụi bẩn hay không.

Trong những trường hợp này, tác nhân thực sự là hợp lý; hiệu suất kỳ vọng của nó ít nhất cũng tốt bằng bất kỳ tác nhân nào khác.

Người ta có thể dễ dàng thấy rằng cùng một tác nhân sẽ là phi lý trong những trường hợp khác nhau. Ví dụ, một khi tất cả bụi bẩn được dọn sạch, tác nhân sẽ dao động không cần thiết qua lại; nếu thước đo hiệu suất bao gồm một hình phạt một điểm cho mỗi lần di chuyển, tác nhân sẽ hoạt động kém. Một tác nhân tốt hơn cho trường hợp này sẽ không làm gì cả một khi nó chắc chắn rằng tất cả các ô vuông đều sạch. Nếu các ô vuông sạch có thể bị bẩn lại, tác nhân nên thỉnh thoảng kiểm tra và làm sạch lại chúng nếu cần. Nếu địa lý của môi trường không được biết, tác nhân sẽ cần khám phá nó. Bài tập 2.VACR yêu cầu bạn thiết kế các tác nhân cho những trường hợp này.

2.2.3 Toàn tri, học tập và tự chủ

Chúng ta cần cẩn thận để phân biệt giữa **tính hợp lý** và **tính toàn tri**. Một **tác nhân toàn tri** biết kết quả thực tế của các hành động của nó và có thể hành động phù hợp; nhưng tính toàn tri là không thể trong thực tế. Hãy xem xét ví dụ sau: Một ngày nọ, tôi đang đi bộ dọc theo đại lộ Champs-Élysées và tôi thấy một người bạn cũ ở bên kia đường. Không có giao thông gần đó và tôi không bận việc gì khác, vì vậy, là người hợp lý, tôi bắt đầu băng qua đường. Trong khi đó, ở độ cao 33.000 feet, một cánh cửa hàng hóa rơi ra khỏi một máy bay phản lực đang bay qua và trước khi tôi sang được bên kia đường, tôi bị đè bẹp. Tôi có phải là phi lý khi băng qua đường không? Không đời nào, không thể có chuyện cáo phó của tôi ghi là “Kẻ ngốc cố băng qua đường” được.

Ví dụ này cho thấy rằng **tính hợp lý không giống với sự hoàn hảo**. Tính hợp lý tối đa hóa hiệu suất kỳ vọng, trong khi sự hoàn hảo tối đa hóa hiệu suất thực tế. Rút lui khỏi yêu cầu về sự hoàn hảo không chỉ là một câu hỏi về sự công bằng đối với các tác nhân. Vấn đề là nếu chúng ta mong đợi một tác nhân làm những gì hóa ra sau đó là hành động tốt nhất, sẽ không thể thiết kế một tác nhân để thực hiện đặc điểm kỹ thuật này—trừ khi chúng ta cải thiện hiệu suất của quả cầu pha lê hoặc cỗ máy thời gian.

Định nghĩa của chúng ta về tính hợp lý không yêu cầu sự toàn tri, bởi vì sự lựa chọn hợp lý chỉ phụ thuộc vào chuỗi tri giác cho đến nay. Chúng ta cũng phải đảm bảo rằng chúng ta đã không vô tình cho phép tác nhân tham gia vào các hoạt động kém thông minh một cách dứt khoát. Ví dụ, nếu một tác nhân không nhìn cả hai bên trước khi băng qua một con đường đông đúc, thì chuỗi tri giác của nó sẽ không cho nó biết rằng có một chiếc xe tải lớn đang lao tới với tốc độ cao. Định nghĩa của chúng ta về tính hợp lý có nói rằng bây giờ có thể băng qua đường không? Hoàn toàn không!

Đầu tiên, sẽ **không hợp lý** để băng qua đường với chuỗi tri giác không có thông tin này: rủi ro tai nạn do băng qua mà không nhìn là quá lớn. Thứ hai, một tác nhân hợp lý nên chọn hành động “nhìn” trước khi bước xuống đường, bởi vì việc nhìn giúp tối đa hóa hiệu suất kỳ vọng. Thực hiện các hành động để sửa đổi các tri giác trong tương lai—đôi khi được gọi là **thu thập thông tin**—là một phần quan trọng của tính hợp lý và được đề cập chuyên sâu trong Chương 15. Một ví dụ thứ hai về việc thu thập thông tin được cung cấp bởi việc khám phá mà một tác nhân dọn dẹp hút bụi phải thực hiện trong một môi trường ban đầu không xác định.

Định nghĩa của chúng ta yêu cầu một tác nhân hợp lý không chỉ **thu thập thông tin** mà còn **học** càng nhiều càng tốt từ những gì nó nhận thức. Cấu hình ban đầu của tác nhân có thể phản ánh một số kiến thức ban đầu về môi trường, nhưng khi tác nhân có kinh nghiệm, điều này có thể được sửa đổi và bổ sung. Có những trường hợp cực đoan trong đó môi trường được biết hoàn toàn từ trước và hoàn toàn có thể dự đoán được. Trong những trường hợp như vậy, tác nhân không cần phải nhận thức hoặc học hỏi; nó chỉ đơn giản là hành động một cách chính xác.

Tất nhiên, những tác nhân như vậy rất mong manh. Hãy xem con bọ hung hèn mọn. Sau khi đào tổ và đẻ trứng, nó lấy một viên phân từ một đồng gần đó để bịt lối vào. Nếu viên phân bị lấy khỏi tay nó trên đường đi, con bọ hung vẫn tiếp tục nhiệm vụ của mình và diễn kịch bịt tổ bằng viên phân không tồn tại, không bao giờ nhận thấy rằng nó đã biến mất. Sự tiến hóa đã xây dựng một giả định vào hành vi của con bọ hung, và khi giả định đó bị vi phạm, hành vi không thành công sẽ xảy ra.

Thông minh hơn một chút là con ong bắp cày sphex. Con ong cái sphex sẽ đào một cái hang, đi ra ngoài và chích một con sâu bướm rồi kéo nó đến cái hang, vào lại cái hang để kiểm tra mọi thứ đều ổn, kéo con sâu bướm vào bên trong và đẻ trứng. Con sâu bướm đóng vai trò là nguồn thức ăn khi trứng nở. Cho đến nay thì tốt, nhưng nếu một nhà côn trùng học di chuyển con sâu bướm cách đó vài inch trong khi con sphex đang kiểm tra, nó sẽ quay lại bước “kéo con sâu bướm” của kế hoạch và sẽ tiếp tục kế hoạch mà không sửa đổi, kiểm tra lại cái hang, ngay cả sau hàng chục lần can thiệp di chuyển sâu bướm. Con sphex không thể học rằng kế hoạch bẩm sinh của nó đang thất bại, và do đó sẽ không thay đổi nó.

Ở mức độ mà một tác nhân dựa vào **kiến thức ban đầu** của nhà thiết kế hơn là vào các quá trình tri giác và học tập của chính nó, chúng ta nói rằng tác nhân đó thiếu **tính tự chủ**. Một tác nhân hợp lý nên **tự chủ**—nó nên học những gì có thể để bù đắp cho kiến thức ban đầu không đầy đủ hoặc không chính xác. Ví dụ, một tác nhân dọn dẹp hút bụi học cách dự đoán bụi bẩn bổ sung sẽ xuất hiện ở đâu và khi nào sẽ làm tốt hơn một tác nhân không làm như vậy.

Như một vấn đề thực tế, người ta hiếm khi yêu cầu sự tự chủ hoàn toàn ngay từ đầu: khi tác nhân có ít hoặc không có kinh nghiệm, nó sẽ phải hành động ngẫu nhiên trừ khi nhà thiết kế cung cấp một số trợ giúp. Giống như sự tiến hóa cung cấp cho động vật đủ phản xạ bẩm sinh để sống sót đủ lâu để tự học, sẽ là hợp lý để cung cấp cho một tác nhân thông minh nhân tạo một số kiến thức ban đầu cũng như khả năng học hỏi. Sau khi có đủ kinh nghiệm về môi trường của nó, hành vi của một tác nhân hợp lý có thể trở nên độc lập một cách hiệu quả với kiến thức ban đầu của nó. Do đó, việc kết hợp **học tập** cho phép người ta thiết kế một tác nhân hợp lý duy nhất sẽ thành công trong một loạt các môi trường rộng lớn.

2.3 Bản chất của Môi trường

Bây giờ chúng ta đã có một định nghĩa về tính hợp lý, chúng ta gần như đã sẵn sàng để suy nghĩ về việc xây dựng các tác nhân hợp lý. Tuy nhiên, trước tiên, chúng ta phải nghĩ về các **môi trường tác vụ**, về cơ bản là những “bài toán” mà các tác nhân hợp lý là “giải pháp.” Chúng ta bắt đầu bằng cách chỉ ra cách chỉ định một môi trường tác vụ, minh họa quy trình bằng một số ví dụ. Sau đó, chúng ta sẽ chỉ ra rằng các môi trường tác vụ có nhiều loại khác nhau. Bản chất của môi trường tác vụ ảnh hưởng trực tiếp đến thiết kế chương trình tác nhân phù hợp cho môi trường đó.

2.3.1 Chỉ định môi trường tác vụ

Trong cuộc thảo luận về tính hợp lý của tác nhân máy hút bụi đơn giản, chúng ta đã phải chỉ định thước đo hiệu suất, môi trường, và các bộ truyền động và cảm biến của tác nhân. Chúng ta nhóm tất cả những điều này dưới tiêu đề của **môi trường tác vụ**. Đối với những người thích từ viết tắt, chúng ta gọi đây là mô tả **PEAS** (Performance, Environment, Actuators, Sensors) – Hiệu suất, Môi trường, Bộ truyền động, Cảm biến. Khi thiết kế một tác nhân, bước đầu tiên phải luôn là chỉ định môi trường tác vụ càng đầy đủ càng tốt.

Thế giới máy hút bụi là một ví dụ đơn giản; hãy xem xét một vấn đề phức tạp hơn: một **tài xế taxi tự động**. Hình 2.4 tóm tắt mô tả PEAS cho môi trường tác vụ của taxi. Chúng ta sẽ thảo luận chi tiết hơn về từng yếu tố trong các đoạn sau.

Đầu tiên, thước đo hiệu suất mà chúng ta muốn tài xế tự động của mình hướng tới là gì? Các phẩm chất đáng mong muốn bao gồm: đến đúng đích; giảm thiểu tiêu thụ nhiên liệu và hao mòn; giảm thiểu thời gian hoặc chi phí chuyến đi; giảm thiểu vi phạm luật giao thông và sự làm phiền đối với các tài xế khác; tối đa hóa an toàn và sự thoải mái của hành khách; tối đa hóa lợi nhuận. Rõ ràng, một số mục tiêu này xung đột với nhau, vì vậy sẽ cần phải có sự đánh đổi.

Tiếp theo, môi trường lái xe mà taxi sẽ phải đối mặt là gì? Bất kỳ tài xế taxi nào cũng phải đối phó với nhiều loại đường khác nhau, từ làn đường nông thôn và ngõ hẻm đô thị đến đường cao tốc 12 làn. Các con đường có các phương tiện giao thông khác, người đi bộ, động vật đi lạc, công trình đường bộ, xe cảnh sát, vũng nước và ổ gà. Taxi cũng phải tương tác với hành khách tiềm năng và hành khách thực tế. Cũng có một số lựa chọn tùy chọn. Taxi có thể cần hoạt động ở Nam California, nơi tuyệt hiếm khi là vấn đề, hoặc ở Alaska, nơi nó hiếm khi không phải là vấn đề. Nó có thể luôn lái ở bên phải, hoặc chúng ta có thể muốn nó đủ linh hoạt để lái ở bên trái khi ở Anh hoặc Nhật Bản. Rõ ràng, môi trường càng bị hạn chế, vấn đề thiết kế càng dễ dàng.

Hiệu suất	An toàn, đến đích, lợi nhuận, hợp pháp, thoải mái
Môi trường	Đường bộ, các phương tiện khác, cảnh sát, người đi bộ, khách hàng, thời tiết
Bộ truyền động	Vô lăng, bàn đạp ga, phanh, đèn xi nhan, còi, màn hình hiển thị, giọng nói
Cảm biến	Camera, radar, đồng hồ tốc độ, GPS, cảm biến động cơ, gia tốc kế, micro, màn hình cảm ứng

Hình 2.4: Mô tả PEAS của môi trường tác vụ cho một tài xế taxi tự động.

Các **bộ truyền động** cho một taxi tự động bao gồm những thứ có sẵn cho một tài xế con người: điều khiển động cơ thông qua bàn đạp ga và điều khiển vô lăng và phanh. Ngoài ra, nó sẽ cần đầu ra cho màn hình hiển thị hoặc bộ tổng hợp giọng nói để nói chuyện lại với hành khách, và có lẽ một số cách để giao tiếp với các phương tiện khác, một cách lịch sự hoặc không.

Các **cảm biến** cơ bản cho taxi sẽ bao gồm một hoặc nhiều camera video để nó có thể nhìn thấy, cũng như cảm biến lidar và siêu âm để phát hiện khoảng cách đến các xe khác và chướng ngại vật. Để tránh vé phạt chạy quá tốc độ, taxi nên có đồng hồ tốc độ, và để điều khiển phương tiện một cách thích hợp, đặc biệt là ở các khúc cua, nó nên có gia tốc kế. Để xác định trạng thái cơ học của phương tiện, nó sẽ cần một loạt các cảm biến động cơ, nhiên liệu và hệ thống điện thông thường. Giống như nhiều tài xế con người, nó có thể muốn truy cập tín hiệu GPS để không bị lạc. Cuối cùng, nó sẽ cần đầu vào màn hình cảm ứng hoặc giọng nói để hành khách yêu cầu một điểm đến.

Trong Hình 2.5, chúng ta đã phác thảo các yếu tố PEAS cơ bản cho một số loại tác nhân bổ sung. Các ví dụ khác xuất hiện trong Bài tập 2.PEAS. Các ví dụ bao gồm môi trường vật lý cũng như ảo. Lưu ý rằng các môi trường tác vụ ảo có thể phức tạp như thế giới “thực”: ví dụ, một **tác nhân phần mềm** (hoặc robot phần mềm hay **softbot**) giao dịch trên các trang web đấu giá và bán lại đối phó với hàng triệu người dùng khác và hàng tỷ đối tượng, nhiều đối tượng có hình ảnh thực.

2.3.2 Các thuộc tính của môi trường tác vụ

Phạm vi của các môi trường tác vụ có thể phát sinh trong AI rõ ràng là rất lớn. Tuy nhiên, chúng ta có thể xác định một số ít các chiều mà theo đó các môi trường tác vụ có thể được phân loại. Các chiều này xác định, ở một mức độ lớn, thiết kế tác nhân phù hợp và khả năng áp dụng của từng họ kỹ thuật chính để triển khai tác nhân. Đầu tiên, chúng ta liệt kê các chiều, sau đó chúng ta phân tích một số môi trường tác vụ để minh họa các ý tưởng. Các định nghĩa ở đây là không chính thức; các chương sau sẽ cung cấp các tuyên bố và ví dụ chính xác hơn về từng loại môi trường.

- **Có thể quan sát hoàn toàn so với có thể quan sát một phần:** Nếu các cảm biến của tác nhân cho phép nó truy cập vào trạng thái hoàn chỉnh của môi trường tại mỗi thời điểm, thì chúng ta nói rằng môi trường tác vụ là **có thể quan sát hoàn toàn**. Một môi trường tác vụ có hiệu quả là có thể quan sát hoàn toàn nếu các cảm biến phát hiện tất cả các khía cạnh có liên quan đến việc lựa chọn hành động; sự liên quan, ngược lại, phụ thuộc vào thước đo hiệu suất. Các môi trường có thể quan sát hoàn toàn rất thuận tiện vì tác nhân không cần duy trì bất kỳ trạng thái nội bộ nào để theo dõi thế giới. Một môi trường có thể là **có thể quan sát một phần** do các cảm biến bị nhiễu và không chính xác hoặc vì các phần của trạng thái đơn giản là bị thiếu trong dữ liệu cảm biến—ví dụ, một tác nhân hút bụi chỉ có cảm biến bụi bản cục bộ không thể biết liệu có bụi bản ở các ô vuông khác hay không, và một chiếc taxi tự động không thể nhìn thấy những gì các tài xế khác đang nghĩ. Nếu tác nhân không có cảm biến nào cả thì môi trường là **không thể quan sát**. Người ta có thể nghĩ rằng trong những trường hợp như vậy, tình trạng khó khăn của tác nhân là vô vọng, nhưng, như chúng ta thảo luận trong Chương 4, các mục tiêu của tác nhân vẫn có thể đạt được, đôi khi với sự chắc chắn.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments	Touchscreen/voice entry of symptoms and findings
Satellite image analysis system	Correct categorization of objects, terrain	Orbiting satellite, downlink, weather	Display of scene categorization	High-resolution digital camera
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, tactile and joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, raw materials, operators	Valves, pumps, heaters, stirrers, displays	Temperature, pressure, flow, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, feedback, speech	Keyboard entry, voice

Hình 2.5: Các ví dụ về loại tác nhân và mô tả PEAS của chúng.

- Một tác nhân so với nhiều tác nhân:** Sự phân biệt giữa môi trường **một tác nhân** và **nhiều tác nhân** có vẻ khá đơn giản. Ví dụ, một tác nhân tự giải một câu đố ô chữ rõ ràng là trong một môi trường một tác nhân, trong khi một tác nhân chơi cờ vua là trong một môi trường hai tác nhân. Tuy nhiên, có một số vấn đề tinh tế. Đầu tiên, chúng ta đã mô tả cách một thực thể có thể được xem là một tác nhân, nhưng chúng ta chưa giải thích thực thể nào phải được xem là tác nhân. Một tác nhân A (ví dụ, tài xế taxi) có phải coi một đối tượng B (một phương tiện khác) là một tác nhân, hay nó có thể được coi chỉ là một đối tượng hành xử theo các định luật vật lý, tương tự như sóng trên bãi biển hay lá cây bay trong gió? Sự khác biệt chính là liệu hành vi của B có được mô tả tốt nhất là tối đa hóa một thước đo hiệu suất có giá trị phụ thuộc vào hành vi của tác nhân A hay không. Ví dụ, trong cờ vua, thực thể đối thủ B đang cố gắng tối đa hóa thước đo hiệu suất của nó, mà theo luật cờ vua, sẽ giảm thiểu thước đo hiệu suất của tác nhân A. Do đó, cờ vua là một môi trường **nhiều tác nhân cạnh tranh**. Mặt khác, trong môi trường lái taxi, việc tránh va chạm sẽ tối đa hóa thước đo hiệu suất của tất cả các tác nhân, vì vậy đó là một môi trường **nhiều tác nhân hợp tác một phần**. Nó cũng cạnh tranh một phần vì, ví dụ, chỉ có một chiếc xe có thể chiếm một chỗ đậu xe. Các vấn đề thiết kế tác nhân trong môi trường nhiều tác nhân thường khá khác với những vấn đề trong môi trường một tác nhân; ví dụ, giao tiếp thường xuất hiện như một hành vi hợp lý trong môi trường nhiều tác nhân; trong một số môi trường

cạnh tranh, hành vi ngẫu nhiên là hợp lý vì nó tránh được những cạm bẫy của tính có thể dự đoán.

- **Xác định so với không xác định:** Nếu trạng thái tiếp theo của môi trường được xác định hoàn toàn bởi trạng thái hiện tại và hành động được thực thi bởi tác nhân, thì chúng ta nói rằng môi trường là **xác định**; nếu không, nó là **không xác định**. Về nguyên tắc, một tác nhân không cần phải lo lắng về sự không chắc chắn trong một môi trường xác định, có thể quan sát hoàn toàn. Tuy nhiên, nếu môi trường có thể quan sát một phần, thì nó có thể xuất hiện là không xác định. Hầu hết các tình huống thực tế đều quá phức tạp nên không thể theo dõi tất cả các khía cạnh không được quan sát; vì mục đích thực tế, chúng phải được coi là không xác định. Lái xe taxi rõ ràng là không xác định theo nghĩa này, bởi vì người ta không bao giờ có thể dự đoán chính xác hành vi của giao thông; hơn nữa, lớp xe có thể nổ bất ngờ và động cơ có thể ngừng hoạt động mà không báo trước. Thế giới máy hút bụi mà chúng ta đã mô tả là xác định, nhưng các biến thể có thể bao gồm các yếu tố không xác định như bụi bẩn xuất hiện ngẫu nhiên và cơ chế hút không đáng tin cậy (Bài tập 2.VFIN). Một lưu ý cuối cùng: từ **stochastic** được một số người sử dụng như một từ đồng nghĩa với “không xác định,” nhưng chúng ta tạo ra một sự phân biệt giữa hai thuật ngữ; chúng ta nói rằng một mô hình của môi trường là stochastic nếu nó xử lý rõ ràng các xác suất (ví dụ, “có 25% khả năng mưa vào ngày mai”) và “không xác định” nếu các khả năng được liệt kê mà không được định lượng (ví dụ, “có khả năng mưa vào ngày mai”).
- **Tập hợp so với tuần tự:** Trong một môi trường tác vụ **tập hợp**, trải nghiệm của tác nhân được chia thành các tập hợp nguyên tử. Trong mỗi tập hợp, tác nhân nhận một tri giác và sau đó thực hiện một hành động duy nhất. Điều quan trọng là, tập hợp tiếp theo không phụ thuộc vào các hành động đã thực hiện trong các tập hợp trước. Nhiều tác vụ phân loại là tập hợp. Ví dụ, một tác nhân phải phát hiện các bộ phận bị lỗi trên dây chuyền lắp ráp dựa trên mỗi quyết định trên bộ phận hiện tại, bất kể các quyết định trước đó; hơn nữa, quyết định hiện tại không ảnh hưởng đến việc bộ phận tiếp theo có bị lỗi hay không. Ngược lại, trong các môi trường **tuần tự**, quyết định hiện tại có thể ảnh hưởng đến tất cả các quyết định trong tương lai. Cờ vua và lái xe taxi là tuần tự: trong cả hai trường hợp, các hành động ngắn hạn có thể có hậu quả dài hạn. Các môi trường tập hợp đơn giản hơn nhiều so với các môi trường tuần tự vì tác nhân không cần phải suy nghĩ trước.
- **Tĩnh so với động:** Nếu môi trường có thể thay đổi trong khi tác nhân đang suy nghĩ, thì chúng ta nói rằng môi trường là **động** đối với tác nhân đó; nếu không, nó là **tĩnh**. Các môi trường tĩnh rất dễ xử lý vì tác nhân không cần phải tiếp tục nhìn vào thế giới trong khi nó đang quyết định một hành động, cũng như không cần phải lo lắng về thời gian trôi qua. Ngược lại, các môi trường động liên tục hỏi tác nhân nó muốn làm gì; nếu nó chưa quyết định, điều đó được tính là quyết định không làm gì. Nếu bản thân môi trường không thay đổi theo thời gian trôi qua nhưng điểm hiệu suất của tác nhân thì có, thì chúng ta nói môi trường là **bán động**. Lái xe taxi rõ ràng là động: những chiếc xe khác và chính chiếc taxi tiếp tục di chuyển trong khi thuật toán lái xe đang phân vân nên làm gì tiếp theo. Cờ vua, khi chơi với đồng hồ, là bán động. Các câu đố ô chữ là tĩnh.
- **Rời rạc so với liên tục:** Sự phân biệt rời rạc/liên tục áp dụng cho trạng thái của môi trường, cách xử lý thời gian, và các tri giác và hành động của tác nhân. Ví dụ, môi trường cờ vua có một số hữu hạn các trạng thái riêng biệt (không bao gồm đồng hồ). Cờ vua cũng có một tập hợp các tri giác và hành động rời rạc. Lái xe taxi là một vấn đề trạng thái liên tục và thời gian liên tục: tốc độ và vị trí của taxi và của các phương tiện khác quét qua một phạm vi các giá trị liên tục và làm như vậy một cách trơn tru theo thời gian. Các hành động lái

xe taxi cũng là liên tục (góc lái, v.v.). Đầu vào từ camera kỹ thuật số là rời rạc, nói một cách nghiêm túc, nhưng thường được coi là đại diện cho các cường độ và vị trí thay đổi liên tục.

- **Đã biết so với chưa biết:** Nói một cách nghiêm túc, sự phân biệt này không đề cập đến bản thân môi trường mà là trạng thái kiến thức của tác nhân (hoặc nhà thiết kế) về các “định luật vật lý” của môi trường. Trong một môi trường **đã biết**, các kết quả (hoặc xác suất kết quả nếu môi trường không xác định) cho tất cả các hành động đều được đưa ra. Rõ ràng, nếu môi trường **chưa biết**, tác nhân sẽ phải học cách nó hoạt động để đưa ra các quyết định tốt. Sự phân biệt giữa môi trường đã biết và chưa biết không giống với sự phân biệt giữa môi trường có thể quan sát hoàn toàn và có thể quan sát một phần. Rất có thể một môi trường đã biết là có thể quan sát một phần—ví dụ, trong các trò chơi bài solitaire, tôi biết các quy tắc nhưng vẫn không thể nhìn thấy các lá bài chưa được lật lên. Ngược lại, một môi trường chưa biết có thể có thể quan sát hoàn toàn—trong một trò chơi điện tử mới, màn hình có thể hiển thị toàn bộ trạng thái trò chơi nhưng tôi vẫn không biết các nút bấm làm gì cho đến khi tôi thử chúng. Như đã lưu ý trên trang 57, bản thân thước đo hiệu suất có thể chưa biết, hoặc là do nhà thiết kế không chắc chắn cách viết nó một cách chính xác hoặc vì người dùng cuối cùng—mà sở thích của họ là quan trọng—không được biết. Ví dụ, một tài xế taxi thường sẽ không biết liệu một hành khách mới thích một hành trình nhanh nhả hay nhanh chóng, một phong cách lái xe thận trọng hay hung hăng. Một trợ lý cá nhân ảo bắt đầu mà không biết gì về sở thích cá nhân của chủ sở hữu mới của nó. Trong những trường hợp như vậy, tác nhân có thể tìm hiểu thêm về thước đo hiệu suất dựa trên các tương tác tiếp theo với nhà thiết kế hoặc người dùng. Điều này, ngược lại, cho thấy rằng môi trường tác vụ nhất thiết phải được xem là một môi trường nhiều tác nhân.

Trường hợp khó nhất là **có thể quan sát một phần, nhiều tác nhân, không xác định, tuần tự, động, liên tục và chưa biết**. Lái xe taxi khó ở tất cả những khía cạnh này, ngoại trừ việc môi trường của tài xế hầu hết là đã biết. Lái một chiếc xe thuê ở một quốc gia mới với địa lý không quen thuộc, luật giao thông khác nhau và hành khách lo lắng thì thú vị hơn nhiều.

Hình 2.6 liệt kê các thuộc tính của một số môi trường quen thuộc. Lưu ý rằng các thuộc tính không phải lúc nào cũng rõ ràng. Ví dụ, chúng ta đã liệt kê tác vụ chẩn đoán y tế là một tác nhân vì quá trình bệnh trong một bệnh nhân không được mô hình hóa một cách có lợi như một tác nhân; nhưng một hệ thống chẩn đoán y tế cũng có thể phải đối phó với những bệnh nhân ương ngạnh và nhân viên hoài nghi, vì vậy môi trường có thể có một khía cạnh nhiều tác nhân. Hơn nữa, chẩn đoán y tế là tập hợp nếu người ta hình dung tác vụ là chọn một chẩn đoán dựa trên một danh sách các triệu chứng; vấn đề là tuần tự nếu tác vụ có thể bao gồm việc đề xuất một loạt các xét nghiệm, đánh giá tiến độ trong quá trình điều trị, xử lý nhiều bệnh nhân, v.v.

Môi trường tác vụ	Có thể quan sát	Tác nhân	Xác định	Tập hợp	Tĩnh	Rời rạc
Câu đố ô chữ	Hoàn toàn	Một	Xác định	Tuần tự	Tĩnh	Rời rạc
Cờ vua có đồng hồ	Hoàn toàn	Nhiều	Xác định	Tuần tự	Bán động	Rời rạc

Môi trường tác vụ	Có thể quan sát	Tác nhân	Xác định	Tập hợp	Tĩnh	Rời rạc
Poker	Một phần	Nhiều	Stochastic	Tuần tự	Tĩnh	Rời rạc
Backgammon	Hoàn toàn	Nhiều	Stochastic	Tuần tự	Tĩnh	Rời rạc
Lái xe taxi	Một phần	Nhiều	Stochastic	Tuần tự	Động	Liên tục
Chẩn đoán y tế	Một phần	Một	Stochastic	Tuần tự	Động	Liên tục
Phân tích hình ảnh	Hoàn toàn	Một	Xác định	Tập hợp	Bán động	Liên tục
Robot nhặt bộ phận	Một phần	Một	Stochastic	Tập hợp	Động	Liên tục
Bộ điều khiển nhà máy lọc dầu	Một phần	Một	Stochastic	Tuần tự	Động	Liên tục
Gia sư tiếng Anh	Một phần	Nhiều	Stochastic	Tuần tự	Động	Rời rạc

Hình 2.6: Ví dụ về các môi trường tác vụ và các đặc điểm của chúng.

Chúng ta đã không bao gồm cột “đã biết/chưa biết” vì, như đã giải thích ở trên, đây không hẳn là một thuộc tính của môi trường. Đối với một số môi trường, chẳng hạn như cờ vua và poker, việc cung cấp cho tác nhân kiến thức đầy đủ về các quy tắc là khá dễ dàng, nhưng dù sao đi nữa, việc xem xét một tác nhân có thể học cách chơi các trò chơi này mà không có kiến thức đó là điều thú vị. Kho lưu trữ mã liên quan đến cuốn sách này (aima.cs.berkeley.edu) bao gồm nhiều triển khai môi trường, cùng với một trình mô phỏng môi trường đa năng để đánh giá hiệu suất của tác nhân. Các thí nghiệm thường được thực hiện không phải cho một môi trường duy nhất mà cho nhiều môi trường được lấy từ một **lớp môi trường**. Ví dụ, để đánh giá một tài xế taxi trong giao thông mô phỏng, chúng ta sẽ muốn chạy nhiều mô phỏng với các điều kiện giao thông, ánh sáng và thời tiết khác nhau. Sau đó, chúng ta quan tâm đến hiệu suất trung bình của tác nhân trên lớp môi trường.

2.4 Cấu trúc của tác nhân

Cho đến nay, chúng ta đã nói về các tác nhân bằng cách mô tả hành vi—hành động được thực hiện sau bất kỳ chuỗi tri giác nào. Bây giờ, chúng ta phải đi sâu vào cách thức hoạt động bên trong của chúng. Nhiệm vụ của AI là thiết kế một **chương trình tác nhân** (agent program) để thực hiện **hàm tác nhân** (agent function)—ánh xạ từ tri giác đến hành động. Chúng ta giả định chương trình này sẽ chạy trên một thiết bị máy tính nào đó với các cảm biến và bộ truyền động vật lý—chúng ta gọi đây là **kiến trúc tác nhân** (agent architecture). .

tác nhân = kiến trúc + chương trình.

Rõ ràng, chương trình chúng ta chọn phải phù hợp với kiến trúc. Nếu chương trình đưa ra các hành động như Đi bộ, thì kiến trúc cần có chân. Kiến trúc có thể chỉ là một máy tính thông thường, hoặc có thể là một chiếc xe robot với nhiều máy tính, camera và cảm biến khác trên bo mạch. Nói chung, kiến trúc làm cho tri giác từ các cảm biến sẵn có cho chương trình, chạy chương trình và đưa các lựa chọn hành động của chương trình đến bộ truyền động khi chúng được tạo ra. Hầu hết cuốn sách này là về việc thiết kế các chương trình tác nhân, mặc dù Chương 26 và 27 sẽ đề cập trực tiếp đến các cảm biến và bộ truyền động.

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
               table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

Hình 2.7 Chương trình **TABLE-DRIVEN-AGENT** được gọi cho mỗi tri giác mới và trả về một hành động mỗi lần. Nó giữ lại toàn bộ chuỗi tri giác trong bộ nhớ.

2.4.1 Các chương trình tác nhân

Các chương trình tác nhân mà chúng ta thiết kế trong cuốn sách này đều có cùng một khung sườn: chúng nhận tri giác hiện tại làm đầu vào từ các cảm biến và trả về một hành động cho các bộ truyền động.

Hãy chú ý sự khác biệt giữa **chương trình tác nhân**, chỉ lấy tri giác hiện tại làm đầu vào, và **hàm tác nhân**, có thể phụ thuộc vào toàn bộ lịch sử tri giác. Chương trình tác nhân không có lựa chọn nào khác ngoài việc chỉ lấy tri giác hiện tại làm đầu vào vì không có gì khác có sẵn từ môi trường; nếu các hành động của tác nhân cần phụ thuộc vào toàn bộ chuỗi tri giác, tác nhân sẽ phải **ghi nhớ** các tri giác.

Chúng tôi mô tả các chương trình tác nhân bằng ngôn ngữ giả mã đơn giản được định nghĩa trong Phụ lục B. (Kho lưu trữ mã trực tuyến chứa các triển khai bằng các ngôn ngữ lập trình thực.) Ví dụ, Hình 2.7 cho thấy một chương trình tác nhân khá đơn giản theo dõi chuỗi tri giác và sau đó sử dụng nó để tra cứu vào một bảng các hành động để quyết định phải làm gì. Bảng này—một ví dụ được đưa ra cho thế giới máy hút bụi trong Hình 2.3—biểu diễn một cách rõ ràng hàm tác nhân mà chương trình tác nhân thể hiện. Để xây dựng một tác nhân hợp lý theo cách này, chúng ta với tư cách là những nhà thiết kế phải xây dựng một bảng chứa hành động thích hợp cho mọi chuỗi tri giác có thể xảy ra.

Việc xem xét tại sao cách tiếp cận xây dựng tác nhân dựa trên bảng lại **chắc chắn thất bại** là điều hữu ích. Gọi P là tập hợp các tri giác có thể và T là thời gian sống của tác nhân (tổng số tri giác nó sẽ nhận được). Bảng tra cứu sẽ chứa $sum_t = 1^T \mid P \mid^t$ mục nhập. Hãy xem xét một chiếc taxi tự động: đầu vào hình ảnh từ một camera duy nhất (tám camera là điển hình) đến với tốc độ khoảng 70 megabyte mỗi giây (30 khung hình mỗi giây, 1080×720 pixel với 24 bit thông tin màu). Điều này mang lại một bảng tra cứu với hơn $10^{600,000,000,000}$ mục nhập cho một giờ lái xe. Ngay cả

bảng tra cứu cho cờ vua—một phần nhỏ, hoạt động tốt của thế giới thực—cũng có ít nhất 10^{150} mục nhập. Trong khi đó, số nguyên tử trong vũ trụ có thể quan sát được chưa đến 10^{80} .

Kích thước khổng lồ của các bảng này có nghĩa là (a) không có tác nhân vật lý nào trong vũ trụ này có đủ không gian để lưu trữ bảng; (b) nhà thiết kế sẽ không có thời gian để tạo bảng; và (c) không có tác nhân nào có thể học được tất cả các mục nhập bảng đúng từ kinh nghiệm của nó.

Mặc dù vậy, **TABLE-DRIVEN-AGENT** vẫn làm được điều chúng ta muốn, giả sử bảng đã được điền đúng: nó thực hiện hàm tác nhân mong muốn.

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```

Hình 2.8 Chương trình tác nhân cho một tác nhân phản xạ đơn giản trong môi trường máy hút bụi hai vị trí. Chương trình này thực hiện hàm tác nhân được lập bảng trong Hình 2.3.

Thách thức chính đối với AI là tìm ra cách viết các chương trình mà, trong chừng mực có thể, tạo ra hành vi hợp lý từ một chương trình nhỏ thay vì từ một bảng khổng lồ.

Chúng ta có nhiều ví dụ cho thấy điều này có thể được thực hiện thành công trong các lĩnh vực khác: ví dụ, các bảng căn bậc hai khổng lồ được các kỹ sư và học sinh sử dụng trước những năm 1970 giờ đây đã được thay thế bằng một chương trình năm dòng cho phương pháp Newton chạy trên máy tính điện tử. Câu hỏi đặt ra là, liệu AI có thể làm cho hành vi thông minh nói chung những gì Newton đã làm cho căn bậc hai không? Chúng tôi tin rằng câu trả lời là có.

Trong phần còn lại của phần này, chúng ta sẽ phân tích bốn loại chương trình tác nhân cơ bản thể hiện các nguyên tắc làm nền tảng cho hầu hết các hệ thống thông minh:

- **Tác nhân phản xạ đơn giản** (Simple reflex agents);
- **Tác nhân phản xạ dựa trên mô hình** (Model-based reflex agents);
- **Tác nhân dựa trên mục tiêu** (Goal-based agents); và
- **Tác nhân dựa trên tiện ích** (Utility-based agents).

Mỗi loại chương trình tác nhân kết hợp các thành phần cụ thể theo những cách cụ thể để tạo ra các hành động. Phần 2.4.6 giải thích một cách tổng quát cách chuyển đổi tất cả các tác nhân này thành **tác nhân học tập** (learning agents) có thể cải thiện hiệu suất của các thành phần của chúng để tạo ra các hành động tốt hơn. Cuối cùng, Phần 2.4.7 mô tả nhiều cách mà bản thân các thành phần có thể được biểu diễn bên trong tác nhân. Sự đa dạng này cung cấp một nguyên tắc tổ chức chính cho lĩnh vực này và cho chính cuốn sách.

2.4.2 Tác nhân phản xạ đơn giản

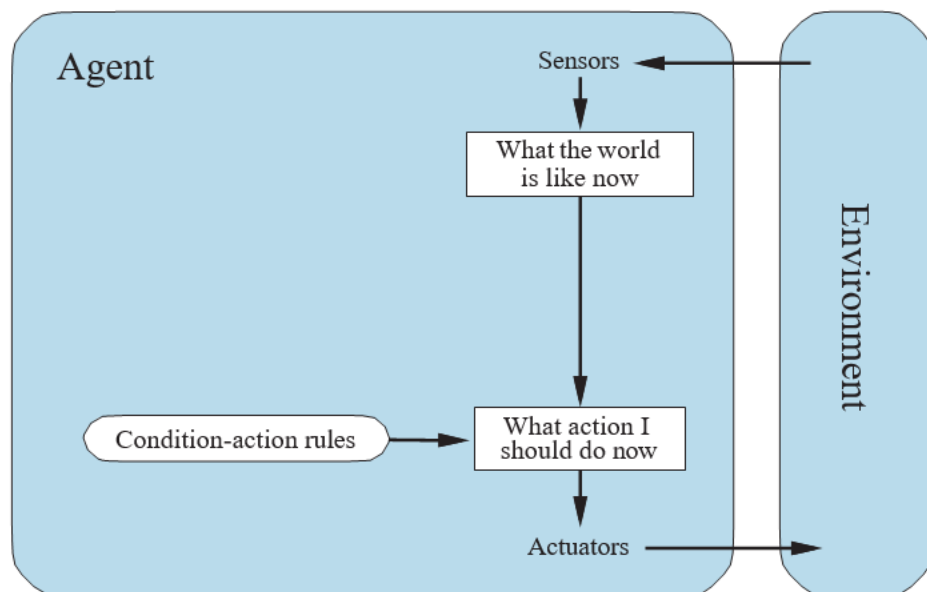
Loại tác nhân đơn giản nhất là **tác nhân phản xạ đơn giản**. Những tác nhân này chọn hành động dựa trên **tri giác hiện tại**, bỏ qua phần còn lại của lịch sử tri giác. Ví dụ, tác nhân máy hút bụi có hàm tác nhân được lập bảng trong Hình 2.3 là một tác nhân phản xạ đơn giản, bởi vì quyết định của nó chỉ dựa trên vị trí hiện tại và liệu vị trí đó có chứa bụi bẩn hay không. Một chương trình tác nhân cho tác nhân này được thể hiện trong Hình 2.8.

Hãy chú ý rằng chương trình tác nhân máy hút bụi rất nhỏ so với bảng tương ứng. Sự giảm đáng kể nhất đến từ việc bỏ qua lịch sử tri giác, làm giảm số lượng chuỗi tri giác liên quan từ 4^T xuống chỉ còn 4. Một sự giảm nhỏ hơn nữa đến từ thực tế là khi ô hiện tại bẩn, hành động không phụ thuộc vào vị trí. Mặc dù chúng ta đã viết chương trình tác nhân bằng cách sử dụng các câu lệnh if-then-else, nó đủ đơn giản để cũng có thể được triển khai dưới dạng một mạch Boolean.

Các hành vi phản xạ đơn giản cũng xảy ra trong các môi trường phức tạp hơn. Hãy tưởng tượng bạn là người lái chiếc taxi tự động. Nếu chiếc xe phía trước phanh và đèn phanh của nó bật sáng, bạn nên nhận thấy điều này và bắt đầu phanh. Nói cách khác, một số xử lý được thực hiện trên đầu vào hình ảnh để thiết lập điều kiện mà chúng ta gọi là “Chiếc xe phía trước đang phanh.” Sau đó, điều này kích hoạt một kết nối đã được thiết lập trong chương trình tác nhân đến hành động “bắt đầu phanh.” Chúng ta gọi kết nối như vậy là một **quy tắc điều kiện-hành động** (condition–action rule), được viết là:

nếu chiếc-xe-phía-trước-đang-phanh thì bắt-đầu-phanh.

Con người cũng có nhiều kết nối như vậy, một số là phản ứng học được (như khi lái xe) và một số là phản xạ bẩm sinh (như chớp mắt khi có thứ gì đó đến gần mắt). Trong suốt cuốn sách, chúng ta sẽ chỉ ra một số cách khác nhau để các kết nối như vậy có thể được học và triển khai.



Hình 2.9 Sơ đồ cấu trúc của một tác nhân phản xạ đơn giản. Chúng ta sử dụng hình chữ nhật để biểu thị trạng thái nội bộ hiện tại của quá trình ra quyết định của tác nhân, và hình bầu dục để biểu thị thông tin nền được sử dụng trong quá trình.

Chương trình trong Hình 2.8 là cụ thể cho một môi trường máy hút bụi cụ thể. Một cách tiếp cận tổng quát và linh hoạt hơn là trước tiên xây dựng một trình thông dịch đa năng cho các quy tắc điều kiện-hành động và sau đó tạo các bộ quy tắc cho các môi trường tác vụ cụ thể. Hình 2.9 đưa ra cấu trúc của chương trình tổng quát này dưới dạng sơ đồ, cho thấy cách các quy tắc điều kiện-hành động cho phép tác nhân tạo kết nối từ tri giác đến hành động. Đừng lo lắng nếu điều này có vẻ tầm thường; nó sẽ trở nên thú vị hơn ngay sau đây.

Một chương trình tác nhân cho Hình 2.9 được thể hiện trong Hình 2.10. Hàm **INTERPRET-INPUT** tạo ra một mô tả trừu tượng của trạng thái hiện tại từ tri giác, và hàm **RULE-MATCH** trả về quy tắc đầu tiên trong bộ quy tắc khớp với mô tả trạng thái đã cho. Lưu ý rằng mô tả về “quy tắc” và “khớp” chỉ mang tính khái niệm; như đã lưu ý ở trên, các triển khai thực tế có thể đơn giản như một tập hợp các công logic thực hiện một mạch Boolean. Ngoài ra, một mạch “thần kinh” có thể được sử dụng, trong đó các công logic được thay thế bằng các đơn vị phi tuyến của mạng nơ-ron nhân tạo (xem Chương 22).

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition-action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Hình 2.10 Một tác nhân phản xạ đơn giản. Nó hoạt động theo một quy tắc có điều kiện khớp với trạng thái hiện tại, được xác định bởi tri giác.

Các tác nhân phản xạ đơn giản có thuộc tính đáng ngưỡng mộ là đơn giản, nhưng chúng có trí thông minh hạn chế. Tác nhân trong Hình 2.10 sẽ chỉ hoạt động nếu quyết định đúng có thể được đưa ra chỉ dựa trên tri giác hiện tại—nghĩa là, chỉ khi môi trường **có thể quan sát đầy đủ**.

Ngay cả một chút không thể quan sát cũng có thể gây ra rắc rối nghiêm trọng. Ví dụ, quy tắc phanh được đưa ra trước đó giả định rằng điều kiện chiếc-xe-phía-trước-đang-phanh có thể được xác định từ tri giác hiện tại—một khung hình video duy nhất. Điều này hoạt động nếu chiếc xe phía trước có một đèn phanh gắn ở trung tâm (và do đó có thể nhận dạng duy nhất). Thật không may, các mẫu cũ hơn có các cấu hình đèn hậu, đèn phanh và đèn xi-nhan khác nhau, và không phải lúc nào cũng có thể biết từ một hình ảnh duy nhất liệu chiếc xe đang phanh hay chỉ bật đèn hậu. Một tác nhân phản xạ đơn giản lái xe phía sau một chiếc xe như vậy sẽ hoặc phanh liên tục và không cần thiết, hoặc tệ hơn, không bao giờ phanh.

Chúng ta có thể thấy một vấn đề tương tự nảy sinh trong thế giới máy hút bụi. Giả sử một tác nhân máy hút bụi phản xạ đơn giản bị tước cảm biến vị trí và chỉ có cảm biến bụi bẩn. Một tác nhân như vậy chỉ có hai tri giác có thể: [Bẩn] và [Sạch]. Nó có thể Hút để đáp lại [Bẩn]; vậy nó nên làm gì để đáp lại [Sạch]? Di chuyển sang Trái sẽ thất bại (mãi mãi) nếu nó bắt đầu ở ô A, và di chuyển sang

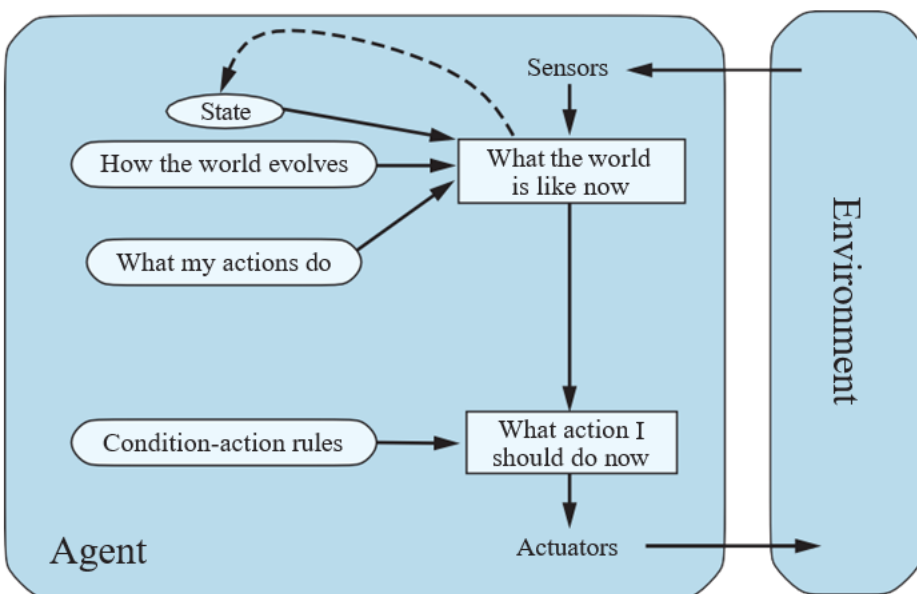
Phải sẽ thất bại (mãi mãi) nếu nó bắt đầu ở ô B. **Vòng lặp vô hạn** thường là không thể tránh khỏi đối với các tác nhân phản xạ đơn giản hoạt động trong các môi trường có thể quan sát một phần.

Thoát khỏi các vòng lặp vô hạn là có thể nếu tác nhân có thể **ngẫu nhiên hóa** các hành động của nó. Ví dụ, nếu tác nhân máy hút bụi nhận thấy [Sạch], nó có thể tung đồng xu để chọn giữa Phải và Trái. Dễ dàng chỉ ra rằng tác nhân sẽ đến được ô kia trong trung bình hai bước. Sau đó, nếu ô đó bẩn, tác nhân sẽ dọn dẹp nó và nhiệm vụ sẽ hoàn thành. Do đó, một tác nhân phản xạ đơn giản ngẫu nhiên hóa có thể vượt trội hơn một tác nhân phản xạ đơn giản có tính xác định.

Chúng ta đã đề cập trong Phần 2.3 rằng hành vi ngẫu nhiên hóa đúng cách có thể hợp lý trong một số môi trường đa tác nhân. Trong các môi trường đơn tác nhân, ngẫu nhiên hóa thường không hợp lý. Đó là một mẹo hữu ích giúp một tác nhân phản xạ đơn giản trong một số tình huống, nhưng trong hầu hết các trường hợp, chúng ta có thể làm tốt hơn nhiều với các tác nhân xác định tinh vi hơn.

2.4.3 Tác nhân phản xạ dựa trên mô hình

Cách hiệu quả nhất để xử lý khả năng quan sát một phần là để tác nhân theo dõi phần thế giới mà nó không thể nhìn thấy ngay lúc này. Tức là, tác nhân nên duy trì một loại **trạng thái nội bộ** (internal state) nào đó phụ thuộc vào lịch sử tri giác và từ đó phản ánh ít nhất một số khía cạnh không được quan sát của trạng thái hiện tại. Đối với vấn đề phanh, trạng thái nội bộ không quá rộng—chỉ là khung hình trước đó từ camera, cho phép tác nhân phát hiện khi hai đèn đỏ ở rìa xe bật hoặc tắt cùng lúc. Đối với các tác vụ lái xe khác như chuyển làn, tác nhân cần theo dõi vị trí của các xe khác nếu nó không thể nhìn thấy tất cả cùng một lúc. Và để việc lái xe có thể diễn ra, tác nhân cần theo dõi chìa khóa của nó đang ở đâu.



Hình 2.11 Một tác nhân phản xạ dựa trên mô hình.

Việc cập nhật thông tin trạng thái nội bộ này theo thời gian đòi hỏi hai loại kiến thức phải được mã hóa trong chương trình tác nhân dưới một dạng nào đó. Đầu tiên, chúng ta cần một số thông tin về cách thế giới thay đổi theo thời gian, có thể chia đại khái thành hai phần: ảnh hưởng của các hành động của tác nhân và cách thế giới tự phát triển độc lập với tác nhân. Ví dụ, khi tác nhân quay vô lăng theo chiều kim đồng hồ, xe rẽ sang phải, và khi trời mưa, camera của xe có thể bị ướt. Kiến thức này về “cách thế giới hoạt động”—dù được triển khai trong các mạch Boolean đơn giản hay trong các lý thuyết khoa học hoàn chỉnh—được gọi là **mô hình chuyển tiếp** (transition model) của thế giới.

Thứ hai, chúng ta cần một số thông tin về cách trạng thái của thế giới được phản ánh trong các tri giác của tác nhân. Ví dụ, khi chiếc xe phía trước bắt đầu phanh, một hoặc nhiều vùng màu đỏ được chiếu sáng xuất hiện trong hình ảnh camera phía trước, và, khi camera bị ướt, các vật thể hình giọt nước xuất hiện trong hình ảnh che khuất một phần đường. Loại kiến thức này được gọi là **mô hình cảm biến** (sensor model).

Cùng nhau, mô hình chuyển tiếp và mô hình cảm biến cho phép một tác nhân theo dõi trạng thái của thế giới—trong chừng mực có thể với những hạn chế của cảm biến của tác nhân. Một tác nhân sử dụng các mô hình như vậy được gọi là **tác nhân dựa trên mô hình** (model-based agent).

Hình 2.11 đưa ra cấu trúc của tác nhân phản xạ dựa trên mô hình với trạng thái nội bộ, cho thấy cách tri giác hiện tại được kết hợp với trạng thái nội bộ cũ để tạo ra mô tả cập nhật của trạng thái hiện tại, dựa trên mô hình của tác nhân về cách thế giới hoạt động. Chương trình tác nhân được thể hiện trong Hình 2.12. Phần thú vị là hàm UPDATE-STATE, chịu trách nhiệm tạo mô tả trạng thái nội bộ mới. Chi tiết về cách các mô hình và trạng thái được biểu diễn khác nhau rất nhiều tùy thuộc vào loại môi trường và công nghệ cụ thể được sử dụng trong thiết kế tác nhân.

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               transition model, a description of how the next state depends on
                 the current state and action
               sensor model, a description of how the current world state is reflected
                 in the agent's percepts
               rules, a set of condition-action rules
               action, the most recent action, initially none

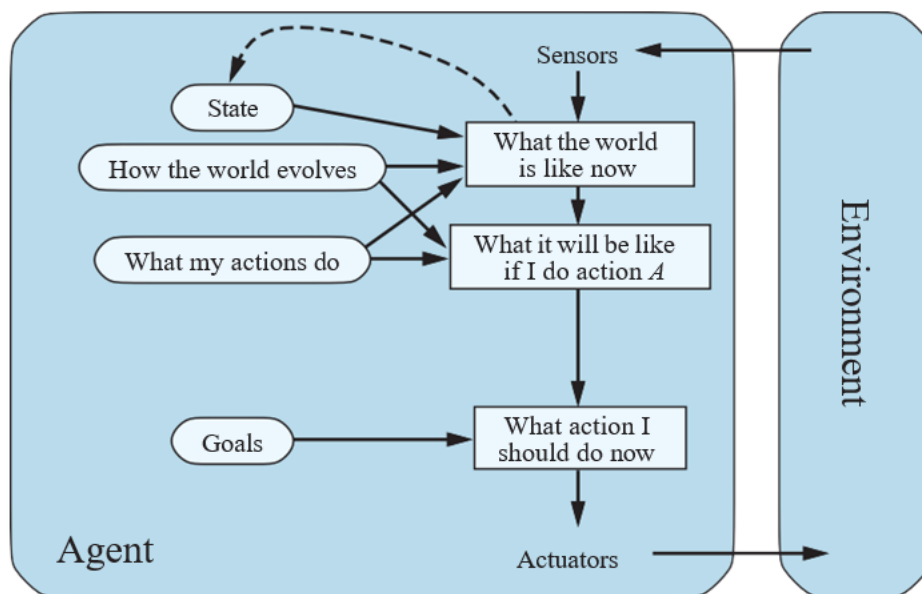
  state ← UPDATE-STATE(state, action, percept, transition model, sensor model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Hình 2.12 Một tác nhân phản xạ dựa trên mô hình. Nó theo dõi trạng thái hiện tại của thế giới, sử dụng một mô hình nội bộ. Sau đó, nó chọn một hành động giống như tác nhân phản xạ.

Bất kể loại biểu diễn nào được sử dụng, hiếm khi tác nhân có thể xác định chính xác trạng thái hiện tại của một môi trường có thể quan sát một phần. Thay vào đó, ô được dán nhãn “thế giới bây giờ như thế nào” (Hình 2.11) đại diện cho “đoán tốt nhất” của tác nhân (hoặc đôi khi là những “đoán tốt nhất,” nếu tác nhân cân nhắc nhiều khả năng). Ví dụ, một chiếc taxi tự động có thể không thể nhìn thấy xung quanh chiếc xe tải lớn đã dừng lại phía trước nó và chỉ có thể đoán về những gì có thể đang gây ra sự tắc nghẽn. Do đó, sự không chắc chắn về trạng thái hiện tại có thể là không thể tránh khỏi, nhưng tác nhân vẫn phải đưa ra quyết định.

2.4.4 Tác nhân dựa trên mục tiêu

Biết một cái gì đó về trạng thái hiện tại của môi trường không phải lúc nào cũng đủ để quyết định phải làm gì. Ví dụ, tại một ngã tư đường, taxi có thể rẽ trái, rẽ phải hoặc đi thẳng. Quyết định đúng đắn phụ thuộc vào nơi taxi đang cố gắng đến. Nói cách khác, cũng như một mô tả trạng thái hiện tại, tác nhân cần một số loại thông tin **mục tiêu** (goal) mô tả các tình huống mong muốn—ví dụ, đang ở một điểm đến cụ thể. Chương trình tác nhân có thể kết hợp điều này với mô hình (cùng thông tin đã được sử dụng trong tác nhân phản xạ dựa trên mô hình) để chọn các hành động đạt được mục tiêu. Hình 2.13 cho thấy cấu trúc của tác nhân dựa trên mục tiêu.



Hình 2.13 Một tác nhân dựa trên mô hình, dựa trên mục tiêu. Nó theo dõi trạng thái thế giới cũng như một tập hợp các mục tiêu mà nó đang cố gắng đạt được, và chọn một hành động sẽ (cuối cùng) dẫn đến việc đạt được các mục tiêu của nó.

Đôi khi việc lựa chọn hành động dựa trên mục tiêu là đơn giản—ví dụ, khi việc thỏa mãn mục tiêu là kết quả ngay lập tức từ một hành động duy nhất. Đôi khi nó sẽ phức tạp hơn—ví dụ, khi tác nhân phải xem xét các chuỗi ngoặt và rẽ dài để tìm cách đạt được mục tiêu. **Tìm kiếm** (các Chương 3, 4, và 6) và **lập kế hoạch** (Chương 11) là các tiểu lĩnh vực của AI dành cho việc tìm kiếm các chuỗi hành động đạt được mục tiêu của tác nhân.

Lưu ý rằng việc ra quyết định kiểu này về cơ bản khác với các quy tắc điều kiện-hành động đã mô tả trước đó, ở chỗ nó liên quan đến việc xem xét tương lai—cả “Điều gì sẽ xảy ra nếu tôi làm điều này?” và “Điều đó có làm tôi hài lòng không?” Trong các thiết kế tác nhân phản xạ, thông tin này không được biểu diễn một cách rõ ràng, bởi vì các quy tắc tích hợp ánh xạ trực tiếp từ tri giác đến hành động. Tác nhân phản xạ phanh khi nó thấy đèn phanh, chấm hết. Nó không biết tại sao. Một tác nhân dựa trên mục tiêu phanh khi nó thấy đèn phanh bởi vì đó là hành động duy nhất mà nó dự đoán sẽ đạt được mục tiêu không đâm vào các xe khác.

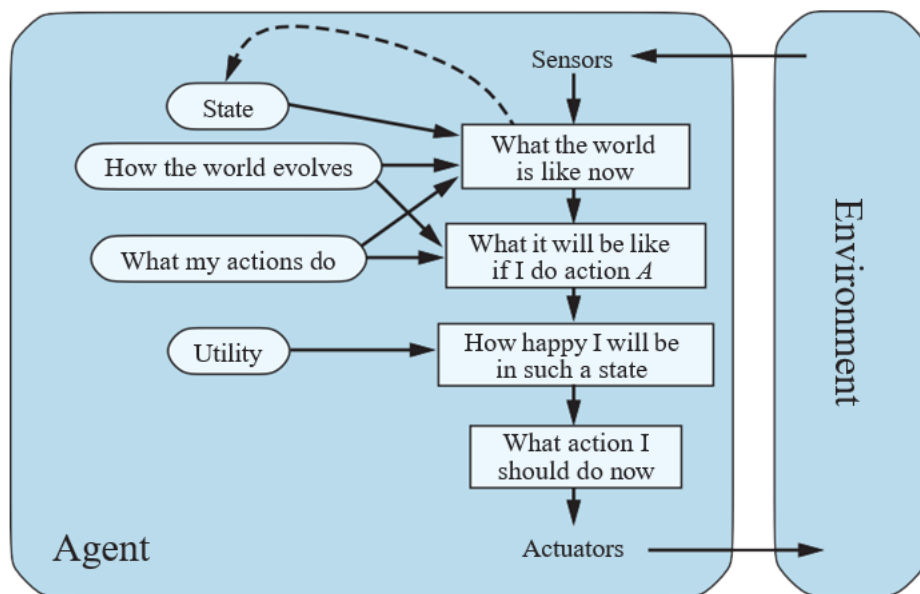
Mặc dù tác nhân dựa trên mục tiêu có vẻ kém hiệu quả hơn, nó linh hoạt hơn vì kiến thức hỗ trợ các quyết định của nó được biểu diễn một cách rõ ràng và có thể được sửa đổi. Ví dụ, hành vi của một tác nhân dựa trên mục tiêu có thể dễ dàng thay đổi để đi đến một điểm đến khác, chỉ bằng cách chỉ định điểm đến đó là mục tiêu. Các quy tắc của tác nhân phản xạ về thời điểm rẽ và thời điểm đi thẳng sẽ chỉ hoạt động cho một điểm đến duy nhất; chúng phải được thay thế hoàn toàn để đi đến một nơi mới.

2.4.5 Tác nhân dựa trên tiện ích

Mục tiêu một mình không đủ để tạo ra hành vi chất lượng cao trong hầu hết các môi trường. Ví dụ, nhiều chuỗi hành động sẽ đưa taxi đến đích (do đó đạt được mục tiêu), nhưng một số nhanh hơn, an toàn hơn, đáng tin cậy hơn hoặc rẻ hơn những chuỗi khác. Các mục tiêu chỉ cung cấp một sự phân biệt nhị phân thô giữa các trạng thái “hạnh phúc” và “không hạnh phúc”. Một thước đo hiệu suất tổng quát hơn nên cho phép so sánh các trạng thái thế giới khác nhau dựa trên mức độ hạnh phúc mà chúng mang lại cho tác nhân. Vì “hạnh phúc” nghe có vẻ không khoa học lắm, các nhà kinh tế và nhà khoa học máy tính sử dụng thuật ngữ **tiện ích** (utility).

Chúng ta đã thấy rằng một thước đo hiệu suất chỉ định một điểm số cho bất kỳ chuỗi trạng thái môi trường nào, vì vậy nó có thể dễ dàng phân biệt giữa các cách đến đích của taxi mong muốn hơn và kém hơn. **Hàm tiện ích** (utility function) của một tác nhân về cơ bản là sự nội hóa của thước đo hiệu suất. Với điều kiện hàm tiện ích nội bộ và thước đo hiệu suất bên ngoài đồng thuận, một tác nhân chọn các hành động để tối đa hóa tiện ích của nó sẽ là hợp lý theo thước đo hiệu suất bên ngoài.

Hãy nhấn mạnh lại rằng đây không phải là cách duy nhất để trở nên hợp lý—chúng ta đã thấy một chương trình tác nhân hợp lý cho thế giới máy hút bụi (Hình 2.8) mà không biết hàm tiện ích của nó là gì—nhưng, giống như các tác nhân dựa trên mục tiêu, một tác nhân dựa trên tiện ích có nhiều lợi thế về tính linh hoạt và học hỏi. Hơn nữa, trong hai loại trường hợp, các mục tiêu là không đủ nhưng một tác nhân dựa trên tiện ích vẫn có thể đưa ra các quyết định hợp lý. Thứ nhất, khi có các mục tiêu xung đột, chỉ một số mục tiêu có thể đạt được (ví dụ: tốc độ và an toàn), hàm tiện ích sẽ xác định sự đánh đổi thích hợp. Thứ hai, khi có một số mục tiêu mà tác nhân có thể nhắm tới, không mục tiêu nào có thể đạt được một cách chắc chắn, tiện ích cung cấp một cách để cân nhắc khả năng thành công so với tầm quan trọng của các mục tiêu.



Hình 2.14 Một tác nhân dựa trên mô hình, dựa trên tiện ích. Nó sử dụng một mô hình của thế giới, cùng với một hàm tiện ích đo lường sở thích của nó giữa các trạng thái của thế giới. Sau đó, nó chọn hành động dẫn đến tiện ích kỳ vọng tốt nhất, trong đó tiện ích kỳ vọng được tính bằng cách tính trung bình trên tất cả các trạng thái kết quả có thể, được trọng số hóa bằng xác suất của kết quả.

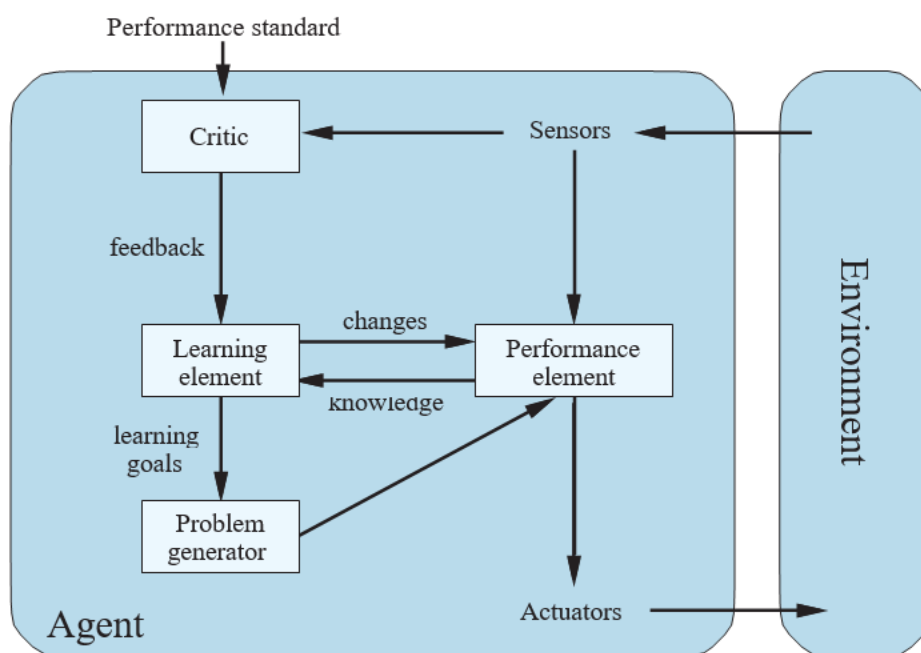
Khả năng quan sát một phần và tính không xác định là phổ biến trong thế giới thực, và do đó, việc ra quyết định trong điều kiện không chắc chắn cũng vậy. Về mặt kỹ thuật, một tác nhân dựa trên tiện ích hợp lý chọn hành động tối đa hóa **tiện ích kỳ vọng** (expected utility) của các kết quả hành động—nghĩa là, tiện ích mà tác nhân mong đợi nhận được, tính trung bình, với xác suất và tiện ích của mỗi kết quả. (Phụ lục A định nghĩa kỳ vọng chính xác hơn.) Trong Chương 15, chúng ta sẽ chỉ ra rằng bất kỳ tác nhân hợp lý nào cũng phải hành động như thể nó sở hữu một hàm tiện ích mà nó cố gắng tối đa hóa giá trị kỳ vọng. Một tác nhân sở hữu một hàm tiện ích rõ ràng có thể đưa ra các quyết định hợp lý với một thuật toán đa năng không phụ thuộc vào hàm tiện ích cụ thể đang được tối đa hóa. Bằng cách này, định nghĩa “toàn cầu” về tính hợp lý—chỉ định là hợp lý những hàm tác nhân có hiệu suất cao nhất—được biến thành một ràng buộc “cục bộ” đối với các thiết kế tác nhân hợp lý có thể được thể hiện trong một chương trình đơn giản.

Cấu trúc tác nhân dựa trên tiện ích xuất hiện trong Hình 2.14. Các chương trình tác nhân dựa trên tiện ích xuất hiện trong Chương 15 và 16, nơi chúng ta thiết kế các tác nhân ra quyết định phải xử lý sự không chắc chắn vốn có trong các môi trường không xác định hoặc có thể quan sát một phần. Việc ra quyết định trong môi trường đa tác nhân cũng được nghiên cứu trong khuôn khổ của lý thuyết tiện ích, như đã giải thích trong Chương 17.

Tại thời điểm này, người đọc có thể đang tự hỏi, “Có đơn giản vậy không? Chúng ta chỉ cần xây dựng các tác nhân tối đa hóa tiện ích kỳ vọng là xong?” Đúng là các tác nhân như vậy sẽ thông minh, nhưng nó không hề đơn giản. Một tác nhân dựa trên tiện ích phải mô hình hóa và theo dõi môi trường của nó, những nhiệm vụ đã liên quan đến rất nhiều nghiên cứu về nhận thức, biểu diễn,

suy luận và học hỏi. Kết quả của nghiên cứu này lấp đầy nhiều chương của cuốn sách này. Việc chọn chuỗi hành động tối đa hóa tiện ích cũng là một nhiệm vụ khó khăn, đòi hỏi các thuật toán khéo léo lấp đầy thêm một số chương nữa. Ngay cả với các thuật toán này, tính hợp lý hoàn hảo thường không thể đạt được trong thực tế vì độ phức tạp tính toán, như chúng ta đã lưu ý trong Chương 1. Chúng ta cũng lưu ý rằng không phải tất cả các tác nhân dựa trên tiện ích đều dựa trên mô hình; chúng ta sẽ thấy trong Chương 23 và 26 rằng một **tác nhân không dựa trên mô hình** (model-free agent) có thể học được hành động nào là tốt nhất trong một tình huống cụ thể mà không cần học chính xác hành động đó thay đổi môi trường như thế nào.

Cuối cùng, tất cả những điều này giả định rằng nhà thiết kế có thể chỉ định hàm tiện ích một cách chính xác; Chương 16, 17 và 23 xem xét vấn đề các hàm tiện ích không xác định một cách sâu sắc hơn.



Hình 2.15 Một tác nhân học tập tổng quát. Hộp “phần tử hiệu suất” đại diện cho những gì chúng ta đã coi là toàn bộ chương trình tác nhân. Bây giờ, hộp “phần tử học tập” có thể sửa đổi chương trình đó để cải thiện hiệu suất của nó.

2.4.6 Tác nhân học tập

Chúng ta đã mô tả các chương trình tác nhân với các phương pháp khác nhau để chọn hành động. Cho đến nay, chúng ta vẫn chưa giải thích cách các chương trình tác nhân được tạo ra. Trong bài báo nổi tiếng ban đầu của mình, Turing (1950) xem xét ý tưởng thực sự lập trình thủ công các cỗ máy thông minh của mình. Ông ước tính công việc này có thể mất bao nhiêu và kết luận, “Một phương pháp nhanh hơn có vẻ là mong muốn.” Phương pháp ông đề xuất là xây dựng các máy học tập và sau đó dạy chúng. Trong nhiều lĩnh vực của AI, đây hiện là phương pháp ưu tiên để tạo ra

các hệ thống tiên tiến. Bất kỳ loại tác nhân nào (dựa trên mô hình, dựa trên mục tiêu, dựa trên tiện ích, v.v.) đều có thể được xây dựng dưới dạng một **tác nhân học tập** (learning agent) (hoặc không).

Học tập có một lợi thế khác, như chúng ta đã lưu ý trước đó: nó cho phép tác nhân hoạt động trong các môi trường ban đầu không xác định và trở nên có năng lực hơn so với kiến thức ban đầu của nó. Trong phần này, chúng ta sẽ giới thiệu ngắn gọn các ý tưởng chính của tác nhân học tập. Trong suốt cuốn sách, chúng ta sẽ bình luận về các cơ hội và phương pháp học tập trong các loại tác nhân cụ thể. Các Chương 19, 21, 22 và 23 đi sâu hơn nhiều vào bản thân các thuật toán học tập.

Một tác nhân học tập có thể được chia thành bốn thành phần khái niệm, như thể hiện trong Hình 2.15. Sự khác biệt quan trọng nhất là giữa **phần tử học tập** (learning element), chịu trách nhiệm thực hiện các cải tiến, và **phần tử hiệu suất** (performance element), chịu trách nhiệm chọn các hành động bên ngoài. Phần tử hiệu suất là những gì chúng ta đã coi là toàn bộ tác nhân: nó nhận tri giác và quyết định các hành động. Phần tử học tập sử dụng phản hồi từ **nhà phê bình** (critic) về việc tác nhân đang hoạt động như thế nào và xác định cách phần tử hiệu suất nên được sửa đổi để hoạt động tốt hơn trong tương lai.

Thiết kế của phần tử học tập phụ thuộc rất nhiều vào thiết kế của phần tử hiệu suất. Khi cố gắng thiết kế một tác nhân học một khả năng nhất định, câu hỏi đầu tiên không phải là “Làm thế nào tôi sẽ khiến nó học được điều này?” mà là “Tác nhân của tôi sẽ sử dụng loại phần tử hiệu suất nào để làm điều này khi nó đã học được cách?” Với một thiết kế cho phần tử hiệu suất, các cơ chế học tập có thể được xây dựng để cải thiện mọi phần của tác nhân.

Nhà phê bình cho phần tử học tập biết tác nhân đang hoạt động tốt như thế nào so với một **tiêu chuẩn hiệu suất** (performance standard) cố định. Nhà phê bình là cần thiết vì bản thân các tri giác không cung cấp dấu hiệu nào về sự thành công của tác nhân. Ví dụ, một chương trình cờ vua có thể nhận một tri giác cho thấy nó đã chiếu bí đối thủ, nhưng nó cần một tiêu chuẩn hiệu suất để biết rằng đây là một điều tốt; bản thân tri giác không nói lên điều đó. Điều quan trọng là tiêu chuẩn hiệu suất phải được cố định. Về mặt khái niệm, người ta nên coi nó nằm ngoài tác nhân hoàn toàn vì tác nhân không được sửa đổi nó để phù hợp với hành vi của chính nó.

Thành phần cuối cùng của tác nhân học tập là **bộ tạo vấn đề** (problem generator). Nó chịu trách nhiệm đề xuất các hành động sẽ dẫn đến những trải nghiệm mới và mang tính thông tin. Nếu phần tử hiệu suất làm theo cách của nó, nó sẽ tiếp tục thực hiện các hành động tốt nhất, với những gì nó biết, nhưng nếu tác nhân sẵn sàng khám phá một chút và thực hiện một số hành động có thể là không tối ưu trong thời gian ngắn, nó có thể khám phá ra các hành động tốt hơn nhiều trong thời gian dài. Công việc của bộ tạo vấn đề là đề xuất các hành động khám phá này. Đây là những gì các nhà khoa học làm khi họ thực hiện các thí nghiệm. Galileo không nghĩ rằng việc thả đá từ đỉnh tháp ở Pisa là có giá trị tự thân. Ông không cố gắng làm vỡ những tảng đá hay sửa đổi bộ não của những người đi bộ không may mắn. Mục đích của ông là sửa đổi bộ não của chính mình bằng cách xác định một lý thuyết tốt hơn về chuyển động của các vật thể.

Phần tử học tập có thể thực hiện các thay đổi đối với bất kỳ thành phần “kiến thức” nào được thể hiện trong các sơ đồ tác nhân (Hình 2.9, 2.11, 2.13 và 2.14). Các trường hợp đơn giản nhất liên quan đến việc học trực tiếp từ chuỗi tri giác. Quan sát các cặp trạng thái liên tiếp của môi trường có thể cho phép tác nhân học “Các hành động của tôi làm gì” và “Thế giới phát triển như thế nào” để đáp lại các hành động của nó. Ví dụ, nếu chiếc taxi tự động tạo ra một áp lực phanh nhất định khi lái xe trên đường ướt, thì nó sẽ sớm tìm ra mức giảm tốc thực sự đạt được là bao nhiêu và liệu nó có bị trượt khỏi đường không. Bộ tạo vấn đề có thể xác định một số phần nhất định của mô

hình cần được cải thiện và đề xuất các thí nghiệm, chẳng hạn như thử phanh trên các bề mặt đường khác nhau trong các điều kiện khác nhau.

Cải thiện các thành phần mô hình của một tác nhân dựa trên mô hình để chúng phù hợp hơn với thực tế gần như luôn là một ý tưởng hay, bất kể tiêu chuẩn hiệu suất bên ngoài. (Trong một số trường hợp, từ quan điểm tính toán, tốt hơn là có một mô hình đơn giản nhưng hơi không chính xác hơn là một mô hình hoàn hảo nhưng cực kỳ phức tạp.) Thông tin từ tiêu chuẩn bên ngoài là cần thiết khi cố gắng học một thành phần phản xạ hoặc một hàm tiện ích.

Ví dụ, giả sử tác nhân lái taxi không nhận được tiền boa từ những hành khách đã bị xóc nảy trong suốt chuyến đi. Tiêu chuẩn hiệu suất bên ngoài phải thông báo cho tác nhân rằng việc mất tiền boa là một đóng góp tiêu cực vào hiệu suất tổng thể của nó; sau đó tác nhân có thể học được rằng những hành động cơ động mạnh mẽ không đóng góp vào tiện ích của chính nó. Theo một nghĩa nào đó, tiêu chuẩn hiệu suất phân biệt một phần của tri giác đến là một **phần thưởng** (reward) (hoặc **hình phạt** (penalty)) cung cấp phản hồi trực tiếp về chất lượng hành vi của tác nhân. Các tiêu chuẩn hiệu suất được lập trình sẵn như đau đớn và đói ở động vật có thể được hiểu theo cách này.

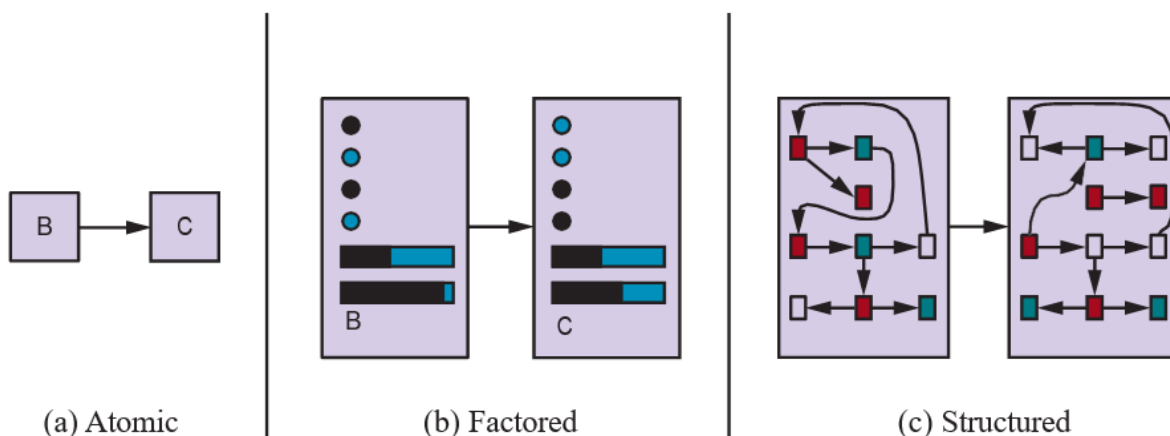
Tổng quát hơn, các lựa chọn của con người có thể cung cấp thông tin về sở thích của con người. Ví dụ, giả sử taxi không biết rằng mọi người thường không thích tiếng ồn lớn, và quyết định thổi còi liên tục như một cách để đảm bảo người đi bộ biết nó đang đến. Hành vi tiếp theo của con người—bịt tai, sử dụng ngôn ngữ xấu, và có thể cắt dây còi—sẽ cung cấp bằng chứng cho tác nhân để cập nhật hàm tiện ích của nó. Vấn đề này được thảo luận thêm trong Chương 23.

Tóm lại, các tác nhân có nhiều thành phần khác nhau, và các thành phần đó có thể được biểu diễn theo nhiều cách trong chương trình tác nhân, vì vậy có vẻ như có nhiều loại phương pháp học tập. Tuy nhiên, có một chủ đề thống nhất duy nhất. Học tập trong các tác nhân thông minh có thể được tóm tắt là một quá trình sửa đổi từng thành phần của tác nhân để đưa các thành phần đó phù hợp hơn với thông tin phản hồi có sẵn, từ đó cải thiện hiệu suất tổng thể của tác nhân.

2.4.7 Cách hoạt động của các thành phần trong chương trình tác nhân

Chúng ta đã mô tả các chương trình tác nhân (một cách tổng quát) bao gồm các thành phần khác nhau, với chức năng là trả lời các câu hỏi như: “Thế giới bây giờ như thế nào?” “Hành động nào tôi nên làm bây giờ?” “Các hành động của tôi làm gì?” Câu hỏi tiếp theo đối với một sinh viên AI là, “Làm thế quái nào mà các thành phần này hoạt động?” Phải mất khoảng một nghìn trang để bắt đầu trả lời câu hỏi đó một cách chính xác, nhưng ở đây chúng ta muốn thu hút sự chú ý của người đọc đến một số điểm khác biệt cơ bản giữa các cách khác nhau mà các thành phần có thể biểu diễn môi trường mà tác nhân cư trú.

Nói một cách đại khái, chúng ta có thể đặt các biểu diễn dọc theo một trục có độ phức tạp và sức mạnh biểu đạt tăng dần—**nguyên tử** (atomic), **phân tách** (factored) và **có cấu trúc** (structured). Để minh họa những ý tưởng này, sẽ hữu ích khi xem xét một thành phần tác nhân cụ thể, chẳng hạn như thành phần xử lý “Các hành động của tôi làm gì.” Thành phần này mô tả những thay đổi có thể xảy ra trong môi trường do kết quả của việc thực hiện một hành động, và Hình 2.16 cung cấp các mô tả sơ đồ về cách các chuyển đổi đó có thể được biểu diễn.



Hình 2.16 Ba cách để biểu diễn các trạng thái và các chuyển đổi giữa chúng. (a) Biểu diễn nguyên tử: một trạng thái (chẳng hạn như B hoặc C) là một hộp đen không có cấu trúc nội bộ; (b) Biểu diễn phân tách: một trạng thái bao gồm một vectơ các giá trị thuộc tính; các giá trị có thể là Boolean, giá trị thực hoặc một trong một tập hợp các ký hiệu cố định. (c) Biểu diễn có cấu trúc: một trạng thái bao gồm các đối tượng, mỗi đối tượng có thể có các thuộc tính riêng cũng như các mối quan hệ với các đối tượng khác.

Trong **biểu diễn nguyên tử** (atomic representation), mỗi trạng thái của thế giới là không thể chia cắt—nó không có cấu trúc nội bộ. Hãy xem xét nhiệm vụ tìm một tuyến đường lái xe từ đầu này sang đầu kia của một quốc gia thông qua một chuỗi các thành phố (chúng ta giải quyết vấn đề này trong Hình 3.1 ở trang 82). Để giải quyết vấn đề này, có thể đủ để giảm trạng thái của thế giới xuống chỉ còn tên của thành phố chúng ta đang ở—một nguyên tử kiến thức duy nhất, một “hộp đen” mà thuộc tính có thể nhận thấy duy nhất của nó là giống hoặc khác với một hộp đen khác. Các thuật toán tiêu chuẩn làm nền tảng cho tìm kiếm và chơi trò chơi (Chương 3, 4 và 6), mô hình Markov ẩn (Chương 14) và quá trình quyết định Markov (Chương 16) đều hoạt động với các biểu diễn nguyên tử.

Một **biểu diễn phân tách** (factored representation) chia mỗi trạng thái thành một tập hợp các biến hoặc thuộc tính cố định, mỗi thuộc tính có thể có một giá trị. Hãy xem xét một mô tả có độ chính xác cao hơn cho cùng một vấn đề lái xe, nơi chúng ta cần quan tâm nhiều hơn là chỉ vị trí nguyên tử trong thành phố này hay thành phố khác; chúng ta có thể cần chú ý đến lượng xăng trong bình, tọa độ GPS hiện tại của chúng ta, liệu đèn cảnh báo dầu có hoạt động hay không, chúng ta có bao nhiêu tiền cho phí cầu đường, đài phát thanh nào đang bật, v.v. Trong khi hai trạng thái nguyên tử khác nhau không có gì chung—chúng chỉ là những hộp đen khác nhau—hai trạng thái phân tách khác nhau có thể chia sẻ một số thuộc tính (chẳng hạn như ở một vị trí GPS cụ thể nào đó) và không chia sẻ những thuộc tính khác (chẳng hạn như có nhiều xăng hoặc không có xăng); điều này giúp việc tìm ra cách biến trạng thái này thành trạng thái khác dễ dàng hơn nhiều. Nhiều lĩnh vực quan trọng của AI dựa trên các biểu diễn phân tách, bao gồm các thuật toán thỏa mãn ràng buộc (Chương 5), logic mệnh đề (Chương 7), lập kế hoạch (Chương 11), mạng lưới Bayesian (các Chương 12, 13, 14, 15 và 18), và các thuật toán học máy khác nhau.

Đối với nhiều mục đích, chúng ta cần hiểu thế giới có những thứ liên quan đến nhau, không chỉ là các biến với các giá trị. Ví dụ, chúng ta có thể nhận thấy một chiếc xe tải lớn phía trước chúng ta

đang lùi vào đường lái xe của một trang trại bò sữa, nhưng một con bò đi lạc đang chắn đường đi của chiếc xe tải. Một biểu diễn phân tách khó có thể được trang bị trước với thuộc tính XeTaiPhiaTruocDangLuiVaoDuongLaiXeTrangTraiBiChanBoiBoDiLac với giá trị đúng hoặc sai. Thay vào đó, chúng ta sẽ cần một **biểu diễn có cấu trúc** (structured representation), trong đó các đối tượng như bò và xe tải cùng các mối quan hệ đa dạng và thay đổi của chúng có thể được mô tả một cách rõ ràng (xem Hình 2.16(c)). Các biểu diễn có cấu trúc làm nền tảng cho các cơ sở dữ liệu quan hệ và logic bậc nhất (các Chương 8, 9 và 10), các mô hình xác suất bậc nhất (Chương 18), và phần lớn sự hiểu biết ngôn ngữ tự nhiên (các Chương 24 và 25). Trên thực tế, phần lớn những gì con người thể hiện bằng ngôn ngữ tự nhiên liên quan đến các đối tượng và mối quan hệ của chúng.

Như chúng ta đã đề cập trước đó, trực mà các biểu diễn nguyên tử, phân tách và có cấu trúc nằm trên đó là trực **biểu đạt** (expressiveness) tăng dần. Nói một cách đại khái, một biểu diễn biểu đạt hơn có thể nắm bắt, ít nhất là một cách ngắn gọn, mọi thứ mà một biểu diễn kém biểu đạt hơn có thể nắm bắt, cộng thêm một số nữa. Thông thường, ngôn ngữ biểu đạt hơn ngắn gọn hơn nhiều; ví dụ, các quy tắc của cờ vua có thể được viết trong một hoặc hai trang của một ngôn ngữ biểu diễn có cấu trúc như logic bậc nhất nhưng yêu cầu hàng nghìn trang khi được viết trong một ngôn ngữ biểu diễn phân tách như logic mệnh đề và khoảng 10^{38} trang khi được viết trong một ngôn ngữ nguyên tử như ngôn ngữ của máy trạng thái hữu hạn. Mặt khác, suy luận và học tập trở nên phức tạp hơn khi sức mạnh biểu đạt của biểu diễn tăng lên. Để đạt được lợi ích của các biểu diễn biểu đạt trong khi tránh những nhược điểm của chúng, các hệ thống thông minh cho thế giới thực có thể cần hoạt động đồng thời ở tất cả các điểm dọc theo trục.

Một trục khác cho biểu diễn liên quan đến việc ánh xạ các khái niệm đến các vị trí trong bộ nhớ vật lý, dù trong máy tính hay trong não. Nếu có một ánh xạ một-đối-một giữa các khái niệm và các vị trí bộ nhớ, chúng ta gọi đó là **biểu diễn cục bộ** (localist representation). Mặt khác, nếu biểu diễn của một khái niệm được trải rộng trên nhiều vị trí bộ nhớ, và mỗi vị trí bộ nhớ được sử dụng như một phần của biểu diễn của nhiều khái niệm khác nhau, chúng ta gọi đó là **biểu diễn phân tán** (distributed representation). Biểu diễn phân tán mạnh mẽ hơn trước nhiều và mất thông tin. Với một biểu diễn cục bộ, ánh xạ từ khái niệm đến vị trí bộ nhớ là tùy ý, và nếu một lỗi truyền tải làm méo một vài bit, chúng ta có thể nhầm lẫn từ Truck (xe tải) với khái niệm không liên quan Truce (đình chiến). Nhưng với một biểu diễn phân tán, bạn có thể nghĩ mỗi khái niệm đại diện cho một điểm trong không gian đa chiều, và nếu bạn làm méo một vài bit, bạn di chuyển đến một điểm gần đó trong không gian đó, điểm đó sẽ có ý nghĩa tương tự.

Tóm tắt

Chương này là một chuyến tham quan lỏng xoáy về AI, mà chúng ta đã hình dung là khoa học về thiết kế tác nhân. Các điểm chính cần nhớ là:

- Một **tác nhân** là một cái gì đó cảm nhận và hành động trong một môi trường. **Hàm tác nhân** cho một tác nhân chỉ định hành động được thực hiện bởi tác nhân để đáp lại bất kỳ chuỗi tri giác nào.
- **Thước đo hiệu suất** đánh giá hành vi của tác nhân trong một môi trường. Một **tác nhân hợp lý** hành động để tối đa hóa **giá trị kỳ vọng** của thước đo hiệu suất, với chuỗi tri giác mà nó đã thấy cho đến nay.

- Một đặc tả **môi trường tác vụ** bao gồm thước đo hiệu suất, môi trường bên ngoài, các bộ truyền động và các cảm biến. Khi thiết kế một tác nhân, bước đầu tiên phải luôn là đặc tả môi trường tác vụ một cách đầy đủ nhất có thể.
- Môi trường tác vụ khác nhau dọc theo một số chiều quan trọng. Chúng có thể là có thể quan sát đầy đủ hoặc một phần, đơn tác nhân hoặc đa tác nhân, xác định hoặc không xác định, tập rời rạc hoặc tuần tự, tĩnh hoặc động, rời rạc hoặc liên tục, và đã biết hoặc chưa biết.
- Trong các trường hợp thước đo hiệu suất không xác định hoặc khó chỉ định một cách chính xác, có một rủi ro đáng kể là tác nhân tối ưu hóa mục tiêu sai. Trong những trường hợp như vậy, thiết kế tác nhân nên phản ánh sự không chắc chắn về mục tiêu thực sự.
- **Chương trình tác nhân** thực hiện hàm tác nhân. Có nhiều thiết kế chương trình tác nhân cơ bản khác nhau phản ánh loại thông tin được làm rõ và sử dụng trong quá trình ra quyết định. Các thiết kế khác nhau về hiệu quả, độ nhỏ gọn và tính linh hoạt. Thiết kế thích hợp của chương trình tác nhân phụ thuộc vào bản chất của môi trường.
- **Tác nhân phản xạ đơn giản** phản ứng trực tiếp với tri giác, trong khi **tác nhân phản xạ dựa trên mô hình** duy trì trạng thái nội bộ để theo dõi các khía cạnh của thế giới không rõ ràng trong tri giác hiện tại. **Tác nhân dựa trên mục tiêu** hành động để đạt được mục tiêu của chúng, và **tác nhân dựa trên tiện ích** cố gắng tối đa hóa “hạnh phúc” kỳ vọng của chính chúng.
- Tất cả các tác nhân có thể cải thiện hiệu suất của chúng thông qua **học tập**.

Ghi chú Thư mục và Lịch sử

Vai trò trung tâm của hành động trong trí thông minh—khái niệm về lý luận thực tiễn—có từ ít nhất là thời **Đạo đức học Nicomachean** của Aristotle. Lý luận thực tiễn cũng là chủ đề của bài báo có ảnh hưởng của McCarthy “Programs with Common Sense” (1958). Các lĩnh vực robot học và lý thuyết điều khiển, về bản chất của chúng, chủ yếu liên quan đến các tác nhân vật lý. Khái niệm **bộ điều khiển** (controller) trong lý thuyết điều khiển giống hệt với khái niệm tác nhân trong AI. Có lẽ đáng ngạc nhiên, AI đã tập trung trong phần lớn lịch sử của nó vào các thành phần cô lập của tác nhân—hệ thống trả lời câu hỏi, chứng minh định lý, hệ thống thị giác, v.v.—thay vì toàn bộ tác nhân. Cuộc thảo luận về các tác nhân trong văn bản của Genesereth và Nilsson (1987) là một ngoại lệ có ảnh hưởng. Quan điểm toàn tác nhân hiện nay được chấp nhận rộng rãi và là một chủ đề trung tâm trong các văn bản gần đây (Padgham và Winikoff, 2004; Jones, 2007; Poole và Mackworth, 2017).

Chương 1 đã truy tìm nguồn gốc của khái niệm hợp lý trong triết học và kinh tế học. Trong AI, khái niệm này chỉ có sự quan tâm ngoại vi cho đến giữa những năm 1980, khi nó bắt đầu thấm nhuần nhiều cuộc thảo luận về nền tảng kỹ thuật thích hợp của lĩnh vực này. Một bài báo của Jon Doyle (1983) đã dự đoán rằng thiết kế tác nhân hợp lý sẽ được coi là sứ mệnh cốt lõi của AI, trong khi các chủ đề phổ biến khác sẽ tách ra để hình thành các ngành học mới.

Sự chú ý cẩn thận đến các thuộc tính của môi trường và hậu quả của chúng đối với thiết kế tác nhân hợp lý là rõ ràng nhất trong truyền thống lý thuyết điều khiển—ví dụ, các hệ thống điều khiển cổ điển (Dorf và Bishop, 2004; Kirk, 2004) xử lý các môi trường có thể quan sát đầy đủ, xác định; điều khiển tối ưu ngẫu nhiên (Kumar và Varaiya, 1986; Bertsekas và Shreve, 2007) xử lý các môi trường có thể quan sát một phần, ngẫu nhiên; và điều khiển lai (Henzinger và Sastry, 1998; Cassandras và Lygeros, 2006) xử lý các môi trường chứa cả các yếu tố rời rạc và liên tục. Sự khác

biệt giữa các môi trường có thể quan sát đầy đủ và một phần cũng là trung tâm trong tài liệu lập trình động được phát triển trong lĩnh vực nghiên cứu hoạt động (Puterman, 1994), mà chúng ta sẽ thảo luận trong Chương 16.

Mặc dù các tác nhân phản xạ đơn giản là trung tâm của tâm lý học hành vi (xem Chương 1), hầu hết các nhà nghiên cứu AI đều coi chúng quá đơn giản để cung cấp nhiều đòn bẩy. (Rosenschein (1985) và Brooks (1986) đã đặt câu hỏi về giả định này; xem Chương 26.) Rất nhiều công việc đã được thực hiện để tìm các thuật toán hiệu quả để theo dõi các môi trường phức tạp (Bar-Shalom et al., 2001; Choset et al., 2005; Simon, 2006), hầu hết trong số đó trong môi trường xác suất. Các tác nhân dựa trên mục tiêu được giả định trong mọi thứ từ quan điểm lý luận thực tiễn của Aristotle đến các bài báo đầu tiên của McCarthy về AI logic. **Shakey the Robot** (Fikes và Nilsson, 1971; Nilsson, 1984) là hiện thân robot đầu tiên của một tác nhân logic, dựa trên mục tiêu. Một phân tích logic đầy đủ về các tác nhân dựa trên mục tiêu xuất hiện trong Genesereth và Nilsson (1987), và một phương pháp lập trình dựa trên mục tiêu được gọi là lập trình hướng tác nhân (agent-oriented programming) đã được Shoham (1993) phát triển. Cách tiếp cận dựa trên tác nhân hiện cực kỳ phổ biến trong kỹ thuật phần mềm (Ciancarini và Wooldridge, 2001). Nó cũng đã xâm nhập vào lĩnh vực hệ điều hành, nơi **tính toán tự trị** (autonomic computing) đề cập đến các hệ thống máy tính và mạng lưới tự giám sát và điều khiển bằng một vòng lặp cảm nhận–hành động và các phương pháp học máy (Kephart và Chess, 2003). Lưu ý rằng một tập hợp các chương trình tác nhân được thiết kế để hoạt động tốt với nhau trong một môi trường đa tác nhân thực sự nhất thiết phải thể hiện tính mô-đun—các chương trình không chia sẻ trạng thái nội bộ và giao tiếp với nhau chỉ thông qua môi trường—việc thiết kế chương trình tác nhân của một tác nhân duy nhất như một tập hợp các tác nhân phụ tự trị là điều phổ biến trong lĩnh vực hệ thống đa tác nhân. Trong một số trường hợp, người ta thậm chí có thể chứng minh rằng hệ thống kết quả đưa ra các giải pháp tối ưu tương tự như một thiết kế nguyên khối.

Quan điểm dựa trên mục tiêu của các tác nhân cũng chi phối truyền thống tâm lý học nhận thức trong lĩnh vực giải quyết vấn đề, bắt đầu với tác phẩm có ảnh hưởng lớn **Human Problem Solving** (Newell và Simon, 1972) và tiếp tục qua tất cả các công trình sau này của Newell (Newell, 1990). Các mục tiêu, được phân tích thêm là mong muốn (chung) và ý định (đang được theo đuổi), là trung tâm của lý thuyết tác nhân có ảnh hưởng được phát triển bởi Michael Bratman (1987).

Như đã lưu ý trong Chương 1, sự phát triển của lý thuyết tiện ích như một cơ sở cho hành vi hợp lý có từ hàng trăm năm trước. Trong AI, nghiên cứu ban đầu tránh tiện ích để ủng hộ mục tiêu, với một số ngoại lệ (Feldman và Sproull, 1977). Sự hồi sinh của sự quan tâm đến các phương pháp xác suất vào những năm 1980 đã dẫn đến việc chấp nhận tối đa hóa tiện ích kỳ vọng là khuôn khổ tổng quát nhất cho việc ra quyết định (Horvitz et al., 1988). Văn bản của Pearl (1988) là văn bản đầu tiên trong AI bao quát lý thuyết xác suất và tiện ích một cách sâu sắc; việc trình bày các phương pháp thực tiễn để suy luận và ra quyết định trong điều kiện không chắc chắn của nó có lẽ là yếu tố lớn nhất trong sự thay đổi nhanh chóng sang các tác nhân dựa trên tiện ích vào những năm 1990 (xem Chương 15). Việc chính thức hóa học tăng cường trong một khuôn khổ lý thuyết quyết định cũng góp phần vào sự thay đổi này (Sutton, 1988). Đáng chú ý là, gần như tất cả nghiên cứu AI cho đến gần đây đều giả định rằng thước đo hiệu suất có thể được chỉ định chính xác và đúng đắn dưới dạng một hàm tiện ích hoặc hàm phần thưởng (Hadfield-Menell et al., 2017a; Russell, 2019).

Thiết kế chung cho các tác nhân học tập được miêu tả trong Hình 2.15 là kinh điển trong tài liệu học máy (Buchanan et al., 1978; Mitchell, 1997). Các ví dụ về thiết kế, như được thể hiện trong

các chương trình, có từ ít nhất là chương trình học chơi cờ đam của Arthur Samuel (1959, 1967). Các tác nhân học tập được thảo luận sâu trong các Chương 19, 21, 22 và 23.

Một số bài báo ban đầu về các cách tiếp cận dựa trên tác nhân được tập hợp bởi Huhns và Singh (1998) và Wooldridge và Rao (1999). Các văn bản về hệ thống đa tác nhân cung cấp một giới thiệu tốt về nhiều khía cạnh của thiết kế tác nhân (Weiss, 2000a; Wooldridge, 2009). Một số chuỗi hội nghị dành riêng cho các tác nhân bắt đầu vào những năm 1990, bao gồm Hội thảo Quốc tế về Lý thuyết, Kiến trúc và Ngôn ngữ Tác nhân (ATAL), Hội nghị Quốc tế về Tác nhân Tự trị (AGENTS) và Hội nghị Quốc tế về Hệ thống Đa Tác nhân (ICMAS). Năm 2002, ba hội nghị này sáp nhập để thành lập Hội nghị Liên hợp Quốc tế về Tác nhân Tự trị và Hệ thống Đa Tác nhân (AAMAS). Từ năm 2000 đến 2012, có các hội thảo hàng năm về Kỹ thuật Phần mềm Hướng Tác nhân (AOSE). Tạp chí **Autonomous Agents and Multi-Agent Systems** được thành lập năm 1998. Cuối cùng, **Dung Beetle Ecology** (Hanski và Cambefort, 1991) cung cấp rất nhiều thông tin thú vị về hành vi của bọ hung. YouTube có các video ghi lại hoạt động đầy cảm hứng của chúng.

CHƯƠNG 3: GIẢI QUYẾT VẤN ĐỀ BẰNG CÁCH TÌM KIẾM

Khi hành động đúng để thực hiện không hiển nhiên ngay lập tức, một tác nhân có thể cần phải lên kế hoạch trước: xem xét một chuỗi các hành động tạo thành một đường dẫn đến một trạng thái mục tiêu. Một tác nhân như vậy được gọi là **tác nhân giải quyết vấn đề** (problem-solving agent), và quá trình tính toán mà nó thực hiện được gọi là **tìm kiếm** (search).

Các tác nhân giải quyết vấn đề sử dụng **biểu diễn nguyên tử**, như đã mô tả trong Phần 2.4.7—tức là, các trạng thái của thế giới được coi là toàn bộ, không có cấu trúc nội bộ nào hiển thị cho các thuật toán giải quyết vấn đề. Các tác nhân sử dụng biểu diễn trạng thái phân tách hoặc có cấu trúc được gọi là **tác nhân lập kế hoạch** (planning agents) và được thảo luận trong Chương 7 và 11.

Chúng ta sẽ đề cập đến một số thuật toán tìm kiếm. Trong chương này, chúng ta chỉ xem xét các môi trường đơn giản nhất: rời rạc, đơn tác nhân, có thể quan sát đầy đủ, xác định, tĩnh, rời rạc và đã biết. Chúng ta phân biệt giữa các **thuật toán có thông tin** (informed algorithms), trong đó tác nhân có thể ước tính nó cách mục tiêu bao xa, và các **thuật toán không có thông tin** (uninformed algorithms), trong đó không có ước tính nào như vậy. Chương 4 nói lỏng các ràng buộc về môi trường và Chương 6 xem xét nhiều tác nhân.

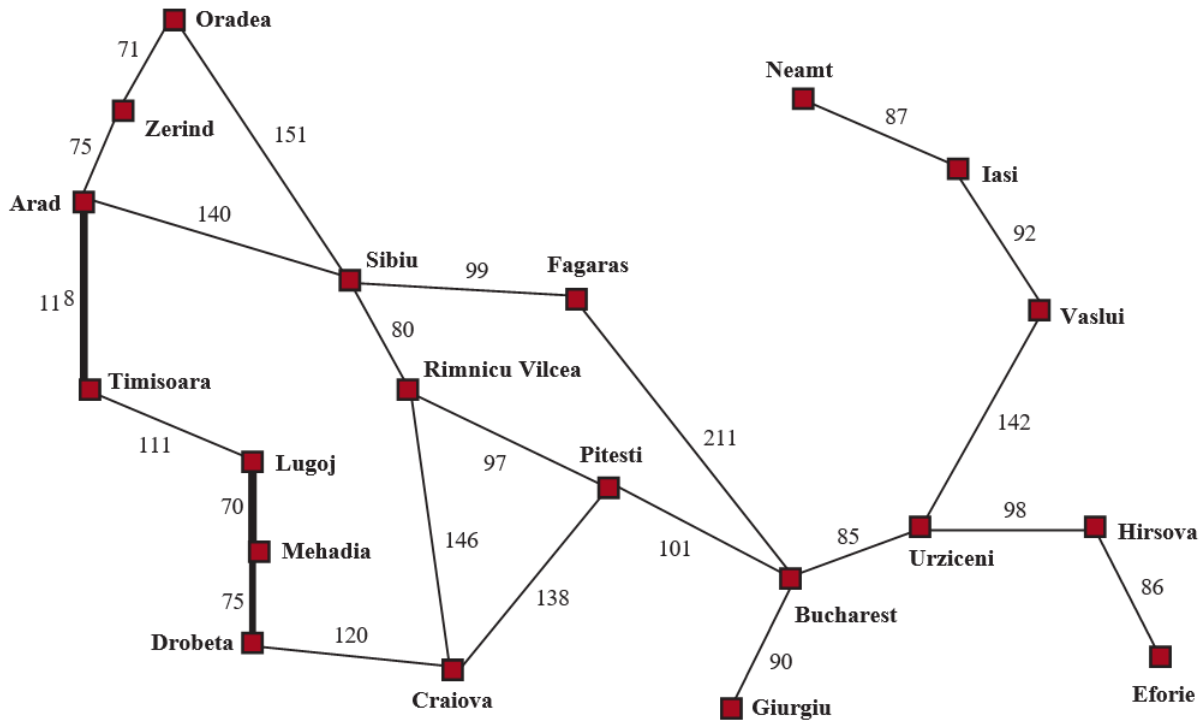
Chương này sử dụng các khái niệm về độ phức tạp tiệm cận (tức là ký hiệu $O(n)$). Độc giả chưa quen với các khái niệm này nên tham khảo Phụ lục A.

3.1 Tác nhân giải quyết vấn đề

Hãy tưởng tượng một tác nhân đang tận hưởng một kỳ nghỉ du lịch ở Romania. Tác nhân muốn ngắm cảnh, cải thiện tiếng Romania, tận hưởng cuộc sống về đêm, tránh bị say, v.v. Vấn đề ra quyết định là một vấn đề phức tạp. Bây giờ, giả sử tác nhân hiện đang ở thành phố Arad và có một vé không hoàn lại để bay ra khỏi Bucharest vào ngày hôm sau. Tác nhân quan sát các biển báo đường phố và thấy rằng có ba con đường dẫn ra khỏi Arad: một về phía Sibiu, một đến Timisoara và một đến Zerind. Không có con đường nào trong số này là mục tiêu, vì vậy trừ khi tác nhân quen thuộc với địa lý của Romania, nó sẽ không biết phải đi theo con đường nào.

Nếu tác nhân không có thông tin bổ sung—nghĩa là, nếu môi trường là chưa biết—thì tác nhân không thể làm gì tốt hơn ngoài việc thực hiện một trong các hành động một cách ngẫu nhiên. Tình huống đáng buồn này được thảo luận trong Chương 4. Trong chương này, chúng ta sẽ giả định các tác nhân của chúng ta luôn có quyền truy cập vào thông tin về thế giới, chẳng hạn như bản đồ trong Hình 3.1. Với thông tin đó, tác nhân có thể tuân theo quy trình giải quyết vấn đề bốn pha này:

- **Xây dựng mục tiêu** (Goal formulation): Tác nhân đặt mục tiêu là đến Bucharest. Các mục tiêu tổ chức hành vi bằng cách giới hạn các mục tiêu và do đó các hành động cần được xem xét.



Hình 3.1 Một bản đồ đường bộ đơn giản hóa của một phần Romania, với khoảng cách đường bộ bằng dặm.

- **Xây dựng vấn đề** (Problem formulation): Tác nhân đưa ra một mô tả về các trạng thái và hành động cần thiết để đạt được mục tiêu—một mô hình trừu tượng của phần thể giới có liên quan. Đối với tác nhân của chúng ta, một mô hình tốt là xem xét các hành động di chuyển từ thành phố này đến một thành phố liền kề, và do đó sự thật duy nhất về trạng thái của thể giới sẽ thay đổi do một hành động là thành phố hiện tại.
- **Tìm kiếm** (Search): Trước khi thực hiện bất kỳ hành động nào trong thể giới thực, tác nhân mô phỏng các chuỗi hành động trong mô hình của nó, tìm kiếm cho đến khi nó tìm thấy một chuỗi các hành động đạt đến mục tiêu. Một chuỗi như vậy được gọi là một **giải pháp** (solution). Tác nhân có thể phải mô phỏng nhiều chuỗi không đạt đến mục tiêu, nhưng cuối cùng nó sẽ tìm thấy một giải pháp (chẳng hạn như đi từ Arad đến Sibiu đến Fagaras đến Bucharest), hoặc nó sẽ thấy rằng không thể có giải pháp nào.
- **Thực thi** (Execution): Tác nhân bây giờ có thể thực thi các hành động trong giải pháp, từng hành động một.

Đây là một thuộc tính quan trọng mà trong một môi trường có thể quan sát đầy đủ, xác định, đã biết, giải pháp cho bất kỳ vấn đề nào là một chuỗi hành động cố định: lái xe đến Sibiu, sau đó Fagaras, sau đó Bucharest. Nếu mô hình là đúng, thì một khi tác nhân đã tìm thấy một giải pháp, nó có thể bỏ qua các tri giác của mình trong khi nó đang thực thi các hành động—có thể nói là nhắm mắt lại—bởi vì giải pháp được đảm bảo sẽ dẫn đến mục tiêu. Các nhà lý thuyết điều khiển gọi đây là một **hệ thống vòng hở** (open-loop system): việc bỏ qua các tri giác phá vỡ vòng lặp giữa tác nhân và môi trường. Nếu có khả năng mô hình không chính xác, hoặc môi trường không

xác định, thì tác nhân sẽ an toàn hơn khi sử dụng cách tiếp cận **vòng kín** (closed-loop) theo dõi các tri giác (xem Phần 4.4).

Trong các môi trường có thể quan sát một phần hoặc không xác định, một giải pháp sẽ là một chiến lược phân nhánh đề xuất các hành động tương lai khác nhau tùy thuộc vào tri giác nào đến. Ví dụ, tác nhân có thể lên kế hoạch lái xe từ Arad đến Sibiu nhưng có thể cần một kế hoạch dự phòng trong trường hợp nó đến Zerind một cách tình cờ hoặc tìm thấy một biển báo ghi “Drum lincis” (Đường cấm).

3.1.1 Vấn đề tìm kiếm và giải pháp

Một vấn đề tìm kiếm có thể được định nghĩa một cách chính thức như sau:

- Một tập hợp các trạng thái có thể mà môi trường có thể ở trong. Chúng ta gọi đây là **không gian trạng thái** (state space).
- **Trạng thái ban đầu** (initial state) mà tác nhân bắt đầu. Ví dụ: Arad.
- Một tập hợp một hoặc nhiều **trạng thái mục tiêu** (goal states). Đôi khi có một trạng thái mục tiêu (ví dụ: Bucharest), đôi khi có một tập hợp nhỏ các trạng thái mục tiêu thay thế, và đôi khi mục tiêu được định nghĩa bởi một thuộc tính áp dụng cho nhiều trạng thái (có khả năng là vô hạn). Ví dụ, trong một thế giới máy hút bụi, mục tiêu có thể là không có bụi bẩn ở bất kỳ vị trí nào, bất kể các sự thật khác về trạng thái. Chúng ta có thể tính đến cả ba khả năng này bằng cách chỉ định một phương thức **IS-GOAL** cho một vấn đề. Trong chương này, chúng ta đôi khi sẽ nói “mục tiêu” để đơn giản, nhưng những gì chúng ta nói cũng áp dụng cho “bất kỳ một trong các trạng thái mục tiêu có thể.”
- Các **hành động** (action) có sẵn cho tác nhân. Với một trạng thái s , **ACTIONS(s)** trả về một tập hợp hữu hạn các hành động có thể được thực thi trong s . Chúng ta nói rằng mỗi hành động này là **áp dụng được** (applicable) trong s . Một ví dụ: $\text{Actions}(\text{Arad}) = \{\text{ToSibiu}, \text{ToTimisoara}, \text{ToZerind}\}$.
- Một **mô hình chuyển tiếp** (transition model), mô tả những gì mỗi hành động làm. **RESULT(s, a)** trả về trạng thái là kết quả của việc thực hiện hành động a trong trạng thái s . Ví dụ, $\text{RESULT}(\text{Arad}, \text{ToZerind}) = \text{Zerind}$.
- Một **hàm chi phí hành động** (action cost function), được ký hiệu là **ACTION-COST(s, a, s')** khi chúng ta đang lập trình hoặc $c(s, a, s')$ khi chúng ta đang làm toán, đưa ra chi phí số của việc áp dụng hành động a trong trạng thái s để đạt đến trạng thái s' . Một tác nhân giải quyết vấn đề nên sử dụng một hàm chi phí phản ánh thước đo hiệu suất của chính nó; ví dụ, đối với các tác nhân tìm đường, chi phí của một hành động có thể là chiều dài theo dặm (như trong Hình 3.1), hoặc có thể là thời gian cần thiết để hoàn thành hành động.

Một chuỗi các hành động tạo thành một **đường dẫn** (path), và một **giải pháp** (solution) là một đường dẫn từ trạng thái ban đầu đến một trạng thái mục tiêu. Chúng ta giả định rằng chi phí hành động là cộng gộp; tức là, tổng chi phí của một đường dẫn là tổng của các chi phí hành động riêng lẻ. Một **giải pháp tối ưu** (optimal solution) có chi phí đường dẫn thấp nhất trong số tất cả các giải pháp. Trong chương này, chúng ta giả định rằng tất cả các chi phí hành động sẽ là dương, để tránh một số biến chứng.

Không gian trạng thái có thể được biểu diễn dưới dạng một **đồ thị** (graph) trong đó các đỉnh là trạng thái và các cạnh có hướng giữa chúng là hành động. Bản đồ Romania được hiển thị trong Hình 3.1 là một đồ thị như vậy, trong đó mỗi con đường chỉ ra hai hành động, một trong mỗi hướng.

3.1.2 Xây dựng vấn đề

Việc xây dựng vấn đề đến Bucharest của chúng ta là một **mô hình**—một mô tả toán học trừu tượng—chứ không phải là thứ thật. Hãy so sánh mô tả trạng thái nguyên tử đơn giản **Arad** với một chuyến đi thực tế xuyên quốc gia, trong đó trạng thái của thế giới bao gồm rất nhiều thứ: những người bạn đồng hành, chương trình radio hiện tại, phong cảnh ngoài cửa sổ, sự gần gũi của các nhân viên thực thi pháp luật, khoảng cách đến điểm dừng chân tiếp theo, tình trạng của con đường, thời tiết, giao thông, v.v. Tất cả những cân nhắc này đều bị loại bỏ khỏi mô hình của chúng ta vì chúng không liên quan đến vấn đề tìm một tuyến đường đến Bucharest.

Quá trình loại bỏ chi tiết khỏi một biểu diễn được gọi là **trừu tượng hóa** (abstraction). Một việc xây dựng vấn đề tốt có mức độ chi tiết phù hợp. Nếu các hành động ở mức độ “di chuyển chân phải về phía trước một centimet” hoặc “quay vô lăng sang trái một độ,” tác nhân có lẽ sẽ không bao giờ tìm thấy lối ra khỏi bãi đậu xe, chứ đừng nói đến Bucharest.

Chúng ta có thể chính xác hơn về mức độ trừu tượng thích hợp không? Hãy nghĩ về các trạng thái và hành động trừu tượng mà chúng ta đã chọn tương ứng với các tập hợp lớn các trạng thái thế giới chi tiết và các chuỗi hành động chi tiết. Bây giờ hãy xem xét một giải pháp cho vấn đề trừu tượng: ví dụ, đường dẫn từ Arad đến Sibiu đến Rimnicu Vilcea đến Pitesti đến Bucharest. Giải pháp trừu tượng này tương ứng với một số lượng lớn các đường dẫn chi tiết hơn. Ví dụ, chúng ta có thể lái xe với radio bật giữa Sibiu và Rimnicu Vilcea, và sau đó tắt nó trong phần còn lại của chuyến đi.

Sự trừu tượng hóa là hợp lệ nếu chúng ta có thể làm rõ bất kỳ giải pháp trừu tượng nào thành một giải pháp trong thế giới chi tiết hơn; một điều kiện đủ là đối với mỗi trạng thái chi tiết “ở Arad,” có một đường dẫn chi tiết đến một trạng thái nào đó “ở Sibiu,” v.v. Sự trừu tượng hóa là hữu ích nếu việc thực hiện mỗi hành động trong giải pháp dễ dàng hơn vấn đề ban đầu; trong trường hợp của chúng ta, hành động “lái xe từ Arad đến Sibiu” có thể được thực hiện mà không cần tìm kiếm hoặc lập kế hoạch thêm bởi một tài xế có kỹ năng trung bình. Do đó, việc lựa chọn một sự trừu tượng hóa tốt liên quan đến việc loại bỏ càng nhiều chi tiết càng tốt trong khi vẫn giữ được tính hợp lệ và đảm bảo rằng các hành động trừu tượng dễ thực hiện. Nếu không có khả năng xây dựng các sự trừu tượng hóa hữu ích, các tác nhân thông minh sẽ hoàn toàn bị choáng ngợp bởi thế giới thực.

3.2 Ví dụ về các vấn đề

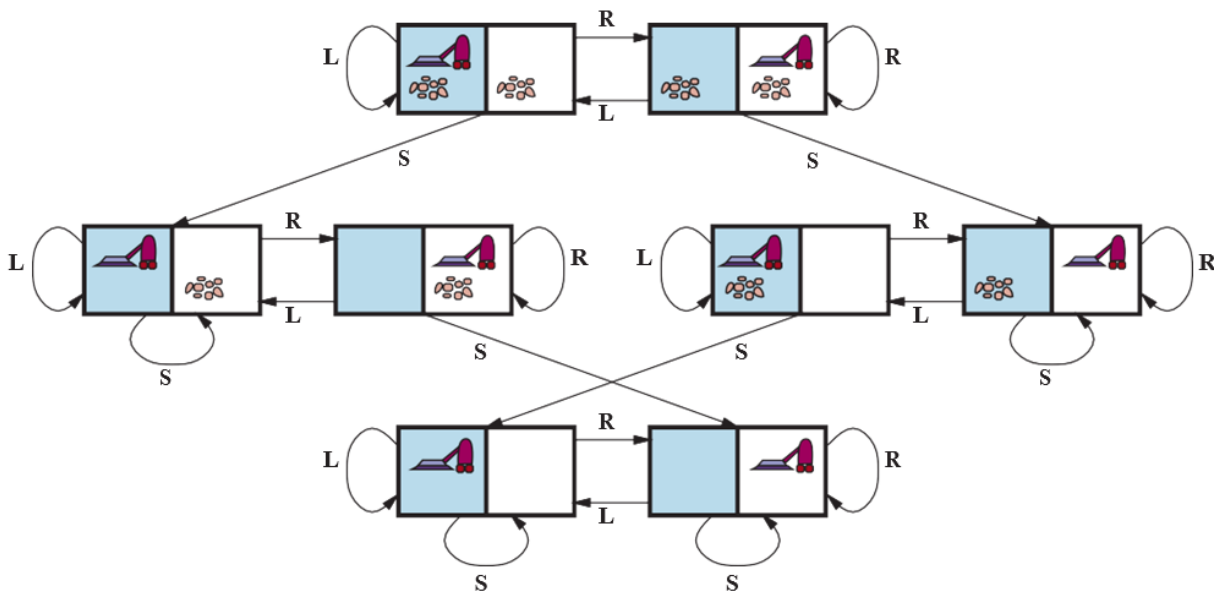
Cách tiếp cận giải quyết vấn đề đã được áp dụng cho một loạt các môi trường tác vụ. Chúng ta liệt kê một số vấn đề nổi tiếng nhất ở đây, phân biệt giữa các vấn đề **tiêu chuẩn hóa** (standardized) và **thế giới thực** (real-world). Một vấn đề tiêu chuẩn hóa nhằm minh họa hoặc rèn luyện các phương pháp giải quyết vấn đề khác nhau. Nó có thể được đưa ra một mô tả chính xác, ngắn gọn và do đó phù hợp làm chuẩn mực để các nhà nghiên cứu so sánh hiệu suất của các thuật toán. Một vấn đề thế giới thực, chẳng hạn như điều hướng robot, là vấn đề mà các giải pháp của nó được con người

thực sự sử dụng và việc xây dựng nó là đặc trưng, không tiêu chuẩn, vì, ví dụ, mỗi robot có các cảm biến khác nhau tạo ra dữ liệu khác nhau.

3.2.1 Vấn đề tiêu chuẩn hóa

Một **vấn đề thế giới lưới** (grid world problem) là một mảng hình chữ nhật hai chiều gồm các ô vuông trong đó các tác nhân có thể di chuyển từ ô này sang ô khác. Thông thường, tác nhân có thể di chuyển đến bất kỳ ô liền kề nào không có chướng ngại vật—theo chiều ngang hoặc chiều dọc và trong một số vấn đề là theo đường chéo. Các ô có thể chứa các đối tượng, mà tác nhân có thể nhặt, đẩy hoặc tác động lên; một bức tường hoặc chướng ngại vật không thể vượt qua khác trong một ô ngăn cản tác nhân di chuyển vào ô đó. Thế giới máy hút bụi từ Phần 2.1 có thể được xây dựng như một vấn đề thế giới lưới như sau:

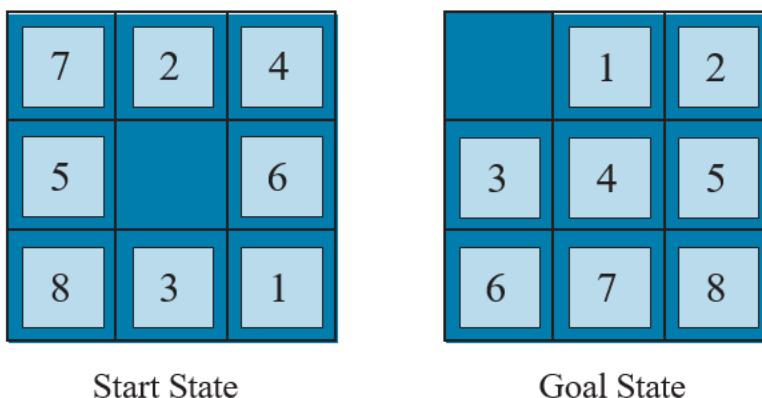
- **Trạng thái:** Một trạng thái của thế giới cho biết những đối tượng nào ở trong những ô nào. Đối với thế giới máy hút bụi, các đối tượng là tác nhân và bất kỳ bụi bẩn nào. Trong phiên bản hai ô đơn giản, tác nhân có thể ở trong một trong hai ô và mỗi ô có thể chứa bụi bẩn hoặc không, vì vậy có $2 \cdot 2 \cdot 2 = 8$ trạng thái (xem Hình 3.2). Nói chung, một môi trường máy hút bụi có n ô có $n \cdot 2^n$ trạng thái.
- **Trạng thái ban đầu:** Bất kỳ trạng thái nào cũng có thể được chỉ định là trạng thái ban đầu.
- **Hành động:** Trong thế giới hai ô, chúng ta đã định nghĩa ba hành động: Hút, di chuyển Trái và di chuyển Phải. Trong một thế giới nhiều ô hai chiều, chúng ta cần nhiều hành động di chuyển hơn. Chúng ta có thể thêm Lên trên và Xuống dưới, cho chúng ta bốn hành động di chuyển tuyệt đối, hoặc chúng ta có thể chuyển sang các hành động **tự tâm** (egocentric), được định nghĩa tương đối với góc nhìn của tác nhân—ví dụ, Tiến, Lùi, Về Phải và Về Trái.



Hình 3.2 Đồ thị không gian trạng thái cho thể giới máy hút bụi hai ô. Có 8 trạng thái và ba hành động cho mỗi trạng thái: L = Trái, R = Phải, S = Hút.

- **Mô hình chuyển tiếp:** Hút loại bỏ bất kỳ bụi bẩn nào khỏi ô của tác nhân; Tiến di chuyển tác nhân về phía trước một ô theo hướng nó đang đối diện, trừ khi nó va vào tường, trong trường hợp đó hành động không có tác dụng. Lùi di chuyển tác nhân theo hướng ngược lại, trong khi Rẽ Phải và Rẽ Trái thay đổi hướng nó đang đối diện 90° .
- **Trạng thái mục tiêu:** Các trạng thái mà trong đó mọi ô đều sạch.
- **Chi phí hành động:** Mỗi hành động có chi phí là 1.

Một loại thể giới lưới khác là **câu đố sokoban** (sokoban puzzle), trong đó mục tiêu của tác nhân là đẩy một số hộp, rải rác trên lưới, đến các vị trí lưu trữ được chỉ định. Có thể có tối đa một hộp trên mỗi ô. Khi một tác nhân di chuyển về phía trước vào một ô có chứa một hộp và có một ô trống ở phía bên kia của hộp, thì cả hộp và tác nhân đều di chuyển về phía trước. Tác nhân không thể đẩy một hộp vào một hộp khác hoặc một bức tường. Đối với một thể giới có n ô không chướng ngại vật và b hộp, có $n \times n! / (b! (n - b)!)$ trạng thái; ví dụ trên một lưới 8×8 với một tá hộp, có hơn 200 nghìn tỷ trạng thái.



Hình 3.3 Một ví dụ điển hình của câu đố 8-puzzle.

Trong một **câu đố xếp gạch trượt** (sliding-tile puzzle), một số gạch (đôi khi được gọi là khối hoặc mảnh) được sắp xếp trong một lưới với một hoặc nhiều khoảng trống để một số gạch có thể trượt vào khoảng trống. Một biến thể là câu đố **Rush Hour**, trong đó ô tô và xe tải trượt xung quanh một lưới 6×6 để cố gắng giải phóng một chiếc ô tô khỏi ùn tắc giao thông. Có lẽ biến thể nổi tiếng nhất là **8-puzzle** (xem Hình 3.3), bao gồm một lưới 3×3 với tám gạch được đánh số và một khoảng trống, và **15-puzzle** trên một lưới 4×4 . Mục tiêu là đạt đến một trạng thái mục tiêu được chỉ định, chẳng hạn như trạng thái được hiển thị ở bên phải của hình. Việc xây dựng tiêu chuẩn của câu đố 8-puzzle như sau:

- **Trạng thái:** Một mô tả trạng thái chỉ định vị trí của mỗi gạch.

- **Trạng thái ban đầu:** Bất kỳ trạng thái nào cũng có thể được chỉ định là trạng thái ban đầu. Lưu ý rằng một thuộc tính chắc lẻ phân chia không gian trạng thái—bất kỳ mục tiêu nào cũng có thể được đạt từ chính xác một nửa số trạng thái ban đầu có thể có.
- **Hành động:** Trong khi trong thế giới vật lý, một gạch là thứ trượt, cách đơn giản nhất để mô tả một hành động là nghĩ về khoảng trống di chuyển sang Trái, Phải, Lên hoặc Xuống. Nếu khoảng trống ở một cạnh hoặc góc thì không phải tất cả các hành động sẽ được áp dụng.
- **Mô hình chuyển tiếp:** Ánh xạ một trạng thái và hành động đến một trạng thái kết quả; ví dụ, nếu chúng ta áp dụng Trái cho trạng thái bắt đầu trong Hình 3.3, trạng thái kết quả sẽ có gạch 5 và khoảng trống hoán đổi.
- **Trạng thái mục tiêu:** Mặc dù bất kỳ trạng thái nào cũng có thể là mục tiêu, chúng ta thường chỉ định một trạng thái với các số theo thứ tự, như trong Hình 3.3.
- **Chi phí hành động:** Mỗi hành động có chi phí là 1.

Lưu ý rằng mọi việc xây dựng vấn đề đều liên quan đến các sự trừu tượng hóa. Các hành động của 8-puzzle được trừu tượng hóa thành các trạng thái bắt đầu và cuối của chúng, bỏ qua các vị trí trung gian nơi gạch đang trượt. Chúng ta đã trừu tượng hóa các hành động như lắc bảng khi gạch bị kẹt và loại trừ việc lấy gạch ra bằng dao và đặt chúng lại. Chúng ta còn lại với một mô tả về các quy tắc, tránh tất cả các chi tiết về thao tác vật lý.

Vấn đề tiêu chuẩn hóa cuối cùng của chúng ta được Donald Knuth (1964) nghĩ ra và minh họa cách không gian trạng thái vô hạn có thể phát sinh. Knuth phỏng đoán rằng bắt đầu với số 4, một chuỗi các phép toán căn bậc hai, làm tròn xuống và giai thừa có thể đạt được bất kỳ số nguyên dương mong muốn nào. Ví dụ, chúng ta có thể đạt được 5 từ 4 như sau:

$$\left\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)}}}}} \right\rfloor = 5.$$

Định nghĩa vấn đề rất đơn giản:

- **Trạng thái:** Số thực dương.
- **Trạng thái ban đầu:** 4.
- **Hành động:** Áp dụng phép toán căn bậc hai, làm tròn xuống hoặc giai thừa (giai thừa chỉ dành cho số nguyên).
- **Mô hình chuyển tiếp:** Như được đưa ra bởi các định nghĩa toán học của các phép toán.
- **Trạng thái mục tiêu:** Số nguyên dương mong muốn.
- **Chi phí hành động:** Mỗi hành động có chi phí là 1.

Không gian trạng thái cho vấn đề này là vô hạn: đối với bất kỳ số nguyên nào lớn hơn 2, toán tử giai thừa sẽ luôn tạo ra một số nguyên lớn hơn. Vấn đề này thú vị vì nó khám phá các số rất lớn: đường dẫn ngắn nhất đến 5 đi qua $\left\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)}}}}} \right\rfloor = 620.448.401.733.239.439.360.000$. Không gian trạng thái vô hạn thường xuyên phát sinh trong các tác vụ liên quan đến việc tạo ra các biểu thức toán học, mạch điện, bằng chứng, chương trình và các đối tượng được định nghĩa đệ quy khác.

3.2.2 Vấn đề thế giới thực

Chúng ta đã thấy cách vấn đề tìm đường được định nghĩa theo các vị trí được chỉ định và các chuyên tiếp dọc theo các cạnh giữa chúng. Các thuật toán tìm đường được sử dụng trong nhiều ứng dụng khác nhau. Một số, chẳng hạn như các trang web và hệ thống trong ô tô cung cấp chỉ đường lái xe, là các phần mở rộng tương đối đơn giản của ví dụ Romania. (Các biến chứng chính là chi phí thay đổi do sự chậm trễ phụ thuộc vào giao thông và việc định tuyến lại do đường bị đóng.) Những vấn đề khác, chẳng hạn như định tuyến luồng video trong mạng máy tính, lập kế hoạch hoạt động quân sự và hệ thống lập kế hoạch du lịch hàng không, liên quan đến các đặc tả phức tạp hơn nhiều. Hãy xem xét các vấn đề du lịch hàng không phải được giải quyết bởi một trang web lập kế hoạch du lịch:

- **Trạng thái:** Mỗi trạng thái rõ ràng bao gồm một vị trí (ví dụ: một sân bay) và thời gian hiện tại. Hơn nữa, vì chi phí của một hành động (một phân đoạn chuyến bay) có thể phụ thuộc vào các phân đoạn trước đó, cơ sở giá vé của chúng và tình trạng của chúng là nội địa hay quốc tế, trạng thái phải ghi lại thông tin bổ sung về các khía cạnh “lịch sử” này.
- **Trạng thái ban đầu:** Sân bay nhà của người dùng.
- **Hành động:** Thực hiện bất kỳ chuyến bay nào từ vị trí hiện tại, ở bất kỳ hạng ghế nào, khởi hành sau thời gian hiện tại, để lại đủ thời gian cho việc trung chuyển trong sân bay nếu cần.
- **Mô hình chuyển tiếp:** Trạng thái kết quả từ việc thực hiện một chuyến bay sẽ có điểm đến của chuyến bay làm vị trí mới và thời gian đến của chuyến bay làm thời gian mới.
- **Trạng thái mục tiêu:** Một thành phố đích. Đôi khi mục tiêu có thể phức tạp hơn, chẳng hạn như “đến đích trên một chuyến bay thẳng.”
- **Chi phí hành động:** Một sự kết hợp của chi phí tiền tệ, thời gian chờ đợi, thời gian bay, thủ tục hải quan và nhập cảnh, chất lượng ghế, thời gian trong ngày, loại máy bay, điểm thưởng của người thường xuyên bay, v.v.

Các hệ thống tư vấn du lịch thương mại sử dụng một việc xây dựng vấn đề loại này, với nhiều biến chứng bổ sung để xử lý cấu trúc giá vé phức tạp của các hãng hàng không. Tuy nhiên, bất kỳ du khách dày dạn nào cũng biết rằng không phải tất cả các chuyến du lịch hàng không đều diễn ra theo kế hoạch. Một hệ thống thực sự tốt nên bao gồm các kế hoạch dự phòng—điều gì sẽ xảy ra nếu chuyến bay này bị trì hoãn và lỡ chuyến bay nối chuyến?

Các **vấn đề du lịch** (touring problem) mô tả một tập hợp các địa điểm phải được ghé thăm, thay vì một điểm đến mục tiêu duy nhất. **Vấn đề người bán hàng rong** (traveling salesperson problem - TSP) là một vấn đề du lịch trong đó mọi thành phố trên bản đồ phải được ghé thăm. Mục đích là tìm một chuyến đi với chi phí $< C$ (hoặc trong phiên bản tối ưu hóa, để tìm một chuyến đi với chi phí thấp nhất có thể). Một lượng lớn nỗ lực đã được dành ra để cải thiện khả năng của các thuật toán TSP. Các thuật toán cũng có thể được mở rộng để xử lý các đội xe. Ví dụ, một thuật toán tìm kiếm và tối ưu hóa để định tuyến xe buýt trường học ở Boston đã tiết kiệm được 5 triệu đô la, cắt giảm giao thông và ô nhiễm không khí, và tiết kiệm thời gian cho tài xế và học sinh (Bertsimas et al., 2019). Ngoài việc lập kế hoạch các chuyến đi, các thuật toán tìm kiếm đã được sử dụng cho các tác vụ như lập kế hoạch di chuyển của máy khoan bảng mạch tự động và của máy xếp hàng trên sàn cửa hàng.

Một **vấn đề bố trí VLSI** (VLSI layout) yêu cầu định vị hàng triệu thành phần và kết nối trên một con chip để giảm thiểu diện tích, giảm thiểu độ trễ mạch, giảm thiểu điện dung lạc và tối đa hóa

năng suất sản xuất. Vấn đề bố trí xuất hiện sau giai đoạn thiết kế logic và thường được chia thành hai phần: bố trí ô và định tuyến kênh. Trong bố trí ô, các thành phần nguyên thủy của mạch được nhóm lại thành các ô, mỗi ô thực hiện một chức năng được công nhận. Mỗi ô có một dấu chân cố định (kích thước và hình dạng) và yêu cầu một số kết nối nhất định với mỗi ô khác. Mục đích là đặt các ô trên chip sao cho chúng không chồng chéo và có chỗ cho các dây kết nối được đặt giữa các ô. Định tuyến kênh tìm một tuyến đường cụ thể cho mỗi dây thông qua các khoảng trống giữa các ô. Các vấn đề tìm kiếm này cực kỳ phức tạp, nhưng chắc chắn đáng để giải quyết.

Điều hướng robot (Robot navigation) là một sự tổng quát hóa của vấn đề tìm đường đã mô tả trước đó. Thay vì đi theo các đường dẫn riêng biệt (chẳng hạn như các con đường ở Romania), một robot có thể đi lang thang, trên thực tế tạo ra các đường dẫn của riêng nó. Đối với một robot hình tròn di chuyển trên một bề mặt phẳng, không gian về cơ bản là hai chiều. Khi robot có cánh tay và chân cũng phải được điều khiển, không gian tìm kiếm trở nên đa chiều—một chiều cho mỗi góc khớp. Các kỹ thuật tiên tiến được yêu cầu chỉ để làm cho không gian tìm kiếm về cơ bản là liên tục trở nên hữu hạn (xem Chương 26). Ngoài sự phức tạp của vấn đề, robot thực tế cũng phải đối phó với các lỗi trong các số đọc cảm biến và điều khiển động cơ của chúng, với khả năng quan sát một phần và với các tác nhân khác có thể làm thay đổi môi trường.

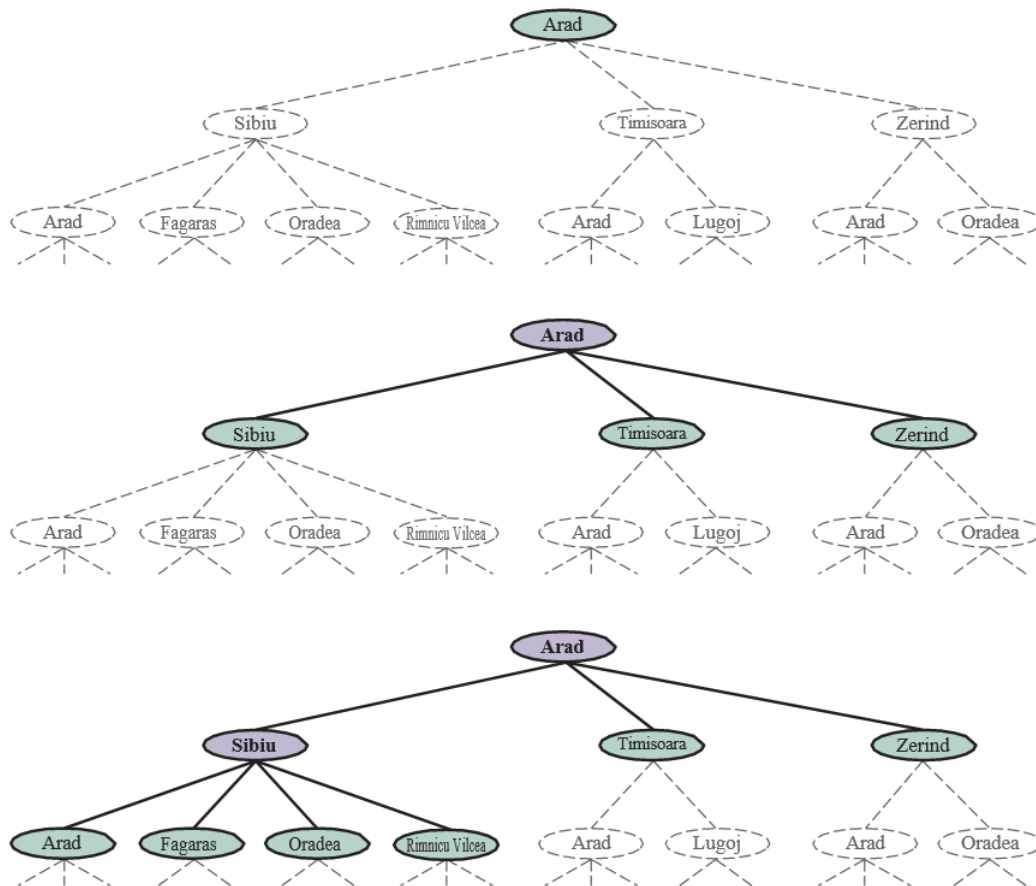
Trình tự lắp ráp tự động (Automatic assembly sequencing) của các đối tượng phức tạp (chẳng hạn như động cơ điện) bởi một robot đã là thực hành công nghiệp tiêu chuẩn từ những năm 1970. Các thuật toán đầu tiên tìm một chuỗi lắp ráp khả thi và sau đó làm việc để tối ưu hóa quy trình. Giảm thiểu lượng lao động thủ công của con người trên dây chuyền lắp ráp có thể tạo ra khoản tiết kiệm đáng kể về thời gian và chi phí. Trong các vấn đề lắp ráp, mục đích là tìm một thứ tự để lắp ráp các bộ phận của một đối tượng. Nếu chọn sai thứ tự, sẽ không có cách nào để thêm một số bộ phận sau này trong chuỗi mà không làm mất một số công việc đã được thực hiện. Kiểm tra một hành động trong chuỗi để xem tính khả thi là một vấn đề tìm kiếm hình học khó khăn liên quan chặt chẽ đến điều hướng robot. Do đó, việc tạo ra các hành động hợp lệ là phần tốn kém của trình tự lắp ráp. Bất kỳ thuật toán thực tế nào cũng phải tránh khám phá tất cả trừ một phần nhỏ của không gian trạng thái. Một vấn đề lắp ráp quan trọng là **thiết kế protein** (protein design), trong đó mục tiêu là tìm một chuỗi axit amin sẽ gấp lại thành một protein ba chiều với các thuộc tính phù hợp để chữa một số bệnh.

3.3 Thuật toán tìm kiếm

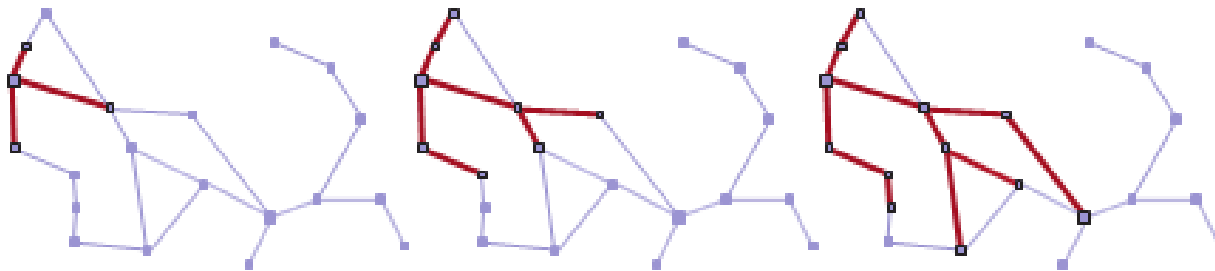
Một **thuật toán tìm kiếm** (search algorithm) lấy một bài toán tìm kiếm làm đầu vào và trả về một giải pháp hoặc một chỉ báo về sự thất bại. Trong chương này, chúng ta xem xét các thuật toán chèn một **cây tìm kiếm** lên đồ thị không gian trạng thái, tạo thành các đường dẫn khác nhau từ trạng thái ban đầu, cố gắng tìm một đường dẫn đạt đến trạng thái mục tiêu. Mỗi **nút** (node) trong cây tìm kiếm tương ứng với một trạng thái trong không gian trạng thái và các cạnh trong cây tìm kiếm tương ứng với các hành động. Nút gốc của cây tương ứng với trạng thái ban đầu của bài toán.

Điều quan trọng là phải hiểu sự khác biệt giữa không gian trạng thái và cây tìm kiếm. Không gian trạng thái mô tả tập hợp các trạng thái (có thể là vô hạn) trong thế giới và các hành động cho phép chuyển đổi từ trạng thái này sang trạng thái khác. Cây tìm kiếm mô tả các đường dẫn giữa các trạng thái này, tiếp cận mục tiêu. Cây tìm kiếm có thể có nhiều đường dẫn đến (và do đó nhiều nút cho) bất kỳ trạng thái nào, nhưng mỗi nút trong cây có một đường dẫn duy nhất trở lại nút gốc (như trong tất cả các cây).

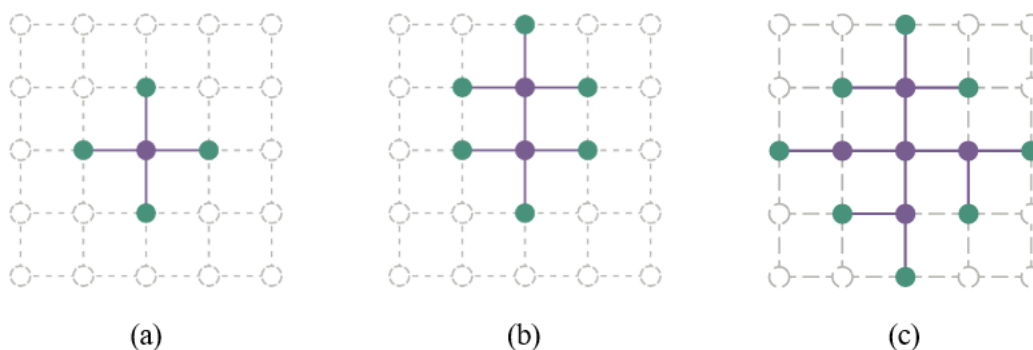
Hình 3.4 cho thấy một vài bước đầu tiên trong việc tìm đường dẫn từ Arad đến Bucharest. Nút gốc của cây tìm kiếm ở trạng thái ban đầu, Arad. Chúng ta có thể **mở rộng** (expand) nút, bằng cách xem xét các **hành động** có sẵn cho trạng thái đó, sử dụng hàm **RESULT** để xem các hành động đó dẫn đến đâu, và tạo một nút mới (được gọi là **nút con** (child node) hoặc **nút kế tiếp** (successor node)) cho mỗi trạng thái kết quả. Mỗi nút con có Arad làm **nút cha** (parent node) của nó.



Hình 3.4 Ba cây tìm kiếm một phần để tìm tuyến đường từ Arad đến Bucharest. Các nút đã được mở rộng có màu tím nhạt với chữ đậm; các nút trên **biên** (frontier) đã được tạo nhưng chưa được mở rộng có màu xanh lá cây; tập hợp các trạng thái tương ứng với hai loại nút này được cho là đã được **đạt đến** (reached). Các nút có thể được tạo tiếp theo được hiển thị bằng các đường đứt nét mờ. Lưu ý trong cây dưới cùng có một chu kỳ từ Arad đến Sibiu đến Arad; đó không thể là một đường dẫn tối ưu, vì vậy tìm kiếm không nên tiếp tục từ đó.



Hình 3.5 Một chuỗi các cây tìm kiếm được tạo bởi một tìm kiếm đồ thị trên bài toán Romania của Hình 3.1. Ở mỗi giai đoạn, chúng ta đã mở rộng mọi nút trên biên, mở rộng mọi đường dẫn với tất cả các hành động áp dụng được mà không dẫn đến một trạng thái đã được đạt đến. Lưu ý rằng ở giai đoạn thứ ba, thành phố trên cùng (Oradea) có hai nút kế tiếp, cả hai đều đã được đạt đến bằng các đường dẫn khác, vì vậy không có đường dẫn nào được mở rộng từ Oradea.



Hình 3.6 Thuộc tính tách biệt của tìm kiếm đồ thị, được minh họa trên một vấn đề lưới hình chữ nhật. **Biên** (màu xanh lá cây) tách biệt **bên trong** (màu tím nhạt) khỏi **bên ngoài** (đường đứt nét mờ). Biên là tập hợp các nút (và các trạng thái tương ứng) đã được đạt đến nhưng chưa được mở rộng; bên trong là tập hợp các nút (và các trạng thái tương ứng) đã được mở rộng; và bên ngoài là tập hợp các trạng thái chưa được đạt đến. Trong (a), chỉ nút gốc đã được mở rộng. Trong (b), nút biên trên cùng được mở rộng. Trong (c), các nút kế tiếp còn lại của nút gốc được mở rộng theo thứ tự kim đồng hồ.

Bây giờ chúng ta phải chọn nút nào từ biên để mở rộng tiếp theo. Đây là bản chất của tìm kiếm—theo đuổi một tùy chọn bây giờ và gác lại những tùy chọn khác để sau này. Giả sử chúng ta chọn mở rộng Sibiu trước. Hình 3.4 (dưới cùng) cho thấy kết quả: một tập hợp 6 nút chưa được mở rộng (được phác thảo bằng chữ đậm). Chúng ta gọi đây là **biên** (frontier) của cây tìm kiếm. Chúng ta nói rằng bất kỳ trạng thái nào đã có một nút được tạo cho nó đã được **đạt đến** (reached) (cho dù nút đó đã được mở rộng hay chưa). Hình 3.5 cho thấy cây tìm kiếm được chòng lên đồ thị không gian trạng thái.

Lưu ý rằng biên tách biệt hai vùng của đồ thị không gian trạng thái: một vùng **bên trong** (interior) nơi mọi trạng thái đã được mở rộng, và một vùng **bên ngoài** (exterior) gồm các trạng thái chưa được đạt đến. Thuộc tính này được minh họa trong Hình 3.6.

3.3.1 Tìm kiếm tốt nhất đầu tiên

Làm thế nào chúng ta quyết định nút nào từ biên để mở rộng tiếp theo? Một cách tiếp cận rất chung được gọi là **tìm kiếm tốt nhất đầu tiên** (best-first search), trong đó chúng ta chọn một nút, n , với giá trị tối thiểu của một số **hàm đánh giá** (evaluation function), $f(n)$. Hình 3.7 cho thấy thuật toán. Ở mỗi lần lặp, chúng ta chọn một nút trên biên với giá trị $f(n)$ tối thiểu, trả về nó nếu trạng thái của nó là trạng thái mục tiêu và ngược lại áp dụng **EXPAND** để tạo các nút con. Mỗi nút con được thêm vào biên nếu nó chưa được đạt đến trước đây, hoặc được thêm lại nếu nó hiện đang được đạt đến với một đường dẫn có chi phí đường dẫn thấp hơn bất kỳ đường dẫn trước đó nào. Thuật toán trả về một chỉ báo về sự thất bại, hoặc một nút đại diện cho một đường dẫn đến một mục tiêu. Bằng cách sử dụng các hàm $f(n)$ khác nhau, chúng ta có được các thuật toán cụ thể khác nhau, mà chương này sẽ đề cập.

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.State
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure

function EXPAND(problem, node) yields nodes
  s ← node.State
  for each action in problem.Actions(s) do
    s' ← problem.RESULT(s, action)
    cost ← node.Path-Cost + problem.Action-Cost(s, action, s')
    yield Node(State=s', Parent=node, Action=action, Path-Cost=cost)
```

Hình 3.7 Thuật toán tìm kiếm tốt nhất đầu tiên, và hàm để mở rộng một nút. Các cấu trúc dữ liệu được sử dụng ở đây được mô tả trong Phần 3.3.2. Xem Phụ lục B để biết yield.

3.3.2 Cấu trúc dữ liệu tìm kiếm

Các thuật toán tìm kiếm yêu cầu một cấu trúc dữ liệu để theo dõi cây tìm kiếm. Một **nút** trong cây được biểu diễn bởi một cấu trúc dữ liệu với bốn thành phần:

- **node.STATE**: trạng thái mà nút tương ứng;
- **node.PARENT**: nút trong cây đã tạo ra nút này;
- **node.ACTION**: hành động đã được áp dụng cho trạng thái của nút cha để tạo ra nút này;

- **node.PATH-COST**: tổng chi phí của đường dẫn từ trạng thái ban đầu đến nút này. Trong các công thức toán học, chúng ta sử dụng $g(node)$ như một từ đồng nghĩa với **PATH-COST**.

Theo các con trỏ **PARENT** quay ngược từ một nút cho phép chúng ta phục hồi các trạng thái và hành động dọc theo đường dẫn đến nút đó. Thực hiện điều này từ một nút mục tiêu sẽ cho chúng ta giải pháp.

Chúng ta cần một cấu trúc dữ liệu để lưu trữ biên. Lựa chọn thích hợp là một loại hàng đợi nào đó, bởi vì các hoạt động trên biên là:

- **IS-EMPTY(frontier)** trả về đúng chỉ khi không có nút nào trong biên.
- **POP(frontier)** loại bỏ nút trên cùng khỏi biên và trả về nó.
- **TOP(frontier)** trả về (nhưng không loại bỏ) nút trên cùng của biên.
- **ADD(node, frontier)** chèn nút vào vị trí thích hợp của nó trong hàng đợi.

Ba loại hàng đợi được sử dụng trong các thuật toán tìm kiếm:

- Một **hàng đợi ưu tiên** (priority queue) đầu tiên sẽ loại bỏ nút có chi phí tối thiểu theo một số hàm đánh giá, f . Nó được sử dụng trong tìm kiếm tốt nhất đầu tiên.
- Một **hàng đợi FIFO** (first-in-first-out) đầu tiên sẽ loại bỏ nút đã được thêm vào hàng đợi đầu tiên; chúng ta sẽ thấy nó được sử dụng trong tìm kiếm theo chiều rộng.
- Một **hàng đợi LIFO** (last-in-first-out) (còn được gọi là một **ngăn xếp** (stack)) đầu tiên sẽ loại bỏ nút được thêm vào gần đây nhất; chúng ta sẽ thấy nó được sử dụng trong tìm kiếm theo chiều sâu.

Các trạng thái đã được đạt đến có thể được lưu trữ dưới dạng một bảng tra cứu (ví dụ: một bảng băm) trong đó mỗi khóa là một trạng thái và mỗi giá trị là nút cho trạng thái đó.

3.3.3 Các đường dẫn dư thừa

Cây tìm kiếm được hiển thị trong Hình 3.4 (dưới cùng) bao gồm một đường dẫn từ Arad đến Sibiu và quay trở lại Arad. Chúng ta nói rằng Arad là một **trạng thái lặp lại** (repeated state) trong cây tìm kiếm, được tạo ra trong trường hợp này bởi một **chu kỳ** (cycle) (còn được gọi là **đường dẫn lặp** (loopy path)). Vì vậy, mặc dù không gian trạng thái chỉ có 20 trạng thái, cây tìm kiếm hoàn chỉnh là vô hạn vì không có giới hạn về số lần người ta có thể đi qua một vòng lặp.

Một chu kỳ là một trường hợp đặc biệt của một **đường dẫn dư thừa** (redundant path). Ví dụ, chúng ta có thể đến Sibiu qua đường dẫn Arad–Sibiu (dài 140 dặm) hoặc đường dẫn Arad–Zerind–Oradea–Sibiu (dài 297 dặm). Đường dẫn thứ hai này là dư thừa—nó chỉ là một cách tồi tệ hơn để đến cùng một trạng thái—và không cần phải được xem xét trong hành trình tìm kiếm các đường dẫn tối ưu của chúng ta.

Hãy xem xét một tác nhân trong một thế giới lưới 10×10 , với khả năng di chuyển đến bất kỳ ô nào trong số 8 ô liền kề. Nếu không có chướng ngại vật, tác nhân có thể đến bất kỳ ô nào trong số 100 ô trong 9 lần di chuyển hoặc ít hơn. Nhưng số lượng đường dẫn có độ dài 9 gần bằng 8^9 (ít hơn một chút vì các cạnh của lưới), hoặc hơn 100 triệu. Nói cách khác, ô trung bình có thể được đạt đến bởi hơn một triệu đường dẫn dư thừa có độ dài 9, và nếu chúng ta loại bỏ các đường dẫn

dư thừa, chúng ta có thể hoàn thành một tìm kiếm nhanh hơn khoảng một triệu lần. Như người ta thường nói, các thuật toán không thể nhớ quá khứ thì sẽ lặp lại nó. Có ba cách tiếp cận cho vấn đề này.

Đầu tiên, chúng ta có thể nhớ tất cả các trạng thái đã đạt đến trước đó (như tìm kiếm tốt nhất đầu tiên làm), cho phép chúng ta phát hiện tất cả các đường dẫn dư thừa và chỉ giữ đường dẫn tốt nhất đến mỗi trạng thái. Điều này phù hợp với các không gian trạng thái có nhiều đường dẫn dư thừa, và là lựa chọn ưu tiên khi bảng các trạng thái đã đạt đến có thể vừa trong bộ nhớ.

Thứ hai, chúng ta có thể không lo lắng về việc lặp lại quá khứ. Có một số việc xây dựng vấn đề mà hiếm khi hoặc không thể có hai đường dẫn đạt đến cùng một trạng thái. Một ví dụ sẽ là một vấn đề lắp ráp trong đó mỗi hành động thêm một bộ phận vào một cụm lắp ráp đang phát triển, và có một thứ tự các bộ phận sao cho có thể thêm A rồi B, nhưng không thể B rồi A. Đối với những vấn đề đó, chúng ta có thể tiết kiệm không gian bộ nhớ nếu chúng ta không theo dõi các trạng thái đã đạt đến và chúng ta không kiểm tra các đường dẫn dư thừa. Chúng ta gọi một thuật toán tìm kiếm là **tìm kiếm đồ thị** (graph search) nếu nó kiểm tra các đường dẫn dư thừa và là **tìm kiếm giống cây** (tree-like search) nếu nó không kiểm tra. Thuật toán **BEST-FIRST-SEARCH** trong Hình 3.7 là một thuật toán tìm kiếm đồ thị; nếu chúng ta loại bỏ tất cả các tham chiếu đến **đã đạt đến**, chúng ta sẽ có một tìm kiếm giống cây sử dụng ít bộ nhớ hơn nhưng sẽ kiểm tra các đường dẫn dư thừa đến cùng một trạng thái, và do đó sẽ chạy chậm hơn.

Thứ ba, chúng ta có thể thỏa hiệp và kiểm tra các chu kỳ, nhưng không kiểm tra các đường dẫn dư thừa nói chung. Vì mỗi nút có một chuỗi con trở cha, chúng ta có thể kiểm tra các chu kỳ mà không cần thêm bộ nhớ bằng cách theo dõi chuỗi cha để xem trạng thái ở cuối đường dẫn có xuất hiện sớm hơn trong đường dẫn hay không. Một số triển khai theo dõi chuỗi này suốt cả đường đi, và do đó loại bỏ tất cả các chu kỳ; các triển khai khác chỉ theo dõi một vài liên kết (ví dụ: đến cha, ông bà, và cụ), và do đó chỉ mất một lượng thời gian không đổi, trong khi loại bỏ tất cả các chu kỳ ngắn (và dựa vào các cơ chế khác để xử lý các chu kỳ dài).

3.3.4 Đo lường hiệu suất giải quyết vấn đề

Trước khi chúng ta đi vào thiết kế của các thuật toán tìm kiếm khác nhau, chúng ta sẽ xem xét các tiêu chí được sử dụng để lựa chọn giữa chúng. Chúng ta có thể đánh giá hiệu suất của một thuật toán theo bốn cách:

- **Tính đầy đủ** (Completeness): Thuật toán có được đảm bảo tìm thấy một giải pháp khi có một giải pháp không, và báo cáo thất bại một cách chính xác khi không có không?
- **Tính tối ưu chi phí** (Cost optimality): Nó có tìm thấy một giải pháp với chi phí đường dẫn thấp nhất trong tất cả các giải pháp không?
- **Độ phức tạp thời gian** (Time complexity): Cần bao lâu để tìm thấy một giải pháp? Điều này có thể được đo bằng giây, hoặc trừu tượng hơn bằng số lượng trạng thái và hành động được xem xét.
- **Độ phức tạp không gian** (Space complexity): Cần bao nhiêu bộ nhớ để thực hiện tìm kiếm?

Để hiểu tính đầy đủ, hãy xem xét một vấn đề tìm kiếm với một mục tiêu duy nhất. Mục tiêu đó có thể ở bất cứ đâu trong không gian trạng thái; do đó, một thuật toán đầy đủ phải có khả năng khám phá một cách có hệ thống mọi trạng thái có thể đạt được từ trạng thái ban đầu. Trong các không gian trạng thái hữu hạn, điều đó rất dễ đạt được: miễn là chúng ta theo dõi các đường dẫn và cắt bỏ những đường dẫn là chu kỳ (ví dụ: Arad đến Sibiu đến Arad), cuối cùng chúng ta sẽ đạt đến mọi trạng thái có thể đạt được.

Trong các không gian trạng thái vô hạn, cần cẩn thận hơn. Ví dụ, một thuật toán liên tục áp dụng toán tử “giai thừa” trong bài toán “4” của Knuth sẽ đi theo một đường dẫn vô hạn từ 4 đến $4!$ đến $(4!)!$, và cứ thế. Tương tự, trên một lưới vô hạn không có chướng ngại vật, việc liên tục di chuyển về phía trước theo một đường thẳng cũng đi theo một đường dẫn vô hạn của các trạng thái mới. Trong cả hai trường hợp, thuật toán không bao giờ quay trở lại một trạng thái nó đã đạt được trước đó, nhưng nó không đầy đủ vì các không gian rộng lớn của không gian trạng thái không bao giờ được đạt đến.

Để đầy đủ, một thuật toán tìm kiếm phải **có hệ thống** (systematic) trong cách nó khám phá một không gian trạng thái vô hạn, đảm bảo rằng nó cuối cùng có thể đạt được bất kỳ trạng thái nào được kết nối với trạng thái ban đầu. Ví dụ, trên lưới vô hạn, một loại tìm kiếm có hệ thống là một đường xoắn ốc bao phủ tất cả các ô cách gốc s bước trước khi di chuyển ra ngoài đến các ô cách gốc $s + 1$ bước. Thật không may, trong một không gian trạng thái vô hạn không có giải pháp, một thuật toán đúng đắn cần phải tiếp tục tìm kiếm mãi mãi; nó không thể kết thúc vì nó không thể biết liệu trạng thái tiếp theo có phải là mục tiêu hay không.

Độ phức tạp thời gian và không gian được xem xét đối với một số thước đo độ khó của vấn đề. Trong khoa học máy tính lý thuyết, thước đo điển hình là kích thước của đồ thị không gian trạng thái, $|V| + |E|$, trong đó $|V|$ là số đỉnh (nút trạng thái) của đồ thị và $|E|$ là số cạnh (cặp trạng thái/hành động riêng biệt). Điều này phù hợp khi đồ thị là một cấu trúc dữ liệu tường minh, chẳng hạn như bản đồ Romania. Nhưng trong nhiều vấn đề AI, đồ thị chỉ được biểu diễn một cách ngầm định bởi trạng thái ban đầu, các hành động và mô hình chuyển tiếp. Đối với một không gian trạng thái ngầm định, độ phức tạp có thể được đo lường theo d , **độ sâu** (depth) hoặc số lượng hành động trong một giải pháp tối ưu; m , số lượng hành động tối đa trong bất kỳ đường dẫn nào; và b , **hệ số phân nhánh** (branching factor) hoặc số nút kế tiếp của một nút cần được xem xét.

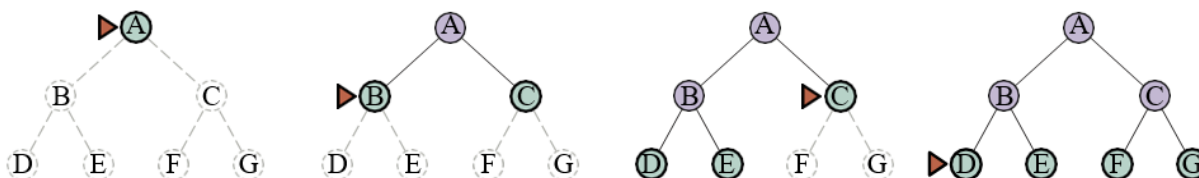
3.4 Chiến lược tìm kiếm không có thông tin

Một **thuật toán tìm kiếm không có thông tin** (uninformed search algorithm) không được cung cấp bất kỳ manh mối nào về mức độ gần của một trạng thái với (các) mục tiêu. Ví dụ, hãy xem xét tác nhân của chúng ta ở Arad với mục tiêu đến Bucharest. Một tác nhân không có thông tin và không có kiến thức về địa lý Romania không có manh mối nào về việc đi đến Zerind hay Sibiu là bước đầu tiên tốt hơn. Ngược lại, một tác nhân **có thông tin** (Phần 3.5), người biết vị trí của mỗi thành phố, biết rằng Sibiu gần Bucharest hơn nhiều và do đó có nhiều khả năng nằm trên đường dẫn ngắn nhất.

3.4.1 Tìm kiếm theo chiều rộng đầu tiên

Khi tất cả các hành động có cùng chi phí, một chiến lược thích hợp là **tìm kiếm theo chiều rộng đầu tiên** (breadth-first search), trong đó nút gốc được mở rộng đầu tiên, sau đó tất cả các nút kế tiếp của nút gốc được mở rộng tiếp theo, sau đó là các nút kế tiếp của chúng, v.v. Đây là một chiến lược tìm kiếm có hệ thống và do đó đầy đủ ngay cả trên các không gian trạng thái vô hạn. Chúng ta có thể triển khai tìm kiếm theo chiều rộng đầu tiên như một lời gọi đến **BEST-FIRST-SEARCH** trong đó hàm đánh giá $f(n)$ là độ sâu của nút—tức là số hành động cần để đạt đến nút.

Tuy nhiên, chúng ta có thể đạt được hiệu quả bổ sung với một vài thủ thuật. Một **hàng đợi vào trước ra trước** (first-in-first-out queue) sẽ nhanh hơn một hàng đợi ưu tiên và sẽ cho chúng ta thứ tự nút chính xác: các nút mới (luôn sâu hơn cha của chúng) đi vào cuối hàng đợi, và các nút cũ, nông hơn các nút mới, được mở rộng trước. Ngoài ra, **đã đạt đến** có thể là một tập hợp các trạng thái chứ không phải là một ánh xạ từ trạng thái đến nút, bởi vì một khi chúng ta đã đạt đến một trạng thái, chúng ta không bao giờ có thể tìm thấy một đường dẫn tốt hơn đến trạng thái đó. Điều đó cũng có nghĩa là chúng ta có thể thực hiện **kiểm tra mục tiêu sớm** (early goal test), kiểm tra xem một nút có phải là giải pháp ngay khi nó được tạo ra, thay vì **kiểm tra mục tiêu muộn** (late goal test) mà tìm kiếm tốt nhất đầu tiên sử dụng, chờ cho đến khi một nút được lấy ra khỏi hàng đợi. Hình 3.8 cho thấy tiến trình của một tìm kiếm theo chiều rộng đầu tiên trên một cây nhị phân đơn giản, và Hình 3.9 cho thấy thuật toán với các cải tiến hiệu quả của việc kiểm tra mục tiêu sớm.



Hình 3.8 Tìm kiếm theo chiều rộng đầu tiên trên một cây nhị phân đơn giản. Ở mỗi giai đoạn, nút sẽ được mở rộng tiếp theo được chỉ ra bằng điểm đánh dấu hình tam giác.

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.State
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
```

```
function UNIFORM-COST-SEARCH(problem) returns a solution node, or failure
  return Best-First-Search(problem, Path-Cost)
```

Hình 3.9 Thuật toán tìm kiếm theo chiều rộng đầu tiên và tìm kiếm chi phí đồng đều.

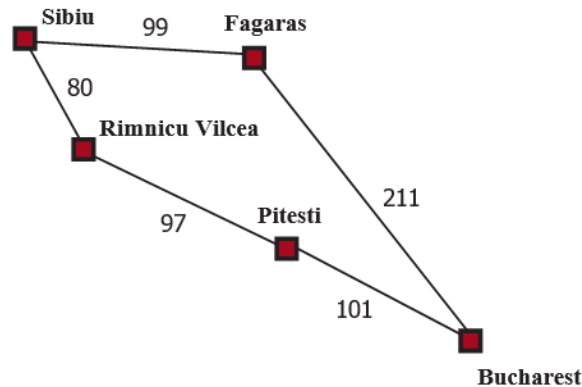
Tìm kiếm theo chiều rộng đầu tiên luôn tìm thấy một giải pháp với số lượng hành động tối thiểu, bởi vì khi nó đang tạo ra các nút ở độ sâu d , nó đã tạo ra tất cả các nút ở độ sâu $d - 1$, vì vậy nếu một trong số chúng là một giải pháp, nó sẽ được tìm thấy. Điều đó có nghĩa là nó **tối ưu chi phí** đối với các vấn đề mà tất cả các hành động có cùng chi phí, nhưng không phải đối với các vấn đề không có thuộc tính đó. Nó đầy đủ trong cả hai trường hợp. Về thời gian và không gian, hãy tưởng tượng tìm kiếm một cây đồng nhất trong đó mỗi trạng thái có b nút kế tiếp. Nút gốc của cây tìm kiếm tạo ra b nút, mỗi nút trong số đó tạo ra thêm b nút, với tổng số b^2 ở cấp độ thứ hai. Mỗi nút này tạo ra thêm b nút, tạo ra b^3 nút ở cấp độ thứ ba, và cứ thế. Bây giờ giả sử rằng giải pháp ở độ sâu d . Khi đó, tổng số nút được tạo ra là:

$$1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$$

Tất cả các nút vẫn còn trong bộ nhớ, vì vậy cả độ phức tạp thời gian và không gian đều là $O(b^d)$. Giới hạn lũy thừa như vậy thật đáng sợ. Là một ví dụ thể giới thực điển hình, hãy xem xét một vấn đề với hệ số phân nhánh $b = 10$, tốc độ xử lý 1 triệu nút/giây và yêu cầu bộ nhớ 1 Kbyte/nút. Một tìm kiếm đến độ sâu $d = 10$ sẽ mất chưa đến 3 giờ, nhưng sẽ yêu cầu 10 terabyte bộ nhớ. Yêu cầu bộ nhớ là một vấn đề lớn hơn đối với tìm kiếm theo chiều rộng đầu tiên so với thời gian thực thi. Nhưng thời gian vẫn là một yếu tố quan trọng. Ở độ sâu $d = 14$, ngay cả với bộ nhớ vô hạn, tìm kiếm sẽ mất 3,5 năm. Nói chung, các bài toán tìm kiếm có độ phức tạp lũy thừa không thể được giải quyết bằng tìm kiếm không có thông tin đối với bất kỳ trường hợp nào ngoài những trường hợp nhỏ nhất.

3.4.2 Thuật toán của Dijkstra hoặc tìm kiếm chi phí đồng đều

Khi các hành động có chi phí khác nhau, một lựa chọn rõ ràng là sử dụng tìm kiếm tốt nhất đầu tiên trong đó hàm đánh giá là chi phí của đường dẫn từ nút gốc đến nút hiện tại. Điều này được cộng đồng khoa học máy tính lý thuyết gọi là **thuật toán của Dijkstra**, và được cộng đồng AI gọi là **tìm kiếm chi phí đồng đều** (uniform-cost search). Ý tưởng là trong khi tìm kiếm theo chiều rộng đầu tiên lan ra theo các làn sóng có độ sâu đồng đều—đầu tiên là độ sâu 1, sau đó là độ sâu 2, v.v.—thì tìm kiếm chi phí đồng đều lan ra theo các làn sóng có **chi phí đường dẫn đồng đều**. Thuật toán có thể được triển khai dưới dạng một lời gọi đến **BEST-FIRST-SEARCH** với **PATH-COST** làm hàm đánh giá, như trong Hình 3.9.



Hình 3.10 Một phần của không gian trạng thái Romania, được chọn để minh họa tìm kiếm chi phí đồng đều.

Hãy xem xét Hình 3.10, trong đó vấn đề là đi từ Sibiu đến Bucharest. Các nút kế tiếp của Sibiu là Rimnicu Vilcea và Fagaras, với chi phí lần lượt là 80 và 99. Nút có chi phí thấp nhất, Rimnicu Vilcea, được mở rộng tiếp theo, thêm Pitesti với chi phí $80 + 97 = 177$. Nút có chi phí thấp nhất bây giờ là Fagaras, vì vậy nó được mở rộng, thêm Bucharest với chi phí $99 + 211 = 310$. Bucharest là mục tiêu, nhưng thuật toán chỉ kiểm tra mục tiêu khi nó mở rộng một nút, chứ không phải khi nó tạo ra một nút, vì vậy nó vẫn chưa phát hiện ra đây là một đường dẫn đến mục tiêu.

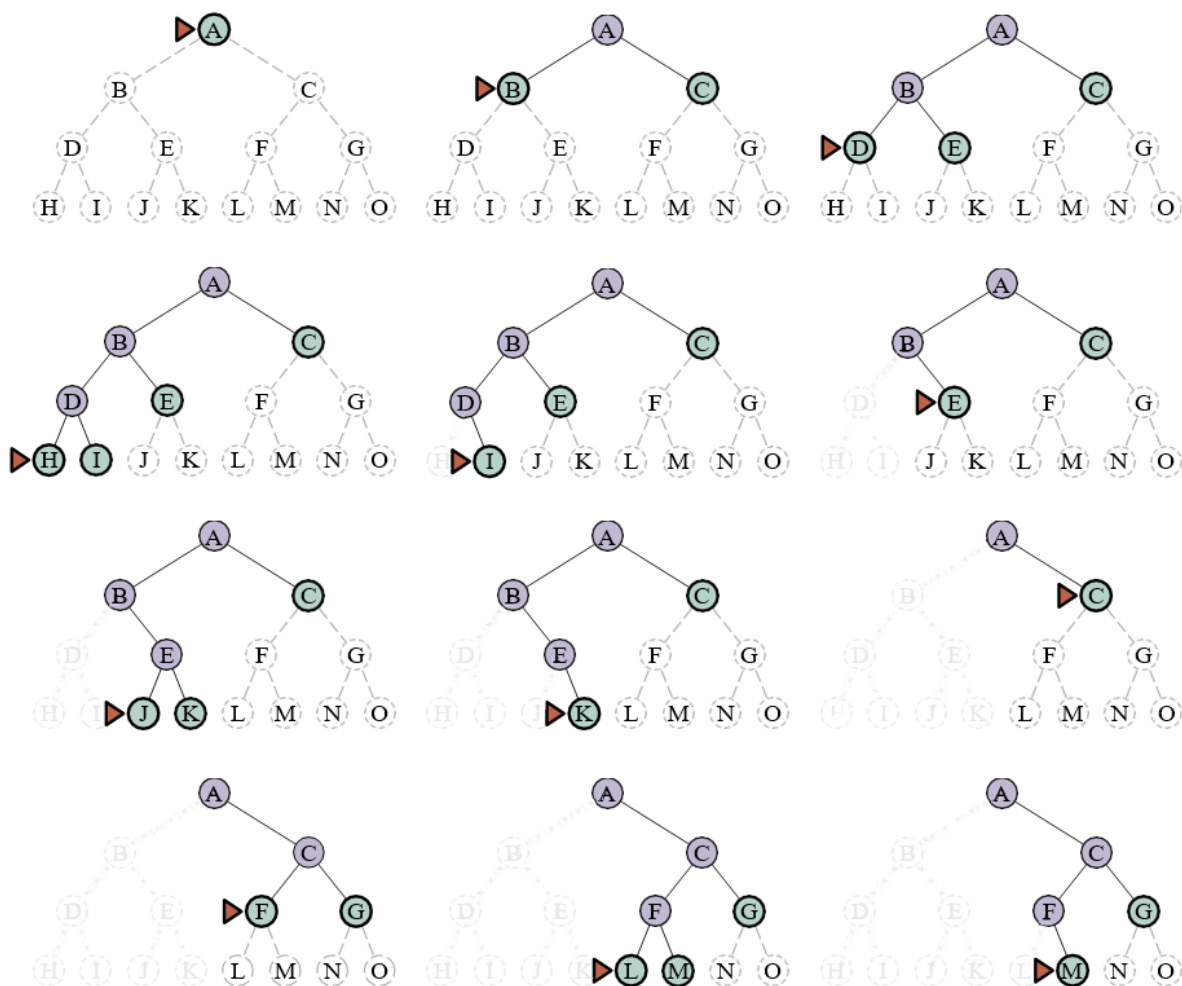
Thuật toán tiếp tục, chọn Pitesti để mở rộng tiếp theo và thêm một đường dẫn thứ hai đến Bucharest với chi phí $80 + 97 + 101 = 278$. Nó có chi phí thấp hơn, vì vậy nó thay thế đường dẫn trước đó trong **đã đạt đến** và được thêm vào biên. Hóa ra nút này bây giờ có chi phí thấp nhất, vì vậy nó được xem xét tiếp theo, được tìm thấy là một mục tiêu và được trả về. Lưu ý rằng nếu chúng ta đã kiểm tra mục tiêu khi tạo một nút thay vì khi mở rộng nút có chi phí thấp nhất, thì chúng ta sẽ trả về một đường dẫn có chi phí cao hơn (đường dẫn qua Fagaras).

Độ phức tạp của tìm kiếm chi phí đồng đều được đặc trưng bằng C^* , chi phí của giải pháp tối ưu, và ϵ , một cận dưới về chi phí của mỗi hành động, với $\epsilon > 0$. Khi đó, độ phức tạp thời gian và không gian trong trường hợp xấu nhất của thuật toán là $O(b^{1+\lceil C^*/\epsilon \rceil})$, có thể lớn hơn nhiều so với b^d . Điều này là do tìm kiếm chi phí đồng đều có thể khám phá các cây hành động lớn với chi phí thấp trước khi khám phá các đường dẫn liên quan đến một hành động có chi phí cao và có thể hữu ích. Khi tất cả các chi phí hành động bằng nhau, $b^{1+\lceil C^*/\epsilon \rceil}$ chỉ là b^{d+1} , và tìm kiếm chi phí đồng đều tương tự như tìm kiếm theo chiều rộng đầu tiên.

Tìm kiếm chi phí đồng đều là **đầy đủ** và **tối ưu chi phí**, bởi vì giải pháp đầu tiên nó tìm thấy sẽ có chi phí ít nhất bằng chi phí của bất kỳ nút nào khác trong biên. Tìm kiếm chi phí đồng đều xem xét tất cả các đường dẫn một cách có hệ thống theo thứ tự chi phí tăng dần, không bao giờ bị mắc kẹt khi đi xuống một đường dẫn vô hạn (giả sử rằng tất cả các chi phí hành động đều $> \epsilon > 0$).

3.4.3 Tìm kiếm theo chiều sâu đầu tiên và vấn đề bộ nhớ

Tìm kiếm theo chiều sâu đầu tiên (depth-first search) luôn mở rộng nút sâu nhất trong biên trước tiên. Nó có thể được triển khai dưới dạng một lời gọi đến **BEST-FIRST-SEARCH** trong đó hàm đánh giá f là âm của độ sâu. Tuy nhiên, nó thường được triển khai không phải dưới dạng tìm kiếm đồ thị mà là tìm kiếm giống cây không giữ một bảng các trạng thái đã đạt đến. Tiến trình của tìm kiếm được minh họa trong Hình 3.11; tìm kiếm tiến hành ngay lập tức đến mức sâu nhất của cây tìm kiếm, nơi các nút không có nút kế tiếp. Sau đó, tìm kiếm “quay lại” nút sâu nhất tiếp theo vẫn còn các nút kế tiếp chưa được mở rộng. Tìm kiếm theo chiều sâu đầu tiên không tối ưu chi phí; nó trả về giải pháp đầu tiên nó tìm thấy, ngay cả khi nó không phải là rẻ nhất.



Hình 3.11 Một tá bước (từ trái sang phải, từ trên xuống dưới) trong tiến trình của một tìm kiếm theo chiều sâu đầu tiên trên một cây nhị phân từ trạng thái bắt đầu A đến mục tiêu M. Biên có màu xanh lá cây, với một hình tam giác đánh dấu nút sẽ được mở rộng tiếp theo. Các nút đã được mở rộng trước đó có màu tím nhạt, và các nút tương lai tiềm năng có các đường đứt nét mờ. Các nút đã mở rộng không có hậu duệ trong biên (các đường rất mờ) có thể bị loại bỏ.

Đối với các không gian trạng thái hữu hạn là cây, nó hiệu quả và đầy đủ; đối với các không gian trạng thái không có chu trình, nó có thể mở rộng cùng một trạng thái nhiều lần thông qua các đường dẫn khác nhau, nhưng sẽ (cuối cùng) khám phá toàn bộ không gian một cách có hệ thống.

Trong các không gian trạng thái có chu trình, nó có thể bị kẹt trong một vòng lặp vô hạn; do đó một số triển khai của tìm kiếm theo chiều sâu đầu tiên kiểm tra mỗi nút mới để tìm chu kỳ. Cuối cùng, trong các không gian trạng thái vô hạn, tìm kiếm theo chiều sâu đầu tiên không có hệ thống: nó có thể bị kẹt khi đi xuống một đường dẫn vô hạn, ngay cả khi không có chu kỳ. Do đó, tìm kiếm theo chiều sâu đầu tiên không đầy đủ.

Với tất cả những tin xấu này, tại sao mọi người lại cân nhắc sử dụng tìm kiếm theo chiều sâu đầu tiên thay vì tìm kiếm theo chiều rộng đầu tiên hoặc tìm kiếm tốt nhất đầu tiên? Câu trả lời là đối với các vấn đề mà tìm kiếm giống cây là khả thi, tìm kiếm theo chiều sâu đầu tiên có nhu cầu về bộ nhớ nhỏ hơn nhiều. Chúng ta không giữ một bảng **đã đạt đến** nào cả, và biên rất nhỏ: hãy nghĩ về biên trong tìm kiếm theo chiều rộng đầu tiên như bề mặt của một hình cầu không ngừng mở rộng, trong khi biên trong tìm kiếm theo chiều sâu đầu tiên chỉ là bán kính của hình cầu.

Đối với một không gian trạng thái giống cây hữu hạn như trong Hình 3.11, một tìm kiếm giống cây theo chiều sâu đầu tiên mất thời gian tỷ lệ thuận với số lượng trạng thái, và có độ phức tạp bộ nhớ chỉ là $O(bm)$, trong đó b là hệ số phân nhánh và m là độ sâu tối đa của cây. Một số vấn đề sẽ yêu cầu exabyte bộ nhớ với tìm kiếm theo chiều rộng đầu tiên có thể được xử lý chỉ với kilobyte bằng cách sử dụng tìm kiếm theo chiều sâu đầu tiên. Vì việc sử dụng bộ nhớ tiết kiệm của nó, tìm kiếm giống cây theo chiều sâu đầu tiên đã được áp dụng làm nền tảng cơ bản của nhiều lĩnh vực AI, bao gồm thỏa mãn ràng buộc (Chương 5), thỏa mãn mệnh đề (Chương 7) và lập trình logic (Chương 9).

Một biến thể của tìm kiếm theo chiều sâu đầu tiên được gọi là **tìm kiếm quay lui** (backtracking search) thậm chí còn sử dụng ít bộ nhớ hơn. (Xem Chương 5 để biết thêm chi tiết.) Trong tìm kiếm quay lui, chỉ có một nút kế tiếp được tạo ra tại một thời điểm chứ không phải tất cả các nút kế tiếp; mỗi nút được mở rộng một phần sẽ nhớ nút kế tiếp nào sẽ được tạo ra tiếp theo. Ngoài ra, các nút kế tiếp được tạo ra bằng cách sửa đổi trực tiếp mô tả trạng thái hiện tại thay vì cấp phát bộ nhớ cho một trạng thái hoàn toàn mới. Điều này làm giảm yêu cầu bộ nhớ xuống chỉ còn một mô tả trạng thái và một đường dẫn gồm $O(m)$ hành động; một khoản tiết kiệm đáng kể so với $O(bm)$ trạng thái cho tìm kiếm theo chiều sâu đầu tiên. Với tìm kiếm quay lui, chúng ta cũng có tùy chọn duy trì một cấu trúc dữ liệu tập hợp hiệu quả cho các trạng thái trên đường dẫn hiện tại, cho phép chúng ta kiểm tra một đường dẫn chu trình trong thời gian $O(1)$ thay vì $O(m)$. Để tìm kiếm quay lui hoạt động, chúng ta phải có khả năng hoàn tác mỗi hành động khi chúng ta quay lui. Tìm kiếm quay lui rất quan trọng đối với sự thành công của nhiều vấn đề với mô tả trạng thái lớn, chẳng hạn như lắp ráp robot.

3.4.4 Tìm kiếm giới hạn độ sâu và tìm kiếm lặp sâu dần

Để ngăn tìm kiếm theo chiều sâu đầu tiên đi lạc xuống một đường dẫn vô hạn, chúng ta có thể sử dụng **tìm kiếm giới hạn độ sâu** (depth-limited search), một phiên bản của tìm kiếm theo chiều sâu đầu tiên trong đó chúng ta cung cấp một **giới hạn độ sâu**, l , và coi tất cả các nút ở độ sâu l như thể chúng không có nút kế tiếp (xem Hình 3.12). Độ phức tạp thời gian là $O(b^l)$ và độ phức tạp không gian là $O(bl)$. Thật không may, nếu chúng ta chọn l không tốt, thuật toán sẽ không tìm thấy giải pháp, làm cho nó trở nên không đầy đủ.

Vì tìm kiếm theo chiều sâu đầu tiên là một tìm kiếm giống cây, chúng ta không thể ngăn nó lãng phí thời gian vào các đường dẫn dư thừa nói chung, nhưng chúng ta có thể loại bỏ các chu kỳ với chi phí là một số thời gian tính toán. Nếu chúng ta chỉ nhìn lên một vài liên kết trong chuỗi cha, chúng ta có thể bắt được hầu hết các chu kỳ; các chu kỳ dài hơn được xử lý bởi giới hạn độ sâu.

Đôi khi một giới hạn độ sâu tốt có thể được chọn dựa trên kiến thức về vấn đề. Ví dụ, trên bản đồ Romania có 20 thành phố. Do đó, $l = 19$ là một giới hạn hợp lệ. Nhưng nếu chúng ta nghiên cứu bản đồ một cách cẩn thận, chúng ta sẽ khám phá ra rằng bất kỳ thành phố nào cũng có thể được đến từ bất kỳ thành phố nào khác trong tối đa 9 hành động. Con số này, được gọi là **đường kính** của đồ thị không gian trạng thái, cho chúng ta một giới hạn độ sâu tốt hơn, dẫn đến một tìm kiếm giới hạn độ sâu hiệu quả hơn. Tuy nhiên, đối với hầu hết các vấn đề, chúng ta sẽ không biết một giới hạn độ sâu tốt cho đến khi chúng ta đã giải quyết vấn đề.

Tìm kiếm lặp sâu dần (iterative deepening search) giải quyết vấn đề chọn một giá trị tốt cho l bằng cách thử tất cả các giá trị: đầu tiên là 0, sau đó là 1, sau đó là 2, v.v.—cho đến khi một giải pháp được tìm thấy hoặc tìm kiếm giới hạn độ sâu trả về giá trị **cắt cụt** (cutoff) thay vì giá trị thất bại. Thuật toán được hiển thị trong Hình 3.12. Tìm kiếm lặp sâu dần kết hợp nhiều lợi ích của tìm kiếm theo chiều sâu đầu tiên và tìm kiếm theo chiều rộng đầu tiên. Giống như tìm kiếm theo chiều sâu đầu tiên, yêu cầu bộ nhớ của nó là khiêm tốn: $O(bd)$ khi có một giải pháp, hoặc $O(bm)$ trên các không gian trạng thái hữu hạn không có giải pháp. Giống như tìm kiếm theo chiều rộng đầu tiên, tìm kiếm lặp sâu dần là tối ưu cho các vấn đề mà tất cả các hành động có cùng chi phí, và đầy đủ trên các không gian trạng thái không có chu trình hữu hạn, hoặc trên bất kỳ không gian trạng thái hữu hạn nào khi chúng ta kiểm tra các chu kỳ của nút trên suốt đường dẫn.

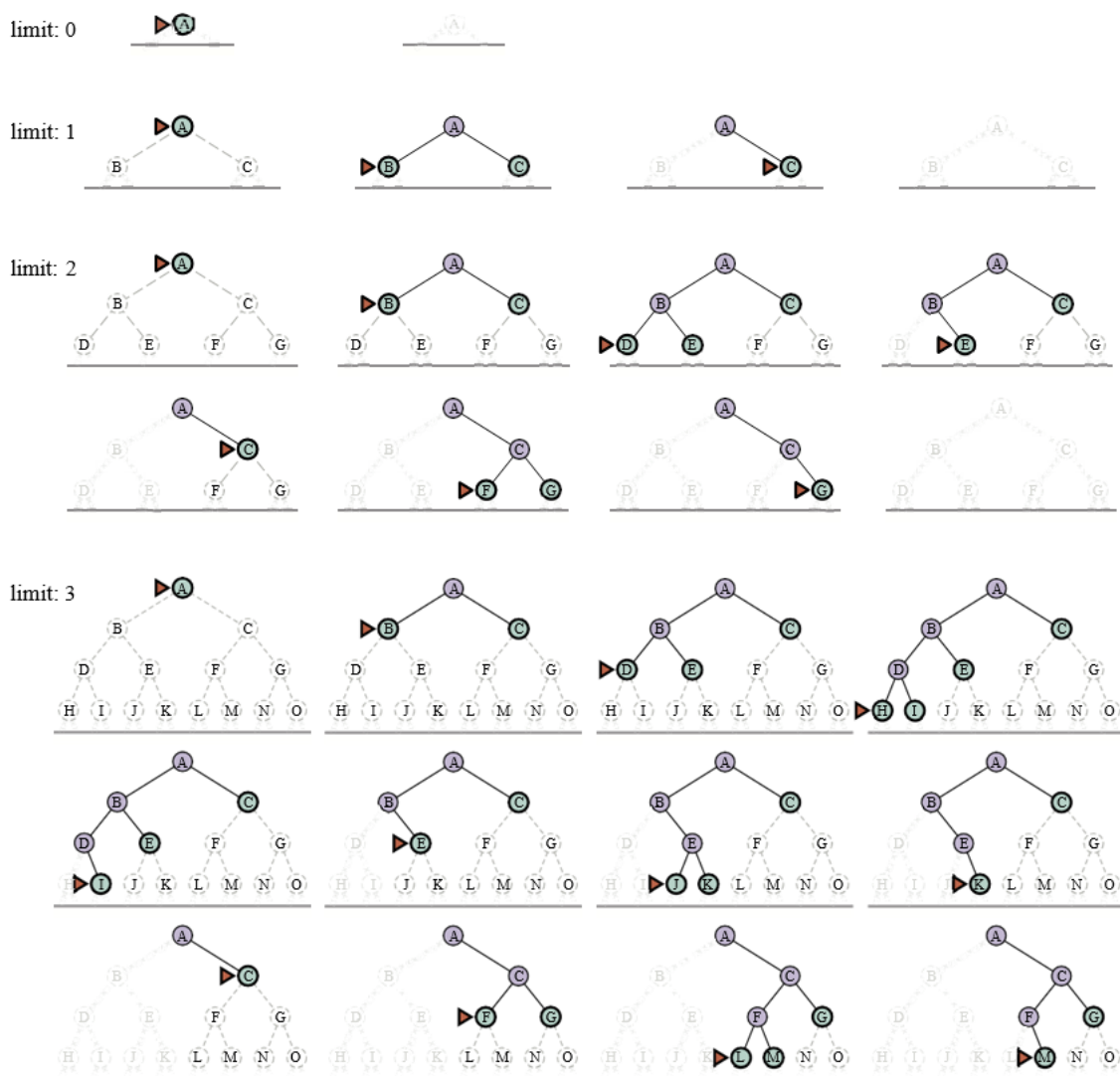
```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution node or failure
  for depth = 0 to  $\infty$  do
    result  $\leftarrow$  Depth-Limited-Search(problem, depth)
    if result  $\neq$  cutoff then return result

function DEPTH-LIMITED-SEARCH(problem, l) returns a node or failure or cutoff
  frontier  $\leftarrow$  a LIFO queue (stack) with NODE(problem.INITIAL) as an element
  result  $\leftarrow$  failure
  while not IS-EMPTY(frontier) do
    node  $\leftarrow$  POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    if Depth(node) > l then
      result  $\leftarrow$  cutoff
    else if not IS-CYCLE(node) do
      for each child in EXPAND(problem, node) do
        add child to frontier
  return result
```

Hình 3.12 Tìm kiếm lặp sâu dần và tìm kiếm giống cây giới hạn độ sâu. Tìm kiếm lặp sâu dần liên tục áp dụng tìm kiếm giới hạn độ sâu với các giới hạn tăng dần. Nó trả về một trong ba loại giá trị khác nhau: hoặc một nút giải pháp; hoặc **failure**, khi nó đã cạn kiệt tất cả các nút và chứng minh không có giải pháp ở bất kỳ độ sâu nào; hoặc **cutoff**, có nghĩa là có thể có một giải pháp ở độ sâu sâu hơn l . Đây là một thuật toán tìm kiếm giống cây không theo dõi các trạng thái đã đạt đến, và do đó sử dụng ít bộ nhớ hơn nhiều so với tìm kiếm tốt nhất đầu tiên, nhưng có nguy cơ ghé thăm

cùng một trạng thái nhiều lần trên các đường dẫn khác nhau. Ngoài ra, nếu việc kiểm tra **IS-CYCLE** không kiểm tra tất cả các chu kỳ, thì thuật toán có thể bị mắc kẹt trong một vòng lặp.

Độ phức tạp thời gian là $O(b^d)$ khi có một giải pháp, hoặc $O(b^m)$ khi không có. Mỗi lần lặp của tìm kiếm lặp sâu dần tạo ra một cấp độ mới, theo cùng một cách mà tìm kiếm theo chiều rộng đầu tiên làm, nhưng tìm kiếm theo chiều rộng đầu tiên làm điều này bằng cách lưu trữ tất cả các nút trong bộ nhớ, trong khi tìm kiếm lặp sâu dần làm điều đó bằng cách lặp lại các cấp độ trước đó, do đó tiết kiệm bộ nhớ với chi phí là nhiều thời gian hơn. Hình 3.13 cho thấy bốn lần lặp của tìm kiếm lặp sâu dần trên một cây tìm kiếm nhị phân, trong đó giải pháp được tìm thấy ở lần lặp thứ tư.



Hình 3.13 Bốn lần lặp của tìm kiếm lặp sâu dần cho mục tiêu M trên một cây nhị phân, với giới hạn độ sâu thay đổi từ 0 đến 3. Lưu ý các nút bên trong tạo thành một đường dẫn duy nhất. Hình tam giác đánh dấu nút sẽ được mở rộng tiếp theo; các nút màu xanh lá cây với viền đậm nằm trên

biên; các nút rất mờ có thể chứng minh không thể là một phần của giải pháp với giới hạn độ sâu này.

Tìm kiếm lặp sâu dần có vẻ lãng phí vì các trạng thái gần đỉnh của cây tìm kiếm được tạo lại nhiều lần. Nhưng đối với nhiều không gian trạng thái, hầu hết các nút đều ở cấp độ dưới cùng, vì vậy việc các cấp độ trên được lặp lại không thành vấn đề. Trong một tìm kiếm lặp sâu dần, các nút ở cấp độ dưới cùng (độ sâu d) được tạo ra một lần, những nút ở cấp độ kế cuối được tạo ra hai lần, v.v., cho đến các nút con của nút gốc, được tạo ra d lần. Vì vậy, tổng số nút được tạo ra trong trường hợp xấu nhất là:

$$N_{IDS} = (d)b^1 + (d-1)b^2 + (d-2)b^3 + \dots + b^d$$

Điều này cho độ phức tạp thời gian là $O(b^d)$ —về mặt tiệm cận giống như tìm kiếm theo chiều rộng đầu tiên. Ví dụ, nếu $b = 10$ và $d = 5$, các con số là:

$$N_{IDS} = 50 + 400 + 3.000 + 20.000 + 100.000 = 123.450 \quad N_{BFS} = 10 + 100 + 1.000 + 10.000 + 100.000 = 111.110$$

Nếu bạn thực sự lo lắng về sự lặp lại, bạn có thể sử dụng một cách tiếp cận kết hợp chạy tìm kiếm theo chiều rộng đầu tiên cho đến khi gần như toàn bộ bộ nhớ có sẵn được sử dụng, và sau đó chạy tìm kiếm lặp sâu dần từ tất cả các nút trong biên. Nói chung, tìm kiếm lặp sâu dần là phương pháp tìm kiếm không có thông tin được ưu tiên khi không gian trạng thái tìm kiếm lớn hơn khả năng chứa của bộ nhớ và độ sâu của giải pháp không được biết.

3.4.5 Tìm kiếm hai chiều

Các thuật toán chúng ta đã đề cập cho đến nay bắt đầu từ một trạng thái ban đầu và có thể đạt đến bất kỳ trạng thái mục tiêu nào trong số nhiều trạng thái mục tiêu có thể có. Một cách tiếp cận thay thế được gọi là **tìm kiếm hai chiều** (bidirectional search) đồng thời tìm kiếm tiến về phía trước từ trạng thái ban đầu và lùi về phía sau từ các trạng thái mục tiêu, với hy vọng rằng hai tìm kiếm sẽ gặp nhau. Động lực là $b^{d/2} + b^{d/2}$ nhỏ hơn nhiều so với b^d (ví dụ, nhỏ hơn 50.000 lần khi $b = d = 10$).

```
function BIBF-SEARCH(problemF, fF, problemB, fB) returns a solution node, or failure
  nodeF ← NODE(problemF.INITIAL) // Nút cho một trạng thái bắt đầu
  nodeB ← NODE(problemB.INITIAL) // Nút cho một trạng thái mục tiêu
  frontierF ← a priority queue ordered by fF, with nodeF as an element
  frontierB ← a priority queue ordered by fB, with nodeB as an element
  reachedF ← a lookup table, with one key nodeF.STATE and value nodeF
  reachedB ← a lookup table, with one key nodeB.STATE and value nodeB
  solution ← failure
  while not TERMINATED(solution, frontierF, frontierB) do
    if fF(TOP(frontierF)) < fB(TOP(frontierB)) then
      solution ← PROCEED(F, problemF, frontierF, reachedF, reachedB, solution)
    else
```

```

        solution ← PROCEED(B, problemB, frontierB, reachedB, reachedF, solution)
    return solution

function PROCEED(dir, problem, frontier, reached, reached2, solution) returns a solution
    // Mở rộng nút trên biên; kiểm tra với biên kia trong reached2.
    // Biến "dir" là hướng: F cho tiến hoặc B cho lùi.
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
        s ← child.State
        if s not in reached or PATH-COST(child) < PATH-COST(reached[s]) then
            reached[s] ← child
            add child to frontier
        if s is in reached2 then
            solution2 ← JOIN-NODES(dir, child, reached2[s])
            if PATH-COST(solution2) < PATH-COST(solution) then
                solution ← solution2
    return solution

```

Hình 3.14 Tìm kiếm tốt nhất đầu tiên hai chiều giữ hai biên và hai bảng các trạng thái đã đạt đến. Khi một đường dẫn trong một biên đạt đến một trạng thái cũng được đạt đến trong nửa kia của tìm kiếm, hai đường dẫn được nối lại (bằng hàm **JOIN-NODES**) để tạo thành một giải pháp. Giải pháp đầu tiên chúng ta có được không được đảm bảo là tốt nhất; hàm **TERMINATED** xác định khi nào dừng tìm kiếm các giải pháp mới.

Để điều này hoạt động, chúng ta cần theo dõi hai biên và hai bảng các trạng thái đã đạt đến, và chúng ta cần có khả năng suy luận ngược: nếu trạng thái s' là một nút kế tiếp của s theo hướng tiến, thì chúng ta cần biết rằng s là một nút kế tiếp của s' theo hướng lùi. Chúng ta có một giải pháp khi hai biên va chạm.

Có nhiều phiên bản khác nhau của tìm kiếm hai chiều, giống như có nhiều thuật toán tìm kiếm một chiều khác nhau. Trong phần này, chúng ta mô tả **tìm kiếm tốt nhất đầu tiên hai chiều**. Mặc dù có hai biên riêng biệt, nút sẽ được mở rộng tiếp theo luôn là một nút có giá trị tối thiểu của hàm đánh giá, trên cả hai biên. Khi hàm đánh giá là chi phí đường dẫn, chúng ta biết rằng giải pháp đầu tiên được tìm thấy sẽ là một giải pháp tối ưu, nhưng với các hàm đánh giá khác nhau, điều đó không nhất thiết phải đúng. Do đó, chúng ta theo dõi giải pháp tốt nhất được tìm thấy cho đến nay và có thể phải cập nhật nó nhiều lần trước khi kiểm tra **TERMINATED** chứng minh rằng không còn giải pháp tốt hơn có thể có.

3.4.6 So sánh các thuật toán tìm kiếm không có thông tin

Hình 3.15 so sánh các thuật toán tìm kiếm không có thông tin dựa trên bốn tiêu chí đánh giá được đưa ra trong Phần 3.3.4. So sánh này dành cho các phiên bản **tìm kiếm giống cây** không kiểm tra các trạng thái lặp lại. Đối với các tìm kiếm đồ thị có kiểm tra, sự khác biệt chính là tìm kiếm theo chiều sâu đầu tiên là đầy đủ đối với các không gian trạng thái hữu hạn, và độ phức tạp không gian và thời gian được giới hạn bởi kích thước của không gian trạng thái (số đỉnh và cạnh, $|V| + |E|$).

Tiêu chí	Tìm kiếm theo chiều rộng đầu tiên	Tìm kiếm chi phí đồng đều	Tìm kiếm theo chiều sâu đầu tiên	Tìm kiếm giới hạn độ sâu	Tìm kiếm lặp sâu dần	Hai chiều (nếu áp dụng)
Đầy đủ?	Có ¹	Có ^{1,2}	Không	Không	Có ¹	Có ^{1,4}
Chi phí tối ưu?	Có ³	Có	Không	Không	Có ³	Có ^{3,4}
Thời gian	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Không gian	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(bd)$	$O(b^{d/2})$

Hình 3.15 Đánh giá các thuật toán tìm kiếm. b là hệ số phân nhánh; m là độ sâu tối đa của cây tìm kiếm; d là độ sâu của giải pháp nông nhất, hoặc là m khi không có giải pháp; l là giới hạn độ sâu. Các chú thích ở trên như sau: ¹ đầy đủ nếu b là hữu hạn, và không gian trạng thái hoặc có một giải pháp hoặc là hữu hạn. ² đầy đủ nếu tất cả chi phí hành động đều $\geq \epsilon > 0$; ³ tối ưu chi phí nếu tất cả chi phí hành động đều giống hệt nhau; ⁴ nếu cả hai hướng đều là tìm kiếm theo chiều rộng đầu tiên hoặc chi phí đồng đều.

3.5 Chiến lược tìm kiếm có thông tin (Heuristic)

Phần này cho thấy cách một chiến lược **tìm kiếm có thông tin**—một chiến lược sử dụng các gợi ý dành riêng cho miền về vị trí của mục tiêu—có thể tìm thấy giải pháp hiệu quả hơn so với một chiến lược không có thông tin. Các gợi ý đến dưới dạng một **hàm heuristic**, ký hiệu là $h(n)$:

$h(n)$

= *chi phí ước tính của đường dẫn rẻ nhất từ trạng thái tại nút n đến một trạng thái mục tiêu.*

Ví dụ, trong các bài toán tìm đường, chúng ta có thể ước tính khoảng cách từ trạng thái hiện tại đến một mục tiêu bằng cách tính khoảng cách đường chim bay trên bản đồ giữa hai điểm. Chúng ta nghiên cứu các heuristic và chúng đến từ đâu chi tiết hơn trong Phần 3.6.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Hình 3.16 Các giá trị của h_{SLD} —khoảng cách đường chim bay đến Bucharest.

3.5.1 Tìm kiếm tốt nhất đầu tiên tham lam

Tìm kiếm tốt nhất đầu tiên tham lam (Greedy best-first search) là một dạng tìm kiếm tốt nhất đầu tiên mở rộng nút có giá trị $h(n)$ thấp nhất trước tiên—nút dường như gần mục tiêu nhất—dựa trên cơ sở rằng điều này có khả năng dẫn đến một giải pháp nhanh chóng. Vì vậy, hàm đánh giá là $f(n) = h(n)$.

Hãy xem điều này hoạt động như thế nào đối với các bài toán tìm đường ở Romania; chúng ta sử dụng **heuristic khoảng cách đường chim bay**, mà chúng ta sẽ gọi là h_{SLD} . Nếu mục tiêu là Bucharest, chúng ta cần biết khoảng cách đường chim bay đến Bucharest, được hiển thị trong Hình 3.16. Ví dụ, $h_{SLD}(Arad) = 366$. Lưu ý rằng các giá trị của h_{SLD} không thể được tính toán từ chính mô tả vấn đề (tức là các hàm **ACTIONS** và **RESULT**). Hơn nữa, phải mất một lượng kiến thức nhất định về thế giới để biết rằng h_{SLD} có tương quan với khoảng cách đường bộ thực tế và do đó là một heuristic hữu ích.

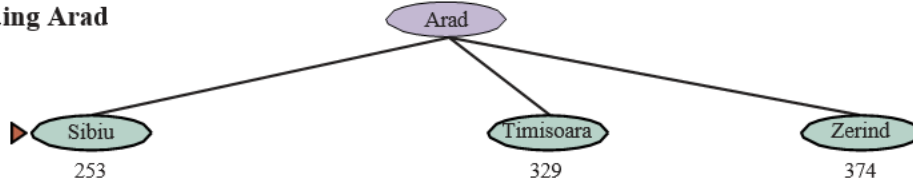
Hình 3.17 cho thấy tiến trình của một tìm kiếm giống cây tốt nhất đầu tiên tham lam sử dụng h_{SLD} để tìm đường dẫn từ Arad đến Bucharest. Nút đầu tiên được mở rộng từ Arad sẽ là Sibiu vì heuristic nói rằng nó gần Bucharest hơn so với Zerind hoặc Timisoara. Nút tiếp theo được mở rộng sẽ là Fagaras vì nó hiện gần nhất theo heuristic. Fagaras đến lượt nó tạo ra Bucharest, là mục tiêu. Đối với bài toán cụ thể này, tìm kiếm tốt nhất đầu tiên tham lam sử dụng h_{SLD} tìm thấy một giải pháp mà không bao giờ mở rộng một nút không nằm trên đường dẫn giải pháp. Tuy nhiên, giải pháp nó tìm thấy không có chi phí tối ưu: đường dẫn qua Sibiu và Fagaras đến Bucharest dài hơn 32 dặm so với đường dẫn qua Rimnicu Vilcea và Pitesti. Đây là lý do tại sao thuật toán được gọi là “tham lam”—ở mỗi lần lặp, nó cố gắng đến gần mục tiêu nhất có thể, nhưng sự tham lam có thể dẫn đến kết quả tồi tệ hơn so với việc cẩn thận.

Tìm kiếm đồ thị tốt nhất đầu tiên tham lam là đầy đủ trong các không gian trạng thái hữu hạn, nhưng không phải trong các không gian vô hạn. Độ phức tạp thời gian và không gian trong trường hợp xấu nhất là $O(|V|)$. Tuy nhiên, với một hàm heuristic tốt, độ phức tạp có thể được giảm đáng kể, trên một số vấn đề đạt đến $O(b^m)$.

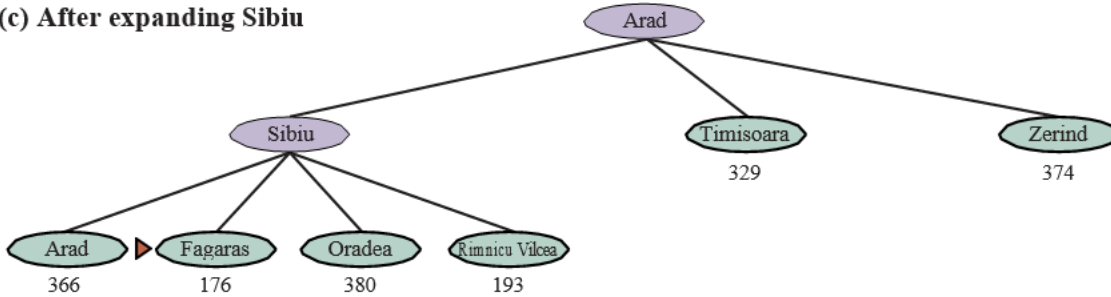
(a) The initial state



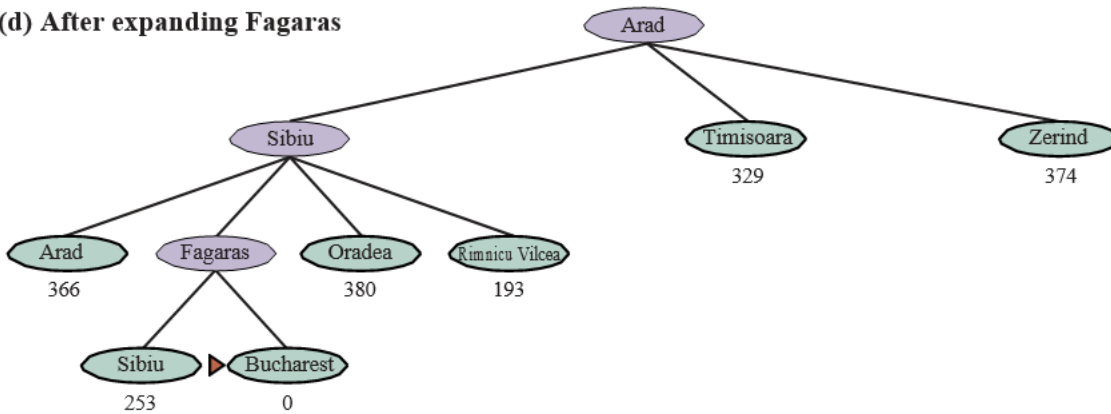
(b) After expanding Arad



(c) After expanding Sibiu



(d) After expanding Fagaras



Hình 3.17 Các giai đoạn trong một tìm kiếm giống cây tốt nhất đầu tiên tham lam cho Bucharest với heuristic khoảng cách đường chim bay h_{SLD} . Các nút được gắn nhãn bằng các giá trị h của chúng.

3.5.2 Tìm kiếm A*

Thuật toán tìm kiếm có thông tin phổ biến nhất là **tìm kiếm A*** (phát âm là “A-star search”), một tìm kiếm tốt nhất đầu tiên sử dụng hàm đánh giá:

$$f(n) = g(n) + h(n)$$

trong đó $g(n)$ là **chi phí đường dẫn** từ trạng thái ban đầu đến nút n , và $h(n)$ là **chi phí ước tính** của đường dẫn ngắn nhất từ n đến một trạng thái mục tiêu, vì vậy chúng ta có:

$$f(n) = \text{chi phí ước tính của đường dẫn từ trạng thái ban đầu đến nút } n + \text{chi phí ước tính từ nút } n \text{ đến một trạng thái mục tiêu.}$$

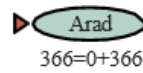
Trong Hình 3.18, chúng ta thấy tiến trình của một tìm kiếm A^* với mục tiêu là đến Bucharest. Các giá trị của g được tính từ chi phí hành động trong Hình 3.1, và các giá trị của h_{SLD} được cho trong Hình 3.16. Lưu ý rằng Bucharest lần đầu tiên xuất hiện trên biên ở bước (e), nhưng nó không được chọn để mở rộng (và do đó không được phát hiện là một giải pháp) vì ở $f = 450$ nó không phải là nút có chi phí thấp nhất trên biên—đó sẽ là Pitesti, ở $f = 417$. Một cách khác để nói điều này là có thể có một giải pháp thông qua Pitesti có chi phí thấp tới 417, vì vậy thuật toán sẽ không chấp nhận một giải pháp có chi phí 450. Ở bước (f), một đường dẫn khác đến Bucharest bây giờ là nút có chi phí thấp nhất, ở $f = 418$, vì vậy nó được chọn và được phát hiện là giải pháp tối ưu.

Tìm kiếm A^* là đầy đủ. Việc A^* có tối ưu chi phí hay không phụ thuộc vào một số thuộc tính của heuristic. Một thuộc tính chính là **tính chấp nhận được** (admissibility): một heuristic chấp nhận được là một heuristic không bao giờ ước tính quá cao chi phí để đạt đến một mục tiêu. (Do đó, một heuristic chấp nhận được là lạc quan.) Với một heuristic chấp nhận được, A^* là tối ưu chi phí, điều mà chúng ta có thể chứng minh bằng phản chứng. Giả sử đường dẫn tối ưu có chi phí C^* , nhưng thuật toán trả về một đường dẫn có chi phí $C > C^*$. Khi đó phải có một số nút n nằm trên đường dẫn tối ưu và chưa được mở rộng (bởi vì nếu tất cả các nút trên đường dẫn tối ưu đã được mở rộng, thì chúng ta đã trả về giải pháp tối ưu đó). Vì vậy, sau đó, sử dụng ký hiệu $g^*(n)$ để chỉ chi phí của đường dẫn tối ưu từ điểm bắt đầu đến n , và $h^*(n)$ để chỉ chi phí của đường dẫn tối ưu từ n đến mục tiêu gần nhất, chúng ta có:

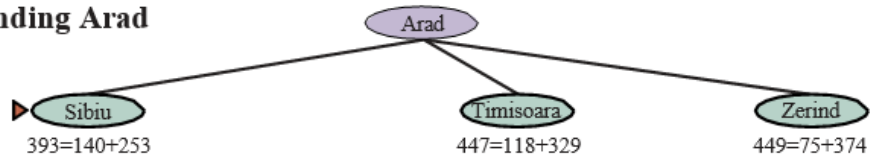
$f(n) > C^*$ (nếu không thì n đã được mở rộng) $f(n) = g(n) + h(n)$ (theo định nghĩa) $f(n) = g^*(n) + h(n)$ (vì n nằm trên một đường dẫn tối ưu) $f(n) \leq g^*(n) + h^*(n)$ (vì tính chấp nhận được, $h(n) \leq h^*(n)$) $f(n) \leq C^*$ (theo định nghĩa, $C^* = g^*(n) + h^*(n)$)

Dòng đầu tiên và dòng cuối cùng tạo thành một sự mâu thuẫn, vì vậy giả định rằng thuật toán có thể trả về một đường dẫn dưới tối ưu phải sai—chắc chắn A^* chỉ trả về các đường dẫn tối ưu chi phí.

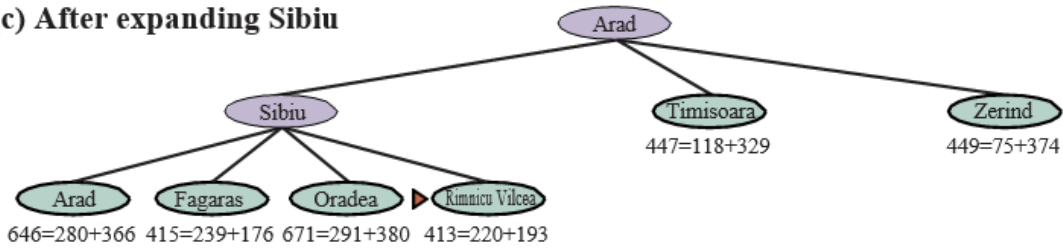
(a) The initial state



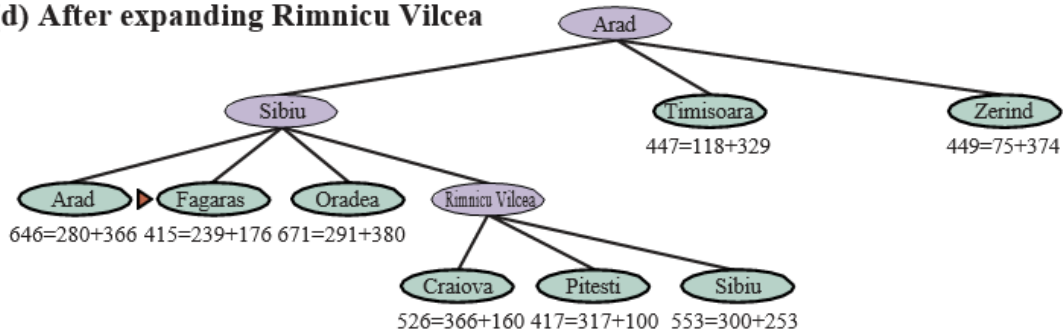
(b) After expanding Arad



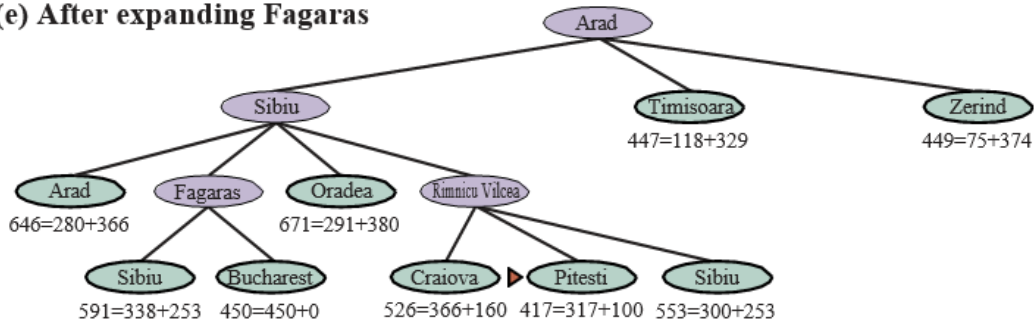
(c) After expanding Sibiu



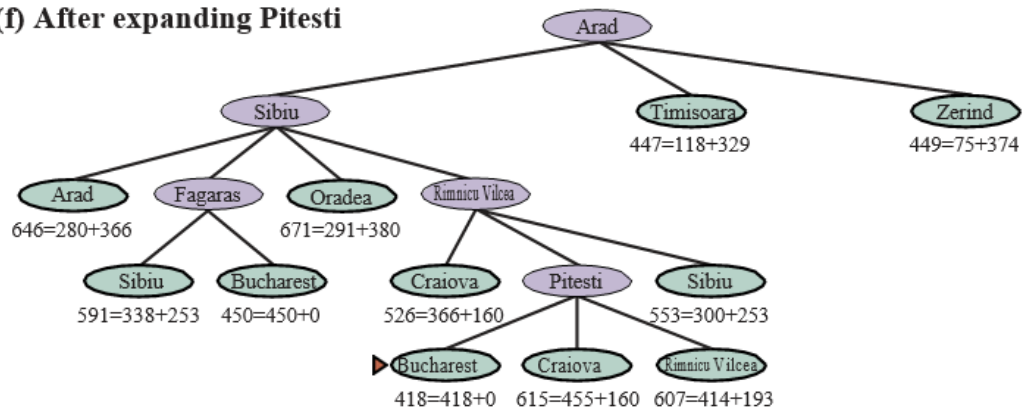
(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras



(f) After expanding Pitesti

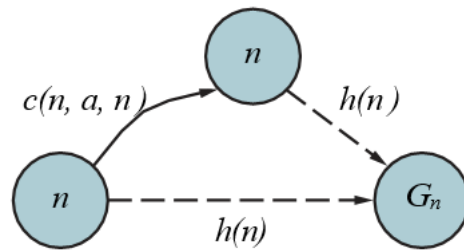


Hình 3.18 Các giai đoạn trong một tìm kiếm A* cho Bucharest. Các nút được gắn nhãn bằng $f = g + h$. Các giá trị h là khoảng cách đường chim bay đến Bucharest lấy từ Hình 3.16.

Một thuộc tính mạnh hơn một chút được gọi là **tính nhất quán** (consistency). Một heuristic $h(n)$ là nhất quán nếu, đối với mọi nút n và mọi nút kế tiếp n' của n được tạo ra bởi một hành động a , chúng ta có:

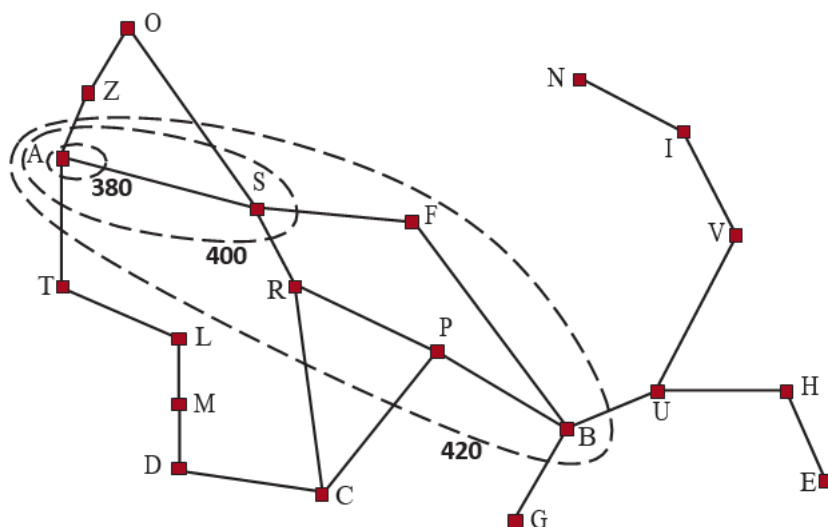
$$h(n) \leq c(n, a, n') + h(n')$$

Đây là một dạng của **bất đẳng thức tam giác**, quy định rằng một cạnh của một tam giác không thể dài hơn tổng của hai cạnh kia (xem Hình 3.19). Một ví dụ về một heuristic nhất quán là khoảng cách đường chim bay h_{SLD} mà chúng ta đã sử dụng để đến Bucharest.



Hình 3.19 Bất đẳng thức tam giác: Nếu heuristic h là nhất quán, thì một số duy nhất $h(n)$ sẽ nhỏ hơn tổng của chi phí $c(n, a, n')$ của hành động từ n đến n' cộng với ước tính heuristic $h(n')$.

Mọi heuristic nhất quán đều chấp nhận được (nhưng không ngược lại), vì vậy với một heuristic nhất quán, A* là tối ưu chi phí. Ngoài ra, với một heuristic nhất quán, lần đầu tiên chúng ta đạt đến một trạng thái, nó sẽ nằm trên một đường dẫn tối ưu, vì vậy chúng ta không bao giờ phải thêm lại một trạng thái vào biên và không bao giờ phải thay đổi một mục trong **đã đạt đến**. Nhưng với một heuristic không nhất quán, chúng ta có thể kết thúc với nhiều đường dẫn đạt đến cùng một trạng thái, và nếu mỗi đường dẫn mới có chi phí đường dẫn thấp hơn đường dẫn trước đó, thì chúng ta sẽ kết thúc với nhiều nút cho trạng thái đó trong biên, gây tốn cả thời gian và không gian. Vì lý do đó, một số triển khai của A* cần thận chỉ nhập một trạng thái vào biên một lần, và nếu một đường dẫn tốt hơn đến trạng thái đó được tìm thấy, tất cả các nút kế tiếp của trạng thái đó sẽ được cập nhật (điều này đòi hỏi các nút phải có con trỏ con cũng như con trỏ cha). Những phức tạp này đã khiến nhiều người triển khai tránh các heuristic không nhất quán, nhưng Felner và cộng sự (2011) lập luận rằng những ảnh hưởng tồi tệ nhất hiếm khi xảy ra trong thực tế và người ta không nên sợ các heuristic không nhất quán.



Hình 3.20 Bản đồ Romania cho thấy các đường đồng mức tại $f = 380, f = 400$, và $f = 420$, với Arad là trạng thái bắt đầu. Các nút bên trong một đường đồng mức đã cho có chi phí $f = g + h$ nhỏ hơn hoặc bằng giá trị đường đồng mức.

Với một heuristic không chấp nhận được, A^* có thể hoặc không tối ưu chi phí. Dưới đây là hai trường hợp nó tối ưu: Thứ nhất, nếu có ít nhất một đường dẫn tối ưu chi phí mà trên đó $h(n)$ là chấp nhận được cho tất cả các nút n trên đường dẫn, thì đường dẫn đó sẽ được tìm thấy, bất kể heuristic nói gì về các trạng thái ngoài đường dẫn. Thứ hai, nếu giải pháp tối ưu có chi phí C^* , và giải pháp tốt thứ hai có chi phí C_2 , và nếu $h(n)$ ước tính quá cao một số chi phí, nhưng không bao giờ nhiều hơn $C_2 - C^*$, thì A^* được đảm bảo sẽ trả về các giải pháp tối ưu chi phí.

3.5.3 Các đường đồng mức tìm kiếm

Một cách hữu ích để hình dung một tìm kiếm là vẽ các **đường đồng mức** trong không gian trạng thái, giống như các đường đồng mức trong bản đồ địa hình. Hình 3.20 cho thấy một ví dụ. Bên trong đường đồng mức được dán nhãn 400, tất cả các nút có $f(n) = g(n) + h(n) \leq 400$, v.v. Sau đó, vì A^* mở rộng nút biên có chi phí f thấp nhất, chúng ta có thể thấy rằng một tìm kiếm A^* lan ra từ nút bắt đầu, thêm các nút trong các dải đồng tâm có chi phí f tăng dần.

Với tìm kiếm chi phí đồng đều, chúng ta cũng có các đường đồng mức, nhưng là của chi phí g , không phải $g + h$. Các đường đồng mức với tìm kiếm chi phí đồng đều sẽ “tròn” xung quanh trạng thái bắt đầu, lan ra đều theo mọi hướng mà không có sự ưu tiên nào đối với mục tiêu. Với tìm kiếm A^* sử dụng một heuristic tốt, các dải $g + h$ sẽ kéo dài về phía một trạng thái mục tiêu (như trong Hình 3.20) và trở nên tập trung hơn xung quanh một đường dẫn tối ưu.

Hình 3.20 Bản đồ Romania cho thấy các đường đồng mức tại $f = 380, f = 400$, và $f = 420$, với Arad là trạng thái bắt đầu. Các nút bên trong một đường đồng mức đã cho có chi phí $f = g + h$ nhỏ hơn hoặc bằng giá trị đường đồng mức.

Nên rõ ràng rằng khi bạn mở rộng một đường dẫn, chi phí g là **đơn điệu** (monotonic): chi phí đường dẫn luôn tăng khi bạn đi dọc theo một đường dẫn, vì chi phí hành động luôn dương. Do đó, bạn có được các đường đồng mức đồng tâm không cắt nhau, và nếu bạn chọn vẽ các đường đủ mịn, bạn có thể đặt một đường giữa bất kỳ hai nút nào trên bất kỳ đường dẫn nào.

Nhưng không rõ liệu chi phí $f = g + h$ có tăng đơn điệu hay không. Khi bạn mở rộng một đường dẫn từ n đến n' , chi phí đi từ $g(n) + h(n)$ đến $g(n) + c(n, a, n') + h(n')$. Hủy bỏ số hạng $g(n)$, chúng ta thấy rằng chi phí của đường dẫn sẽ tăng đơn điệu nếu và chỉ nếu $h(n) \leq c(n, a, n') + h(n')$; nói cách khác nếu và chỉ nếu heuristic là **nhất quán**. Nhưng lưu ý rằng một đường dẫn có thể đóng góp một vài nút liên tiếp với cùng điểm $g(n) + h(n)$; điều này sẽ xảy ra bất cứ khi nào sự giảm trong h chính xác bằng chi phí hành động vừa được thực hiện (ví dụ, trong một bài toán lưới, khi n ở cùng hàng với mục tiêu và bạn thực hiện một bước về phía mục tiêu, g được tăng thêm 1 và h được giảm đi 1). Nếu C^* là chi phí của đường dẫn giải pháp tối ưu, thì chúng ta có thể nói những điều sau:

- A* mở rộng tất cả các nút có thể đạt được từ trạng thái ban đầu trên một đường dẫn mà mọi nút trên đường dẫn đều có $f(n) < C^*$. Chúng ta nói đây là các **nút chắc chắn được mở rộng** (surely expanded nodes).
- A* sau đó có thể mở rộng một số nút ngay trên “đường đồng mức mục tiêu” (nơi $f(n) = C^*$) trước khi chọn một nút mục tiêu.
- A* không mở rộng nút nào có $f(n) > C^*$.

Chúng ta nói rằng A* với một heuristic nhất quán là **tối ưu hiệu quả** (optimally efficient) theo nghĩa là bất kỳ thuật toán nào mở rộng các đường dẫn tìm kiếm từ trạng thái ban đầu và sử dụng cùng thông tin heuristic, phải mở rộng tất cả các nút chắc chắn được mở rộng bởi A* (vì bất kỳ nút nào trong số chúng có thể là một phần của giải pháp tối ưu). Trong số các nút có $f(n) = C^*$, một thuật toán có thể may mắn và chọn nút tối ưu trước trong khi một thuật toán khác không may mắn; chúng ta không xem xét sự khác biệt này trong việc xác định hiệu quả tối ưu.

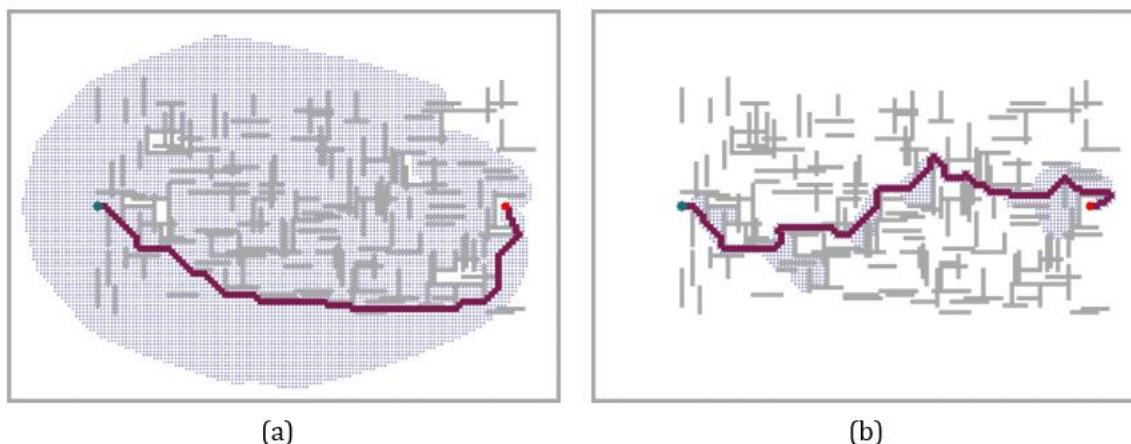
A* hiệu quả vì nó **cắt tỉa** (prunes) các nút cây tìm kiếm không cần thiết để tìm một giải pháp tối ưu. Trong Hình 3.18(b), chúng ta thấy rằng Timisoara có $f = 447$ và Zerind có $f = 449$. Mặc dù chúng là con của nút gốc và sẽ là một trong những nút đầu tiên được mở rộng bởi tìm kiếm chi phí đồng đều hoặc tìm kiếm theo chiều rộng đầu tiên, chúng không bao giờ được mở rộng bởi tìm kiếm A* vì giải pháp với $f = 418$ được tìm thấy trước. Khái niệm cắt tỉa—loại bỏ các khả năng khỏi sự xem xét mà không cần phải kiểm tra chúng—rất quan trọng đối với nhiều lĩnh vực của AI.

Việc tìm kiếm A* là đầy đủ, tối ưu chi phí và tối ưu hiệu quả trong số tất cả các thuật toán như vậy là khá thỏa mãn. Thật không may, điều đó không có nghĩa là A* là câu trả lời cho tất cả các nhu cầu tìm kiếm của chúng ta. Vấn đề là đối với nhiều bài toán, số nút được mở rộng có thể là hàm mũ theo độ dài của giải pháp. Ví dụ, hãy xem xét một phiên bản của thế giới chân không với một chân không siêu mạnh có thể dọn dẹp bất kỳ ô vuông nào với chi phí 1 đơn vị, mà không cần phải ghé thăm ô vuông đó; trong kịch bản đó, các ô vuông có thể được dọn dẹp theo bất kỳ thứ tự nào. Với N ô vuông ban đầu bị bẩn, có 2^N trạng thái mà một số tập con đã được dọn dẹp; tất cả các trạng thái đó nằm trên một đường dẫn giải pháp tối ưu, và do đó thỏa mãn $f(n) < C^*$, vì vậy tất cả chúng sẽ được A* ghé thăm.

3.5.4 Tìm kiếm thỏa mãn: Heuristic không chấp nhận được và A* có trọng số

Tìm kiếm A* có nhiều phẩm chất tốt, nhưng nó mở rộng rất nhiều nút. Chúng ta có thể khám phá ít nút hơn (tốn ít thời gian và không gian hơn) nếu chúng ta sẵn sàng chấp nhận các giải pháp dưới tối ưu, nhưng “đủ tốt”—những gì chúng ta gọi là **giải pháp thỏa mãn** (satisficing solutions). Nếu chúng ta cho phép tìm kiếm A* sử dụng một **heuristic không chấp nhận được**—một heuristic có thể ước tính quá cao—thì chúng ta có nguy cơ bỏ lỡ giải pháp tối ưu, nhưng heuristic có khả năng chính xác hơn, do đó giảm số nút được mở rộng. Ví dụ, các kỹ sư đường bộ biết khái niệm **chỉ số đường vòng** (detour index), là một số nhân được áp dụng cho khoảng cách đường chim bay để tính đến độ cong điển hình của các con đường. Một chỉ số đường vòng là 1.3 có nghĩa là nếu hai thành phố cách nhau 10 dặm theo khoảng cách đường chim bay, một ước tính tốt về đường dẫn tốt nhất giữa chúng là 13 dặm. Đối với hầu hết các địa phương, chỉ số đường vòng dao động từ 1.2 đến 1.6.

Chúng ta có thể áp dụng ý tưởng này cho bất kỳ bài toán nào, không chỉ những bài toán liên quan đến đường, với một cách tiếp cận được gọi là __tìm kiếm A* có trọng số__* (weighted A* search) trong đó chúng ta đặt nặng giá trị heuristic hơn, cho chúng ta hàm đánh giá $f(n) = g(n) + W \times h(n)$, với một số $W > 1$.



Hình 3.21 Hai tìm kiếm trên cùng một lưới: (a) một tìm kiếm A* và (b) một tìm kiếm A* có trọng số với trọng số $W = 2$. Các thanh màu xám là chướng ngại vật, đường màu tím là đường dẫn từ điểm bắt đầu màu xanh lá cây đến mục tiêu màu đỏ, và các chấm nhỏ là các trạng thái đã được mỗi tìm kiếm đạt đến. Đối với bài toán cụ thể này, A* có trọng số khám phá ít trạng thái hơn 7 lần và tìm thấy một đường dẫn có chi phí cao hơn 5%.

Hình 3.21 cho thấy một bài toán tìm kiếm trên một thế giới lưới. Trong (a), một tìm kiếm A* tìm thấy giải pháp tối ưu, nhưng phải khám phá một phần lớn của không gian trạng thái để tìm thấy nó. Trong (b), một tìm kiếm A* có trọng số tìm thấy một giải pháp có chi phí cao hơn một chút, nhưng thời gian tìm kiếm nhanh hơn nhiều. Chúng ta thấy rằng tìm kiếm có trọng số tập trung đường đồng mức của các trạng thái đã đạt đến về phía một mục tiêu. Điều đó có nghĩa là ít trạng thái được khám phá hơn, nhưng nếu đường dẫn tối ưu từng đi ra ngoài đường đồng mức của tìm kiếm có trọng số (như trong trường hợp này), thì đường dẫn tối ưu sẽ không được tìm thấy. Nói chung, nếu giải pháp tối ưu có chi phí C^* , một tìm kiếm A* có trọng số sẽ tìm thấy một giải pháp có chi phí nằm đâu đó giữa C^* và $W \times C^*$; nhưng trong thực tế chúng ta thường nhận được kết quả gần C^* hơn nhiều so với $W \times C^*$. Chúng ta đã xem xét các tìm kiếm đánh giá các trạng thái bằng cách kết hợp g và h theo nhiều cách khác nhau; A* có trọng số có thể được xem là một sự khái quát hóa của những cái khác:

- Tìm kiếm A*: $g(n) + h(n)$ ($W = 1$)
- Tìm kiếm chi phí đồng đều: $g(n)$ ($W = 0$)
- Tìm kiếm tốt nhất đầu tiên tham lam: $h(n)$ ($W = \infty$)
- Tìm kiếm A* có trọng số: $g(n) + W \times h(n)$ ($1 < W < \infty$)

Bạn có thể gọi A* có trọng số là “tìm kiếm hơi tham lam”: giống như tìm kiếm tốt nhất đầu tiên tham lam, nó tập trung tìm kiếm về phía một mục tiêu; mặt khác, nó sẽ không bỏ qua chi phí đường dẫn hoàn toàn và sẽ đình chỉ một đường dẫn đang tiến triển ít với chi phí lớn.

Có nhiều thuật toán tìm kiếm dưới tối ưu khác nhau, có thể được đặc trưng bởi các tiêu chí về những gì được coi là “đủ tốt”. Trong **tìm kiếm dưới tối ưu có giới hạn** (bounded suboptimal search), chúng ta tìm kiếm một giải pháp được đảm bảo nằm trong một hằng số W của chi phí tối ưu. A* có trọng số cung cấp sự đảm bảo này. Trong **tìm kiếm chi phí có giới hạn** (bounded-cost search), chúng ta tìm kiếm một giải pháp có chi phí nhỏ hơn một hằng số C . Và trong **tìm kiếm chi phí không có giới hạn** (unbounded-cost search), chúng ta chấp nhận một giải pháp có bất kỳ chi phí nào, miễn là chúng ta có thể tìm thấy nó nhanh chóng.

Một ví dụ về thuật toán tìm kiếm chi phí không có giới hạn là **tìm kiếm nhanh** (speedy search), là một phiên bản của tìm kiếm tốt nhất đầu tiên tham lam sử dụng làm heuristic số hành động ước tính cần thiết để đạt đến một mục tiêu, bất kể chi phí của những hành động đó. Do đó, đối với các bài toán mà tất cả các hành động có cùng chi phí, nó giống như tìm kiếm tốt nhất đầu tiên tham lam, nhưng khi các hành động có chi phí khác nhau, nó có xu hướng dẫn tìm kiếm tìm thấy một giải pháp nhanh chóng, ngay cả khi nó có thể có chi phí cao.

3.5.5 Tìm kiếm có giới hạn bộ nhớ

Vấn đề chính với A* là việc sử dụng bộ nhớ của nó. Trong phần này, chúng ta sẽ đề cập đến một số thủ thuật triển khai giúp tiết kiệm không gian, và sau đó là một số thuật toán hoàn toàn mới tận dụng tốt hơn không gian có sẵn.

Bộ nhớ được chia giữa biên và các trạng thái đã đạt đến. Trong việc triển khai tìm kiếm tốt nhất đầu tiên của chúng ta, một trạng thái nằm trên biên được lưu trữ ở hai nơi: là một nút trong biên (để chúng ta có thể quyết định mở rộng nút nào tiếp theo) và là một mục nhập trong bảng các trạng thái đã đạt đến (để chúng ta biết liệu chúng ta đã ghé thăm trạng thái đó trước đây chưa). Đối với nhiều bài toán (chẳng hạn như khám phá một lưới), việc trùng lặp này không phải là một vấn đề, vì kích thước của biên nhỏ hơn nhiều so với **đã đạt đến**, vì vậy việc trùng lặp các trạng thái trong biên đòi hỏi một lượng bộ nhớ tương đối nhỏ. Nhưng một số triển khai chỉ giữ một trạng thái ở một trong hai nơi, tiết kiệm một chút không gian với chi phí làm phức tạp (và có lẽ làm chậm) thuật toán.

Một khả năng khác là xóa các trạng thái khỏi **đã đạt đến** khi chúng ta có thể chứng minh rằng chúng không còn cần thiết nữa. Đối với một số bài toán, chúng ta có thể sử dụng thuộc tính tách (separation property) (Hình 3.6 trên trang 90), cùng với việc cấm các hành động quay đầu, để đảm bảo rằng tất cả các hành động hoặc di chuyển ra ngoài biên hoặc vào một trạng thái biên khác. Trong trường hợp đó, chúng ta chỉ cần kiểm tra biên để tìm các đường dẫn dư thừa và chúng ta có thể loại bỏ bảng **đã đạt đến**.

Đối với các bài toán khác, chúng ta có thể giữ **số lượng tham chiếu** (reference counts) về số lần một trạng thái đã được đạt đến và xóa nó khỏi bảng **đã đạt đến** khi không còn cách nào để đạt đến trạng thái đó nữa. Ví dụ, trên một thế giới lưới nơi mỗi trạng thái chỉ có thể được đạt đến từ bốn nút lân cận của nó, một khi chúng ta đã đạt đến một trạng thái bốn lần, chúng ta có thể xóa nó khỏi bảng.

Bây giờ chúng ta hãy xem xét các thuật toán mới được thiết kế để bảo tồn việc sử dụng bộ nhớ.

Tìm kiếm chùm tia (Beam search) giới hạn kích thước của biên. Cách tiếp cận dễ nhất là chỉ giữ k nút có điểm f tốt nhất, loại bỏ bất kỳ nút mở rộng nào khác. Điều này tất nhiên làm cho tìm kiếm không đầy đủ và dưới tối ưu, nhưng chúng ta có thể chọn k để sử dụng tốt bộ nhớ có sẵn và thuật toán thực thi nhanh vì nó mở rộng ít nút hơn. Đối với nhiều bài toán, nó có thể tìm thấy các giải pháp tốt gần tối ưu. Bạn có thể nghĩ tìm kiếm chi phí đồng đều hoặc A^* là lan ra khắp nơi trong các đường đồng mức đồng tâm, và nghĩ tìm kiếm chùm tia là chỉ khám phá một phần tập trung của các đường đồng mức đó, phần chứa k ứng cử viên tốt nhất.

Một phiên bản thay thế của tìm kiếm chùm tia không giữ một giới hạn nghiêm ngặt về kích thước của biên mà thay vào đó giữ mọi nút có điểm f nằm trong δ của điểm f tốt nhất. Bằng cách đó, khi có một vài nút có điểm mạnh, chỉ một vài nút sẽ được giữ lại, nhưng nếu không có nút mạnh nào, nhiều nút hơn sẽ được giữ lại cho đến khi một nút mạnh xuất hiện.

```
function RECURSIVE-BEST-FIRST-SEARCH(problem) returns a solution or failure
    solution, fvalue ← RBFS(problem, NODE(problem.INITIAL),  $\infty$ )
    return solution

function RBFS(problem, node, f limit) returns a solution or failure, and a new  $f$ -cost limit
    if problem.IS-GOAL(node.STATE) then return node
    successors ← LIST(EXPAND(node))
    if successors is empty then return failure,  $\infty$ 
    for each s in successors do // update  $f$  with value from previous search
        s.f ← max(s.PATH-COST +  $h(s)$ , node.f)
    while true do
        best ← the node in successors with lowest  $f$ -value
        if best.f > f limit then return failure, best.f
        alternative ← the second-lowest  $f$ -value among successors
        result, best.f ← RBFS(problem, best, min(f limit, alternative))
        if result ≠ failure then return result, best.f
```

Hình 3.22 Thuật toán cho tìm kiếm tốt nhất đầu tiên đệ quy.

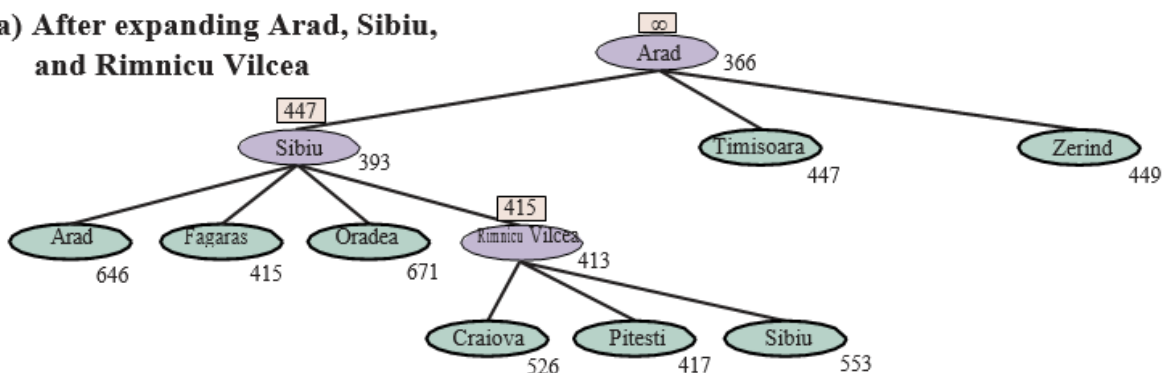
Tìm kiếm A* lặp sâu dần (IDA*) (Iterative-deepening A* search) đối với A^* cũng giống như tìm kiếm lặp sâu dần đối với tìm kiếm theo chiều sâu đầu tiên: IDA* mang lại cho chúng ta những lợi ích của A^* mà không yêu cầu giữ tất cả các trạng thái đã đạt đến trong bộ nhớ, với chi phí là ghé thăm một số trạng thái nhiều lần. Đây là một thuật toán rất quan trọng và thường được sử dụng cho các bài toán không vừa với bộ nhớ.

Trong tìm kiếm lặp sâu dần tiêu chuẩn, giới hạn là độ sâu, được tăng thêm một mỗi lần lặp. Trong IDA*, giới hạn là chi phí $f(g + h)$; ở mỗi lần lặp, giá trị giới hạn là chi phí f nhỏ nhất của bất kỳ nút nào vượt quá giới hạn ở lần lặp trước đó. Nói cách khác, mỗi lần lặp tìm kiếm một cách cạn kiệt một đường đồng mức f , tìm một nút ngay bên ngoài đường đồng mức đó và sử dụng chi phí f của nút đó làm đường đồng mức tiếp theo. Đối với các bài toán như 8-puzzle, nơi chi phí f của mỗi đường dẫn là một số nguyên, điều này hoạt động rất tốt, dẫn đến tiến trình ổn định về phía

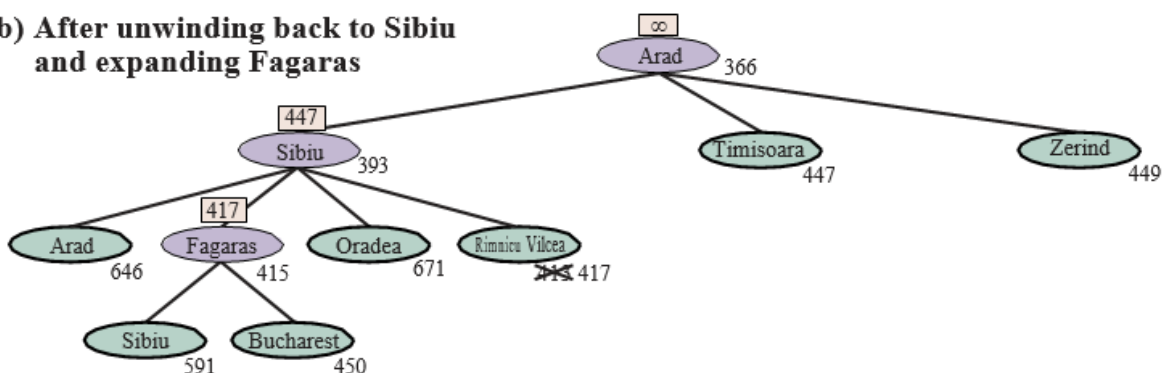
mục tiêu mỗi lần lặp. Nếu giải pháp tối ưu có chi phí C^* , thì không thể có nhiều hơn C^* lần lặp (ví dụ, không quá 31 lần lặp trên các bài toán 8-puzzle khó nhất). Nhưng đối với một bài toán mà mỗi nút có một chi phí f khác nhau, mỗi đường đồng mức mới có thể chỉ chứa một nút mới, và số lần lặp có thể bằng số trạng thái.

Tìm kiếm tốt nhất đầu tiên đệ quy (RBFS) (Recursive best-first search) (Hình 3.22) cố gắng bắt chước hoạt động của tìm kiếm tốt nhất đầu tiên tiêu chuẩn, nhưng chỉ sử dụng không gian tuyến tính. RBFS giống với một tìm kiếm theo chiều sâu đầu tiên đệ quy, nhưng thay vì tiếp tục vô thời hạn xuống đường dẫn hiện tại, nó sử dụng biến **flimit** để theo dõi giá trị f của đường dẫn thay thế tốt nhất có sẵn từ bất kỳ tổ tiên nào của nút hiện tại. Nếu nút hiện tại vượt quá giới hạn này, đệ quy sẽ quay lại đường dẫn thay thế. Khi đệ quy quay lại, RBFS thay thế giá trị f của mỗi nút dọc theo đường dẫn bằng một **giá trị được sao lưu** (backed-up value)—giá trị f tốt nhất của các nút con của nó. Bằng cách này, RBFS ghi nhớ giá trị f của lá tốt nhất trong cây con bị lãng quên và do đó có thể quyết định xem có đáng để mở rộng lại cây con đó vào một thời điểm sau đó hay không. Hình 3.23 cho thấy cách RBFS đến Bucharest. RBFS hiệu quả hơn một chút so với IDA*, nhưng vẫn bị tái tạo nút quá mức. Trong ví dụ trong Hình 3.23, RBFS đi theo đường dẫn qua Rimnicu Vilcea, sau đó “thay đổi ý định” và thử Fagaras, và sau đó lại thay đổi ý định trở lại. Những thay đổi ý định này xảy ra bởi vì mỗi khi đường dẫn tốt nhất hiện tại được mở rộng, giá trị f của nó có khả năng tăng lên— h thường ít lạc quan hơn đối với các nút gần mục tiêu hơn. Khi điều này xảy ra, đường dẫn tốt thứ hai có thể trở thành đường dẫn tốt nhất, vì vậy tìm kiếm phải quay lui để đi theo nó. Mỗi thay đổi ý định tương ứng với một lần lặp của IDA* và có thể yêu cầu nhiều lần tái mở rộng các nút bị lãng quên để tạo lại đường dẫn tốt nhất và mở rộng nó thêm một nút nữa.

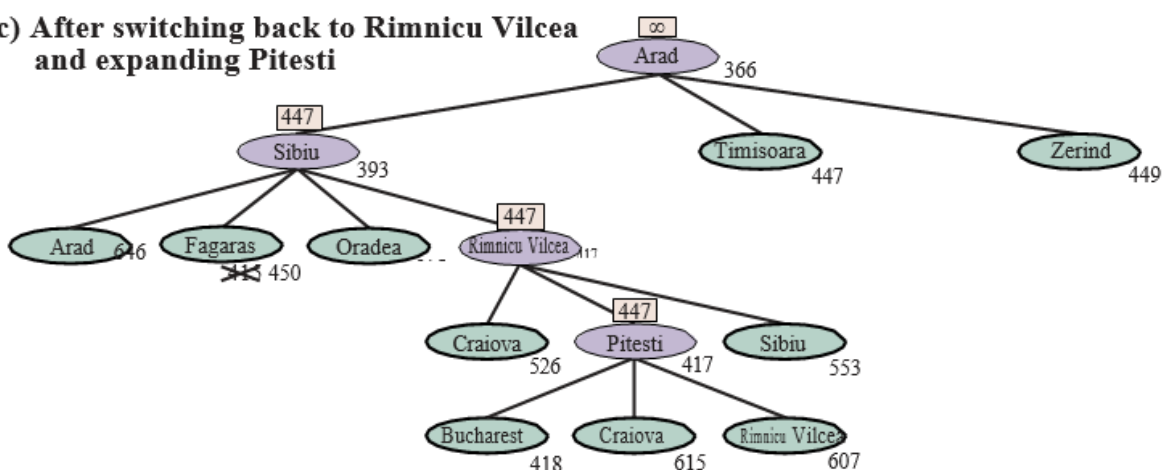
(a) After expanding Arad, Sibiu, and Rimnicu Vilcea



(b) After unwinding back to Sibiu and expanding Fagaras



(c) After switching back to Rimnicu Vilcea and expanding Pitesti



Hình 3.23 Các giai đoạn trong một tìm kiếm RBFS cho tuyến đường ngắn nhất đến Bucharest. Giá trị giới hạn f cho mỗi lời gọi đệ quy được hiển thị trên mỗi nút hiện tại, và mỗi nút được gắn nhãn với chi phí f của nó. (a) Đường dẫn qua Rimnicu Vilcea được đi theo cho đến khi lá tốt nhất hiện tại (Pitesti) có giá trị tệ hơn đường dẫn thay thế tốt nhất (Fagaras). (b) Đệ quy quay lại và giá trị lá tốt nhất của cây con bị lãng quên (417) được sao lưu đến Rimnicu Vilcea; sau đó Fagaras được mở rộng, tiết lộ một giá trị lá tốt nhất là 450. (c) Đệ quy quay lại và giá trị lá tốt nhất của cây con bị lãng quên (450) được sao lưu đến Fagaras; sau đó Rimnicu Vilcea được mở rộng. Lần này, vì đường dẫn thay thế tốt nhất (qua Timisoara) có chi phí ít nhất là 447, việc mở rộng tiếp tục đến Bucharest.

RBFS là tối ưu nếu hàm heuristic $h(n)$ là chấp nhận được. Độ phức tạp không gian của nó là tuyến tính theo độ sâu của giải pháp tối ưu sâu nhất, nhưng độ phức tạp thời gian của nó khá khó để đặc trưng: nó phụ thuộc cả vào độ chính xác của hàm heuristic và tần suất đường dẫn tốt nhất thay đổi khi các nút được mở rộng. Nó mở rộng các nút theo thứ tự chi phí f tăng dần, ngay cả khi f không đơn điệu.

IDA* và RBFS gặp vấn đề do sử dụng quá ít bộ nhớ. Giữa các lần lặp, IDA* chỉ giữ lại một số duy nhất: giới hạn chi phí f hiện tại. RBFS giữ lại nhiều thông tin hơn trong bộ nhớ, nhưng nó chỉ sử dụng không gian tuyến tính: ngay cả khi có nhiều bộ nhớ hơn, RBFS không có cách nào để sử dụng nó. Vì chúng quên hầu hết những gì chúng đã làm, cả hai thuật toán có thể kết thúc việc khám phá lại cùng một trạng thái nhiều lần.

Do đó, có vẻ hợp lý khi xác định chúng ta có bao nhiêu bộ nhớ, và cho phép một thuật toán sử dụng tất cả bộ nhớ đó. Hai thuật toán làm điều này là **MA*** (memory-bounded A*) và **SMA*** (simplified MA*). **SMA*** — tốt — đơn giản hơn, vì vậy chúng ta sẽ mô tả nó. SMA* tiến hành giống như A*, mở rộng lá tốt nhất cho đến khi bộ nhớ đầy. Tại thời điểm này, nó không thể thêm một nút mới vào cây tìm kiếm mà không bỏ một nút cũ. SMA* luôn bỏ nút lá tệ nhất—nút có giá trị f cao nhất. Giống như RBFS, SMA* sau đó sao lưu giá trị của nút bị lãng quên đến nút cha của nó. Bằng cách này, tổ tiên của một cây con bị lãng quên biết được chất lượng của đường dẫn tốt nhất trong cây con đó. Với thông tin này, SMA* chỉ tạo lại cây con khi tất cả các đường dẫn khác đã được chứng minh là trông tệ hơn đường dẫn mà nó đã quên. Một cách khác để nói điều này là nếu tất cả các hậu duệ của một nút n bị lãng quên, thì chúng ta sẽ không biết phải đi đường nào từ n , nhưng chúng ta vẫn sẽ có ý tưởng về mức độ đáng giá để đi bất cứ đâu từ n .

Thuật toán hoàn chỉnh được mô tả trong kho mã trực tuyến đi kèm với cuốn sách này. Có một điểm tinh tế đáng nói. Chúng ta đã nói rằng SMA* mở rộng lá tốt nhất và xóa lá tệ nhất. Điều gì sẽ xảy ra nếu tất cả các nút lá đều có cùng giá trị f ? Để tránh chọn cùng một nút để xóa và mở rộng, SMA* mở rộng lá tốt nhất mới nhất và xóa lá tệ nhất cũ nhất. Những điều này trùng nhau khi chỉ có một lá, nhưng trong trường hợp đó, cây tìm kiếm hiện tại phải là một đường dẫn duy nhất từ gốc đến lá lấp đầy toàn bộ bộ nhớ. Nếu lá không phải là nút mục tiêu, thì ngay cả khi nó nằm trên một đường dẫn giải pháp tối ưu, giải pháp đó không thể đạt được với bộ nhớ có sẵn. Do đó, nút có thể bị loại bỏ chính xác như thể nó không có nút kế tiếp.

SMA* là đầy đủ nếu có bất kỳ giải pháp nào có thể đạt được—tức là, nếu d , độ sâu của nút mục tiêu nông nhất, nhỏ hơn kích thước bộ nhớ (được biểu thị bằng các nút). Nó là tối ưu nếu bất kỳ giải pháp tối ưu nào có thể đạt được; nếu không, nó trả về giải pháp tốt nhất có thể đạt được. Trong thực tế, SMA* là một lựa chọn khá mạnh mẽ để tìm các giải pháp tối ưu, đặc biệt là khi không gian trạng thái là một đồ thị, chi phí hành động không đồng nhất và việc tạo nút tốn kém so với chi phí duy trì biên và tập hợp **đã đạt đến**.

Tuy nhiên, trên các bài toán rất khó, thường thì SMA* sẽ bị buộc phải liên tục chuyển đổi qua lại giữa nhiều đường dẫn giải pháp ứng cử viên, chỉ một tập hợp con nhỏ trong số đó có thể vừa với bộ nhớ. (Điều này giống với vấn đề **thrashing** trong các hệ thống phân trang đĩa.) Khi đó, thời gian bổ sung cần thiết cho việc tái tạo lặp đi lặp lại cùng một nút có nghĩa là các bài toán có thể giải quyết được trên thực tế bằng A*, với bộ nhớ không giới hạn, trở nên không thể giải quyết được đối với SMA*. Tức là, các giới hạn bộ nhớ có thể làm cho một bài toán trở nên không thể giải quyết được từ quan điểm của thời gian tính toán. Mặc dù không có lý thuyết hiện tại nào giải

thích sự đánh đổi giữa thời gian và bộ nhớ, dường như đây là một vấn đề không thể tránh khỏi. Cách thoát duy nhất là từ bỏ yêu cầu tối ưu.

3.5.6 Tìm kiếm heuristic hai chiều

Với tìm kiếm tốt nhất đầu tiên một chiều, chúng ta đã thấy rằng việc sử dụng $f(n) = g(n) + h(n)$ làm hàm đánh giá cho chúng ta một tìm kiếm A* được đảm bảo tìm thấy các giải pháp có chi phí tối ưu (giả sử h là chấp nhận được) trong khi vẫn hiệu quả tối ưu về số nút được mở rộng.

Với tìm kiếm tốt nhất đầu tiên hai chiều, chúng ta cũng có thể thử sử dụng $f(n) = g(n) + h(n)$, nhưng thật không may, không có gì đảm bảo rằng điều này sẽ dẫn đến một giải pháp có chi phí tối ưu, cũng như không hiệu quả tối ưu, ngay cả với một heuristic chấp nhận được. Với tìm kiếm hai chiều, hóa ra không phải các nút riêng lẻ mà là các cặp nút (một từ mỗi biên) có thể được chứng minh là chắc chắn được mở rộng, vì vậy bất kỳ bằng chứng nào về hiệu quả sẽ phải xem xét các cặp nút (Eckerle et al., 2017).

Chúng ta sẽ bắt đầu với một số ký hiệu mới. Chúng ta sử dụng $f_F(n) = g_F(n) + h_F(n)$ cho các nút đi theo hướng tiến (với trạng thái ban đầu là gốc) và $f_B(n) = g_B(n) + h_B(n)$ cho các nút đi theo hướng lùi (với một trạng thái mục tiêu là gốc). Mặc dù cả tìm kiếm tiến và lùi đều đang giải quyết cùng một vấn đề, chúng có các hàm đánh giá khác nhau vì, ví dụ, các heuristic khác nhau tùy thuộc vào việc bạn đang cố gắng đạt được mục tiêu hay trạng thái ban đầu. Chúng ta sẽ giả sử các heuristic là chấp nhận được.

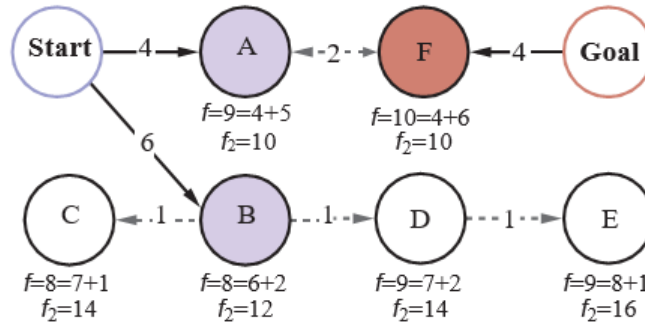
Xem xét một đường dẫn tiến từ trạng thái ban đầu đến một nút m và một đường dẫn lùi từ mục tiêu đến một nút n . Chúng ta có thể định nghĩa một cận dưới về chi phí của một giải pháp đi theo đường dẫn từ trạng thái ban đầu đến m , sau đó bằng cách nào đó đến n , sau đó đi theo đường dẫn đến mục tiêu là:

$$lb(m, n) = \max(g_F(m) + g_B(n), f_F(m), f_B(n))$$

Nói cách khác, chi phí của một đường dẫn như vậy phải ít nhất là tổng chi phí đường dẫn của hai phần (vì kết nối còn lại giữa chúng phải có chi phí không âm), và chi phí cũng phải ít nhất bằng chi phí f ước tính của một trong hai phần (vì các ước tính heuristic là lạc quan). Theo đó, định lý là đối với bất kỳ cặp nút m, n nào có $lb(m, n)$ nhỏ hơn chi phí tối ưu C^* , chúng ta phải mở rộng hoặc m hoặc n , bởi vì đường dẫn đi qua cả hai chúng là một giải pháp tối ưu tiềm năng. Khó khăn là chúng ta không biết chắc chắn nút nào là tốt nhất để mở rộng, và do đó không có thuật toán tìm kiếm hai chiều nào có thể được đảm bảo là hiệu quả tối ưu—bất kỳ thuật toán nào cũng có thể mở rộng lên đến gấp đôi số nút tối thiểu nếu nó luôn chọn thành viên sai của một cặp để mở rộng trước. Một số thuật toán tìm kiếm heuristic hai chiều quản lý rõ ràng một hàng đợi các cặp (m, n) , nhưng chúng ta sẽ gắn bó với tìm kiếm tốt nhất đầu tiên hai chiều (Hình 3.14), có hai hàng đợi ưu tiên biên và đưa cho nó một hàm đánh giá bất chước tiêu chí lb :

$$f_2(n) = \max(2g(n), g(n) + h(n))$$

Nút được mở rộng tiếp theo sẽ là nút giảm thiểu giá trị f_2 này; nút có thể đến từ một trong hai biên. Hàm f_2 này đảm bảo rằng chúng ta sẽ không bao giờ mở rộng một nút (từ bất kỳ biên nào) với $g(n) > C^*$. Chúng ta nói rằng hai nửa của tìm kiếm “gặp nhau ở giữa” theo nghĩa là khi hai biên chạm vào nhau, không có nút nào bên trong một trong hai biên có chi phí đường dẫn lớn hơn giới hạn C^* . Hình 3.24 giải thích một ví dụ về tìm kiếm hai chiều.



Hình 3.24 Tìm kiếm hai chiều duy trì hai biên: ở bên trái, các nút A và B là các nút kế tiếp của Bắt đầu; ở bên phải, nút F là một nút kế tiếp ngược của Mục tiêu. Mỗi nút được gắn nhãn với các giá trị $f = g + h$ và giá trị $f_2 = \max(2g, g + h)$. (Các giá trị g là tổng của các chi phí hành động như được hiển thị trên mỗi mũi tên; các giá trị h là tùy ý và không thể suy ra từ bất cứ điều gì trong hình.) Giải pháp tối ưu, Bắt đầu-A-F-Mục tiêu, có chi phí $C^* = 4 + 2 + 4 = 10$, vì vậy điều đó có nghĩa là một thuật toán hai chiều gặp nhau ở giữa không nên mở rộng bất kỳ nút nào có $g > C^* = 5$; và quả thực nút tiếp theo được mở rộng sẽ là A hoặc F (mỗi nút có $g = 4$), dẫn chúng ta đến một giải pháp tối ưu. Nếu chúng ta mở rộng nút có chi phí f thấp nhất trước, thì B và C sẽ đến tiếp theo, và D và E sẽ bằng A, nhưng tất cả chúng đều có $g > C^*$ và do đó không bao giờ được mở rộng khi f_2 là hàm đánh giá.

Chúng ta đã mô tả một cách tiếp cận trong đó heuristic h_F ước tính khoảng cách đến mục tiêu (hoặc, khi bài toán có nhiều trạng thái mục tiêu, khoảng cách đến mục tiêu gần nhất) và h_B ước tính khoảng cách đến điểm bắt đầu. Đây được gọi là **tìm kiếm từ đầu đến cuối** (front-to-end search). Một cách thay thế, được gọi là **tìm kiếm từ biên đến biên** (front-to-front search), cố gắng ước tính khoảng cách đến biên kia. Rõ ràng, nếu một biên có hàng triệu nút, sẽ không hiệu quả khi áp dụng hàm heuristic cho tất cả chúng và lấy giá trị nhỏ nhất. Nhưng nó có thể hoạt động để lấy mẫu một vài nút từ biên. Trong một số miền bài toán cụ thể, có thể tóm tắt biên—ví dụ, trong một bài toán tìm kiếm lưới, chúng ta có thể tính toán một hộp giới hạn của biên một cách gia tăng và sử dụng khoảng cách đến hộp giới hạn làm heuristic.

Tìm kiếm hai chiều đôi khi hiệu quả hơn tìm kiếm một chiều, đôi khi không. Nói chung, nếu chúng ta có một heuristic rất tốt, thì tìm kiếm A* tạo ra các đường đồng mức tìm kiếm tập trung vào mục tiêu, và việc thêm tìm kiếm hai chiều không giúp ích nhiều. Với một heuristic trung bình, tìm kiếm hai chiều gặp nhau ở giữa có xu hướng mở rộng ít nút hơn và được ưu tiên. Trong trường hợp xấu nhất của một heuristic kém, tìm kiếm không còn tập trung vào mục tiêu, và tìm kiếm hai chiều có độ phức tạp tiệm cận tương tự như A*. Tìm kiếm hai chiều với hàm đánh giá f_2 và một heuristic h chấp nhận được là đầy đủ và tối ưu.

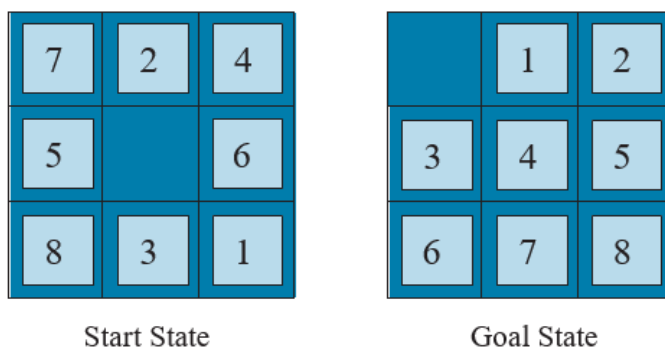
3.6 Các hàm Heuristic

Trong phần này, chúng ta xem xét cách độ chính xác của một heuristic ảnh hưởng đến hiệu suất tìm kiếm và cũng xem xét cách các heuristic có thể được phát minh. Là ví dụ chính, chúng ta sẽ

quay lại 8-puzzle. Như đã đề cập trong Phần 3.2, mục tiêu của trò chơi xếp hình là trượt các ô vuông theo chiều ngang hoặc chiều dọc vào khoảng trống cho đến khi cấu hình khớp với cấu hình mục tiêu (Hình 3.25).

Có $9!/2 = 181,400$ trạng thái có thể đạt được trong một 8-puzzle, vì vậy một tìm kiếm có thể dễ dàng giữ tất cả chúng trong bộ nhớ. Nhưng đối với 15-puzzle, có $16!/2$ trạng thái—hơn 10 nghìn tỷ—vì vậy để tìm kiếm không gian đó, chúng ta sẽ cần sự trợ giúp của một hàm heuristic chấp nhận được tốt. Có một lịch sử lâu dài về các heuristic như vậy cho 15-puzzle; dưới đây là hai ứng cử viên thường được sử dụng:

- **h_1 = số ô sai vị trí** (không bao gồm ô trống). Đối với Hình 3.25, cả tám ô đều sai vị trí, vì vậy trạng thái bắt đầu có $h_1 = 8$. h_1 là một heuristic chấp nhận được vì bất kỳ ô nào sai vị trí sẽ yêu cầu ít nhất một lần di chuyển để đưa nó đến đúng vị trí.



Hình 3.25 Một trường hợp điển hình của 8-puzzle. Giải pháp ngắn nhất dài 26 hành động.

- **h_2 = tổng khoảng cách của các ô so với vị trí mục tiêu của chúng.** Vì các ô không thể di chuyển theo đường chéo, khoảng cách là tổng của khoảng cách ngang và dọc—đôi khi được gọi là khoảng cách khối thành phố (city-block distance) hoặc khoảng cách Manhattan (Manhattan distance). h_2 cũng chấp nhận được vì tất cả những gì một lần di chuyển có thể làm là di chuyển một ô tiến gần hơn một bước đến mục tiêu. Các ô từ 1 đến 8 trong trạng thái bắt đầu của Hình 3.25 cho khoảng cách Manhattan là: $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$. Như mong đợi, không có heuristic nào trong số này ước tính quá cao chi phí giải pháp thực sự, là 26.

3.6.1 Ảnh hưởng của độ chính xác heuristic đến hiệu suất

Một cách để mô tả chất lượng của một heuristic là **hệ số phân nhánh hiệu quả b^*** . Nếu tổng số nút được tạo ra bởi A^* cho một bài toán cụ thể là N và độ sâu giải pháp là d , thì b^* là hệ số phân nhánh mà một cây đồng nhất có độ sâu d sẽ phải có để chứa $N + 1$ nút. Do đó,

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

Ví dụ, nếu A^* tìm thấy một giải pháp ở độ sâu 5 sử dụng 52 nút, thì hệ số phân nhánh hiệu quả là 1.92. Hệ số phân nhánh hiệu quả có thể thay đổi tùy theo các trường hợp bài toán, nhưng thường

đối với một miền cụ thể (chẳng hạn như 8-puzzles), nó khá ổn định trên tất cả các trường hợp bài toán không tầm thường. Do đó, các phép đo thực nghiệm của b^* trên một tập hợp nhỏ các bài toán có thể cung cấp một hướng dẫn tốt về tính hữu ích tổng thể của heuristic. Một heuristic được thiết kế tốt sẽ có giá trị b^* gần 1, cho phép các bài toán khá lớn được giải quyết với chi phí tính toán hợp lý. Korf và Reid (1998) lập luận rằng một cách tốt hơn để mô tả ảnh hưởng của việc cắt tỉa A^* với một heuristic h đã cho là nó làm giảm **độ sâu hiệu quả** (effective depth) bằng một hằng số k_h so với độ sâu thực tế. Điều này có nghĩa là tổng chi phí tìm kiếm là $O(b^{d-k_h})$ so với $O(b^d)$ đối với một tìm kiếm không có thông tin. Các thí nghiệm của họ trên Rubik's Cube và các bài toán n -puzzle cho thấy rằng công thức này đưa ra các dự đoán chính xác cho tổng chi phí tìm kiếm cho các trường hợp bài toán được lấy mẫu trên một phạm vi rộng các độ dài giải pháp—ít nhất là đối với các độ dài giải pháp lớn hơn k_h .

Đối với Hình 3.26, chúng tôi đã tạo các bài toán 8-puzzle ngẫu nhiên và giải chúng bằng tìm kiếm theo chiều rộng đầu tiên không có thông tin và với tìm kiếm A^* sử dụng cả h_1 và h_2 , báo cáo số nút trung bình được tạo ra và hệ số phân nhánh hiệu quả tương ứng cho mỗi chiến lược tìm kiếm và cho mỗi độ dài giải pháp. Kết quả cho thấy h_2 tốt hơn h_1 và cả hai đều tốt hơn so với không có heuristic nào cả.

d	Search Cost (nodes generated)			Effective Branching Factor		
	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Hình 3.26 So sánh chi phí tìm kiếm và hệ số phân nhánh hiệu quả cho các bài toán 8-puzzle sử dụng tìm kiếm theo chiều rộng đầu tiên, A^* với h_1 (các ô sai vị trí) và A^* với h_2 (khoảng cách Manhattan). Dữ liệu được lấy trung bình trên 100 câu đố cho mỗi độ dài giải pháp d từ 6 đến 28.

Người ta có thể hỏi liệu h_2 có luôn tốt hơn h_1 hay không. Câu trả lời là “Về cơ bản là có.” Dễ dàng thấy từ các định nghĩa của hai heuristic rằng đối với bất kỳ nút n nào, $h_2(n) \geq h_1(n)$. Do đó, chúng ta nói rằng h_2 **thống trị** (dominates) h_1 . Sự thống trị này chuyển trực tiếp thành hiệu quả: A^* sử dụng h_2 sẽ không bao giờ mở rộng nhiều nút hơn A^* sử dụng h_1 (ngoại trừ trong trường hợp không may mắn phá vỡ sự cân bằng). Lập luận rất đơn giản. Nhớ lại quan sát trên trang 108 rằng mọi nút có $f(n) < C^*$ chắc chắn sẽ được mở rộng. Điều này giống như nói rằng mọi nút có

$h(n) < C^* - g(n)$ chắc chắn được mở rộng khi h nhất quán. Nhưng vì h_2 lớn ít nhất bằng h_1 đối với tất cả các nút, mọi nút chắc chắn được mở rộng bởi tìm kiếm A^* với h_2 cũng chắc chắn được mở rộng với h_1 , và h_1 có thể khiến các nút khác cũng được mở rộng. Do đó, nhìn chung tốt hơn là sử dụng một hàm heuristic có giá trị cao hơn, miễn là nó nhất quán và thời gian tính toán cho heuristic không quá dài.

3.6.2 Tạo Heuristic từ các bài toán đã được nói lỏng

Chúng ta đã thấy rằng cả h_1 (số ô sai vị trí) và h_2 (khoảng cách Manhattan) đều là các heuristic khá tốt cho 8-puzzle và h_2 thì tốt hơn. Làm thế nào người ta có thể nghĩ ra h_2 ? Liệu một máy tính có thể tự động phát minh ra một heuristic như vậy không?

h_1 và h_2 là các ước tính về độ dài đường dẫn còn lại cho 8-puzzle, nhưng chúng cũng là độ dài đường dẫn hoàn toàn chính xác cho các phiên bản đơn giản hóa của câu đố. Nếu các quy tắc của câu đố được thay đổi sao cho một ô có thể di chuyển đến bất cứ đâu thay vì chỉ đến ô trống liền kề, thì h_1 sẽ đưa ra độ dài chính xác của giải pháp ngắn nhất. Tương tự, nếu một ô có thể di chuyển một ô theo bất kỳ hướng nào, ngay cả trên một ô đã có người, thì h_2 sẽ đưa ra độ dài chính xác của giải pháp ngắn nhất. Một bài toán có ít hạn chế hơn về các hành động được gọi là một **bài toán đã được nói lỏng** (relaxed problem). Đồ thị không gian trạng thái của bài toán đã được nói lỏng là một siêu đồ thị của không gian trạng thái ban đầu vì việc loại bỏ các hạn chế tạo ra các cạnh bổ sung trong đồ thị.

Vì bài toán đã được nói lỏng thêm các cạnh vào đồ thị không gian trạng thái, nên bất kỳ giải pháp tối ưu nào trong bài toán gốc, theo định nghĩa, cũng là một giải pháp trong bài toán đã được nói lỏng; nhưng bài toán đã được nói lỏng có thể có các giải pháp tốt hơn nếu các cạnh được thêm vào cung cấp các lối tắt. Do đó, chi phí của một giải pháp tối ưu cho một bài toán đã được nói lỏng là một heuristic chấp nhận được cho bài toán gốc. Hơn nữa, vì heuristic được suy ra là một chi phí chính xác cho bài toán đã được nói lỏng, nó phải tuân theo bất đẳng thức tam giác và do đó là nhất quán (xem trang 106).

Nếu một định nghĩa bài toán được viết bằng một ngôn ngữ hình thức, có thể tự động xây dựng các bài toán đã được nói lỏng. Ví dụ, nếu các hành động của 8-puzzle được mô tả là: Một ô có thể di chuyển từ ô X đến ô Y nếu X liền kề với Y và Y trống. Chúng ta có thể tạo ra ba bài toán đã được nói lỏng bằng cách loại bỏ một hoặc cả hai điều kiện: (a) Một ô có thể di chuyển từ ô X đến ô Y nếu X liền kề với Y. (b) Một ô có thể di chuyển từ ô X đến ô Y nếu Y trống. (c) Một ô có thể di chuyển từ ô X đến ô Y.

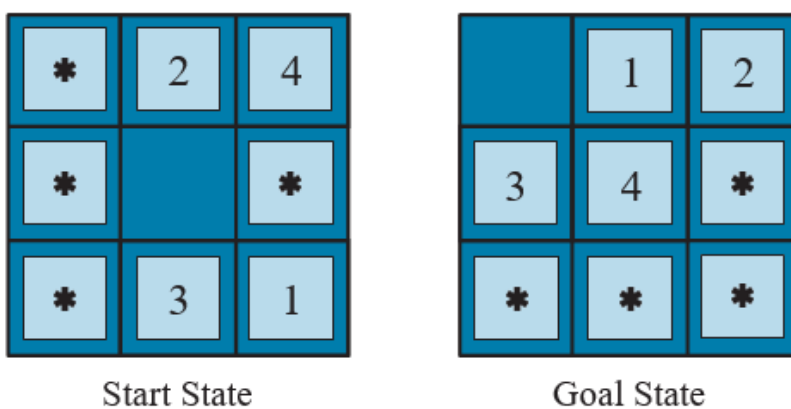
Từ (a), chúng ta có thể suy ra h_2 (khoảng cách Manhattan). Lập luận là h_2 sẽ là điểm số thích hợp nếu chúng ta lần lượt di chuyển từng ô đến đích của nó. Heuristic được suy ra từ (b) được thảo luận trong Bài tập 3.GASC. Từ (c), chúng ta có thể suy ra h_1 (các ô sai vị trí) vì nó sẽ là điểm số thích hợp nếu các ô có thể di chuyển đến đích dự định của chúng trong một hành động. Lưu ý rằng điều quan trọng là các bài toán đã được nói lỏng được tạo ra bằng kỹ thuật này có thể được giải quyết về cơ bản mà không cần tìm kiếm, bởi vì các quy tắc nói lỏng cho phép bài toán được phân tách thành tám **bài toán con** (subproblem) độc lập. Nếu bài toán đã được nói lỏng khó giải quyết, thì các giá trị của heuristic tương ứng sẽ tồn kém để có được.

Một chương trình có tên ABSOLVER có thể tự động tạo ra các heuristic từ các định nghĩa bài toán, sử dụng phương pháp “bài toán đã được nói lỏng” và các kỹ thuật khác nhau (Prieditis, 1993). ABSOLVER đã tạo ra một heuristic mới cho 8-puzzle tốt hơn bất kỳ heuristic hiện có nào và tìm thấy heuristic hữu ích đầu tiên cho câu đố Rubik’s Cube nổi tiếng.

Nếu một bộ sưu tập các heuristic chấp nhận được $h_1 \dots h_m$ có sẵn cho một bài toán và không có heuristic nào trong số chúng rõ ràng là tốt hơn những heuristic khác, chúng ta nên chọn cái nào? Hóa ra, chúng ta có thể có điều tốt nhất của tất cả các thế giới, bằng cách định nghĩa: $h(n) = \max\{h_1(n), \dots, h_m(n)\}$. Heuristic tổng hợp này chọn hàm nào chính xác nhất trên nút đang được xem xét. Vì các thành phần h_i là chấp nhận được, h là chấp nhận được (và nếu tất cả h_i đều nhất quán, h là nhất quán). Hơn nữa, h **thống trị** (dominates) tất cả các heuristic thành phần của nó. Nhược điểm duy nhất là $h(n)$ mất nhiều thời gian hơn để tính toán. Nếu đó là một vấn đề, một lựa chọn thay thế là chọn ngẫu nhiên một trong các heuristic ở mỗi lần đánh giá hoặc sử dụng một thuật toán học máy để dự đoán heuristic nào sẽ tốt nhất. Làm như vậy có thể dẫn đến một heuristic không nhất quán (ngay cả khi mọi h_i đều nhất quán), nhưng trong thực tế, nó thường dẫn đến việc giải quyết vấn đề nhanh hơn.

3.6.3 Tạo Heuristic từ các bài toán con: Cơ sở dữ liệu mẫu

Các heuristic chấp nhận được cũng có thể được suy ra từ chi phí giải pháp của một bài toán con của một bài toán đã cho. Ví dụ, Hình 3.27 cho thấy một bài toán con của trường hợp 8-puzzle trong Hình 3.25. Bài toán con này liên quan đến việc đưa các ô 1, 2, 3, 4 và ô trống vào đúng vị trí của chúng. Rõ ràng, chi phí của giải pháp tối ưu của bài toán con này là một cận dưới về chi phí của bài toán hoàn chỉnh. Nó hóa ra chính xác hơn khoảng cách Manhattan trong một số trường hợp.



Hình 3.27 Một bài toán con của trường hợp 8-puzzle được đưa ra trong Hình 3.25. Nhiệm vụ là đưa các ô 1, 2, 3, 4 và ô trống vào đúng vị trí của chúng, mà không cần lo lắng về những gì xảy ra với các ô khác.

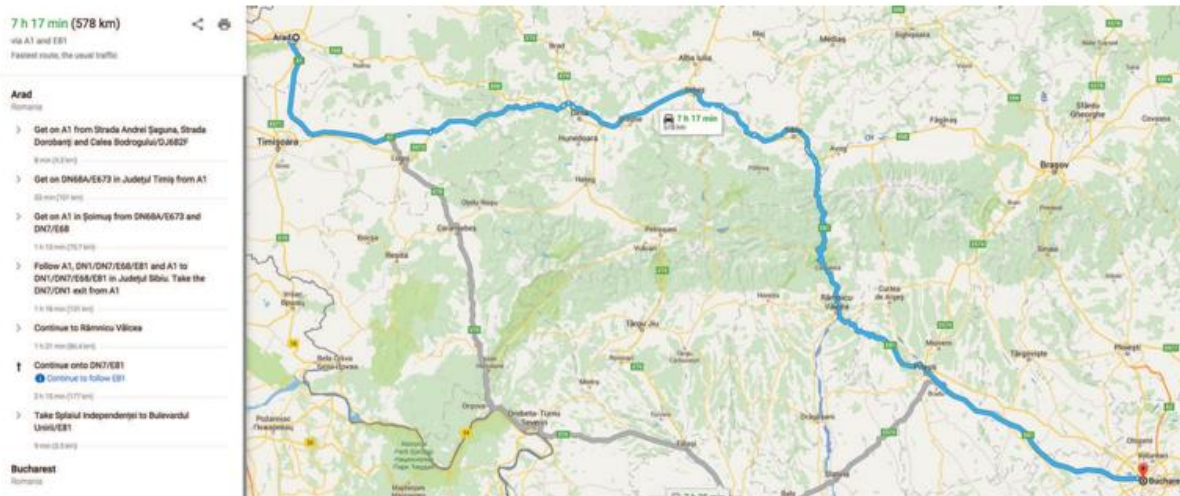
Ý tưởng đằng sau **cơ sở dữ liệu mẫu** (pattern databases) là lưu trữ các chi phí giải pháp chính xác này cho mọi trường hợp bài toán con có thể có—trong ví dụ của chúng ta, mọi cấu hình có thể có của bốn ô và ô trống. (Sẽ có $9 \times 8 \times 7 \times 6 \times 5 = 15,120$ mẫu trong cơ sở dữ liệu. Danh tính của bốn ô khác không liên quan cho mục đích giải quyết bài toán con, nhưng các bước di chuyển của các ô đó có tính vào chi phí giải pháp của bài toán con.) Sau đó, chúng ta tính toán một heuristic chấp nhận được h_{DB} cho mỗi trạng thái gặp phải trong quá trình tìm kiếm chỉ bằng cách tra cứu cấu hình bài toán con tương ứng trong cơ sở dữ liệu. Bản thân cơ sở dữ liệu được xây dựng bằng cách tìm kiếm ngược từ mục tiêu và ghi lại chi phí của mỗi mẫu mới gặp phải; chi phí của tìm kiếm này được khấu hao trên các trường hợp bài toán tiếp theo và do đó có ý nghĩa nếu chúng ta mong đợi được yêu cầu giải quyết nhiều bài toán.

Việc chọn các ô 1-2-3-4 đi cùng với ô trống là khá tùy ý; chúng ta cũng có thể xây dựng cơ sở dữ liệu cho 5-6-7-8, cho 2-4-6-8, v.v. Mỗi cơ sở dữ liệu mang lại một heuristic chấp nhận được, và các heuristic này có thể được kết hợp, như đã giải thích trước đó, bằng cách lấy giá trị tối đa. Một heuristic kết hợp loại này chính xác hơn nhiều so với khoảng cách Manhattan; số nút được tạo ra khi giải quyết các 15-puzzle ngẫu nhiên có thể giảm đi 1000 lần. Tuy nhiên, với mỗi cơ sở dữ liệu bổ sung, có những lợi ích giảm dần và tăng chi phí bộ nhớ và tính toán.

Người ta có thể tự hỏi liệu các heuristic thu được từ cơ sở dữ liệu 1-2-3-4 và 5-6-7-8 có thể được cộng lại không, vì hai bài toán con dường như không chồng chéo. Liệu điều này vẫn cho một heuristic chấp nhận được không? Câu trả lời là không, bởi vì các giải pháp của bài toán con 1-2-3-4 và bài toán con 5-6-7-8 cho một trạng thái nhất định gần như chắc chắn sẽ chia sẻ một số bước di chuyển—rất khó có khả năng 1-2-3-4 có thể được di chuyển vào vị trí mà không chạm vào 5-6-7-8, và ngược lại. Nhưng nếu chúng ta không đếm những bước di chuyển đó thì sao—nếu chúng ta không trừu tượng hóa các ô khác thành các ngôi sao, mà thay vào đó làm cho chúng biến mất thì sao? Tức là, chúng ta không ghi lại tổng chi phí giải quyết bài toán con 1-2-3-4, mà chỉ là số lần di chuyển liên quan đến 1-2-3-4. Sau đó, tổng của hai chi phí vẫn là một cận dưới về chi phí giải quyết toàn bộ bài toán. Đây là ý tưởng đằng sau **cơ sở dữ liệu mẫu rời rạc** (disjoint pattern databases). Với các cơ sở dữ liệu như vậy, có thể giải quyết các 15-puzzle ngẫu nhiên trong vài mili giây—số nút được tạo ra giảm đi 10,000 lần so với việc sử dụng khoảng cách Manhattan. Đối với 24-puzzles, có thể đạt được tốc độ tăng lên khoảng một triệu lần. Cơ sở dữ liệu mẫu rời rạc hoạt động cho các câu đố trượt ô vì bài toán có thể được chia nhỏ theo cách mà mỗi lần di chuyển chỉ ảnh hưởng đến một bài toán con—vì chỉ có một ô được di chuyển tại một thời điểm.

3.6.4 Tạo Heuristic bằng các điểm mốc

Có các dịch vụ trực tuyến lưu trữ bản đồ với hàng chục triệu đỉnh và tìm chỉ đường lái xe với chi phí tối ưu trong vài mili giây (Hình 3.28). Làm thế nào họ có thể làm điều đó, khi các thuật toán tìm kiếm tốt nhất mà chúng ta đã xem xét cho đến nay chậm hơn khoảng một triệu lần? Có nhiều thủ thuật, nhưng thủ thuật quan trọng nhất là **tính toán trước** (precomputation) một số chi phí đường dẫn tối ưu. Mặc dù việc tính toán trước có thể tốn thời gian, nhưng nó chỉ cần được thực hiện một lần, và sau đó có thể được khấu hao trên hàng tỷ yêu cầu tìm kiếm của người dùng.



Hình 3.28 Một dịch vụ web cung cấp chỉ đường lái xe, được tính toán bằng một thuật toán tìm kiếm.

Chúng ta có thể tạo ra một heuristic hoàn hảo bằng cách tính toán trước và lưu trữ chi phí của đường dẫn tối ưu giữa mọi cặp đỉnh. Điều đó sẽ tốn không gian $O(|V|^2)$ và thời gian $O(|E|^3)$ —thực tế đối với các đồ thị có 10 nghìn đỉnh, nhưng không phải 10 triệu.

Một cách tiếp cận tốt hơn là chọn một vài (có lẽ 10 hoặc 20) **điểm mốc** (landmark point) từ các đỉnh. Sau đó, đối với mỗi điểm mốc L và đối với mỗi đỉnh v khác trong đồ thị, chúng ta tính toán và lưu trữ $C^*(v, L)$, chi phí chính xác của đường dẫn tối ưu từ v đến L . (Chúng ta cũng cần $C^*(L, v)$; trên một đồ thị vô hướng, điều này giống như $C^*(v, L)$; trên một đồ thị có hướng—ví dụ, với các đường một chiều—chúng ta cần tính toán điều này một cách riêng biệt.) Với các bảng C^* đã được lưu trữ, chúng ta có thể dễ dàng tạo ra một heuristic hiệu quả (mặc dù không chấp nhận được): giá trị nhỏ nhất, trên tất cả các điểm mốc, của chi phí để đi từ nút hiện tại đến điểm mốc, và sau đó đến mục tiêu:

$$h_L(n) = \min_{L \in \text{Landmarks}} C^*(n, L) + C^*(L, \text{goal})$$

Nếu đường dẫn tối ưu tình cờ đi qua một điểm mốc, heuristic này sẽ chính xác; nếu không, nó không chấp nhận được—nó ước tính quá cao chi phí đến mục tiêu. Trong một tìm kiếm A*, nếu bạn có các heuristic chính xác, thì một khi bạn đến một nút nằm trên một đường dẫn tối ưu, mọi nút bạn mở rộng từ đó trở đi sẽ nằm trên một đường dẫn tối ưu. Hãy nghĩ về các đường đồng mức như đi theo đường dẫn tối ưu này. Tìm kiếm sẽ đi theo đường dẫn tối ưu, ở mỗi lần lặp thêm một hành động với chi phí c để đến một trạng thái kết quả có giá trị h sẽ nhỏ hơn c , có nghĩa là tổng điểm $f = g + h$ sẽ không đổi ở C^* dọc theo đường dẫn.

Một số thuật toán tìm đường tiết kiệm thời gian hơn nữa bằng cách thêm các **lối tắt** (shortcuts)—các cạnh nhân tạo trong đồ thị định nghĩa một đường dẫn đa hành động tối ưu. Ví dụ, nếu có các lối tắt được xác định trước giữa tất cả 100 thành phố lớn nhất ở Hoa Kỳ, và chúng ta đang cố gắng điều hướng từ khuôn viên Berkeley ở California đến NYU ở New York, chúng ta có thể đi lối tắt giữa Sacramento và Manhattan và đi được 90% đường dẫn trong một hành động.

$h_L(n)$ hiệu quả nhưng không chấp nhận được. Nhưng với một chút cẩn thận hơn, chúng ta có thể đưa ra một heuristic vừa hiệu quả vừa chấp nhận được:

$$h_{DH}(n) = \max_{L \in Landmarks} |C^*(n, L) - C^*(goal, L)|$$

Đây được gọi là **heuristic vi sai** (differential heuristic) (vì phép trừ). Hãy nghĩ về điều này với một điểm mốc nằm đâu đó ngoài mục tiêu. Nếu mục tiêu tình cờ nằm trên đường dẫn tối ưu từ n đến điểm mốc, thì điều này đang nói “xem xét toàn bộ đường dẫn từ n đến L , sau đó trừ đi phần cuối của đường dẫn đó, từ mục tiêu đến L , cho chúng ta chi phí chính xác của đường dẫn từ n đến mục tiêu.” Trong chừng mực mà mục tiêu hơi lệch khỏi đường dẫn tối ưu đến điểm mốc, heuristic sẽ không chính xác, nhưng vẫn chấp nhận được. Các điểm mốc không nằm ngoài mục tiêu sẽ không hữu ích; một điểm mốc nằm chính xác ở giữa n và mục tiêu sẽ cho $h_{DH} = 0$, điều này không hữu ích.

Có một vài cách để chọn các điểm mốc. Chọn các điểm ngẫu nhiên thì nhanh, nhưng chúng ta nhận được kết quả tốt hơn nếu chúng ta cẩn thận để trải các điểm mốc ra để chúng không quá gần nhau. Một cách tiếp cận tham lam là chọn một điểm mốc đầu tiên một cách ngẫu nhiên, sau đó tìm điểm xa nhất từ đó và thêm nó vào tập hợp các điểm mốc, và tiếp tục, ở mỗi lần lặp thêm điểm tối đa hóa khoảng cách đến điểm mốc gần nhất. Nếu bạn có nhật ký các yêu cầu tìm kiếm trong quá khứ của người dùng, thì bạn có thể chọn các điểm mốc thường xuyên được yêu cầu trong các tìm kiếm. Đối với heuristic vi sai, tốt hơn là các điểm mốc được trải ra xung quanh chu vi của đồ thị. Do đó, một kỹ thuật tốt là tìm trọng tâm của đồ thị, sắp xếp k hình nêm xung quanh trọng tâm, và trong mỗi hình nêm chọn đỉnh xa nhất từ tâm.

Các điểm mốc hoạt động đặc biệt tốt trong các bài toán tìm đường vì cách các con đường được bố trí trên thế giới: rất nhiều giao thông thực sự muốn đi lại giữa các điểm mốc, vì vậy các kỹ sư dân sự xây dựng các con đường rộng nhất và nhanh nhất dọc theo các tuyến đường này; tìm kiếm điểm mốc giúp dễ dàng khôi phục các tuyến đường này hơn.

3.6.5 Học để tìm kiếm tốt hơn

Chúng ta đã trình bày một số chiến lược tìm kiếm cố định—tìm kiếm theo chiều rộng, A^* , v.v.—đã được các nhà khoa học máy tính thiết kế và lập trình cẩn thận. Liệu một tác nhân có thể học cách tìm kiếm tốt hơn không? Câu trả lời là có, và phương pháp này dựa trên một khái niệm quan trọng được gọi là **không gian trạng thái siêu cấp** (metalevel state space). Mỗi trạng thái trong một không gian trạng thái siêu cấp nắm bắt trạng thái bên trong (tính toán) của một chương trình đang tìm kiếm trong một không gian trạng thái thông thường như bản đồ Romania. (Để giữ hai khái niệm này tách biệt, chúng ta gọi bản đồ Romania là **không gian trạng thái cấp đối tượng** (object-level state space).) Ví dụ, trạng thái bên trong của thuật toán A^* bao gồm cây tìm kiếm hiện tại. Mỗi hành động trong không gian trạng thái siêu cấp là một bước tính toán làm thay đổi trạng thái bên trong; ví dụ, mỗi bước tính toán trong A^* mở rộng một nút lá và thêm các nút kế tiếp của nó vào cây. Do đó, Hình 3.18, cho thấy một chuỗi các cây tìm kiếm ngày càng lớn, có thể được coi là mô tả một đường dẫn trong không gian trạng thái siêu cấp, trong đó mỗi trạng thái trên đường dẫn là một cây tìm kiếm cấp đối tượng.

Bây giờ, đường dẫn trong Hình 3.18 có năm bước, bao gồm một bước, việc mở rộng Fagaras, không đặc biệt hữu ích. Đối với các bài toán khó hơn, sẽ có nhiều bước đi sai như vậy, và một

thuật toán **học siêu cấp** (metalevel learning) có thể học từ những kinh nghiệm này để tránh khám phá các cây con không có triển vọng. Các kỹ thuật được sử dụng cho loại học này được mô tả trong Chương 23. Mục tiêu của việc học là để giảm thiểu tổng chi phí giải quyết bài toán, cân bằng giữa chi phí tính toán và chi phí đường dẫn.

3.6.6 Học Heuristic từ kinh nghiệm

Chúng ta đã thấy rằng một cách để phát minh ra một heuristic là nghĩ ra một bài toán đã được nói lòng mà một giải pháp tối ưu có thể được tìm thấy dễ dàng. Một cách khác là học hỏi từ kinh nghiệm. “Kinh nghiệm” ở đây có nghĩa là giải quyết nhiều 8-puzzles, chẳng hạn. Mỗi giải pháp tối ưu cho một bài toán 8-puzzle cung cấp một ví dụ (mục tiêu, đường dẫn). Từ những ví dụ này, một thuật toán học có thể được sử dụng để xây dựng một hàm h mà có thể (với may mắn) xấp xỉ chi phí đường dẫn thực sự cho các trạng thái khác phát sinh trong quá trình tìm kiếm. Hầu hết các cách tiếp cận này học một xấp xỉ không hoàn hảo cho hàm heuristic, và do đó có nguy cơ không chấp nhận được. Điều này dẫn đến một sự đánh đổi không thể tránh khỏi giữa thời gian học, thời gian chạy tìm kiếm và chi phí giải pháp. Các kỹ thuật cho học máy được trình bày trong Chương 19. Các phương pháp học tăng cường được mô tả trong Chương 23 cũng có thể áp dụng cho tìm kiếm.

Một số kỹ thuật học máy hoạt động tốt hơn khi được cung cấp các **đặc trưng** (features) của một trạng thái có liên quan đến việc dự đoán giá trị heuristic của trạng thái, thay vì chỉ mô tả trạng thái thô. Ví dụ, đặc trưng “số ô sai vị trí” có thể hữu ích trong việc dự đoán khoảng cách thực tế của một trạng thái 8-puzzle so với mục tiêu. Hãy gọi đặc trưng này là $x_1(n)$. Chúng ta có thể lấy 100 cấu hình 8-puzzle được tạo ngẫu nhiên và thu thập số liệu thống kê về chi phí giải pháp thực tế của chúng. Chúng ta có thể thấy rằng khi $x_1(n)$ là 5, chi phí giải pháp trung bình là khoảng 14, và cứ như vậy. Tất nhiên, chúng ta có thể sử dụng nhiều đặc trưng. Một đặc trưng thứ hai $x_2(n)$ có thể là “số cặp ô liền kề không liền kề trong trạng thái mục tiêu.” Làm thế nào $x_1(n)$ và $x_2(n)$ nên được kết hợp để dự đoán $h(n)$? Một cách tiếp cận phổ biến là sử dụng một tổ hợp tuyến tính:

$$h(n) = c_1 x_1(n) + c_2 x_2(n)$$

Các hằng số c_1 và c_2 được điều chỉnh để đưa ra sự phù hợp tốt nhất với dữ liệu thực tế trên các cấu hình được tạo ngẫu nhiên. Người ta mong đợi cả c_1 và c_2 đều là số dương vì các ô sai vị trí và các cặp liền kề không đúng làm cho bài toán khó giải hơn. Lưu ý rằng heuristic này thỏa mãn điều kiện $h(n) = 0$ cho các trạng thái mục tiêu, nhưng nó không nhất thiết phải chấp nhận được hoặc nhất quán.

Tóm tắt

Chương này đã giới thiệu các thuật toán tìm kiếm mà một tác nhân có thể sử dụng để chọn các chuỗi hành động trong nhiều môi trường khác nhau—miễn là chúng là tuần tự, một tác nhân, có thể quan sát đầy đủ, xác định, tĩnh, rời rạc và được biết hoàn toàn. Có những sự đánh đổi phải được thực hiện giữa lượng thời gian tìm kiếm, lượng bộ nhớ có sẵn và chất lượng của giải pháp. Chúng ta có thể hiệu quả hơn nếu chúng ta có kiến thức phụ thuộc vào miền dưới dạng một hàm heuristic

ước tính một trạng thái nhất định cách mục tiêu bao xa, hoặc nếu chúng ta tính toán trước các giải pháp một phần liên quan đến các mẫu hoặc điểm mốc.

- Trước khi một tác nhân có thể bắt đầu tìm kiếm, một bài toán được xác định rõ phải được xây dựng.
- Một bài toán bao gồm năm phần: trạng thái ban đầu, một tập hợp các hành động, một mô hình chuyển đổi mô tả kết quả của các hành động đó, một tập hợp các trạng thái mục tiêu và một hàm chi phí hành động.
- Môi trường của bài toán được biểu diễn bằng một đồ thị không gian trạng thái. Một đường dẫn qua không gian trạng thái (một chuỗi các hành động) từ trạng thái ban đầu đến trạng thái mục tiêu là một giải pháp.
- Các thuật toán tìm kiếm thường xử lý các trạng thái và hành động như các nguyên tử, không có bất kỳ cấu trúc nội bộ nào (mặc dù chúng ta đã giới thiệu các đặc trưng của trạng thái khi đến lúc thực hiện việc học).
- Các thuật toán tìm kiếm được đánh giá dựa trên tính đầy đủ, tính tối ưu chi phí, độ phức tạp thời gian và độ phức tạp không gian.
- Các phương pháp tìm kiếm không có thông tin chỉ có quyền truy cập vào định nghĩa bài toán. Các thuật toán xây dựng một cây tìm kiếm trong nỗ lực tìm một giải pháp. Các thuật toán khác nhau dựa trên nút nào chúng mở rộng trước:
 - **Tìm kiếm tốt nhất đầu tiên** (Best-first search) chọn các nút để mở rộng bằng cách sử dụng một hàm đánh giá.
 - **Tìm kiếm theo chiều rộng đầu tiên** (Breadth-first search) mở rộng các nút nông nhất trước; nó đầy đủ, tối ưu cho chi phí hành động đơn vị, nhưng có độ phức tạp không gian theo hàm mũ.
 - **Tìm kiếm chi phí đồng nhất** (Uniform-cost search) mở rộng nút có chi phí đường dẫn thấp nhất, $g(n)$, và tối ưu cho chi phí hành động chung.
 - **Tìm kiếm theo chiều sâu đầu tiên** (Depth-first search) mở rộng nút sâu nhất chưa được mở rộng trước. Nó không đầy đủ cũng không tối ưu, nhưng có độ phức tạp không gian tuyến tính. Tìm kiếm theo chiều sâu giới hạn thêm một giới hạn độ sâu.
 - **Tìm kiếm sâu dần lặp lại** (Iterative deepening search) gọi tìm kiếm theo chiều sâu đầu tiên với các giới hạn độ sâu tăng dần cho đến khi tìm thấy một mục tiêu. Nó đầy đủ khi kiểm tra chu kỳ đầy đủ được thực hiện, tối ưu cho chi phí hành động đơn vị, có độ phức tạp thời gian tương đương với tìm kiếm theo chiều rộng đầu tiên và có độ phức tạp không gian tuyến tính.
 - **Tìm kiếm hai chiều** (Bidirectional search) mở rộng hai biên, một xung quanh trạng thái ban đầu và một xung quanh mục tiêu, dừng lại khi hai biên gặp nhau.
- Các phương pháp tìm kiếm có thông tin có quyền truy cập vào một hàm heuristic $h(n)$ ước tính chi phí của một giải pháp từ n . Chúng có thể có quyền truy cập vào thông tin bổ sung như cơ sở dữ liệu mẫu với chi phí giải pháp.
 - **Tìm kiếm tốt nhất đầu tiên tham lam** (Greedy best-first search) mở rộng các nút có $h(n)$ tối thiểu. Nó không tối ưu nhưng thường hiệu quả.
 - **Tìm kiếm A*** mở rộng các nút có $f(n) = g(n) + h(n)$ tối thiểu. A* đầy đủ và tối ưu, với điều kiện $h(n)$ là chấp nhận được. Độ phức tạp không gian của A* vẫn là một vấn đề đối với nhiều bài toán.

- __Tìm kiếm A__ hai chiều __* đôi khi hiệu quả hơn A* bản thân nó.
- **IDA*** (tìm kiếm A* sâu dần lặp lại) là một phiên bản sâu dần lặp lại của A*, và do đó giải quyết vấn đề độ phức tạp không gian.
- **RBFS** (tìm kiếm tốt nhất đầu tiên đệ quy) và **SMA*** (A* giới hạn bộ nhớ đơn giản hóa) là các thuật toán tìm kiếm tối ưu, mạnh mẽ sử dụng lượng bộ nhớ hạn chế; nếu có đủ thời gian, chúng có thể giải quyết các bài toán mà A* hết bộ nhớ.
- **Tìm kiếm chùm** (Beam search) đặt giới hạn về kích thước của biên; điều đó làm cho nó không đầy đủ và không tối ưu, nhưng nó thường tìm thấy các giải pháp khá tốt và chạy nhanh hơn các tìm kiếm đầy đủ.
- __Tìm kiếm A__ có trọng số __* (Weighted A* search) tập trung tìm kiếm vào một mục tiêu, mở rộng ít nút hơn, nhưng hy sinh tính tối ưu.
- Hiệu suất của các thuật toán tìm kiếm heuristic phụ thuộc vào chất lượng của hàm heuristic. Đôi khi người ta có thể xây dựng các heuristic tốt bằng cách nói lỏng định nghĩa bài toán, bằng cách lưu trữ các chi phí giải pháp được tính toán trước cho các bài toán con trong một cơ sở dữ liệu mẫu, bằng cách xác định các điểm mốc hoặc bằng cách học hỏi từ kinh nghiệm với loại bài toán.

Ghi chú Thư mục và Lịch sử

Chủ đề tìm kiếm không gian trạng thái có nguồn gốc từ những năm đầu của AI. Công trình của Newell và Simon về Logic Theorist (1957) và GPS (1961) đã dẫn đến việc thành lập các thuật toán tìm kiếm như công cụ chính cho các nhà nghiên cứu AI thập niên 1960 và việc thành lập giải quyết vấn đề như nhiệm vụ AI chính tắc. Công trình trong nghiên cứu hoạt động của Richard Bellman (1957) đã cho thấy tầm quan trọng của chi phí đường dẫn cộng để đơn giản hóa các thuật toán tối ưu hóa. Văn bản của Nils Nilsson (1971) đã thiết lập lĩnh vực này trên một nền tảng lý thuyết vững chắc.

8-puzzle là một người anh em họ nhỏ hơn của 15-puzzle, mà lịch sử của nó được kể lại chi tiết bởi Slocum và Sonneveld (2006). Vào năm 1880, 15-puzzle đã thu hút sự chú ý rộng rãi từ công chúng và các nhà toán học (Johnson và Story, 1879; Tait, 1880). Các biên tập viên của Tạp chí Toán học Hoa Kỳ đã nói, “Câu đố ‘15’ trong vài tuần qua đã được công chúng Mỹ chú ý nổi bật, và có thể nói một cách an toàn rằng nó đã thu hút sự chú ý của chín trong số mười người thuộc cả hai giới và mọi lứa tuổi và điều kiện trong cộng đồng,” trong khi tờ Weekly News-Democrat of Emporia, Kansas đã viết vào ngày 12 tháng 3 năm 1880 rằng “Nó đã trở thành một dịch bệnh theo đúng nghĩa đen trên khắp đất nước.”

Nhà thiết kế trò chơi người Mỹ nổi tiếng Sam Loyd đã tuyên bố sai rằng đã phát minh ra 15-puzzle (Loyd, 1959); thực tế nó được phát minh bởi Noyes Chapman, một người quản lý bưu điện ở Canastota, New York, vào giữa những năm 1870 (mặc dù một bằng sáng chế chung bao gồm các khối trượt đã được cấp cho Ernest Kinsey vào năm 1878). Ratner và Warmuth (1986) đã chỉ ra rằng phiên bản tổng quát $n \times n$ của 15-puzzle thuộc về lớp các bài toán NP-đầy đủ.

Rubik’s Cube tất nhiên được phát minh vào năm 1974 bởi Erno Rubik, người cũng đã khám phá ra một thuật toán để tìm các giải pháp tốt, nhưng không tối ưu. Korf (1997) đã tìm thấy các giải pháp tối ưu cho một số trường hợp bài toán ngẫu nhiên bằng cách sử dụng cơ sở dữ liệu mẫu và tìm kiếm IDA*. Rokicki et al. (2014) đã chứng minh rằng bất kỳ trường hợp nào cũng có thể được

giải quyết trong 26 lần di chuyển (nếu bạn coi một lần xoay 180° là hai lần di chuyển; 20 nếu nó được tính là một). Bằng chứng đã tiêu tốn 35 năm CPU tính toán; nó không dẫn ngay đến một thuật toán hiệu quả. Agostinelli et al. (2019) đã sử dụng học tăng cường, mạng học sâu và tìm kiếm cây Monte Carlo để học một bộ giải hiệu quả hơn nhiều cho Rubik's cube. Nó không được đảm bảo tìm thấy một giải pháp tối ưu về chi phí, nhưng làm như vậy khoảng 60% thời gian và thời gian giải pháp điển hình là dưới một giây.

Mỗi bài toán tìm kiếm trong thế giới thực được liệt kê trong chương đã là chủ đề của một nỗ lực nghiên cứu đáng kể. Các phương pháp để chọn các chuyến bay tối ưu của hãng hàng không phần lớn vẫn là độc quyền, nhưng Carl de Marcken đã chỉ ra bằng cách giảm xuống các bài toán quyết định Diophantine rằng giá vé máy bay và các hạn chế đã trở nên phức tạp đến mức bài toán chọn một chuyến bay tối ưu là không thể quyết định một cách hình thức (Robinson, 2002). Bài toán người bán hàng du lịch (TSP) là một bài toán tổ hợp tiêu chuẩn trong khoa học máy tính lý thuyết (Lawler et al., 1992). Karp (1972) đã chứng minh bài toán quyết định TSP là NP-khó, nhưng các phương pháp xấp xỉ heuristic hiệu quả đã được phát triển (Lin và Kernighan, 1973). Arora (1998) đã nghĩ ra một sơ đồ xấp xỉ đa thức hoàn toàn cho các TSP Euclidean. Các phương pháp bố trí VLSI được khảo sát bởi LaPaugh (2010), và nhiều bài báo tối ưu hóa bố trí xuất hiện trong các tạp chí VLSI. Điều hướng robot được thảo luận trong Chương 26. Trình tự lắp ráp tự động lần đầu tiên được FREDDY chứng minh (Michie, 1972); một đánh giá toàn diện được đưa ra bởi (Bahubalendruni và Biswal, 2016).

Các thuật toán tìm kiếm không có thông tin là một chủ đề trung tâm của khoa học máy tính (Cormen et al., 2009) và nghiên cứu hoạt động (Dreyfus, 1969). Tìm kiếm theo chiều rộng đầu tiên được Moore (1959) xây dựng để giải quyết các mê cung. Phương pháp quy hoạch động (dynamic programming) (Bellman, 1957; Bellman và Dreyfus, 1962), ghi lại một cách có hệ thống các giải pháp cho tất cả các bài toán con có độ dài tăng dần, có thể được coi là một dạng tìm kiếm theo chiều rộng đầu tiên.

Thuật toán của Dijkstra trong dạng nó thường được trình bày (Dijkstra, 1959) có thể áp dụng cho các đồ thị hữu hạn rõ ràng. Nilsson (1971) đã giới thiệu một phiên bản thuật toán của Dijkstra mà ông gọi là tìm kiếm chi phí đồng nhất (vì thuật toán “lan truyền dọc theo các đường đồng mức chi phí đường dẫn bằng nhau”) cho phép các đồ thị vô hạn được định nghĩa một cách ngầm. Công trình của Nilsson cũng giới thiệu ý tưởng về các danh sách đóng và mở, và thuật ngữ “tìm kiếm đồ thị.” Cái tên BEST-FIRST-SEARCH được giới thiệu trong Sổ tay AI (Barr và Feigenbaum, 1981). Các thuật toán Floyd–Warshall (Floyd, 1962) và Bellman-Ford (Bellman, 1958; Ford, 1956) cho phép chi phí bước âm (miễn là không có chu kỳ âm).

Một phiên bản của tìm kiếm sâu dần lặp lại được thiết kế để sử dụng đồng hồ cờ vua hiệu quả lần đầu tiên được Slate và Atkin (1977) sử dụng trong chương trình chơi cờ CHESS 4.5. Thuật toán B của Martelli (1977) cũng bao gồm một khía cạnh sâu dần lặp lại. Kỹ thuật sâu dần lặp lại được giới thiệu bởi Bertram Raphael (1976) và nổi lên trong công trình của Korf (1985a).

Việc sử dụng thông tin heuristic trong giải quyết vấn đề xuất hiện trong một bài báo đầu tiên của Simon và Newell (1958), nhưng cụm từ “tìm kiếm heuristic” và việc sử dụng các hàm heuristic ước tính khoảng cách đến mục tiêu đến sau đó một chút (Newell và Ernst, 1965; Lin, 1965). Doran và Michie (1966) đã thực hiện các nghiên cứu thực nghiệm rộng rãi về tìm kiếm heuristic. Mặc dù họ đã phân tích độ dài đường dẫn và “độ xuyên thấu” (tỷ lệ độ dài đường dẫn so với tổng số nút đã được kiểm tra cho đến nay), họ dường như đã bỏ qua thông tin được cung cấp bởi chi phí đường

dẫn $g(n)$. Thuật toán A^* , kết hợp chi phí đường dẫn hiện tại vào tìm kiếm heuristic, được phát triển bởi Hart, Nilsson và Raphael (1968). Dechter và Pearl (1985) đã nghiên cứu các điều kiện mà theo đó A^* hiệu quả tối ưu (về số nút được mở rộng).

Bài báo A^* gốc (Hart et al., 1968) đã giới thiệu điều kiện nhất quán trên các hàm heuristic. Điều kiện đơn điệu được giới thiệu bởi Pohl (1977) như một sự thay thế đơn giản hơn, nhưng Pearl (1984) đã chỉ ra rằng hai điều kiện này là tương đương.

Pohl (1977) đi tiên phong trong việc nghiên cứu mối quan hệ giữa lỗi trong các hàm heuristic và độ phức tạp thời gian của A^* . Các kết quả cơ bản đã đạt được cho tìm kiếm giống cây với chi phí hành động đơn vị và một trạng thái mục tiêu duy nhất (Pohl, 1977; Gaschnig, 1979; Huyn et al., 1980; Pearl, 1984) và với nhiều trạng thái mục tiêu (Dinh et al., 2007). Korf và Reid (1998) đã chỉ ra cách dự đoán số nút chính xác được mở rộng (không chỉ là một xấp xỉ tiệm cận) trên nhiều miền bài toán thực tế. “Hệ số phân nhánh hiệu quả” được Nilsson (1971) đề xuất như một thước đo hiệu quả thực nghiệm. Đối với tìm kiếm đồ thị, Helmert và Röger (2008) đã lưu ý rằng một số bài toán nổi tiếng chứa số lượng nút theo hàm mũ trên các đường dẫn giải pháp chi phí tối ưu, ngụ ý độ phức tạp thời gian theo hàm mũ cho A^* .

Có nhiều biến thể của thuật toán A^* . Pohl (1970) đã giới thiệu tìm kiếm A^* có trọng số, và sau đó là một phiên bản động (1973), trong đó trọng số thay đổi theo độ sâu của cây. Ebendt và Drechsler (2009) tổng hợp các kết quả và kiểm tra một số ứng dụng. Hatem và Ruml (2014) cho thấy một phiên bản đơn giản và cải tiến của A^* có trọng số dễ thực hiện hơn. Wilt và Ruml (2014) giới thiệu tìm kiếm tốc độ như một lựa chọn thay thế cho tìm kiếm tham lam tập trung vào việc giảm thiểu thời gian tìm kiếm, và Wilt và Ruml (2016) cho thấy rằng các heuristic tốt nhất cho tìm kiếm thỏa mãn là khác với các heuristic cho tìm kiếm tối ưu. Burns et al. (2012) đưa ra một số thủ thuật triển khai để viết mã tìm kiếm nhanh, và Felner (2018) xem xét cách việc triển khai thay đổi khi sử dụng kiểm tra mục tiêu sớm.

Pohl (1971) đã giới thiệu tìm kiếm hai chiều. Holte et al. (2016) mô tả phiên bản tìm kiếm hai chiều được đảm bảo gặp nhau ở giữa, làm cho nó có thể áp dụng rộng rãi hơn. Eckerle et al. (2017) mô tả tập hợp các cặp nút chắc chắn được mở rộng và chỉ ra rằng không có tìm kiếm hai chiều nào có thể hiệu quả tối ưu. Thuật toán NBS (Chen et al., 2017) sử dụng rõ ràng một hàng đợi các cặp nút.

Một sự kết hợp của A^* hai chiều và các điểm mốc đã biết đã được sử dụng để tìm chỉ đường lái xe hiệu quả cho dịch vụ bản đồ trực tuyến của Microsoft (Goldberg et al., 2006). Sau khi lưu trữ một tập hợp các đường dẫn giữa các điểm mốc, thuật toán có thể tìm một đường dẫn chi phí tối ưu giữa bất kỳ cặp điểm nào trong một đồ thị 24 triệu điểm của Hoa Kỳ, tìm kiếm ít hơn 0,1% đồ thị. Korf (1987) chỉ ra cách sử dụng các mục tiêu con, các toán tử macro và sự trừu tượng để đạt được tốc độ tăng đáng kể so với các kỹ thuật trước đó. Delleng et al. (2009) mô tả cách sử dụng tìm kiếm hai chiều, các điểm mốc, cấu trúc phân cấp và các thủ thuật khác để tìm chỉ đường lái xe. Anderson et al. (2008) mô tả một kỹ thuật liên quan, được gọi là **tìm kiếm từ thô đến mịn** (coarse-to-fine search), có thể được coi là việc xác định các điểm mốc ở các cấp độ trừu tượng phân cấp khác nhau. Korf (1987) mô tả các điều kiện mà theo đó tìm kiếm từ thô đến mịn cung cấp một tốc độ tăng theo hàm mũ. Knoblock (1991) cung cấp các kết quả thực nghiệm và phân tích để định lượng các lợi thế của tìm kiếm phân cấp.

A^* và các thuật toán tìm kiếm không gian trạng thái khác có liên quan chặt chẽ đến các kỹ thuật **phân nhánh và cận** (branch-and-bound) được sử dụng rộng rãi trong nghiên cứu hoạt động

(Lawler và Wood, 1966; Rayward-Smith et al., 1996). Kumar và Kanal (1988) đã cố gắng “hợp nhất lớn” tìm kiếm heuristic, quy hoạch động và các kỹ thuật phân nhánh và cận dưới tên CDP— “quy trình quyết định tổng hợp.”

Vì hầu hết các máy tính trong những năm 1960 chỉ có vài nghìn từ bộ nhớ chính, tìm kiếm heuristic giới hạn bộ nhớ là một chủ đề nghiên cứu sớm. The Graph Traverser (Doran và Michie, 1966), một trong những chương trình tìm kiếm sớm nhất, cam kết một hành động sau khi tìm kiếm tốt nhất đầu tiên lên đến giới hạn bộ nhớ. IDA* (Korf, 1985b) là thuật toán tìm kiếm heuristic giới hạn bộ nhớ, tối ưu độ dài đầu tiên được sử dụng rộng rãi, và một số lượng lớn các biến thể đã được phát triển. Một phân tích về hiệu quả của IDA* và những khó khăn của nó với các heuristic giá trị thực xuất hiện trong Patrick et al. (1992).

Phiên bản gốc của RBFS (Korf, 1993) thực sự phức tạp hơn một chút so với thuật toán được hiển thị trong Hình 3.22, thực sự gần hơn với một thuật toán được phát triển độc lập gọi là **mở rộng lặp lại** (iterative expansion) hoặc IE (Russell, 1992). RBFS sử dụng một cận dưới cũng như cận trên; hai thuật toán hoạt động giống hệt nhau với các heuristic chấp nhận được, nhưng RBFS mở rộng các nút theo thứ tự tốt nhất đầu tiên ngay cả với một heuristic không chấp nhận được. Ý tưởng theo dõi đường dẫn thay thế tốt nhất đã xuất hiện sớm hơn trong việc triển khai Prolog thành lịch của A* của Bratko (2009) và trong thuật toán DTA* (Russell và Wefald, 1991). Công trình sau này cũng thảo luận về không gian trạng thái siêu cấp và học siêu cấp.

Thuật toán MA* xuất hiện trong Chakrabarti et al. (1989). SMA*, hay Simplified MA*, xuất hiện từ một nỗ lực để triển khai MA* (Russell, 1992). Kaindl và Khorsand (1994) đã áp dụng SMA* để tạo ra một thuật toán tìm kiếm hai chiều nhanh hơn đáng kể so với các thuật toán trước đó. Korf và Zhang (2000) mô tả một cách tiếp cận chia để trị, và Zhou và Hansen (2002) giới thiệu tìm kiếm đồ thị A* giới hạn bộ nhớ và một chiến lược để chuyển sang tìm kiếm theo chiều rộng đầu tiên để tăng hiệu quả bộ nhớ (Zhou và Hansen, 2006).

Ý tưởng rằng các heuristic chấp nhận được có thể được suy ra bằng cách nói lòng bài toán xuất hiện trong bài báo quan trọng của Held và Karp (1970), những người đã sử dụng heuristic cây bao trùm tối thiểu để giải quyết TSP. (Xem Bài tập 3.MSTR.) Việc tự động hóa quy trình nói lòng đã được Prieditis (1993) triển khai thành công. Có một tài liệu ngày càng tăng về việc áp dụng học máy để khám phá các hàm heuristic (Samadi et al., 2008; Arfaee et al., 2010; Thayer et al., 2011; Lelis et al., 2012).

Việc sử dụng cơ sở dữ liệu mẫu để suy ra các heuristic chấp nhận được là do Gasser (1995) và Culberson và Schaeffer (1996, 1998); cơ sở dữ liệu mẫu rời rạc được mô tả bởi Korf và Felner (2002); một phương pháp tương tự sử dụng các mẫu biểu tượng là do Edelkamp (2009). Felner et al. (2007) chỉ ra cách nền cơ sở dữ liệu mẫu để tiết kiệm không gian. Việc diễn giải xác suất của các heuristic đã được Pearl (1984) và Hansson và Mayer (1989) điều tra. **Heuristics** của Pearl (1984) và **Heuristic Search** của Edelkamp và Schrodl (2012) là những cuốn sách giáo khoa có ảnh hưởng về tìm kiếm. Các bài báo về các thuật toán tìm kiếm mới xuất hiện tại Hội nghị chuyên đề quốc tế về tìm kiếm tổ hợp (SoCS) và Hội nghị quốc tế về lập kế hoạch và lập lịch tự động (ICAPS), cũng như trong các hội nghị AI chung như AAAI và IJCAI, và các tạp chí như Artificial Intelligence và Journal of the ACM.

CHƯƠNG 4: TÌM KIẾM TRONG MÔI TRƯỜNG PHỨC TẠP

Trong chương này chúng ta nói lỏng các giả định đơn giản hóa của chương trước, để đến gần hơn với thế giới thực.

Chương 3 đã giải quyết các bài toán trong môi trường **hoàn toàn có thể quan sát được, xác định, tĩnh, đã biết** và lời giải là một chuỗi các hành động. Trong chương này, chúng ta sẽ nói lỏng những ràng buộc đó. Chúng ta bắt đầu với bài toán tìm kiếm một trạng thái tốt mà không cần quan tâm đến đường đi để đến đó, bao gồm cả trạng thái **rời rạc** (Mục 4.1) và **liên tục** (Mục 4.2). Sau đó, chúng ta nói lỏng các giả định về **tính xác định** (Mục 4.3) và **khả năng quan sát** (Mục 4.4). Trong một thế giới không xác định, tác nhân sẽ cần một **kế hoạch có điều kiện** và thực hiện các hành động khác nhau tùy thuộc vào những gì nó quan sát—ví dụ, dừng lại nếu đèn đỏ và đi nếu đèn xanh. Với khả năng quan sát một phần, tác nhân cũng sẽ cần theo dõi các trạng thái có thể xảy ra mà nó có thể đang ở. Cuối cùng, Mục 4.5 hướng dẫn tác nhân đi qua một không gian chưa biết mà nó phải học khi di chuyển, sử dụng **tìm kiếm trực tuyến**.

Tìm kiếm cục bộ

Bài toán tối ưu Hàm mục tiêu “Bản đồ” không gian trạng thái

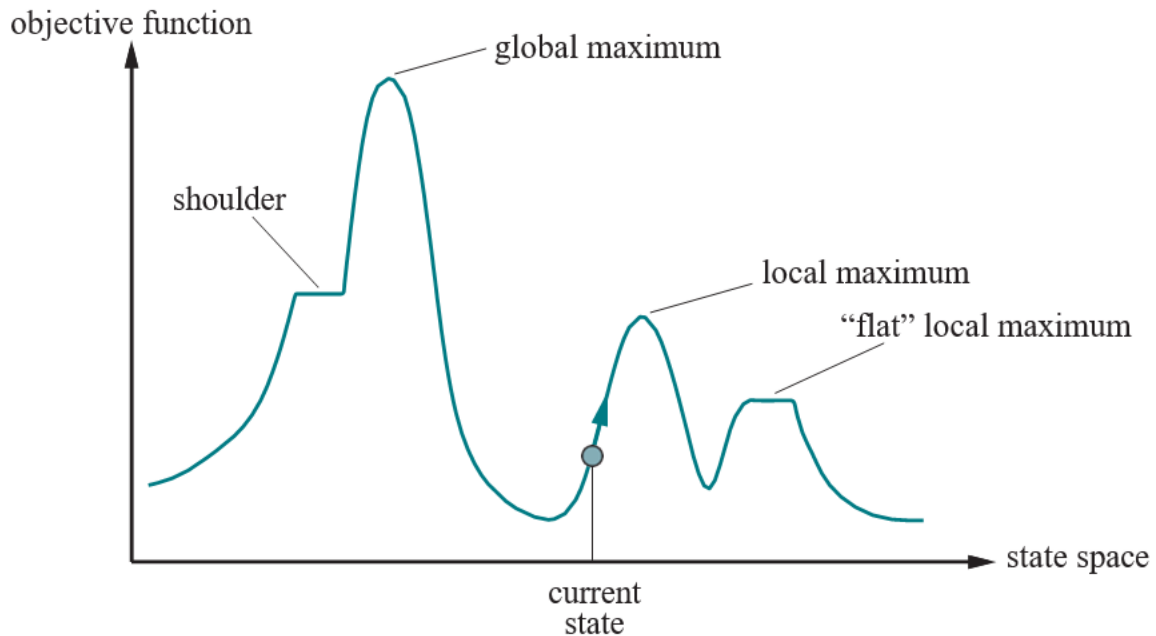
4.1 Tìm kiếm cục bộ và các bài toán tối ưu

Trong các bài toán tìm kiếm ở Chương 3, chúng ta muốn tìm các **đường đi** qua không gian tìm kiếm, chẳng hạn như đường đi từ Arad đến Bucharest. Nhưng đôi khi chúng ta chỉ quan tâm đến trạng thái cuối cùng, không phải đường đi để đến đó. Ví dụ, trong bài toán 8 quân hậu (Hình 4.3), chúng ta chỉ quan tâm đến việc tìm một cấu hình cuối cùng hợp lệ của 8 quân hậu (vì nếu bạn biết cấu hình, việc tái tạo lại các bước đã tạo ra nó là điều đơn giản). Điều này cũng đúng đối với nhiều ứng dụng quan trọng như thiết kế mạch tích hợp, bố trí mặt bằng nhà máy, lập lịch trình công việc, lập trình tự động, tối ưu hóa mạng viễn thông, lập kế hoạch cây trồng và quản lý danh mục đầu tư.

Các thuật toán **tìm kiếm cục bộ** hoạt động bằng cách tìm kiếm từ một trạng thái bắt đầu đến các trạng thái lân cận, mà không cần theo dõi các đường đi, cũng như tập hợp các trạng thái đã đạt được. Điều đó có nghĩa là chúng không có hệ thống—chúng có thể không bao giờ khám phá một phần của không gian tìm kiếm nơi thực sự có một giải pháp. Tuy nhiên, chúng có hai lợi thế chính: (1) chúng sử dụng rất ít bộ nhớ; và (2) chúng thường có thể tìm thấy các giải pháp hợp lý trong không gian trạng thái lớn hoặc vô hạn mà các thuật toán có hệ thống không phù hợp.

Các thuật toán tìm kiếm cục bộ cũng có thể giải quyết các **bài toán tối ưu**, trong đó mục tiêu là tìm trạng thái tốt nhất theo một **hàm mục tiêu**.

Để hiểu tìm kiếm cục bộ, hãy xem xét các trạng thái của một bài toán được sắp xếp trên một **“bản đồ” không gian trạng thái**, như trong Hình 4.1. Mỗi điểm (trạng thái) trong bản đồ có một “độ cao”, được xác định bởi giá trị của hàm mục tiêu. Nếu độ cao tương ứng với một hàm mục tiêu, thì mục tiêu là tìm đỉnh cao nhất—một **cực đại toàn cục**—và chúng ta gọi quá trình này là **leo đồi (hill climbing)**. Nếu độ cao tương ứng với chi phí, thì mục tiêu là tìm thung lũng thấp nhất—một **cực tiểu toàn cục**—và chúng ta gọi nó là **giảm độ dốc (gradient descent)**.



Hình 4.1 Một “bản đồ” không gian trạng thái một chiều trong đó độ cao tương ứng với hàm mục tiêu. Mục tiêu là tìm cực đại toàn cục.

4.1.1 Tìm kiếm leo đồi

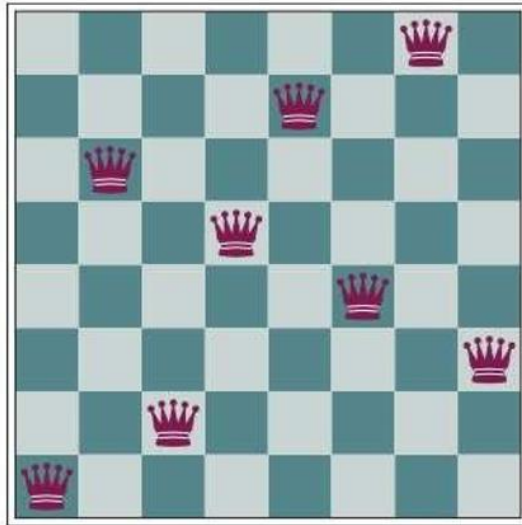
Thuật toán tìm kiếm **leo đồi** được thể hiện trong Hình 4.2. Nó theo dõi một trạng thái hiện tại và trong mỗi lần lặp, di chuyển đến trạng thái lân cận có giá trị cao nhất—tức là nó đi theo hướng mang lại **độ dốc dâng lên dốc nhất**. Nó kết thúc khi đạt đến một “đỉnh” nơi không có trạng thái lân cận nào có giá trị cao hơn. Leo đồi không nhìn xa hơn các trạng thái lân cận ngay lập tức của trạng thái hiện tại. Điều này giống như việc cố gắng tìm đỉnh núi Everest trong một màn sương dày đặc trong khi bị mất trí nhớ. Lưu ý rằng một cách để sử dụng tìm kiếm leo đồi là sử dụng giá trị âm của hàm chi phí heuristic làm hàm mục tiêu; điều đó sẽ leo cục bộ đến trạng thái có khoảng cách heuristic nhỏ nhất đến mục tiêu.

Để minh họa việc leo đồi, chúng ta sẽ sử dụng bài toán 8 quân hậu (Hình 4.3). Chúng ta sẽ sử dụng một **công thức trạng thái hoàn chỉnh**, có nghĩa là mọi trạng thái đều có tất cả các thành phần của một giải pháp, nhưng chúng có thể không ở đúng vị trí. Trong trường hợp này, mọi trạng thái đều có 8 quân hậu trên bàn cờ, mỗi cột một quân. Trạng thái ban đầu được chọn ngẫu nhiên và các trạng thái kế tiếp của một trạng thái là tất cả các trạng thái có thể được tạo ra bằng cách di chuyển một quân hậu duy nhất đến một ô vuông khác trong cùng một cột (vì vậy mỗi trạng thái có $8 \times 7 = 56$ trạng thái kế tiếp). Hàm chi phí heuristic h là số cặp quân hậu đang tấn công nhau; điều này sẽ bằng 0 chỉ khi có giải pháp. (Nó được tính là một cuộc tấn công nếu hai quân cờ ở cùng một hàng, ngay cả khi có một quân cờ ở giữa chúng.) Hình 4.3(b) cho thấy một trạng thái có $h = 17$. Hình cũng cho thấy các giá trị h của tất cả các trạng thái kế tiếp của nó.

```
function HILL-CLIMBING( problem) returns a state that is a local maximum
  current ← problem.Initial
  while true do
    neighbor ← a highest-valued successor state of current
```

if $\text{Value}(\text{neighbor}) \leq \text{Value}(\text{current})$ then return current
current ← neighbor

Hình 4.2 Thuật toán tìm kiếm leo đồi, là kỹ thuật tìm kiếm cục bộ cơ bản nhất. Ở mỗi bước, nút hiện tại được thay thế bằng nút lân cận tốt nhất.



(a)

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	17	13	16	13	16
17	14	17	15	17	14	16	16
17	17	16	18	15	17	15	17
18	14	17	15	15	14	17	16
14	14	13	17	12	14	12	18

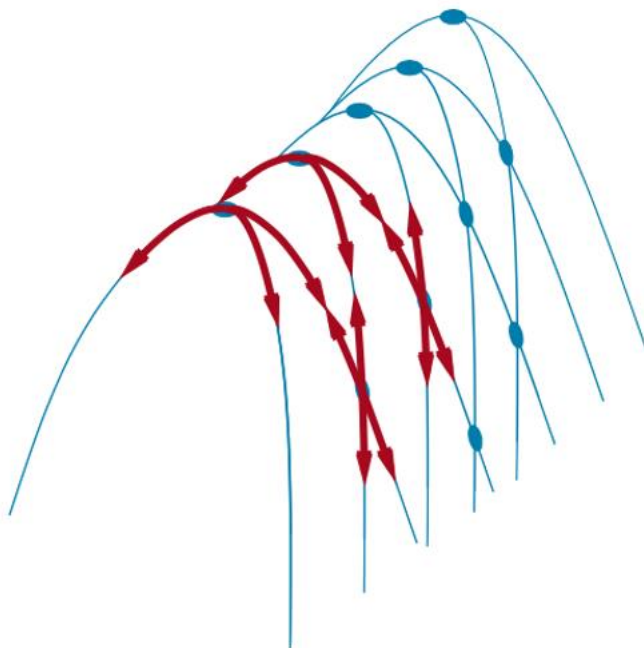
(b)

Hình 4.3 (a) Bài toán 8 quân hậu: đặt 8 quân hậu trên bàn cờ sao cho không có quân hậu nào tấn công quân hậu nào khác. (Một quân hậu tấn công bất kỳ quân cờ nào trong cùng hàng, cột hoặc đường chéo.) Vị trí này gần như là một giải pháp, ngoại trừ hai quân hậu ở cột thứ tư và thứ bảy tấn công nhau dọc theo đường chéo. (b) Một trạng thái 8 quân hậu với ước tính chi phí heuristic $h = 17$. Bàn cờ hiển thị giá trị của h cho mỗi trạng thái kế tiếp có thể có được bằng cách di chuyển một quân hậu trong cột của nó. Có 8 nước đi được cho là tốt nhất, với $h = 12$. Thuật toán leo đồi sẽ chọn một trong số này.

Leo đồi đôi khi được gọi là **tìm kiếm cục bộ tham lam** vì nó “chộp lấy” một trạng thái lân cận tốt mà không suy nghĩ trước về việc sẽ đi đâu tiếp theo. Mặc dù tham lam được coi là một trong bảy tội lỗi chết người, nhưng các thuật toán tham lam thường hoạt động khá tốt. Leo đồi có thể đạt được tiến bộ nhanh chóng hướng tới một giải pháp vì thường khá dễ để cải thiện một trạng thái tồi. Ví dụ, từ trạng thái trong Hình 4.3(b), chỉ mất năm bước để đạt đến trạng thái trong Hình 4.3(a), có $h = 1$ và gần như là một giải pháp. Thật không may, leo đồi có thể bị mắc kẹt vì bất kỳ lý do nào sau đây:

- **Cực đại cục bộ:** Một cực đại cục bộ là một đỉnh cao hơn mỗi trạng thái lân cận của nó nhưng thấp hơn cực đại toàn cục. Các thuật toán leo đồi khi đạt đến vùng lân cận của một cực đại cục bộ sẽ bị kéo lên đỉnh nhưng sau đó sẽ bị mắc kẹt vì không còn nơi nào khác để đi. Hình 4.1 minh họa vấn đề này một cách sơ đồ. Cụ thể hơn, trạng thái trong Hình 4.3(a) là một cực đại cục bộ (tức là một cực tiểu cục bộ cho chi phí h); mọi lần di chuyển một quân hậu duy nhất đều làm cho tình hình trở nên tồi tệ hơn.

- **Sườn núi (Ridges):** Một sườn núi được thể hiện trong Hình 4.4. Sườn núi tạo ra một chuỗi các cực đại cục bộ rất khó để các thuật toán tham lam vượt qua.



Hình 4.4 Minh họa tại sao sườn núi gây khó khăn cho việc leo đồi. Lưới các trạng thái (vòng tròn đậm) được đặt trên một sườn núi dâng lên từ trái sang phải, tạo ra một chuỗi các cực đại cục bộ không được kết nối trực tiếp với nhau. Từ mỗi cực đại cục bộ, tất cả các hành động có sẵn đều hướng xuống dốc. Các cấu trúc liên kết như thế này phổ biến trong không gian trạng thái có chiều thấp, chẳng hạn như các điểm trong mặt phẳng hai chiều. Nhưng trong không gian trạng thái với hàng trăm hoặc hàng nghìn chiều, hình ảnh trực quan này không còn đúng, và thường có ít nhất một vài chiều giúp có thể thoát khỏi sườn núi và cao nguyên.

- **Cao nguyên (Plateaus):** Một cao nguyên là một khu vực bằng phẳng của “bản đồ” không gian trạng thái. Nó có thể là một **cực đại cục bộ phẳng**, từ đó không có lối thoát lên dốc, hoặc một **vai dốc**, từ đó có thể có tiến triển. (Xem Hình 4.1.) Một tìm kiếm leo đồi có thể bị lạc khi lang thang trên cao nguyên.

Trong mỗi trường hợp, thuật toán đạt đến một điểm mà không có tiến bộ nào được thực hiện. Bắt đầu từ một trạng thái 8 quân hậu được tạo ngẫu nhiên, việc leo đồi dốc nhất bị mắc kẹt 86% thời gian, chỉ giải quyết được 14% các trường hợp bài toán. Mặt khác, nó hoạt động nhanh chóng, chỉ mất trung bình 4 bước khi thành công và 3 bước khi bị mắc kẹt—không tệ đối với một không gian trạng thái với $8^8 \approx 17$ triệu trạng thái.

Làm thế nào chúng ta có thể giải quyết được nhiều bài toán hơn? Một câu trả lời là tiếp tục đi khi chúng ta đạt đến một cao nguyên—để cho phép một **bước đi ngang** với hy vọng rằng cao nguyên đó thực sự là một vai dốc, như trong Hình 4.1. Nhưng nếu chúng ta thực sự đang ở trên một cực đại cục bộ phẳng, thì cách tiếp cận này sẽ lang thang trên cao nguyên mãi mãi. Do đó, chúng ta có thể giới hạn số bước đi ngang liên tiếp, dừng lại sau, chẳng hạn, 100 bước đi ngang liên tiếp. Điều này nâng tỷ lệ phần trăm các trường hợp bài toán được giải quyết bằng leo đồi từ 14% lên 94%.

Thành công đi kèm với một cái giá: thuật toán mất trung bình khoảng 21 bước cho mỗi trường hợp thành công và 64 bước cho mỗi trường hợp thất bại.

Nhiều biến thể của leo đồi đã được phát minh. **Leo đồi ngẫu nhiên** chọn ngẫu nhiên trong số các bước đi lên dốc; xác suất chọn có thể thay đổi theo độ dốc của bước đi lên. Điều này thường hội tụ chậm hơn so với leo đồi nhất, nhưng trong một số “bản đồ” trạng thái, nó tìm thấy các giải pháp tốt hơn. **Leo đồi chọn đầu tiên** thực hiện leo đồi ngẫu nhiên bằng cách tạo ra các trạng thái kế tiếp một cách ngẫu nhiên cho đến khi một trạng thái tốt hơn trạng thái hiện tại được tạo ra. Đây là một chiến lược tốt khi một trạng thái có nhiều (ví dụ: hàng nghìn) trạng thái kế tiếp.

Một biến thể khác là **leo đồi khởi động ngẫu nhiên**, áp dụng câu ngạn ngữ, “Nếu lúc đầu bạn không thành công, hãy cố gắng, cố gắng một lần nữa.” Nó thực hiện một loạt các tìm kiếm leo đồi từ các trạng thái ban đầu được tạo ngẫu nhiên, cho đến khi tìm thấy một mục tiêu. Nó hoàn thành với xác suất 1, bởi vì cuối cùng nó sẽ tạo ra một trạng thái mục tiêu làm trạng thái ban đầu. Nếu mỗi tìm kiếm leo đồi có xác suất p thành công, thì số lần khởi động lại được mong đợi cần thiết là $1/p$. Đối với các trường hợp 8 quân hậu không cho phép đi ngang, $p \approx 0,14$, vì vậy chúng ta cần khoảng 7 lần lặp để tìm mục tiêu (6 lần thất bại và 1 lần thành công). Số bước được mong đợi là chi phí của một lần lặp thành công cộng với $(1 - p)/p$ lần chi phí thất bại, hoặc khoảng 22 bước tất cả. Khi chúng ta cho phép các bước đi ngang, cần trung bình $1/0,94 \approx 1,06$ lần lặp và $(1 \times 21) + (0,06/0,94) \times 64 \approx 25$ bước. Đối với 8 quân hậu, thì việc leo đồi khởi động ngẫu nhiên thực sự rất hiệu quả. Ngay cả đối với ba triệu quân hậu, cách tiếp cận này có thể tìm thấy giải pháp trong vài giây.

Sự thành công của leo đồi phụ thuộc rất nhiều vào hình dạng của “bản đồ” không gian trạng thái: nếu có ít cực đại cục bộ và cao nguyên, leo đồi khởi động ngẫu nhiên sẽ tìm thấy một giải pháp tốt rất nhanh chóng. Mặt khác, nhiều bài toán thực tế có một bản đồ giống như một gia đình nhím hơi rải rác trên một sàn phẳng, với những con nhím nhỏ sống trên đầu mỗi chiếc kim của con nhím. Các bài toán NP-khó (xem Phụ lục A) thường có số lượng cực đại cục bộ theo cấp số nhân để bị mắc kẹt. Mặc dù vậy, một cực đại cục bộ hợp lý tốt thường có thể được tìm thấy sau một số ít lần khởi động lại.

4.1.1 Mô phỏng luyện kim (Simulated annealing)

Một thuật toán leo đồi mà không bao giờ thực hiện các bước đi “xuống dốc” hướng tới các trạng thái có giá trị thấp hơn (hoặc chi phí cao hơn) luôn dễ bị mắc kẹt trong một cực đại cục bộ. Ngược lại, một bước đi ngẫu nhiên thuần túy di chuyển đến một trạng thái kế tiếp mà không quan tâm đến giá trị cuối cùng cũng sẽ tìm thấy cực đại toàn cục, nhưng sẽ cực kỳ không hiệu quả. Do đó, có vẻ hợp lý khi cố gắng kết hợp leo đồi với một bước đi ngẫu nhiên theo cách mang lại cả hiệu quả và tính đầy đủ.

Mô phỏng luyện kim là một thuật toán như vậy. Trong luyện kim, **luyện kim** là quá trình được sử dụng để tôi luyện hoặc làm cứng kim loại và thủy tinh bằng cách nung chúng đến nhiệt độ cao và sau đó làm nguội dần dần, do đó cho phép vật liệu đạt đến trạng thái tinh thể năng lượng thấp. Để giải thích mô phỏng luyện kim, chúng ta chuyển quan điểm từ leo đồi sang **giảm độ dốc** (tức là giảm thiểu chi phí) và tưởng tượng nhiệm vụ đưa một quả bóng bàn vào khe sâu nhất trên một bề mặt rất gập ghềnh. Nếu chúng ta chỉ để quả bóng lăn, nó sẽ dừng lại ở một cực tiểu cục bộ. Nếu chúng ta lắc bề mặt, chúng ta có thể nảy quả bóng ra khỏi cực tiểu cục bộ—có lẽ vào một cực tiểu

cực bộ sâu hơn, nơi nó sẽ dành nhiều thời gian hơn. Bí quyết là chỉ lắc đủ mạnh để nảy quả bóng ra khỏi cực tiểu cực bộ nhưng không đủ mạnh để nó bật ra khỏi cực tiểu toàn cục. Giải pháp mô phỏng luyện kim là bắt đầu bằng cách lắc mạnh (tức là ở nhiệt độ cao) và sau đó giảm dần cường độ lắc (tức là hạ nhiệt độ).

Cấu trúc tổng thể của thuật toán mô phỏng luyện kim (Hình 4.5) tương tự như leo đồi. Tuy nhiên, thay vì chọn nước đi tốt nhất, nó chọn một nước đi ngẫu nhiên. Nếu nước đi cải thiện tình hình, nó luôn được chấp nhận. Ngược lại, thuật toán chấp nhận nước đi với một xác suất nhỏ hơn 1. Xác suất giảm theo cấp số nhân với “mức độ tệ” của nước đi—lượng ΔE mà giá trị bị giảm đi. Xác suất cũng giảm khi “nhiệt độ” T giảm xuống: các nước đi “tồi” có nhiều khả năng được cho phép ở lúc bắt đầu khi T cao, và chúng trở nên ít có khả năng hơn khi T giảm. Nếu lịch trình giảm T xuống 0 đủ chậm, thì một thuộc tính của phân bố Boltzmann, $e^{\Delta E/T}$, là tất cả xác suất được tập trung vào các cực đại toàn cục, mà thuật toán sẽ tìm thấy với xác suất tiến đến 1.

```
function SIMULATED-ANNEALING( problem, schedule) returns a solution state
  current ← problem.Initial
  for t = 1 to ∞ do
    T ← schedule(t)
    if T = 0 then return current
    next ← a randomly selected successor of current
    ΔE ← VALUE(current) – VALUE(next)
    if ΔE > 0 then current ← next
    else current ← next only with probability  $e^{\Delta E/T}$ 
```

Hình 4.5 Thuật toán mô phỏng luyện kim, một phiên bản của leo đồi ngẫu nhiên trong đó một số nước đi xuống dốc được cho phép. Đầu vào **schedule** xác định giá trị của “nhiệt độ” T như là một hàm của thời gian.

Mô phỏng luyện kim đã được sử dụng để giải các bài toán bố trí VLSI bắt đầu từ những năm 1980. Nó đã được áp dụng rộng rãi cho việc lập lịch trình nhà máy và các tác vụ tối ưu hóa quy mô lớn khác.

4.1.2 Tìm kiếm chùm cực bộ (*Local beam search*)

Việc chỉ giữ một nút trong bộ nhớ có vẻ là một phản ứng cực đoan đối với vấn đề hạn chế bộ nhớ. Thuật toán **tìm kiếm chùm cực bộ** theo dõi k trạng thái thay vì chỉ một. Nó bắt đầu với k trạng thái được tạo ngẫu nhiên. Ở mỗi bước, tất cả các trạng thái kế tiếp của tất cả k trạng thái được tạo ra. Nếu bất kỳ trạng thái nào là mục tiêu, thuật toán sẽ dừng lại. Ngược lại, nó chọn k trạng thái kế tiếp tốt nhất từ danh sách đầy đủ và lặp lại.

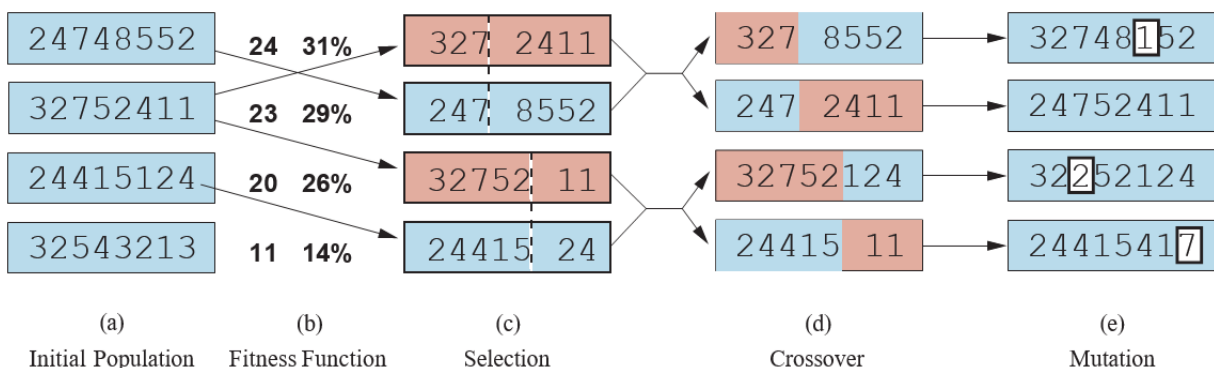
Thoạt nhìn, một tìm kiếm chùm cực bộ với k trạng thái có vẻ không khác gì việc chạy k lần khởi động lại ngẫu nhiên song song thay vì tuần tự. Trên thực tế, hai thuật toán này khá khác nhau. Trong tìm kiếm khởi động lại ngẫu nhiên, mỗi quá trình tìm kiếm chạy độc lập với các quá trình khác. Trong tìm kiếm chùm cực bộ, thông tin hữu ích được truyền giữa các luồng tìm kiếm song song. Trên thực tế, các trạng thái tạo ra các trạng thái kế tiếp tốt nhất nói với những trạng thái khác, “Hãy đến đây, cỏ xanh hơn!” Thuật toán nhanh chóng từ bỏ các tìm kiếm không hiệu quả và chuyển nguồn lực của nó đến nơi đang có nhiều tiến bộ nhất.

Tìm kiếm chùm cục bộ có thể gặp phải vấn đề thiếu sự đa dạng giữa k trạng thái—chúng có thể bị nhóm lại trong một vùng nhỏ của không gian trạng thái, làm cho việc tìm kiếm không hơn gì một phiên bản chậm hơn k lần của leo đồi. Một biến thể được gọi là **tìm kiếm chùm ngẫu nhiên**, tương tự như leo đồi ngẫu nhiên, giúp giảm bớt vấn đề này. Thay vì chọn k trạng thái kế tiếp hàng đầu, tìm kiếm chùm ngẫu nhiên chọn các trạng thái kế tiếp với xác suất tỷ lệ thuận với giá trị của trạng thái kế tiếp, do đó tăng tính đa dạng.

4.1.3 Thuật toán tiến hóa (Evolutionary algorithms)

Các **thuật toán tiến hóa** có thể được xem như các biến thể của tìm kiếm chùm ngẫu nhiên được thúc đẩy rõ ràng bởi phép ẩn dụ của **chọn lọc tự nhiên** trong sinh học: có một **quần thể** cá thể (trạng thái), trong đó các cá thể phù hợp nhất (giá trị cao nhất) tạo ra **con cái** (trạng thái kế tiếp) tạo thành thế hệ tiếp theo, một quá trình được gọi là **tái tổ hợp**. Có vô số dạng thuật toán tiến hóa, khác nhau theo những cách sau:

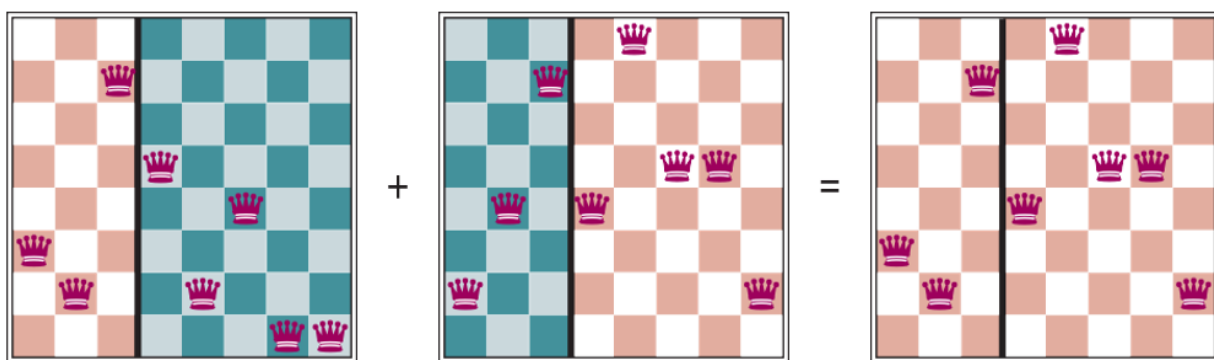
- **Kích thước của quần thể.**
- **Biểu diễn của mỗi cá thể.** Trong các **thuật toán di truyền**, mỗi cá thể là một chuỗi trên một bảng chữ cái hữu hạn (thường là một chuỗi Boolean), giống như DNA là một chuỗi trên bảng chữ cái ACGT. Trong các **chiến lược tiến hóa**, một cá thể là một chuỗi các số thực, và trong **lập trình di truyền**, một cá thể là một chương trình máy tính.
- **Số lượng kết hợp, ρ** , là số lượng cha mẹ kết hợp với nhau để tạo thành con cái. Trường hợp phổ biến nhất là $\rho = 2$: hai cha mẹ kết hợp “gen” của họ (các phần của biểu diễn của họ) để tạo thành con cái. Khi $\rho = 1$ chúng ta có tìm kiếm chùm ngẫu nhiên (có thể được xem là sinh sản vô tính). Có thể có $\rho > 2$, điều này hiếm khi xảy ra trong tự nhiên nhưng đủ để mô phỏng trên máy tính.
- **Quá trình chọn lọc** để chọn các cá thể sẽ trở thành cha mẹ của thế hệ tiếp theo: một khả năng là chọn từ tất cả các cá thể với xác suất tỷ lệ thuận với điểm phù hợp của họ. Một khả năng khác là chọn ngẫu nhiên n cá thể ($n\rho$), và sau đó chọn ρ cá thể phù hợp nhất làm cha mẹ.
- **Quy trình tái tổ hợp.** Một cách tiếp cận phổ biến (giả sử $\rho = 2$), là chọn ngẫu nhiên một **điểm giao chéo** để tách mỗi chuỗi cha mẹ, và tái tổ hợp các phần để tạo thành hai đứa con, một đứa có phần đầu tiên của cha mẹ 1 và phần thứ hai của cha mẹ 2; đứa kia có phần thứ hai của cha mẹ 1 và phần đầu tiên của cha mẹ 2.
- **Tỷ lệ đột biến**, xác định tần suất con cái có đột biến ngẫu nhiên đối với biểu diễn của chúng. Khi một con cái đã được tạo ra, mỗi bit trong thành phần của nó sẽ bị lật với xác suất bằng tỷ lệ đột biến.
- **Thành phần của thế hệ tiếp theo.** Đây có thể chỉ là những đứa con mới được tạo thành, hoặc nó có thể bao gồm một vài cha mẹ có điểm cao nhất từ thế hệ trước (một phương pháp gọi là **chọn lọc tinh hoa**, đảm bảo rằng độ phù hợp tổng thể sẽ không bao giờ giảm theo thời gian). Thực hành **loại bỏ**, trong đó tất cả các cá thể dưới một ngưỡng nhất định bị loại bỏ, có thể dẫn đến tăng tốc độ (Baum và cộng sự, 1995).



Hình 4.6 Một thuật toán di truyền, được minh họa cho các chuỗi chữ số đại diện cho các trạng thái 8 quân hậu. Quần thể ban đầu trong (a) được xếp hạng theo một hàm phù hợp trong (b) dẫn đến các cặp để giao phối trong (c). Chúng tạo ra con cái trong (d), chịu đột biến trong (e).

Hình 4.6(a) cho thấy một quần thể gồm bốn chuỗi 8 chữ số, mỗi chuỗi đại diện cho một trạng thái của bài toán 8 quân hậu: chữ số thứ c đại diện cho số hàng của quân hậu ở cột c . Trong (b), mỗi trạng thái được đánh giá bằng hàm phù hợp. Giá trị phù hợp cao hơn là tốt hơn, vì vậy đối với bài toán 8 quân hậu, chúng ta sử dụng số cặp quân hậu không tấn công nhau, có giá trị $8 \times 7 / 2 = 28$ cho một giải pháp. Các giá trị của bốn trạng thái trong (b) là 24, 23, 20 và 11. Các điểm phù hợp sau đó được chuẩn hóa thành xác suất, và các giá trị kết quả được hiển thị bên cạnh các giá trị phù hợp trong (b).

Trong (c), hai cặp cha mẹ được chọn, phù hợp với xác suất trong (b). Lưu ý rằng một cá thể được chọn hai lần và một cá thể không được chọn lần nào. Đối với mỗi cặp được chọn, một **điểm giao chéo** (đường chấm chấm) được chọn ngẫu nhiên. Trong (d), chúng ta giao chéo các chuỗi cha mẹ tại các điểm giao chéo, tạo ra con cái mới. Ví dụ, đưa con đầu tiên của cặp đầu tiên nhận ba chữ số đầu tiên (327) từ cha mẹ đầu tiên và các chữ số còn lại (48552) từ cha mẹ thứ hai. Các trạng thái 8 quân hậu liên quan đến bước tái tổ hợp này được thể hiện trong Hình 4.7.



Hình 4.7 Các trạng thái 8 quân hậu tương ứng với hai cha mẹ đầu tiên trong Hình 4.6(c) và đưa con đầu tiên trong Hình 4.6(d). Các cột màu xanh lá cây bị mất trong bước giao chéo và các cột màu đỏ được giữ lại. (Để diễn giải các số trong Hình 4.6: hàng 1 là hàng dưới cùng, và 8 là hàng trên cùng.)

Cuối cùng, trong (e), mỗi vị trí trong mỗi chuỗi chịu đột biến ngẫu nhiên với một xác suất nhỏ độc lập. Một chữ số đã bị đột biến trong đũa con đầu tiên, thứ ba và thứ tư. Trong bài toán 8 quân hậu, điều này tương ứng với việc chọn một quân hậu ngẫu nhiên và di chuyển nó đến một ô vuông ngẫu nhiên trong cột của nó. Thường thì quần thể đa dạng sớm trong quá trình, vì vậy giao chéo thường thực hiện các bước lớn trong không gian trạng thái sớm trong quá trình tìm kiếm (như trong mô phỏng luyện kim). Sau nhiều thế hệ chọn lọc hướng tới độ phù hợp cao hơn, quần thể trở nên ít đa dạng hơn, và các bước nhỏ hơn là điển hình. Hình 4.8 mô tả một thuật toán thực hiện tất cả các bước này.

```
function GENETIC-ALGORITHM( population, fitness) returns an individual
repeat
  weights ← WEIGHTED-BY(population, fitness)
  population2 ← empty list
  for i = 1 to SIZE( population) do
    parent1, parent2 ← Weighted-Random-Choices(population, weights, 2)
    child ← Reproduce(parent1, parent2)
    if (small random probability) then child ← MUTATE(child)
    add child to population2
  population ← population2
until some individual is fit enough, or enough time has elapsed
return the best individual in population, according to fitness

function REPRODUCE(parent1, parent2) returns an individual
n ← Length(parent1)
c ← random number from 1 to n
return Append(Substring(parent1, 1, c), Substring(parent2, c + 1, n))
```

Hình 4.8 Một thuật toán di truyền. Trong hàm, **population** là một danh sách có thứ tự các cá thể, **weights** là một danh sách các giá trị phù hợp tương ứng cho mỗi cá thể và **fitness** là một hàm để tính toán các giá trị này.

Các thuật toán di truyền tương tự như tìm kiếm chùm ngẫu nhiên, nhưng có thêm hoạt động giao chéo. Điều này có lợi nếu có các khối thực hiện các chức năng hữu ích. Ví dụ, có thể việc đặt ba quân hậu đầu tiên vào các vị trí 2, 4 và 6 (nơi chúng không tấn công nhau) tạo thành một khối hữu ích có thể được kết hợp với các khối hữu ích khác xuất hiện ở các cá thể khác để xây dựng một giải pháp. Có thể chứng minh bằng toán học rằng, nếu các khối không phục vụ một mục đích—ví dụ nếu các vị trí của mã di truyền được hoán vị ngẫu nhiên—thì giao chéo không mang lại lợi thế nào.

Lý thuyết về thuật toán di truyền giải thích cách hoạt động này bằng cách sử dụng ý tưởng về một **schema**, là một chuỗi con trong đó một số vị trí có thể không được chỉ định. Ví dụ, schema **246***** mô tả tất cả các trạng thái 8 quân hậu trong đó ba quân hậu đầu tiên lần lượt ở các vị trí 2, 4 và 6. Các chuỗi khớp với schema (chẳng hạn như 24613578) được gọi là các **trường hợp** của schema. Có thể chứng minh rằng nếu độ phù hợp trung bình của các trường hợp của một schema cao hơn mức trung bình, thì số lượng các trường hợp của schema sẽ tăng theo thời gian.

Tiến hóa và Tìm kiếm

Lý thuyết về tiến hóa được phát triển bởi Charles Darwin trong *Về nguồn gốc các loài bằng con đường chọn lọc tự nhiên* (1859) và độc lập bởi Alfred Russel Wallace (1858). Ý tưởng trung tâm rất đơn giản: các biến dị xảy ra trong sinh sản và sẽ được bảo tồn trong các thế hệ liên tiếp gần như tỷ lệ thuận với tác động của chúng đối với độ phù hợp sinh sản.

Lý thuyết của Darwin được phát triển mà không có kiến thức về cách các đặc điểm của sinh vật có thể được thừa hưởng và sửa đổi. Các quy luật xác suất chi phối các quá trình này lần đầu tiên được xác định bởi Gregor Mendel (1866), một nhà sư đã thí nghiệm với đậu ngọt. Rất lâu sau đó, Watson và Crick (1953) đã xác định cấu trúc của phân tử DNA và bảng chữ cái của nó, AGTC (adenine, guanine, thymine, cytosine). Trong mô hình tiêu chuẩn, biến dị xảy ra cả do đột biến điểm trong chuỗi chữ cái và do “giao chéo” (trong đó DNA của một đứa con được tạo ra bằng cách kết hợp các đoạn DNA dài từ mỗi cha mẹ).

Sự tương đồng với các thuật toán tìm kiếm cục bộ đã được mô tả; sự khác biệt chính giữa tìm kiếm chùm ngẫu nhiên và tiến hóa là việc sử dụng sinh sản hữu tính, trong đó các trạng thái kế tiếp được tạo ra từ nhiều cá thể hơn là chỉ một. Tuy nhiên, các cơ chế thực tế của tiến hóa phong phú hơn nhiều so với hầu hết các thuật toán di truyền cho phép. Ví dụ, đột biến có thể liên quan đến đảo ngược, sao chép và di chuyển các đoạn DNA lớn; một số virus mượn DNA từ một sinh vật và chèn nó vào một sinh vật khác; và có những gen chuyển vị không làm gì ngoài việc sao chép chính chúng hàng nghìn lần trong bộ gen.

Thậm chí còn có những gen đầu độc các tế bào từ những người bạn đời tiềm năng không mang gen đó, do đó làm tăng cơ hội sao chép của chính chúng. Quan trọng nhất là thực tế là chính các gen mã hóa các cơ chế mà bộ gen được tái tạo và dịch thành một sinh vật. Trong các thuật toán di truyền, các cơ chế đó là một chương trình riêng biệt không được biểu diễn trong các chuỗi đang được thao tác. Tiến hóa Darwin có vẻ không hiệu quả, đã tạo ra một cách mù quáng khoảng 10^{43} sinh vật mà không cải thiện các heuristic tìm kiếm của nó một chút nào. Nhưng học tập đóng một vai trò trong tiến hóa. Mặc dù nhà tự nhiên học vĩ đại người Pháp Jean Lamarck (1809) đã sai khi đề xuất rằng các đặc điểm có được bằng cách thích nghi trong suốt cuộc đời của một sinh vật sẽ được truyền lại cho con cái của nó, nhưng lý thuyết tương tự bề ngoài của James Baldwin (1896) là đúng: học tập có thể làm giảm hiệu quả bản đồ phù hợp, dẫn đến sự tăng tốc độ tiến hóa. Một sinh vật có một đặc điểm không hoàn toàn thích nghi với môi trường của nó sẽ truyền lại đặc điểm đó nếu nó cũng có đủ tính dẻo để học cách thích nghi với môi trường theo cách có lợi. Các mô phỏng trên máy tính (Hinton và Nowlan, 1987) xác nhận rằng **hiệu ứng Baldwin** này là có thật, và một hậu quả là những thứ khó học sẽ kết thúc trong bộ gen, nhưng những thứ dễ học thì không cần thiết phải ở đó (Morgan và Griffiths, 2015).

Rõ ràng, hiệu ứng này khó có thể đáng kể nếu các bit liên kết hoàn toàn không liên quan đến nhau, bởi vì khi đó sẽ có ít khối liên kết cung cấp một lợi ích nhất quán. Các thuật toán di truyền hoạt động tốt nhất khi các schema tương ứng với các thành phần có ý nghĩa của một giải pháp. Ví dụ, nếu chuỗi là một biểu diễn của một ăng-ten, thì các schema có thể đại diện cho các thành phần của ăng-ten, chẳng hạn như gương phản xạ và bộ làm lệch. Một thành phần tốt có khả năng tốt trong nhiều thiết kế khác nhau. Điều này cho thấy rằng việc sử dụng thành công các thuật toán di truyền đòi hỏi phải có sự thiết kế cẩn thận về cách biểu diễn.

Trong thực tế, các thuật toán di truyền có vị trí của chúng trong bối cảnh rộng lớn của các phương pháp tối ưu hóa (Marler và Arora, 2004), đặc biệt là đối với các bài toán có cấu trúc phức tạp như bố trí mạch điện hoặc lập lịch trình công việc, và gần đây hơn là để tối ưu hóa kiến trúc của các mạng thần kinh sâu (Miikkulainen và cộng sự, 2019). Không rõ sức hấp dẫn của các thuật toán di truyền xuất phát từ sự ưu việt của chúng trên các tác vụ cụ thể, hay từ phép ẩn dụ hấp dẫn của tiến hóa.

4.2 Tìm kiếm cục bộ trong không gian liên tục

Trong Chương 2, chúng ta đã giải thích sự khác biệt giữa môi trường rời rạc và liên tục, chỉ ra rằng hầu hết các môi trường trong thế giới thực là liên tục. Một không gian hành động liên tục có một hệ số phân nhánh vô hạn, và do đó không thể được xử lý bởi hầu hết các thuật toán mà chúng ta đã đề cập cho đến nay (ngoại trừ tìm kiếm leo đồi chọn đầu tiên và mô phỏng luyện kim).

Mục này cung cấp một giới thiệu rất ngắn gọn về một số kỹ thuật tìm kiếm cục bộ cho các không gian liên tục. Kho tàng tài liệu về chủ đề này là rất lớn; nhiều kỹ thuật cơ bản có nguồn gốc từ thế kỷ 17, sau sự phát triển của vi tích phân bởi Newton và Leibniz. Chúng ta tìm thấy cách sử dụng các kỹ thuật này ở một số nơi trong cuốn sách này, bao gồm các chương về học tập, thị giác và robot.

Chúng ta bắt đầu với một ví dụ. Giả sử chúng ta muốn đặt ba sân bay mới ở bất cứ đâu tại Romania, sao cho tổng bình phương khoảng cách đường thẳng từ mỗi thành phố trên bản đồ đến sân bay gần nhất của nó là nhỏ nhất. (Xem Hình 3.1 để biết bản đồ Romania.) **Không gian trạng thái** sau đó được xác định bởi tọa độ của ba sân bay: (x_1, y_1) , (x_2, y_2) và (x_3, y_3) . Đây là một không gian sáu chiều; chúng ta cũng nói rằng các trạng thái được xác định bởi sáu **biến**. Nói chung, các trạng thái được xác định bởi một vector n -chiều của các biến, x . Di chuyển trong không gian này tương ứng với việc di chuyển một hoặc nhiều sân bay trên bản đồ. **Hàm mục tiêu** $f(x) = f(x_1, y_1, x_2, y_2, x_3, y_3)$ tương đối dễ tính toán cho bất kỳ trạng thái cụ thể nào một khi chúng ta tính toán các thành phố gần nhất. Gọi C_i là tập hợp các thành phố mà sân bay gần nhất (trong trạng thái x) là sân bay i . Khi đó, chúng ta có

$$f(x) = f(x_1, y_1, x_2, y_2, x_3, y_3) = \sum_{i=1}^3 \sum_{c \in C_i} (x_i - x_c)^2 + (y_i - y_c)^2$$

Phương trình này đúng không chỉ cho trạng thái x mà còn cho các trạng thái trong vùng lân cận cục bộ của x . Tuy nhiên, nó không đúng trên toàn cục; nếu chúng ta đi quá xa khỏi x (bằng cách thay đổi vị trí của một hoặc nhiều sân bay với một lượng lớn) thì tập hợp các thành phố gần nhất cho sân bay đó sẽ thay đổi, và chúng ta cần tính toán lại C_i .

Một cách để xử lý một không gian trạng thái liên tục là **rời rạc hóa** nó. Ví dụ, thay vì cho phép các vị trí (x_i, y_i) là bất kỳ điểm nào trong không gian hai chiều liên tục, chúng ta có thể giới hạn chúng ở các điểm cố định trên một lưới hình chữ nhật với khoảng cách có kích thước *delta* (delta). Khi đó, thay vì có vô số trạng thái kế tiếp, mỗi trạng thái trong không gian sẽ chỉ có 12 trạng thái kế tiếp, tương ứng với việc tăng một trong 6 biến lên *pmdelta*. Sau đó, chúng ta có thể áp dụng bất kỳ thuật toán tìm kiếm cục bộ nào của chúng ta cho không gian rời rạc này. Ngoài ra, chúng ta có thể làm cho hệ số phân nhánh hữu hạn bằng cách lấy mẫu các trạng thái kế tiếp một cách ngẫu nhiên, di chuyển theo một hướng ngẫu nhiên với một lượng nhỏ, *delta*. Các phương pháp đo

lường tiến độ bằng sự thay đổi trong giá trị của hàm mục tiêu giữa hai điểm gần nhau được gọi là các phương pháp **gradient thực nghiệm**. Tìm kiếm gradient thực nghiệm cũng giống như leo đồi dốc nhất trong một phiên bản rời rạc hóa của không gian trạng thái. Việc giảm giá trị của *delta* theo thời gian có thể mang lại một giải pháp chính xác hơn, nhưng không nhất thiết phải hội tụ đến một cực đại toàn cục trong giới hạn.

Thường thì chúng ta có một hàm mục tiêu được biểu diễn dưới dạng toán học sao cho chúng ta có thể sử dụng vi tích phân để giải bài toán một cách **phân tích** thay vì **thực nghiệm**. Nhiều phương pháp cố gắng sử dụng **gradient** của bản đồ để tìm một cực đại. Gradient của hàm mục tiêu là một vector *nabla f* cho biết độ lớn và hướng của độ dốc dốc nhất. Đối với bài toán của chúng ta, chúng ta có

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3} \right).$$

Trong một số trường hợp, chúng ta có thể tìm thấy một cực đại bằng cách giải phương trình $\nabla f = 0$. (Điều này có thể được thực hiện, ví dụ, nếu chúng ta chỉ đặt một sân bay; giải pháp là trung bình cộng của tọa độ của tất cả các thành phố.) Tuy nhiên, trong nhiều trường hợp, phương trình này không thể giải được dưới dạng đóng. Ví dụ, với ba sân bay, biểu thức cho gradient phụ thuộc vào các thành phố gần nhất với mỗi sân bay trong trạng thái hiện tại. Điều này có nghĩa là chúng ta có thể tính toán gradient một cách cục bộ (nhưng không phải toàn cục); ví dụ,

$$\frac{\partial f}{\partial x_1} = 2 \sum_{c \in C_1} (x_1 - x_c).$$

Với một biểu thức đúng cục bộ cho gradient, chúng ta có thể thực hiện leo đồi dốc nhất bằng cách cập nhật trạng thái hiện tại theo công thức

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha \nabla f(\mathbf{x}),$$

trong đó *alpha* (alpha) là một hằng số nhỏ thường được gọi là **kích thước bước**. Tồn tại rất nhiều phương pháp để điều chỉnh *alpha*. Vấn đề cơ bản là nếu *alpha* quá nhỏ, cần quá nhiều bước; nếu *alpha* quá lớn, việc tìm kiếm có thể vượt qua cực đại. Kỹ thuật **tìm kiếm đường thẳng** cố gắng vượt qua tình thế tiến thoái lưỡng nan này bằng cách mở rộng hướng gradient hiện tại—thường bằng cách lặp lại nhân đôi *alpha*—cho đến khi *f* bắt đầu giảm trở lại. Điểm xảy ra điều này trở thành trạng thái hiện tại mới. Có một số trường phái tư tưởng về cách chọn hướng mới tại thời điểm này.

Đối với nhiều bài toán, thuật toán hiệu quả nhất là phương pháp **Newton–Raphson** đáng kính. Đây là một kỹ thuật chung để tìm **ng nghiệm** của các hàm—tức là giải các phương trình có dạng $g(x) = 0$. Nó hoạt động bằng cách tính toán một ước tính mới cho nghiệm *x* theo công thức của Newton

$$x \leftarrow x - g(x)/g'(x).$$

Để tìm một cực đại hoặc cực tiểu của f , chúng ta cần tìm $\mathbf{x}$ sao cho gradient là một vector 0 (tức là $\nabla f(\mathbf{x}) = 0$). Do đó, $g(\mathbf{x})$ trong công thức của Newton trở thành $\nabla f(\mathbf{x})$, và phương trình cập nhật có thể được viết dưới dạng ma trận–vector là

$$\mathbf{x} \leftarrow \mathbf{x} - \mathbf{H}_f^{-1}(\mathbf{x}) \nabla f(\mathbf{x}),$$

trong đó $H_f(\mathbf{x})$ là ma trận **Hessian** của các đạo hàm bậc hai, với các phần tử H_{ij} được cho bởi $\partial^2 f / \partial x_i \partial x_j$. Đối với ví dụ về sân bay của chúng ta, chúng ta có thể thấy từ Phương trình (4.2) rằng $H_f(\mathbf{x})$ đặc biệt đơn giản: các phần tử ngoài đường chéo bằng 0 và các phần tử trên đường chéo cho sân bay i chỉ bằng hai lần số thành phố trong C_i . Một phép tính cho thấy một bước cập nhật sẽ di chuyển sân bay i trực tiếp đến trọng tâm của C_i , đây là cực tiểu của biểu thức cực bộ cho f từ Phương trình (4.1). Tuy nhiên, đối với các bài toán có số chiều cao, việc tính toán n^2 mục của Hessian và nghịch đảo nó có thể tốn kém, vì vậy nhiều phiên bản gần đúng của phương pháp Newton–Raphson đã được phát triển.

Các phương pháp tìm kiếm cực bộ phải chịu các cực đại cực bộ, sườn núi và cao nguyên trong không gian trạng thái liên tục nhiều như trong không gian rời rạc. Khởi động lại ngẫu nhiên và mô phỏng luyện kim thường hữu ích. Tuy nhiên, các không gian liên tục có chiều cao là những nơi lớn mà rất dễ bị lạc.

Một chủ đề cuối cùng là **tối ưu hóa có ràng buộc**. Một bài toán tối ưu hóa là có ràng buộc nếu các giải pháp phải thỏa mãn một số ràng buộc cứng về các giá trị của các biến. Ví dụ, trong bài toán định vị sân bay của chúng ta, chúng ta có thể ràng buộc các địa điểm phải nằm trong Romania và trên đất liền (thay vì ở giữa các hồ). Độ khó của các bài toán tối ưu hóa có ràng buộc phụ thuộc vào bản chất của các ràng buộc và hàm mục tiêu. Thể loại được biết đến nhiều nhất là các bài toán **quy hoạch tuyến tính**, trong đó các ràng buộc phải là các bất đẳng thức tuyến tính tạo thành một tập hợp lồi và hàm mục tiêu cũng tuyến tính. Độ phức tạp về thời gian của quy hoạch tuyến tính là đa thức theo số biến.

Quy hoạch tuyến tính có lẽ là phương pháp được nghiên cứu rộng rãi nhất và hữu ích nhất cho việc tối ưu hóa. Nó là một trường hợp đặc biệt của bài toán tổng quát hơn là **tối ưu hóa lồi**, cho phép vùng ràng buộc là bất kỳ vùng lồi nào và mục tiêu là bất kỳ hàm nào lồi trong vùng ràng buộc. Trong những điều kiện nhất định, các bài toán tối ưu hóa lồi cũng có thể giải được bằng đa thức và có thể khả thi trong thực tế với hàng nghìn biến. Một số bài toán quan trọng trong học máy và lý thuyết điều khiển có thể được xây dựng dưới dạng bài toán tối ưu hóa lồi (xem Chương 21).

4.3 Tìm kiếm với các hành động không xác định

Trong Chương 3, chúng ta giả định một môi trường hoàn toàn có thể quan sát được, xác định và đã biết. Do đó, một tác nhân có thể quan sát trạng thái ban đầu, tính toán một chuỗi các hành động để đạt đến mục tiêu và thực hiện các hành động với “mắt nhắm”, không bao giờ phải sử dụng các giác quan của nó.

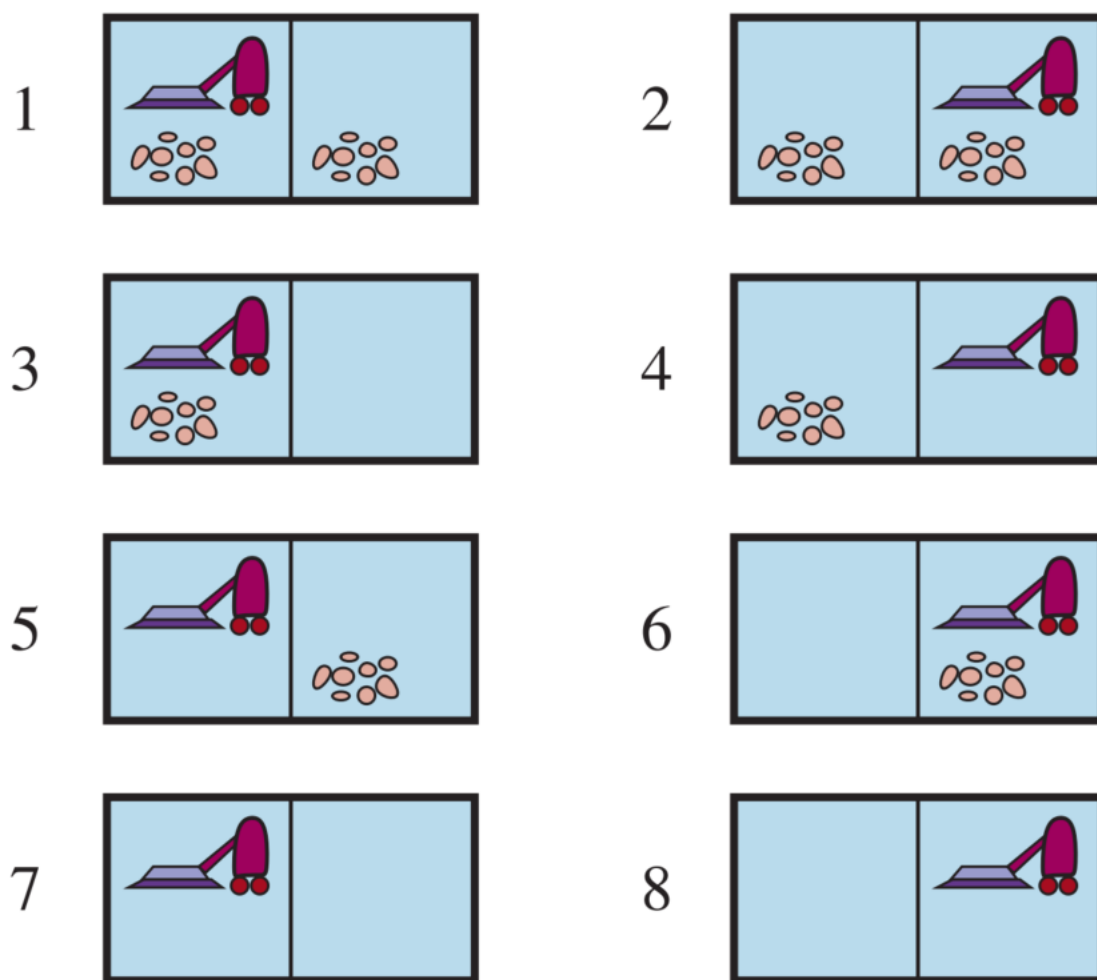
Tuy nhiên, khi môi trường có thể quan sát được một phần, tác nhân không chắc chắn về trạng thái mà nó đang ở; và khi môi trường không xác định, tác nhân không biết nó sẽ chuyển sang trạng thái nào sau khi thực hiện một hành động. Điều đó có nghĩa là thay vì suy nghĩ “Tôi đang ở trạng thái

s_1 và nếu tôi thực hiện hành động a tôi sẽ kết thúc ở trạng thái s_2 ,” một tác nhân bây giờ sẽ suy nghĩ “Tôi đang ở trạng thái s_1 hoặc s_3 , và nếu tôi thực hiện hành động a tôi sẽ kết thúc ở trạng thái s_2 , s_4 hoặc s_5 .” Chúng ta gọi một tập hợp các trạng thái vật lý mà tác nhân tin là có thể xảy ra là một **trạng thái niềm tin**.

Trong các môi trường có thể quan sát được một phần và không xác định, giải pháp cho một bài toán không còn là một chuỗi, mà là một **kế hoạch có điều kiện** (đôi khi được gọi là **kế hoạch dự phòng** hoặc **chiến lược**) chỉ định phải làm gì tùy thuộc vào những gì giác quan tác nhân nhận được trong khi thực hiện kế hoạch. Chúng ta kiểm tra tính không xác định trong mục này và khả năng quan sát một phần trong mục tiếp theo.

4.3.1 Thế giới chân không thất thường

Thế giới chân không từ Chương 2 có tám trạng thái, như trong Hình 4.9. Có ba hành động—**Right**, **Left**, và **Suck**—và mục tiêu là dọn sạch tất cả bụi bẩn (trạng thái 7 và 8). Nếu môi trường hoàn toàn có thể quan sát được, xác định và đã biết hoàn toàn, thì bài toán rất dễ giải quyết với bất kỳ thuật toán nào trong Chương 3, và giải pháp là một chuỗi hành động. Ví dụ, nếu trạng thái ban đầu là 1, thì chuỗi hành động [**Suck**, **Right**, **Suck**] sẽ đạt đến trạng thái mục tiêu, 8.



Hình 4.9 Tám trạng thái có thể có của thế giới chân không; trạng thái 7 và 8 là trạng thái mục tiêu.

Bây giờ giả sử rằng chúng ta giới thiệu tính không xác định dưới dạng một máy hút bụi mạnh nhưng thất thường. Trong thế giới chân không thất thường, hành động **Suck** hoạt động như sau:

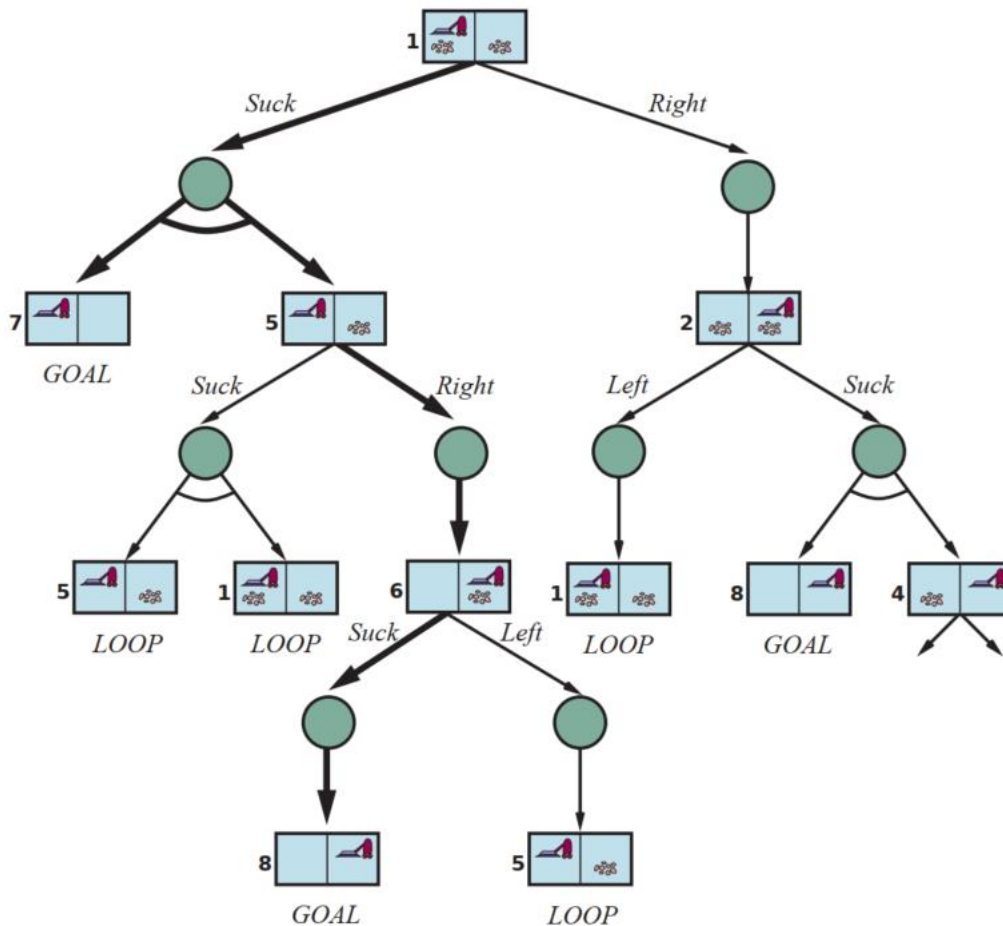
- Khi áp dụng cho một ô vuông bẩn, hành động làm sạch ô vuông và đôi khi cũng làm sạch bụi bẩn ở một ô vuông lân cận.
- Khi áp dụng cho một ô vuông sạch, hành động đôi khi làm bẩn thảm.

Để cung cấp một công thức chính xác cho bài toán này, chúng ta cần khái quát hóa khái niệm **mô hình chuyển đổi** từ Chương 3. Thay vì xác định mô hình chuyển đổi bằng một hàm **RESULT** trả về một trạng thái kết quả duy nhất, chúng ta sử dụng một hàm **RESULTS** trả về một tập hợp các trạng thái kết quả có thể xảy ra. Ví dụ, trong thế giới chân không thất thường, hành động **Suck** trong trạng thái 1 làm sạch hoặc chỉ vị trí hiện tại, hoặc cả hai vị trí: $Results(1, Suck) = \{5, 7\}$ Nếu chúng ta bắt đầu ở trạng thái 1, không có chuỗi hành động đơn lẻ nào giải quyết được bài toán, nhưng **kế hoạch có điều kiện** sau đây thì có: $[Suck, if State = 5 then [Right, Suck] else []]$ Ở đây chúng ta thấy rằng một kế hoạch có điều kiện có thể chứa các bước if-then-else; điều này có nghĩa là các giải pháp là các cây hơn là các chuỗi. Ở đây, điều kiện trong câu lệnh if kiểm tra xem trạng thái hiện tại là gì; đây là điều mà tác nhân sẽ có thể quan sát được trong thời gian chạy, nhưng không biết vào thời gian lập kế hoạch.

Ngoài ra, chúng ta có thể có một công thức kiểm tra giác quan thay vì trạng thái. Nhiều bài toán trong thế giới thực, vật lý là các bài toán dự phòng, vì việc dự đoán chính xác tương lai là không thể. Vì lý do này, nhiều người mở mắt khi đi lại.

4.3.2 Cây tìm kiếm AND-OR

Làm thế nào để chúng ta tìm ra các giải pháp có điều kiện cho các bài toán không xác định? Tương tự như trong Chương 3, chúng ta bắt đầu bằng cách xây dựng các cây tìm kiếm, nhưng ở đây các cây có một đặc điểm khác. Trong một môi trường xác định, sự phân nhánh duy nhất được tạo ra bởi các lựa chọn của chính tác nhân trong mỗi trạng thái: Tôi có thể thực hiện hành động này hoặc hành động kia. Chúng ta gọi các nút này là các **nút OR**. Trong thế giới chân không, ví dụ, tại một nút OR, tác nhân chọn Left, Right hoặc Suck. Trong một môi trường không xác định, sự phân nhánh cũng được tạo ra bởi sự lựa chọn của môi trường về kết quả cho mỗi hành động. Chúng ta gọi các nút này là các **nút AND**. Ví dụ, hành động Suck trong trạng thái 1 dẫn đến trạng thái niềm tin $\{5, 7\}$, vì vậy tác nhân sẽ cần tìm một kế hoạch cho trạng thái 5 và cho trạng thái 7. Hai loại nút này xen kẽ nhau, dẫn đến một **cây AND-OR** như được minh họa trong Hình 4.10.



Hình 4.10 Hai cấp độ đầu tiên của cây tìm kiếm cho thế giới chân không thất thường. Các nút trạng thái là các nút OR, nơi một hành động phải được chọn. Tại các nút AND, được hiển thị dưới dạng hình tròn, mọi kết quả phải được xử lý, như được chỉ ra bởi cung nối các nhánh đi ra. Giải pháp được tìm thấy được hiển thị bằng các đường đậm.

Một giải pháp cho một bài toán tìm kiếm AND–OR là một cây con của cây tìm kiếm hoàn chỉnh mà (1) có một nút mục tiêu ở mỗi lá, (2) chỉ định một hành động tại mỗi nút OR của nó và (3) bao gồm mọi nhánh kết quả tại mỗi nút AND của nó. Giải pháp được hiển thị bằng các đường đậm trong hình; nó tương ứng với kế hoạch được đưa ra trong Phương trình (4.3).

Hình 4.11 đưa ra một thuật toán đệ quy, tìm kiếm theo chiều sâu cho tìm kiếm đồ thị AND–OR. Một khía cạnh quan trọng của thuật toán là cách nó xử lý các chu kỳ, thường phát sinh trong các bài toán không xác định (ví dụ, nếu một hành động đôi khi không có tác dụng hoặc nếu một tác dụng ngoài ý muốn có thể được sửa chữa). Nếu trạng thái hiện tại giống hệt với một trạng thái trên đường đi từ gốc, thì nó trả về thất bại. Điều này không có nghĩa là không có giải pháp từ trạng thái hiện tại; nó chỉ đơn giản có nghĩa là nếu có một giải pháp không tuần hoàn, nó phải có thể đạt được từ trạng thái hiện tại trước đó, vì vậy phiên bản mới có thể bị loại bỏ. Với kiểm tra này, chúng ta đảm bảo rằng thuật toán sẽ kết thúc trong mọi không gian trạng thái hữu hạn, bởi vì mọi đường đi phải đạt đến mục tiêu, một ngõ cụt hoặc một trạng thái lặp lại. Lưu ý rằng thuật toán không

kiểm tra xem trạng thái hiện tại có phải là sự lặp lại của một trạng thái trên một đường đi khác từ gốc hay không, điều này rất quan trọng đối với hiệu quả.

Các đồ thị AND-OR có thể được khám phá theo chiều rộng hoặc theo chiều tốt nhất. Khái niệm về một hàm heuristic phải được sửa đổi để ước tính chi phí của một giải pháp có điều kiện thay vì một chuỗi, nhưng khái niệm về tính chấp nhận được vẫn được giữ nguyên và có một tương tự của thuật toán A* để tìm các giải pháp tối ưu. (Xem các ghi chú thư mục ở cuối chương.)

```
function AND-OR-SEARCH(problem) returns a conditional plan, or failure
  return Or-Search(problem, problem.Initial, [ ])
```

```
function OR-SEARCH(problem, state, path) returns a conditional plan, or failure
  if problem.IS-GOAL(state) then return the empty plan
  if IS-CYCLE(state, path) then return failure
  for each action in problem.Actions(state) do
    plan ← AND-SEARCH(problem, RESULTS(state, action), [state] + [path])
    if plan ≠ failure then return [action] + [plan]
  return failure
```

```
function AND-SEARCH(problem, states, path) returns a conditional plan, or failure
  for each si in states do
    plani ← OR-SEARCH(problem, si, path)
    if plani = failure then return failure
  return [if s1 then plan1 else if s2 then plan2 else ... if sn-1 then plann-1 else plann]
```

Hình 4.11 Một thuật toán để tìm kiếm các đồ thị AND-OR được tạo ra bởi các môi trường không xác định. Một giải pháp là một kế hoạch có điều kiện xem xét mọi kết quả không xác định và đưa ra một kế hoạch cho từng kết quả đó.

4.3.3 Cố gắng, cố gắng lại

Hãy xem xét một thế giới chân không trơn trượt, giống hệt với thế giới chân không thông thường (không thất thường) ngoại trừ việc các hành động di chuyển đôi khi thất bại, để lại tác nhân ở cùng một vị trí. Ví dụ, di chuyển Right trong trạng thái 1 dẫn đến trạng thái niềm tin $\{1, 2\}$. Hình 4.12 cho thấy một phần của đồ thị tìm kiếm; rõ ràng, không còn bất kỳ giải pháp không tuần hoàn nào từ trạng thái 1, và AND-OR-SEARCH sẽ trả về thất bại. Tuy nhiên, có một **giải pháp tuần hoàn**, đó là tiếp tục thử Right cho đến khi nó hoạt động. Chúng ta có thể biểu diễn điều này bằng một cấu trúc *while* mới:

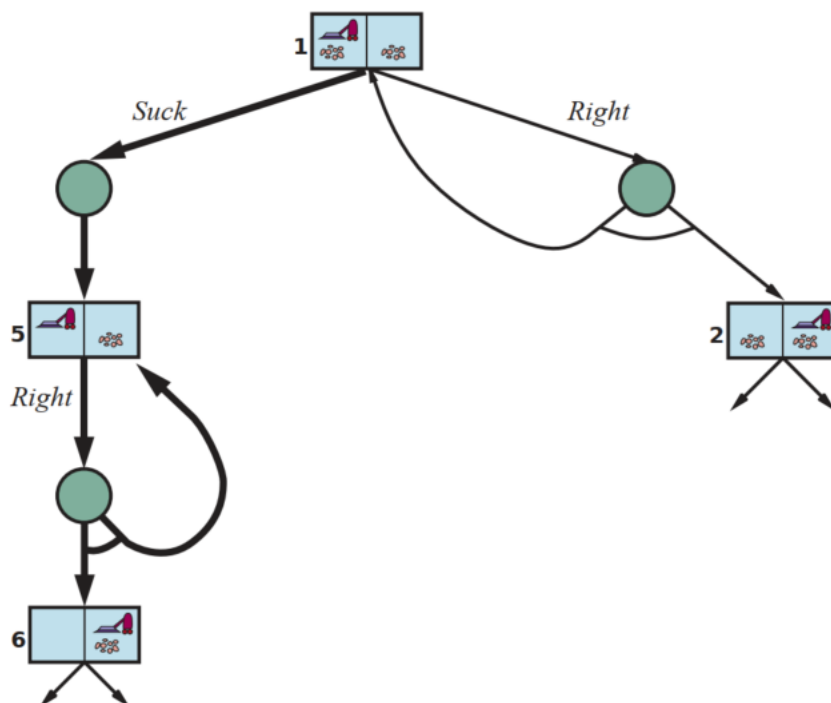
[Suck, while State = 5 do Right, Suck]

hoặc bằng cách thêm một nhãn để biểu thị một phần nào đó của kế hoạch và tham chiếu đến nhãn đó sau này:

[Suck, L1: Right, if State = 5 then L1 else Suck]

Khi nào một kế hoạch tuần hoàn là một giải pháp? Một điều kiện tối thiểu là mọi lá đều là trạng thái mục tiêu và một lá có thể đạt được từ mọi điểm trong kế hoạch. Ngoài ra, chúng ta cần xem

xét nguyên nhân của tính không xác định. Nếu thực sự cơ chế truyền động của robot hút bụi hoạt động một lúc nào đó, nhưng trượt ngẫu nhiên và độc lập vào những dịp khác, thì tác nhân có thể tin tưởng rằng nếu hành động được lặp lại đủ lần, cuối cùng nó sẽ hoạt động và kế hoạch sẽ thành công. Nhưng nếu tính không xác định là do một sự thật không được quan sát về robot hoặc môi trường—có lẽ một dây đai truyền động đã bị đứt và robot sẽ không bao giờ di chuyển—thì việc lặp lại hành động sẽ không giúp ích gì.



Hình 4.12 Một phần của đồ thị tìm kiếm cho một thế giới chân không trơn trượt, trong đó chúng ta đã hiển thị (một số) chu kỳ một cách rõ ràng. Tất cả các giải pháp cho bài toán này đều là các kế hoạch tuần hoàn vì không có cách nào để di chuyển một cách đáng tin cậy.

Một cách để hiểu quyết định này là nói rằng công thức bài toán ban đầu (hoàn toàn có thể quan sát được, không xác định) bị từ bỏ để chuyển sang một công thức khác (có thể quan sát được một phần, xác định) trong đó sự thất bại của kế hoạch tuần hoàn được gán cho một thuộc tính không được quan sát của dây đai truyền động. Trong Chương 12, chúng ta sẽ thảo luận về cách quyết định khả năng nào trong số một số khả năng không chắc chắn có nhiều khả năng hơn.

4.4 Tìm kiếm trong môi trường có thể quan sát được một phần

Bây giờ chúng ta chuyển sang bài toán về khả năng quan sát một phần, nơi các giác quan của tác nhân không đủ để xác định chính xác trạng thái. Điều đó có nghĩa là một số hành động của tác nhân sẽ nhằm mục đích giảm sự không chắc chắn về trạng thái hiện tại.

4.4.1 Tìm kiếm mà không có quan sát

Khi các giác quan của tác nhân không cung cấp bất kỳ thông tin nào cả, chúng ta có cái được gọi là **bài toán không có cảm biến** (hoặc một **bài toán phù hợp**). Lúc đầu, bạn có thể nghĩ rằng tác

nhân không có cảm biến không có hy vọng giải quyết một bài toán nếu nó không có ý tưởng về trạng thái mà nó bắt đầu, nhưng các giải pháp không có cảm biến lại phổ biến và hữu ích một cách đáng ngạc nhiên, chủ yếu là vì chúng không dựa vào cảm biến hoạt động đúng cách. Ví dụ, trong các hệ thống sản xuất, nhiều phương pháp khéo léo đã được phát triển để định hướng các bộ phận một cách chính xác từ một vị trí ban đầu không xác định bằng cách sử dụng một chuỗi các hành động mà không cần cảm biến nào cả. Đôi khi một kế hoạch không có cảm biến tốt hơn ngay cả khi có sẵn một kế hoạch có điều kiện với cảm biến. Ví dụ, các bác sĩ thường kê đơn thuốc kháng sinh phổ rộng thay vì sử dụng kế hoạch có điều kiện là làm xét nghiệm máu, sau đó chờ kết quả về, và sau đó kê một loại kháng sinh cụ thể hơn. Kế hoạch không có cảm biến tiết kiệm thời gian và tiền bạc, và tránh được nguy cơ nhiễm trùng trở nên tồi tệ hơn trước khi có kết quả xét nghiệm.

Hãy xem xét một phiên bản không có cảm biến của thế giới chân không (xác định). Giả sử rằng tác nhân biết địa lý của thế giới của nó, nhưng không biết vị trí của chính nó hoặc sự phân bố của bụi bẩn. Trong trường hợp đó, trạng thái niềm tin ban đầu của nó là $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (xem Hình 4.9). Bây giờ, nếu tác nhân di chuyển Right, nó sẽ ở một trong các trạng thái $\{2, 4, 6, 8\}$ —tác nhân đã có được thông tin mà không cần cảm nhận bất cứ điều gì! Sau [Right, Suck], tác nhân sẽ luôn kết thúc ở một trong các trạng thái $\{4, 8\}$. Cuối cùng, sau [Right, Suck, Left, Suck], tác nhân được đảm bảo đạt đến trạng thái mục tiêu 7, bất kể trạng thái bắt đầu là gì. Chúng ta nói rằng tác nhân có thể **cưỡng chế** thế giới vào trạng thái 7.

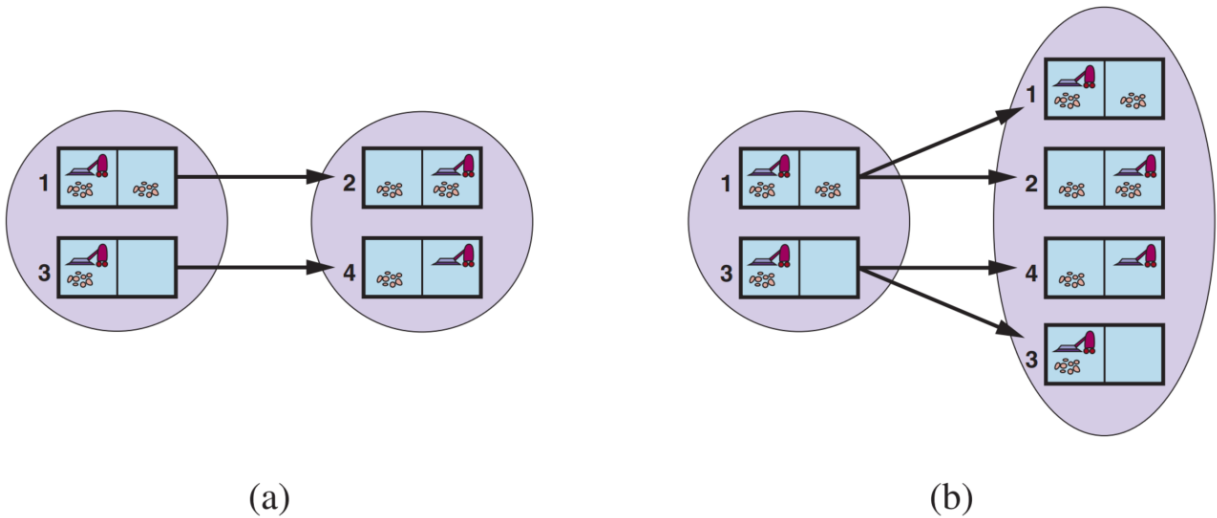
Giải pháp cho một bài toán không có cảm biến là một chuỗi các hành động, không phải là một kế hoạch có điều kiện (vì không có sự cảm nhận). Nhưng chúng ta tìm kiếm trong **không gian của các trạng thái niềm tin** thay vì các trạng thái vật lý. Trong không gian trạng thái niềm tin, bài toán hoàn toàn có thể quan sát được vì tác nhân luôn biết trạng thái niềm tin của chính nó. Hơn nữa, giải pháp (nếu có) cho một bài toán không có cảm biến luôn là một chuỗi các hành động. Điều này là do, như trong các bài toán thông thường của Chương 3, các giác quan nhận được sau mỗi hành động hoàn toàn có thể dự đoán được—chúng luôn trông rỗng! Vì vậy không có điều kiện dự phòng nào để lập kế hoạch. Điều này đúng ngay cả khi môi trường không xác định.

Chúng ta có thể giới thiệu các thuật toán mới cho các bài toán tìm kiếm không có cảm biến. Nhưng thay vào đó, chúng ta có thể sử dụng các thuật toán hiện có từ Chương 3 nếu chúng ta chuyển đổi bài toán vật lý cơ bản thành một **bài toán trạng thái niềm tin**, trong đó chúng ta tìm kiếm trên các trạng thái niềm tin thay vì các trạng thái vật lý. Bài toán ban đầu, P , có các thành phần $Actions_P$, $Result_P$, v.v., và bài toán trạng thái niềm tin có các thành phần sau:

- **Trạng thái:** Không gian trạng thái niềm tin chứa mọi tập con có thể có của các trạng thái vật lý. Nếu P có N trạng thái, thì bài toán trạng thái niềm tin có 2^N trạng thái niềm tin, mặc dù nhiều trong số đó có thể không thể đạt được từ trạng thái ban đầu.
- **Trạng thái ban đầu:** Thông thường là trạng thái niềm tin bao gồm tất cả các trạng thái trong P , mặc dù trong một số trường hợp, tác nhân sẽ có nhiều kiến thức hơn thế này.
- **Hành động:** Điều này hơi khó. Giả sử tác nhân đang ở trạng thái niềm tin $b = s_1, s_2$, nhưng $ACTIONS_P(s_1) \neq ACTIONS_P(s_2)$; thì tác nhân không chắc chắn về hành động nào là hợp pháp. Nếu chúng ta giả định rằng các hành động bất hợp pháp không có tác dụng đối với môi trường, thì việc lấy hợp của tất cả các hành động trong bất kỳ trạng thái vật lý nào trong trạng thái niềm tin hiện tại b là an toàn: $Actions(b) = \bigcup \{s \in b\} ACTIONS_P(s)$. Mặt khác, nếu một hành động bất hợp pháp có thể dẫn đến thảm họa, thì an

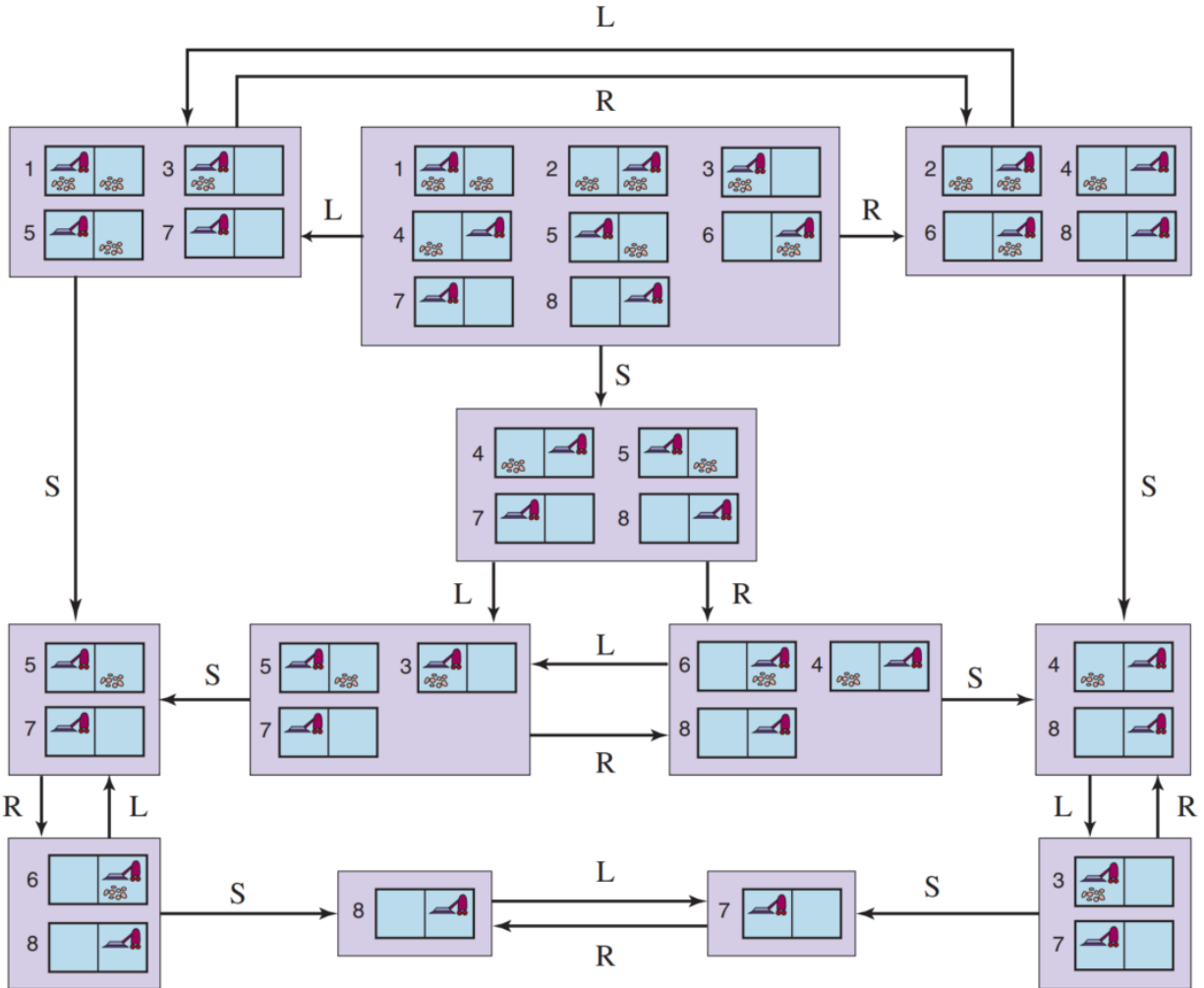
toàn hơn là chỉ cho phép giao, tức là tập hợp các hành động hợp pháp trong tất cả các trạng thái. Đối với thế giới chân không, mọi trạng thái đều có các hành động hợp pháp giống nhau, vì vậy cả hai phương pháp đều cho cùng một kết quả.

- **Mô hình chuyển đổi:** Đối với các hành động xác định, trạng thái niềm tin mới có một trạng thái kết quả cho mỗi trạng thái khả thi hiện tại (mặc dù một số trạng thái kết quả có thể giống nhau): $b' = RESULT(b, a) = \{s' : s' = RESULT_P(s, a) \text{ and } s \in b\}$. Với tính không xác định, trạng thái niềm tin mới bao gồm tất cả các kết quả có thể có của việc áp dụng hành động cho bất kỳ trạng thái nào trong trạng thái niềm tin hiện tại: $b' = RESULT(b, a) = \{s' : s' \in RESULTS_P(s, a) \text{ and } s \in b\} = \bigcup_{s \in b} Results_P(s, a)$. Kích thước của b' sẽ bằng hoặc nhỏ hơn b đối với các hành động xác định, nhưng có thể lớn hơn b đối với các hành động không xác định (xem Hình 4.13).
- **Kiểm tra mục tiêu:** Tác nhân có thể đạt được mục tiêu nếu bất kỳ trạng thái s nào trong trạng thái niềm tin thỏa mãn kiểm tra mục tiêu của bài toán cơ bản, $IS_GOAL_P(s)$. Tác nhân nhất thiết phải đạt được mục tiêu nếu mọi trạng thái đều thỏa mãn $IS_GOAL_P(s)$. Chúng ta đặt mục tiêu là nhất thiết phải đạt được mục tiêu.
- **Chi phí hành động:** Điều này cũng khó. Nếu cùng một hành động có thể có các chi phí khác nhau trong các trạng thái khác nhau, thì chi phí của việc thực hiện một hành động trong một trạng thái niềm tin nhất định có thể là một trong số các giá trị. (Điều này làm phát sinh một lớp bài toán mới, mà chúng ta khám phá trong Bài tập 4.MVAL.) Hiện tại, chúng ta giả định rằng chi phí của một hành động là như nhau trong tất cả các trạng thái và do đó có thể được chuyển trực tiếp từ bài toán vật lý cơ bản.



Hình 4.13 (a) Dự đoán trạng thái niềm tin tiếp theo cho thế giới chân không không có cảm biến với hành động xác định, Right. (b) Dự đoán cho cùng một trạng thái niềm tin và hành động trong phiên bản trơn trượt của thế giới chân không không có cảm biến.

Hình 4.14 cho thấy phần có thể đạt được của không gian trạng thái niềm tin cho thế giới chân không không có cảm biến, xác định. Chỉ có 12 trạng thái niềm tin có thể đạt được trong số $2^8 = 256$ trạng thái niềm tin có thể có.



Hình 4.14 Phần có thể đạt được của không gian trạng thái niềm tin cho thế giới chân không không có cảm biến, xác định. Mỗi hộp hình chữ nhật tương ứng với một trạng thái niềm tin duy nhất. Tại bất kỳ thời điểm nào, tác nhân có một trạng thái niềm tin nhưng không biết nó đang ở trạng thái vật lý nào. Trạng thái niềm tin ban đầu (hoàn toàn không biết) là hộp trên cùng ở giữa.

Các định nghĩa trên cho phép xây dựng tự động công thức bài toán trạng thái niềm tin từ định nghĩa của bài toán vật lý cơ bản. Sau khi điều này được thực hiện, chúng ta có thể giải các bài toán không có cảm biến bằng bất kỳ thuật toán tìm kiếm thông thường nào của Chương 3.

Trong tìm kiếm đồ thị thông thường, các trạng thái mới đạt được được kiểm tra để xem liệu chúng có được đạt đến trước đó hay không. Điều này cũng hoạt động đối với các trạng thái niềm tin; ví dụ, trong Hình 4.14, chuỗi hành động [Suck, Left, Suck] bắt đầu từ trạng thái ban đầu đạt đến trạng thái niềm tin giống như [Right, Left, Suck], cụ thể là {5, 7}. Bây giờ, hãy xem xét trạng thái niềm tin đạt được bởi [Left], cụ thể là {1, 3, 5, 7}. Rõ ràng, điều này không giống hệt {5, 7}, nhưng nó là một siêu tập. Chúng ta có thể loại bỏ (cắt tia) bất kỳ trạng thái niềm tin siêu tập nào như vậy. Tại sao? Bởi vì một giải pháp từ {1, 3, 5, 7} phải là một giải pháp cho mỗi trạng thái riêng lẻ 1, 3, 5 và 7, và do đó nó là một giải pháp cho bất kỳ sự kết hợp nào của các trạng thái riêng lẻ này, chẳng hạn như {5, 7}; do đó chúng ta không cần phải cố gắng giải {1, 3, 5, 7}, chúng ta có thể tập trung vào việc cố gắng giải trạng thái niềm tin dễ dàng hơn nhiều {5, 7}.

Ngược lại, nếu $\{1, 3, 5, 7\}$ đã được tạo ra và được tìm thấy là có thể giải được, thì bất kỳ tập con nào, chẳng hạn như $\{5, 7\}$, đều được đảm bảo là có thể giải được. (Nếu tôi có một giải pháp hoạt động khi tôi rất bối rối về trạng thái tôi đang ở, nó vẫn sẽ hoạt động khi tôi bớt bối rối hơn.) Mức độ cắt tĩa bổ sung này có thể cải thiện đáng kể hiệu quả của việc giải bài toán không có cảm biến.

Tuy nhiên, ngay cả với cải tiến này, việc giải bài toán không có cảm biến như chúng ta đã mô tả hiếm khi khả thi trong thực tế. Một vấn đề là sự rộng lớn của không gian trạng thái niềm tin—chúng ta đã thấy trong chương trước rằng thường thì một không gian tìm kiếm có kích thước N là quá lớn, và bây giờ chúng ta có không gian tìm kiếm có kích thước 2^N . Hơn nữa, mỗi phần tử của không gian tìm kiếm là một tập hợp có tới N phần tử. Đối với N lớn, chúng ta thậm chí sẽ không thể biểu diễn một trạng thái niềm tin duy nhất mà không hết dung lượng bộ nhớ.

Một giải pháp là biểu diễn trạng thái niềm tin bằng một mô tả nhỏ gọn hơn. Bằng tiếng Anh, chúng ta có thể nói tác nhân biết “Không có gì” ở trạng thái ban đầu; sau khi di chuyển Left, chúng ta có thể nói, “Không ở cột ngoài cùng bên phải,” và cứ thế. Chương 7 giải thích cách thực hiện điều này trong một sơ đồ biểu diễn chính thức.

Một cách tiếp cận khác là tránh các thuật toán tìm kiếm tiêu chuẩn, vốn coi các trạng thái niềm tin là các hộp đen giống như bất kỳ trạng thái bài toán nào khác. Thay vào đó, chúng ta có thể nhìn vào bên trong các trạng thái niềm tin và phát triển các thuật toán **tìm kiếm trạng thái niềm tin tăng dần** xây dựng giải pháp từng trạng thái vật lý một. Ví dụ, trong thế giới chân không không có cảm biến, trạng thái niềm tin ban đầu là $\{1, 2, 3, 4, 5, 6, 7, 8\}$, và chúng ta phải tìm một chuỗi hành động hoạt động trong cả 8 trạng thái. Chúng ta có thể làm điều này bằng cách đầu tiên tìm một giải pháp hoạt động cho trạng thái 1; sau đó chúng ta kiểm tra xem nó có hoạt động cho trạng thái 2 không; nếu không, quay lại và tìm một giải pháp khác cho trạng thái 1, v.v.

Giống như một tìm kiếm AND–OR phải tìm một giải pháp cho mọi nhánh tại một nút AND, thuật toán này phải tìm một giải pháp cho mọi trạng thái trong trạng thái niềm tin; sự khác biệt là tìm kiếm AND–OR có thể tìm một giải pháp khác nhau cho mỗi nhánh, trong khi một tìm kiếm trạng thái niềm tin tăng dần phải tìm một giải pháp hoạt động cho tất cả các trạng thái.

Ưu điểm chính của cách tiếp cận tăng dần là nó thường có thể phát hiện thất bại một cách nhanh chóng—khi một trạng thái niềm tin không thể giải quyết được, thường thì một tập con nhỏ của trạng thái niềm tin, bao gồm vài trạng thái đầu tiên được kiểm tra, cũng không thể giải quyết được. Trong một số trường hợp, điều này dẫn đến tăng tốc độ tỷ lệ thuận với kích thước của các trạng thái niềm tin, mà bản thân chúng có thể lớn bằng chính không gian trạng thái vật lý.

4.4.2 Tìm kiếm trong môi trường có thể quan sát được một phần

Nhiều bài toán không thể giải quyết được nếu không có cảm biến. Ví dụ, bài toán 8-ô không có cảm biến là không thể. Mặt khác, một chút cảm biến có thể hữu ích rất nhiều: chúng ta có thể giải bài toán 8-ô nếu chúng ta chỉ có thể nhìn thấy ô vuông ở góc trên cùng bên trái. Giải pháp liên quan đến việc di chuyển từng ô một vào ô vuông có thể quan sát được và theo dõi vị trí của nó từ đó trở đi.

Đối với một bài toán có thể quan sát được một phần, đặc tả bài toán sẽ chỉ định một hàm **PERCEPT(s)** trả về cảm nhận mà tác nhân nhận được trong một trạng thái nhất định. Nếu cảm

biến là không xác định, thì chúng ta có thể sử dụng một hàm **PERCEPTS** trả về một tập hợp các cảm nhận có thể có. Đối với các bài toán có thể quan sát được hoàn toàn, $\text{PERCEPT}(s) = s$ cho mọi trạng thái s , và đối với các bài toán không có cảm biến, $\text{PERCEPT}(s) = \text{null}$.

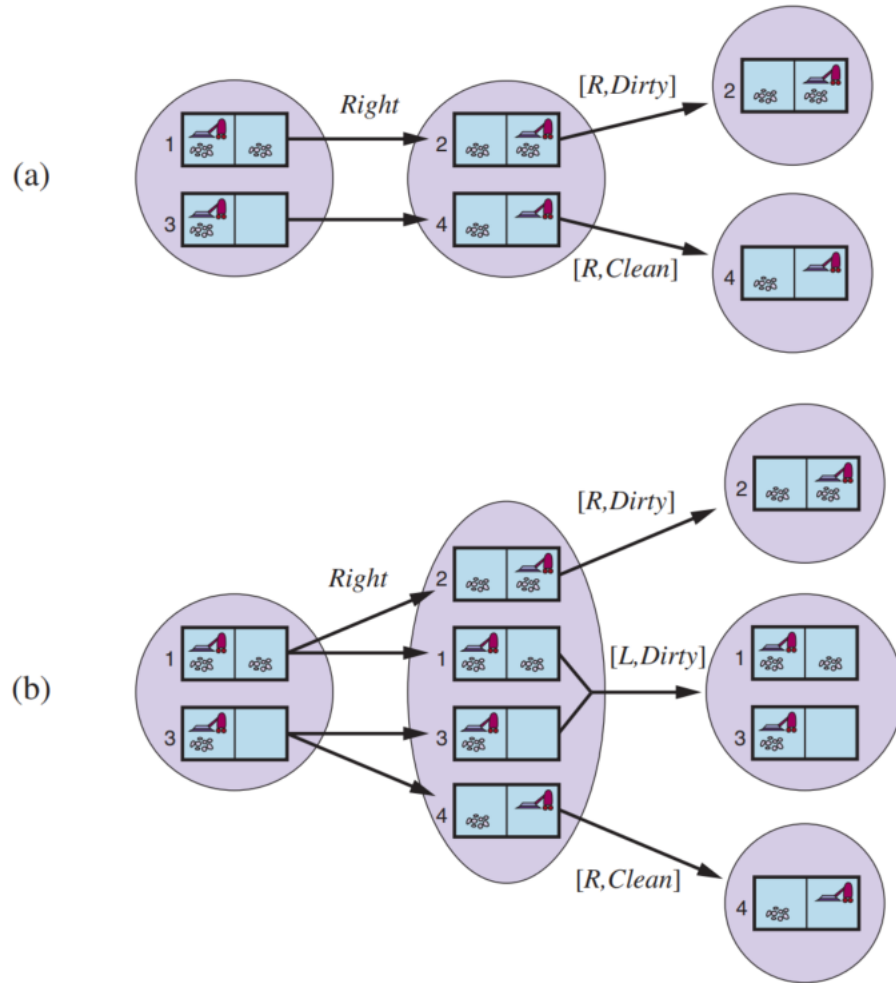
Hãy xem xét một thế giới chân không có cảm biến cục bộ, trong đó tác nhân có một cảm biến vị trí cho cảm nhận L ở ô vuông bên trái và R ở ô vuông bên phải, và một cảm biến bụi bẩn cho cảm nhận Dirty khi ô vuông hiện tại bị bẩn và Clean khi nó sạch. Do đó, **PERCEPT** ở trạng thái 1 là [L, Dirty]. Với khả năng quan sát một phần, thường thì một số trạng thái sẽ tạo ra cùng một cảm nhận; trạng thái 3 cũng sẽ tạo ra [L, Dirty]. Do đó, với cảm nhận ban đầu này, trạng thái niềm tin ban đầu sẽ là $\{1, 3\}$. Chúng ta có thể nghĩ về mô hình chuyển đổi giữa các trạng thái niềm tin cho các bài toán có thể quan sát được một phần xảy ra trong ba giai đoạn, như trong Hình 4.15:

- Giai đoạn **dự đoán** tính toán trạng thái niềm tin là kết quả của hành động, **RESULT(b, a)**, chính xác như chúng ta đã làm với các bài toán không có cảm biến. Để nhấn mạnh rằng đây là một dự đoán, chúng ta sử dụng ký hiệu $\text{hat}b = \text{RESULT}(b, a)$, trong đó “*hat*” trên b có nghĩa là “ước tính,” và chúng ta cũng sử dụng **PREDICT(b, a)** làm từ đồng nghĩa với **RESULT(b, a)**.
- Giai đoạn **các cảm nhận có thể có** tính toán tập hợp các cảm nhận có thể được quan sát trong trạng thái niềm tin đã dự đoán (sử dụng chữ cái o cho quan sát): **POSSIBLE-PERCEPTS(hatb) = $\{o : o = \text{PERCEPT}(s) \text{ và } s \in \text{hat}b\}$**
- Giai đoạn **cập nhật** tính toán, cho mỗi cảm nhận có thể có, trạng thái niềm tin sẽ là kết quả từ cảm nhận đó. Trạng thái niềm tin được cập nhật b_o là tập hợp các trạng thái trong $\text{hat}b$ có thể đã tạo ra cảm nhận đó: $b_o = \text{UPDATE}(\text{hat}b, o) = \{s : o = \text{PERCEPT}(s) \text{ và } s \in \text{hat}b\}$

Tác nhân cần phải xử lý các cảm nhận có thể có tại thời điểm lập kế hoạch, vì nó sẽ không biết các cảm nhận thực tế cho đến khi nó thực hiện kế hoạch. Lưu ý rằng tính không xác định trong môi trường vật lý có thể mở rộng trạng thái niềm tin trong giai đoạn dự đoán, nhưng mỗi trạng thái niềm tin được cập nhật b_o không thể lớn hơn trạng thái niềm tin đã dự đoán $\text{hat}b$; các quan sát chỉ có thể giúp giảm sự không chắc chắn. Hơn nữa, đối với cảm biến xác định, các trạng thái niềm tin cho các cảm nhận có thể có khác nhau sẽ rời nhau, tạo thành một phân hoạch của trạng thái niềm tin đã dự đoán ban đầu.

Kết hợp ba giai đoạn này lại với nhau, chúng ta thu được các trạng thái niềm tin có thể có là kết quả của một hành động nhất định và các cảm nhận có thể có sau đó:

$$\begin{aligned} & \text{RESULTS}(b, a) \\ &= \{b_o \mid b_o = \text{UPDATE}(\text{PREDICT}(b, a), o) \text{ và } o \in \text{POSSIBLE-PERCEPTS}(\text{PREDICT}(b, a))\} \end{aligned}$$



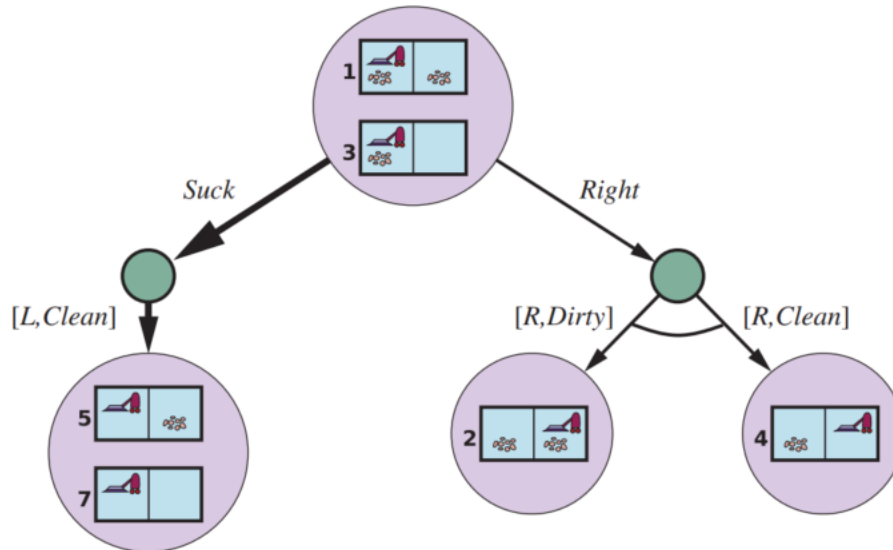
Hình 4.15 Hai ví dụ về chuyển đổi trong thế giới chân không có cảm biến cục bộ. (a) Trong thế giới xác định, Right được áp dụng trong trạng thái niềm tin ban đầu, dẫn đến một trạng thái niềm tin đã dự đoán mới với hai trạng thái vật lý có thể có; đối với các trạng thái đó, các cảm nhận có thể có là [R, Dirty] và [R, Clean], dẫn đến hai trạng thái niềm tin, mỗi trạng thái là một đơn tập. (b) Trong thế giới trơn trượt, Right được áp dụng trong trạng thái niềm tin ban đầu, cho một trạng thái niềm tin mới với bốn trạng thái vật lý; đối với các trạng thái đó, các cảm nhận có thể có là [L, Dirty], [R, Dirty] và [R, Clean], dẫn đến ba trạng thái niềm tin như được hiển thị.

4.3.2 Giải quyết các bài toán có thể quan sát được một phần

Mục trước đã chỉ ra cách suy ra hàm RESULTS cho một bài toán trạng thái niềm tin không xác định từ một bài toán vật lý cơ bản, với hàm PERCEPT. Với công thức này, thuật toán tìm kiếm AND-OR của Hình 4.11 có thể được áp dụng trực tiếp để suy ra một giải pháp. Hình 4.16 cho thấy một phần của cây tìm kiếm cho thế giới chân không có cảm biến cục bộ, giả sử cảm nhận ban đầu là [L, Dirty]. Giải pháp là kế hoạch có điều kiện

$[Suck, Right, \text{if } Rstate = \{6\} \text{ then Suck else } []]$

Lưu ý rằng, vì chúng ta đã cung cấp một bài toán trạng thái niềm tin cho thuật toán tìm kiếm AND–OR, nó đã trả về một kế hoạch có điều kiện kiểm tra trạng thái niềm tin thay vì trạng thái thực tế. Điều này là đúng như vậy: trong một môi trường có thể quan sát được một phần, tác nhân sẽ không biết trạng thái thực tế.



Hình 4.16 Cấp độ đầu tiên của cây tìm kiếm AND–OR cho một bài toán trong thế giới chân không có cảm biến cục bộ; Suck là hành động đầu tiên trong giải pháp.

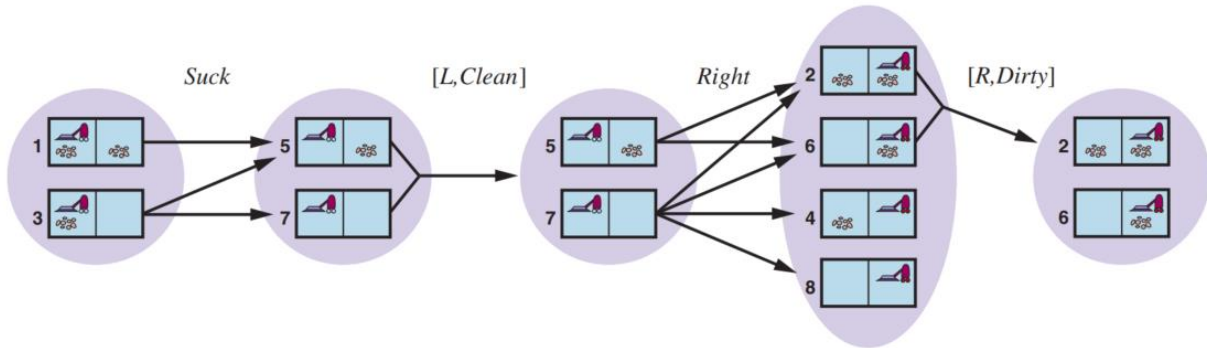
Cũng như trường hợp các thuật toán tìm kiếm tiêu chuẩn được áp dụng cho các bài toán không có cảm biến, thuật toán tìm kiếm AND–OR coi các trạng thái niềm tin là các hộp đen, giống như bất kỳ trạng thái nào khác. Người ta có thể cải thiện điều này bằng cách kiểm tra các trạng thái niềm tin đã được tạo ra trước đó là các tập con hoặc siêu tập của trạng thái hiện tại, giống như đối với các bài toán không có cảm biến. Người ta cũng có thể suy ra các thuật toán tìm kiếm tăng dần, tương tự như những thuật toán được mô tả cho các bài toán không có cảm biến, cung cấp tốc độ tăng đáng kể so với cách tiếp cận hộp đen.

4.4.4 Một tác nhân cho môi trường có thể quan sát được một phần

Một tác nhân cho môi trường có thể quan sát được một phần xây dựng một bài toán, gọi một thuật toán tìm kiếm (chẳng hạn như AND-OR-SEARCH) để giải quyết nó và thực hiện giải pháp. Có hai điểm khác biệt chính giữa tác nhân này và tác nhân cho môi trường xác định hoàn toàn có thể quan sát được. Thứ nhất, giải pháp sẽ là một kế hoạch có điều kiện hơn là một chuỗi; để thực hiện một biểu thức if–then–else, tác nhân sẽ cần kiểm tra điều kiện và thực hiện nhánh thích hợp của điều kiện. Thứ hai, tác nhân sẽ cần duy trì trạng thái niềm tin của mình khi nó thực hiện các hành động và nhận các cảm nhận. Quá trình này giống với quá trình dự đoán–quan sát–cập nhật trong Phương trình (4.5) nhưng thực tế đơn giản hơn vì cảm nhận được môi trường cung cấp thay vì được tác nhân tính toán. Với một trạng thái niềm tin ban đầu b , một hành động a , và một cảm nhận o , trạng thái niềm tin mới là:

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o)$$

Quá trình này được gọi là **giám sát, lọc và ước tính trạng thái**. Phương trình (4.6) được gọi là **bộ ước tính trạng thái đệ quy** vì nó tính toán trạng thái niềm tin mới từ trạng thái trước đó thay vì bằng cách kiểm tra toàn bộ chuỗi cảm nhận. Nếu tác nhân không muốn bị “tụt lại phía sau”, việc tính toán phải diễn ra nhanh chóng khi các cảm nhận đến. Khi môi trường trở nên phức tạp hơn, tác nhân sẽ chỉ có thời gian để tính toán một trạng thái niềm tin gần đúng, có lẽ tập trung vào ý nghĩa của cảm nhận đối với các khía cạnh của môi trường đang được quan tâm hiện tại. Hầu hết công việc về bài toán này đã được thực hiện cho các môi trường ngẫu nhiên, trạng thái liên tục với các công cụ của lý thuyết xác suất, như đã giải thích trong Chương 14.



Hình 4.17 Hai chu kỳ dự đoán–cập nhật của việc duy trì trạng thái niềm tin trong thế giới chân không nhà trẻ có cảm biến cục bộ.

Trong mục này, chúng ta sẽ đưa ra một ví dụ trong một môi trường rời rạc với các cảm biến xác định và các hành động không xác định. Ví dụ liên quan đến một robot với một nhiệm vụ ước tính trạng thái cụ thể được gọi là **định vị**: xác định vị trí của nó, với một bản đồ của thế giới và một chuỗi các cảm nhận và hành động. Robot của chúng ta được đặt trong môi trường giống như mê cung của Hình 4.18. Robot được trang bị bốn cảm biến sonar cho biết liệu có chướng ngại vật hay không—bức tường bên ngoài hoặc một ô vuông màu đậm trong hình—ở mỗi trong bốn hướng la bàn. Cảm nhận có dạng một vectơ bit, một bit cho mỗi hướng bắc, đông, nam và tây theo thứ tự đó, vì vậy 1011 có nghĩa là có chướng ngại vật ở phía bắc, nam và tây, nhưng không phải phía đông.

Chúng ta giả định rằng các cảm biến cung cấp dữ liệu hoàn toàn chính xác và robot có một bản đồ chính xác của môi trường. Nhưng thật không may, hệ thống định vị của robot bị hỏng, vì vậy khi nó thực hiện một hành động Right, nó di chuyển ngẫu nhiên đến một trong các ô vuông liền kề. Nhiệm vụ của robot là xác định vị trí hiện tại của nó.

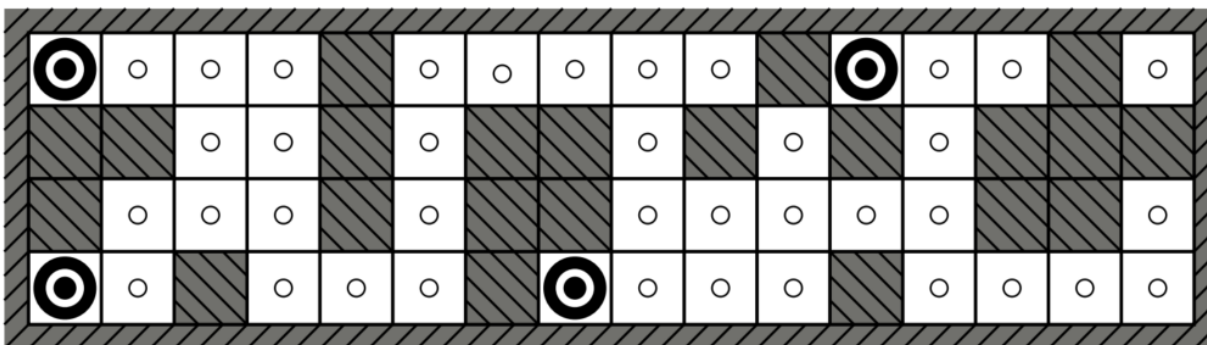
Giả sử robot vừa được bật, và nó không biết nó đang ở đâu—trạng thái niềm tin ban đầu b của nó bao gồm tập hợp tất cả các vị trí. Robot sau đó nhận được cảm nhận 1011 và thực hiện cập nhật bằng cách sử dụng phương trình $b_o = \text{UPDATE}(1011)$, cho ra 4 vị trí được hiển thị trong Hình 4.18(a). Bạn có thể kiểm tra mê cung để thấy rằng đó là bốn vị trí duy nhất tạo ra cảm nhận 1011.

Tiếp theo, robot thực hiện một hành động Right, nhưng kết quả là không xác định. Trạng thái niềm tin mới, $b_a = \text{PREDICT}(b_o, \text{Right})$, chứa tất cả các vị trí cách các vị trí trong b_o một bước. Khi cảm nhận thứ hai, 1010, đến, robot thực hiện $\text{UPDATE}(b_a, 1010)$ và thấy rằng trạng thái niềm

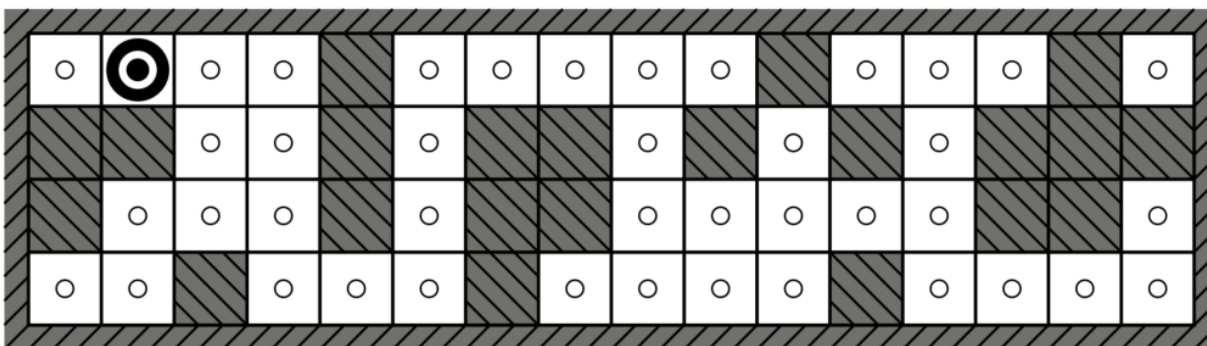
tin đã thu gọn lại thành một vị trí duy nhất được hiển thị trong Hình 4.18(b). Đó là vị trí duy nhất có thể là kết quả của

$Update(Predict(Update(b, 1011), Right), 1010)$

Với các hành động không xác định, bước PREDICT làm cho trạng thái niềm tin phát triển, nhưng bước UPDATE lại thu nhỏ nó lại—miễn là các cảm nhận cung cấp một số thông tin nhận dạng hữu ích. Đôi khi các cảm nhận không giúp ích nhiều cho việc định vị: Nếu có một hoặc nhiều hành lang dài đông–tây, thì một robot có thể nhận được một chuỗi dài các cảm nhận 1010, nhưng không bao giờ biết nó đang ở đâu trong hành lang. Nhưng đối với các môi trường có sự thay đổi hợp lý về địa lý, việc định vị thường hội tụ nhanh chóng đến một điểm duy nhất, ngay cả khi các hành động là không xác định. Điều gì sẽ xảy ra nếu các cảm biến bị lỗi? Nếu chúng ta chỉ có thể lý luận bằng logic Boolean, thì chúng ta phải coi mỗi bit cảm biến là đúng hoặc sai, điều này cũng giống như không có thông tin cảm nhận nào cả. Nhưng chúng ta sẽ thấy rằng lý luận xác suất (Chương 12), cho phép chúng ta trích xuất thông tin hữu ích từ một cảm biến bị lỗi miễn là nó sai ít hơn một nửa thời gian.



(a) Possible locations of robot after $E_1 = 1011$



(b) Possible locations of robot after $E_1 = 1011, E_2 = 1010$

Hình 4.18 Các vị trí có thể có của robot, $\odot$, (a) sau một quan sát, $E_1 = 1011$, và (b) sau khi di chuyển một ô vuông và thực hiện một quan sát thứ hai, $E_2 = 1010$. Khi các cảm biến không có nhiễu và mô hình chuyển đổi chính xác, chỉ có một vị trí có thể có cho robot nhất quán với chuỗi hai quan sát này.

4.5 Tác nhân tìm kiếm trực tuyến và môi trường không xác định

Cho đến nay, chúng ta đã tập trung vào các tác nhân sử dụng thuật toán tìm kiếm ngoại tuyến. Chúng tính toán một giải pháp hoàn chỉnh trước khi thực hiện hành động đầu tiên của mình. Ngược lại, một tác nhân tìm kiếm **trực tuyến** xen kẽ giữa tính toán và hành động: đầu tiên nó thực hiện một hành động, sau đó nó quan sát môi trường và tính toán hành động tiếp theo. Tìm kiếm trực tuyến là một ý tưởng hay trong các môi trường động hoặc bán động, nơi có một hình phạt cho việc ngồi yên và tính toán quá lâu. Tìm kiếm trực tuyến cũng hữu ích trong các miền không xác định vì nó cho phép tác nhân tập trung nỗ lực tính toán của mình vào các trường hợp dự phòng thực sự phát sinh thay vì những trường hợp có thể xảy ra nhưng có thể sẽ không xảy ra.

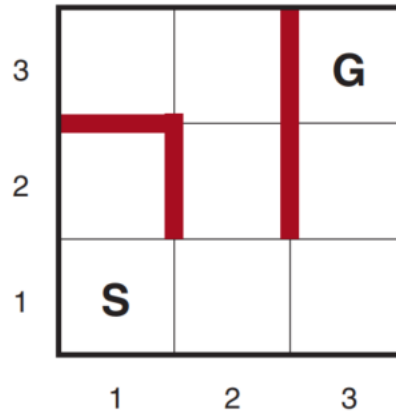
Tất nhiên, có một sự đánh đổi: tác nhân càng lập kế hoạch trước nhiều, nó càng ít khi thấy mình rơi vào tình thế tiến thoái lưỡng nan. Trong các môi trường không xác định, nơi tác nhân không biết những trạng thái nào tồn tại hoặc hành động của nó làm gì, tác nhân phải sử dụng các hành động của mình như những thử nghiệm để tìm hiểu về môi trường. Một ví dụ điển hình về tìm kiếm trực tuyến là **bài toán lập bản đồ**: một robot được đặt trong một tòa nhà không xác định và phải khám phá để xây dựng một bản đồ mà sau này có thể được sử dụng để đi từ A đến B. Các phương pháp thoát khỏi mê cung—kiến thức bắt buộc đối với các anh hùng đầy tham vọng của thời cổ đại—cũng là ví dụ về các thuật toán tìm kiếm trực tuyến. Tuy nhiên, khám phá không gian không phải là hình thức khám phá trực tuyến duy nhất. Hãy xem xét một em bé sơ sinh: nó có nhiều hành động có thể nhưng không biết kết quả của bất kỳ hành động nào trong số đó, và nó chỉ trải qua một vài trong số các trạng thái có thể mà nó có thể đạt được.

4.5.1 Các bài toán tìm kiếm trực tuyến

Một bài toán tìm kiếm trực tuyến được giải quyết bằng cách xen kẽ tính toán, cảm nhận và hành động. Chúng ta sẽ bắt đầu bằng cách giả định một môi trường xác định và hoàn toàn có thể quan sát được (Chương 16 sẽ nói lòng các giả định này) và quy định rằng tác nhân chỉ biết những điều sau:

- $ACTIONS(s)$, các hành động hợp pháp trong trạng thái s ;
- $c(s, a, s')$, chi phí của việc áp dụng hành động a trong trạng thái s để đến trạng thái s' . Lưu ý rằng điều này không thể được sử dụng cho đến khi tác nhân biết rằng s' là kết quả.
- $IS-GOAL(s)$, kiểm tra mục tiêu.

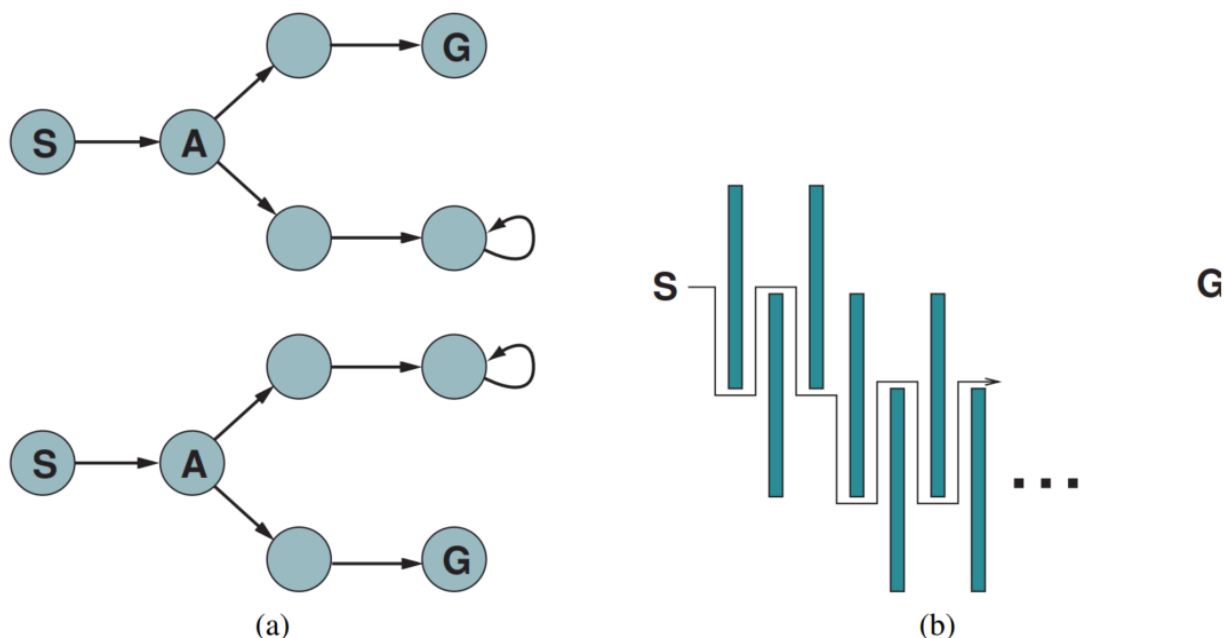
Đặc biệt lưu ý rằng tác nhân không thể xác định $RESULT(s, a)$ ngoại trừ bằng cách thực sự ở trong s và thực hiện a . Ví dụ, trong bài toán mê cung được hiển thị trong Hình 4.19, tác nhân không biết rằng đi Up từ (1, 1) dẫn đến (1, 2); cũng không, sau khi đã làm điều đó, nó biết rằng đi Down sẽ đưa nó trở lại (1, 1). Mức độ thiếu hiểu biết này có thể được giảm bớt trong một số ứng dụng—ví dụ, một robot thám hiểm có thể biết các hành động di chuyển của nó hoạt động như thế nào và chỉ không biết về vị trí của các chướng ngại vật.



Hình 4.19 Một bài toán mê cung đơn giản. Tác nhân bắt đầu tại S và phải đến G nhưng không biết gì về môi trường.

Cuối cùng, tác nhân có thể có quyền truy cập vào một hàm heuristic chấp nhận được $h(s)$ ước tính khoảng cách từ trạng thái hiện tại đến một trạng thái mục tiêu. Ví dụ, trong Hình 4.19, tác nhân có thể biết vị trí của mục tiêu và có thể sử dụng heuristic khoảng cách Manhattan (trang 116). Thông thường, mục tiêu của tác nhân là đạt đến một trạng thái mục tiêu trong khi giảm thiểu chi phí. (Một mục tiêu khả thi khác chỉ đơn giản là khám phá toàn bộ môi trường.) Chi phí là tổng chi phí đường đi mà tác nhân phải chịu khi nó di chuyển. Phổ biến là so sánh chi phí này với chi phí đường đi mà tác nhân sẽ phải chịu nếu nó biết trước không gian tìm kiếm—tức là đường đi tối ưu trong môi trường đã biết. Trong ngôn ngữ của các thuật toán trực tuyến, so sánh này được gọi là **tỷ lệ cạnh tranh**; chúng ta muốn nó càng nhỏ càng tốt.

Các nhà thám hiểm trực tuyến dễ bị tổn thương bởi các **ngõ cụt**: các trạng thái mà từ đó không thể đạt được trạng thái mục tiêu nào. Nếu tác nhân không biết mỗi hành động làm gì, nó có thể thực hiện hành động “nhảy xuống hố sâu không đáy”, và do đó không bao giờ đạt được mục tiêu. Nói chung, không có thuật toán nào có thể tránh các ngõ cụt trong tất cả các không gian trạng thái. Hãy xem xét hai không gian trạng thái ngõ cụt trong Hình 4.20(a). Một thuật toán tìm kiếm trực tuyến đã truy cập các trạng thái S và A không thể biết liệu nó đang ở trong không gian trạng thái trên cùng hay dưới cùng; hai cái trông giống hệt nhau dựa trên những gì tác nhân đã thấy. Do đó, không có cách nào nó có thể biết cách chọn hành động chính xác trong cả hai không gian trạng thái. Đây là một ví dụ về một **lập luận phản biện**—chúng ta có thể tưởng tượng một đối thủ xây dựng không gian trạng thái trong khi tác nhân khám phá nó và đặt các mục tiêu và ngõ cụt ở bất cứ đâu nó chọn, như trong Hình 4.20(b).



Hình 4.20 (a) Hai không gian trạng thái có thể dẫn một tác nhân tìm kiếm trực tuyến vào ngõ cụt. Bất kỳ tác nhân nào cũng sẽ thất bại trong ít nhất một trong các không gian này. (b) Một môi trường hai chiều có thể khiến một tác nhân tìm kiếm trực tuyến đi theo một lộ trình không hiệu quả tùy ý đến mục tiêu. Bất kể tác nhân đưa ra lựa chọn nào, đối thủ sẽ chặn lộ trình đó bằng một bức tường dài, mỏng khác, để đường đi được đi theo dài hơn nhiều so với đường đi tốt nhất có thể.

Ngõ cụt là một khó khăn thực sự đối với việc khám phá của robot—cầu thang, đoạn dốc, vách đá, đường một chiều và thậm chí cả địa hình tự nhiên đều có các trạng thái mà từ đó một số hành động là **không thể đảo ngược**—không có cách nào để quay lại trạng thái trước đó. Thuật toán khám phá mà chúng ta sẽ trình bày chỉ được đảm bảo hoạt động trong các không gian trạng thái có thể khám phá một cách an toàn—tức là một số trạng thái mục tiêu có thể đạt được từ mọi trạng thái có thể đạt được. Các không gian trạng thái chỉ có các hành động có thể đảo ngược, chẳng hạn như mê cung và 8-ô, rõ ràng là có thể khám phá một cách an toàn (nếu chúng có bất kỳ giải pháp nào cả). Chúng ta sẽ đề cập đến chủ đề khám phá an toàn chi tiết hơn trong Mục 23.3.2.

Ngay cả trong các môi trường có thể khám phá một cách an toàn, không có tỷ lệ cạnh tranh bị chặn nào có thể được đảm bảo nếu có các đường đi có chi phí không bị chặn. Điều này dễ dàng được chứng minh trong các môi trường có các hành động không thể đảo ngược, nhưng trên thực tế nó vẫn đúng đối với trường hợp có thể đảo ngược, như Hình 4.20(b) cho thấy. Vì lý do này, người ta thường đặc trưng hiệu suất của các thuật toán tìm kiếm trực tuyến bằng kích thước của toàn bộ không gian trạng thái hơn là chỉ độ sâu của mục tiêu nông nhất.

4.5.2 Các tác nhân tìm kiếm trực tuyến

Sau mỗi hành động, một tác nhân trực tuyến trong một môi trường có thể quan sát được sẽ nhận được một cảm nhận cho nó biết nó đã đạt đến trạng thái nào; từ thông tin này, nó có thể bổ sung bản đồ của môi trường. Bản đồ được cập nhật sau đó được sử dụng để lập kế hoạch nơi sẽ đi tiếp. Sự xen kẽ giữa lập kế hoạch và hành động này có nghĩa là các thuật toán tìm kiếm trực tuyến khá

khác so với các thuật toán tìm kiếm ngoại tuyến mà chúng ta đã thấy trước đây: các thuật toán ngoại tuyến khám phá mô hình của không gian trạng thái, trong khi các thuật toán trực tuyến khám phá thế giới thực. Ví dụ, A^* có thể mở rộng một nút ở một phần của không gian và sau đó ngay lập tức mở rộng một nút ở một phần xa xôi của không gian, bởi vì việc mở rộng nút liên quan đến các hành động mô phỏng chứ không phải hành động thực.

Mặt khác, một thuật toán trực tuyến chỉ có thể khám phá những người kế thừa cho một trạng thái mà nó đang chiếm giữ về mặt vật lý. Để tránh phải đi khắp nơi đến một trạng thái xa xôi để mở rộng nút tiếp theo, có vẻ tốt hơn là mở rộng các nút theo thứ tự cục bộ. Tìm kiếm theo chiều sâu chính xác có thuộc tính này bởi vì (ngoại trừ khi thuật toán đang quay lui) nút tiếp theo được mở rộng là một con của nút trước đó đã được mở rộng.

Một tác nhân khám phá theo chiều sâu trực tuyến (cho các hành động xác định nhưng không xác định) được hiển thị trong Hình 4.21. Tác nhân này lưu trữ bản đồ của mình trong một bảng, $result[s, a]$, ghi lại trạng thái là kết quả của việc thực hiện hành động a trong trạng thái s . (Đối với các hành động không xác định, tác nhân có thể ghi lại một tập hợp các trạng thái dưới $results[s, a]$). Bất cứ khi nào trạng thái hiện tại có các hành động chưa được khám phá, tác nhân sẽ thử một trong các hành động đó. Khó khăn nảy sinh khi tác nhân đã thử tất cả các hành động trong một trạng thái. Trong tìm kiếm theo chiều sâu ngoại tuyến, trạng thái chỉ đơn giản là bị loại khỏi hàng đợi; trong một tìm kiếm trực tuyến, tác nhân phải **quay lui** trong thế giới vật lý. Trong tìm kiếm theo chiều sâu, điều này có nghĩa là quay lại trạng thái mà từ đó tác nhân gần đây nhất đã vào trạng thái hiện tại. Để đạt được điều đó, thuật toán giữ một bảng khác liệt kê, cho mỗi trạng thái, các trạng thái tiền nhiệm mà tác nhân chưa quay lại. Nếu tác nhân đã hết các trạng thái để quay lại, thì tìm kiếm của nó đã hoàn tất.

```

function ONLINE-DFS-AGENT(problem,  $s'$ ) returns an action
     $s$ ,  $a$ , the previous state and action, initially null
    result, a table mapping ( $s$ ,  $a$ ) to  $s'$ , initially empty
    untried, a table mapping  $s$  to a list of untried actions
    unbacktracked, a table mapping  $s$  to a list of states never backtracked to

    if problem.IS-GOAL( $s'$ ) then return stop
    if  $s'$  is a new state (not in untried) then untried[ $s'$ ]  $\leftarrow$  problem.ACTIONS( $s'$ )
    if  $s$  is not null then
        result[ $s$ ,  $a$ ]  $\leftarrow$   $s'$ 
        add  $s$  to the front of unbacktracked[ $s'$ ]
    if untried[ $s'$ ] is empty then
        if unbacktracked[ $s'$ ] is empty then return stop
         $a \leftarrow$  an action  $b$  such that result[ $s'$ ,  $b$ ] = POP(unbacktracked[ $s'$ ]))  $s' \leftarrow$  null
    else  $a \leftarrow$  POP(untried[ $s'$ ]))
     $s \leftarrow s'$ 
    return  $a$ 

```

Hình 4.21 Một tác nhân tìm kiếm trực tuyến sử dụng khám phá theo chiều sâu. Tác nhân chỉ có thể khám phá một cách an toàn trong các không gian trạng thái trong đó mọi hành động đều có thể được “hoàn tác” bằng một hành động khác.

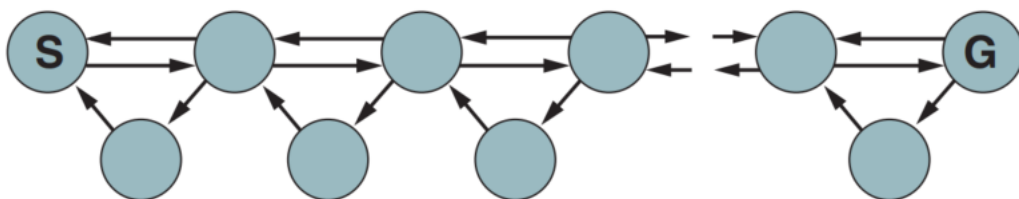
Chúng tôi khuyên người đọc nên theo dõi tiến trình của ONLINE-DFS-AGENT khi áp dụng cho mê cung trong Hình 4.19. Khá dễ dàng để thấy rằng tác nhân sẽ, trong trường hợp xấu nhất, cuối cùng đi qua mọi liên kết trong không gian trạng thái chính xác hai lần. Đối với việc khám phá, điều này là tối ưu; mặt khác, đối với việc tìm kiếm mục tiêu, tỷ lệ cạnh tranh của tác nhân có thể tùy ý tồi tệ nếu nó đi vào một chuyến đi chơi dài khi có một mục tiêu ngay bên cạnh trạng thái ban đầu. Một biến thể trực tuyến của việc lặp lại làm sâu sắc giải quyết vấn đề này; đối với một môi trường là một cây đồng nhất, tỷ lệ cạnh tranh của một tác nhân như vậy là một hằng số nhỏ.

Vì phương pháp quay lui của nó, ONLINE-DFS-AGENT chỉ hoạt động trong các không gian trạng thái mà các hành động có thể đảo ngược. Có các thuật toán phức tạp hơn một chút hoạt động trong các không gian trạng thái chung, nhưng không có thuật toán nào như vậy có tỷ lệ cạnh tranh bị chặn.

4.5.3 Tìm kiếm cục bộ trực tuyến

Giống như tìm kiếm theo chiều sâu, tìm kiếm leo đồi có thuộc tính cục bộ trong việc mở rộng nút của nó. Trên thực tế, vì nó chỉ giữ một trạng thái hiện tại trong bộ nhớ, tìm kiếm leo đồi đã là một thuật toán tìm kiếm trực tuyến! Thật không may, thuật toán cơ bản không tốt lắm cho việc khám phá vì nó để tác nhân ngồi ở các cực đại cục bộ mà không có nơi nào để đi. Hơn nữa, việc khởi động lại ngẫu nhiên không thể được sử dụng, bởi vì tác nhân không thể dịch chuyển tức thời đến một trạng thái bắt đầu mới.

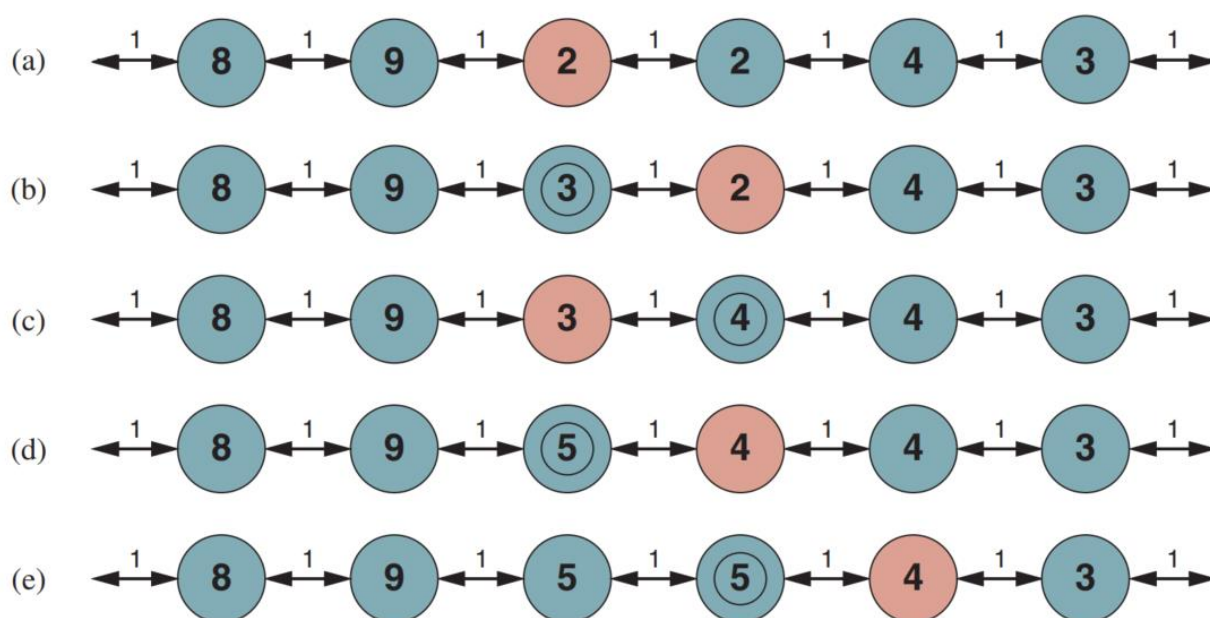
Thay vì khởi động lại ngẫu nhiên, người ta có thể xem xét sử dụng một **bước đi ngẫu nhiên** để khám phá môi trường. Một bước đi ngẫu nhiên chỉ đơn giản là chọn ngẫu nhiên một trong các hành động có sẵn từ trạng thái hiện tại; ưu tiên có thể được dành cho các hành động chưa được thử. Dễ dàng chứng minh rằng một bước đi ngẫu nhiên cuối cùng sẽ tìm thấy một mục tiêu hoặc hoàn thành việc khám phá của nó, miễn là không gian là hữu hạn và có thể khám phá một cách an toàn. Mặt khác, quá trình này có thể rất chậm. Hình 4.22 cho thấy một môi trường trong đó một bước đi ngẫu nhiên sẽ mất số bước theo cấp số mũ để tìm thấy mục tiêu, bởi vì, đối với mỗi trạng thái trong hàng trên cùng ngoại trừ S, tiến trình lùi lại có khả năng gấp đôi so với tiến trình về phía trước. Tất nhiên, ví dụ này là được tạo ra, nhưng có nhiều không gian trạng thái trong thế giới thực có cấu trúc liên kết gây ra những loại “bẫy” này cho các bước đi ngẫu nhiên.



Hình 4.22 Một môi trường trong đó một bước đi ngẫu nhiên sẽ mất số bước theo cấp số mũ để tìm thấy mục tiêu.

Việc bổ sung bộ nhớ thay vì tính ngẫu nhiên vào leo đồi hóa ra là một cách tiếp cận hiệu quả hơn. Ý tưởng cơ bản là lưu trữ một “ước tính tốt nhất hiện tại” $H(s)$ về chi phí để đạt đến mục tiêu từ mỗi trạng thái đã được truy cập. $H(s)$ bắt đầu chỉ là ước tính heuristic $h(s)$ và được cập nhật khi tác nhân có được kinh nghiệm trong không gian trạng thái.

Hình 4.23 cho thấy một ví dụ đơn giản trong một không gian trạng thái một chiều. Trong (a), tác nhân dường như bị mắc kẹt trong một cực tiểu cục bộ phẳng ở trạng thái màu đỏ. Thay vì ở yên đó, tác nhân nên đi theo con đường dường như là tốt nhất đến mục tiêu dựa trên các ước tính chi phí hiện tại cho các hàng xóm của nó. Chi phí ước tính để đạt đến mục tiêu thông qua một hàng xóm s' là chi phí để đến s' cộng với chi phí ước tính để đến mục tiêu từ đó—tức là $c(s, s') + H(s')$. Trong ví dụ, có hai hành động, với chi phí ước tính là $1 + 9$ sang trái và $1 + 2$ sang phải, vì vậy có vẻ tốt nhất là di chuyển sang phải.



Hình 4.23 Năm lần lặp của LRTA* trên một không gian trạng thái một chiều. Mỗi trạng thái được gán nhãn $H(s)$, ước tính chi phí hiện tại để đạt đến mục tiêu, và mỗi liên kết có chi phí hành động là 1. Trạng thái màu đỏ đánh dấu vị trí của tác nhân, và các ước tính chi phí được cập nhật ở mỗi lần lặp có một vòng tròn đôi.

Trong (b) rõ ràng là ước tính chi phí là 2 cho trạng thái màu đỏ trong (a) là quá lạc quan. Vì bước di chuyển tốt nhất có chi phí 1 và dẫn đến một trạng thái cách mục tiêu ít nhất 2 bước, trạng thái màu đỏ phải cách mục tiêu ít nhất 3 bước, vì vậy H của nó nên được cập nhật tương ứng, như được hiển thị trong Hình 4.23(b). Tiếp tục quá trình này, tác nhân sẽ di chuyển qua lại hai lần nữa, cập nhật H mỗi lần và “san phẳng” cực tiểu cục bộ cho đến khi nó thoát ra khỏi đó sang bên phải.

Một tác nhân thực hiện sơ đồ này, được gọi là **học A* thời gian thực (LRTA*)**, được hiển thị trong Hình 4.24. Giống như ONLINE-DFS-AGENT, nó xây dựng một bản đồ của môi trường trong bảng *result*. Nó cập nhật ước tính chi phí cho trạng thái mà nó vừa rời đi và sau đó chọn bước di chuyển “có vẻ tốt nhất” theo ước tính chi phí hiện tại của nó. Một chi tiết quan trọng là các hành động chưa được thử trong một trạng thái s luôn được giả định là dẫn ngay lập tức đến

mục tiêu với chi phí thấp nhất có thể, cụ thể là $h(s)$. Sự **lạc quan trong điều không chắc chắn** này khuyến khích tác nhân khám phá các con đường mới, có thể hứa hẹn.

```

function LRTA*-AGENT(problem,  $s'$ ,  $h$ ) returns an action
     $s$ ,  $a$ , the previous state and action, initially null
    result, a table mapping ( $s$ ,  $a$ ) to  $s'$ , initially empty
     $H$ , a table mapping  $s$  to a cost estimate, initially empty

    if IS-GOAL( $s'$ ) then return stop
    if  $s'$  is a new state (not in  $H$ ) then  $H[s'] \leftarrow h(s')$ 
    if  $s$  is not null then
         $result[s, a] \leftarrow s'$ 
         $H[s] \leftarrow \min_{b \in ACTIONS(s)} LRTA^*-COST(problem, s, b, result[s, b], H)$ 
         $a \leftarrow \operatorname{argmin}_{b \in ACTIONS(s)} LRTA^*-COST(problem, s', b, result[s', b], H)$ 
         $s \leftarrow s'$ 
    return  $a$ 

function LRTA*-COST(problem,  $s$ ,  $a$ ,  $s'$ ,  $H$ ) returns a cost estimate
    if  $s'$  is undefined then return  $h(s)$ 
    else return problem.ACTION-COST( $s, a, s'$ ) +  $H[s']$ 

```

Hình 4.24 LRTA*-AGENT chọn một hành động theo các giá trị của các trạng thái lân cận, được cập nhật khi tác nhân di chuyển trong không gian trạng thái.

Một tác nhân LRTA* được đảm bảo sẽ tìm thấy một mục tiêu trong bất kỳ môi trường hữu hạn, có thể khám phá một cách an toàn nào. Tuy nhiên, không giống như A*, nó không hoàn chỉnh cho các không gian trạng thái vô hạn—có những trường hợp nó có thể bị lạc vô hạn. Nó có thể khám phá một môi trường n trạng thái trong $O(n^2)$ bước trong trường hợp xấu nhất, nhưng thường hoạt động tốt hơn nhiều. Tác nhân LRTA* chỉ là một trong một họ lớn các tác nhân trực tuyến mà người ta có thể định nghĩa bằng cách chỉ định quy tắc lựa chọn hành động và quy tắc cập nhật theo những cách khác nhau. Chúng ta sẽ thảo luận về họ này, ban đầu được phát triển cho các môi trường ngẫu nhiên, trong Chương 23.

4.5.4 Học trong tìm kiếm trực tuyến

Sự thiếu hiểu biết ban đầu của các tác nhân tìm kiếm trực tuyến cung cấp một số cơ hội để học hỏi. Thứ nhất, các tác nhân học một “bản đồ” của môi trường—chính xác hơn, kết quả của mỗi hành động trong mỗi trạng thái—chỉ đơn giản bằng cách ghi lại từng kinh nghiệm của chúng. Thứ hai, các tác nhân tìm kiếm cục bộ có được các ước tính chính xác hơn về chi phí của mỗi trạng thái bằng cách sử dụng các quy tắc cập nhật cục bộ, như trong LRTA*. Trong Chương 23, chúng ta sẽ chỉ ra rằng các bản cập nhật này cuối cùng hội tụ đến các giá trị chính xác cho mọi trạng thái, với

điều kiện tác nhân khám phá không gian trạng thái theo đúng cách. Khi các giá trị chính xác đã được biết, các quyết định tối ưu có thể được đưa ra chỉ đơn giản bằng cách di chuyển đến người kế thừa có chi phí thấp nhất—tức là, leo đồi thuần túy sau đó là một chiến lược tối ưu.

Nếu bạn làm theo gợi ý của chúng tôi để theo dõi hành vi của ONLINE-DFS-AGENT trong môi trường của Hình 4.19, bạn sẽ nhận thấy rằng tác nhân không thông minh lắm. Ví dụ, sau khi nó đã thấy rằng hành động Up đi từ (1, 1) đến (1, 2), tác nhân vẫn không có ý tưởng rằng hành động Down quay trở lại (1, 1) hoặc hành động Up cũng đi từ (2, 1) đến (2, 2), từ (2, 2) đến (2, 3), v.v. Nói chung, chúng ta muốn tác nhân học rằng Up tăng tọa độ y trừ khi có một bức tường cản đường, rằng Down giảm nó, v.v.

Để điều này xảy ra, chúng ta cần hai điều. Thứ nhất, chúng ta cần một biểu diễn chính thức và có thể thao tác rõ ràng cho các loại quy tắc chung này; cho đến nay, chúng ta đã ẩn thông tin bên trong hộp đen được gọi là hàm RESULT. Các Chương 8 đến 11 được dành cho vấn đề này. Thứ hai, chúng ta cần các thuật toán có thể xây dựng các quy tắc chung phù hợp từ các quan sát cụ thể do tác nhân thực hiện. Những điều này được đề cập trong Chương 19.

Nếu chúng ta dự đoán rằng chúng ta sẽ được yêu cầu giải quyết nhiều bài toán tương tự trong tương lai, thì việc đầu tư thời gian (và bộ nhớ) để làm cho các tìm kiếm trong tương lai đó dễ dàng hơn là điều hợp lý. Có một số cách để làm điều này, tất cả đều thuộc tiêu đề **tìm kiếm tăng dần**. Chúng ta có thể giữ cây tìm kiếm trong bộ nhớ và sử dụng lại các phần của nó không thay đổi trong bài toán mới. Chúng ta có thể giữ các giá trị heuristic h và cập nhật chúng khi chúng ta có được thông tin mới—hoặc vì thế giới đã thay đổi hoặc vì chúng ta đã tính toán một ước tính tốt hơn. Hoặc chúng ta có thể giữ các giá trị g đường đi tốt nhất, sử dụng chúng để ghép lại một giải pháp mới và cập nhật chúng khi thế giới thay đổi.

Tóm tắt

Chương này đã xem xét các thuật toán tìm kiếm cho các bài toán trong môi trường có thể quan sát được một phần, không xác định, không xác định và liên tục.

- Các phương pháp **tìm kiếm cục bộ** như leo đồi chỉ giữ một số lượng nhỏ các trạng thái trong bộ nhớ. Chúng đã được áp dụng cho các bài toán tối ưu hóa, trong đó ý tưởng là tìm một trạng thái có điểm số cao, mà không phải lo lắng về đường đi đến trạng thái đó. Một số thuật toán tìm kiếm cục bộ ngẫu nhiên đã được phát triển, bao gồm cả **tôi luyện mô phỏng**, trả về các giải pháp tối ưu khi được cung cấp một lịch trình làm mát thích hợp.
- Nhiều phương pháp tìm kiếm cục bộ cũng áp dụng cho các bài toán trong không gian liên tục. Các bài toán **quy hoạch tuyến tính** và **tối ưu hóa lồi** tuân theo một số hạn chế nhất định về hình dạng của không gian trạng thái và bản chất của hàm mục tiêu, và cho phép các thuật toán thời gian đa thức thường cực kỳ hiệu quả trong thực tế. Đối với một số bài toán được hình thành tốt về mặt toán học, chúng ta có thể tìm thấy cực đại bằng cách sử dụng phép vi tích phân để tìm nơi gradient bằng 0; đối với các bài toán khác, chúng ta phải sử dụng gradient thực nghiệm, đo lường sự khác biệt về độ thích ứng giữa hai điểm gần nhau.

- Một **thuật toán tiến hóa** là một tìm kiếm leo đồi ngẫu nhiên trong đó một quần thể các trạng thái được duy trì. Các trạng thái mới được tạo ra bằng cách đột biến và bằng cách lai ghép, kết hợp các cặp trạng thái.
- Trong các môi trường **không xác định**, các tác nhân có thể áp dụng tìm kiếm AND–OR để tạo ra các **kế hoạch có điều kiện** đạt đến mục tiêu bất kể kết quả nào xảy ra trong quá trình thực hiện.
- Khi môi trường có thể quan sát được một phần, **trạng thái niềm tin** biểu thị tập hợp các trạng thái có thể có mà tác nhân có thể ở trong.
- Các thuật toán tìm kiếm tiêu chuẩn có thể được áp dụng trực tiếp vào **không gian trạng thái niềm tin** để giải quyết các bài toán không có cảm biến, và tìm kiếm AND–OR trạng thái niềm tin có thể giải quyết các bài toán có thể quan sát được một phần nói chung. Các thuật toán tăng dần xây dựng các giải pháp từng trạng thái một trong một trạng thái niềm tin thường hiệu quả hơn.
- Các bài toán **khám phá** phát sinh khi tác nhân không có ý tưởng về các trạng thái và hành động của môi trường của nó. Đối với các môi trường có thể khám phá một cách an toàn, các tác nhân tìm kiếm trực tuyến có thể xây dựng một bản đồ và tìm một mục tiêu nếu có. Việc cập nhật các ước tính heuristic từ kinh nghiệm cung cấp một phương pháp hiệu quả để thoát khỏi các cực tiểu cục bộ.

Ghi chú thư mục và lịch sử

Các kỹ thuật tìm kiếm cục bộ có một lịch sử lâu đời trong toán học và khoa học máy tính. Thật vậy, phương pháp Newton–Raphson (Newton, 1671; Raphson, 1690) có thể được coi là một phương pháp tìm kiếm cục bộ rất hiệu quả cho các không gian liên tục trong đó thông tin gradient có sẵn. Brent (1973) là một tài liệu tham khảo kinh điển cho các thuật toán tối ưu hóa không yêu cầu thông tin như vậy. **Tìm kiếm chùm**, mà chúng ta đã trình bày như một thuật toán tìm kiếm cục bộ, có nguồn gốc là một biến thể có chiều rộng bị chặn của quy hoạch động để nhận dạng giọng nói trong hệ thống HARP (Lowerre, 1976). Một thuật toán liên quan được Pearl phân tích sâu (1984, Ch. 5).

Chủ đề tìm kiếm cục bộ đã được tiếp thêm sinh lực vào đầu những năm 1990 bởi những kết quả tốt đáng ngạc nhiên cho các bài toán thỏa mãn ràng buộc lớn như n-hậu (Minton et al., 1992) và tính thỏa mãn Boolean (Selman et al., 1992) và bằng cách kết hợp tính ngẫu nhiên, nhiều tìm kiếm đồng thời và các cải tiến khác. Sự phục hưng của những gì Christos Papadimitriou đã gọi là các thuật toán “Thời đại mới” cũng làm dấy lên sự quan tâm ngày càng tăng trong số các nhà khoa học máy tính lý thuyết (Koutsoupias và Papadimitriou, 1992; Aldous và Vazirani, 1994).

Trong lĩnh vực nghiên cứu hoạt động, một biến thể của leo đồi được gọi là **tìm kiếm tabu** đã trở nên phổ biến (Glover và Laguna, 1997). Thuật toán này duy trì một danh sách tabu gồm k trạng thái đã truy cập trước đó không thể được truy cập lại; cũng như cải thiện hiệu quả khi tìm kiếm đô thị, danh sách này có thể cho phép thuật toán thoát khỏi một số cực tiểu cục bộ.

Một cải tiến hữu ích khác trên leo đồi là thuật toán STAGE (Boyan và Moore, 1998). Ý tưởng là sử dụng các cực đại cục bộ được tìm thấy bằng cách khởi động lại leo đồi ngẫu nhiên để có được ý tưởng về hình dạng tổng thể của cảnh quan. Thuật toán phù hợp với một bề mặt bậc hai trơn tru cho tập hợp các cực đại cục bộ và sau đó tính toán cực đại toàn cầu của bề mặt đó một cách phân tích. Điều này trở thành điểm khởi động lại mới. Gomes et al. (1998) đã chỉ ra rằng thời gian chạy của các thuật toán quay lui có hệ thống thường có một **phân phối đuôi nặng**, có nghĩa là xác suất của một thời gian chạy rất dài lớn hơn so với dự đoán nếu thời gian chạy được phân phối theo cấp số mũ. Khi phân phối thời gian chạy có đuôi nặng, việc khởi động lại ngẫu nhiên tìm thấy một giải pháp nhanh hơn, trung bình, so với một lần chạy duy nhất đến khi hoàn thành. Hoos và Stutzle (2004) cung cấp một cuốn sách bao gồm chủ đề này.

Tôi luyện mô phỏng lần đầu tiên được Kirkpatrick et al. (1983) mô tả, người đã mượn trực tiếp từ thuật toán Metropolis (được sử dụng để mô phỏng các hệ thống phức tạp trong vật lý (Metropolis et al., 1953) và được cho là đã được phát minh tại một bữa tiệc tối ở Los Alamos). Tôi luyện mô phỏng hiện là một lĩnh vực của riêng nó, với hàng trăm bài báo được xuất bản mỗi năm.

Tìm kiếm các giải pháp tối ưu trong không gian liên tục là chủ đề của một số lĩnh vực, bao gồm lý thuyết tối ưu hóa, lý thuyết điều khiển tối ưu và vi tích phân biến phân. Các kỹ thuật cơ bản được Bishop (1995) giải thích tốt; Press et al. (2007) bao gồm một loạt các thuật toán và cung cấp phần mềm hoạt động.

Các nhà nghiên cứu đã lấy cảm hứng cho các thuật toán tìm kiếm và tối ưu hóa từ nhiều lĩnh vực nghiên cứu khác nhau: luyện kim (tôi luyện mô phỏng); sinh học (thuật toán di truyền); khoa học thần kinh (mạng lưới thần kinh); leo núi (leo đồi); kinh tế học (thuật toán dựa trên thị trường (Dias et al., 2006)); vật lý (đàn hạt (Li và Yao, 2012) và thủy tinh spin (Mezard et al., 1987)); hành vi động vật (học tăng cường, bộ tối ưu hóa sói xám (Mirjalili và Lewis, 2014)); điều học (tìm kiếm chim cú cu (Yang và Deb, 2014)); côn trùng học (đàn kiến (Dorigo et al., 2008), đàn ong (Karaboga và Basturk, 2007), đom đóm (Yang, 2009) và tối ưu hóa đom đóm (Krishnanand và Ghose, 2009)); và những lĩnh vực khác.

Quy hoạch tuyến tính (LP) lần đầu tiên được nhà toán học Leonid Kantorovich (1939) nghiên cứu một cách có hệ thống. Nó là một trong những ứng dụng đầu tiên của máy tính; thuật toán đơn hình (Dantzig, 1949) vẫn được sử dụng mặc dù có độ phức tạp theo cấp số mũ trong trường hợp xấu nhất. Karmarkar (1984) đã phát triển họ phương pháp điểm nội có hiệu quả hơn nhiều, được chứng minh là có độ phức tạp đa thức cho lớp bài toán tối ưu hóa lồi tổng quát hơn bởi Nesterov và Nemirovski (1994).

CHƯƠNG 5 CÁC BÀI TOÁN THỎA MÃN RÀNG BUỘC

Trong chương này, chúng ta sẽ xem xét cách xử lý các trạng thái không chỉ như những “hộp đen” nhỏ, mà còn như những cấu trúc bên trong, từ đó dẫn đến các phương pháp tìm kiếm mới và hiểu sâu hơn về cấu trúc của bài toán.

Các Chương 3 và 4 đã khám phá ý tưởng rằng các bài toán có thể được giải quyết bằng cách tìm kiếm trên không gian trạng thái: một đồ thị trong đó các nút là trạng thái và các cạnh là các hành động. Chúng ta đã thấy rằng các heuristic chuyên biệt cho từng miền có thể ước tính chi phí để đạt được mục tiêu từ một trạng thái nhất định, nhưng từ quan điểm của thuật toán tìm kiếm, mỗi trạng thái là một thể nguyên tử, hay không thể chia nhỏ—một hộp đen không có cấu trúc bên trong. Đối với mỗi bài toán, chúng ta cần mã chuyên biệt cho từng miền để mô tả các chuyển đổi giữa các trạng thái.

Trong chương này, chúng ta sẽ “mở hộp đen” bằng cách sử dụng một biểu diễn được phân tách cho mỗi trạng thái: một tập hợp các biến, mỗi biến có một giá trị. Một bài toán được giải quyết khi mỗi biến có một giá trị thỏa mãn tất cả các ràng buộc trên biến đó. Một bài toán được mô tả theo cách này được gọi là một **bài toán thỏa mãn ràng buộc**, hay **CSP**.

Các thuật toán tìm kiếm CSP tận dụng cấu trúc của các trạng thái và sử dụng các heuristic tổng quát thay vì các heuristic chuyên biệt cho từng miền để cho phép giải quyết các bài toán phức tạp. Ý tưởng chính là loại bỏ một lúc các phần lớn của không gian tìm kiếm bằng cách xác định các tổ hợp biến/giá trị vi phạm các ràng buộc. CSP có lợi thế bổ sung là các hành động và mô hình chuyển đổi có thể được suy ra từ mô tả bài toán.

5.1 Định nghĩa Bài toán Thỏa mãn Ràng buộc

Một bài toán thỏa mãn ràng buộc bao gồm ba thành phần: **X**, **D**, và **C**:

- **X** là một tập hợp các biến, $\{X_1, \dots, X_n\}$.
- **D** là một tập hợp các miền, $\{D_1, \dots, D_n\}$, mỗi miền cho một biến.
- **C** là một tập hợp các ràng buộc chỉ định các tổ hợp giá trị được phép.

Một miền, D_i , bao gồm một tập hợp các giá trị được phép, $\{v_1, \dots, v_k\}$, cho biến X_i . Ví dụ, một biến Boolean sẽ có miền $\{\text{true}, \text{false}\}$. Các biến khác nhau có thể có các miền khác nhau với kích thước khác nhau. Mỗi ràng buộc C_j bao gồm một cặp (phạm vi, quan hệ), trong đó **phạm vi** là một tuple các biến tham gia vào ràng buộc và **quan hệ** là một quan hệ xác định các giá trị mà các biến đó có thể nhận. Một quan hệ có thể được biểu diễn dưới dạng một tập hợp rõ ràng của tất cả các tuple giá trị thỏa mãn ràng buộc, hoặc dưới dạng một hàm có thể tính toán liệu một tuple có phải là thành viên của quan hệ hay không. Ví dụ, nếu X_1 và X_2 đều có miền $\{1, 2, 3\}$, thì ràng buộc nói rằng X_1 phải lớn hơn X_2 có thể được viết là $\langle (X_1, X_2), \{(3, 1), (3, 2), (2, 1)\} \rangle$ hoặc là $\langle (X_1, X_2), X_1 > X_2 \rangle$.

Các CSP xử lý các **gán giá trị** cho các biến, $\{X_i = v_i, X_j = v_j, \dots\}$. Một gán giá trị không vi phạm bất kỳ ràng buộc nào được gọi là một **gán giá trị nhất quán** hoặc **hợp lệ**. Một **gán giá trị đầy đủ** là một gán giá trị trong đó mọi biến đều được gán một giá trị, và một **nghiệm** cho một CSP là một gán giá trị nhất quán, đầy đủ. Một **gán giá trị một phần** là một gán giá trị để lại một số biến chưa được gán, và một **nghiệm một phần** là một gán giá trị một phần mà nhất quán. Giải một CSP nói chung là một bài toán **NP-đầy đủ**, mặc dù có những lớp con quan trọng của CSP có thể được giải quyết rất hiệu quả.

5.1.1 Bài toán ví dụ: Tô màu bản đồ

Giả sử rằng, sau khi đã chán với Romania, chúng ta đang xem xét một bản đồ của Úc cho thấy mỗi bang và vùng lãnh thổ của nó (Hình 5.1(a)). Chúng ta được giao nhiệm vụ tô màu cho mỗi vùng bằng màu đỏ, xanh lá cây, hoặc xanh dương sao cho không có hai vùng lân cận nào có cùng màu. Để diễn đạt bài toán này dưới dạng một CSP, chúng ta định nghĩa các biến là các vùng:

$$X = \{WA, NT, Q, NSW, V, SA, T\}.$$

Miền của mỗi biến là tập hợp $D_i = \{red, green, blue\}$. Các ràng buộc yêu cầu các vùng lân cận phải có các màu khác nhau. Vì có chín nơi các vùng tiếp giáp, có chín ràng buộc:

$$C = \{SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V\}.$$

Ở đây chúng ta đang sử dụng các chữ viết tắt; $SA \neq WA$ là viết tắt của $\langle (SA, WA), SA \neq WA \rangle$, trong đó $SA \neq WA$ có thể được liệt kê đầy đủ lần lượt là $\{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)\}$.

Có nhiều nghiệm khả thi cho bài toán này, chẳng hạn như:

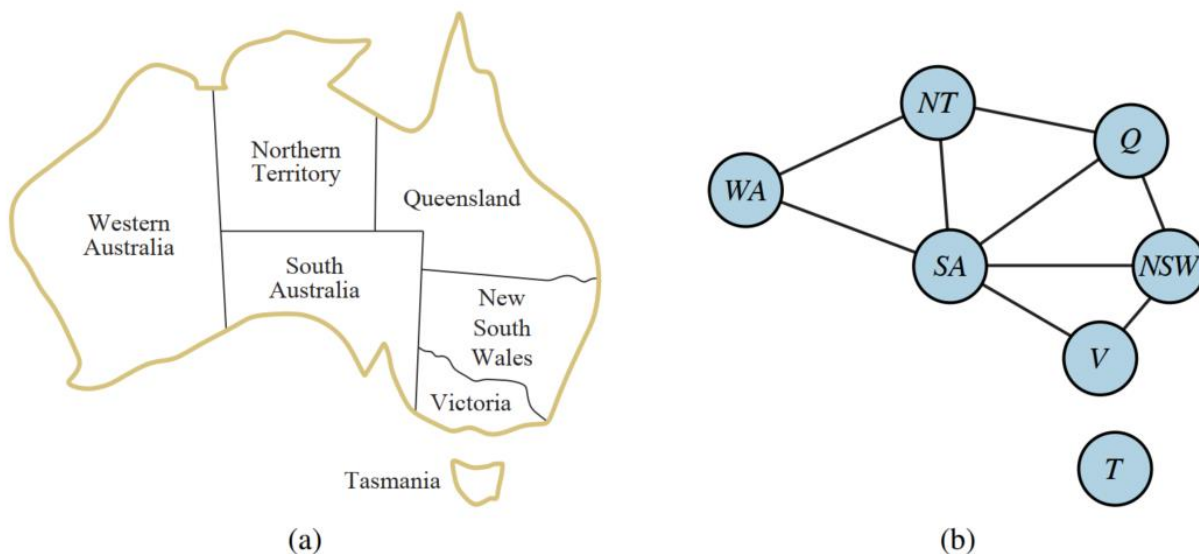
$$\{WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = red\}.$$

Có thể hữu ích khi hình dung một CSP dưới dạng một **đồ thị ràng buộc**, như được hiển thị trong Hình 5.1(b). Các nút của đồ thị tương ứng với các biến của bài toán, và một cạnh nối hai biến bất kỳ tham gia vào một ràng buộc.

Tại sao phải diễn đạt một bài toán dưới dạng CSP? Một lý do là các CSP tạo ra một biểu diễn tự nhiên cho nhiều loại bài toán; thường dễ dàng diễn đạt một bài toán dưới dạng một CSP. Một lý do khác là nhiều năm phát triển đã giúp các bộ giải CSP trở nên nhanh và hiệu quả. Thứ ba là một bộ giải CSP có thể nhanh chóng cắt bỏ những phần lớn của không gian tìm kiếm mà một bộ tìm kiếm không gian trạng thái nguyên tử không thể. Ví dụ, một khi chúng ta đã chọn $\{SA = blue\}$ trong bài toán Úc, chúng ta có thể kết luận rằng không có biến nào trong năm biến lân cận có thể nhận giá trị **blue**. Một thủ tục tìm kiếm không sử dụng ràng buộc sẽ phải xem xét $3^5 = 243$ gán giá trị cho năm biến lân cận; với các ràng buộc, chúng ta chỉ có $2^5 = 32$ gán giá trị để xem xét, giảm 87%.

Trong tìm kiếm không gian trạng thái nguyên tử, chúng ta chỉ có thể hỏi: trạng thái cụ thể này có phải là mục tiêu không? Không? Còn cái này thì sao? Với CSP, một khi chúng ta phát hiện ra rằng một gán giá trị một phần vi phạm một ràng buộc, chúng ta có thể loại bỏ ngay lập tức các tính

chính tiếp theo của gán giá trị một phần đó. Hơn nữa, chúng ta có thể thấy tại sao gán giá trị không phải là một nghiệm—chúng ta thấy biến nào vi phạm một ràng buộc—vì vậy chúng ta có thể tập trung sự chú ý vào các biến quan trọng. Kết quả là, nhiều bài toán không thể giải quyết được bằng tìm kiếm không gian trạng thái nguyên tử có thể được giải quyết nhanh chóng khi được diễn đạt dưới dạng một CSP.



Hình 5.1 (a) Các bang và vùng lãnh thổ chính của Úc. Việc tô màu bản đồ này có thể được xem như một bài toán thỏa mãn ràng buộc (CSP). Mục tiêu là gán màu cho mỗi vùng sao cho không có vùng lân cận nào có cùng màu. (b) Bài toán tô màu bản đồ được biểu diễn dưới dạng đồ thị ràng buộc.

5.1.2 Bài toán ví dụ: Lập lịch sản xuất

Các nhà máy có bài toán lập lịch cho các công việc trong một ngày, tuân theo nhiều ràng buộc khác nhau. Trong thực tế, nhiều bài toán trong số này được giải quyết bằng kỹ thuật CSP. Hãy xem xét bài toán lập lịch lắp ráp một chiếc xe hơi. Toàn bộ công việc bao gồm các nhiệm vụ, và chúng ta có thể mô hình hóa mỗi nhiệm vụ như một biến, trong đó giá trị của mỗi biến là thời gian bắt đầu của nhiệm vụ, được biểu thị bằng một số nguyên số phút. Các **ràng buộc ưu tiên** có thể khẳng định rằng một nhiệm vụ phải xảy ra trước một nhiệm vụ khác—ví dụ, một bánh xe phải được lắp trước khi nắp chụp được gắn vào—và chỉ có một số lượng nhiệm vụ nhất định có thể diễn ra cùng một lúc. Các ràng buộc cũng có thể chỉ định rằng một nhiệm vụ mất một khoảng thời gian nhất định để hoàn thành.

Chúng ta xem xét một phần nhỏ của việc lắp ráp xe hơi, bao gồm 15 nhiệm vụ: lắp trục (trước và sau), gắn cả bốn bánh xe (phải và trái, trước và sau), siết chặt các đai ốc cho mỗi bánh xe, gắn nắp chụp, và kiểm tra lắp ráp cuối cùng. Chúng ta có thể biểu diễn các nhiệm vụ bằng 15 biến:

$$X = \left\{ AxleF, AxleB, Wheel_{RF}, Wheel_{LF}, Wheel_{RB}, Wheel_{LB}, Nuts_{RF}, Nuts_{LF}, Nuts_{RB}, \right. \\ \left. Nuts_{LB}, Cap_{RF}, Cap_{LF}, Cap_{RB}, Cap_{LB}, Inspect \right\}.$$

Tiếp theo, chúng ta biểu diễn các ràng buộc ưu tiên giữa các nhiệm vụ riêng lẻ. Bất cứ khi nào một nhiệm vụ T_1 phải xảy ra trước nhiệm vụ T_2 , và nhiệm vụ T_1 mất thời gian d_1 để hoàn thành, chúng ta thêm một ràng buộc số học có dạng:

$$T_1 + d_1 \leq T_2 .$$

Trong ví dụ của chúng ta, các trục phải được đặt đúng chỗ trước khi các bánh xe được lắp vào, và mất 10 phút để lắp một trục, vì vậy chúng ta viết:

$$AxleF + 10 \leq Wheel_{RF} ; AxleF + 10 \leq Wheel_{LF} ;$$

$$AxleB + 10 \leq Wheel_{RB} ; AxleB + 10 \leq Wheel_{LB} .$$

Tiếp theo chúng ta nói rằng đối với mỗi bánh xe, chúng ta phải gắn bánh xe (mất 1 phút), sau đó siết chặt các đai ốc (2 phút), và cuối cùng gắn nắp chụp (1 phút, nhưng chưa được biểu diễn ở đây):

$$Wheel_{RF} + 1 \leq Nuts_{RF} ; Nuts_{RF} + 2 \leq Cap_{RF} ;$$

$$Wheel_{LF} + 1 \leq Nuts_{LF} ; Nuts_{LF} + 2 \leq Cap_{LF} ;$$

$$Wheel_{RB} + 1 \leq Nuts_{RB} ; Nuts_{RB} + 2 \leq Cap_{RB} ;$$

$$Wheel_{LB} + 1 \leq Nuts_{LB} ; Nuts_{LB} + 2 \leq Cap_{LB} .$$

Giả sử chúng ta có bốn công nhân để lắp bánh xe, nhưng họ phải dùng chung một công cụ giúp đặt trục vào đúng chỗ. Chúng ta cần một **ràng buộc lựa chọn** để nói rằng AxleF và AxleB không được trùng nhau về thời gian; một trong hai phải đến trước hoặc đến sau:

$$(AxleF + 10 \leq AxleB) \text{ hoặc } (AxleB + 10 \leq AxleF) .$$

Điều này trông giống như một ràng buộc phức tạp hơn, kết hợp số học và logic. Nhưng nó vẫn quy về một tập hợp các cặp giá trị mà AxleF và AxleB có thể nhận.

Chúng ta cũng cần khẳng định rằng việc kiểm tra diễn ra cuối cùng và mất 3 phút. Đối với mọi biến ngoại trừ Inspect, chúng ta thêm một ràng buộc có dạng $X + d_X \leq Inspect$. Cuối cùng, giả sử có một yêu cầu hoàn thành toàn bộ lắp ráp trong 30 phút. Chúng ta có thể đạt được điều đó bằng cách giới hạn miền của tất cả các biến:

$$D_{\{i\}} = \{0, 1, 2, 3, \dots, 30\}.$$

Bài toán cụ thể này rất dễ giải, nhưng các CSP đã được áp dụng thành công cho các bài toán lập lịch sản xuất như thế này với hàng nghìn biến.

5.1.3 Các biến thể của mô hình CSP

Loại CSP đơn giản nhất bao gồm các biến có miền rời rạc, hữu hạn. Các bài toán tô màu bản đồ và lập lịch với giới hạn thời gian đều thuộc loại này. Bài toán 8 con hậu (Hình 4.3) cũng có thể được xem là một CSP có miền hữu hạn, trong đó các biến $Q_1, \dots, Q_8$ tương ứng với các con hậu trong cột 1 đến 8, và miền của mỗi biến chỉ định các số hàng có thể có cho con hậu trong cột đó,

$D_i = \{1, 2, 3, 4, 5, 6, 7, 8\}$. Các ràng buộc nói rằng không có hai con hậu nào có thể ở cùng một hàng hoặc đường chéo.

Một miền rời rạc có thể là vô hạn, chẳng hạn như tập hợp các số nguyên hoặc chuỗi. (Nếu chúng ta không đặt thời hạn cho bài toán lập lịch công việc, sẽ có vô số thời gian bắt đầu cho mỗi biến.) Với các miền vô hạn, chúng ta phải sử dụng các ràng buộc ngầm như $T_1 + d_1 \leq T_2$ thay vì các tuple giá trị rõ ràng. Các thuật toán giải đặc biệt (mà chúng ta không thảo luận ở đây) tồn tại cho các ràng buộc tuyến tính trên các biến số nguyên—tức là, các ràng buộc, như đã nêu ở trên, trong đó mỗi biến chỉ xuất hiện ở dạng tuyến tính. Có thể chứng minh rằng không có thuật toán nào tồn tại để giải các ràng buộc phi tuyến tổng quát trên các biến số nguyên—bài toán là không thể quyết định.

Các bài toán thỏa mãn ràng buộc với các **miền liên tục** rất phổ biến trong thế giới thực và được nghiên cứu rộng rãi trong lĩnh vực nghiên cứu vận trù học. Ví dụ, việc lập lịch các thí nghiệm trên Kính viễn vọng không gian Hubble yêu cầu thời gian quan sát rất chính xác; thời gian bắt đầu và kết thúc của mỗi lần quan sát và thao tác là các biến có giá trị liên tục phải tuân theo nhiều ràng buộc thiên văn, ưu tiên và năng lượng khác nhau. Lớp CSP miền liên tục nổi tiếng nhất là các bài toán **quy hoạch tuyến tính**, trong đó các ràng buộc phải là các phương trình hoặc bất phương trình tuyến tính. Các bài toán quy hoạch tuyến tính có thể được giải trong thời gian đa thức theo số lượng biến. Các bài toán với các loại ràng buộc và hàm mục tiêu khác nhau cũng đã được nghiên cứu—quy hoạch bậc hai, quy hoạch hình nón bậc hai, v.v. Những bài toán này tạo thành một lĩnh vực quan trọng của toán học ứng dụng.

Ngoài việc kiểm tra các loại biến có thể xuất hiện trong CSP, điều hữu ích là xem xét các loại ràng buộc. Loại đơn giản nhất là **ràng buộc một ngôi**, giới hạn giá trị của một biến duy nhất. Ví dụ, trong bài toán tô màu bản đồ, có thể người dân Nam Úc không dung thứ cho màu xanh lá cây; chúng ta có thể thể hiện điều đó bằng ràng buộc một ngôi $\langle (SA), SA \neq \text{green} \rangle$. (Việc chỉ định ban đầu miền của một biến cũng có thể được xem là một ràng buộc một ngôi.)

Một **ràng buộc hai ngôi** liên quan đến hai biến. Ví dụ, $SA \neq NSW$ là một ràng buộc hai ngôi. Một **CSP hai ngôi** là một CSP chỉ có các ràng buộc một ngôi và hai ngôi; nó có thể được biểu diễn dưới dạng đồ thị ràng buộc, như trong Hình 5.1(b).

Chúng ta cũng có thể định nghĩa các ràng buộc bậc cao hơn. Ràng buộc ba ngôi $\text{Between}(X, Y, Z)$, ví dụ, có thể được định nghĩa là $\langle (X, Y, Z), X < Y < Z \text{ hoặc } X > Y > Z \rangle$.

Một ràng buộc liên quan đến một số lượng biến tùy ý được gọi là một **ràng buộc toàn cục**. (Tên này là truyền thống nhưng gây nhầm lẫn vì một ràng buộc toàn cục không nhất thiết phải liên quan đến tất cả các biến trong một bài toán). Một trong những ràng buộc toàn cục phổ biến nhất là **Alldiff**, nói rằng tất cả các biến liên quan đến ràng buộc phải có các giá trị khác nhau. Trong các bài toán Sudoku (xem Mục 5.2.6), tất cả các biến trong một hàng, cột hoặc ô vuông 3×3 phải thỏa mãn ràng buộc Alldiff.

Một ví dụ khác được cung cấp bởi các câu đố **mật mã số** (Hình 5.2(a)). Mỗi chữ cái trong một câu đố mật mã số đại diện cho một chữ số khác nhau. Đối với trường hợp trong Hình 5.2(a), điều này sẽ được biểu diễn dưới dạng ràng buộc toàn cục $\text{Alldiff}(F, T, U, W, R, O)$. Các ràng buộc cộng trên bốn cột của câu đố có thể được viết dưới dạng các ràng buộc n-ngôi sau:

$$O + O = R + 10 \cdot C_1$$

$$C_1 + W + W = U + 10 \cdot C_2$$

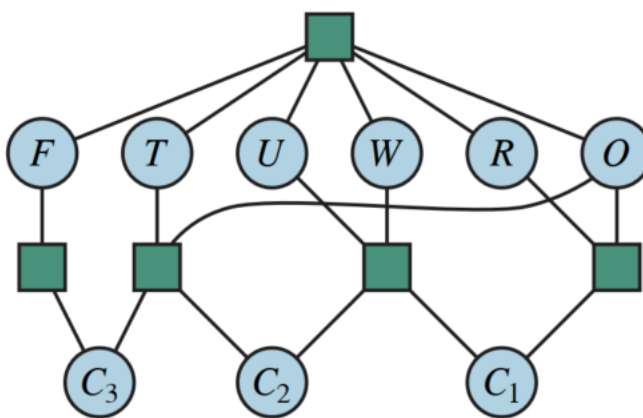
$$C_2 + T + T = O + 10 \cdot C_3$$

$$C_3 = F$$

trong đó C_1 , C_2 , và C_3 là các biến phụ trợ đại diện cho chữ số được mang sang hàng chục, hàng trăm, hoặc hàng nghìn. Các ràng buộc này có thể được biểu diễn trong một **siêu đồ thị ràng buộc**, chẳng hạn như đồ thị được hiển thị trong Hình 5.2(b). Một siêu đồ thị bao gồm các nút thông thường (các hình tròn trong hình) và các siêu nút (các hình vuông), đại diện cho các ràng buộc n-ngôi—các ràng buộc liên quan đến n biến.

Ngoài ra, như Bài tập 5.NARY yêu cầu bạn chứng minh, mọi ràng buộc miền hữu hạn có thể được giảm xuống một tập hợp các ràng buộc hai ngôi nếu đủ các biến phụ trợ được giới thiệu. Điều này có nghĩa là chúng ta có thể biến đổi bất kỳ CSP nào thành một CSP chỉ có các ràng buộc hai ngôi—điều này chắc chắn làm cho cuộc sống của người thiết kế thuật toán trở nên đơn giản hơn. Một cách khác để chuyển đổi một CSP n-ngôi thành một CSP hai ngôi là biến đổi **đồ thị đối ngẫu**: tạo một đồ thị mới trong đó sẽ có một biến cho mỗi ràng buộc trong đồ thị gốc, và một ràng buộc hai ngôi cho mỗi cặp ràng buộc trong đồ thị gốc có chung biến. Ví dụ, hãy xem xét một CSP với các biến $X = \{X, Y, Z\}$, mỗi biến có miền $\{1, 2, 3, 4, 5\}$, và với hai ràng buộc $C_1: \langle (X, Y, Z), X + Y = Z \rangle$ và $C_2: \langle (X, Y), X + 1 = Y \rangle$. Khi đó, đồ thị đối ngẫu sẽ có các biến $X = \{C_1, C_2\}$, trong đó miền của biến C_1 trong đồ thị đối ngẫu là tập hợp các tuple $\{(x_i, y_j, z_k)\}$ từ ràng buộc C_1 trong bài toán gốc, và tương tự miền của C_2 là tập hợp các tuple $\{(x_i, y_j)\}$. Đồ thị đối ngẫu có ràng buộc hai ngôi $\langle (C_1, C_2), R_1 \rangle$, trong đó R_1 là một quan hệ mới định nghĩa ràng buộc giữa C_1 và C_2 ; trong trường hợp này nó sẽ là $R_1 = \{((1, 2, 3), (1, 2)), ((2, 3, 5), (2, 3))\}$.

$$\begin{array}{r} T \quad W \quad O \\ + \quad T \quad W \quad O \\ \hline F \quad O \quad U \quad R \end{array}$$



(a)

(b)

Hình 5.2 (a) Một bài toán mật mã số. Mỗi chữ cái đại diện cho một chữ số riêng biệt; mục đích là tìm một sự thay thế các chữ số cho các chữ cái sao cho tổng số học thu được là chính xác, với ràng buộc bổ sung là không được có số 0 ở đầu. (b) Siêu đồ thị ràng buộc cho bài toán mật mã số, hiển

thị ràng buộc Alldiff (ô vuông ở trên cùng) cũng như các ràng buộc cộng cột (bốn ô vuông ở giữa). Các biến C_1 , C_2 , và C_3 đại diện cho các chữ số nhớ cho ba cột từ phải sang trái.

Tuy nhiên, có hai lý do tại sao chúng ta có thể thích một ràng buộc toàn cục như Alldiff hơn là một tập hợp các ràng buộc hai ngôi. Thứ nhất, việc viết mô tả bài toán bằng cách sử dụng Alldiff dễ dàng hơn và ít lỗi hơn. Thứ hai, có thể thiết kế các thuật toán suy luận chuyên dụng cho các ràng buộc toàn cục hiệu quả hơn so với việc hoạt động với các ràng buộc nguyên thủy. Chúng ta mô tả các thuật toán suy luận này trong Mục 5.2.5.

Các ràng buộc chúng ta đã mô tả cho đến nay đều là các **ràng buộc tuyệt đối**, việc vi phạm chúng loại bỏ một nghiệm tiềm năng. Nhiều CSP trong thế giới thực bao gồm các **ràng buộc ưu tiên** chỉ ra những nghiệm nào được ưa chuộng hơn. Ví dụ, trong một bài toán lập lịch lớp học đại học có các ràng buộc tuyệt đối rằng không giáo sư nào có thể dạy hai lớp cùng một lúc. Nhưng chúng ta cũng có thể cho phép các ràng buộc ưu tiên: Giáo sư R có thể thích dạy vào buổi sáng, trong khi Giáo sư N thích dạy vào buổi chiều. Một lịch trình có Giáo sư R dạy vào lúc 2 giờ chiều vẫn sẽ là một nghiệm được phép (trừ khi Giáo sư R là trưởng khoa) nhưng sẽ không phải là một nghiệm tối ưu. Các ràng buộc ưu tiên thường có thể được mã hóa dưới dạng chi phí trên các gán giá trị biến riêng lẻ—ví dụ, gán một vị trí buổi chiều cho Giáo sư R tốn 2 điểm so với hàm mục tiêu tổng thể, trong khi một vị trí buổi sáng tốn 1 điểm. Với công thức này, các CSP có ưu tiên có thể được giải quyết bằng các phương pháp tìm kiếm tối ưu hóa, dựa trên đường đi hoặc cục bộ. Chúng ta gọi một bài toán như vậy là **bài toán tối ưu hóa ràng buộc**, hay **COP**. Các chương trình tuyến tính là một lớp của COP.

5.2 Lan truyền Ràng buộc: Suy luận trong CSP

Một thuật toán tìm kiếm không gian trạng thái nguyên tử chỉ tiến hành theo một cách: bằng cách mở rộng một nút để truy cập các nút kế nhiệm. Một thuật toán CSP có nhiều lựa chọn. Nó có thể tạo ra các nút kế nhiệm bằng cách chọn một gán giá trị biến mới, hoặc nó có thể thực hiện một loại suy luận cụ thể được gọi là **lan truyền ràng buộc**: sử dụng các ràng buộc để giảm số lượng giá trị hợp lệ cho một biến, từ đó có thể giảm các giá trị hợp lệ cho một biến khác, và cứ thế tiếp tục. Ý tưởng là điều này sẽ để lại ít lựa chọn hơn để xem xét khi chúng ta thực hiện lựa chọn tiếp theo về một gán giá trị biến. Lan truyền ràng buộc có thể được đan xen với tìm kiếm, hoặc có thể được thực hiện như một bước tiền xử lý, trước khi tìm kiếm bắt đầu. Đôi khi bước tiền xử lý này có thể giải quyết toàn bộ bài toán, do đó không cần tìm kiếm gì cả.

Ý tưởng chính là **tính nhất quán cục bộ**. Nếu chúng ta coi mỗi biến là một nút trong một đồ thị (xem Hình 5.1(b)) và mỗi ràng buộc hai ngôi là một cạnh, thì quá trình thực thi tính nhất quán cục bộ trong mỗi phần của đồ thị sẽ làm cho các giá trị không nhất quán bị loại bỏ trên toàn bộ đồ thị. Có nhiều loại tính nhất quán cục bộ khác nhau, mà chúng ta sẽ lần lượt đề cập.

5.2.1 Tính nhất quán nút

Một biến duy nhất (tương ứng với một nút trong đồ thị CSP) là **nhất quán nút** nếu tất cả các giá trị trong miền của biến thỏa mãn các ràng buộc một ngôi của biến đó. Ví dụ, trong biến thể của bài toán tô màu bản đồ Úc (Hình 5.1) trong đó người dân Nam Úc không thích màu xanh lá cây, biến SA bắt đầu với miền {red, green, blue}, và chúng ta có thể làm cho nó nhất quán nút bằng cách

loại bỏ green, để lại SA với miền bị rút gọn là {red, blue}. Chúng ta nói rằng một đồ thị là nhất quán nút nếu mọi biến trong đồ thị đều nhất quán nút.

Việc loại bỏ tất cả các ràng buộc một ngôi trong một CSP rất dễ dàng bằng cách giảm miền của các biến có ràng buộc một ngôi ngay từ đầu quá trình giải quyết. Như đã đề cập trước đó, cũng có thể biến đổi tất cả các ràng buộc n-ngôi thành hai ngôi. Vì lý do này, một số bộ giải CSP chỉ làm việc với các ràng buộc hai ngôi, mong đợi người dùng loại bỏ các ràng buộc khác trước đó. Chúng ta đưa ra giả định đó cho phần còn lại của chương này, trừ khi có ghi chú.

5.2.2 Tính nhất quán cung

Một biến trong một CSP là **nhất quán cung** nếu mọi giá trị trong miền của nó thỏa mãn các ràng buộc hai ngôi của biến đó. Chính xác hơn, X_i nhất quán cung đối với một biến khác X_j nếu với mọi giá trị trong miền hiện tại D_i có một số giá trị trong miền D_j thỏa mãn ràng buộc hai ngôi trên cung (X_i, X_j) . Một đồ thị là nhất quán cung nếu mọi biến đều nhất quán cung với mọi biến khác. Ví dụ, hãy xem xét ràng buộc $Y = X^2$ trong đó miền của cả X và Y là tập hợp các chữ số thập phân. Chúng ta có thể viết ràng buộc này một cách rõ ràng là $\langle (X, Y), \{(0, 0), (1, 1), (2, 4), (3, 9)\} \rangle$.

Để làm cho X nhất quán cung đối với Y , chúng ta giảm miền của X xuống $\{0, 1, 2, 3\}$. Nếu chúng ta cũng làm cho Y nhất quán cung đối với X , thì miền của Y trở thành $\{0, 1, 4, 9\}$, và toàn bộ CSP là nhất quán cung. Mặt khác, tính nhất quán cung không thể làm gì cho bài toán tô màu bản đồ Úc.

Hãy xem xét ràng buộc bất đẳng thức sau trên (SA, WA):

$(\text{textr\textcolor{red}{red}, green}), (\text{textr\textcolor{red}{red}, blue}), (\text{textgreen}, red), (\text{textgreen}, blue), (\text{textblue}, red),$

$(\text{textblue}, green)$. Bất kể bạn chọn giá trị nào cho SA (hoặc cho WA), đều có một giá trị hợp lệ cho biến kia. Vì vậy, việc áp dụng tính nhất quán cung không có tác dụng gì đối với miền của cả hai biến.

Thuật toán phổ biến nhất để thực thi tính nhất quán cung được gọi là **AC-3** (xem Hình 5.3). Để làm cho mọi biến nhất quán cung, thuật toán AC-3 duy trì một hàng đợi các cung để xem xét. Ban đầu, hàng đợi chứa tất cả các cung trong CSP. (Mỗi ràng buộc hai ngôi trở thành hai cung, một theo mỗi hướng.) AC-3 sau đó lấy ra một cung tùy ý (X_i, X_j) từ hàng đợi và làm cho X_i nhất quán cung đối với X_j . Nếu điều này không làm thay đổi D_i , thuật toán chỉ đơn giản chuyển sang cung tiếp theo. Nhưng nếu điều này sửa đổi D_i (làm cho miền nhỏ hơn), thì chúng ta thêm vào hàng đợi tất cả các cung (X_k, X_i) trong đó X_k là một nút lân cận của X_i . Chúng ta cần làm điều đó vì sự thay đổi trong D_i có thể cho phép giảm thêm trong D_k , ngay cả khi chúng ta đã xem xét X_k trước đó. Nếu D_i bị rút gọn xuống rỗng, thì chúng ta biết rằng toàn bộ CSP không có nghiệm nhất quán, và AC-3 có thể ngay lập tức trả về thất bại. Ngược lại, chúng ta tiếp tục kiểm tra, cố gắng loại bỏ các giá trị khỏi miền của các biến cho đến khi không còn cung nào trong hàng đợi. Tại thời điểm đó, chúng ta còn lại một CSP tương đương với CSP gốc—chúng đều có cùng các nghiệm—nhưng CSP nhất quán cung sẽ nhanh hơn để tìm kiếm vì các biến của nó có miền nhỏ hơn. Trong một số trường hợp, nó giải quyết bài toán hoàn toàn (bằng cách giảm mọi miền xuống kích thước 1) và trong những trường hợp khác nó chứng minh rằng không có nghiệm nào tồn tại (bằng cách giảm một số miền xuống kích thước 0).

Độ phức tạp của AC-3 có thể được phân tích như sau. Giả sử một CSP có n biến, mỗi biến có kích thước miền tối đa là d , và với c ràng buộc hai ngôi (cung). Mỗi cung (X_k, X_i) chỉ có thể được chèn vào hàng đợi d lần vì X_i có tối đa d giá trị để xóa. Việc kiểm tra tính nhất quán của một cung có thể được thực hiện trong thời gian $O(d^2)$, vì vậy chúng ta có tổng thời gian trường hợp xấu nhất là $O(cd^3)$.

```
function AC-3( csp) returns false if an inconsistency is found and true otherwise
  queue ← a queue of arcs, initially all the arcs in csp
  while queue is not empty do
    (Xi, Xj) ← POP(queue)
    if REVISE(csp, Xi, Xj) then
      if size of Di = 0 then return false
      for each Xk in Xi.Neighbors - {Xj} do
        add (Xk, Xi) to queue
  return true
```

```
function REVISE( csp, Xi, Xj) returns true iff we revise the domain of Xi
  revised ← false
  for each x in Di do
    if no value y in Dj allows (x,y) to satisfy the constraint between Xi and Xj then
      delete x from Di
      revised ← true
  return revised
```

Hình 5.3 Thuật toán nhất quán cung AC-3. Sau khi áp dụng AC-3, hoặc mọi cung đều nhất quán cung, hoặc một số biến có miền rỗng, cho thấy rằng CSP không thể được giải quyết. Tên “AC-3” được người phát minh ra thuật toán sử dụng (Mackworth, 1977) vì nó là phiên bản thứ ba được phát triển trong bài báo.

5.2.3 Tính nhất quán đường đi

Giả sử chúng ta phải tô màu bản đồ Úc chỉ với hai màu, đỏ và xanh dương. Tính nhất quán cung không làm gì cả vì mọi ràng buộc có thể được thỏa mãn riêng lẻ với màu đỏ ở một đầu và màu xanh dương ở đầu kia. Nhưng rõ ràng là không có nghiệm cho bài toán: vì Tây Úc, Lãnh thổ Bắc Úc, và Nam Úc đều tiếp xúc với nhau, chúng ta cần ít nhất ba màu chỉ riêng cho chúng.

Tính nhất quán cung thắt chặt các miền (ràng buộc một ngôi) bằng cách sử dụng các cung (ràng buộc hai ngôi). Để tiến bộ trên các bài toán như tô màu bản đồ, chúng ta cần một khái niệm nhất quán mạnh hơn. **Tính nhất quán đường đi** thắt chặt các ràng buộc hai ngôi bằng cách sử dụng các ràng buộc ngầm được suy ra bằng cách xem xét ba biến.

Một tập hợp hai biến $\{X_i, X_j\}$ là nhất quán đường đi đối với một biến thứ ba X_m nếu, với mọi gán giá trị $\{X_i = a, X_j = b\}$ nhất quán với các ràng buộc (nếu có) trên $\{X_i, X_j\}$, có một gán giá trị cho X_m thỏa mãn các ràng buộc trên $\{X_i, X_m\}$ và $\{X_m, X_j\}$. Tên gọi này đề cập đến tính nhất quán tổng thể của đường đi từ X_i đến X_j với X_m ở giữa.

Hãy xem tính nhất quán đường đi hoạt động như thế nào trong việc tô màu bản đồ Úc với hai màu. Chúng ta sẽ làm cho tập hợp $\{WA, SA\}$ nhất quán đường đi đối với NT. Chúng ta bắt đầu bằng cách liệt kê các gán giá trị nhất quán cho tập hợp. Trong trường hợp này, chỉ có hai: $\{WA = \text{red}, SA = \text{blue}\}$ và $\{WA = \text{blue}, SA = \text{red}\}$. Chúng ta có thể thấy rằng với cả hai gán giá trị này, NT không thể là màu đỏ cũng không thể là màu xanh dương (vì nó sẽ xung đột với WA hoặc SA). Vì không có lựa chọn hợp lệ nào cho NT, chúng ta loại bỏ cả hai gán giá trị, và cuối cùng chúng ta không còn gán giá trị hợp lệ nào cho $\{WA, SA\}$. Do đó, chúng ta biết rằng không thể có nghiệm cho bài toán này.

5.2.4 Tính nhất quán k

Các dạng lan truyền mạnh hơn có thể được định nghĩa bằng khái niệm **tính nhất quán k** . Một CSP là **nhất quán k** nếu, với bất kỳ tập hợp $k - 1$ biến nào và với bất kỳ gán giá trị nhất quán nào cho các biến đó, một giá trị nhất quán luôn có thể được gán cho bất kỳ biến thứ k nào. Tính nhất quán 1 nói rằng, với tập hợp rỗng, chúng ta có thể làm cho bất kỳ tập hợp một biến nào nhất quán: đây là cái mà chúng ta gọi là tính nhất quán nút. Tính nhất quán 2 giống như tính nhất quán cung. Đối với các đồ thị ràng buộc hai ngôi, tính nhất quán 3 giống như tính nhất quán đường đi.

Một CSP là **nhất quán k mạnh** nếu nó nhất quán k và cũng nhất quán $(k - 1)$, nhất quán $(k - 2)$, ..., cho đến nhất quán 1. Bây giờ giả sử chúng ta có một CSP với n nút và làm cho nó nhất quán n mạnh (tức là, nhất quán k mạnh cho $k = n$). Sau đó, chúng ta có thể giải bài toán như sau: Đầu tiên, chúng ta chọn một giá trị nhất quán cho X_1 . Sau đó, chúng ta được đảm bảo có thể chọn một giá trị cho X_2 vì đồ thị là nhất quán 2, cho X_3 vì nó là nhất quán 3, v.v. Đối với mỗi biến X_i , chúng ta chỉ cần tìm kiếm qua d giá trị trong miền để tìm một giá trị nhất quán với $X_1, \dots, X_i - 1$. Tổng thời gian chạy chỉ là $O(n^2d)$.

Tất nhiên, không có bữa trưa nào miễn phí: thỏa mãn ràng buộc nói chung là **NP-đầy đủ**, và bất kỳ thuật toán nào để thiết lập tính nhất quán n phải mất thời gian hàm mũ theo n trong trường hợp xấu nhất. Tệ hơn nữa, tính nhất quán n cũng yêu cầu không gian hàm mũ theo n . Trong thực tế, việc xác định mức độ kiểm tra tính nhất quán phù hợp chủ yếu là một khoa học thực nghiệm. Tính toán tính nhất quán 2 là phổ biến, và tính nhất quán 3 ít phổ biến hơn.

5.2.5 Ràng buộc toàn cục

Hãy nhớ rằng một ràng buộc toàn cục là một ràng buộc liên quan đến một số lượng biến tùy ý (nhưng không nhất thiết phải là tất cả các biến). Các ràng buộc toàn cục xảy ra thường xuyên trong các bài toán thực tế và có thể được xử lý bằng các thuật toán chuyên dụng hiệu quả hơn các phương pháp đa năng được mô tả cho đến nay. Ví dụ, ràng buộc **Alldiff** nói rằng tất cả các biến liên quan phải có các giá trị riêng biệt (như trong bài toán mật mã số ở trên và câu đố Sudoku dưới đây). Một dạng đơn giản của phát hiện tính không nhất quán cho các ràng buộc Alldiff hoạt động như sau: nếu m biến liên quan đến ràng buộc, và nếu chúng có tổng cộng n giá trị riêng biệt có thể có, và $m > n$, thì ràng buộc không thể được thỏa mãn.

Điều này dẫn đến thuật toán đơn giản sau: Đầu tiên, loại bỏ bất kỳ biến nào trong ràng buộc có miền đơn lẻ, và xóa giá trị của biến đó khỏi miền của các biến còn lại. Lặp lại quá trình này miễn là còn các biến đơn lẻ. Nếu tại bất kỳ thời điểm nào một miền rỗng được tạo ra hoặc có nhiều biến hơn các giá trị miền còn lại, thì một sự không nhất quán đã được phát hiện.

Phương pháp này có thể phát hiện sự không nhất quán trong gán giá trị $\{WA = red, NSW = red\}$ cho Hình 5.1. Lưu ý rằng các biến SA, NT và Q được kết nối một cách hiệu quả bởi một ràng buộc Alldiff vì mỗi cặp phải có hai màu khác nhau. Sau khi áp dụng AC-3 với gán giá trị một phần, miền của SA, NT và Q đều bị giảm xuống $\{green, blue\}$. Tức là, chúng ta có ba biến và chỉ có hai màu, do đó ràng buộc Alldiff bị vi phạm. Do đó, một thủ tục nhất quán đơn giản cho một ràng buộc bậc cao hơn đôi khi hiệu quả hơn việc áp dụng tính nhất quán cung cho một tập hợp tương đương các ràng buộc hai ngôi.

Một ràng buộc bậc cao hơn quan trọng khác là **ràng buộc tài nguyên**, đôi khi được gọi là **ràng buộc Atmost**. Ví dụ, trong một bài toán lập lịch, hãy để $P_1, \dots, P_4$ biểu thị số lượng nhân sự được giao cho mỗi trong bốn nhiệm vụ. Ràng buộc rằng không có nhiều hơn 10 nhân sự được giao tổng cộng được viết là $Atmost(10, P_1, P_2, P_3, P_4)$. Chúng ta có thể phát hiện một sự không nhất quán đơn giản bằng cách kiểm tra tổng các giá trị tối thiểu của các miền hiện tại; ví dụ, nếu mỗi biến có miền $\{3, 4, 5, 6\}$, ràng buộc Atmost không thể được thỏa mãn. Chúng ta cũng có thể thực thi tính nhất quán bằng cách xóa giá trị tối đa của bất kỳ miền nào nếu nó không nhất quán với các giá trị tối thiểu của các miền khác. Do đó, nếu mỗi biến trong ví dụ của chúng ta có miền $\{2, 3, 4, 5, 6\}$, các giá trị 5 và 6 có thể bị xóa khỏi mỗi miền.

Đối với các bài toán lớn bị giới hạn tài nguyên với các giá trị số nguyên—chẳng hạn như các bài toán hậu cần liên quan đến việc di chuyển hàng nghìn người trong hàng trăm phương tiện—thường không thể biểu diễn miền của mỗi biến dưới dạng một tập hợp lớn các số nguyên và dần dần giảm tập hợp đó bằng các phương pháp kiểm tra tính nhất quán. Thay vào đó, các miền được biểu diễn bằng các cận trên và dưới và được quản lý bằng **lan truyền cận**. Ví dụ, trong một bài toán lập lịch hãng hàng không, giả sử có hai chuyến bay, F_1 và F_2 , mà các máy bay có sức chứa lần lượt là 165 và 385. Các miền ban đầu cho số lượng hành khách trên các chuyến bay F_1 và F_2 là

$$D_1 = [0, 165] \text{ và } D_2 = [0, 385].$$

Bây giờ giả sử chúng ta có ràng buộc bổ sung rằng hai chuyến bay cùng nhau phải chở 420 người: $F_1 + F_2 = 420$. Lan truyền các ràng buộc cận, chúng ta giảm các miền xuống

$D_1 = [35, 165]$ và $D_2 = [255, 385]$. Chúng ta nói rằng một CSP là **nhất quán cận** nếu với mọi biến X, và đối với cả giá trị cận dưới và cận trên của X, tồn tại một số giá trị của Y thỏa mãn ràng buộc giữa X và Y cho mọi biến Y. Loại lan truyền cận này được sử dụng rộng rãi trong các bài toán ràng buộc thực tế.

5.2.6 Sudoku

Câu đố Sudoku phổ biến đã giới thiệu hàng triệu người đến với các bài toán thỏa mãn ràng buộc, mặc dù họ có thể không nhận ra điều đó. Một bảng Sudoku bao gồm 81 ô vuông, một số trong đó ban đầu được điền các chữ số từ 1 đến 9. Câu đố là điền vào tất cả các ô vuông còn lại sao cho không có chữ số nào xuất hiện hai lần trong bất kỳ hàng, cột, hoặc ô vuông 3×3 nào (xem Hình 5.4). Một hàng, cột hoặc ô vuông được gọi là một **đơn vị**.

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

(b)

Hình 5.4 (a) Một câu đố Sudoku và (b) nghiệm của nó.

Các câu đố Sudoku xuất hiện trên báo và sách câu đố có một đặc điểm là chỉ có đúng một nghiệm. Mặc dù một số có thể khó giải bằng tay, mất hàng chục phút, một bộ giải CSP có thể xử lý hàng nghìn câu đố mỗi giây.

Một câu đố Sudoku có thể được coi là một CSP với 81 biến, một cho mỗi ô vuông. Chúng ta sử dụng tên biến A1 đến A9 cho hàng trên cùng (từ trái sang phải), xuống đến I1 đến I9 cho hàng dưới cùng. Các ô vuông trống có miền $\{1,2,3,4,5,6,7,8,9\}$ và các ô vuông được điền sẵn có miền bao gồm một giá trị duy nhất. Ngoài ra, có 27 ràng buộc Alldiff khác nhau, một cho mỗi đơn vị (hàng, cột và ô vuông 9 ô):

$Alldiff(A1, A2, A3, A4, A5, A6, A7, A8, A9)$

$Alldiff(B1, B2, B3, B4, B5, B6, B7, B8, B9)$

...

$Alldiff(A1, B1, C1, D1, E1, F1, G1, H1, I1)$

$Alldiff(A2, B2, C2, D2, E2, F2, G2, H2, I2)$

...

$Alldiff(A1, A2, A3, B1, B2, B3, C1, C2, C3)$

$Alldiff(A4, A5, A6, B4, B5, B6, C4, C5, C6)$

...

Hãy xem tính nhất quán cung có thể đưa chúng ta đi xa đến đâu. Giả sử rằng các ràng buộc Alldiff đã được mở rộng thành các ràng buộc hai ngôi (chẳng hạn như $A1 \neq A2$) để chúng ta có thể áp

dụng trực tiếp thuật toán AC-3. Hãy xem xét biến E6 từ Hình 5.4(a)—ô vuông trống giữa 2 và 8 trong ô vuông ở giữa. Từ các ràng buộc trong ô vuông đó, chúng ta có thể loại bỏ 1, 2, 7 và 8 khỏi miền của E6. Từ các ràng buộc trong cột của nó, chúng ta có thể loại bỏ 5, 6, 2, 8, 9 và 3 (mặc dù 2 và 8 đã bị loại bỏ). Điều đó để lại cho E6 một miền là {4} ; nói cách khác, chúng ta biết câu trả lời cho E6. Bây giờ hãy xem xét biến I6—ô vuông ở ô vuông giữa dưới cùng được bao quanh bởi 1, 3 và 3. Áp dụng tính nhất quán cung trong cột của nó, chúng ta loại bỏ 5, 6, 2, 4 (vì bây giờ chúng ta biết E6 phải là 4), 8, 9 và 3. Chúng ta loại bỏ 1 bằng tính nhất quán cung với I5, và chúng ta chỉ còn lại giá trị 7 trong miền của I6. Bây giờ có 8 giá trị đã biết trong cột 6, vì vậy tính nhất quán cung có thể suy ra rằng A6 phải là 1. Suy luận tiếp tục theo các dòng này, và cuối cùng, AC-3 có thể giải toàn bộ câu đố—tất cả các biến đều có miền của chúng bị giảm xuống một giá trị duy nhất, như được hiển thị trong Hình 5.4(b).

Tất nhiên, Sudoku sẽ sớm mất đi sự hấp dẫn nếu mọi câu đố có thể được giải bằng một ứng dụng cơ học của AC-3, và thực tế AC-3 chỉ hoạt động với những câu đố Sudoku dễ nhất. Những câu đố khó hơn một chút có thể được giải bằng PC-2, nhưng với chi phí tính toán lớn hơn: có 255.960 ràng buộc đường đi khác nhau để xem xét trong một câu đố Sudoku. Để giải những câu đố khó nhất và để tiến bộ một cách hiệu quả, chúng ta sẽ phải thông minh hơn.

Thật vậy, sự hấp dẫn của các câu đố Sudoku đối với người giải là nhu cầu phải tháo vát trong việc áp dụng các chiến lược suy luận phức tạp hơn. Những người đam mê đặt cho chúng những cái tên đầy màu sắc, chẳng hạn như “bộ ba trần trụi”. Chiến lược đó hoạt động như sau: trong bất kỳ đơn vị nào (hàng, cột hoặc ô vuông), tìm ba ô vuông mà mỗi ô có một miền chứa cùng ba số hoặc một tập hợp con của các số đó. Ví dụ, ba miền có thể là {1,8}, {3,8} và {1,3,8} . Từ đó chúng ta không biết ô vuông nào chứa 1, 3 hoặc 8, nhưng chúng ta biết rằng ba số đó phải được phân phối trong ba ô vuông. Do đó, chúng ta có thể loại bỏ 1, 3 và 8 khỏi miền của mọi ô vuông khác trong đơn vị đó.

Thật thú vị khi lưu ý rằng chúng ta có thể đi xa đến mức nào mà không nói nhiều điều cụ thể về Sudoku. Tất nhiên chúng ta phải nói rằng có 81 biến, rằng miền của chúng là các chữ số từ 1 đến 9, và có 27 ràng buộc Alldiff. Nhưng ngoài ra, tất cả các chiến lược—tính nhất quán cung, tính nhất quán đường đi, v.v.—đều áp dụng chung cho tất cả các CSP, không chỉ riêng cho các bài toán Sudoku. Ngay cả “bộ ba trần trụi” cũng thực sự là một chiến lược để thực thi tính nhất quán của các ràng buộc Alldiff và không cụ thể cho Sudoku. Đây là sức mạnh của mô hình CSP: đối với mỗi lĩnh vực bài toán mới, chúng ta chỉ cần định nghĩa bài toán dưới dạng các ràng buộc; sau đó các cơ chế giải ràng buộc chung có thể đảm nhiệm.

5.3 Tìm kiếm quay lui cho CSP

Đôi khi chúng ta có thể hoàn thành quá trình lan truyền ràng buộc nhưng vẫn còn các biến có nhiều giá trị khả dĩ. Trong trường hợp đó, chúng ta phải tìm kiếm một giải pháp. Trong phần này, chúng ta sẽ đề cập đến các thuật toán **tìm kiếm quay lui** hoạt động trên các gán giá trị một phần; trong phần tiếp theo, chúng ta sẽ xem xét các thuật toán tìm kiếm cục bộ trên các gán giá trị đầy đủ.

Hãy xem xét cách một tìm kiếm giới hạn độ sâu tiêu chuẩn (từ Chương 3) có thể giải quyết các CSP. Một trạng thái sẽ là một gán giá trị một phần, và một hành động sẽ mở rộng gán giá trị, thêm, chẳng hạn, NSW = red hoặc SA = blue cho bài toán tô màu bản đồ Úc. Đối với một CSP có n biến với kích thước miền là d , chúng ta sẽ có một cây tìm kiếm trong đó tất cả các gán giá trị đầy đủ (và do đó tất cả các nghiệm) là các nút lá ở độ sâu n . Nhưng hãy lưu ý rằng hệ số phân nhánh ở

cấp cao nhất sẽ là $n \cdot d$ vì bất kỳ trong số d giá trị nào cũng có thể được gán cho bất kỳ trong số n biến nào. Ở cấp độ tiếp theo, hệ số phân nhánh là $(n - 1)d$, và cứ thế tiếp tục trong n cấp. Vì vậy, cây có $n \cdot d^n$ nút lá, mặc dù chỉ có d^n gán giá trị đầy đủ có thể có!

Chúng ta có thể lấy lại hệ số n đó bằng cách nhận ra một thuộc tính quan trọng của CSP: **tính giao hoán**. Một bài toán là giao hoán nếu thứ tự áp dụng của bất kỳ tập hợp hành động nào không quan trọng. Trong các CSP, không có sự khác biệt nếu chúng ta gán NSW = red trước và sau đó là SA = blue, hay ngược lại. Do đó, chúng ta chỉ cần xem xét một biến duy nhất tại mỗi nút trong cây tìm kiếm. Tại gốc, chúng ta có thể đưa ra lựa chọn giữa SA = red, SA = green, và SA = blue, nhưng chúng ta sẽ không bao giờ chọn giữa NSW = red và SA = blue. Với giới hạn này, số nút lá là d^n , như chúng ta mong đợi. Ở mỗi cấp của cây, chúng ta phải chọn biến nào chúng ta sẽ xử lý, nhưng chúng ta không bao giờ phải quay lui qua lựa chọn đó.

Hình 5.5 cho thấy một thủ tục tìm kiếm quay lui cho các CSP. Nó lặp đi lặp lại chọn một biến chưa được gán, và sau đó lần lượt thử tất cả các giá trị trong miền của biến đó, cố gắng mở rộng từng giá trị thành một nghiệm thông qua một lời gọi đệ quy. Nếu lời gọi thành công, nghiệm được trả về, và nếu nó thất bại, gán giá trị được khôi phục về trạng thái trước đó, và chúng ta thử giá trị tiếp theo. Nếu không có giá trị nào hoạt động thì chúng ta trả về thất bại. Một phần của cây tìm kiếm cho bài toán Úc được hiển thị trong Hình 5.6, nơi chúng ta đã gán các biến theo thứ tự WA, NT, Q,... Lưu ý rằng **BACKTRACKING-SEARCH** chỉ giữ một biểu diễn duy nhất của một trạng thái (gán giá trị) và thay đổi biểu diễn đó thay vì tạo ra các biểu diễn mới (xem trang 98).

Trong khi các thuật toán tìm kiếm không thông tin của Chương 3 chỉ có thể được cải thiện bằng cách cung cấp cho chúng các heuristic chuyên biệt cho từng miền, hóa ra tìm kiếm quay lui có thể được cải thiện bằng cách sử dụng các heuristic độc lập với miền tận dụng biểu diễn được phân tách của các CSP. Trong bốn phần sau, chúng ta sẽ chỉ ra cách thực hiện điều này:

- **(5.3.1)** Biến nào nên được gán tiếp theo (**SELECT-UNASSIGNED-VARIABLE**), và theo thứ tự nào các giá trị của nó nên được thử (**ORDER-DOMAIN-VALUES**)?
- **(5.3.2)** Những suy luận nào nên được thực hiện ở mỗi bước trong tìm kiếm (**INFERENCE**)?
- **(5.3.3)** Chúng ta có thể **BACKTRACK** nhiều hơn một bước khi thích hợp không?
- **(5.3.4)** Chúng ta có thể lưu và sử dụng lại các kết quả một phần từ tìm kiếm không?

```
function BACKTRACKING-SEARCH(csp) returns a solution or failure
  return BACKTRACK(csp, {})
```

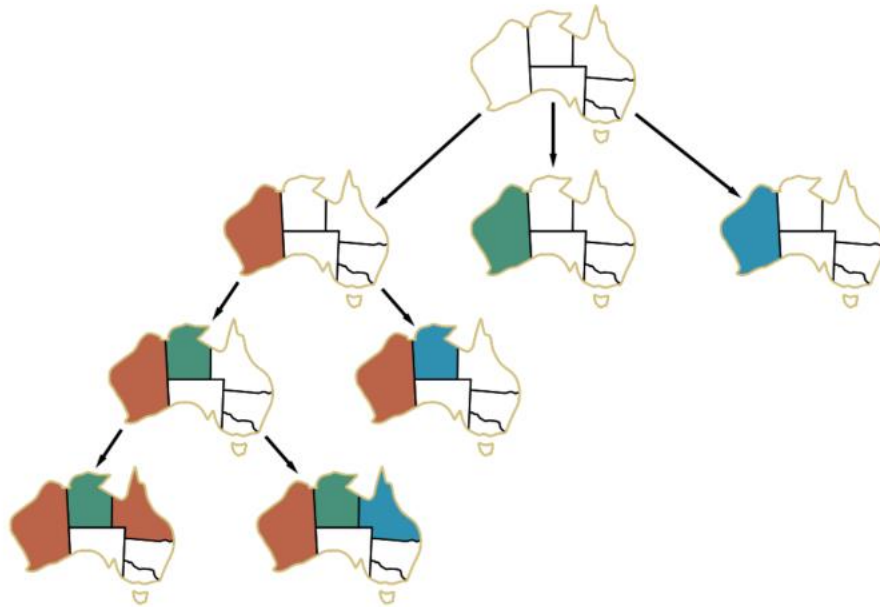
```
function BACKTRACK(csp, assignment) returns a solution or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
  for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
    if value is consistent with assignment then
      add {var = value} to assignment
      inferences ← INFERENCE(csp, var, assignment)
      if inferences ≠ failure then
```

```

add inferences to csp
result ← BACKTRACK(csp, assignment)
if result ≠ failure then return result
remove inferences from csp
remove {var = value} from assignment
return failure

```

Hình 5.5 Một thuật toán quay lui đơn giản cho các bài toán thỏa mãn ràng buộc. Thuật toán được mô hình hóa theo tìm kiếm đệ quy theo chiều sâu của Chương 3. Các hàm **SELECT-UNASSIGNED-VARIABLE** và **ORDER-DOMAIN-VALUES** thực hiện các heuristic đa năng được thảo luận trong Mục 5.3.1. Hàm **INFERENCE** có thể tùy chọn áp đặt tính nhất quán cung, đường đi hoặc k, nếu muốn. Nếu một lựa chọn giá trị dẫn đến thất bại (được nhận thấy bởi **INFERENCE** hoặc **BACKTRACK**), thì các gán giá trị (bao gồm cả những gán giá trị được thực hiện bởi **INFERENCE**) được rút lại và một giá trị mới được thử.



Hình 5.6 Một phần của cây tìm kiếm cho bài toán tô màu bản đồ trong Hình 5.1.

5.3.1 Thứ tự biến và giá trị

Thuật toán quay lui chứa dòng lệnh $var \leftarrow \text{Select-Unassigned-Variable}(csp, assignment)$ Chiến lược đơn giản nhất cho **SELECT-UNASSIGNED-VARIABLE** là sắp xếp tĩnh: chọn các biến theo thứ tự, $\{X_1, X_2, \dots\}$. Chiến lược đơn giản tiếp theo là chọn ngẫu nhiên. Cả hai chiến lược đều không tối ưu. Ví dụ, sau khi gán WA = red và NT = green trong Hình 5.6, chỉ có một giá trị khả dĩ cho SA, vì vậy việc gán SA = blue tiếp theo có ý nghĩa hơn là gán Q. Trên thực tế, sau khi SA được gán, các lựa chọn cho Q, NSW và V đều bị ép buộc.

Ý tưởng trực quan này—chọn biến có ít giá trị “hợp lệ” nhất—được gọi là **heuristic giá trị còn lại tối thiểu (MRV)**. Nó cũng đã được gọi là heuristic “biến bị ràng buộc nhất” hoặc “thất bại trước”, cái sau vì nó chọn một biến có nhiều khả năng gây ra thất bại sớm nhất, do đó cắt tỉa cây

tìm kiếm. Nếu một biến X nào đó không còn giá trị hợp lệ nào, heuristic MRV sẽ chọn X và thất bại sẽ được phát hiện ngay lập tức—tránh các tìm kiếm vô nghĩa qua các biến khác. Heuristic MRV thường hoạt động tốt hơn so với sắp xếp ngẫu nhiên hoặc tĩnh, đôi khi theo cấp số nhân, mặc dù kết quả thay đổi tùy thuộc vào bài toán.

Heuristic MRV không giúp ích gì trong việc chọn vùng đầu tiên để tô màu ở Úc, vì ban đầu mọi vùng đều có ba màu hợp lệ. Trong trường hợp này, **heuristic bậc** trở nên hữu ích. Nó cố gắng giảm hệ số phân nhánh trên các lựa chọn trong tương lai bằng cách chọn biến liên quan đến số lượng ràng buộc lớn nhất đối với các biến chưa được gán khác. Trong Hình 5.1, SA là biến có bậc cao nhất, 5; các biến khác có bậc 2 hoặc 3, ngoại trừ T, có bậc 0. Nếu chúng ta gán SA trước, chúng ta có thể đi vòng quanh năm vùng đất liền theo thứ tự kim đồng hồ hoặc ngược kim đồng hồ và gán cho mỗi vùng một màu khác với SA và khác với vùng trước đó. Heuristic giá trị còn lại tối thiểu thường là một hướng dẫn mạnh hơn, nhưng heuristic bậc có thể hữu ích như một cách để phá vỡ sự hòa.

Khi một biến đã được chọn, thuật toán phải quyết định thứ tự để kiểm tra các giá trị của nó. **Heuristic giá trị ràng buộc ít nhất** có hiệu quả cho việc này. Nó ưu tiên giá trị loại bỏ ít lựa chọn nhất cho các biến lân cận trong đồ thị ràng buộc. Ví dụ, giả sử trong Hình 5.1 chúng ta đã tạo ra gán giá trị một phần với WA = red và NT = green và lựa chọn tiếp theo của chúng ta là Q. Blue sẽ là một lựa chọn tồi vì nó loại bỏ giá trị hợp lệ cuối cùng còn lại cho nút lân cận của Q, SA. Do đó, heuristic giá trị ràng buộc ít nhất ưu tiên màu đỏ hơn màu xanh dương. Nói chung, heuristic đang cố gắng để lại sự linh hoạt tối đa cho các gán giá trị biến tiếp theo.

Tại sao việc chọn biến lại nên thất bại trước, nhưng việc chọn giá trị lại nên thất bại sau? Mọi biến đều phải được gán cuối cùng, vì vậy bằng cách chọn những biến có khả năng thất bại trước, chúng ta sẽ có trung bình ít gán giá trị thành công hơn để quay lui. Đối với việc sắp xếp giá trị, mẹo là chúng ta chỉ cần một nghiệm; do đó, việc tìm kiếm các giá trị có khả năng nhất trước tiên là hợp lý. Nếu chúng ta muốn liệt kê tất cả các nghiệm thay vì chỉ tìm một, thì việc sắp xếp giá trị sẽ không liên quan.

5.3.2 Xen kẽ tìm kiếm và suy luận

Chúng ta đã thấy cách AC-3 có thể giảm miền của các biến trước khi chúng ta bắt đầu tìm kiếm. Nhưng suy luận có thể mạnh mẽ hơn nữa trong quá trình tìm kiếm: mỗi khi chúng ta đưa ra một lựa chọn giá trị cho một biến, chúng ta có một cơ hội hoàn toàn mới để suy ra các lần giảm miền mới trên các biến lân cận.

Một trong những dạng suy luận đơn giản nhất được gọi là **kiểm tra phía trước**. Bất cứ khi nào một biến X được gán, quá trình kiểm tra phía trước sẽ thiết lập tính nhất quán cung cho nó: đối với mỗi biến Y chưa được gán được kết nối với X bằng một ràng buộc, hãy xóa khỏi miền của Y bất kỳ giá trị nào không nhất quán với giá trị được chọn cho X .

Hình 5.7 cho thấy sự tiến triển của tìm kiếm quay lui trên CSP Úc với kiểm tra phía trước. Có hai điểm quan trọng cần lưu ý về ví dụ này. Thứ nhất, lưu ý rằng sau khi WA = red và Q = green được gán, miền của NT và SA bị giảm xuống một giá trị duy nhất; chúng ta đã loại bỏ hoàn toàn việc phân nhánh trên các biến này bằng cách lan truyền thông tin từ WA và Q. Một điểm thứ hai cần lưu ý là sau V = blue, miền của SA là rỗng. Do đó, kiểm tra phía trước đã phát hiện ra rằng gán

giá trị một phần $\{WA = red, Q = green, V = blue\}$ không nhất quán với các ràng buộc của bài toán, và thuật toán quay lui ngay lập tức.

Đối với nhiều bài toán, tìm kiếm sẽ hiệu quả hơn nếu chúng ta kết hợp heuristic MRV với kiểm tra phía trước. Hãy xem xét Hình 5.7 sau khi gán $\{WA=red\}$. Trực giác, có vẻ như gán giá trị đó ràng buộc các nút lân cận của nó, NT và SA, vì vậy chúng ta nên xử lý các biến đó tiếp theo, và sau đó tất cả các biến khác sẽ vào đúng vị trí. Đó chính xác là những gì xảy ra với MRV: NT và SA mỗi biến có hai giá trị, vì vậy một trong số chúng được chọn trước, sau đó là biến kia, sau đó là Q, NSW, và V theo thứ tự. Cuối cùng T vẫn có ba giá trị, và bất kỳ giá trị nào trong số chúng đều hoạt động. Chúng ta có thể xem kiểm tra phía trước như một cách hiệu quả để tính toán tăng dần thông tin mà heuristic MRV cần để thực hiện công việc của nó.

Mặc dù kiểm tra phía trước phát hiện nhiều sự không nhất quán, nhưng nó không phát hiện tất cả. Vấn đề là nó không nhìn đủ xa về phía trước. Ví dụ, hãy xem xét hàng $Q = green$ của Hình 5.7. Chúng ta đã làm cho WA và Q nhất quán cùng, nhưng chúng ta đã để cả NT và SA với blue là giá trị khả dĩ duy nhất của chúng, đây là một sự không nhất quán, vì chúng là các nút lân cận.

Thuật toán được gọi là **MAC** (viết tắt của Maintaining Arc Consistency) phát hiện những sự không nhất quán như thế này. Sau khi một biến X_i được gán một giá trị, thủ tục INFERENCE gọi AC-3, nhưng thay vì một hàng đợi tất cả các cung trong CSP, chúng ta bắt đầu chỉ với các cung (X_j, X_i) cho tất cả X_j là các biến chưa được gán là nút lân cận của X_i . Từ đó, AC-3 thực hiện lan truyền ràng buộc theo cách thông thường, và nếu bất kỳ biến nào có miền của nó bị giảm xuống tập hợp rỗng, lời gọi đến AC-3 sẽ thất bại và chúng ta biết phải quay lui ngay lập tức. Chúng ta có thể thấy rằng MAC mạnh hơn hẳn so với kiểm tra phía trước vì kiểm tra phía trước thực hiện điều tương tự như MAC trên các cung ban đầu trong hàng đợi của MAC; nhưng không giống như MAC, kiểm tra phía trước không lan truyền ràng buộc một cách đệ quy khi các thay đổi được thực hiện cho miền của các biến.

	<i>WA</i>	<i>NT</i>	<i>Q</i>	<i>NSW</i>	<i>V</i>	<i>SA</i>	<i>T</i>	
Initial domains								
After <i>WA</i> =red								
After <i>Q</i> =green								
After <i>V</i> =blue								

Hình 5.7 Sự tiến triển của một tìm kiếm tô màu bản đồ với kiểm tra phía trước. $WA = red$ được gán đầu tiên; sau đó kiểm tra phía trước xóa màu đỏ khỏi miền của các biến lân cận NT và SA. Sau khi $Q = green$ được gán, màu xanh lá cây bị xóa khỏi miền của NT, SA và NSW. Sau khi $V = blue$ được gán, màu xanh dương bị xóa khỏi miền của NSW và SA, để lại SA không có giá trị hợp lệ.

5.3.3 Quay lui thông minh: Nhìn lại phía sau

Thuật toán **BACKTRACKING-SEARCH** trong Hình 5.5 có một chính sách rất đơn giản về những gì cần làm khi một nhánh của tìm kiếm thất bại: quay lại biến trước đó và thử một giá trị

khác cho nó. Điều này được gọi là **quay lui theo thời gian** vì điểm quyết định gần đây nhất được xem xét lại. Trong phần này, chúng ta sẽ xem xét các khả năng tốt hơn.

Hãy xem xét điều gì sẽ xảy ra khi chúng ta áp dụng quay lui đơn giản trong Hình 5.1 với một thứ tự biến cố định Q, NSW, V, T, SA, WA, NT . Giả sử chúng ta đã tạo ra gán giá trị một phần $\{Q = red, NSW = green, V = blue, T = red\}$. Khi chúng ta thử biến tiếp theo, SA , chúng ta thấy rằng mọi giá trị đều vi phạm một ràng buộc. Chúng ta quay lại T và thử một màu mới cho Tasmania! Rõ ràng điều này thật ngớ ngẩn—việc tô màu lại Tasmania không thể giúp giải quyết vấn đề với Nam Úc.

Một cách tiếp cận thông minh hơn là quay lại một biến có thể khắc phục sự cố—một biến chịu trách nhiệm làm cho một trong các giá trị có thể có của SA trở nên bất khả thi. Để làm điều này, chúng ta sẽ theo dõi một tập hợp các gán giá trị mâu thuẫn với một số giá trị của SA . Tập hợp này (trong trường hợp này là $\{Q = red, NSW = green, V = blue\}$), được gọi là **tập hợp mâu thuẫn** cho SA . Phương pháp **nhảy lùi** quay lại gán giá trị gần đây nhất trong tập hợp mâu thuẫn; trong trường hợp này, nhảy lùi sẽ bỏ qua Tasmania và thử một giá trị mới cho V . Phương pháp này được triển khai dễ dàng bằng cách sửa đổi **BACKTRACK** sao cho nó tích lũy tập hợp mâu thuẫn trong khi kiểm tra một giá trị hợp lệ để gán. Nếu không tìm thấy giá trị hợp lệ nào, thuật toán sẽ trả về phần tử gần đây nhất của tập hợp mâu thuẫn cùng với chỉ báo thất bại.

Người đọc tinh mắt có thể đã nhận thấy rằng kiểm tra phía trước có thể cung cấp tập hợp mâu thuẫn mà không cần thêm công việc: bất cứ khi nào kiểm tra phía trước dựa trên gán giá trị $X = x$ xóa một giá trị khỏi miền của Y , nó sẽ thêm $X = x$ vào tập hợp mâu thuẫn của Y . Nếu giá trị cuối cùng bị xóa khỏi miền của Y , thì các gán giá trị trong tập hợp mâu thuẫn của Y được thêm vào tập hợp mâu thuẫn của X . Tức là, bây giờ chúng ta biết rằng $X = x$ dẫn đến mâu thuẫn (trong Y), và do đó nên thử một gán giá trị khác cho X .

Người đọc tinh tường hơn có thể đã nhận thấy một điều kỳ lạ: nhảy lùi xảy ra khi mọi giá trị trong một miền đều mâu thuẫn với gán giá trị hiện tại; nhưng kiểm tra phía trước phát hiện sự kiện này và ngăn không cho tìm kiếm đi đến một nút như vậy! Trên thực tế, có thể chỉ ra rằng mọi nhánh bị cắt tỉa bởi nhảy lùi cũng bị cắt tỉa bởi kiểm tra phía trước. Do đó, nhảy lùi đơn giản là dư thừa trong một tìm kiếm kiểm tra phía trước hoặc, thực sự, trong một tìm kiếm sử dụng kiểm tra tính nhất quán mạnh hơn, chẳng hạn như MAC—bạn chỉ cần thực hiện một trong hai.

Mặc dù có những quan sát của đoạn trước, ý tưởng đằng sau nhảy lùi vẫn là một ý tưởng hay: quay lui dựa trên nguyên nhân thất bại. Nhảy lùi nhận thấy thất bại khi miền của một biến trở nên rỗng, nhưng trong nhiều trường hợp, một nhánh đã thất bại từ lâu trước khi điều này xảy ra. Hãy xem xét lại gán giá trị một phần $\{WA = red, NSW = red\}$ (mà, từ cuộc thảo luận trước của chúng ta, là không nhất quán). Giả sử chúng ta thử $T = red$ tiếp theo và sau đó gán NT, Q, V, SA . Chúng ta biết rằng không có gán giá trị nào có thể hoạt động cho bốn biến cuối cùng này, vì vậy cuối cùng chúng ta hết các giá trị để thử tại NT . Bây giờ, câu hỏi là, quay lui về đâu? Nhảy lùi không thể hoạt động, bởi vì NT có các giá trị nhất quán với các biến được gán trước đó— NT không có một tập hợp mâu thuẫn đầy đủ của các biến trước đó đã khiến nó thất bại. Tuy nhiên, chúng ta biết rằng bốn biến NT, Q, V , và SA , được gộp lại, đã thất bại vì một tập hợp các biến trước đó, phải là những biến mâu thuẫn trực tiếp với bốn biến đó.

Điều này dẫn đến một khái niệm khác—và sâu sắc hơn—về tập hợp mâu thuẫn cho một biến như NT : đó là tập hợp các biến trước đó đã khiến NT , cùng với bất kỳ biến tiếp theo nào, không có

nghiệm nhất quán. Trong trường hợp này, tập hợp là WA và NSW, vì vậy thuật toán nên quay lui đến NSW và bỏ qua Tasmania. Một thuật toán nhảy lùi sử dụng các tập hợp mâu thuẫn được định nghĩa theo cách này được gọi là **nhảy lùi theo mâu thuẫn**.

Bây giờ chúng ta phải giải thích cách các tập hợp mâu thuẫn mới này được tính toán. Phương pháp này trên thực tế khá đơn giản. Sự thất bại “cuối cùng” của một nhánh tìm kiếm luôn xảy ra vì miền của một biến trở nên rỗng; biến đó có một tập hợp mâu thuẫn tiêu chuẩn. Trong ví dụ của chúng ta, SA thất bại, và tập hợp mâu thuẫn của nó là (ví dụ) {WA, NT, Q}. Chúng ta nhảy lùi đến Q, và Q hấp thụ tập hợp mâu thuẫn từ SA (trừ chính Q, tất nhiên) vào tập hợp mâu thuẫn trực tiếp của nó, là {NT, NSW}; tập hợp mâu thuẫn mới là {WA, NT, NSW}. Tức là, không có nghiệm nào từ Q trở đi, với gán giá trị trước đó cho {WA, NT, NSW}. Do đó, chúng ta quay lui đến NT, biến gần đây nhất trong số này. NT hấp thụ {WA, NT, NSW} - {NT} vào tập hợp mâu thuẫn trực tiếp của nó {WA}, cho ra {WA, NSW} (như đã nêu trong đoạn trước). Bây giờ thuật toán nhảy lùi đến NSW, như chúng ta mong đợi. Tóm lại: cho X_j là biến hiện tại, và cho $\text{conf}(X_j)$ là tập hợp mâu thuẫn của nó. Nếu mọi giá trị có thể có cho X_j đều thất bại, hãy nhảy lùi đến biến gần đây nhất X_i trong $\text{conf}(X_j)$ và tính toán lại tập hợp mâu thuẫn cho X_i như sau:

$$\text{conf}(X_i) \leftarrow \text{conf}(X_i) \cup \text{conf}(X_j) - \{X_j\}.$$

5.3.4 Học ràng buộc

Khi chúng ta đi đến một mâu thuẫn, nhảy lùi có thể cho chúng ta biết phải quay lại bao xa, vì vậy chúng ta không lãng phí thời gian thay đổi các biến sẽ không khắc phục được vấn đề. Nhưng chúng ta cũng muốn tránh gặp phải cùng một vấn đề một lần nữa. Khi tìm kiếm đi đến một mâu thuẫn, chúng ta biết rằng một số tập hợp con của tập hợp mâu thuẫn chịu trách nhiệm cho vấn đề. **Học ràng buộc** là ý tưởng tìm một tập hợp tối thiểu các biến từ tập hợp mâu thuẫn gây ra vấn đề. Tập hợp các biến này, cùng với các giá trị tương ứng của chúng, được gọi là một **no-good**. Sau đó chúng ta ghi lại no-good, bằng cách thêm một ràng buộc mới vào CSP để cấm tổ hợp các gán giá trị này hoặc bằng cách giữ một bộ nhớ đệm riêng biệt của các no-good.

Ví dụ, hãy xem xét trạng thái $\{WA=\text{red}, NT=\text{green}, Q=\text{blue}\}$ ở hàng dưới cùng của Hình 5.6. Kiểm tra phía trước có thể cho chúng ta biết trạng thái này là một no-good vì không có gán giá trị hợp lệ nào cho SA. Trong trường hợp cụ thể này, việc ghi lại no-good sẽ không giúp ích gì, bởi vì một khi chúng ta cắt tia nhánh này khỏi cây tìm kiếm, chúng ta sẽ không bao giờ gặp lại tổ hợp này nữa. Nhưng giả sử rằng cây tìm kiếm trong Hình 5.6 thực sự là một phần của một cây tìm kiếm lớn hơn bắt đầu bằng cách gán giá trị cho V và T trước. Khi đó, việc ghi lại $\{WA=\text{red}, NT=\text{green}, Q=\text{blue}\}$ dưới dạng một no-good sẽ rất hữu ích vì chúng ta sẽ gặp lại cùng một vấn đề cho mỗi tập hợp gán giá trị có thể có cho V và T.

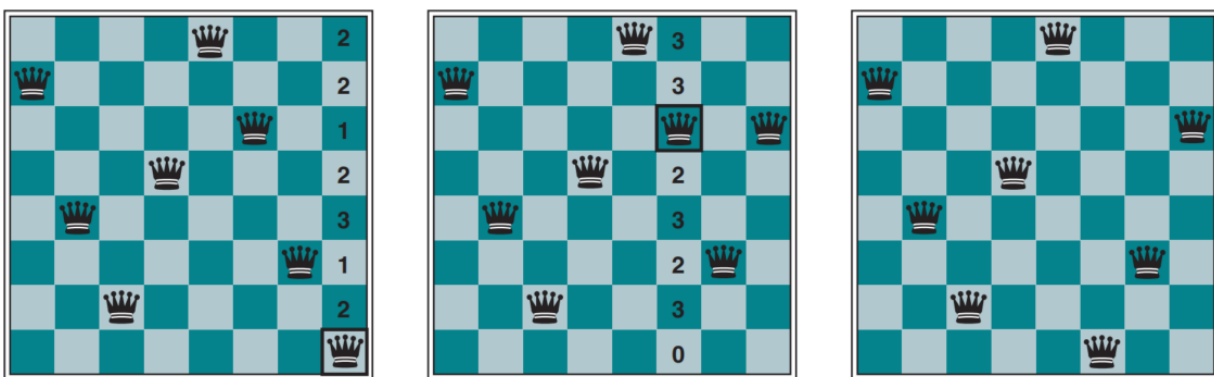
Các no-good có thể được sử dụng hiệu quả bằng cách kiểm tra phía trước hoặc nhảy lùi. Học ràng buộc là một trong những kỹ thuật quan trọng nhất được sử dụng bởi các bộ giải CSP hiện đại để đạt được hiệu quả trên các bài toán phức tạp.

5.4 Tìm kiếm cục bộ cho CSP

Các thuật toán **tìm kiếm cục bộ** (xem Mục 4.1) tỏ ra rất hiệu quả trong việc giải quyết nhiều CSP. Chúng sử dụng một mô hình trạng thái hoàn chỉnh (như đã giới thiệu trong Mục 4.1.1) trong đó mỗi trạng thái gán một giá trị cho mọi biến, và tìm kiếm thay đổi giá trị của một biến tại một thời điểm.

Lấy ví dụ, chúng ta sẽ sử dụng bài toán 8 con hậu, được định nghĩa là một CSP ở trang 167. Trong Hình 5.8, chúng ta bắt đầu ở bên trái với một gán giá trị hoàn chỉnh cho 8 biến; thông thường điều này sẽ vi phạm một số ràng buộc. Sau đó, chúng ta chọn ngẫu nhiên một biến bị mâu thuẫn, đó là Q_8 , cột ngoài cùng bên phải. Chúng ta muốn thay đổi giá trị thành một giá trị nào đó đưa chúng ta đến gần hơn với một giải pháp; cách tiếp cận rõ ràng nhất là chọn giá trị dẫn đến số lượng mâu thuẫn tối thiểu với các biến khác—**heuristic mâu thuẫn tối thiểu**.

Trong hình, chúng ta thấy có hai hàng chỉ vi phạm một ràng buộc; chúng ta chọn $Q_8 = 3$ (tức là chúng ta di chuyển con hậu đến cột thứ 8, hàng thứ 3). Ở lần lặp tiếp theo, trong bảng ở giữa của hình, chúng ta chọn Q_6 làm biến để thay đổi, và lưu ý rằng việc di chuyển con hậu đến hàng thứ 8 không gây ra mâu thuẫn nào. Tại thời điểm này, không còn biến nào bị mâu thuẫn nữa, vì vậy chúng ta có một giải pháp. Thuật toán được hiển thị trong Hình 5.9.



Hình 5.8 Một giải pháp hai bước sử dụng mâu thuẫn tối thiểu cho bài toán 8 con hậu. Ở mỗi giai đoạn, một con hậu được chọn để gán lại trong cột của nó. Số lượng mâu thuẫn (trong trường hợp này, số lượng con hậu tấn công) được hiển thị trong mỗi ô vuông. Thuật toán di chuyển con hậu đến ô vuông có mâu thuẫn tối thiểu, phá vỡ các trường hợp hòa một cách ngẫu nhiên.

```
function MIN-CONFLICTS(csp, max_steps) returns a solution or failure
inputs: csp, a constraint satisfaction problem
       max_steps, the number of steps allowed before giving up
current ← an initial complete assignment for csp
for i = 1 to max_steps do
  if current is a solution for csp then return current
  var ← a randomly chosen conflicted variable from csp.VARIABLES
  value ← the value v for var that minimizes CONFLICTS(csp, var, v, current)
  set var = value in current
return failure
```

Hình 5.9 Thuật toán tìm kiếm cục bộ **MIN-CONFLICTS** cho CSP. Trạng thái ban đầu có thể được chọn ngẫu nhiên hoặc bằng một quá trình gán tham lam chọn một giá trị mâu thuẫn tối thiểu cho mỗi biến lần lượt. Hàm **CONFLICTS** đếm số lượng ràng buộc bị vi phạm bởi một giá trị cụ thể, với phần còn lại của gán giá trị hiện tại.

Heuristic mâu thuẫn tối thiểu hiệu quả đáng ngạc nhiên đối với nhiều CSP. Điều đáng kinh ngạc là, trên bài toán n -queens, nếu bạn không tính vị trí ban đầu của các con hậu, thời gian chạy của mâu thuẫn tối thiểu gần như độc lập với kích thước bài toán. Nó giải quyết ngay cả bài toán triệu con hậu trong trung bình 50 bước (sau gán giá trị ban đầu). Quan sát đáng chú ý này là động lực dẫn đến rất nhiều nghiên cứu trong những năm 1990 về tìm kiếm cục bộ và sự khác biệt giữa các bài toán dễ và khó, mà chúng ta sẽ đề cập trong Mục 7.6.3. Nói một cách đơn giản, n -queens là dễ đối với tìm kiếm cục bộ vì các nghiệm được phân bố dày đặc trong không gian trạng thái. Mâu thuẫn tối thiểu cũng hoạt động tốt cho các bài toán khó. Ví dụ, nó đã được sử dụng để lập lịch các quan sát cho Kính viễn vọng không gian Hubble, giảm thời gian cần thiết để lập lịch một tuần quan sát từ ba tuần (!) xuống còn khoảng 10 phút.

Tất cả các kỹ thuật tìm kiếm cục bộ từ Mục 4.1 đều là ứng cử viên để áp dụng cho các CSP, và một số trong số đó đã được chứng minh là đặc biệt hiệu quả. Cảnh quan của một CSP dưới heuristic mâu thuẫn tối thiểu thường có một loạt các cao nguyên. Có thể có hàng triệu gán giá trị biến chỉ cách một mâu thuẫn so với một nghiệm. **Tìm kiếm cao nguyên**—cho phép di chuyển sang một bên đến một trạng thái khác có cùng điểm số—có thể giúp tìm kiếm cục bộ tìm đường ra khỏi cao nguyên này. Sự lang thang trên cao nguyên này có thể được định hướng bằng một kỹ thuật gọi là **tìm kiếm cấm**: giữ một danh sách nhỏ các trạng thái đã truy cập gần đây và cấm thuật toán quay lại các trạng thái đó. **Mô phỏng luyện kim** cũng có thể được sử dụng để thoát khỏi các cao nguyên.

Một kỹ thuật khác được gọi là **trọng số ràng buộc** nhằm mục đích tập trung tìm kiếm vào các ràng buộc quan trọng. Mỗi ràng buộc được gán một trọng số bằng số, ban đầu tất cả đều là 1. Ở mỗi bước của tìm kiếm, thuật toán chọn một cặp biến/giá trị để thay đổi sẽ dẫn đến tổng trọng số thấp nhất của tất cả các ràng buộc bị vi phạm. Sau đó, các trọng số được điều chỉnh bằng cách tăng trọng số của mỗi ràng buộc bị vi phạm bởi gán giá trị hiện tại. Điều này có hai lợi ích: nó thêm địa hình cho các cao nguyên, đảm bảo rằng có thể cải thiện từ trạng thái hiện tại, và nó cũng thêm việc học: theo thời gian, các ràng buộc khó được gán trọng số cao hơn. Một lợi thế khác của tìm kiếm cục bộ là nó có thể được sử dụng trong một cài đặt trực tuyến (xem Mục 4.5) khi bài toán thay đổi. Hãy xem xét một bài toán lập lịch cho các chuyến bay hàng tuần của một hãng hàng không. Lịch trình có thể liên quan đến hàng nghìn chuyến bay và hàng chục nghìn gán giá trị nhân sự, nhưng thời tiết xấu ở một sân bay có thể làm cho lịch trình không khả thi. Chúng ta muốn sửa chữa lịch trình với số lượng thay đổi tối thiểu. Điều này có thể được thực hiện dễ dàng với một thuật toán tìm kiếm cục bộ bắt đầu từ lịch trình hiện tại. Một tìm kiếm quay lui với tập hợp các ràng buộc mới thường yêu cầu nhiều thời gian hơn và có thể tìm thấy một giải pháp với nhiều thay đổi so với lịch trình hiện tại.

5.5 Cấu trúc của các bài toán

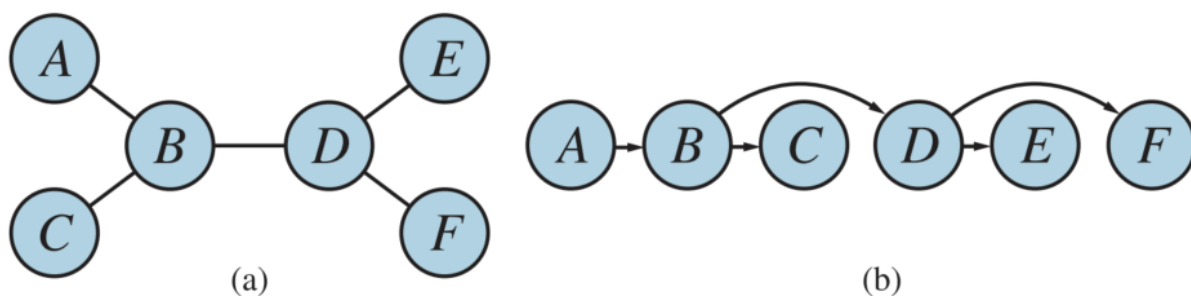
Trong phần này, chúng ta sẽ xem xét các cách mà cấu trúc của bài toán, được biểu diễn bởi đồ thị ràng buộc, có thể được sử dụng để tìm giải pháp một cách nhanh chóng. Hầu hết các cách tiếp cận ở đây cũng áp dụng cho các bài toán khác ngoài CSP, chẳng hạn như suy luận xác suất.

Cách duy nhất chúng ta có thể hy vọng giải quyết thế giới thực rộng lớn là phân tách nó thành các **bài toán con**. Nhìn lại đồ thị ràng buộc cho Úc (Hình 5.1(b)), lặp lại là Hình 5.12(a)), một sự thật nổi bật: Tasmania không được kết nối với đất liền. Trực giác, rõ ràng là việc tô màu Tasmania và tô màu đất liền là hai **bài toán con độc lập**—bất kỳ giải pháp nào cho đất liền kết hợp với bất kỳ giải pháp nào cho Tasmania đều tạo ra một giải pháp cho toàn bộ bản đồ.

Sự độc lập có thể được xác định đơn giản bằng cách tìm các **thành phần liên thông** của đồ thị ràng buộc. Mỗi thành phần tương ứng với một bài toán con CSP_i . Nếu gán giá trị S_i là một giải pháp của CSP_i , thì $S_i \text{ cup } S_j$ là một giải pháp của $CSP_i \text{ cup } CSP_j$. Tại sao điều này lại quan trọng? Giả sử mỗi CSP_i có c biến từ tổng số n biến, trong đó c là một hằng số. Khi đó có n/c bài toán con, mỗi bài toán mất nhiều nhất d^c công việc để giải quyết, trong đó d là kích thước của miền. Do đó, tổng công việc là $O(d^c n/c)$, tuyến tính theo n ; nếu không có sự phân tách, tổng công việc là $O(d^n)$, hàm mũ theo n . Hãy làm cho điều này cụ thể hơn: chia một CSP Boolean với 100 biến thành bốn bài toán con làm giảm thời gian giải quyết trường hợp xấu nhất từ khoảng thời gian của vũ trụ xuống còn chưa đầy một giây.

Các bài toán con hoàn toàn độc lập rất hấp dẫn, nhưng hiếm. May mắn thay, một số cấu trúc đồ thị khác cũng dễ giải quyết. Ví dụ, một đồ thị ràng buộc là một **cây** khi bất kỳ hai biến nào cũng được kết nối bởi chỉ một đường đi. Chúng ta sẽ chỉ ra rằng bất kỳ CSP có cấu trúc cây nào cũng có thể được giải quyết trong thời gian tuyến tính theo số lượng biến. Chìa khóa là một khái niệm mới về **tính nhất quán**, được gọi là **tính nhất quán cung có hướng** hoặc **DAC**. Một CSP được định nghĩa là nhất quán cung có hướng dưới một thứ tự các biến $X_1, X_2, \dots, X_n$ nếu và chỉ nếu mọi X_i là nhất quán cung với mỗi X_j với $j < i$.

Để giải một CSP có cấu trúc cây, trước tiên hãy chọn bất kỳ biến nào làm gốc của cây, và chọn một thứ tự các biến sao cho mỗi biến xuất hiện sau cha của nó trong cây. Một thứ tự như vậy được gọi là một **sắp xếp tô pô**. Hình 5.10(a) cho thấy một cây mẫu và (b) cho thấy một thứ tự có thể có. Bất kỳ cây nào có n nút đều có $n - 1$ cạnh, vì vậy chúng ta có thể làm cho đồ thị này nhất quán cung có hướng trong $O(n)$ bước, mỗi bước phải so sánh tối đa d^2 giá trị miền có thể có cho hai biến, với tổng thời gian là $O(nd^2)$. Một khi chúng ta có một đồ thị nhất quán cung có hướng, chúng ta có thể chỉ cần duyệt qua danh sách các biến và chọn bất kỳ giá trị nào còn lại. Vì mỗi cạnh từ một nút cha đến con của nó là nhất quán cung, chúng ta biết rằng đối với bất kỳ giá trị nào chúng ta chọn cho nút cha, sẽ có một giá trị hợp lệ còn lại để chọn cho nút con. Điều đó có nghĩa là chúng ta sẽ không phải quay lui; chúng ta có thể di chuyển tuyến tính qua các biến. Thuật toán hoàn chỉnh được hiển thị trong Hình 5.11.



Hình 5.10 (a) Đồ thị ràng buộc của một CSP có cấu trúc cây. (b) Một thứ tự tuyến tính của các biến nhất quán với cây với A là gốc. Điều này được gọi là một sắp xếp tô pô của các biến.

```
function TREE-CSP-SOLVER(csp) returns a solution, or failure
inputs: csp, a CSP with components X, D, C
n ← number of variables in X
```

```

assignment ← an empty assignment
root ← any variable in X
X ← TopologicalSort(X, root)
for j = n down to 2 do
    MAKE-ARC-CONSISTENT(PARENT(Xj), Xj)
    if it cannot be made consistent then return failure
for i = 1 to n do
    assignment[Xi] ← any consistent value from Di
    if there is no consistent value then return failure
return assignment

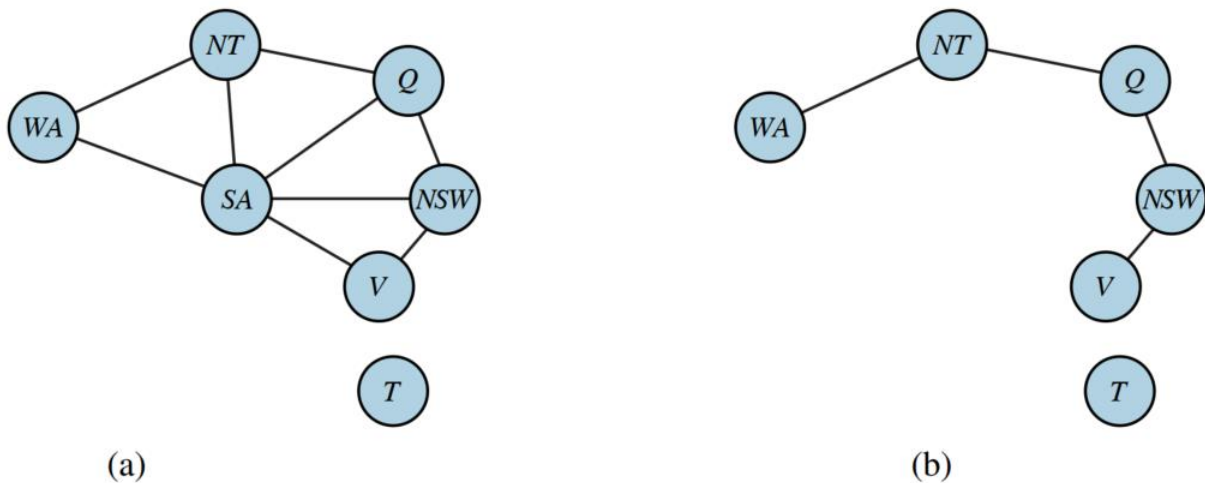
```

Hình 5.11 Thuật toán **TREE-CSP-SOLVER** để giải quyết các CSP có cấu trúc cây. Nếu CSP có một giải pháp, chúng ta sẽ tìm thấy nó trong thời gian tuyến tính; nếu không, chúng ta sẽ phát hiện một mâu thuẫn.

Bây giờ chúng ta đã có một thuật toán hiệu quả cho cây, chúng ta có thể xem xét liệu các đồ thị ràng buộc tổng quát hơn có thể được giảm xuống thành cây bằng cách nào đó hay không. Có hai cách để làm điều này: bằng cách loại bỏ các nút (Mục 5.5.1) hoặc bằng cách gộp các nút lại với nhau (Mục 5.5.2).

5.5.1 Điều kiện tập cắt

Cách đầu tiên để giảm một đồ thị ràng buộc thành một cây liên quan đến việc gán giá trị cho một số biến để các biến còn lại tạo thành một cây. Hãy xem xét đồ thị ràng buộc cho Úc, được hiển thị lại trong Hình 5.12(a). Nếu không có Nam Úc, đồ thị sẽ trở thành một cây, như trong (b). May mắn thay, chúng ta có thể xóa Nam Úc (trong đồ thị, không phải quốc gia) bằng cách cố định một giá trị cho SA và xóa khỏi miền của các biến khác bất kỳ giá trị nào không nhất quán với giá trị được chọn cho SA.



Hình 5.12 (a) Đồ thị ràng buộc gốc từ Hình 5.1. (b) Sau khi loại bỏ SA, đồ thị ràng buộc trở thành một rừng gồm hai cây.

Bây giờ, bất kỳ giải pháp nào cho CSP sau khi SA và các ràng buộc của nó được loại bỏ sẽ nhất quán với giá trị được chọn cho SA. (Điều này hoạt động cho các CSP hai ngôi; tình huống phức

tập hơn với các ràng buộc bậc cao hơn.) Do đó, chúng ta có thể giải cây còn lại bằng thuật toán được đưa ra ở trên và do đó giải quyết toàn bộ bài toán. Tất nhiên, trong trường hợp tổng quát (trái ngược với tô màu bản đồ), giá trị được chọn cho SA có thể là sai, vì vậy chúng ta sẽ cân thử từng giá trị có thể có. Thuật toán tổng quát như sau:

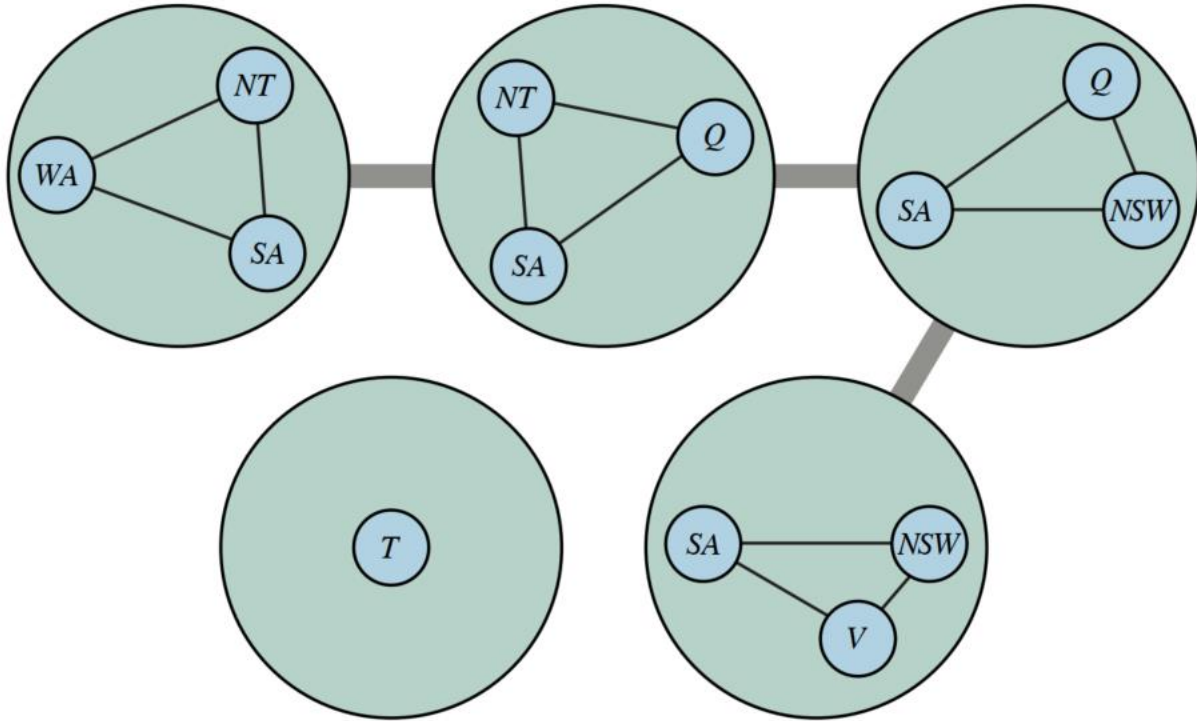
1. Chọn một tập hợp con S của các biến của CSP sao cho đồ thị ràng buộc trở thành một cây sau khi loại bỏ S . S được gọi là một **tập cắt chu trình**.
2. Đối với mỗi gán giá trị có thể có cho các biến trong S thỏa mãn tất cả các ràng buộc trên S , a) xóa khỏi miền của các biến còn lại bất kỳ giá trị nào không nhất quán với gán giá trị cho S , và b) nếu CSP còn lại có một giải pháp, hãy trả về nó cùng với gán giá trị cho S .

Nếu tập cắt chu trình có kích thước c , thì tổng thời gian chạy là $O(d^c \cdot (n - c)d^2)$: chúng ta phải thử từng trong số d^c tổ hợp giá trị cho các biến trong S , và đối với mỗi tổ hợp chúng ta phải giải một bài toán cây có kích thước $n - c$. Nếu đồ thị “gần như là một cây,” thì c sẽ nhỏ và việc tiết kiệm so với quay lui thẳng sẽ rất lớn—đối với ví dụ 100 biến Boolean của chúng ta, nếu chúng ta có thể tìm thấy một tập cắt có kích thước $c = 20$, điều này sẽ đưa chúng ta xuống từ khoảng thời gian của Vũ trụ xuống còn vài phút. Tuy nhiên, trong trường hợp xấu nhất, c có thể lớn tới $(n - 2)$. Việc tìm kiếm tập cắt chu trình nhỏ nhất là **NP-hard**, nhưng một số thuật toán xấp xỉ hiệu quả đã được biết đến. Cách tiếp cận thuật toán tổng thể được gọi là **điều kiện tập cắt**; nó xuất hiện trở lại trong Chương 13, nơi nó được sử dụng để suy luận về xác suất.

5.5.2 Phân rã cây

Cách thứ hai để giảm đồ thị ràng buộc thành một cây dựa trên việc xây dựng một **phân rã cây** của đồ thị ràng buộc: một phép biến đổi đồ thị gốc thành một cây trong đó mỗi nút trong cây bao gồm một tập hợp các biến, như trong Hình 5.13. Một phân rã cây phải thỏa mãn ba yêu cầu này:

- Mọi biến trong bài toán gốc xuất hiện trong ít nhất một trong các nút cây.
- Nếu hai biến được kết nối bởi một ràng buộc trong bài toán gốc, chúng phải xuất hiện cùng nhau (cùng với ràng buộc) trong ít nhất một trong các nút cây.
- Nếu một biến xuất hiện trong hai nút trong cây, nó phải xuất hiện trong mọi nút dọc theo đường đi kết nối các nút đó.



Hình 5.13 Một phân rã cây của đồ thị ràng buộc trong Hình 5.12(a).

Hai điều kiện đầu tiên đảm bảo rằng tất cả các biến và ràng buộc được biểu diễn trong phân rã cây. Điều kiện thứ ba có vẻ khá kỹ thuật, nhưng cho phép chúng ta nói rằng bất kỳ biến nào từ bài toán gốc phải có cùng giá trị ở bất cứ nơi nào nó xuất hiện: các ràng buộc trong cây nói rằng một biến trong một nút của cây phải có cùng giá trị với biến tương ứng trong nút liền kề trong cây. Ví dụ, SA xuất hiện trong cả bốn nút được kết nối trong Hình 5.13, vì vậy mỗi cạnh trong phân rã cây do đó bao gồm ràng buộc rằng giá trị của SA trong một nút phải giống với giá trị của SA trong nút tiếp theo. Bạn có thể xác minh từ Hình 5.12 rằng phân rã này có ý nghĩa.

Một khi chúng ta có một đồ thị có cấu trúc cây, chúng ta có thể áp dụng **TREE-CSP-SOLVER** để có được một giải pháp trong thời gian $O(nd^2)$, trong đó n là số nút cây và d là kích thước của miền lớn nhất. Nhưng lưu ý rằng trong cây, một miền là một tập hợp các bộ giá trị, không chỉ là các giá trị riêng lẻ.

Ví dụ, nút trên cùng bên trái trong Hình 5.13 đại diện cho, ở cấp độ của bài toán gốc, một bài toán con với các biến $\{WA, NT, SA\}$, miền $\{\text{red, green, blue}\}$ và các ràng buộc $WA \neq NT$, $SA \neq NT$, $WA \neq SA$. Ở cấp độ của cây, nút đại diện cho một biến duy nhất, mà chúng ta có thể gọi là SANTWA, có giá trị phải là một bộ ba màu, chẳng hạn như (red, green, blue), nhưng không phải (red, red, blue), vì điều đó sẽ vi phạm ràng buộc $SA \neq NT$ từ bài toán gốc. Sau đó, chúng ta có thể di chuyển từ nút đó đến nút liền kề, với biến mà chúng ta có thể gọi là SANTQ, và thấy rằng chỉ có một bộ, (red, green, blue), nhất quán với lựa chọn cho SANTWA. Chính xác quá trình tương tự được lặp lại cho hai nút tiếp theo, và độc lập chúng ta có thể đưa ra bất kỳ lựa chọn nào cho T.

Chúng ta có thể giải quyết bất kỳ bài toán phân rã cây nào trong thời gian $O(nd^2)$ với **TREE-CSP-SOLVER**, sẽ hiệu quả miễn là d vẫn nhỏ. Quay trở lại ví dụ của chúng ta với 100 biến

Boolean, nếu mỗi nút có 10 biến, thì $d = 2^{10}$ và chúng ta sẽ có thể giải quyết bài toán trong vài giây. Nhưng nếu có một nút có 30 biến, sẽ mất hàng thế kỷ.

Một đồ thị cho trước chấp nhận nhiều phân rã cây; trong việc chọn một phân rã, mục tiêu là làm cho các bài toán con càng nhỏ càng tốt. (Đặt tất cả các biến vào một nút về mặt kỹ thuật là một cây, nhưng không hữu ích.) **Độ rộng cây** của một phân rã cây của một đồ thị là nhỏ hơn một so với kích thước của nút lớn nhất; độ rộng cây của chính đồ thị được định nghĩa là độ rộng tối thiểu trong số tất cả các phân rã cây của nó. Nếu một đồ thị có độ rộng cây w thì bài toán có thể được giải quyết trong thời gian $O(nd^{w+1})$ với phân rã cây tương ứng. Do đó, các CSP với đồ thị ràng buộc có độ rộng cây bị chặn có thể giải được trong thời gian đa thức. Thật không may, việc tìm kiếm phân rã với độ rộng cây tối thiểu là **NP-hard**, nhưng có các phương pháp heuristic hoạt động tốt trong thực tế. Cách nào tốt hơn: phân rã tập cắt với thời gian $O(d^c \cdot (n - c)d^2)$, hay phân rã cây với thời gian $O(nd^{w+1})$? Bất cứ khi nào bạn có một tập cắt chu trình có kích thước c , cũng có một độ rộng cây có kích thước $w < c + 1$, và nó có thể nhỏ hơn nhiều trong một số trường hợp. Vì vậy, xem xét thời gian ủng hộ phân rã cây, nhưng lợi thế của cách tiếp cận tập cắt chu trình là nó có thể được thực thi trong bộ nhớ tuyến tính, trong khi phân rã cây yêu cầu bộ nhớ hàm mũ theo w .

5.5.3 Đối xứng giá trị

Cho đến nay, chúng ta đã xem xét cấu trúc của đồ thị ràng buộc. Cũng có thể có cấu trúc quan trọng trong các giá trị của biến, hoặc trong cấu trúc của chính các quan hệ ràng buộc. Hãy xem xét bài toán tô màu bản đồ với d màu. Đối với mỗi giải pháp nhất quán, thực sự có một tập hợp $d!$ giải pháp được hình thành bằng cách hoán vị tên màu. Ví dụ, trên bản đồ Úc, chúng ta biết rằng WA, NT và SA phải có các màu khác nhau, nhưng có $3! = 6$ cách để gán ba màu cho ba vùng. Điều này được gọi là **đối xứng giá trị**. Chúng ta muốn giảm không gian tìm kiếm bằng một thừa số $d!$ bằng cách phá vỡ tính đối xứng trong các gán giá trị.

Chúng ta làm điều này bằng cách giới thiệu một **ràng buộc phá vỡ đối xứng**. Đối với ví dụ của chúng ta, chúng ta có thể áp đặt một ràng buộc thứ tự tùy ý, $NT < SA < WA$, yêu cầu ba giá trị phải theo thứ tự bằng chữ cái. Ràng buộc này đảm bảo rằng chỉ có một trong số $d!$ giải pháp là có thể: $\{NT = \text{blue}, SA = \text{green}, WA = \text{red}\}$.

Đối với tô màu bản đồ, việc tìm một ràng buộc loại bỏ đối xứng là dễ dàng. Nói chung, việc loại bỏ tất cả đối xứng là **NP-hard**, nhưng phá vỡ đối xứng giá trị đã được chứng minh là quan trọng và hiệu quả trên nhiều bài toán.

Tóm tắt

- Các bài toán thỏa mãn ràng buộc (**CSP**) biểu diễn một trạng thái bằng một tập hợp các cặp biến/giá trị và biểu diễn các điều kiện cho một giải pháp bằng một tập hợp các ràng buộc trên các biến. Nhiều bài toán quan trọng trong thế giới thực có thể được mô tả như các CSP.
- Một số kỹ thuật suy luận sử dụng các ràng buộc để loại trừ các gán giá trị biến nhất định. Chúng bao gồm tính nhất quán nút, cung, đường đi và k .

- Tìm kiếm quay lui, một dạng của tìm kiếm theo chiều sâu, thường được sử dụng để giải quyết các CSP. Suy luận có thể được đan xen với tìm kiếm.
- Các heuristic giá trị còn lại tối thiểu và bậc là các phương pháp độc lập với miền để quyết định chọn biến nào tiếp theo trong một tìm kiếm quay lui. Heuristic giá trị ràng buộc ít nhất giúp quyết định giá trị nào nên thử đầu tiên cho một biến đã cho. Quay lui xảy ra khi không tìm thấy gán giá trị hợp lệ cho một biến. **Nhảy lùi theo mâu thuẫn** quay lui trực tiếp đến nguồn gốc của vấn đề. **Học ràng buộc** ghi lại các mâu thuẫn khi chúng được gặp phải trong quá trình tìm kiếm để tránh cùng một mâu thuẫn sau này trong tìm kiếm.
- Tìm kiếm cục bộ sử dụng heuristic mâu thuẫn tối thiểu cũng đã được áp dụng cho các bài toán thỏa mãn ràng buộc với thành công lớn.
- Độ phức tạp của việc giải một CSP liên quan chặt chẽ đến cấu trúc của đồ thị ràng buộc của nó. Các bài toán có cấu trúc cây có thể được giải quyết trong thời gian tuyến tính. **Điều kiện tập cắt** có thể giảm một CSP tổng quát thành một CSP có cấu trúc cây và khá hiệu quả (chỉ yêu cầu bộ nhớ tuyến tính) nếu tìm thấy một tập cắt nhỏ. Các kỹ thuật **phân rã cây** biến đổi CSP thành một cây các bài toán con và hiệu quả nếu độ rộng cây của đồ thị ràng buộc là nhỏ; tuy nhiên chúng cần bộ nhớ hàm mũ theo độ rộng cây của đồ thị ràng buộc. Kết hợp điều kiện tập cắt với phân rã cây có thể cho phép một sự đánh đổi tốt hơn giữa bộ nhớ và thời gian.

Ghi chú thư mục và lịch sử

Nhà toán học Hy Lạp **Diophantus** (k. 200–284) đã trình bày và giải quyết các bài toán liên quan đến các ràng buộc đại số trên các phương trình, mặc dù ông không phát triển một phương pháp luận tổng quát. Chúng ta bây giờ gọi các phương trình trên các miền số nguyên là **phương trình Diophantine**. Nhà toán học Ấn Độ **Brahmagupta** (k. 650) là người đầu tiên chỉ ra một giải pháp tổng quát trên miền số nguyên cho phương trình $ax + by = c$. Các phương pháp có hệ thống để giải các phương trình tuyến tính bằng cách loại bỏ biến đã được **Gauss** (1829) nghiên cứu; việc giải các ràng buộc bất đẳng thức tuyến tính có từ **Fourier** (1827).

Các bài toán thỏa mãn ràng buộc miền hữu hạn cũng có một lịch sử lâu đời. Ví dụ, tô màu đồ thị (mà tô màu bản đồ là một trường hợp đặc biệt) là một bài toán cũ trong toán học. Phỏng đoán bốn màu (rằng mọi đồ thị phẳng có thể được tô màu bằng bốn hoặc ít hơn bốn màu) lần đầu tiên được đưa ra bởi **Francis Guthrie**, một sinh viên của De Morgan, vào năm 1852. Nó đã chống lại giải pháp—mặc dù có một số tuyên bố đã được xuất bản ngược lại—cho đến khi một bằng chứng được đưa ra bởi **Appel** và **Haken** (1977) (xem cuốn sách *Four Colors Suffice* (Wilson, 2004)). Những người theo chủ nghĩa thuần túy thất vọng vì một phần của bằng chứng dựa vào máy tính, vì vậy **Georges Gonthier** (2008), sử dụng công cụ chứng minh định lý COQ, đã đưa ra một bằng chứng chính thức rằng chương trình chứng minh của Appel và Haken là đúng.

Các lớp cụ thể của các bài toán thỏa mãn ràng buộc xuất hiện trong suốt lịch sử khoa học máy tính. Một trong những ví dụ ban đầu có ảnh hưởng nhất là **SKETCHPAD** (Sutherland, 1963), giải quyết các ràng buộc hình học trong sơ đồ và là tiền thân của các chương trình vẽ và công cụ CAD hiện đại. Việc xác định các CSP như một lớp tổng quát là nhờ **Ugo Montanari** (1974). Việc giảm các CSP bậc cao hơn thành các CSP hoàn toàn hai ngôi với các biến phụ (xem Bài tập 5.NARY) ban

đầu là nhờ nhà logic học thế kỷ 19 **Charles Sanders Peirce**. Nó đã được giới thiệu vào tài liệu CSP bởi **Dechter** (1990b) và được trình bày chi tiết bởi **Bacchus** và **van Beek** (1998). Các CSP có ưu tiên giữa các giải pháp được nghiên cứu rộng rãi trong tài liệu tối ưu hóa; xem **Bistarelli et al.** (1997) để biết một tổng quát hóa của khuôn khổ CSP để cho phép các ưu tiên.

Các phương pháp lan truyền ràng buộc đã trở nên phổ biến nhờ thành công của **Waltz** (1975) trên các bài toán gán nhãn đường đa diện cho thị giác máy tính. Waltz chỉ ra rằng trong nhiều bài toán, việc lan truyền loại bỏ hoàn toàn nhu cầu quay lui. **Montanari** (1974) đã giới thiệu khái niệm đồ thị ràng buộc và lan truyền bằng tính nhất quán đường đi. **Alan Mackworth** (1977) đã đề xuất thuật toán AC-3 để thực thi tính nhất quán cung cũng như ý tưởng tổng quát về việc kết hợp quay lui với một mức độ thực thi tính nhất quán nào đó. AC-4, một thuật toán nhất quán cung hiệu quả hơn được phát triển bởi **Mohr** và **Henderson** (1986), chạy trong thời gian trường hợp xấu nhất là $O(cd^2)$ nhưng có thể chậm hơn AC-3 trong các trường hợp trung bình. Thuật toán PC-2 (**Mackworth**, 1977) đạt được tính nhất quán đường đi theo cùng một cách mà AC-3 đạt được tính nhất quán cung.

Ngay sau khi bài báo của Mackworth xuất hiện, các nhà nghiên cứu bắt đầu thử nghiệm sự đánh đổi giữa chi phí thực thi tính nhất quán và lợi ích về mặt giảm tìm kiếm. **Haralick** và **Elliott** (1980) ủng hộ thuật toán kiểm tra phía trước tối thiểu được mô tả bởi **McGregor** (1979), trong khi **Gaschnig** (1979) đề xuất kiểm tra tính nhất quán cung đầy đủ sau mỗi gán giá trị biến—một thuật toán sau này được gọi là MAC bởi **Sabin** và **Freuder** (1994). Bài báo sau này cung cấp bằng chứng khá thuyết phục rằng trên các CSP khó hơn, việc kiểm tra tính nhất quán cung đầy đủ mang lại hiệu quả. **Freuder** (1978, 1982) đã điều tra khái niệm k-nhất quán và mối quan hệ của nó với độ phức tạp của việc giải quyết các CSP. **Dechter** và **Dechter** (1987) đã giới thiệu tính nhất quán cung có hướng. **Apt** (1999) mô tả một khuôn khổ thuật toán chung trong đó các thuật toán lan truyền tính nhất quán có thể được phân tích, và các cuộc khảo sát được đưa ra bởi **Bessière** (2006) và **Barták et al.** (2010).

Các phương pháp đặc biệt để xử lý các ràng buộc bậc cao hơn hoặc toàn cục lần đầu tiên được phát triển trong bối cảnh của **lập trình logic ràng buộc**. **Marriott** và **Stuckey** (1998) cung cấp một phạm vi bao quát tuyệt vời về nghiên cứu trong lĩnh vực này. Ràng buộc **Alldiff** đã được nghiên cứu bởi **Regin** (1994), **Stergiou** và **Walsh** (1999), và **van Hoeve** (2001). Có các thuật toán suy luận phức tạp hơn cho Alldiff (xem **van Hoeve** và **Katriel**, 2006) lan truyền nhiều ràng buộc hơn nhưng tốn kém hơn về mặt tính toán để chạy. Các ràng buộc giới hạn đã được đưa vào lập trình logic ràng buộc bởi **Van Hentenryck et al.** (1998). Một cuộc khảo sát về các ràng buộc toàn cục được cung cấp bởi **van Hoeve** và **Katriel** (2006).

Sudoku đã trở thành CSP được biết đến rộng rãi nhất và được **Simonis** (2005) mô tả như vậy. **Agerbeck** và **Hansen** (2008) mô tả một số chiến lược và chỉ ra rằng Sudoku trên một bảng $n^2 \times n^2$ nằm trong lớp các bài toán **NP-hard**.

Vào năm 1850, **C. F. Gauss** đã mô tả một thuật toán quay lui đệ quy để giải bài toán 8 con hậu, đã được xuất bản trên tạp chí cờ vua của Đức **Schachzeitung** vào năm 1848. Gauss gọi phương pháp của mình là **Tatonniren**, có nguồn gốc từ từ tiếng Pháp *tatonner*—mò mẫm, như thể trong bóng tối.

Theo **Donald Knuth** (liên lạc cá nhân), **R. J. Walker** đã giới thiệu thuật ngữ *backtrack* vào những năm 1950. **Walker** (1960) đã mô tả thuật toán quay lui cơ bản và sử dụng nó để tìm tất cả các giải

pháp cho bài toán 13 con hậu. **Golomb** và **Baumert** (1965) đã xây dựng, với các ví dụ, lớp tổng quát của các bài toán tổ hợp mà quay lui có thể được áp dụng và giới thiệu cái mà chúng ta gọi là heuristic MRV. **Bitner** và **Reingold** (1975) đã cung cấp một cuộc khảo sát có ảnh hưởng về các kỹ thuật quay lui. **Brelaz** (1979) đã sử dụng heuristic bậc như một cách để phá vỡ sự hòa sau khi áp dụng heuristic MRV. Thuật toán kết quả, mặc dù đơn giản, vẫn là phương pháp tốt nhất để tô màu k-đồ thị tùy ý. **Haralick** và **Elliott** (1980) đã đề xuất heuristic giá trị ràng buộc ít nhất.

Phương pháp nhảy lùi cơ bản là nhờ **John Gaschnig** (1977, 1979). **Kondrak** và **van Beek** (1997) đã chỉ ra rằng thuật toán này về cơ bản bị bao hàm bởi kiểm tra phía trước. **Nhảy lùi theo mâu thuẫn** được phát minh bởi **Prosser** (1993). **Dechter** (1990a) đã giới thiệu nhảy lùi dựa trên đồ thị, giới hạn độ phức tạp của các thuật toán dựa trên nhảy lùi như một hàm của đồ thị ràng buộc (**Dechter** và **Frost**, 2002). Một dạng rất tổng quát của quay lui thông minh đã được phát triển sớm bởi **Stallman** và **Sussman** (1977). Kỹ thuật **quay lui theo hướng phụ thuộc** của họ kết hợp nhảy lùi với học no-good (**McAllester**, 1990) và dẫn đến sự phát triển của các hệ thống duy trì sự thật (**Doyle**, 1979), mà chúng ta thảo luận trong Mục 10.6.2. Mối liên hệ giữa hai lĩnh vực được phân tích bởi **de Kleer** (1989).

Công trình của Stallman và Sussman cũng đã giới thiệu ý tưởng **học ràng buộc**, trong đó các kết quả một phần thu được bằng tìm kiếm có thể được lưu và sử dụng lại sau này trong tìm kiếm. Ý tưởng đã được **Dechter** (1990a) chính thức hóa. **Đánh dấu lại** (Gaschnig, 1979) là một phương pháp đặc biệt đơn giản trong đó các gán giá trị cặp đôi nhất quán và không nhất quán được lưu và sử dụng để tránh kiểm tra lại các ràng buộc. Đánh dấu lại có thể được kết hợp với nhảy lùi theo mâu thuẫn; **Kondrak** và **van Beek** (1997) trình bày một thuật toán lai chứng minh rằng bao hàm một trong hai phương pháp được thực hiện riêng biệt.

Phương pháp **quay lui động** (**Ginsberg**, 1993) giữ lại các gán giá trị một phần thành công từ các tập hợp biến sau này khi quay lui trên một lựa chọn trước đó không làm mất hiệu lực thành công sau này. **Moskewicz** et al. (2001) chỉ ra cách các kỹ thuật này và các kỹ thuật khác được sử dụng để tạo ra một bộ giải SAT hiệu quả. Các nghiên cứu thực nghiệm về một số phương pháp quay lui ngẫu nhiên đã được thực hiện bởi **Gomes** et al. (2000) và **Gomes** và **Selman** (2001). **Van Beek** (2006) khảo sát quay lui.

Tìm kiếm cục bộ trong các bài toán thỏa mãn ràng buộc đã trở nên phổ biến nhờ công trình của **Kirkpatrick** et al. (1983) về mô phỏng luyện kim (xem Chương 4), được sử dụng rộng rãi cho bố cục VLSI và các bài toán lập lịch. **Beck** et al. (2011) đưa ra một cái nhìn tổng quan về công trình gần đây về lập lịch công việc. Heuristic mâu thuẫn tối thiểu lần đầu tiên được đề xuất bởi **Gu** (1989) và được phát triển độc lập bởi **Minton** et al. (1992). **Sosic** và **Gu** (1994) đã chỉ ra cách nó có thể được áp dụng để giải bài toán 3.000.000 con hậu trong vòng chưa đầy một phút. Thành công đáng kinh ngạc của tìm kiếm cục bộ sử dụng mâu thuẫn tối thiểu trên bài toán n-queens đã dẫn đến một sự đánh giá lại bản chất và sự phổ biến của các bài toán “dễ” và “khó”. **Peter Cheeseman** et al. (1991) đã khám phá độ khó của các CSP được tạo ngẫu nhiên và phát hiện ra rằng hầu hết các bài toán như vậy hoặc là dễ một cách tầm thường hoặc không có giải pháp. Chỉ khi các tham số của trình tạo bài toán được đặt trong một phạm vi hẹp nhất định, trong đó khoảng một nửa số bài toán có thể giải được, chúng ta mới tìm thấy các trường hợp bài toán “khó”. Chúng ta thảo luận thêm về hiện tượng này trong Chương 7.

Konolige (1994) đã chỉ ra rằng tìm kiếm cục bộ kém hơn tìm kiếm quay lui trên các bài toán có một mức độ cấu trúc cục bộ nhất định; điều này dẫn đến công trình kết hợp tìm kiếm cục bộ và

suy luận, chẳng hạn như của **Pinkas** và **Dechter** (1995). **Hoos** và **Tsang** (2006) cung cấp một cuộc khảo sát về các kỹ thuật tìm kiếm cục bộ, và sách giáo khoa được cung cấp bởi **Hoos** và **Stützle** (2004) và **Aarts** và **Lenstra** (2003).

Công trình liên quan đến cấu trúc và độ phức tạp của các CSP có nguồn gốc từ **Freuder** (1985) và **Mackworth** và **Freuder** (1985), những người đã chỉ ra rằng tìm kiếm trên các cây nhất quán cung hoạt động mà không cần bất kỳ quay lui nào. Một kết quả tương tự, với các phần mở rộng cho các siêu đồ thị không chu trình, đã được phát triển trong cộng đồng cơ sở dữ liệu (**Beeri et al.**, 1983). **Bayardo** và **Miranker** (1994) trình bày một thuật toán cho các CSP có cấu trúc cây chạy trong thời gian tuyến tính mà không cần bất kỳ tiền xử lý nào. **Dechter** (1990a) mô tả cách tiếp cận tập cắt chu trình.

Kể từ khi các bài báo đó được xuất bản, đã có rất nhiều tiến bộ trong việc phát triển các kết quả tổng quát hơn liên quan đến độ phức tạp của việc giải một CSP với cấu trúc của đồ thị ràng buộc của nó. Khái niệm **độ rộng cây** được giới thiệu bởi các nhà lý thuyết đồ thị **Robertson** và **Seymour** (1986). **Dechter** và **Pearl** (1987, 1989), dựa trên công trình của Freuder, đã áp dụng một khái niệm liên quan (mà họ gọi là *độ rộng cảm ứng* nhưng giống hệt với độ rộng cây) cho các bài toán thỏa mãn ràng buộc và phát triển cách tiếp cận phân rã cây được phác thảo trong Mục 5.5.

Dựa trên công trình này và các kết quả từ lý thuyết cơ sở dữ liệu, **Gottlob et al.** (1999a, 1999b) đã phát triển một khái niệm, *hypertreewidth*, dựa trên đặc trưng của CSP như một siêu đồ thị. Ngoài việc chỉ ra rằng bất kỳ CSP nào có độ rộng cây siêu đồ thị w có thể được giải trong thời gian $O(n^{w+1} \log n)$, họ cũng chỉ ra rằng độ rộng cây siêu đồ thị bao hàm tất cả các thước đo “độ rộng” đã được định nghĩa trước đây theo nghĩa là có những trường hợp mà độ rộng cây siêu đồ thị bị chặn và các thước đo khác là không bị chặn.

Thuật toán RELSAT của **Bayardo** và **Schrag** (1997) đã kết hợp học ràng buộc và nhảy lùi và được chứng minh là vượt trội hơn nhiều thuật toán khác cùng thời. Điều này đã dẫn đến các thuật toán tìm kiếm AND-OR có thể áp dụng cho cả CSP và suy luận xác suất (**Dechter** và **Mateescu**, 2007). **Brown et al.** (1988) giới thiệu ý tưởng phá vỡ đối xứng trong các CSP, và **Gent et al.** (2006) đưa ra một cuộc khảo sát.

Lĩnh vực thỏa mãn ràng buộc phân tán xem xét việc giải quyết các CSP khi có một tập hợp các tác nhân, mỗi tác nhân kiểm soát một tập hợp con của các biến ràng buộc. Đã có các hội thảo thường niên về bài toán này kể từ năm 2000 và phạm vi bao quát tốt ở những nơi khác (**Collin et al.**, 1999; **Pearce et al.**, 2008).

So sánh các thuật toán CSP chủ yếu là một khoa học thực nghiệm: ít kết quả lý thuyết cho thấy một thuật toán chiếm ưu thế hơn một thuật toán khác trên tất cả các bài toán; thay vào đó, chúng ta cần chạy các thử nghiệm để xem thuật toán nào hoạt động tốt hơn trên các trường hợp bài toán điển hình. Như **Hooker** (1995) chỉ ra, chúng ta cần cẩn thận để phân biệt giữa thử nghiệm cạnh tranh—như xảy ra trong các cuộc thi giữa các thuật toán dựa trên thời gian chạy—và thử nghiệm khoa học, có mục tiêu là xác định các thuộc tính của một thuật toán quyết định hiệu quả của nó trên một lớp bài toán.

Các sách giáo khoa của **Apt** (2003), **Dechter** (2003), **Tsang** (1993), và **Lecoutre** (2009), và bộ sưu tập của **Rossi et al.** (2006), là những nguồn tài liệu tuyệt vời về xử lý ràng buộc. Có một số bài báo khảo sát hay, bao gồm của **Dechter** và **Frost** (2002), và **Barták et al.** (2010). **Carbonnel**

và **Cooper** (2016) khảo sát các lớp CSP khả thi. **Kondrak** và **van Beek** (1997) đưa ra một cuộc khảo sát phân tích về các thuật toán tìm kiếm quay lui, và **Bacchus** và **van Run** (1995) đưa ra một cuộc khảo sát mang tính thực nghiệm hơn. Lập trình ràng buộc được đề cập trong các cuốn sách của **Apt** (2003) và **Fruhworth** và **Abdennadher** (2003). Các bài báo về thỏa mãn ràng buộc xuất hiện thường xuyên trên *Artificial Intelligence* và trong tạp chí chuyên ngành *Constraints*; các bộ giải SAT mới nhất được mô tả trong Cuộc thi SAT quốc tế hàng năm. Địa điểm hội nghị chính là Hội nghị quốc tế về Nguyên tắc và Thực hành Lập trình Ràng buộc, thường được gọi là CP.

CHƯƠNG 6 TÌM KIẾM ĐỐI KHÁNG VÀ TRÒ CHƠI

Trong chương này, chúng ta khám phá các môi trường mà các tác nhân khác đang chống lại chúng ta.

Tìm kiếm đối kháng Kinh tế Cắt tỉa Thông tin không hoàn hảo

Trong chương này, chúng ta sẽ đề cập đến các môi trường cạnh tranh, trong đó hai hoặc nhiều tác nhân có mục tiêu xung đột, tạo ra các bài toán tìm kiếm đối kháng. Thay vì đối phó với sự hỗn loạn của các cuộc giao tranh trong thế giới thực, chúng ta sẽ tập trung vào các trò chơi, chẳng hạn như cờ vua, cờ vây và poker. Đối với các nhà nghiên cứu AI, bản chất đơn giản của các trò chơi này là một lợi thế: trạng thái của một trò chơi rất dễ biểu diễn và các tác nhân thường bị giới hạn trong một số ít hành động mà hiệu quả của chúng được xác định bởi các quy tắc chính xác. Các trò chơi vật lý, chẳng hạn như croquet và khúc côn cầu trên băng, có mô tả phức tạp hơn, phạm vi hành động khả thi lớn hơn và các quy tắc khá không chính xác để xác định tính hợp pháp của các hành động. Ngoại trừ bóng đá robot, các trò chơi vật lý này chưa thu hút nhiều sự quan tâm trong cộng đồng AI.

6.1 Lý thuyết trò chơi

Có ít nhất ba quan điểm mà chúng ta có thể áp dụng đối với các môi trường đa tác nhân. Quan điểm đầu tiên, thích hợp khi có rất nhiều tác nhân, là xem xét chúng một cách tổng hợp như một **nền kinh tế**, cho phép chúng ta làm những việc như dự đoán rằng nhu cầu tăng sẽ làm giá tăng, mà không cần phải dự đoán hành động của bất kỳ tác nhân cá nhân nào.

Thứ hai, chúng ta có thể coi các tác nhân đối kháng chỉ là một phần của môi trường—một phần làm cho môi trường **không xác định**. Nhưng nếu chúng ta mô hình hóa các đối thủ theo cách mà, ví dụ, mưa đôi khi rơi và đôi khi không, chúng ta sẽ bỏ lỡ ý tưởng rằng các đối thủ của chúng ta đang tích cực cố gắng đánh bại chúng ta, trong khi mưa được cho là không có ý định đó.

Quan điểm thứ ba là mô hình hóa một cách rõ ràng các tác nhân đối kháng bằng các kỹ thuật **tìm kiếm cây trò chơi đối kháng**. Đó là những gì chương này đề cập. Chúng ta bắt đầu với một lớp trò chơi bị hạn chế và xác định nước đi tối ưu và một thuật toán để tìm nó: **tìm kiếm minimax**, một sự khái quát hóa của tìm kiếm AND-OR (từ Hình 4.11). Chúng ta sẽ chỉ ra rằng **cắt tỉa** làm cho việc tìm kiếm hiệu quả hơn bằng cách bỏ qua các phần của cây tìm kiếm không tạo ra sự khác biệt đối với nước đi tối ưu. Đối với các trò chơi không tầm thường, chúng ta thường sẽ không có đủ thời gian để chắc chắn tìm thấy nước đi tối ưu (ngay cả khi có cắt tỉa); chúng ta sẽ phải cắt bớt việc tìm kiếm tại một số điểm. Đối với mỗi trạng thái mà chúng ta chọn dừng tìm kiếm, chúng ta sẽ hỏi ai đang chiến thắng. Để trả lời câu hỏi này, chúng ta có một lựa chọn: chúng ta có thể áp dụng một **hàm đánh giá heuristic** để ước tính ai đang chiến thắng dựa trên các đặc điểm của trạng thái (Mục 6.3), hoặc chúng ta có thể lấy trung bình kết quả của nhiều mô phỏng trò chơi nhanh từ trạng thái đó đến cuối (Mục 6.4).

Mục 6.5 thảo luận về các trò chơi có yếu tố may rủi (thông qua việc tung xúc xắc hoặc xáo bài) và Mục 6.6 đề cập đến các trò chơi có **thông tin không hoàn hảo** (chẳng hạn như poker và bridge, nơi không phải tất cả các lá bài đều hiển thị cho tất cả người chơi).

6.1.1 Trò chơi hai người chơi tổng bằng không

Các trò chơi được nghiên cứu phổ biến nhất trong AI (chẳng hạn như cờ vua và cờ vây) là những gì các nhà lý thuyết trò chơi gọi là **trò chơi xác định, hai người chơi, luân phiên, thông tin hoàn hảo, tổng bằng không**. “**Thông tin hoàn hảo**” là một từ đồng nghĩa với “có thể quan sát hoàn toàn”,¹ và “**tổng bằng không**” có nghĩa là điều gì tốt cho người chơi này thì cũng tệ cho người chơi kia: không có kết quả “cùng thắng”. Đối với các trò chơi, chúng ta thường sử dụng thuật ngữ **nước đi** làm từ đồng nghĩa với “hành động” và **vị trí** làm từ đồng nghĩa với “trạng thái”.

Chúng ta sẽ gọi hai người chơi của chúng ta là **MAX** và **MIN**, vì những lý do sẽ sớm trở nên rõ ràng. MAX đi trước, và sau đó người chơi luân phiên di chuyển cho đến khi trò chơi kết thúc. Vào cuối trò chơi, điểm được trao cho người chơi chiến thắng và hình phạt được trao cho người thua cuộc. Một trò chơi có thể được định nghĩa chính thức với các yếu tố sau:

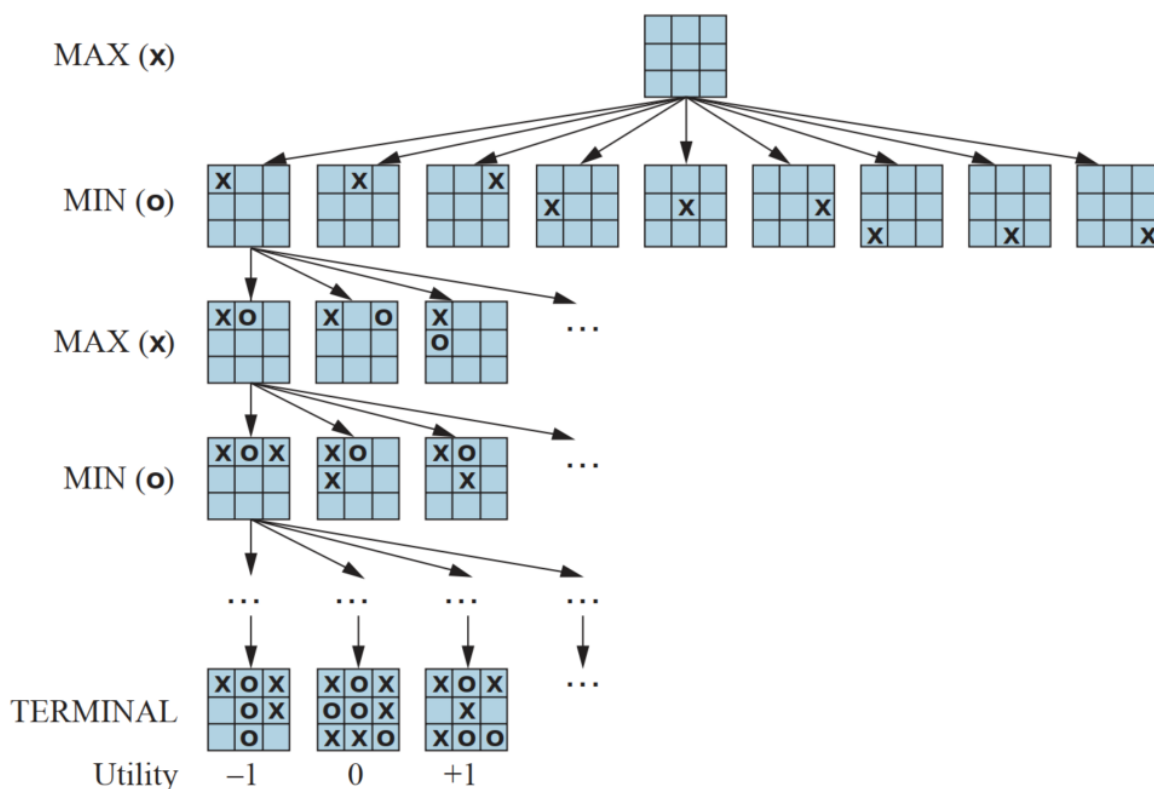
- **S_0 : Trạng thái ban đầu**, chỉ định cách trò chơi được thiết lập khi bắt đầu.
- **TO-MOVE(s)**: Người chơi đến lượt di chuyển trong trạng thái s .
- **ACTIONS(s)**: Tập hợp các **nước đi hợp lệ** trong trạng thái s .
- **RESULT(s, a)**: **Mô hình chuyển đổi**, xác định trạng thái kết quả từ việc thực hiện hành động a trong trạng thái s .
- **IS-TERMINAL(s)**: Một **kiểm tra kết thúc**, đúng khi trò chơi kết thúc và sai trong trường hợp khác. Các trạng thái mà trò chơi đã kết thúc được gọi là **trạng thái kết thúc**.
- **UTILITY(s, p)**: Một **hàm tiện ích** (cũng được gọi là hàm mục tiêu hoặc hàm thanh toán), xác định giá trị số cuối cùng cho người chơi p khi trò chơi kết thúc ở trạng thái kết thúc s . Trong cờ vua, kết quả là thắng, thua hoặc hòa, với các giá trị 1,0 hoặc 1/2.² Một số trò chơi có phạm vi kết quả khả thi rộng hơn—ví dụ, các khoản thanh toán trong backgammon dao động từ 0 đến 192.

Giống như trong Chương 3, trạng thái ban đầu, hàm ACTIONS và hàm RESULT xác định **đồ thị không gian trạng thái**—một đồ thị trong đó các đỉnh là trạng thái, các cạnh là nước đi và một trạng thái có thể được đến bằng nhiều đường dẫn. Như trong Chương 3, chúng ta có thể chồng một **cây tìm kiếm** lên một phần của đồ thị đó để xác định nước đi nào cần thực hiện. Chúng ta định nghĩa **cây trò chơi** hoàn chỉnh là một cây tìm kiếm theo mọi chuỗi nước đi cho đến một trạng thái kết thúc. Cây trò chơi có thể là vô hạn nếu bản thân không gian trạng thái không bị giới hạn hoặc nếu các quy tắc của trò chơi cho phép lặp lại các vị trí vô hạn.

Hình 6.1 cho thấy một phần của cây trò chơi cho trò chơi tic-tac-toe (cờ ca-rô). Từ trạng thái ban đầu, MAX có chín nước đi khả thi. Lượt chơi luân phiên giữa việc MAX đặt một chữ X và MIN đặt một chữ O cho đến khi chúng ta đến các nút lá tương ứng với các trạng thái kết thúc sao cho một người chơi có ba ô vuông liên tiếp hoặc tất cả các ô vuông đều được điền. Con số trên mỗi nút lá cho biết giá trị tiện ích của trạng thái kết thúc theo quan điểm của MAX; các giá trị cao là tốt cho MAX và xấu cho MIN (đó là cách mà người chơi có tên của họ).

Đối với tic-tac-toe, cây trò chơi tương đối nhỏ—ít hơn $9! = 362,880$ nút cuối (chỉ với 5,478 trạng thái khác nhau). Nhưng đối với cờ vua có hơn 10^{40} nút, vì vậy cây trò chơi được coi là một cấu trúc lý thuyết mà chúng ta không thể thực hiện trong thế giới vật lý.

1 Một số tác giả phân biệt, sử dụng “trò chơi thông tin không hoàn hảo” cho một trò chơi như poker, nơi người chơi có được thông tin riêng tư về bài của mình mà những người chơi khác không có, và “trò chơi quan sát một phần” để chỉ một trò chơi như StarCraft II, nơi mỗi người chơi có thể nhìn thấy môi trường gần đó, nhưng không thể nhìn thấy môi trường ở xa. 2 Cờ vua được coi là một trò chơi “tổng bằng không”, mặc dù tổng kết quả của hai người chơi là +1 cho mỗi trò chơi, không phải bằng không. “Tổng không đổi” sẽ là một thuật ngữ chính xác hơn, nhưng tổng bằng không là truyền thống và có ý nghĩa nếu bạn tưởng tượng mỗi người chơi phải trả phí vào cửa là 1/2.



Hình 6.1 Một cây trò chơi (một phần) cho trò chơi tic-tac-toe. Nút trên cùng là trạng thái ban đầu và MAX đi trước, đặt một chữ X vào một ô trống. Chúng ta hiển thị một phần của cây, luân phiên các nước đi của MIN (O) và MAX (X), cho đến khi chúng ta cuối cùng đến các trạng thái kết thúc, có thể được gán các tiện ích theo các quy tắc của trò chơi.

6.2 Các Quyết định tối ưu trong trò chơi

MAX muốn tìm một chuỗi hành động dẫn đến chiến thắng, nhưng MIN có quyền can thiệp vào điều đó. Điều này có nghĩa là chiến lược của MAX phải là một **kế hoạch có điều kiện**—một chiến lược dự phòng chỉ định một phản hồi cho mỗi nước đi có thể có của MIN. Trong các trò chơi có kết quả nhị phân (thắng hoặc thua), chúng ta có thể sử dụng tìm kiếm AND-OR (trang 143) để tạo ra kế hoạch có điều kiện. Trên thực tế, đối với các trò chơi như vậy, định nghĩa của một chiến lược chiến thắng cho trò chơi giống hệt với định nghĩa của một giải pháp cho một bài toán lập kế hoạch không xác định: trong cả hai trường hợp, kết quả mong muốn phải được đảm bảo bất kể “phía bên kia” làm gì. Đối với các trò chơi có nhiều điểm kết quả, chúng ta cần một thuật toán tổng quát hơn một chút được gọi là **tìm kiếm minimax**.

Hãy xem xét trò chơi đơn giản trong Hình 6.2. Các nước đi có thể có của MAX tại nút gốc được gắn nhãn a_1, a_2 và a_3 . Các phản hồi có thể có cho a_1 của MIN là b_1, b_2, b_3 , v.v. Trò chơi cụ thể này kết thúc sau mỗi một nước đi của MAX và MIN. (Lưu ý: Trong một số trò chơi, từ “nước đi” có nghĩa là cả hai người chơi đã thực hiện một hành động; do đó từ **ply** được sử dụng để chỉ rõ một cách rõ ràng một nước đi của một người chơi, đưa chúng ta xuống một cấp độ sâu hơn trong cây trò chơi.) Các tiện ích của các trạng thái kết thúc trong trò chơi này dao động từ 2 đến 14.

Với một cây trò chơi, **chiến lược tối ưu** có thể được xác định bằng cách tìm ra **giá trị minimax** của mỗi trạng thái trong cây, mà chúng ta viết là $MINIMAX(s)$. Giá trị minimax là tiện ích (đối với MAX) của việc ở trong trạng thái đó, giả sử rằng cả hai người chơi đều chơi tối ưu từ đó cho đến cuối trò chơi. Giá trị minimax của một trạng thái kết thúc chỉ là tiện ích của nó. Trong một trạng thái không kết thúc, MAX thích di chuyển đến một trạng thái có giá trị tối đa khi đến lượt MAX di chuyển và MIN thích một trạng thái có giá trị tối thiểu (tức là giá trị tối thiểu cho MAX và do đó giá trị tối đa cho MIN). Vì vậy, chúng ta có:

$$MINIMAX(s) = \begin{cases} UTILITY(s, MAX) & \text{if IS-TERMINAL}(s) \\ \max_{a \in Actions(s)} MINIMAX(RESULT(s, a)) & \text{if TO-MOVE}(s) = MAX \\ \min_{a \in Actions(s)} MINIMAX(RESULT(s, a)) & \text{if TO-MOVE}(s) = MIN \end{cases}$$

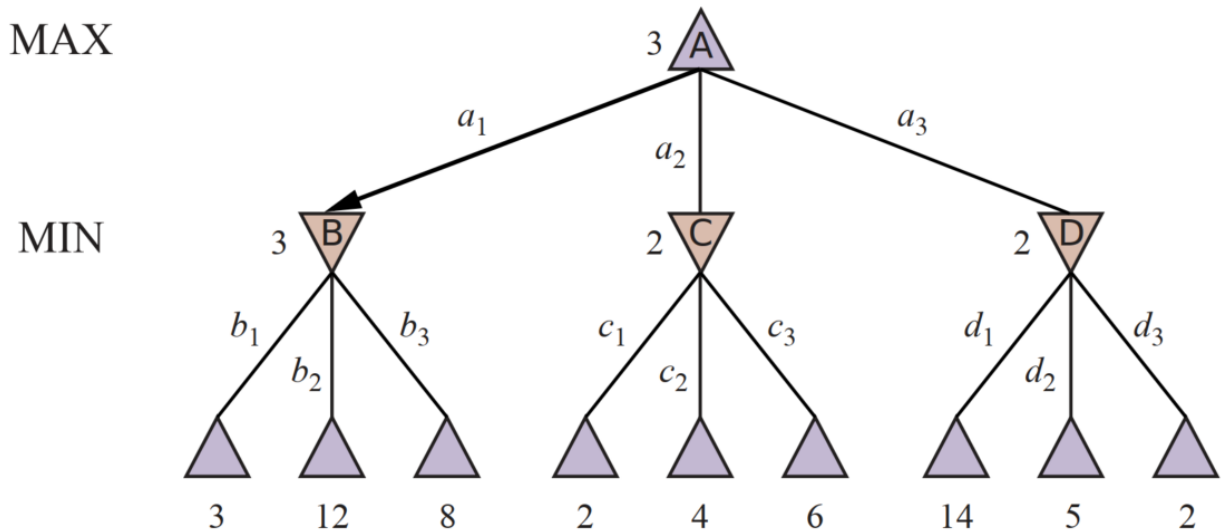
Hãy áp dụng các định nghĩa này cho cây trò chơi trong Hình 6.2. Các nút kết thúc ở cấp dưới cùng nhận giá trị tiện ích của chúng từ hàm $UTILITY$ của trò chơi. Nút MIN đầu tiên, được gắn nhãn B, có ba trạng thái kế thừa với các giá trị 3, 12 và 8, vì vậy giá trị minimax của nó là 3. Tương tự, hai nút MIN khác có giá trị minimax là 2. Nút gốc là nút MAX; các trạng thái kế thừa của nó có giá trị minimax là 3, 2 và 2; vì vậy nó có giá trị minimax là 3. Chúng ta cũng có thể xác định **quyết định minimax** tại gốc: hành động a_1 là lựa chọn tối ưu cho MAX vì nó dẫn đến trạng thái có giá trị minimax cao nhất.

Định nghĩa này về trò chơi tối ưu cho MAX giả định rằng MIN cũng chơi tối ưu. Điều gì xảy ra nếu MIN không chơi tối ưu? Khi đó MAX sẽ làm tốt ít nhất là như khi đối mặt với một người chơi tối ưu, có thể tốt hơn. Tuy nhiên, điều đó không có nghĩa là luôn luôn tốt nhất để chơi nước đi tối ưu minimax khi đối mặt với một đối thủ không tối ưu. Hãy xem xét một tình huống mà trò chơi tối ưu của cả hai bên sẽ dẫn đến một trận hòa, nhưng có một nước đi mạo hiểm cho MAX dẫn đến một trạng thái trong đó có 10 nước đi phản hồi khả thi của MIN mà tất cả đều có vẻ hợp lý, nhưng 9 trong số đó là một trận thua cho MIN và một là một trận thua cho MAX. Nếu MAX tin rằng MIN không có đủ sức mạnh tính toán để khám phá nước đi tối ưu, MAX có thể muốn thử nước đi mạo hiểm, với lý do rằng cơ hội 9/10 để chiến thắng tốt hơn một trận hòa chắc chắn.

6.2.1 Thuật toán tìm kiếm Minimax

Bây giờ chúng ta có thể tính toán $MINIMAX(s)$, chúng ta có thể biến nó thành một thuật toán tìm kiếm tìm ra nước đi tốt nhất cho MAX bằng cách thử tất cả các hành động và chọn hành động mà trạng thái kết quả có giá trị $MINIMAX$ cao nhất. Hình 6.3 cho thấy thuật toán. Nó là một thuật toán đệ quy tiến hành xuống tận các nút lá của cây và sau đó sao lưu các giá trị minimax thông qua cây khi đệ quy kết thúc. Ví dụ, trong Hình 6.2, thuật toán đầu tiên đệ quy xuống ba nút dưới cùng bên trái và sử dụng hàm $UTILITY$ trên chúng để khám phá rằng các giá trị của chúng lần

lượt là 3, 12 và 8. Sau đó, nó lấy giá trị tối thiểu của các giá trị này, 3, và trả về nó làm giá trị được sao lưu của nút B. Một quá trình tương tự cho các giá trị được sao lưu là 2 cho C và 2 cho D. Cuối cùng, chúng ta lấy giá trị tối đa của 3, 2 và 2 để có được giá trị được sao lưu là 3 cho nút gốc.



Hình 6.2 Một cây trò chơi hai ply. Các nút hình tam giác là “nút MAX”, trong đó đến lượt MAX di chuyển, và các nút hình vuông là “nút MIN”. Các nút kết thúc hiển thị các giá trị tiện ích cho MAX; các nút khác được gắn nhãn với các giá trị minimax của chúng. Nước đi tốt nhất của MAX tại gốc là a_1 , vì nó dẫn đến trạng thái có giá trị minimax cao nhất, và phản hồi tốt nhất của MIN là b_1 , vì nó dẫn đến trạng thái có giá trị minimax thấp nhất.

```
function MINIMAX-SEARCH(game, state) returns an action
  player ← game.TO-MOVE(state)
  value, move ← MAX-VALUE(game, state)
  return move
```

```
function MAX-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v, move ← -∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
    if v2 > v then
      v, move ← v2, a
  return v, move
```

```
function MIN-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v, move ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
```

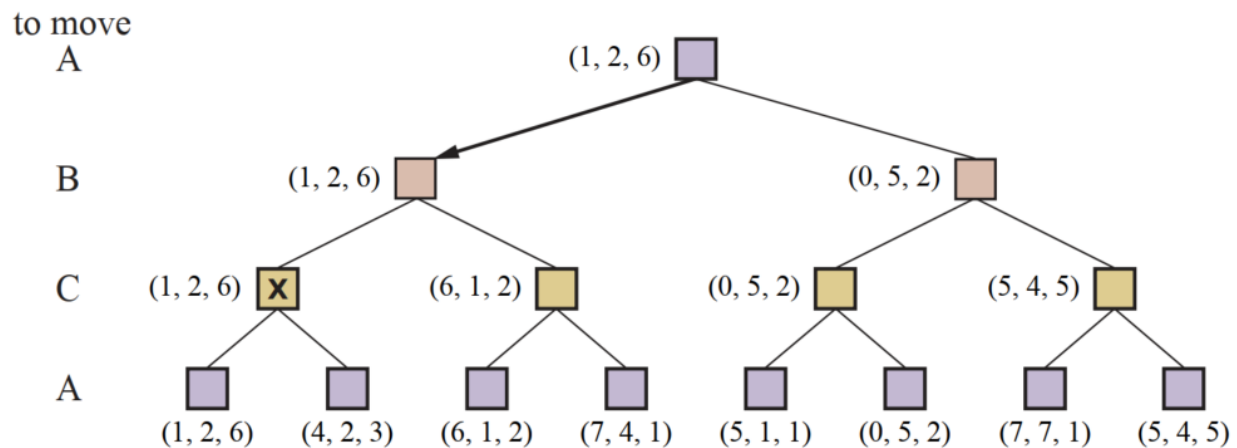
```

if v2 < v then
    v, move ← v2, a
return v, move

```

Hình 6.3 Một thuật toán để tính toán nước đi tối ưu bằng cách sử dụng minimax—nước đi dẫn đến một trạng thái kết thúc có tiện ích tối đa, với giả định rằng đối thủ chơi để giảm thiểu tiện ích. Các hàm MAX-VALUE và MIN-VALUE đi qua toàn bộ cây trò chơi, xuống tận các nút lá, để xác định giá trị được sao lưu của một trạng thái và nước đi để đến đó.

Thuật toán minimax thực hiện một khám phá **độ sâu đầu tiên** hoàn chỉnh của cây trò chơi. Nếu độ sâu tối đa của cây là m và có b nước đi hợp lệ tại mỗi điểm, thì độ phức tạp thời gian của thuật toán minimax là $O(b^m)$. Độ phức tạp không gian là $O(b^m)$ đối với một thuật toán tạo tất cả các hành động cùng một lúc, hoặc $O(m)$ đối với một thuật toán tạo các hành động từng cái một (xem trang 98). Độ phức tạp theo cấp số nhân làm cho MINIMAX không thực tế đối với các trò chơi phức tạp; ví dụ, cờ vua có hệ số phân nhánh khoảng 35 và trò chơi trung bình có độ sâu khoảng 80 ply, và không thể tìm kiếm $35^{80} \approx 10^{123}$ trạng thái. Tuy nhiên, MINIMAX đóng vai trò là cơ sở cho việc phân tích toán học các trò chơi. Bằng cách xấp xỉ phân tích minimax theo nhiều cách khác nhau, chúng ta có thể suy ra các thuật toán thực tế hơn.



Hình 6.4 Ba ply đầu tiên của một cây trò chơi với ba người chơi (A, B, C). Mỗi nút được gắn nhãn với các giá trị theo quan điểm của mỗi người chơi. Nước đi tốt nhất được đánh dấu tại gốc.

6.2.2 Quyết định tối ưu trong các trò chơi nhiều người chơi

Nhiều trò chơi phổ biến cho phép nhiều hơn hai người chơi. Chúng ta hãy xem xét cách mở rộng ý tưởng minimax cho các trò chơi nhiều người chơi. Điều này khá đơn giản từ quan điểm kỹ thuật, nhưng đặt ra một số vấn đề khái niệm mới thú vị.

Đầu tiên, chúng ta cần thay thế giá trị đơn cho mỗi nút bằng một **vector giá trị**. Ví dụ, trong một trò chơi ba người chơi với người chơi A, B và C, một vector $\langle v_A, v_B, v_C \rangle$ được liên kết với mỗi nút. Đối với các trạng thái kết thúc, vector này đưa ra tiện ích của trạng thái từ quan điểm của mỗi người chơi. (Trong các trò chơi hai người chơi, tổng bằng không, vector hai

phần tử có thể được giảm xuống thành một giá trị duy nhất vì các giá trị luôn ngược nhau.) Cách đơn giản nhất để thực hiện điều này là yêu cầu hàm UTILITY trả về một vector tiện ích.

Bây giờ chúng ta phải xem xét các trạng thái không kết thúc. Hãy xem xét nút được đánh dấu X trong cây trò chơi được hiển thị trong Hình 6.4. Trong trạng thái đó, người chơi C chọn làm gì. Hai lựa chọn dẫn đến các trạng thái kết thúc với các vector tiện ích là $\langle v_A = 1, v_B = 2, v_C = 6 \rangle$ và $\langle v_A = 4, v_B = 2, v_C = 3 \rangle$. Vì 6 lớn hơn 3, C nên chọn nước đi đầu tiên. Điều này có nghĩa là nếu trạng thái X đạt được, trò chơi tiếp theo sẽ dẫn đến một trạng thái kết thúc với tiện ích là $\langle v_A = 1, v_B = 2, v_C = 6 \rangle$. Do đó, giá trị được sao lưu của X là vector này. Nói chung, giá trị được sao lưu của một nút n là vector tiện ích của trạng thái kế thừa có giá trị cao nhất đối với người chơi lựa chọn tại n .

Bất cứ ai chơi các trò chơi nhiều người chơi, chẳng hạn như Diplomacy hoặc Settlers of Catan, đều nhanh chóng nhận ra rằng có nhiều điều đang diễn ra hơn trong các trò chơi hai người chơi. Các trò chơi nhiều người chơi thường liên quan đến các **liên minh**, dù chính thức hay không chính thức, giữa các người chơi. Các liên minh được tạo ra và phá vỡ khi trò chơi tiếp diễn. Làm thế nào để chúng ta hiểu được hành vi như vậy? Liệu các liên minh có phải là một hệ quả tự nhiên của các chiến lược tối ưu cho mỗi người chơi trong một trò chơi nhiều người chơi không? Hóa ra là chúng có thể.

Ví dụ, giả sử A và B ở vị trí yếu và C ở vị trí mạnh hơn. Khi đó, thường là tối ưu cho cả A và B để tấn công C thay vì tấn công lẫn nhau, để C không tiêu diệt từng người trong số họ một. Bằng cách này, sự hợp tác nảy sinh từ hành vi hoàn toàn ích kỷ. Tất nhiên, ngay sau khi C suy yếu dưới đòn tấn công chung, liên minh sẽ mất giá trị, và một trong hai A hoặc B có thể vi phạm thỏa thuận.

Trong một số trường hợp, các liên minh rõ ràng chỉ làm cụ thể hóa những gì đã xảy ra. Trong các trường hợp khác, một sự kỳ thị xã hội gắn liền với việc phá vỡ một liên minh, vì vậy người chơi phải cân bằng lợi thế trước mắt của việc phá vỡ một liên minh với bất lợi lâu dài của việc bị coi là không đáng tin cậy. Xem Mục 17.2 để biết thêm về những phức tạp này.

Nếu trò chơi không phải là tổng bằng không, thì sự hợp tác cũng có thể xảy ra chỉ với hai người chơi. Giả sử, ví dụ, có một trạng thái kết thúc với tiện ích là $\langle v_A = 1000, v_B = 1000 \rangle$ và 1000 là tiện ích cao nhất có thể cho mỗi người chơi. Khi đó, chiến lược tối ưu là cả hai người chơi đều làm mọi thứ có thể để đạt được trạng thái này—nghĩa là, người chơi sẽ tự động hợp tác để đạt được một mục tiêu cùng mong muốn.

6.2.3 Cắt tia Alpha-Beta

Số lượng trạng thái trò chơi là hàm mũ theo độ sâu của cây. Không có thuật toán nào có thể loại bỏ hoàn toàn số mũ, nhưng đôi khi chúng ta có thể cắt nó làm đôi, tính toán quyết định minimax đúng mà không cần kiểm tra mọi trạng thái bằng cách **cắt tia** (xem trang 108) các phần lớn của cây không tạo ra sự khác biệt cho kết quả. Kỹ thuật cụ thể mà chúng ta xem xét được gọi là **cắt tia alpha-beta**.

Hãy xem xét lại cây trò chơi hai ply từ Hình 6.2. Chúng ta hãy xem lại việc tính toán quyết định tối ưu một lần nữa, lần này chú ý cẩn thận đến những gì chúng ta biết tại mỗi điểm trong quá trình.

Các bước được giải thích trong Hình 6.5. Kết quả là chúng ta có thể xác định quyết định minimax mà không cần phải đánh giá hai trong số các nút lá.

Một cách khác để xem xét điều này là một sự đơn giản hóa của công thức cho MINIMAX. Hãy để hai trạng thái kế thừa chưa được đánh giá của nút C trong Hình 6.5 có các giá trị x và y . Khi đó, giá trị của nút gốc được cho bởi

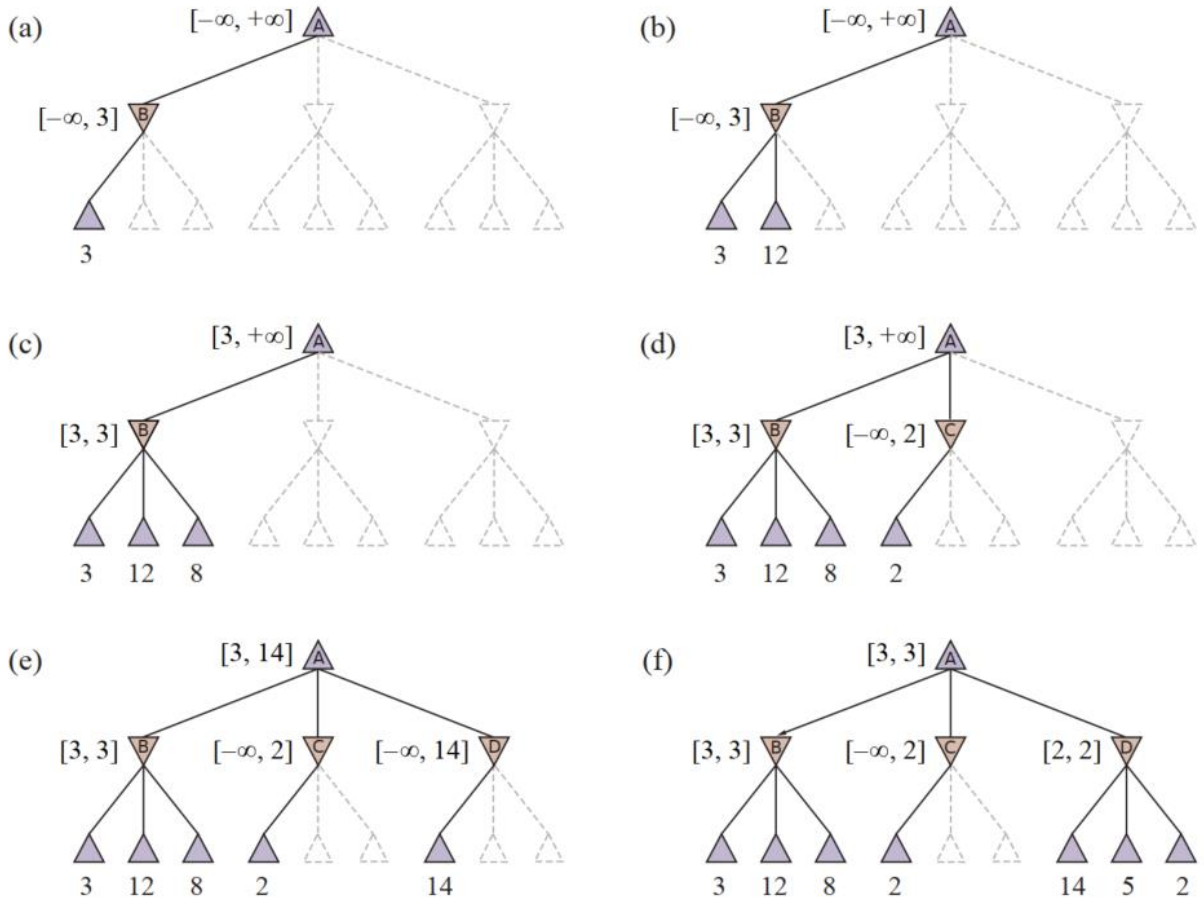
$$\begin{aligned} MINIMAX(root) &= \max(\min(3,12,8), \min(2, x, y), \min(14,5,2)) \\ &= \max(3, \min(2, x, y), 2) \\ &= \max(3, z, 2) \text{ trong đó } z = \min(2, x, y) \leq 2 \\ &= 3. \end{aligned}$$

Nói cách khác, giá trị của nút gốc và do đó quyết định minimax là độc lập với các giá trị của các nút lá x và y , và do đó chúng có thể được cắt tía.

Cắt tía Alpha-beta có thể được áp dụng cho các cây có bất kỳ độ sâu nào, và thường có thể cắt tía toàn bộ cây con thay vì chỉ các nút lá. Nguyên tắc chung là: hãy xem xét một nút n ở đâu đó trong cây (xem Hình 6.6), sao cho Người chơi có lựa chọn di chuyển đến n . Nếu Người chơi có một lựa chọn tốt hơn ở cùng cấp độ (ví dụ: m' trong Hình 6.6) hoặc ở bất kỳ điểm nào cao hơn trong cây (ví dụ: m trong Hình 6.6), thì Người chơi sẽ không bao giờ di chuyển đến n . Vì vậy, một khi chúng ta đã tìm ra đủ về n (bằng cách kiểm tra một số hậu duệ của nó) để đi đến kết luận này, chúng ta có thể cắt tía nó.

Hãy nhớ rằng tìm kiếm minimax là **độ sâu đầu tiên**, vì vậy tại bất kỳ thời điểm nào chúng ta chỉ cần xem xét các nút dọc theo một đường dẫn duy nhất trong cây. Cắt tía alpha-beta có tên của nó từ hai tham số bổ sung trong $MAX-VALUE(state, \alpha, \beta)$ (xem Hình 6.7) mô tả các giới hạn trên các giá trị được sao lưu xuất hiện ở bất cứ đâu dọc theo đường dẫn:

- α = giá trị của lựa chọn tốt nhất (tức là giá trị cao nhất) mà chúng ta đã tìm thấy cho đến nay tại bất kỳ điểm lựa chọn nào dọc theo đường dẫn cho MAX. Hãy nghĩ: α = “ít nhất là”.
- β = giá trị của lựa chọn tốt nhất (tức là giá trị thấp nhất) mà chúng ta đã tìm thấy cho đến nay tại bất kỳ điểm lựa chọn nào dọc theo đường dẫn cho MIN. Hãy nghĩ: β = “nhiều nhất là”.



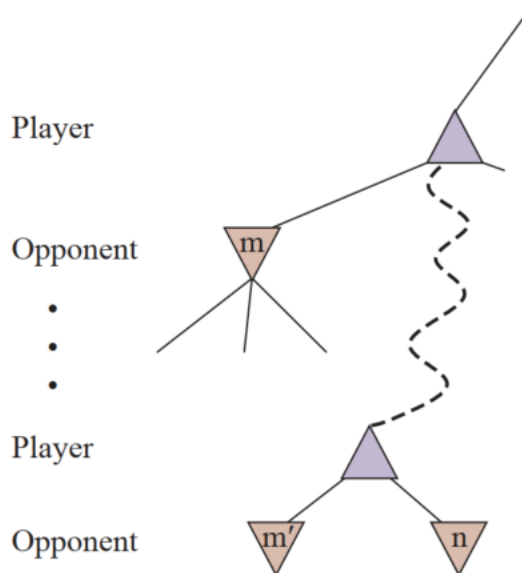
Hình 6.5 Các giai đoạn trong việc tính toán quyết định tối ưu cho cây trò chơi trong Hình 6.2. Tại mỗi điểm, chúng ta hiển thị phạm vi các giá trị có thể có cho mỗi nút. (a) Nút lá đầu tiên bên dưới B có giá trị là 3. Do đó, B, là một nút MIN, có giá trị nhiều nhất là 3. (b) Nút lá thứ hai bên dưới B có giá trị là 12; MIN sẽ tránh nước đi này, vì vậy giá trị của B vẫn nhiều nhất là 3. (c) Nút lá thứ ba bên dưới B có giá trị là 8; chúng ta đã thấy tất cả các trạng thái kế thừa của B, vì vậy giá trị của B chính xác là 3. Bây giờ chúng ta có thể suy ra rằng giá trị của nút gốc ít nhất là 3, vì MAX có một lựa chọn trị giá 3 tại gốc. (d) Nút lá đầu tiên bên dưới C có giá trị là 2. Do đó, C, là một nút MIN, có giá trị nhiều nhất là 2. Nhưng chúng ta biết rằng B trị giá 3, vì vậy MAX sẽ không bao giờ chọn C. Do đó, không có lý do gì để xem xét các trạng thái kế thừa khác của C. Đây là một ví dụ về **cắt tỉa alpha-beta**. (e) Nút lá đầu tiên bên dưới D có giá trị là 14, vì vậy D trị giá nhiều nhất là 14. Điều này vẫn cao hơn lựa chọn thay thế tốt nhất của MAX (tức là 3), vì vậy chúng ta cần tiếp tục khám phá các trạng thái kế thừa của D. Cũng lưu ý rằng bây giờ chúng ta có các giới hạn trên tất cả các trạng thái kế thừa của gốc, vì vậy giá trị của gốc cũng nhiều nhất là 14. (f) Trạng thái kế thừa thứ hai của D trị giá 5, vì vậy một lần nữa chúng ta cần tiếp tục khám phá. Trạng thái kế thừa thứ ba trị giá 2, vì vậy bây giờ D trị giá chính xác là 2. Quyết định của MAX tại gốc là di chuyển đến B, đưa ra giá trị là 3.

Thuật toán tìm kiếm alpha-beta cập nhật các giá trị của α và β khi nó tiếp tục và cắt tỉa các nhánh còn lại tại một nút (tức là, chấm dứt lời gọi đệ quy) ngay khi giá trị của nút hiện tại được biết là tệ

hơn giá trị α hoặc β hiện tại tương ứng cho MAX hoặc MIN. Thuật toán hoàn chỉnh được đưa ra trong Hình 6.7. Hình 6.5 theo dõi tiến trình của thuật toán trên một cây trò chơi.

6.2.4 Sắp xếp nước đi

Hiệu quả của việc cắt tỉa alpha-beta phụ thuộc rất nhiều vào thứ tự các trạng thái được kiểm tra. Ví dụ, trong Hình 6.5(e) và (f), chúng ta không thể cắt tỉa bất kỳ trạng thái kế thừa nào của D cả vì các trạng thái kế thừa tệ nhất (từ quan điểm của MIN) đã được tạo ra đầu tiên. Nếu trạng thái kế thừa thứ ba của D được tạo ra trước, với giá trị là 2, chúng ta đã có thể cắt tỉa hai trạng thái kế thừa còn lại. Điều này cho thấy rằng có thể đáng để thử kiểm tra trước các trạng thái kế thừa có khả năng tốt nhất.



Hình 6.6 Trường hợp tổng quát cho việc cắt tỉa alpha-beta. Nếu m hoặc m' tốt hơn n cho Người chơi, chúng ta sẽ không bao giờ đến được n trong trò chơi.

Nếu điều này có thể được thực hiện một cách hoàn hảo, alpha-beta sẽ chỉ cần kiểm tra $O(b^{m/2})$ nút để chọn nước đi tốt nhất, thay vì $O(b^m)$ đối với minimax. Điều này có nghĩa là hệ số phân nhánh hiệu quả trở thành $\sqrt{b}$ thay vì b —đối với cờ vua, khoảng 6 thay vì 35. Nói cách khác, alpha-beta với sắp xếp nước đi hoàn hảo có thể giải một cây sâu gấp đôi so với minimax trong cùng một khoảng thời gian. Với sắp xếp nước đi ngẫu nhiên, tổng số nút được kiểm tra sẽ xấp xỉ $O(b^{3m/4})$ đối với b vừa phải. Rõ ràng, chúng ta không thể đạt được sắp xếp nước đi hoàn hảo—trong trường hợp đó, hàm sắp xếp có thể được sử dụng để chơi một trò chơi hoàn hảo! Nhưng chúng ta thường có thể tiến khá gần. Đối với cờ vua, một hàm sắp xếp khá đơn giản (chẳng hạn như thử các nước đi bắt quân trước, sau đó là các mối đe dọa, sau đó là các nước đi về phía trước và sau đó là các nước đi lùi) sẽ đưa bạn đến gần với kết quả $O(b^{m/2})$ tốt nhất trong khoảng một hệ số 2.

Việc thêm các lược đồ sắp xếp nước đi động, chẳng hạn như thử trước các nước đi được tìm thấy là tốt nhất trong quá khứ, đưa chúng ta đến khá gần giới hạn lý thuyết. Quá khứ có thể là nước đi trước—thường thì các mối đe dọa tương tự vẫn còn—hoặc nó có thể đến từ việc khám phá trước đó của nước đi hiện tại thông qua một quá trình **làm sâu dần** (xem trang 98). Đầu tiên, tìm kiếm

sâu một ply và ghi lại thứ hạng của các nước đi dựa trên đánh giá của chúng. Sau đó tìm kiếm sâu hơn một ply, sử dụng thứ hạng trước đó để thông báo việc sắp xếp nước đi; và cứ như vậy. Thời gian tìm kiếm tăng lên từ việc làm sâu dần có thể được bù đắp nhiều hơn từ việc sắp xếp nước đi tốt hơn. Các nước đi tốt nhất được gọi là **nước đi sát thủ**, và việc thử chúng trước được gọi là **heuristic nước đi sát thủ**.

Trong Mục 3.3.3, chúng ta đã lưu ý rằng các đường dẫn dư thừa đến các trạng thái lặp lại có thể gây ra sự gia tăng theo cấp số nhân trong chi phí tìm kiếm, và việc giữ một bảng các trạng thái đã đạt được trước đó có thể giải quyết vấn đề này. Trong tìm kiếm cây trò chơi, các trạng thái lặp lại có thể xảy ra do **hoán vị**—các hoán vị khác nhau của chuỗi nước đi kết thúc ở cùng một vị trí, và vấn đề có thể được giải quyết bằng một **bảng hoán vị** lưu trữ giá trị heuristic của các trạng thái.

```
function ALPHA-BETA-SEARCH(game, state) returns an action
  player ← game.TO-MOVE(state)
  value, move ← MAX-VALUE(game, state, -∞, +∞)
  return move

function MAX-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v, move ← -∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a), α, β)
    if v2 > v then
      v, move ← v2, a
      α ← MAX(α, v)
    if v ≥ β then return v, move
  return v, move

function MIN-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v, move ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a), α, β)
    if v2 < v then
      v, move ← v2, a
      β ← MIN(β, v)
    if v ≤ α then return v, move
  return v, move
```

Hình 6.7 Thuật toán tìm kiếm alpha-beta. Lưu ý rằng các hàm này giống với các hàm MINIMAX-SEARCH trong Hình 6.3, ngoại trừ việc chúng ta duy trì các giới hạn trong các biến *alpha* và *beta*, và sử dụng chúng để cắt bớt tìm kiếm khi một giá trị nằm ngoài các giới hạn.

Ví dụ, giả sử Trắng có một nước đi w_1 có thể được Đen trả lời bằng b_1 và một nước đi không liên quan w_2 ở phía bên kia của bàn cờ có thể được trả lời bằng b_2 , và chúng ta tìm kiếm chuỗi nước đi $[w_1, b_1, w_2, b_2]$; hãy gọi trạng thái kết quả là s . Sau khi khám phá một cây con lớn bên dưới s , chúng ta tìm thấy giá trị được sao lưu của nó, mà chúng ta lưu trữ trong bảng hoán vị. Khi chúng

ta sau đó tìm kiếm chuỗi nước đi $[w_2, b_2, w_1, b_1]$, chúng ta lại kết thúc ở trạng thái s , và chúng ta có thể tra cứu giá trị thay vì lặp lại việc tìm kiếm. Trong cờ vua, việc sử dụng các bảng hoán vị rất hiệu quả, cho phép chúng ta tăng gấp đôi độ sâu tìm kiếm có thể đạt được trong cùng một khoảng thời gian.

Ngay cả với cắt tỉa alpha-beta và sắp xếp nước đi thông minh, minimax sẽ không hoạt động cho các trò chơi như cờ vua và cờ vây, bởi vì vẫn còn quá nhiều trạng thái để khám phá trong thời gian có sẵn. Trong bài báo đầu tiên về việc chơi trò chơi trên máy tính, *Lập trình máy tính để chơi cờ vua* (Shannon, 1950), Claude Shannon đã nhận ra vấn đề này và đề xuất hai chiến lược: một **chiến lược Loại A** xem xét tất cả các nước đi có thể có đến một độ sâu nhất định trong cây tìm kiếm, và sau đó sử dụng một hàm đánh giá heuristic để ước tính tiện ích của các trạng thái ở độ sâu đó. Nó khám phá một phần rộng nhưng nông của cây. Một **chiến lược Loại B** bỏ qua các nước đi có vẻ tệ, và đi theo các đường đi đầy hứa hẹn “càng xa càng tốt”. Nó khám phá một phần sâu nhưng hẹp của cây. Trong lịch sử, hầu hết các chương trình cờ vua là Loại A (chúng ta sẽ đề cập trong mục tiếp theo), trong khi các chương trình cờ vây thường là Loại B (được đề cập trong Mục 6.4), bởi vì hệ số phân nhánh trong cờ vây cao hơn nhiều. Gần đây hơn, các chương trình Loại B đã thể hiện khả năng chơi ở cấp độ vô địch thế giới trên nhiều trò chơi khác nhau, bao gồm cả cờ vua (Silver và cộng sự, 2018).

6.3 Tìm kiếm cây Alpha-Beta Heuristic

Để tận dụng thời gian tính toán có hạn, chúng ta có thể cắt bớt tìm kiếm sớm và áp dụng một **hàm đánh giá heuristic** cho các trạng thái, coi các nút không kết thúc như thể chúng là các nút kết thúc. Nói cách khác, chúng ta thay thế hàm UTILITY bằng EVAL, ước tính tiện ích của một trạng thái. Chúng ta cũng thay thế kiểm tra kết thúc bằng một **kiểm tra cắt bớt**, phải trả về true cho các trạng thái kết thúc, nhưng nếu không thì có thể tự do quyết định khi nào cắt bớt tìm kiếm, dựa trên độ sâu tìm kiếm và bất kỳ thuộc tính nào của trạng thái mà nó chọn để xem xét. Điều đó cho chúng ta công thức H-MINIMAX(s, d) cho giá trị minimax heuristic của trạng thái s ở độ sâu tìm kiếm d :

$$\text{H-MINIMAX}(s, d) = \begin{cases} \text{EVAL}(s, \text{MAX}) & \text{if IS-CUTOFF}(s, d) \\ \max_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MIN}. \end{cases}$$

6.3.1 Các hàm đánh giá

Một hàm đánh giá heuristic **EVAL(s, p)** trả về một ước tính về tiện ích mong đợi của trạng thái s đối với người chơi p , giống như các hàm heuristic của Chương 3 trả về một ước tính về khoảng cách đến mục tiêu. Đối với các trạng thái kết thúc, phải là $\text{EVAL}(s, p) = \text{UTILITY}(s, p)$ và đối với các trạng thái không kết thúc, đánh giá phải nằm ở đâu đó giữa thua và thắng: $\text{UTILITY}(\text{thua}, p) \leq \text{EVAL}(s, p) \leq \text{UTILITY}(\text{thắng}, p)$.

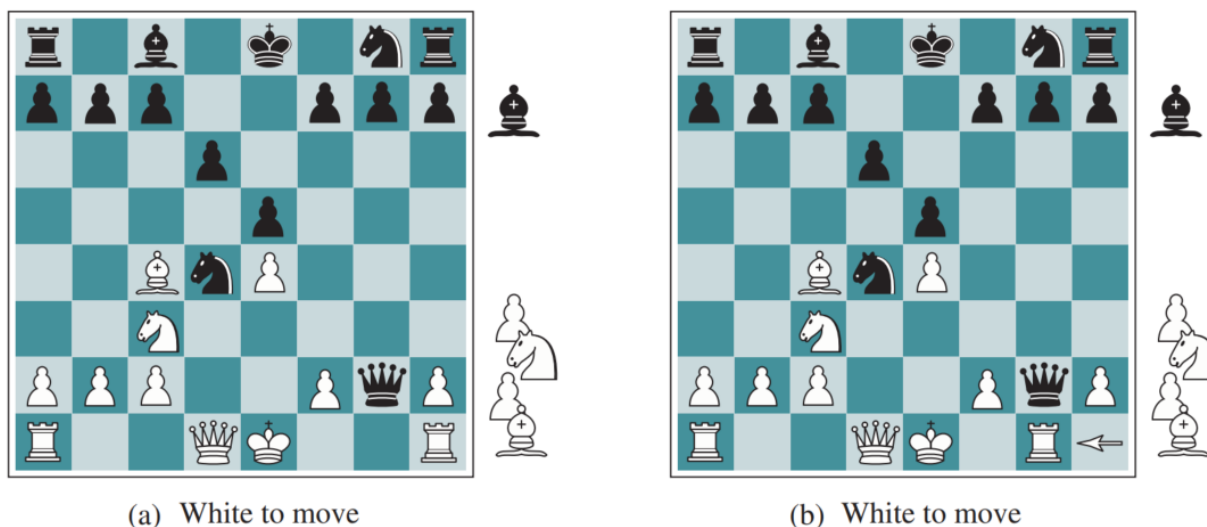
Ngoài những yêu cầu đó, điều gì tạo nên một hàm đánh giá tốt? Thứ nhất, việc tính toán không được mất quá nhiều thời gian! (Toàn bộ vấn đề là để tìm kiếm nhanh hơn.) Thứ hai, hàm đánh giá phải tương quan mạnh mẽ với cơ hội chiến thắng thực tế. Người ta có thể tự hỏi về cụm từ “cơ hội chiến thắng”. Rốt cuộc, cờ vua không phải là một trò chơi may rủi: chúng ta biết trạng thái hiện tại một cách chắc chắn, và không có xúc xắc nào liên quan; nếu không người chơi nào mắc lỗi, kết

quả đã được định trước. Nhưng nếu việc tìm kiếm phải bị cắt bớt ở các trạng thái không kết thúc, thì thuật toán nhất thiết sẽ không chắc chắn về kết quả cuối cùng của các trạng thái đó (mặc dù sự không chắc chắn đó có thể được giải quyết bằng các tài nguyên tính toán vô hạn).

Chúng ta hãy làm cho ý tưởng này cụ thể hơn. Hầu hết các hàm đánh giá hoạt động bằng cách tính toán các **đặc điểm** khác nhau của trạng thái—ví dụ, trong cờ vua, chúng ta sẽ có các đặc điểm cho số lượng quân tốt trắng, quân tốt đen, quân hậu trắng, quân hậu đen, v.v. Các đặc điểm, khi được tổng hợp lại, xác định các loại hoặc lớp tương đương khác nhau của các trạng thái: các trạng thái trong mỗi loại có cùng giá trị cho tất cả các đặc điểm. Ví dụ, một loại có thể chứa tất cả các trận đấu cuối cùng hai tốt đấu một tốt. Bất kỳ loại nào cũng sẽ chứa một số trạng thái dẫn đến chiến thắng (với lối chơi hoàn hảo), một số dẫn đến hòa, và một số dẫn đến thua.

Hàm đánh giá không biết trạng thái nào là gì, nhưng nó có thể trả về một giá trị duy nhất ước tính tỷ lệ các trạng thái với mỗi kết quả. Ví dụ, giả sử kinh nghiệm của chúng ta cho thấy rằng 82% các trạng thái gặp phải trong loại hai tốt đấu một tốt dẫn đến chiến thắng (tiền ích +1); 2% dẫn đến thua (0), và 16% dẫn đến hòa (1/2). Khi đó, một đánh giá hợp lý cho các trạng thái trong loại này là giá trị kỳ vọng: $(0.82 \times 1) + (0.02 \times 0) + (0.16 \times 1/2) = 0.90$. Về nguyên tắc, giá trị kỳ vọng có thể được xác định cho mỗi loại trạng thái, dẫn đến một hàm đánh giá hoạt động cho bất kỳ trạng thái nào.

Trong thực tế, loại phân tích này yêu cầu quá nhiều loại và do đó quá nhiều kinh nghiệm để ước tính tất cả các xác suất. Thay vào đó, hầu hết các hàm đánh giá tính toán các đóng góp số riêng biệt từ mỗi đặc điểm và sau đó kết hợp chúng để tìm ra tổng giá trị.



Hình 6.8 Hai vị trí cờ vua chỉ khác nhau ở vị trí của quân xe ở phía dưới bên phải. Trong (a), Đen có lợi thế một mã và hai tốt, đủ để thắng ván cờ. Trong (b), Trắng sẽ bắt quân hậu, cho nó một lợi thế đủ mạnh để thắng.

Trong nhiều thế kỷ, người chơi cờ vua đã phát triển các cách đánh giá giá trị của một vị trí chỉ bằng ý tưởng này. Ví dụ, các sách cờ vua nhập môn đưa ra một **giá trị vật chất** gần đúng cho mỗi quân cờ: mỗi quân tốt trị giá 1, một mã hoặc tượng trị giá 3, một xe 5, và quân hậu 9. Các đặc điểm

khác như “cấu trúc tốt đẹp” và “an toàn vua” có thể trị giá nửa tốt, chẳng hạn. Các giá trị đặc điểm này sau đó chỉ đơn giản là được cộng lại để có được đánh giá của vị trí.

Về mặt toán học, loại hàm đánh giá này được gọi là **hàm tuyến tính có trọng số** vì nó có thể được biểu thị là:

$$EVAL(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s) = \sum_{i=1}^n w_i f_i(s)$$

trong đó mỗi f_i là một đặc điểm của vị trí (chẳng hạn như “số lượng tượng trắng”) và mỗi w_i là một trọng số (nói lên đặc điểm đó quan trọng như thế nào). Các trọng số nên được chuẩn hóa sao cho tổng luôn nằm trong khoảng từ thua (0) đến thắng (+1). Một lợi thế an toàn tương đương với một quân tốt mang lại một khả năng đáng kể để chiến thắng, và một lợi thế an toàn tương đương với ba quân tốt sẽ mang lại chiến thắng gần như chắc chắn, như được minh họa trong Hình 6.8(a). Chúng ta đã nói rằng hàm đánh giá nên tương quan mạnh mẽ với cơ hội chiến thắng thực tế, nhưng nó không cần phải tương quan tuyến tính: nếu trạng thái s có khả năng chiến thắng gấp đôi trạng thái s' , chúng ta không yêu cầu rằng $EVAL(s)$ phải gấp đôi $EVAL(s')$; tất cả những gì chúng ta yêu cầu là $EVAL(s) > EVAL(s')$.

Việc cộng các giá trị của các đặc điểm dường như là một điều hợp lý để làm, nhưng trên thực tế nó liên quan đến một giả định mạnh mẽ: rằng sự đóng góp của mỗi đặc điểm là độc lập với các giá trị của các đặc điểm khác. Vì lý do này, các chương trình hiện tại cho cờ vua và các trò chơi khác cũng sử dụng các kết hợp phi tuyến tính của các đặc điểm. Ví dụ, một cặp tượng có thể trị giá hơn gấp đôi giá trị của một tượng đơn lẻ, và một tượng có giá trị hơn trong giai đoạn cuối trận so với trước đó—khi đặc điểm số nước đi cao hoặc đặc điểm số quân cờ còn lại thấp.

Các đặc điểm và trọng số đến từ đâu? Chúng không phải là một phần của luật cờ vua, nhưng chúng là một phần của văn hóa kinh nghiệm chơi cờ vua của con người. Trong các trò chơi mà loại kinh nghiệm này không có sẵn, trọng số của hàm đánh giá có thể được ước tính bằng các kỹ thuật **học máy** của Chương 23. Áp dụng các kỹ thuật này cho cờ vua đã xác nhận rằng một tượng thực sự trị giá khoảng ba quân tốt, và có vẻ như hàng thế kỷ kinh nghiệm của con người có thể được tái tạo chỉ trong vài giờ học máy.

6.3.2 Cắt bớt tìm kiếm

Bước tiếp theo là sửa đổi ALPHA-BETA-SEARCH để nó sẽ gọi hàm heuristic EVAL khi thích hợp để cắt bớt tìm kiếm. Chúng ta thay thế hai dòng trong Hình 6.7 đề cập đến *IS-TERMINAL* bằng dòng sau:

```
if game.IS-CUTOFF(state, depth) then return game.EVAL(state, player), null
```

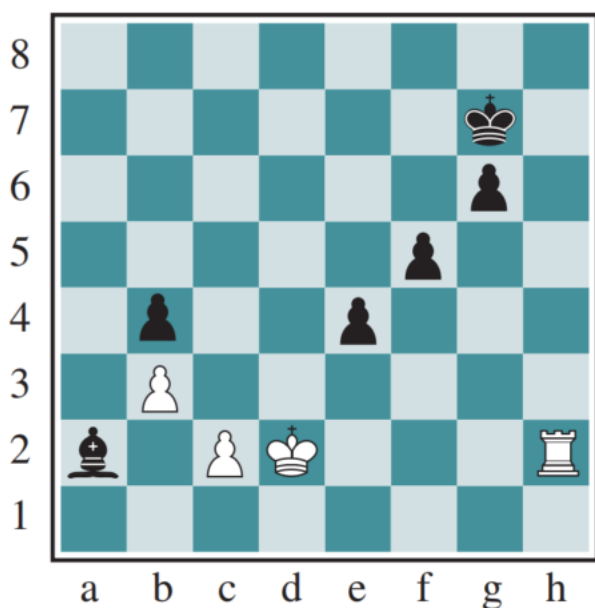
Chúng ta cũng phải sắp xếp một số thủ tục để độ sâu hiện tại được tăng lên trong mỗi lần gọi đệ quy. Cách tiếp cận đơn giản nhất để kiểm soát lượng tìm kiếm là đặt một giới hạn độ sâu cố định sao cho *IS-CUTOFF(state, depth)* trả về true cho tất cả độ sâu lớn hơn một độ sâu cố định d nào đó (cũng như cho tất cả các trạng thái kết thúc). Độ sâu d được chọn sao cho một nước đi được chọn trong thời gian được phân bổ. Một cách tiếp cận mạnh mẽ hơn là áp dụng **làm sâu dần**. (Xem Chương 3.) Khi hết thời gian, chương trình trả về nước đi được chọn bởi tìm kiếm hoàn thành sâu nhất. Như một phần thưởng, nếu trong mỗi vòng làm sâu dần, chúng ta giữ các mục trong bảng

hoán vị, các vòng tiếp theo sẽ nhanh hơn và chúng ta có thể sử dụng các đánh giá để cải thiện việc sắp xếp nước đi.

Các cách tiếp cận đơn giản này có thể dẫn đến lỗi do bản chất xấp xỉ của hàm đánh giá. Hãy xem xét lại hàm đánh giá đơn giản cho cờ vua dựa trên lợi thế vật chất. Giả sử chương trình tìm kiếm đến giới hạn độ sâu, đạt đến vị trí trong Hình 6.8(b), nơi Đen dẫn trước một mã và hai tốt. Nó sẽ báo cáo đây là giá trị heuristic của trạng thái, do đó tuyên bố rằng trạng thái này là một chiến thắng có khả năng của Đen. Nhưng nước đi tiếp theo của Trắng bắt hậu của Đen mà không có sự đền bù nào. Do đó, vị trí thực sự thuận lợi cho Trắng, nhưng điều này chỉ có thể được nhìn thấy bằng cách nhìn xa hơn.

Hàm đánh giá chỉ nên được áp dụng cho các vị trí **quiescent**—tức là, các vị trí mà không có nước đi đang chờ xử lý (chẳng hạn như một nước bắt hậu) có thể làm thay đổi đáng kể đánh giá. Đối với các vị trí không quiescent, *IS-CUTOFF* trả về false, và tìm kiếm tiếp tục cho đến khi các vị trí quiescent đạt được. Việc **tìm kiếm quiescent** bổ sung này đôi khi bị giới hạn chỉ xem xét một số loại nước đi nhất định, chẳng hạn như nước đi bắt quân, sẽ nhanh chóng giải quyết sự không chắc chắn trong vị trí.

Hiệu ứng chân trời khó loại bỏ hơn. Nó phát sinh khi chương trình phải đối mặt với một nước đi của đối thủ gây ra thiệt hại nghiêm trọng và cuối cùng là không thể tránh khỏi, nhưng có thể tạm thời tránh được bằng cách sử dụng các chiến thuật trì hoãn. Hãy xem xét vị trí cờ vua trong Hình 6.9. Rõ ràng là không có cách nào để tượng đen thoát ra. Ví dụ, xe trắng có thể bắt nó bằng cách di chuyển đến h1, sau đó a1, sau đó a2; một nước bắt ở độ sâu 6 ply.



Hình 6.9 Hiệu ứng chân trời. Với lượt đi của Đen, tượng đen chắc chắn sẽ bị diệt. Nhưng Đen có thể trì hoãn sự kiện đó bằng cách chiếu vua trắng bằng các quân tốt của nó, khuyến khích vua bắt tốt. Điều này đẩy việc mất tượng không thể tránh khỏi qua chân trời, và do đó các nước thí tốt được thuật toán tìm kiếm coi là những nước đi tốt thay vì những nước đi xấu.

Nhưng Đen có một chuỗi nước đi đẩy việc bắt tượng “qua chân trời”. Giả sử Đen tìm kiếm đến độ sâu 8 ply. Hầu hết các nước đi của Đen sẽ dẫn đến việc bắt tượng cuối cùng, và do đó sẽ được đánh dấu là nước đi “xấu”. Nhưng Đen cũng sẽ xem xét chuỗi nước đi bắt đầu bằng việc chiếu vua bằng một quân tốt, và dụ vua bắt tốt đó. Sau đó, Đen có thể làm điều tương tự với quân tốt thứ hai. Điều đó chiếm đủ nước đi mà việc bắt tượng sẽ không được khám phá trong phần còn lại của tìm kiếm của Đen. Đen nghĩ rằng đường đi này đã cứu tượng với cái giá của hai quân tốt, trong khi thực tế tất cả những gì nó đã làm là lãng phí tốt và đẩy việc bắt tượng không thể tránh khỏi ra ngoài chân trời mà Đen có thể nhìn thấy.

Một chiến lược để giảm thiểu hiệu ứng chân trời là cho phép **mở rộng đơn lẻ**, các nước đi “rõ ràng tốt hơn” tất cả các nước đi khác trong một vị trí nhất định, ngay cả khi việc tìm kiếm thường sẽ bị cắt bớt tại thời điểm đó. Trong ví dụ của chúng ta, một tìm kiếm sẽ tiết lộ rằng ba nước đi của xe trắng—h2 đến h1, sau đó h1 đến a1, và sau đó a1 bắt tượng trên a2—lần lượt là những nước đi rõ ràng tốt hơn, vì vậy ngay cả khi một chuỗi nước đi tốt đẩy chúng ta đến chân trời, những nước đi rõ ràng tốt hơn này sẽ được trao cơ hội để mở rộng tìm kiếm. Điều này làm cho cây sâu hơn, nhưng vì thường có ít mở rộng đơn lẻ, chiến lược này không thêm nhiều nút tổng thể vào cây và đã được chứng minh là hiệu quả trong thực tế.

6.3.3 Cắt tia về phía trước

Cắt tia alpha-beta cắt tia các nhánh của cây không thể có ảnh hưởng đến đánh giá cuối cùng, nhưng **cắt tia về phía trước** cắt tia các nước đi có vẻ là nước đi kém, nhưng có thể là nước đi tốt. Do đó, chiến lược này tiết kiệm thời gian tính toán với rủi ro mắc lỗi. Theo thuật ngữ của Shannon, đây là một chiến lược Loại B. Rõ ràng, hầu hết người chơi cờ vua đều làm điều này, chỉ xem xét một vài nước đi từ mỗi vị trí (ít nhất là một cách có ý thức).

Một cách tiếp cận để cắt tia về phía trước là **tìm kiếm chùm** (xem trang 133): trên mỗi ply, chỉ xem xét một “chùm” n nước đi tốt nhất (theo hàm đánh giá) thay vì xem xét tất cả các nước đi có thể có. Thật không may, cách tiếp cận này khá nguy hiểm vì không có gì đảm bảo rằng nước đi tốt nhất sẽ không bị cắt tia.

Thuật toán **PROBCUT**, hay cắt tia xác suất (Buro, 1995), là một phiên bản cắt tia về phía trước của tìm kiếm alpha-beta sử dụng số liệu thống kê thu được từ kinh nghiệm trước đó để giảm bớt cơ hội nước đi tốt nhất sẽ bị cắt tia. Tìm kiếm alpha-beta cắt tia bất kỳ nút nào được chứng minh là nằm ngoài cửa sổ (α, β) hiện tại. PROBCUT cũng cắt tia các nút có khả năng nằm ngoài cửa sổ. Nó tính toán xác suất này bằng cách thực hiện một tìm kiếm nông để tính toán giá trị được sao lưu v của một nút và sau đó sử dụng kinh nghiệm trong quá khứ để ước tính khả năng một điểm số v ở độ sâu d trong cây sẽ nằm ngoài (α, β) là bao nhiêu. Buro đã áp dụng kỹ thuật này cho chương trình Othello của mình, LOGISTELLO, và thấy rằng một phiên bản của chương trình của anh ấy với PROBCUT đã đánh bại phiên bản thông thường 64% thời gian, ngay cả khi phiên bản thông thường được cho gấp đôi thời gian.

Một kỹ thuật khác, **giảm nước đi muộn**, hoạt động dưới giả định rằng việc sắp xếp nước đi đã được thực hiện tốt, và do đó các nước đi xuất hiện muộn hơn trong danh sách các nước đi có thể có ít có khả năng là nước đi tốt. Nhưng thay vì cắt tia chúng hoàn toàn, chúng ta chỉ giảm độ sâu mà chúng ta tìm kiếm các nước đi này, do đó tiết kiệm thời gian. Nếu tìm kiếm giảm bớt trả về một giá trị trên giá trị α hiện tại, chúng ta có thể chạy lại tìm kiếm với độ sâu đầy đủ.

Kết hợp tất cả các kỹ thuật được mô tả ở đây dẫn đến một chương trình có thể chơi cờ vua (hoặc các trò chơi khác) một cách đáng tin cậy. Hãy giả sử chúng ta đã triển khai một hàm đánh giá cho cờ vua, một kiểm tra cắt bớt hợp lý với tìm kiếm quiescent. Hãy cũng giả sử rằng, sau nhiều tháng lập trình tẻ nhạt, chúng ta có thể tạo và đánh giá khoảng một triệu nút mỗi giây trên một PC mới nhất. Hệ số phân nhánh cho cờ vua là khoảng 35, trung bình, và 35^5 là khoảng 50 triệu, vì vậy nếu chúng ta sử dụng tìm kiếm minimax, chúng ta chỉ có thể nhìn trước năm ply trong khoảng một phút tính toán; các quy tắc cạnh tranh sẽ không cho chúng ta đủ thời gian để tìm kiếm sáu ply. Mặc dù không kém cỏi, một chương trình như vậy có thể bị đánh bại bởi một người chơi cờ vua trung bình, người thỉnh thoảng có thể lập kế hoạch trước sáu hoặc tám ply.

Với tìm kiếm alpha-beta và một bảng hoán vị lớn, chúng ta đạt được khoảng 14 ply, dẫn đến trình độ chơi của một chuyên gia. Chúng ta có thể đổi PC của mình lấy một máy trạm với 8 GPU, giúp chúng ta đạt hơn một tỷ nút mỗi giây, nhưng để đạt được trình độ đại kiện tướng, chúng ta vẫn cần một hàm đánh giá được điều chỉnh rộng rãi và một cơ sở dữ liệu lớn về các nước đi cuối trận. Các chương trình cờ vua hàng đầu như STOCKFISH có tất cả những điều này, thường đạt độ sâu 30 hoặc hơn trong cây tìm kiếm và vượt xa khả năng của bất kỳ người chơi nào.

6.3.4 Tìm kiếm so với tra cứu

Việc một chương trình cờ vua bắt đầu một ván đấu bằng cách xem xét một cây gồm hàng tỷ trạng thái trò chơi, chỉ để kết luận rằng nó sẽ chơi tốt lên e4 (nước đi đầu tiên phổ biến nhất), dường như là quá mức cần thiết. Sách mô tả lối chơi hay trong khai cuộc và tàn cuộc trong cờ vua đã có từ hơn một thế kỷ (Tattersall, 1911). Do đó, không có gì đáng ngạc nhiên khi nhiều chương trình chơi trò chơi sử dụng **tra cứu bảng** thay vì tìm kiếm cho phần khai cuộc và tàn cuộc của trò chơi.

Đối với khai cuộc, máy tính chủ yếu dựa vào chuyên môn của con người. Lời khuyên tốt nhất của các chuyên gia con người về cách chơi từng khai cuộc có thể được sao chép từ sách và nhập vào các bảng để máy tính sử dụng. Ngoài ra, máy tính có thể thu thập số liệu thống kê từ một cơ sở dữ liệu các ván đấu đã chơi trước đó để xem những chuỗi khai cuộc nào thường dẫn đến chiến thắng nhất. Đối với vài nước đi đầu tiên, có ít khả năng, và hầu hết các vị trí sẽ nằm trong bảng. Thông thường sau khoảng 10 hoặc 15 nước đi, chúng ta kết thúc ở một vị trí hiếm khi thấy, và chương trình phải chuyển từ tra cứu bảng sang tìm kiếm.

Gần cuối ván đấu, lại có ít vị trí có thể có hơn, và do đó việc tra cứu dễ dàng hơn. Nhưng ở đây, máy tính mới là chuyên gia: phân tích của máy tính về các ván đấu cuối cùng vượt xa khả năng của con người. Những người mới chơi có thể thắng một ván đấu cuối cùng vua-và-xe-đầu-với-vua (KRK) bằng cách tuân theo một vài quy tắc đơn giản. Các ván đấu cuối cùng khác, chẳng hạn như vua, tượng và mã đầu với vua (KBNK), rất khó thành thạo và không có mô tả chiến lược ngắn gọn.

Mặt khác, một máy tính có thể giải quyết hoàn toàn ván đấu cuối cùng bằng cách tạo ra một **chính sách**, đó là một ánh xạ từ mọi trạng thái có thể có đến nước đi tốt nhất trong trạng thái đó. Sau đó, máy tính có thể chơi hoàn hảo bằng cách tra cứu nước đi đúng trong bảng này. Bảng được xây dựng bằng **tìm kiếm minimax thoái lui**: bắt đầu bằng cách xem xét tất cả các cách đặt các quân KBNK lên bàn cờ. Một số vị trí là chiến thắng cho trắng; đánh dấu chúng như vậy. Sau đó, đảo ngược các quy tắc của cờ vua để thực hiện các nước đi ngược lại thay vì các nước đi. Bất kỳ nước đi nào của Trắng mà, bất kể Đen đáp lại bằng nước đi nào, kết thúc ở một vị trí được đánh dấu là chiến thắng, cũng phải là một chiến thắng. Tiếp tục tìm kiếm này cho đến khi tất cả các vị trí có

thể có được giải quyết là thắng, thua hoặc hòa, và bạn có một bảng tra cứu không thể sai lầm cho tất cả các ván đấu cuối cùng với các quân cờ đó. Điều này đã được thực hiện không chỉ cho các ván đấu cuối cùng KBNK, mà còn cho tất cả các ván đấu cuối cùng với bảy hoặc ít hơn quân cờ. Các bảng chứa 400 nghìn tỷ vị trí. Một bảng tám quân cờ sẽ yêu cầu 40 triệu tỷ vị trí.

6.4 Tìm kiếm cây Monte Carlo

Trò chơi Cờ vây minh họa hai điểm yếu lớn của tìm kiếm cây alpha-beta heuristic: Thứ nhất, Cờ vây có hệ số phân nhánh bắt đầu từ 361, có nghĩa là tìm kiếm alpha-beta sẽ chỉ giới hạn ở 4 hoặc 5 ply. Thứ hai, rất khó để xác định một hàm đánh giá tốt cho Cờ vây vì giá trị vật chất không phải là một chỉ số mạnh và hầu hết các vị trí đang thay đổi cho đến khi kết thúc ván đấu. Để đối phó với hai thách thức này, các chương trình Cờ vây hiện đại đã từ bỏ tìm kiếm alpha-beta và thay vào đó sử dụng một chiến lược gọi là **tìm kiếm cây Monte Carlo (MCTS)**.

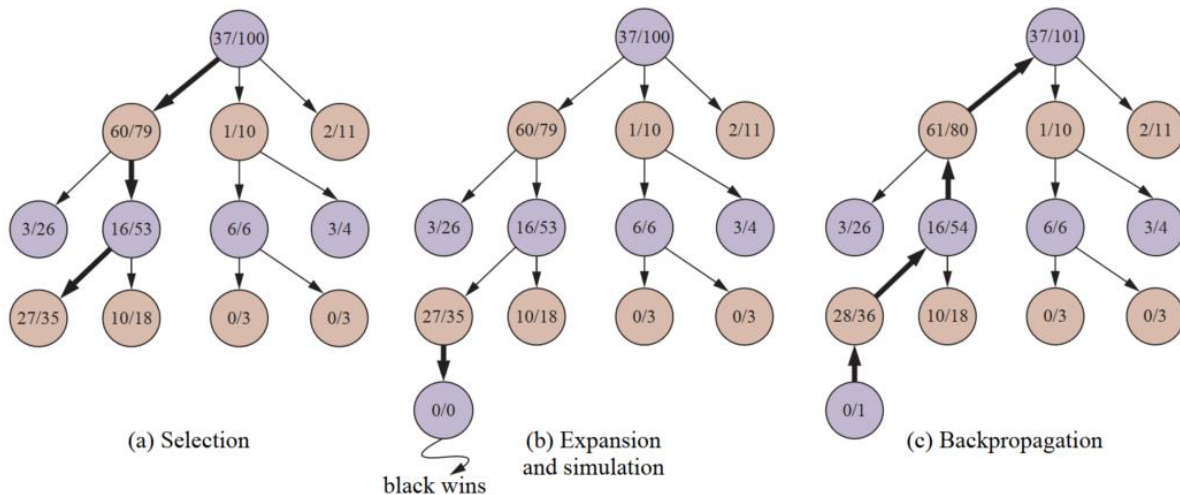
Chiến lược MCTS cơ bản không sử dụng hàm đánh giá heuristic. Thay vào đó, giá trị của một trạng thái được ước tính là tiện ích trung bình qua một số lần **mô phỏng** các ván đấu hoàn chỉnh bắt đầu từ trạng thái đó. Một mô phỏng (cũng được gọi là **playout** hoặc **rollout**) chọn nước đi đầu tiên cho một người chơi, sau đó cho người chơi kia, lặp lại cho đến khi đạt được một vị trí kết thúc. Tại thời điểm đó, các quy tắc của trò chơi (không phải heuristic dễ sai lầm) xác định ai đã thắng hoặc thua, và với tỷ số nào. Đối với các trò chơi mà kết quả duy nhất là thắng hoặc thua, “tiện ích trung bình” cũng giống như “tỷ lệ thắng”.

Làm thế nào để chúng ta chọn những nước đi nào để thực hiện trong quá trình playout? Nếu chúng ta chỉ chọn ngẫu nhiên, thì sau nhiều lần mô phỏng, chúng ta sẽ nhận được câu trả lời cho câu hỏi “nước đi tốt nhất là gì nếu cả hai người chơi đều chơi ngẫu nhiên?”. Đối với một số trò chơi đơn giản, đó tình cờ là cùng một câu trả lời với “nước đi tốt nhất là gì nếu cả hai người chơi đều chơi tốt?”, nhưng đối với hầu hết các trò chơi thì không. Để có được thông tin hữu ích từ playout, chúng ta cần một **chính sách playout** thiên về các nước đi tốt. Đối với Cờ vây và các trò chơi khác, các chính sách playout đã được học thành công từ việc tự chơi bằng cách sử dụng mạng nơ-ron. Đôi khi, các heuristic cụ thể của trò chơi được sử dụng, chẳng hạn như “xem xét các nước đi bắt quân” trong cờ vua hoặc “chiếm ô góc” trong Othello.

Với một chính sách playout, tiếp theo chúng ta cần quyết định hai điều: chúng ta bắt đầu playout từ những vị trí nào, và chúng ta phân bổ bao nhiêu playout cho mỗi vị trí? Câu trả lời đơn giản nhất, được gọi là **tìm kiếm Monte Carlo thuần túy**, là thực hiện N mô phỏng bắt đầu từ trạng thái hiện tại của trò chơi, và theo dõi nước đi nào có thể có từ vị trí hiện tại có tỷ lệ thắng cao nhất.

Đối với một số trò chơi ngẫu nhiên, điều này hội tụ đến lối chơi tối ưu khi N tăng lên, nhưng đối với hầu hết các trò chơi thì điều đó là không đủ—chúng ta cần một **chính sách lựa chọn** tập trung một cách có chọn lọc các tài nguyên tính toán vào các phần quan trọng của cây trò chơi. Nó cân bằng hai yếu tố: **khám phá** các trạng thái đã có ít playout, và **khai thác** các trạng thái đã làm tốt trong các playout trong quá khứ, để có được một ước tính chính xác hơn về giá trị của chúng. (Xem Mục 16.3 để biết thêm về sự đánh đổi giữa khám phá/khai thác.) Tìm kiếm cây Monte Carlo làm điều đó bằng cách duy trì một cây tìm kiếm và phát triển nó trên mỗi lần lặp của bốn bước sau, như được hiển thị trong Hình 6.10:

- Lựa chọn:** Bắt đầu từ nút gốc của cây tìm kiếm, chúng ta chọn một nước đi (được hướng dẫn bởi chính sách lựa chọn), dẫn đến một nút kế thừa, và lặp lại quá trình đó, di chuyển xuống cây đến một nút lá. Hình 6.10(a) cho thấy một cây tìm kiếm với nút gốc đại diện cho một trạng thái mà trắng vừa di chuyển, và trắng đã thắng 37 trong số 100 playout đã được thực hiện cho đến nay. Mũi tên dày cho thấy việc lựa chọn một nước đi của đen dẫn đến một nút mà đen đã thắng 60/79 playout. Đây là tỷ lệ thắng tốt nhất trong số ba nước đi, vì vậy việc chọn nó là một ví dụ về khai thác. Nhưng cũng hợp lý khi chọn nút 2/11 vì mục đích khám phá—với chỉ 11 playout, nút này vẫn có sự không chắc chắn cao trong đánh giá của nó, và có thể cuối cùng là tốt nhất nếu chúng ta có thêm thông tin về nó. Lựa chọn tiếp tục đến nút lá được đánh dấu 27/35.
- Mở rộng:** Chúng ta phát triển cây tìm kiếm bằng cách tạo một con mới của nút đã chọn; Hình 6.10(b) cho thấy nút mới được đánh dấu với 0/0. (Một số phiên bản tạo ra nhiều hơn một con trong bước này.)
- Mô phỏng:** Chúng ta thực hiện một playout từ nút con mới được tạo, chọn nước đi cho cả hai người chơi theo chính sách playout. Những nước đi này không được ghi lại trong cây tìm kiếm. Trong hình, kết quả mô phỏng là một chiến thắng cho đen.
- Lan truyền ngược:** Bây giờ chúng ta sử dụng kết quả của mô phỏng để cập nhật tất cả các nút cây tìm kiếm đi lên đến nút gốc. Vì đen đã thắng playout, các nút của đen được tăng cả số lần thắng và số lần playout, vì vậy 27/35 trở thành 28/36 và 60/79 trở thành 61/80. Vì trắng đã thua, các nút của trắng chỉ được tăng số lần playout, vì vậy 16/53 trở thành 16/54 và nút gốc 37/100 trở thành 37/101.



Hình 6.10 Một lần lặp của quá trình chọn một nước đi bằng tìm kiếm cây Monte Carlo (MCTS) sử dụng số liệu lựa chọn UCT, được hiển thị sau khi 100 lần lặp đã được thực hiện. Trong (a), chúng ta chọn các nước đi, đi xuống hết cây, kết thúc tại nút lá được đánh dấu 27/35 (cho 27 lần thắng của đen trong số 35 playout). Trong (b), chúng ta mở rộng nút đã chọn và thực hiện một mô phỏng (playout), kết thúc bằng một chiến thắng cho đen. Trong (c), kết quả của mô phỏng được lan truyền ngược lên cây.

function MONTE-CARLO-TREE-SEARCH(state) returns an action

```
tree ← NODE(state)
while Is-Time-Remaining() do
  leaf ← Select(tree)
  child ← Expand(leaf)
  result ← Simulate(child)
  Back-Propagate(result, child)
return the move in ACTIONS(state) whose node has highest number of playouts
```

Hình 6.11 Thuật toán tìm kiếm cây Monte Carlo. Một cây trò chơi, *tree*, được khởi tạo, và sau đó chúng ta lặp lại một chu trình SELECT / EXPAND / SIMULATE / BACK-PROPAGATE cho đến khi chúng ta hết thời gian, và trả về nước đi đã dẫn đến nút có số playout cao nhất.

Chúng ta lặp lại bốn bước này cho một số lần lặp đã định, hoặc cho đến khi thời gian được phân bổ đã hết, và sau đó trả về nước đi có số playout cao nhất.

Một chính sách lựa chọn rất hiệu quả được gọi là “**giới hạn tin cậy trên áp dụng cho cây**” hoặc **UCT**. Chính sách này xếp hạng mỗi nước đi có thể có dựa trên một công thức giới hạn tin cậy trên gọi là **UCB1**. (Xem Mục 16.3.3 để biết thêm chi tiết.) Đối với một nút n , công thức là:

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\log N(PARENT(n))}{N(n)}}$$

trong đó $U(n)$ là tổng tiện ích của tất cả các playout đã đi qua nút n , $N(n)$ là số playout qua nút n , và $PARENT(n)$ là nút cha của n trong cây. Do đó, $\frac{U(n)}{N(n)}$ là thuật ngữ khai thác: tiện ích trung bình của n . Thuật ngữ với căn bậc hai là thuật ngữ khám phá: nó có số đếm $N(n)$ trong mẫu số, có nghĩa là thuật ngữ sẽ cao đối với các nút chỉ được khám phá một vài lần. Trong tử số, nó có logarit của số lần chúng ta đã khám phá cha của n . Điều này có nghĩa là nếu chúng ta đang chọn n một tỷ lệ phần trăm khác không nào đó của thời gian, thuật ngữ khám phá sẽ tiến về không khi số đếm tăng lên, và cuối cùng các playout được đưa cho nút có tiện ích trung bình cao nhất. C là một hằng số cân bằng khai thác và khám phá. Có một lập luận lý thuyết rằng C nên là $\sqrt{2}$, nhưng trong thực tế, các lập trình viên trò chơi thử nhiều giá trị khác nhau cho C và chọn giá trị hoạt động tốt nhất. (Một số chương trình sử dụng các công thức hơi khác; ví dụ, AlphaGo thêm vào một thuật ngữ cho xác suất nước đi, được tính bằng một mạng nơ-ron được đào tạo từ việc tự chơi trong quá khứ.) Với $C = 1.4$, nút 60/79 trong Hình 6.10 có điểm UCB1 cao nhất, nhưng với $C = 1.5$, nó sẽ là nút 2/11.

Hình 6.11 cho thấy thuật toán MCTS UCT hoàn chỉnh. Khi các lần lặp kết thúc, nước đi có số playout cao nhất được trả về. Bạn có thể nghĩ rằng sẽ tốt hơn nếu trả về nút có tiện ích trung bình cao nhất, nhưng ý tưởng là một nút với 65/100 lần thắng tốt hơn một nút với 2/3 lần thắng, vì nút sau có nhiều sự không chắc chắn. Trong mọi trường hợp, công thức UCB1 đảm bảo rằng nút có nhiều playout nhất gần như luôn là nút có tỷ lệ thắng cao nhất, vì quá trình lựa chọn ủng hộ tỷ lệ thắng ngày càng nhiều hơn khi số lượng playout tăng lên.

Thời gian để tính toán một playout là tuyến tính, không phải hàm mũ, theo độ sâu của cây trò chơi, vì chỉ có một nước đi được thực hiện tại mỗi điểm lựa chọn. Điều đó cho chúng ta nhiều thời gian cho nhiều playout. Ví dụ: hãy xem xét một trò chơi có hệ số phân nhánh là 32, nơi ván đấu trung

biên kéo dài 100 ply. Nếu chúng ta có đủ sức mạnh tính toán để xem xét một tỷ trạng thái trò chơi trước khi chúng ta phải thực hiện một nước đi, thì minimax có thể tìm kiếm sâu 6 ply, alpha-beta với sắp xếp nước đi hoàn hảo có thể tìm kiếm 12 ply, và tìm kiếm Monte Carlo có thể thực hiện 10 triệu playout. Cách tiếp cận nào sẽ tốt hơn? Điều đó phụ thuộc vào độ chính xác của hàm heuristic so với các chính sách lựa chọn và playout.

Quan niệm thông thường là tìm kiếm Monte Carlo có lợi thế hơn alpha-beta cho các trò chơi như Cờ vây, nơi hệ số phân nhánh rất cao (và do đó alpha-beta không thể tìm kiếm đủ sâu), hoặc khi khó xác định một hàm đánh giá tốt. Điều mà alpha-beta làm là chọn đường dẫn đến một nút có điểm hàm đánh giá cao nhất có thể đạt được, với giả định rằng đối thủ sẽ cố gắng giảm thiểu điểm số. Do đó, nếu hàm đánh giá không chính xác, alpha-beta sẽ không chính xác. Một tính toán sai trên một nút duy nhất có thể dẫn alpha-beta đến việc chọn (hoặc tránh) một cách sai lầm một đường dẫn đến nút đó. Nhưng tìm kiếm Monte Carlo dựa trên tổng hợp của nhiều playout, và do đó không dễ bị tổn thương bởi một lỗi duy nhất. Có thể kết hợp MCTS và các hàm đánh giá bằng cách thực hiện một playout cho một số nước đi nhất định, nhưng sau đó cắt bớt playout và áp dụng một hàm đánh giá.

Cũng có thể kết hợp các khía cạnh của tìm kiếm alpha-beta và Monte Carlo. Ví dụ, trong các trò chơi có thể kéo dài nhiều nước đi, chúng ta có thể muốn sử dụng **kết thúc playout sớm**, trong đó chúng ta dừng một playout đang mất quá nhiều nước đi, và hoặc đánh giá nó bằng một hàm đánh giá heuristic hoặc chỉ tuyên bố đó là một trận hòa.

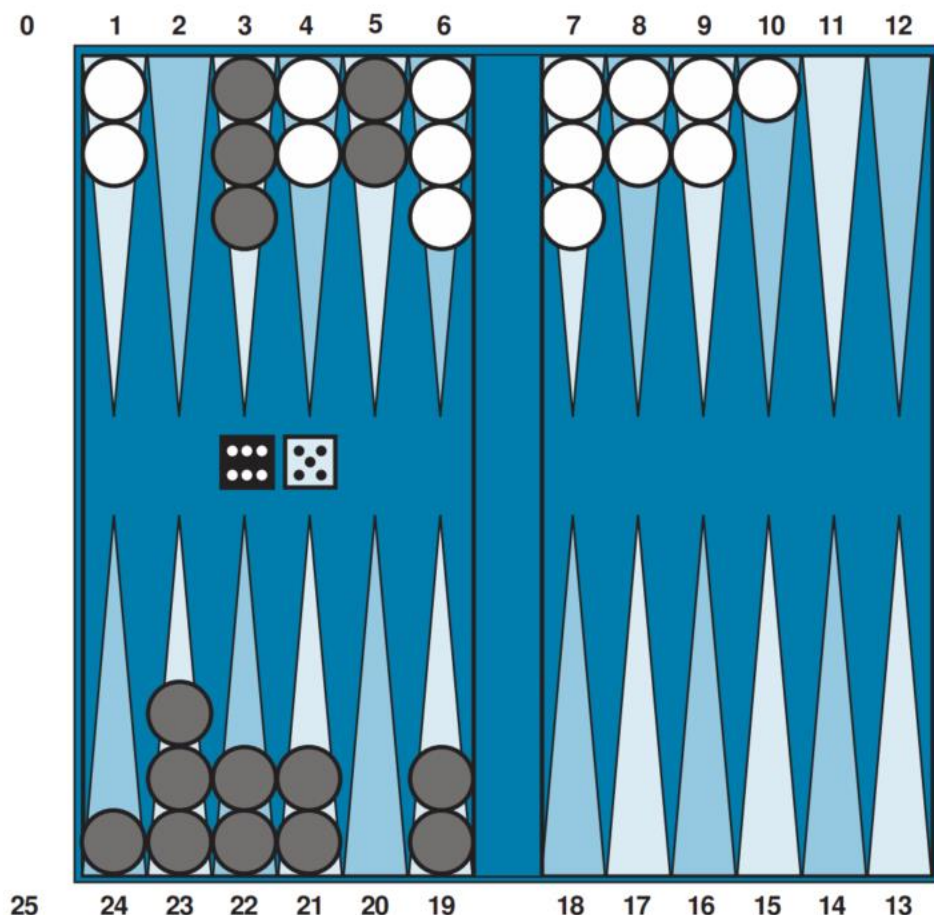
Tìm kiếm Monte Carlo có thể được áp dụng cho các trò chơi hoàn toàn mới, trong đó không có kinh nghiệm để dựa vào để xác định một hàm đánh giá. Miễn là chúng ta biết các quy tắc của trò chơi, tìm kiếm Monte Carlo không cần bất kỳ thông tin bổ sung nào. Các chính sách lựa chọn và playout có thể tận dụng tốt kiến thức chuyên môn được tạo thủ công khi nó có sẵn, nhưng các chính sách tốt có thể được học bằng cách sử dụng mạng nơ-ron được đào tạo chỉ bằng việc tự chơi.

Tìm kiếm Monte Carlo có một nhược điểm khi có khả năng một nước đi duy nhất có thể thay đổi cục diện của trò chơi, bởi vì bản chất ngẫu nhiên của tìm kiếm Monte Carlo có nghĩa là nó có thể không xem xét nước đi đó. Nói cách khác, cắt tỉa Loại B trong tìm kiếm Monte Carlo có nghĩa là một đường lối chơi quan trọng có thể không được khám phá chút nào. Tìm kiếm Monte Carlo cũng có một nhược điểm khi có các trạng thái trò chơi “rõ ràng” là một chiến thắng cho một bên này hay bên kia (theo kiến thức của con người và một hàm đánh giá), nhưng vẫn sẽ mất nhiều nước đi trong một playout để xác minh người chiến thắng. Người ta đã giữ quan niệm rằng tìm kiếm alpha-beta phù hợp hơn cho các trò chơi như cờ vua với hệ số phân nhánh thấp và các hàm đánh giá tốt, nhưng gần đây các cách tiếp cận Monte Carlo đã chứng minh thành công trong cờ vua và các trò chơi khác.

Ý tưởng chung về việc mô phỏng các nước đi trong tương lai, quan sát kết quả và sử dụng kết quả để xác định nước đi nào là tốt là một loại **học tăng cường**, được đề cập trong Chương 23.

6.5 Trò chơi ngẫu nhiên

Trò chơi ngẫu nhiên đưa chúng ta đến gần hơn một chút với sự không thể đoán trước của đời thực bằng cách bao gồm một yếu tố ngẫu nhiên, chẳng hạn như việc tung xúc xắc. Cờ tào cáo là một trò chơi ngẫu nhiên điển hình kết hợp may mắn và kỹ năng. Trong vị trí cờ tào cáo của Hình 6.12, Đen đã tung được 6-5 và có bốn nước đi có thể có (mỗi nước đi di chuyển một quân cờ về phía trước (theo chiều kim đồng hồ) 5 vị trí, và một quân cờ về phía trước 6 vị trí).

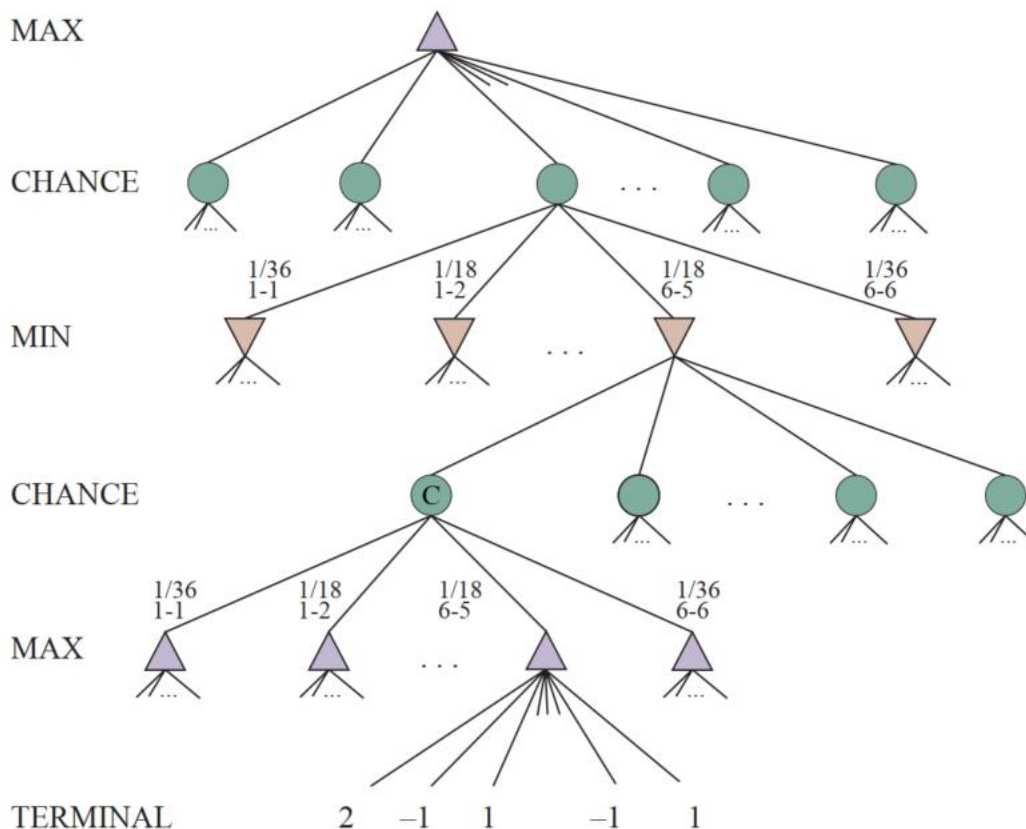


Hình 6.12 Một vị trí cờ tào cáo điển hình. Mục tiêu của trò chơi là di chuyển tất cả các quân cờ của mình ra khỏi bàn cờ. Đen di chuyển theo chiều kim đồng hồ về phía 25, và Trắng di chuyển ngược chiều kim đồng hồ về phía 0. Một quân cờ có thể di chuyển đến bất kỳ vị trí nào trừ khi có nhiều quân cờ của đối thủ ở đó; nếu có một quân cờ của đối thủ, nó sẽ bị bắt và phải bắt đầu lại. Trong vị trí được hiển thị, Đen đã tung được 6-5 và phải chọn trong số bốn nước đi hợp lệ: (5-11, 5-10), (5-11, 19-24), (5-10, 10-16), và (5-11, 11-16), trong đó ký hiệu (5-11, 11-16) có nghĩa là di chuyển một quân cờ từ vị trí 5 đến 11, và sau đó di chuyển một quân cờ từ 11 đến 16.

Tại thời điểm này, Đen biết những nước đi nào có thể được thực hiện, nhưng không biết Trắng sẽ tung ra những gì và do đó không biết các nước đi hợp lệ của Trắng sẽ là gì. Điều đó có nghĩa là Đen không thể xây dựng một cây trò chơi tiêu chuẩn như chúng ta đã thấy trong cờ vua và cờ caro. Một cây trò chơi trong cờ tào cáo phải bao gồm các **nút ngẫu nhiên** ngoài các nút MAX và MIN. Các nút ngẫu nhiên được hiển thị dưới dạng hình tròn trong Hình 6.13. Các nhánh dẫn từ mỗi nút ngẫu nhiên biểu thị các lần tung xúc xắc có thể có; mỗi nhánh được gán nhãn với lần tung và xác suất của nó. Có 36 cách để tung hai con xúc xắc, mỗi cách có khả năng như nhau; nhưng vì 6-5 cũng giống như 5-6, chỉ có 21 lần tung khác biệt. Sáu lần tung đôi (1-1 đến 6-6) mỗi lần có xác suất là $1/36$, vì vậy chúng ta nói $P(1 - 1) = 1/36$. 15 lần tung khác biệt còn lại mỗi lần có xác suất là $1/18$.

Bước tiếp theo là hiểu cách đưa ra các quyết định đúng. Rõ ràng, chúng ta vẫn muốn chọn nước đi dẫn đến vị trí tốt nhất. Tuy nhiên, các vị trí không có giá trị minimax xác định. Thay vào đó, chúng

ta chỉ có thể tính toán **giá trị kỳ vọng** của một vị trí: giá trị trung bình trên tất cả các kết quả có thể có của các nút ngẫu nhiên.



Hình 6.13 Cây trò chơi sơ đồ cho một vị trí cờ tào cáo.

Điều này dẫn chúng ta đến **giá trị expectiminimax** cho các trò chơi có nút ngẫu nhiên, một sự tổng quát hóa của giá trị minimax cho các trò chơi tất định. Các nút kết thúc và các nút MAX và MIN hoạt động chính xác giống như trước đây (với lưu ý rằng các nước đi hợp lệ cho MAX và MIN sẽ phụ thuộc vào kết quả của lần tung xúc xắc trong nút ngẫu nhiên trước đó). Đối với các nút ngẫu nhiên, chúng ta tính toán giá trị kỳ vọng, là tổng của giá trị trên tất cả các kết quả, được trọng số bằng xác suất của mỗi hành động ngẫu nhiên:

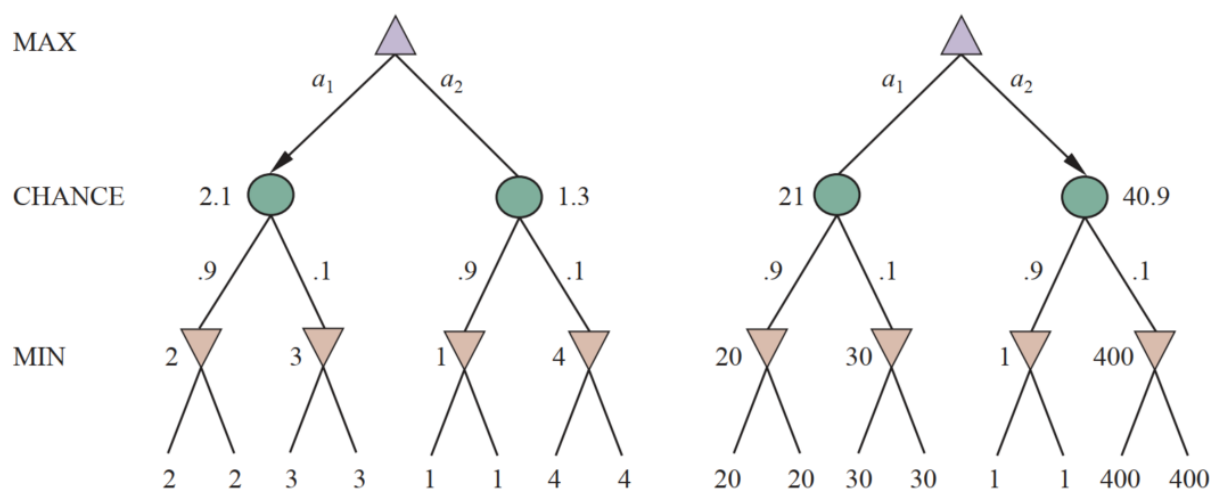
$$\text{EXPECTIMINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \\ \sum_r P(r) \text{EXPECTIMINIMAX}(\text{RESULT}(s, r)) & \text{if TO-MOVE}(s) = \text{CHANCE} \end{cases}$$

trong đó r đại diện cho một lần tung xúc xắc có thể có (hoặc sự kiện ngẫu nhiên khác) và $\text{RESULT}(s, r)$ là cùng một trạng thái với s , với thực tế bổ sung rằng kết quả của lần tung xúc xắc là r .

6.5.1 Các hàm đánh giá cho trò chơi may rủi

Cũng như với minimax, xấp xỉ rõ ràng cần thực hiện với expectiminimax là cắt bớt tìm kiếm tại một số điểm và áp dụng một hàm đánh giá cho mỗi nút lá. Người ta có thể nghĩ rằng các hàm đánh giá cho các trò chơi như cờ tào cáo nên giống như các hàm đánh giá cho cờ vua—chúng chỉ cần gán các giá trị cao hơn cho các vị trí tốt hơn. Nhưng trên thực tế, sự hiện diện của các nút ngẫu nhiên có nghĩa là người ta phải cẩn thận hơn về ý nghĩa của các giá trị.

Hình 6.14 cho thấy điều gì xảy ra: với một hàm đánh giá gán các giá trị $[1,2,3,4]$ cho các nút lá, nước đi a_1 là tốt nhất; với các giá trị $[1,20,30,400]$, nước đi a_2 là tốt nhất. Do đó, chương trình hoạt động hoàn toàn khác nếu chúng ta thực hiện một thay đổi đối với một số giá trị đánh giá, ngay cả khi thứ tự ưu tiên vẫn giữ nguyên.



Hình 6.14 Một phép biến đổi bảo toàn thứ tự trên các giá trị lá làm thay đổi nước đi tốt nhất.

Hóa ra, để tránh vấn đề này, hàm đánh giá phải trả về các giá trị là **một phép biến đổi tuyến tính dương** của xác suất chiến thắng (hoặc của tiện ích kỳ vọng, đối với các trò chơi có kết quả khác ngoài thắng/thua). Mỗi quan hệ với xác suất này là một thuộc tính quan trọng và tổng quát của các tình huống liên quan đến sự không chắc chắn, và chúng ta sẽ thảo luận thêm về nó trong Chương 15.

Nếu chương trình biết trước tất cả các lần tung xúc xắc sẽ xảy ra trong phần còn lại của trò chơi, việc giải một trò chơi với xúc xắc sẽ giống như giải một trò chơi không có xúc xắc, mà minimax thực hiện trong thời gian $O(b^m)$, trong đó b là hệ số phân nhánh và m là độ sâu tối đa của cây trò chơi. Bởi vì expectiminimax cũng đang xem xét tất cả các chuỗi tung xúc xắc có thể có, nó sẽ mất $O(b^m n^m)$, trong đó n là số lần tung khác biệt.

Ngay cả khi việc tìm kiếm bị giới hạn ở một độ sâu nhỏ d , chi phí bổ sung so với minimax làm cho việc xem xét việc nhìn xa hơn trong hầu hết các trò chơi may rủi là không thực tế. Trong cờ tào cáo, n là 21 và b thường là khoảng 20, nhưng trong một số tình huống có thể lên tới 4000 đối với các lần tung xúc xắc đôi. Chúng ta có thể chỉ quản lý được ba ply tìm kiếm.

Một cách khác để suy nghĩ về vấn đề là như thế này: lợi thế của alpha-beta là nó bỏ qua những diễn biến trong tương lai mà sẽ không xảy ra, với lối chơi tốt nhất. Do đó, nó tập trung vào các sự

kiện có khả năng xảy ra. Nhưng trong một trò chơi mà mỗi nước đi có một lần tung hai con xúc xắc đi trước, không có chuỗi nước đi có khả năng xảy ra; ngay cả nước đi có khả năng xảy ra nhất cũng chỉ xảy ra $2/36$ lần, bởi vì để nước đi diễn ra, xúc xắc trước tiên phải ra đúng cách để nó hợp lệ. Đây là một vấn đề chung bất cứ khi nào sự không chắc chắn xuất hiện: các khả năng được nhân lên một cách khủng khiếp, và việc hình thành các kế hoạch hành động chi tiết trở nên vô nghĩa vì thế giới có thể sẽ không diễn ra như vậy.

Bạn có thể đã nghĩ rằng một cái gì đó giống như cắt tia alpha-beta có thể được áp dụng cho cây trò chơi với các nút ngẫu nhiên. Hóa ra là nó có thể. Phân tích cho các nút MIN và MAX không thay đổi, nhưng chúng ta cũng có thể cắt tia các nút ngẫu nhiên, bằng một chút khéo léo. Hãy xem xét nút ngẫu nhiên C trong Hình 6.13 và điều gì xảy ra với giá trị của nó khi chúng ta đánh giá các con của nó. Có thể tìm thấy một giới hạn trên cho giá trị của C trước khi chúng ta đã xem xét tất cả các con của nó không? (Nhớ lại rằng đây là điều mà alpha-beta cần để cắt tia một nút và cây con của nó.)

Ban đầu, có vẻ như điều đó là không thể vì giá trị của C là giá trị trung bình của các giá trị của các con của nó, và để tính toán giá trị trung bình của một tập hợp các số, chúng ta phải xem xét tất cả các số. Nhưng nếu chúng ta đặt giới hạn cho các giá trị có thể có của hàm tiện ích, thì chúng ta có thể đi đến các giới hạn cho giá trị trung bình mà không cần xem xét mọi số. Ví dụ, giả sử tất cả các giá trị tiện ích đều nằm giữa -2 và $+2$; khi đó giá trị của các nút lá bị giới hạn, và lần lượt chúng ta có thể đặt một giới hạn trên cho giá trị của một nút ngẫu nhiên mà không cần xem xét tất cả các con của nó.

Trong các trò chơi mà hệ số phân nhánh cho các nút ngẫu nhiên cao—hãy xem xét một trò chơi như Yahtzee, nơi bạn tung 5 con xúc xắc trong mỗi lượt—bạn có thể muốn xem xét cắt tia về phía trước lấy mẫu một số lượng nhỏ hơn các nhánh ngẫu nhiên có thể có. Hoặc bạn có thể muốn tránh sử dụng hàm đánh giá hoàn toàn, và thay vào đó chọn tìm kiếm cây Monte Carlo, trong đó mỗi playout bao gồm các lần tung xúc xắc ngẫu nhiên.

6.6 Trò chơi quan sát một phần

Bobby Fischer tuyên bố rằng “cờ vua là chiến tranh,” nhưng cờ vua thiếu ít nhất một đặc điểm chính của các cuộc chiến tranh thực sự, đó là khả năng **quan sát một phần**. Trong “sương mù chiến tranh,” vị trí của các đơn vị địch thường không được biết cho đến khi được tiết lộ bằng tiếp xúc trực tiếp. Do đó, chiến tranh bao gồm việc sử dụng trinh sát và gián điệp để thu thập thông tin và sử dụng ngụy trang và lừa dối để làm bối rối kẻ thù.

Các **trò chơi quan sát một phần** chia sẻ những đặc điểm này và do đó khác biệt về mặt định tính so với các trò chơi trong các phần trước. Các trò chơi điện tử như StarCraft đặc biệt thách thức, vì chúng vừa có thể quan sát một phần, vừa có nhiều tác nhân, không tất định, năng động và không xác định.

Trong các trò chơi quan sát một phần tất định, sự không chắc chắn về trạng thái của bàn cờ hoàn toàn phát sinh từ việc thiếu quyền truy cập vào các lựa chọn của đối thủ. Lớp này bao gồm các trò chơi của trẻ em như Tàu chiến (trong đó các con tàu của mỗi người chơi được đặt ở những vị trí ẩn đối với đối thủ) và Cờ chiến lược (trong đó vị trí quân cờ được biết nhưng loại quân cờ bị ẩn). Chúng ta sẽ xem xét trò chơi **Kriegspiel**, một biến thể quan sát một phần của cờ vua trong đó các quân cờ hoàn toàn vô hình đối với đối thủ. Các trò chơi khác cũng có các phiên bản quan sát một phần: Cờ vây ma, cờ ca-rô ma và Shogi màn hình.

6.6.1 Kriegspiel: Cờ vua quan sát một phần

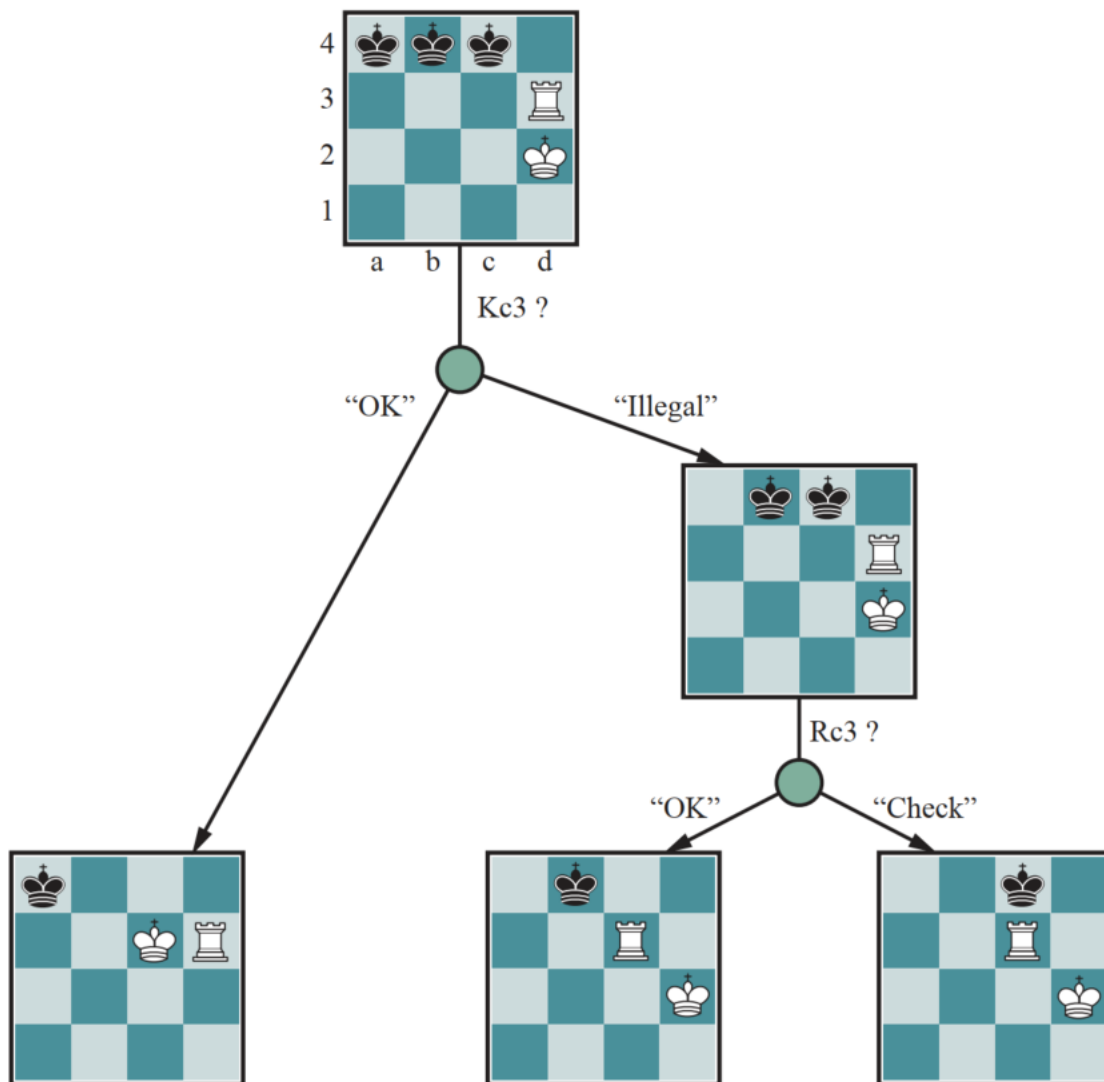
Các quy tắc của Kriegspiel như sau: Trắng và Đen mỗi người chỉ nhìn thấy một bàn cờ chỉ chứa các quân cờ của riêng họ. Một **trọng tài**, người có thể nhìn thấy tất cả các quân cờ, phân xử trò chơi và định kỳ đưa ra các thông báo mà cả hai người chơi đều nghe thấy. Đầu tiên, Trắng đề xuất với trọng tài một nước đi sẽ hợp lệ nếu không có quân cờ đen. Nếu các quân cờ đen ngăn cản nước đi, trọng tài thông báo “bất hợp pháp”, và Trắng tiếp tục đề xuất các nước đi cho đến khi tìm thấy một nước hợp lệ—học hỏi thêm về vị trí của các quân cờ của Đen trong quá trình đó.

Khi một nước đi hợp lệ được đề xuất, trọng tài thông báo một hoặc nhiều điều sau đây: “Bắt quân trên ô X” nếu có một nước bắt, và “Chiếu bởi D” nếu vua đen bị chiếu, trong đó D là hướng của nước chiếu, và có thể là một trong “Mã,” “Hàng,” “Cột,” “Đường chéo dài,” hoặc “Đường chéo ngắn.” Nếu Đen bị chiếu hết hoặc bị bế tắc, trọng tài sẽ nói như vậy; nếu không, đến lượt Đen di chuyển.

Kriegspiel có vẻ bất khả thi một cách đáng sợ, nhưng con người xử lý nó khá tốt và các chương trình máy tính đang bắt đầu bắt kịp. Nó giúp gợi lại khái niệm về **trạng thái niềm tin** như được định nghĩa trong Mục 4.4 và được minh họa trong Hình 4.14—tập hợp tất cả các trạng thái bàn cờ có thể có về mặt logic dựa trên toàn bộ lịch sử các nhận thức cho đến nay. Ban đầu, trạng thái niềm tin của Trắng là một tập hợp đơn vì các quân cờ của Đen chưa di chuyển. Sau khi Trắng thực hiện một nước đi và Đen đáp lại, trạng thái niềm tin của Trắng chứa 20 vị trí, vì Đen có 20 câu trả lời cho bất kỳ nước đi khai cuộc nào. Theo dõi trạng thái niềm tin khi trò chơi diễn ra chính xác là vấn đề của **ước tính trạng thái**, trong đó bước cập nhật được đưa ra trong Phương trình (4.6) trên trang 150. Chúng ta có thể ánh xạ ước tính trạng thái Kriegspiel trực tiếp lên khuôn khổ quan sát một phần, không tất định của Mục 4.4 nếu chúng ta coi đối thủ là nguồn gốc của sự không tất định; nghĩa là, các KẾT QUẢ của nước đi của Trắng được cấu thành từ kết quả (có thể dự đoán được) của nước đi của chính Trắng và kết quả không thể đoán trước được đưa ra bởi câu trả lời của Đen.

Với một trạng thái niềm tin hiện tại, Trắng có thể hỏi, “Tôi có thể thắng ván đấu không?” Đối với một trò chơi quan sát một phần, khái niệm về một chiến lược bị thay đổi; thay vì chỉ định một nước đi để thực hiện cho mỗi nước đi có thể có của đối thủ, chúng ta cần một nước đi cho mọi chuỗi nhận thức có thể được nhận.

Đối với Kriegspiel, một chiến lược chiến thắng, hoặc **chiếu hết được đảm bảo**, là một chiến lược mà, đối với mỗi chuỗi nhận thức có thể có, dẫn đến một nước chiếu hết thực tế cho mọi trạng thái bàn cờ có thể có trong trạng thái niềm tin hiện tại, bất kể đối thủ di chuyển như thế nào. Với định nghĩa này, trạng thái niềm tin của đối thủ là không liên quan—chiến lược phải hoạt động ngay cả khi đối thủ có thể nhìn thấy tất cả các quân cờ. Điều này giúp đơn giản hóa đáng kể việc tính toán. Hình 6.15 cho thấy một phần của một nước chiếu hết được đảm bảo cho tàn cuộc KRK (vua và xe đấu với vua). Trong trường hợp này, Đen chỉ có một quân cờ (vua), vì vậy một trạng thái niềm tin cho Trắng có thể được hiển thị trong một bàn cờ duy nhất bằng cách đánh dấu mỗi vị trí có thể có của vua Đen.



Hình 6.15 Một phần của một nước chiếu hết được đảm bảo trong tàn cuộc KRK, được hiển thị trên một bàn cờ thu nhỏ. Trong trạng thái niềm tin ban đầu, vua của Đen ở một trong ba vị trí có thể có. Bằng cách kết hợp các nước đi thăm dò, chiến lược này thu hẹp điều này xuống còn một. Việc hoàn thành nước chiếu hết được để lại như một bài tập.

Thuật toán tìm kiếm AND-OR tổng quát có thể được áp dụng cho không gian trạng thái niềm tin để tìm các nước chiếu hết được đảm bảo, giống như trong Mục 4.4. Thuật toán trạng thái niềm tin tăng dần được đề cập trong Mục 4.4.2 thường tìm thấy các nước chiếu hết giữa trận đấu lên đến độ sâu 9—vượt xa khả năng của hầu hết người chơi con người.

Ngoài các nước chiếu hết được đảm bảo, Kriegspiel thừa nhận một khái niệm hoàn toàn mới không có ý nghĩa trong các trò chơi có thể quan sát đầy đủ: **chiếu hết xác suất**. Các nước chiếu hết như vậy vẫn được yêu cầu phải hoạt động trong mọi trạng thái bàn cờ trong trạng thái niềm tin; chúng là xác suất đối với việc ngẫu nhiên hóa các nước đi của người chơi chiến thắng. Để có được ý tưởng cơ bản, hãy xem xét vấn đề tìm một vua đen đơn độc chỉ bằng cách sử dụng vua trắng. Chỉ bằng cách di chuyển ngẫu nhiên, vua trắng cuối cùng sẽ va chạm với vua đen ngay cả khi vua đen

cố gắng tránh số phận này, vì Đen không thể tiếp tục đoán đúng các nước đi né tránh vô thời hạn. Theo thuật ngữ của lý thuyết xác suất, việc phát hiện xảy ra với xác suất 1.

Tàn cuộc KBNK—vua, tượng và mã đấu với vua—được thắng theo nghĩa này; Trắng đưa cho Đen một chuỗi lựa chọn ngẫu nhiên vô hạn, trong đó Đen sẽ đoán sai một và tiết lộ vị trí của mình, dẫn đến chiếu hết. Mặt khác, tàn cuộc KBBK được thắng với xác suất $1 - \epsilon$. Trắng chỉ có thể buộc thắng bằng cách để một trong các con tượng của mình không được bảo vệ trong một nước đi. Nếu Đen tình cờ ở đúng vị trí và bắt tượng (một nước đi sẽ bất hợp pháp nếu các con tượng được bảo vệ), trò chơi sẽ hòa. Trắng có thể chọn thực hiện nước đi rủi ro tại một thời điểm được chọn ngẫu nhiên ở giữa một chuỗi rất dài, do đó giảm ϵ xuống một hằng số nhỏ tùy ý, nhưng không thể giảm ϵ xuống không.

Đôi khi một chiến lược chiếu hết hoạt động cho một số trạng thái bàn cờ trong trạng thái niềm tin hiện tại nhưng không phải các trạng thái khác. Việc thử một chiến lược như vậy có thể thành công, dẫn đến một nước **chiếu hết ngẫu nhiên**—ngẫu nhiên theo nghĩa là Trắng không thể biết rằng đó sẽ là chiếu hết—nếu các quân cờ của Đen tình cờ ở đúng vị trí. (Hầu hết các nước chiếu hết trong các ván đấu giữa con người đều có bản chất ngẫu nhiên này.) Ý tưởng này dẫn tự nhiên đến câu hỏi về khả năng một chiến lược nhất định sẽ thắng là bao nhiêu, lần lượt dẫn đến câu hỏi về khả năng mỗi trạng thái bàn cờ trong trạng thái niềm tin hiện tại là trạng thái bàn cờ thực sự là bao nhiêu.

Khuynh hướng đầu tiên của người ta có thể là đề xuất rằng tất cả các trạng thái bàn cờ trong trạng thái niềm tin hiện tại đều có khả năng như nhau—nhưng điều này không thể đúng. Ví dụ, hãy xem xét trạng thái niềm tin của Trắng sau nước đi đầu tiên của Đen. Theo định nghĩa (giả sử rằng Đen chơi tối ưu), Đen phải đã chơi một nước đi tối ưu, vì vậy tất cả các trạng thái bàn cờ do các nước đi không tối ưu tạo ra phải được gán xác suất bằng không.

► Lập luận này cũng không hoàn toàn đúng, bởi vì mục tiêu của mỗi người chơi không chỉ là di chuyển các quân cờ đến đúng ô mà còn là giảm thiểu thông tin mà đối thủ có về vị trí của họ. Chơi bất kỳ chiến lược “tối ưu” có thể dự đoán nào đều cung cấp cho đối thủ thông tin. Do đó, lối chơi tối ưu trong các trò chơi quan sát một phần đòi hỏi sự sẵn sàng chơi một cách ngẫu nhiên. (Đây là lý do tại sao các thanh tra vệ sinh nhà hàng thực hiện các chuyến kiểm tra ngẫu nhiên.) Điều này có nghĩa là thỉnh thoảng chọn các nước đi có vẻ “vốn dĩ” yếu—nhưng chúng có được sức mạnh từ chính sự không thể đoán trước của chúng, bởi vì đối thủ không có khả năng đã chuẩn bị bất kỳ sự phòng thủ nào chống lại chúng.

Từ những cân nhắc này, có vẻ như xác suất liên quan đến các trạng thái bàn cờ trong trạng thái niềm tin hiện tại chỉ có thể được tính toán khi biết một chiến lược ngẫu nhiên tối ưu; lần lượt, việc tính toán chiến lược đó dường như yêu cầu biết xác suất của các trạng thái khác nhau mà bàn cờ có thể ở. Tình huống khó xử này có thể được giải quyết bằng cách áp dụng khái niệm lý thuyết trò chơi về một **giải pháp cân bằng**, mà chúng ta theo đuổi thêm trong Chương 16. Một sự cân bằng chỉ định một chiến lược ngẫu nhiên tối ưu cho mỗi người chơi. Việc tính toán sự cân bằng quá tốn kém đối với Kriegspiel. Hiện tại, việc thiết kế các thuật toán hiệu quả cho lối chơi Kriegspiel tổng quát là một chủ đề nghiên cứu mở. Hầu hết các hệ thống thực hiện việc nhìn trước độ sâu giới hạn trong không gian trạng thái niềm tin của riêng họ, bỏ qua trạng thái niềm tin của đối thủ. Các hàm đánh giá giống với các hàm đánh giá cho trò chơi có thể quan sát được nhưng bao gồm một thành phần cho kích thước của trạng thái niềm tin—nhỏ hơn thì tốt hơn! Chúng ta sẽ quay lại các trò chơi quan sát một phần dưới chủ đề Lý thuyết trò chơi trong Mục 17.2.

6.6.2 Trò chơi bài

Các trò chơi bài như bridge, whist, hearts, và poker có đặc điểm quan sát một phần ngẫu nhiên, trong đó thông tin bị thiếu được tạo ra bởi việc chia bài ngẫu nhiên.

Thoạt nhìn, có vẻ như các trò chơi bài này giống như các trò chơi xúc xắc: các lá bài được chia ngẫu nhiên và xác định các nước đi có sẵn cho mỗi người chơi, nhưng tất cả “xúc xắc” đều được tung ngay từ đầu! Mặc dù phép loại suy này hóa ra không chính xác, nó gợi ý một thuật toán: coi phần đầu của trò chơi như một nút ngẫu nhiên với mọi lần chia bài có thể có là một kết quả, và sau đó sử dụng công thức **EXPECTIMINIMAX** để chọn nước đi tốt nhất. Lưu ý rằng trong cách tiếp cận này, nút ngẫu nhiên duy nhất là nút gốc; sau đó trò chơi trở nên có thể quan sát đầy đủ. Cách tiếp cận này đôi khi được gọi là **trung bình hóa sự thấu thị** vì nó giả định rằng một khi việc chia bài thực tế đã xảy ra, trò chơi trở nên có thể quan sát đầy đủ đối với cả hai người chơi. Mặc dù có sức hấp dẫn trực quan, chiến lược này có thể dẫn đến sai lầm. Hãy xem xét câu chuyện sau:

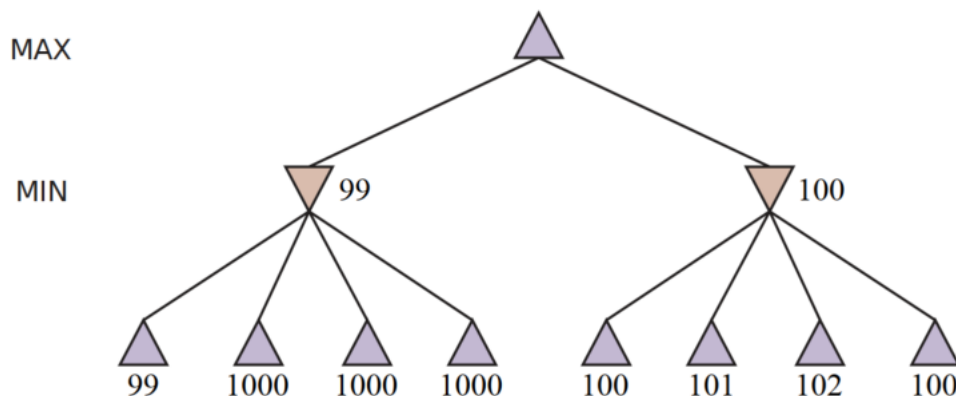
Ngày 1: Con đường A dẫn đến một hũ vàng; Con đường B dẫn đến một ngã ba. Bạn có thể thấy rằng ngã rẽ bên trái dẫn đến hai hũ vàng, và ngã rẽ bên phải dẫn đến việc bạn bị xe buýt cán qua.

Ngày 2: Con đường A dẫn đến một hũ vàng; Con đường B dẫn đến một ngã ba. Bạn có thể thấy rằng ngã rẽ bên phải dẫn đến hai hũ vàng, và ngã rẽ bên trái dẫn đến việc bạn bị xe buýt cán qua.

Ngày 3: Con đường A dẫn đến một hũ vàng; Con đường B dẫn đến một ngã ba. Bạn được cho biết rằng một ngã rẽ dẫn đến hai hũ vàng, và một ngã rẽ dẫn đến việc bạn bị xe buýt cán qua. Thật không may, bạn không biết ngã rẽ nào là cái nào.

Trung bình hóa sự thấu thị dẫn đến lý luận sau: vào Ngày 1, B là lựa chọn đúng; vào Ngày 2, B là lựa chọn đúng; vào Ngày 3, tình huống giống như Ngày 1 hoặc Ngày 2, vì vậy B vẫn phải là lựa chọn đúng.

Bây giờ chúng ta có thể thấy trung bình hóa sự thấu thị thất bại như thế nào: nó không xem xét trạng thái niềm tin mà tác nhân sẽ ở sau khi hành động. Một trạng thái niềm tin hoàn toàn thiếu kiến thức là không mong muốn, đặc biệt khi một khả năng là cái chết chắc chắn. Bởi vì nó giả định rằng mọi trạng thái tương lai sẽ tự động là một trong những kiến thức hoàn hảo, cách tiếp cận thấu thị không bao giờ chọn các hành động thu thập thông tin (như nước đi đầu tiên trong Hình 6.15); nó cũng sẽ không chọn các hành động che giấu thông tin khỏi đối thủ hoặc cung cấp thông tin cho một đối tác, bởi vì nó giả định rằng họ đã biết thông tin đó; và nó sẽ không bao giờ **bluff** trong poker, vì nó giả định đối thủ có thể nhìn thấy bài của mình. Trong Chương 16, chúng ta sẽ chỉ ra cách xây dựng các thuật toán thực hiện tất cả những điều này nhờ giải quyết vấn đề quyết định quan sát một phần thực sự, dẫn đến một chiến lược cân bằng tối ưu (xem Mục 17.2).



Hình 6.16 Một cây trò chơi hai ply mà heuristic minimax có thể mắc lỗi.

Bất chấp những hạn chế, trung bình hóa sự thiếu thốn có thể là một chiến lược hiệu quả, với một số thủ thuật để làm cho nó hoạt động tốt hơn. Trong hầu hết các trò chơi bài, số lần chia bài có thể có khá lớn. Ví dụ, trong ván bridge, mỗi người chơi chỉ nhìn thấy hai trong số bốn bộ bài; có hai bộ bài không nhìn thấy được, mỗi bộ 13 lá, vì vậy số lần chia bài là $\binom{26}{13} \approx 10,400,600$. Giải quyết ngay cả một lần chia bài là khá khó khăn, vì vậy giải quyết mười triệu lần là không thể. Một cách để xử lý con số khổng lồ này là **trừu tượng hóa**: tức là bằng cách coi các bộ bài tương tự là giống hệt nhau. Ví dụ, rất quan trọng là những quân át và vua nào có trong một bộ bài, nhưng việc bộ bài có quân 4 hay 5 thì không quan trọng bằng, và có thể được trừu tượng hóa.

Một cách khác để xử lý số lượng lớn là **cắt tỉa về phía trước**: chỉ xem xét một mẫu ngẫu nhiên nhỏ gồm N lần chia bài, và lại tính toán điểm EXPECTIMINIMAX. Ngay cả đối với N khá nhỏ—chẳng hạn, 100 đến 1.000—phương pháp này đưa ra một xấp xỉ tốt. Nó cũng có thể được áp dụng cho các trò chơi tất định như Kriegspiel, nơi chúng ta lấy mẫu trên các trạng thái có thể có của trò chơi thay vì trên các lần chia bài có thể có, miễn là chúng ta có một cách nào đó để ước tính khả năng xảy ra của mỗi trạng thái. Việc tìm kiếm heuristic với một giới hạn độ sâu cũng có thể hữu ích thay vì tìm kiếm toàn bộ cây trò chơi.

Cho đến nay chúng ta đã giả định rằng mỗi lần chia bài có khả năng như nhau. Điều đó có ý nghĩa đối với các trò chơi như whist và hearts. Nhưng đối với bridge, ván chơi được bắt đầu bằng một giai đoạn đấu giá trong đó mỗi đội chỉ ra số mảnh khố mà nó dự kiến sẽ thắng. Vì người chơi đấu giá dựa trên các lá bài họ cầm, những người chơi khác học được một cái gì đó về xác suất $P(s)$ của mỗi lần chia bài. Việc tính toán điều này khi quyết định cách chơi một bộ bài là phức tạp, vì những lý do được đề cập trong mô tả của chúng tôi về Kriegspiel: người chơi có thể đấu giá theo cách để giảm thiểu thông tin được truyền đạt cho đối thủ của họ.

Máy tính đã đạt đến trình độ siêu phàm trong poker. Chương trình poker Libratus đã đối đầu với bốn trong số những người chơi poker hàng đầu thế giới trong một trận đấu không giới hạn Texas hold 'em kéo dài 20 ngày và đánh bại tất cả họ một cách dứt khoát. Vì có rất nhiều trạng thái có thể có trong poker, Libratus sử dụng trừu tượng hóa để giảm không gian trạng thái: nó có thể coi hai bộ bài AAA72 và AAA64 là tương đương (chúng đều là “ba quân át và một số quân bài thấp”), và nó có thể coi một cược 200 đô la là giống với 201 đô la. Nhưng Libratus cũng theo dõi những

người chơi khác, và nếu nó phát hiện họ đang khai thác một sự trù tuợng, nó sẽ thực hiện một số tính toán bổ sung qua đêm để lấp đầy lỗ hổng đó. Nhìn chung, nó đã sử dụng 25 triệu giờ CPU trên một siêu máy tính để giành chiến thắng.

Chi phí tính toán mà Libratus phải chịu (và chi phí tương tự của ALPHAZERO và các hệ thống khác) cho thấy rằng việc chơi trò chơi ở đẳng cấp vô địch thế giới có thể không đạt được đối với các nhà nghiên cứu có ngân sách hạn chế. Ở một mức độ nào đó, điều đó là đúng: giống như bạn không nên mong đợi có thể lắp ráp một chiếc xe đua Công thức Một vô địch từ các phụ tùng trong nhà để xe của mình, có một lợi thế khi có quyền truy cập vào siêu máy tính hoặc phần cứng chuyên dụng như Bộ xử lý Tensor. Điều đó đặc biệt đúng khi đào tạo một hệ thống, nhưng việc đào tạo cũng có thể được thực hiện thông qua nguồn cung cấp cộng đồng. Ví dụ, hệ thống LEELAZERO mã nguồn mở là một sự tái triển khai của ALPHAZERO, đào tạo thông qua việc tự chơi trên máy tính của những người tham gia tình nguyện. Sau khi được đào tạo, các yêu cầu tính toán cho việc chơi giải đấu thực tế là khiêm tốn. ALPHASTAR đã thắng các trò chơi StarCraft II chạy trên một máy tính để bàn thông thường với một GPU duy nhất, và ALPHAZERO cũng có thể được chạy trong chế độ đó.

6.7 Hạn chế của thuật toán tìm kiếm trò chơi

Vì việc tính toán các quyết định tối ưu trong các trò chơi phức tạp là không thể, tất cả các thuật toán phải đưa ra một số giả định và xấp xỉ. Tìm kiếm alpha-beta sử dụng hàm đánh giá heuristic như một phép xấp xỉ, và tìm kiếm Monte Carlo tính toán một giá trị trung bình gần đúng trên một lựa chọn ngẫu nhiên các ván chơi. Lựa chọn thuật toán nào để sử dụng phụ thuộc một phần vào các tính năng của mỗi trò chơi: khi hệ số phân nhánh cao hoặc khó xác định một hàm đánh giá, tìm kiếm Monte Carlo được ưu tiên. Nhưng cả hai thuật toán đều có những hạn chế cơ bản.

Một hạn chế của tìm kiếm alpha-beta là tính dễ bị tổn thương trước các lỗi trong hàm heuristic. Hình 6.16 cho thấy một cây trò chơi hai ply mà minimax gợi ý chọn nhánh bên phải vì $100 > 99$. Đó là nước đi đúng nếu tất cả các đánh giá đều hoàn toàn chính xác. Nhưng giả sử rằng việc đánh giá của mỗi nút có một lỗi độc lập với các nút khác và được phân bố ngẫu nhiên với độ lệch chuẩn là σ . Khi đó, nhánh bên trái thực sự tốt hơn 71% thời gian khi $\sigma = 5$, và 58% thời gian khi $\sigma = 2$ (vì một trong bốn nút lá bên phải có khả năng trượt xuống dưới 99 trong những trường hợp này). Nếu lỗi trong hàm đánh giá không độc lập, thì khả năng mắc lỗi sẽ tăng lên. Rất khó để bù đắp cho điều này vì chúng ta không có một mô hình tốt về sự phụ thuộc giữa các giá trị của các nút anh em.

Một hạn chế thứ hai của cả alpha-beta và Monte Carlo là chúng được thiết kế để tính toán (các giới hạn trên) các giá trị của các nước đi hợp lệ. Nhưng đôi khi có một nước đi rõ ràng là tốt nhất (ví dụ khi chỉ có một nước đi hợp lệ), và trong trường hợp đó, không có ích gì khi lãng phí thời gian tính toán để tìm ra giá trị của nước đi—tốt hơn là chỉ thực hiện nước đi đó. Một thuật toán tìm kiếm tốt hơn sẽ sử dụng ý tưởng về **tiện ích của một lần mở rộng nút**, chọn các lần mở rộng nút có tiện ích cao—nghĩa là, những lần có khả năng dẫn đến việc khám phá một nước đi tốt hơn đáng kể. Nếu không có lần mở rộng nút nào có tiện ích cao hơn chi phí của chúng (về mặt thời gian), thì thuật toán nên ngừng tìm kiếm và thực hiện một nước đi. Điều này không chỉ hoạt động cho các tình huống rõ ràng là tốt nhất mà còn cho trường hợp các nước đi đối xứng, trong đó không có tìm kiếm nào có thể chỉ ra rằng một nước đi tốt hơn nước đi khác.

Loại lý luận về những phép tính nào cần thực hiện này được gọi là **siêu lý luận** (lý luận về lý luận). Nó không chỉ áp dụng cho việc chơi trò chơi mà còn cho bất kỳ loại lý luận nào. Tất cả các phép tính được thực hiện nhằm cố gắng đưa ra các quyết định tốt hơn, tất cả đều có chi phí và tất cả đều có một khả năng dẫn đến một cải thiện nhất định trong chất lượng quyết định. Tim kiếm Monte Carlo cố gắng thực hiện siêu lý luận để phân bổ tài nguyên cho các phần quan trọng nhất của cây, nhưng không làm như vậy một cách tối ưu.

Một hạn chế thứ ba là cả alpha-beta và Monte Carlo đều thực hiện tất cả lý luận của chúng ở cấp độ các nước đi riêng lẻ. Rõ ràng, con người chơi trò chơi khác: họ có thể lý luận ở một cấp độ trừu tượng hơn, xem xét một mục tiêu cấp cao hơn—ví dụ, bẫy hậu của đối thủ—và sử dụng mục tiêu đó để tạo ra các kế hoạch hợp lý một cách có chọn lọc. Trong Chương 11, chúng ta sẽ nghiên cứu loại lập kế hoạch này, và trong Mục 11.4, chúng ta sẽ chỉ ra cách lập kế hoạch với một hệ thống phân cấp từ biểu diễn trừu tượng đến cụ thể.

Một vấn đề thứ tư là khả năng kết hợp học máy vào quá trình tìm kiếm trò chơi. Các chương trình trò chơi ban đầu dựa vào kiến thức chuyên môn của con người để tạo ra các hàm đánh giá, sách khai cuộc, chiến lược tìm kiếm và các thủ thuật hiệu quả bằng tay. Chúng ta chỉ mới bắt đầu thấy các chương trình như **ALPHAZERO** (Silver et al., 2018), dựa vào học máy từ việc tự chơi hơn là kiến thức chuyên môn do con người tạo ra dành riêng cho trò chơi. Chúng ta sẽ đề cập sâu về học máy bắt đầu với Chương 19.

Tóm tắt

Chúng ta đã xem xét nhiều trò chơi khác nhau để hiểu ý nghĩa của lối chơi tối ưu, để hiểu cách chơi tốt trong thực tế và để có cảm nhận về cách một tác nhân nên hành động trong bất kỳ loại môi trường đối kháng nào. Các ý tưởng quan trọng nhất như sau:

- Một trò chơi có thể được định nghĩa bởi trạng thái ban đầu (cách bàn cờ được thiết lập), các hành động hợp lệ trong mỗi trạng thái, kết quả của mỗi hành động, một bài kiểm tra kết thúc (cho biết khi nào trò chơi kết thúc), và một hàm tiện ích áp dụng cho các trạng thái kết thúc để cho biết ai đã thắng và điểm cuối cùng là bao nhiêu.
- Trong các trò chơi hai người chơi, rời rạc, tất định, theo lượt, tổng bằng không với thông tin hoàn hảo, thuật toán **minimax** có thể chọn các nước đi tối ưu bằng cách liệt kê cây trò chơi theo chiều sâu.
- Thuật toán tìm kiếm **alpha-beta** tính toán cùng một nước đi tối ưu như minimax, nhưng đạt được hiệu quả lớn hơn nhiều bằng cách loại bỏ các cây con đã được chứng minh là không liên quan.
- Thông thường, không khả thi để xem xét toàn bộ cây trò chơi (ngay cả với alpha-beta), vì vậy chúng ta cần cắt tìm kiếm tại một số điểm và áp dụng một **hàm đánh giá heuristic** để ước tính tiện ích của một trạng thái.
- Một giải pháp thay thế được gọi là **tìm kiếm cây Monte Carlo (MCTS)** đánh giá các trạng thái không phải bằng cách áp dụng một hàm heuristic, mà bằng cách chơi trò chơi cho đến cuối cùng và sử dụng các quy tắc của trò chơi để xem ai đã thắng. Vì các nước đi được

chọn trong quá trình chơi có thể không phải là các nước đi tối ưu, nên quá trình này được lặp lại nhiều lần và đánh giá là giá trị trung bình của các kết quả.

- Nhiều chương trình trò chơi tính toán trước các bảng các nước đi tốt nhất trong khai cuộc và tàn cuộc để chúng có thể tra cứu một nước đi thay vì tìm kiếm.
- Các trò chơi may rủi có thể được xử lý bằng **expectiminimax**, một phần mở rộng của thuật toán minimax đánh giá một nút ngẫu nhiên bằng cách lấy tiện ích trung bình của tất cả các con của nó, được trọng số bằng xác suất của mỗi con.
- Trong các trò chơi có thông tin không hoàn hảo, chẳng hạn như Kriegspiel và poker, lối chơi tối ưu yêu cầu lý luận về các **trạng thái niềm tin** hiện tại và tương lai của mỗi người chơi. Một phép xấp xỉ đơn giản có thể đạt được bằng cách lấy giá trị trung bình của một hành động trên mỗi cấu hình có thể có của thông tin bị thiếu.
- Các chương trình đã đánh bại một cách thuyết phục các nhà vô địch con người trong cờ vua, cờ đam, Othello, Cờ vây, poker và nhiều trò chơi khác. Con người vẫn giữ ưu thế trong một vài trò chơi có thông tin không hoàn hảo, chẳng hạn như bridge và Kriegspiel. Trong các trò chơi điện tử như StarCraft và Dota 2, các chương trình cạnh tranh với các chuyên gia con người, nhưng một phần thành công của chúng có thể là do khả năng thực hiện nhiều hành động rất nhanh chóng.

Ghi chú thư mục và lịch sử

Năm 1846, Charles Babbage đã thảo luận về tính khả thi của cờ vua và cờ đam trên máy tính (Morrison và Morrison, 1961). Ông không hiểu sự phức tạp theo cấp số nhân của cây tìm kiếm, tuyên bố rằng “các sự kết hợp liên quan đến Động cơ Phân tích vượt trội hơn rất nhiều so với bất kỳ yêu cầu nào, ngay cả đối với trò chơi cờ vua.” Babbage cũng thiết kế, nhưng không chế tạo, một máy chuyên dụng để chơi cờ ca-rô. cỗ máy chơi trò chơi đầu tiên được chế tạo vào khoảng năm 1890 bởi kỹ sư người Tây Ban Nha Leonardo Torres y Quevedo. Nó chuyên về tàn cuộc cờ vua “KRK” (vua và xe đấu với vua), đảm bảo chiến thắng khi bên có xe được di chuyển. Thuật toán minimax được truy vết đến một bài báo năm 1912 của Ernst Zermelo, nhà phát triển của lý thuyết tập hợp hiện đại.

Chơi trò chơi là một trong những nhiệm vụ đầu tiên được thực hiện trong AI, với những nỗ lực ban đầu của các nhà tiên phong như Konrad Zuse (1945), Norbert Wiener trong cuốn sách **Điều khiển học** của ông (1948), và Alan Turing (1953). Nhưng chính bài báo của Claude Shannon **Lập trình một máy tính để chơi cờ vua** (1950) đã đưa ra tất cả các ý tưởng chính: một biểu diễn cho các vị trí bàn cờ, một hàm đánh giá, tìm kiếm tạm ngừng (quiescence search) và một số ý tưởng cho tìm kiếm cây trò chơi có chọn lọc. Slater (1950) có ý tưởng về một hàm đánh giá như một tổ hợp tuyến tính của các đặc điểm, và nhấn mạnh đặc điểm di động trong cờ vua.

John McCarthy đã nghĩ ra ý tưởng về tìm kiếm alpha-beta vào năm 1956, mặc dù ý tưởng này không xuất hiện trên báo in cho đến sau này (Hart và Edwards, 1961). Knuth và Moore (1975) đã chứng minh tính đúng đắn của alpha-beta và phân tích độ phức tạp thời gian của nó, trong khi Pearl (1982b) đã chỉ ra rằng alpha-beta là tối ưu tiệm cận trong số tất cả các thuật toán tìm kiếm cây trò chơi có độ sâu cố định.

Berliner (1979) đã giới thiệu B*, một thuật toán tìm kiếm heuristic duy trì các giới hạn khoảng trên giá trị có thể có của một nút trong cây trò chơi thay vì đưa ra một ước tính giá trị điểm duy nhất. Tìm kiếm số âm mưu của David McAllester (1988) mở rộng các nút lá mà, bằng cách thay đổi giá trị của chúng, có thể khiến chương trình ưu tiên một nước đi mới ở gốc cây. MGSS* (Russell và Wefald, 1989) sử dụng các kỹ thuật lý thuyết quyết định của Chương 15 để ước tính giá trị của việc mở rộng mỗi nút lá về mặt cải thiện dự kiến trong chất lượng quyết định ở gốc.

Thuật toán SSS* (Stockman, 1979) có thể được coi là một A* hai người chơi không bao giờ mở rộng nhiều nút hơn alpha-beta. Yêu cầu bộ nhớ làm cho nó không thực tế, nhưng một phiên bản không gian tuyến tính đã được phát triển từ thuật toán RBFS (Korf và Chickering, 1996). Baum và Smith (1997) đề xuất một sự thay thế dựa trên xác suất cho minimax, cho thấy rằng nó dẫn đến các lựa chọn tốt hơn trong một số trò chơi nhất định. Thuật toán expectiminimax được đề xuất bởi Donald Michie (1966). Bruce Ballard (1983) đã mở rộng cắt tỉa alpha-beta để bao gồm các cây có nút ngẫu nhiên.

Cuốn sách Heuristics của Pearl (1984) đã phân tích kỹ lưỡng nhiều thuật toán chơi trò chơi.

Mô phỏng Monte Carlo được tiên phong bởi Metropolis và Ulam (1949) cho các tính toán liên quan đến việc phát triển bom nguyên tử. Tìm kiếm cây Monte Carlo (MCTS) được giới thiệu bởi Abramson (1987). Tesauro và Galperin (1997) đã chỉ ra cách tìm kiếm Monte Carlo có thể được kết hợp với một hàm đánh giá cho trò chơi cờ tào cáo. Việc kết thúc sớm ván chơi được nghiên cứu bởi Lorentz (2015). ALPHAGO đã kết thúc các ván chơi và áp dụng một hàm đánh giá (Silver et al., 2016). Kocsis và Szepesvari (2006) đã tinh chỉnh cách tiếp cận với cơ chế lựa chọn “Giới hạn tin cậy trên áp dụng cho các cây” (Upper Confidence Bounds applied to Trees). Chaslot et al. (2008) cho thấy MCTS có thể được áp dụng cho nhiều trò chơi khác nhau và Browne et al. (2012) đưa ra một cuộc khảo sát.

Koller và Pfeffer (1997) mô tả một hệ thống để giải quyết hoàn toàn các trò chơi quan sát một phần. Nó xử lý các trò chơi lớn hơn các hệ thống trước đó, nhưng không phải phiên bản đầy đủ của các trò chơi phức tạp như poker và bridge. Frank et al. (1998) mô tả một số biến thể của tìm kiếm Monte Carlo cho các trò chơi quan sát một phần, bao gồm một biến thể trong đó MIN có thông tin đầy đủ nhưng MAX thì không. Schofield và Thielscher (2015) điều chỉnh một hệ thống chơi trò chơi tổng quát cho các trò chơi quan sát một phần.

Ferguson đã tạo ra các chiến lược ngẫu nhiên bằng tay để chiến thắng Kriegspiel với một tượng và mã (1992) hoặc hai tượng (1995) đấu với một vua. Các chương trình Kriegspiel đầu tiên tập trung vào việc tìm kiếm các nước chiếu hết tàn cuộc và thực hiện tìm kiếm AND-OR trong không gian trạng thái niềm tin (Sakuta và Iida, 2002; Bolognesi và Ciancarini, 2003). Các thuật toán trạng thái niềm tin tăng dần cho phép tìm thấy các nước chiếu hết giữa trận đấu phức tạp hơn nhiều (Russell và Wolfe, 2005; Wolfe và Russell, 2007), nhưng ước tính trạng thái hiệu quả vẫn là trở ngại chính cho lối chơi tổng quát hiệu quả (Parker et al., 2005). Ciancarini và Favini (2010) áp dụng MCTS cho Kriegspiel, và Wang et al. (2018b) mô tả một phiên bản MCTS dựa trên trạng thái niềm tin cho Cờ vây ma.

Các cột mốc cờ vua đã được đánh dấu bằng các người chiến thắng liên tiếp của Giải thưởng Fredkin: BELLE (Condon và Thompson, 1982), chương trình đầu tiên đạt được trạng thái master; DEEP THOUGHT (Hsu et al., 1990), chương trình đầu tiên đạt được trạng thái kiện tướng quốc tế; và Deep Blue (Campbell et al., 2002; Hsu, 2004), đã đánh bại nhà vô địch thế giới Garry

Kasparov trong một trận đấu biểu diễn năm 1997. Deep Blue đã chạy tìm kiếm alpha-beta ở tốc độ hơn 100 triệu vị trí mỗi giây, và có thể tạo ra các phân mở rộng đặc biệt để thỉnh thoảng đạt đến độ sâu 40 ply.

Các chương trình cờ vua hàng đầu ngày nay (ví dụ: STOCKFISH, KOMODO, HOUDINI) vượt xa bất kỳ người chơi nào. Các chương trình này đã giảm hệ số phân nhánh hiệu quả xuống dưới 3 (so với hệ số phân nhánh thực tế khoảng 35), tìm kiếm đến khoảng 20 ply với tốc độ khoảng một triệu nút mỗi giây trên một máy tính 1 lõi tiêu chuẩn. Chúng sử dụng các kỹ thuật cắt tỉa như heuristic nước đi rộng, tạo ra một giới hạn dưới tốt cho giá trị của một vị trí, sử dụng tìm kiếm nông trong đó đối thủ được di chuyển hai lần ở đầu. Cắt tỉa vô ích cũng quan trọng, giúp quyết định trước những nước đi nào sẽ gây ra beta cutoff trong các nút kế tiếp. SUNFISH là một chương trình cờ vua đơn giản hóa cho mục đích giảng dạy; lõi của nó ít hơn 200 dòng Python.

Ý tưởng về **phân tích hồi cứu** để tính toán các bảng tàn cuộc là do Bellman (1965). Sử dụng ý tưởng này, Ken Thompson (1986, 1996) và Lewis Stiller (1992, 1996) đã giải quyết tất cả các tàn cuộc cờ vua với tối đa năm quân cờ. Stiller đã phát hiện một trường hợp trong đó tồn tại một nước chiếu hết bắt buộc nhưng yêu cầu 262 nước đi; điều này gây ra một số lo ngại vì các quy tắc của cờ vua yêu cầu một nước bắt hoặc nước đi quân tốt phải xảy ra trong vòng 50 nước đi, nếu không thì sẽ tuyên bố hòa. Năm 2012, Vladimir Makhnychev và Victor Zakharov đã biên soạn **Cơ sở dữ liệu tàn cuộc Lomonosov**, đã giải quyết tất cả các vị trí tàn cuộc với tối đa bảy quân cờ—một số yêu cầu hơn 500 nước đi mà không có nước bắt. Bảng 7 quân tiêu tốn 140 terabyte; một bảng 8 quân sẽ lớn hơn 100 lần.

Năm 2017, ALPHAZERO (Silver et al., 2018) đã đánh bại STOCKFISH (nhà vô địch cờ vua máy tính TCEC 2017) trong một thử nghiệm 1000 ván đấu, với 155 trận thắng và 6 trận thua. Các trận đấu bổ sung cũng dẫn đến chiến thắng quyết định cho ALPHAZERO, ngay cả khi nó chỉ được cấp 1/10 thời gian được phân bổ cho STOCKFISH.

Đại kiện tướng Larry Kaufman đã ngạc nhiên trước sự thành công của chương trình Monte Carlo này và lưu ý, “Có thể sự thống trị hiện tại của các công cụ cờ vua minimax có thể sắp kết thúc, nhưng còn quá sớm để nói như vậy.” Garry Kasparov nhận xét “Đó là một thành tựu đáng chú ý, ngay cả khi chúng ta nên mong đợi nó sau ALPHAGO. Nó tiếp cận cách tiếp cận giống con người Loại B đối với cờ vua máy mà Claude Shannon và Alan Turing mơ ước thay vì sử dụng vũ lực.” Ông tiếp tục dự đoán “Cờ vua đã bị rung chuyển tận gốc bởi ALPHAZERO, nhưng đây chỉ là một ví dụ nhỏ về những gì sẽ đến. Các ngành học cứng nhắc như giáo dục và y học cũng sẽ bị rung chuyển” (Sadler và Regan, 2019).

Cờ đam là trò chơi cổ điển đầu tiên được chơi bởi máy tính (Strachey, 1952). Arthur Samuel (1959, 1967) đã phát triển một chương trình cờ đam tự học hàm đánh giá của mình thông qua việc tự chơi bằng cách sử dụng một dạng học tăng cường. Đây là một thành tựu khá lớn khi Samuel có thể tạo ra một chương trình chơi tốt hơn ông, trên một máy tính IBM 704 chỉ có 10.000 từ bộ nhớ và bộ xử lý 0,000001 GHz. MENACE—Máy có thể giáo dục noughts and crosses Engine (Michie, 1963)—cũng sử dụng học tăng cường để trở nên thành thạo cờ ca-rô. Bộ xử lý của nó thậm chí còn chậm hơn: một bộ sưu tập 304 hộp diêm chứa các hạt màu để biểu thị nước đi tốt nhất đã học được trong mỗi vị trí.

Năm 1992, chương trình cờ đam CHINOOK của Jonathan Schaeffer đã thách đấu huyền thoại Marion Tinsley, người đã là nhà vô địch thế giới trong hơn 20 năm. Tinsley đã thắng trận đấu,

nhưng thua hai ván—là trận thua thứ tư và thứ năm trong toàn bộ sự nghiệp của ông. Sau khi Tinsley giải nghệ vì lý do sức khỏe, CHINOOK đã giành lấy vương miện. Câu chuyện được Schaeffer ghi lại (2008).

Năm 2007, Schaeffer và nhóm của ông đã “giải quyết” cờ đam (Schaeffer et al., 2007): trò chơi này là một trận hòa với lối chơi hoàn hảo. Richard Bellman (1965) đã dự đoán điều này: “Trong cờ đam, số lượng các nước đi có thể có trong bất kỳ tình huống nào là rất nhỏ đến mức chúng ta có thể tự tin mong đợi một giải pháp máy tính kỹ thuật số hoàn chỉnh cho vấn đề chơi tối ưu trong trò chơi này.” Bellman đã không lường trước được quy mô của nỗ lực: bảng tàn cuộc cho 10 quân cờ có 39 nghìn tỷ mục. Với bảng này, phải mất 18 năm CPU tìm kiếm alpha-beta để giải quyết trò chơi.

I. J. Good, người được Alan Turing dạy trò chơi Cờ vây, đã viết (1965a) “Tôi nghĩ sẽ còn khó hơn nhiều để lập trình một máy tính chơi một ván cờ vây hợp lý so với cờ vua.” Ông đã đúng: đến năm 2015, các chương trình Cờ vây chỉ chơi ở cấp độ nghiệp dư. Các tài liệu ban đầu được tóm tắt bởi Bouzy và Cazenave (2001) và Müller (2002).

Nhận dạng mẫu hình ảnh được đề xuất như một kỹ thuật đầy hứa hẹn cho Cờ vây bởi Zobrist (1970), trong khi Schraudolph et al. (1994) đã phân tích việc sử dụng học tăng cường, Lubberts và Miikkulainen (2001) đã đề xuất mạng nơ-ron, và Brüggmann (1993) đã giới thiệu tìm kiếm cây Monte Carlo cho Cờ vây. ALPHAGO (Silver et al., 2016) đã kết hợp bốn ý tưởng đó để đánh bại các chuyên gia hàng đầu Lee Sedol (với tỷ số 4-1 vào năm 2015) và Ke Jie (với tỷ số 3-0 vào năm 2016).

Ke Jie nhận xét “Sau khi nhân loại đã dành hàng ngàn năm để cải thiện chiến thuật của chúng ta, máy tính nói với chúng ta rằng con người hoàn toàn sai. Tôi sẽ đi xa hơn khi nói rằng không một con người nào đã chạm đến rìa của sự thật về Cờ vây.” Lee Sedol đã giải nghệ khỏi Cờ vây, than thở, “Ngay cả khi tôi trở thành số một, có một thực thể không thể bị đánh bại.”

Năm 2018, ALPHAZERO đã vượt qua ALPHAGO ở Cờ vây, và cũng đánh bại các chương trình hàng đầu trong cờ vua và shogi, học thông qua việc tự chơi mà không có bất kỳ kiến thức chuyên môn nào của con người và không có quyền truy cập vào bất kỳ trò chơi nào trong quá khứ. (Tất nhiên, nó dựa vào con người để xác định kiến trúc cơ bản là tìm kiếm cây Monte Carlo với mạng nơ-ron sâu và học tăng cường, và để mã hóa các quy tắc của trò chơi.) Sự thành công của ALPHAZERO đã dẫn đến sự quan tâm ngày càng tăng đối với học tăng cường như một thành phần chính của AI tổng quát (xem Chương 23). Tiến thêm một bước, hệ thống MUZERO hoạt động mà thậm chí không được cho biết các quy tắc của trò chơi mà nó đang chơi—nó phải tìm ra các quy tắc bằng cách chơi. MUZERO đã đạt được kết quả tiên tiến trong Pacman, cờ vua, Cờ vây và 75 trò chơi Atari (Schrittwieser et al., 2019). Nó học cách khái quát hóa; ví dụ, nó học được rằng trong Pacman, hành động “lên” di chuyển người chơi lên một ô vuông (trừ khi có một bức tường ở đó), mặc dù nó chỉ quan sát kết quả của hành động “lên” ở một tỷ lệ nhỏ các vị trí trên bàn cờ.

Othello, còn được gọi là Reversi, có không gian tìm kiếm nhỏ hơn cờ vua, nhưng việc xác định một hàm đánh giá là khó khăn, bởi vì lợi thế vật chất không quan trọng bằng tính di động. Các chương trình đã ở cấp độ siêu phàm kể từ năm 1997 (Buro, 2002).

Cờ tào cáo, một trò chơi may rủi, đã được Gerolamo Cardano phân tích toán học (1663), và được chơi trên máy tính với chương trình BKG (Berliner, 1980b), sử dụng một hàm đánh giá được xây

dụng thủ công và chỉ tìm kiếm đến độ sâu 1. Nó là chương trình đầu tiên đánh bại một nhà vô địch thế giới con người trong một trò chơi lớn (Berliner, 1980a), mặc dù Berliner dễ dàng thừa nhận rằng BKG đã rất may mắn với xúc xắc. TD-GAMMON của Gerry Tesauro (1995) đã học hàm đánh giá của nó bằng cách sử dụng mạng nơ-ron được đào tạo bằng cách tự chơi. Nó liên tục chơi ở cấp độ nhà vô địch thế giới và khiến các nhà phân tích con người thay đổi ý kiến của họ về nước đi khai cuộc tốt nhất cho một số lần tung xúc xắc.

Poker, giống như Cờ vây, đã chứng kiến những tiến bộ đáng ngạc nhiên trong những năm gần đây. Bowling et al. (2015) đã sử dụng lý thuyết trò chơi (xem Mục 17.2) để xác định chiến lược tối ưu chính xác cho một phiên bản poker chỉ có hai người chơi và một số lần tổ cố định với các kích thước cược cố định. Năm 2017, lần đầu tiên, các nhà vô địch poker đã bị đánh bại trong poker đối đầu (hai người chơi) không giới hạn Texas hold 'em trong hai trận đấu riêng biệt chống lại các chương trình Libratus (Brown và Sandholm, 2017) và DeepStack (Moravčík et al., 2017). Năm 2019, Pluribus (Brown và Sandholm, 2019) đã đánh bại các người chơi chuyên nghiệp hàng đầu trong các trò chơi Texas hold 'em với sáu người chơi. Các trò chơi nhiều người chơi đưa ra một số mối quan tâm chiến lược mà chúng ta sẽ đề cập trong Chương 17. Petosa và Balch (2019) thực hiện một phiên bản ALPHAZERO nhiều người chơi.

Bridge: Smith et al. (1998) báo cáo về cách BRIDGE BARON đã thắng giải vô địch bridge máy tính năm 1998, sử dụng các kế hoạch phân cấp (xem Chương 11) và các hành động cấp cao, chẳng hạn như finessing và squeezing, mà người chơi bridge đã quen thuộc. Ginsberg (2001) mô tả cách chương trình GIB của ông, dựa trên mô phỏng Monte Carlo (được đề xuất lần đầu tiên cho bridge bởi Levy (1989)), đã thắng giải vô địch máy tính tiếp theo và hoạt động tốt một cách đáng ngạc nhiên trước các người chơi chuyên gia. Trong thế kỷ 21, giải vô địch bridge máy tính đã bị thông trị bởi hai chương trình thương mại, JACK và WBRIDGE5. Cả hai đều chưa được mô tả trong các bài báo đã xuất bản, nhưng cả hai đều được cho là sử dụng các kỹ thuật Monte Carlo. Nhìn chung, các chương trình bridge ở cấp độ nhà vô địch con người khi thực sự chơi các bộ bài, nhưng lại tụt hậu trong giai đoạn đấu giá, vì chúng không hoàn toàn hiểu các quy ước được con người sử dụng để giao tiếp với đối tác của họ. Các lập trình viên bridge đã tập trung hơn vào việc tạo ra các chương trình hữu ích và mang tính giáo dục, khuyến khích mọi người tham gia trò chơi, hơn là vào việc đánh bại các nhà vô địch con người.

Scrabble là một trò chơi mà người chơi nghiệp dư gặp khó khăn trong việc đưa ra các từ có điểm số cao, nhưng đối với máy tính, thật dễ dàng để tìm thấy điểm số cao nhất có thể cho một bộ bài nhất định (Gordon, 1994); phần khó là lập kế hoạch trước trong một trò chơi quan sát một phần, ngẫu nhiên. Tuy nhiên, vào năm 2006, chương trình QUACKLE đã đánh bại cựu vô địch thế giới, David Boys, 3-2. Boys đã đón nhận nó một cách tốt đẹp, nói rằng, “Vẫn tốt hơn là một con người hơn là một máy tính.” Một mô tả tốt về một chương trình hàng đầu, MAVEN, được đưa ra bởi Sheppard (2002).

Các trò chơi điện tử như StarCraft II liên quan đến hàng trăm đơn vị quan sát một phần di chuyển trong thời gian thực với các không gian quan sát và hành động gần như liên tục, có chiều cao với các quy tắc phức tạp. Oriol Vinyals, người là nhà vô địch StarCraft của Tây Ban Nha ở tuổi 15, đã mô tả cách trò chơi có thể đóng vai trò là một thử nghiệm và một thách thức lớn cho học tăng cường (Vinyals et al., 2017a). Năm 2019, Vinyals và nhóm tại DeepMind đã công bố chương trình ALPHASTAR, dựa trên học sâu và học tăng cường, đã đánh bại các người chơi chuyên gia 10 ván đấu với 1, và xếp hạng trong top 0,02% người chơi được xếp hạng chính thức (Vinyals et al., 2019). ALPHASTAR đã thực hiện các bước để giới hạn số lượng hành động mỗi phút mà nó có

thể thực hiện trong các đợt quan trọng, để đáp lại các nhà phê bình cảm thấy nó có một lợi thế không công bằng.

Máy tính đã đánh bại những người chơi hàng đầu trong các trò chơi điện tử phổ biến khác như Super Smash Bros. (Firoiu et al., 2017), Quake III (Jaderberg et al., 2019), và Dota 2 (Fernandez và Mahlmann, 2018), tất cả đều sử dụng các kỹ thuật học sâu.

Các trò chơi vật lý như bóng đá robot (Visser et al., 2008; Barrett và Stone, 2015), bida (Lam và Greenspan, 2008; Archibald et al., 2009), và bóng bàn (Silva et al., 2015) đã thu hút một số sự chú ý trong AI. Chúng kết hợp tất cả các phức tạp của trò chơi điện tử với sự lộn xộn của thế giới thực.

Các cuộc thi trò chơi máy tính diễn ra hàng năm, bao gồm Thế vận hội Máy tính kể từ năm 1989. Cuộc thi trò chơi tổng quát (Love et al., 2006) kiểm tra các chương trình phải học cách chơi một trò chơi không xác định chỉ dựa trên một mô tả logic về các quy tắc của trò chơi. Hiệp hội Trò chơi Máy tính Quốc tế (ICGA) xuất bản Tạp chí ICGA và tổ chức hai hội nghị luân phiên hai năm một lần, Hội nghị Quốc tế về Máy tính và Trò chơi (ICCG hoặc CG) và Hội nghị Quốc tế về Tiến bộ trong Trò chơi Máy tính (ACG). IEEE xuất bản Giao dịch IEEE về Trò chơi và tổ chức một Hội nghị thường niên về Trí tuệ tính toán và Trò chơi.

Chương 7: Tác nhân logic

Trong chương này, chúng ta thiết kế các tác nhân có thể tạo ra các biểu diễn của một thế giới phức tạp, sử dụng quá trình suy luận để rút ra các biểu diễn mới về thế giới và sử dụng các biểu diễn này để suy luận hành động cần thực hiện.

Con người dường như biết nhiều điều, và những gì họ biết giúp họ hành động. Trong AI, **tác nhân dựa trên tri thức** sử dụng quá trình **lập luận** dựa trên một **biểu diễn** nội bộ của tri thức để quyết định hành động nào cần thực hiện.

Các tác nhân giải quyết vấn đề trong Chương 3 và 4 cũng biết một số điều, nhưng chỉ theo một nghĩa rất hạn chế và cứng nhắc. Chúng biết những hành động nào có sẵn và kết quả của việc thực hiện một hành động cụ thể từ một trạng thái cụ thể, nhưng chúng không biết các sự thật chung. Một tác nhân tìm đường không biết rằng không thể có một con đường dài một số kilomet âm. Một tác nhân giải trò chơi 8-puzzle không biết rằng hai ô không thể chiếm cùng một không gian. Tri thức mà chúng có rất hữu ích để tìm đường từ điểm bắt đầu đến mục tiêu, nhưng không hữu ích cho bất cứ điều gì khác.

Các biểu diễn nguyên tử được sử dụng bởi các tác nhân giải quyết vấn đề cũng rất hạn chế. Ví dụ, trong một môi trường có thể quan sát một phần, lựa chọn duy nhất của tác nhân giải quyết vấn đề để biểu diễn những gì nó biết về trạng thái hiện tại là liệt kê tất cả các trạng thái cụ thể có thể. Tôi có thể đưa cho một người mục tiêu lái xe đến một thị trấn ở Mỹ có dân số dưới 10.000, nhưng để nói điều đó với một tác nhân giải quyết vấn đề, tôi chỉ có thể mô tả mục tiêu một cách chính thức như một tập hợp rõ ràng của khoảng 16.000 thị trấn thỏa mãn mô tả đó.

Chương 5 giới thiệu biểu diễn phân tích đầu tiên của chúng ta, trong đó các trạng thái được biểu diễn dưới dạng gán giá trị cho các biến; đây là một bước đi đúng hướng, cho phép một số phần của tác nhân hoạt động một cách độc lập với miền và cho phép các thuật toán hiệu quả hơn. Trong chương này, chúng ta thực hiện bước này đến kết luận logic của nó—chúng ta phát triển **logic** như một lớp biểu diễn chung để hỗ trợ các tác nhân dựa trên tri thức. Những tác nhân này có thể kết hợp và tái kết hợp thông tin để phục vụ vô số mục đích. Điều này có thể rất xa so với nhu cầu của thời điểm hiện tại—như khi một nhà toán học chứng minh một định lý hoặc một nhà thiên văn tính toán tuổi thọ của Trái Đất. Các tác nhân dựa trên tri thức có thể chấp nhận các nhiệm vụ mới dưới dạng các mục tiêu được mô tả rõ ràng; chúng có thể đạt được năng lực nhanh chóng bằng cách được chỉ dẫn hoặc học tri thức mới về môi trường; và chúng có thể thích ứng với các thay đổi trong môi trường bằng cách cập nhật tri thức liên quan.

Chúng ta bắt đầu trong Mục 7.1 với thiết kế tổng thể của tác nhân. Mục 7.2 giới thiệu một môi trường mới đơn giản, thế giới wumpus, và minh họa hoạt động của một tác nhân dựa trên tri thức mà không đi vào bất kỳ chi tiết kỹ thuật nào. Sau đó, chúng ta giải thích các nguyên tắc chung của **logic** trong Mục 7.3 và các chi tiết cụ thể của **logic mệnh đề** trong Mục 7.4. Logic mệnh đề là một biểu diễn phân tích; mặc dù ít biểu đạt hơn **logic bậc nhất** (Chương 8), là biểu diễn có cấu trúc chuẩn, logic mệnh đề minh họa tất cả các khái niệm cơ bản của logic. Nó cũng đi kèm với các công nghệ suy luận phát triển tốt, mà chúng ta mô tả trong Mục 7.5 và 7.6. Cuối cùng, Mục 7.7 kết hợp khái niệm tác nhân dựa trên tri thức với công nghệ logic mệnh đề để xây dựng một số tác nhân đơn giản cho thế giới wumpus.

7.1 Tác nhân dựa trên tri thức

Thành phần trung tâm của một tác nhân dựa trên tri thức là cơ sở tri thức (KB) của nó. Một cơ sở tri thức là một tập hợp các câu. (Ở đây, “câu” được sử dụng như một thuật ngữ kỹ thuật. Nó có liên quan nhưng không giống với các câu trong tiếng Anh và các ngôn ngữ tự nhiên khác.) Mỗi câu được biểu diễn bằng một ngôn ngữ gọi là ngôn ngữ biểu diễn tri thức và biểu diễn một số khẳng định về thế giới. Khi một câu được coi là đã cho mà không được suy ra từ các câu khác, chúng ta gọi nó là một tiên đề.

Phải có một cách để thêm các câu mới vào cơ sở tri thức và một cách để truy vấn những gì đã biết. Tên tiêu chuẩn cho các thao tác này là TELL và ASK, tương ứng. Cả hai thao tác đều có thể liên quan đến suy luận—tức là rút ra các câu mới từ các câu cũ. Suy luận phải tuân theo yêu cầu rằng khi một câu hỏi được ASK từ cơ sở tri thức, câu trả lời phải được suy ra từ những gì đã được TELL cho cơ sở tri thức trước đó. Sau này trong chương này, chúng ta sẽ chính xác hơn về từ quan trọng “suy ra”. Hiện tại, hãy hiểu nó có nghĩa là quá trình suy luận không nên bị đặt khi nó tiến hành.

```
function KB-AGENT(percept) returns an action
  persistent: KB, a knowledge base
               t, a counter, initially 0, indicating time

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

Hình 7.1 Một tác tử dựa trên tri thức chung. Với một *percept*, tác tử thêm *percept* đó vào cơ sở tri thức của nó, hỏi cơ sở tri thức về hành động tốt nhất, và nói với cơ sở tri thức rằng nó đã thực sự thực hiện hành động đó.

Hình 7.1 cho thấy phác thảo của một chương trình tác nhân dựa trên tri thức. Giống như tất cả các tác nhân của chúng ta, nó nhận một tri giác làm đầu vào và trả về một hành động. Tác nhân duy trì một cơ sở tri thức, *KB*, ban đầu có thể chứa một số tri thức nền.

Mỗi khi chương trình tác nhân được gọi, nó thực hiện ba việc. Đầu tiên, nó TELL cho cơ sở tri thức những gì nó tri giác. Thứ hai, nó ASK cơ sở tri thức hành động nào nên thực hiện. Trong quá trình trả lời truy vấn này, có thể thực hiện nhiều lập luận về trạng thái hiện tại của thế giới, về kết quả của các chuỗi hành động có thể, v.v. Thứ ba, chương trình tác nhân TELL cho cơ sở tri thức hành động nào đã được chọn và trả về hành động đó để nó có thể được thực thi.

Các chi tiết của ngôn ngữ biểu diễn được ẩn bên trong ba hàm thực hiện giao diện giữa các cảm biến và cơ cấu chấp hành ở một bên và hệ thống biểu diễn và lập luận cốt lõi ở bên kia. MAKE-PERCEPT-SENTENCE xây dựng một câu khẳng định rằng tác nhân đã tri giác tri giác đã cho tại thời điểm đã cho. MAKE-ACTION-QUERY xây dựng một câu hỏi hành động nào nên được thực hiện tại thời điểm hiện tại. Cuối cùng, MAKE-ACTION-SENTENCE xây dựng một câu khẳng định rằng hành động đã chọn đã được thực thi. Các chi tiết của cơ chế suy luận được ẩn bên trong TELL và ASK. Các mục sau sẽ tiết lộ các chi tiết này.

Tác nhân trong Hình 7.1 có vẻ khá giống với các tác nhân có trạng thái nội bộ được mô tả trong Chương 2. Tuy nhiên, do định nghĩa của TELL và ASK, tác nhân dựa trên tri thức không phải là một chương trình tùy ý để tính toán các hành động. Nó có thể được mô tả ở mức tri thức, nơi chúng ta chỉ cần chỉ định những gì tác nhân biết và mục tiêu của nó là gì, để xác định hành vi của nó.

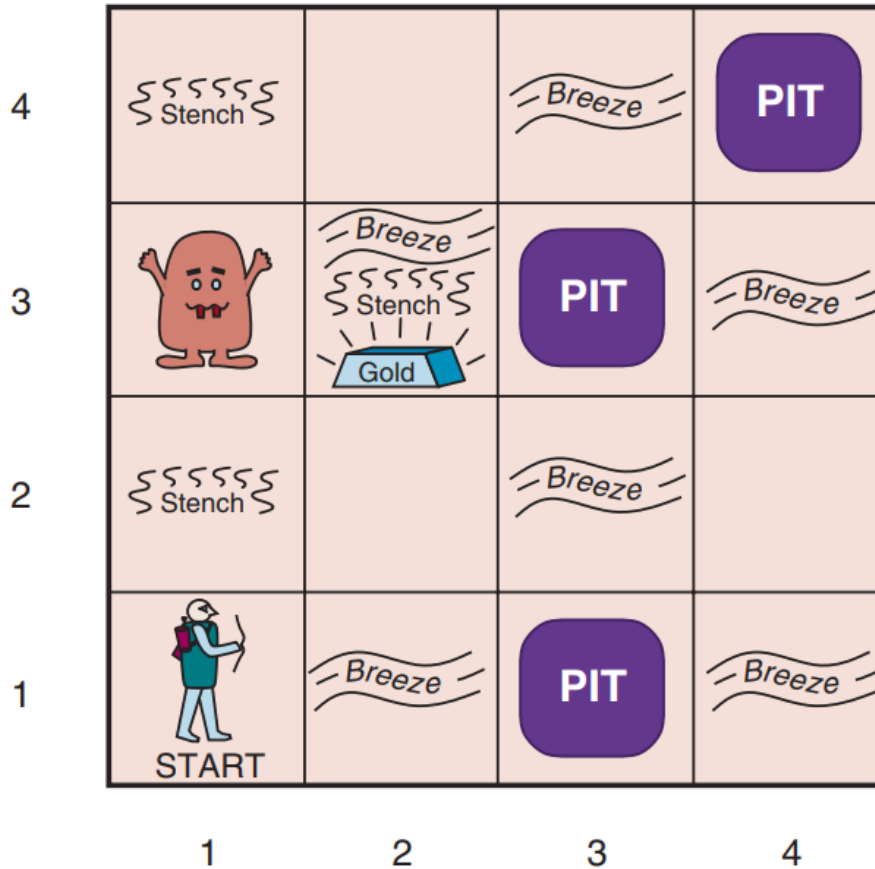
Ví dụ, một taxi tự động có thể có mục tiêu đưa hành khách từ San Francisco đến Hạt Marin và có thể biết rằng Cầu Cổng Vàng là liên kết duy nhất giữa hai địa điểm này. Sau đó, chúng ta có thể mong đợi nó băng qua Cầu Cổng Vàng vì nó biết rằng điều đó sẽ đạt được mục tiêu của nó. Lưu ý rằng phân tích này độc lập với cách taxi hoạt động ở mức triển khai. Không quan trọng việc tri thức địa lý của nó được triển khai dưới dạng danh sách liên kết hay bản đồ pixel, hoặc liệu nó lập luận bằng cách thao tác các chuỗi ký hiệu được lưu trữ trong các thanh ghi hay bằng cách truyền các tín hiệu nhiễu trong một mạng lưới tế bào thần kinh.

Một tác nhân dựa trên tri thức có thể được xây dựng đơn giản bằng cách TELL cho nó những gì nó cần biết. Bắt đầu với một cơ sở tri thức trống, nhà thiết kế tác nhân có thể TELL từng câu một cho đến khi tác nhân biết cách hoạt động trong môi trường của nó. Đây được gọi là cách tiếp cận khai báo để xây dựng hệ thống. Ngược lại, cách tiếp cận thủ tục mã hóa các hành vi mong muốn trực tiếp dưới dạng mã chương trình. Trong những năm 1970 và 1980, những người ủng hộ hai cách tiếp cận này đã tham gia vào các cuộc tranh luận nảy lửa. Bây giờ chúng ta hiểu rằng một tác nhân thành công thường kết hợp cả hai yếu tố khai báo và thủ tục trong thiết kế của nó, và tri thức khai báo thường có thể được biên dịch thành mã thủ tục hiệu quả hơn.

Chúng ta cũng có thể cung cấp cho một tác nhân dựa trên tri thức các cơ chế cho phép nó tự học. Những cơ chế này, được thảo luận trong Chương 19, tạo ra tri thức chung về môi trường từ một chuỗi tri giác. Một tác nhân học tập có thể hoàn toàn tự chủ.

7.2 Thế giới Wumpus

Trong mục này, chúng ta mô tả một môi trường trong đó các tác nhân dựa trên tri thức có thể thể hiện giá trị của chúng. Thế giới wumpus là một hang động bao gồm các phòng được kết nối bởi các lối đi. Ẩn nấp đâu đó trong hang là con wumpus khủng khiếp, một con quái vật ăn thịt bất cứ ai bước vào phòng của nó. Wumpus có thể bị bắn bởi một tác nhân, nhưng tác nhân chỉ có một mũi tên. Một số phòng chứa các hố không đáy sẽ bắt bất cứ ai đi vào những phòng này (ngoại trừ wumpus, quá lớn để rơi vào). Điểm cứu rỗi duy nhất của môi trường âm đạm này là khả năng tìm thấy một đồng vàng. Mặc dù thế giới wumpus khá nhạt nhẽo theo tiêu chuẩn trò chơi máy tính hiện đại, nó minh họa một số điểm quan trọng về trí tuệ.



Hình 7.2 Một thế giới wumpus điển hình. Tác tử ở góc dưới cùng bên trái, quay mặt về hướng đông (sang phải).

Một thế giới wumpus mẫu được hiển thị trong Hình 7.2. Định nghĩa chính xác của môi trường tác vụ được đưa ra, như gợi ý trong Mục 2.3, bằng mô tả PEAS:

- **Thước đo hiệu suất:** +1000 cho việc leo ra khỏi hang với vàng, -1000 cho việc rơi vào hố hoặc bị wumpus ăn thịt, -1 cho mỗi hành động được thực hiện và -10 cho việc sử dụng hết mũi tên. Trò chơi kết thúc khi tác nhân chết hoặc khi tác nhân leo ra khỏi hang.
- **Môi trường:** Một lưới phòng 4×4, với các bức tường bao quanh lưới. Tác nhân luôn bắt đầu ở ô được gắn nhãn [1,1], hướng về phía đông. Vị trí của vàng và wumpus được chọn ngẫu nhiên, với phân phối đều, từ các ô khác với ô bắt đầu. Ngoài ra, mỗi ô khác với ô bắt đầu có thể là một hố, với xác suất 0,2.
- **Cơ cấu chấp hành:** Tác nhân có thể di chuyển Tiến lên, Rẽ trái 90° hoặc Rẽ phải 90°. Tác nhân sẽ chết một cách đau đớn nếu nó vào một ô chứa hố hoặc một wumpus còn sống. (An toàn, mặc dù có mùi hôi, khi vào một ô có wumpus đã chết.) Nếu một tác nhân cố gắng di chuyển tiến lên và va vào tường, thì tác nhân không di chuyển. Hành động Lấy có thể được sử dụng để nhặt vàng nếu nó ở cùng ô với tác nhân. Hành động Bắn có thể được sử dụng để bắn một mũi tên theo đường thẳng theo hướng tác nhân đang đối diện. Mũi tên tiếp tục cho đến khi nó trúng (và do đó giết chết) wumpus hoặc trúng tường. Tác nhân chỉ có một

mũi tên, vì vậy chỉ hành động Bắt đầu tiên có hiệu lực. Cuối cùng, hành động Leo có thể được sử dụng để leo ra khỏi hang, nhưng chỉ từ ô [1,1].

- **Cảm biến:** Tác nhân có năm cảm biến, mỗi cảm biến cung cấp một bit thông tin:
 - Trong các ô liền kề trực tiếp (không chéo) với wumpus, tác nhân sẽ tri giác mùi Hôi thối.
 - Trong các ô liền kề trực tiếp với một hố, tác nhân sẽ tri giác Gió lửa.
 - Trong ô có vàng, tác nhân sẽ tri giác Ánh kim.
 - Khi một tác nhân đi vào tường, nó sẽ tri giác Va chạm.
 - Khi wumpus bị giết, nó phát ra một tiếng Thét thảm thiết có thể được tri giác ở bất cứ đâu trong hang.

Các tri giác sẽ được cung cấp cho chương trình tác nhân dưới dạng một danh sách năm ký hiệu; ví dụ, nếu có mùi hôi và gió lửa, nhưng không có ánh kim, va chạm hoặc tiếng thét, chương trình tác nhân sẽ nhận được [Stench, Breeze, None, None, None].

Chúng ta có thể đặc trưng môi trường wumpus dọc theo các chiều khác nhau được đưa ra trong Chương 2. Rõ ràng, nó là tất định, rời rạc, tĩnh và đơn tác nhân. (Wumpus không di chuyển, may mắn thay.) Nó là tuần tự, vì phần thưởng có thể chỉ đến sau nhiều hành động được thực hiện. Nó là một phần có thể quan sát được, vì một số khía cạnh của trạng thái không thể tri giác trực tiếp: vị trí của tác nhân, tình trạng sức khỏe của wumpus và khả năng có sẵn một mũi tên. Đối với vị trí của các hố và wumpus: chúng ta có thể coi chúng như các phần không được quan sát của trạng thái—trong trường hợp đó, mô hình chuyển tiếp cho môi trường hoàn toàn đã biết, và việc tìm ra vị trí của các hố hoàn thành tri thức của tác nhân về trạng thái. Ngoài ra, chúng ta có thể nói rằng bản thân mô hình chuyển tiếp là không xác định vì tác nhân không biết hành động Tiến lên nào là chết người—trong trường hợp đó, việc khám phá vị trí của các hố và wumpus hoàn thành tri thức của tác nhân về mô hình chuyển tiếp.

Đối với một tác nhân trong môi trường, thách thức chính là sự thiếu hiểu biết ban đầu về cấu hình của môi trường; vượt qua sự thiếu hiểu biết này dường như đòi hỏi lập luận logic. Trong hầu hết các trường hợp của thế giới wumpus, tác nhân có thể lấy vàng một cách an toàn. Thỉnh thoảng, tác nhân phải lựa chọn giữa việc về nhà tay không và mạo hiểm cái chết để tìm vàng. Khoảng 21% các môi trường hoàn toàn không công bằng, vì vàng nằm trong một hố hoặc được bao quanh bởi các hố.

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK			
1,1	2,1	3,1	4,1
A			
OK	OK		

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2 P?	3,2	4,2
OK			
1,1	2,1 A	3,1 P?	4,1
V	B		
OK	OK		

(a)

(b)

Hình 7.3 Bước đầu tiên được thực hiện bởi tác tử trong thế giới wumpus. (a) Tình huống ban đầu, sau percept [None, None, None, None, None]. (b) Sau khi di chuyển đến [2,1] và cảm nhận [None, Breeze, None, None, None].

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A	2,2	3,2	4,2
S			
OK	OK		
1,1	2,1 B	3,1 P!	4,1
V	V		
OK	OK		

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 A	3,3 P?	4,3
	S G		
	B		
1,2 S	2,2	3,2	4,2
V	V		
OK	OK		
1,1	2,1 B	3,1 P!	4,1
V	V		
OK	OK		

(a)

(b)

Hình 7.4 Hai giai đoạn sau trong quá trình tiến triển của tác tử. (a) Sau khi di chuyển đến [1,1] và sau đó [1,2], và cảm nhận [Stench, None, None, None, None]. (b) Sau khi di chuyển đến [2,2] và sau đó [2,3], và cảm nhận [Stench, Breeze, Glitter, None, None].

Hãy xem một tác nhân wumpus dựa trên tri thức khám phá môi trường được hiển thị trong Hình 7.2. Chúng ta sử dụng một ngôn ngữ biểu diễn tri thức không chính thức bao gồm việc viết các ký hiệu trong một lưới (như trong Hình 7.3 và 7.4).

Cơ sở tri thức ban đầu của tác nhân chứa các quy tắc của môi trường, như đã mô tả trước đó; đặc biệt, nó biết rằng nó đang ở [1,1] và [1,1] là một ô an toàn; chúng ta biểu thị điều đó bằng “A” và “OK”, tương ứng, trong ô [1,1].

Tri giác đầu tiên là [None, None, None, None, None], từ đó tác nhân có thể kết luận rằng các ô liền kề của nó, [1,2] và [2,1], không có nguy hiểm—chúng là OK. Hình 7.3(a) cho thấy trạng thái tri thức của tác nhân tại thời điểm này.

Một tác nhân thận trọng sẽ chỉ di chuyển vào một ô mà nó biết là OK. Giả sử tác nhân quyết định di chuyển về phía trước đến [2,1]. Tác nhân tri giác một luồng gió (được biểu thị bằng “B”) trong [2,1], vì vậy phải có một hồ trong một ô liền kề. Hồ không thể ở [1,1], theo quy tắc của trò chơi, vì vậy phải có một hồ ở [2,2] hoặc [3,1] hoặc cả hai. Ký hiệu “P?” trong Hình 7.3(b) chỉ ra một hồ có thể có trong những ô đó. Tại thời điểm này, chỉ có một ô đã biết là OK và chưa được ghé thăm. Vì vậy, tác nhân thận trọng sẽ quay lại, quay trở lại [1,1], và sau đó tiến đến [1,2].

Tác nhân tri giác mùi hôi trong [1,2], dẫn đến trạng thái tri thức được hiển thị trong Hình 7.4(a). Mùi hôi trong [1,2] có nghĩa là phải có một wumpus gần đó. Nhưng wumpus không thể ở [1,1], theo quy tắc của trò chơi, và nó không thể ở [2,2] (hoặc tác nhân đã phát hiện mùi hôi khi nó ở [2,1]). Do đó, tác nhân có thể suy luận rằng wumpus ở [1,3]. Ký hiệu W! chỉ ra suy luận này. Hơn nữa, việc không có gió lửa trong [1,2] ngụ ý rằng không có hồ ở [2,2]. Tuy nhiên, tác nhân đã suy luận rằng phải có một hồ ở [2,2] hoặc [3,1], vì vậy điều này có nghĩa là nó phải ở [3,1]. Đây là một suy luận khá khó, vì nó kết hợp tri thức thu được ở các thời điểm khác nhau ở các nơi khác nhau và dựa vào việc không có tri giác để thực hiện một bước quan trọng.

Tác nhân bây giờ đã chứng minh với chính nó rằng không có hồ hoặc wumpus ở [2,2], vì vậy an toàn để di chuyển đến đó. Chúng tôi không hiển thị trạng thái tri thức của tác nhân tại [2,2]; chúng tôi chỉ giả định rằng tác nhân quay và di chuyển đến [2,3], cho chúng ta Hình 7.4(b). Trong [2,3], tác nhân phát hiện ánh kim, vì vậy nó nên lấy vàng và sau đó trở về nhà.

Lưu ý rằng trong mỗi trường hợp tác nhân rút ra kết luận từ thông tin có sẵn, kết luận đó được đảm bảo là chính xác nếu thông tin có sẵn là chính xác. Đây là một tính chất cơ bản của lập luận logic. Trong phần còn lại của chương này, chúng tôi mô tả cách xây dựng các tác nhân logic có thể biểu diễn thông tin và rút ra các kết luận như những kết luận được mô tả trong các đoạn trước.

7.3 Logic

Phần này tóm tắt các khái niệm cơ bản về biểu diễn logic và suy luận. Những ý tưởng tuyệt vời này độc lập với bất kỳ hình thức cụ thể nào của logic. Do đó, chúng ta sẽ tạm hoãn các chi tiết kỹ thuật của những hình thức đó đến phần tiếp theo, sử dụng ví dụ quen thuộc về số học thông thường thay thế.

Trong Mục 7.1, chúng ta đã nói rằng các cơ sở tri thức bao gồm các câu. Những câu này được biểu diễn theo cú pháp của ngôn ngữ biểu diễn, quy định tất cả các câu được hình thành tốt. Khái niệm cú pháp khá rõ ràng trong số học thông thường: “ $x + y = 4$ ” là một câu được hình thành tốt, trong khi “ $x4y + =$ ” thì không.

Một hệ thống logic cũng phải định nghĩa ngữ nghĩa, hoặc ý nghĩa, của các câu. Ngữ nghĩa xác định tính đúng đắn của mỗi câu đối với mỗi thế giới có thể. Ví dụ, ngữ nghĩa của số học quy định rằng câu “ $x + y = 4$ ” đúng trong một thế giới nơi x là 2 và y là 2, nhưng sai trong một thế giới nơi x là 1 và y là 1. Trong các hệ thống logic tiêu chuẩn, mỗi câu phải hoặc đúng hoặc sai trong mỗi thế giới có thể—không có trường hợp “ở giữa”.²

Khi cần chính xác, chúng ta sử dụng thuật ngữ **model** (mô hình) thay cho “thế giới có thể”. Trong khi thế giới có thể được coi là các môi trường (có khả năng) thực tế mà tác nhân có thể hoặc không thể tồn tại, thì các mô hình là các khái niệm toán học, mỗi mô hình có một giá trị chân lý cố định (đúng hoặc sai) cho mỗi câu liên quan. Một cách không chính thức, chúng ta có thể nghĩ về một thế giới có thể như, ví dụ, có x nam và y nữ ngồi quanh bàn chơi bài, và câu $x + y = 4$ đúng khi có tổng cộng bốn người. Một cách chính thức, các mô hình có thể chỉ là tất cả các phép gán số nguyên không âm cho các biến x và y. Mỗi phép gán như vậy xác định tính đúng đắn của bất kỳ câu số học nào có biến là x và y. Nếu một câu α đúng trong mô hình m, chúng ta nói rằng m **thỏa mãn** α hoặc đôi khi m là một mô hình của α . Chúng ta sử dụng ký hiệu $M(\alpha)$ để chỉ tập hợp tất cả các mô hình của α .

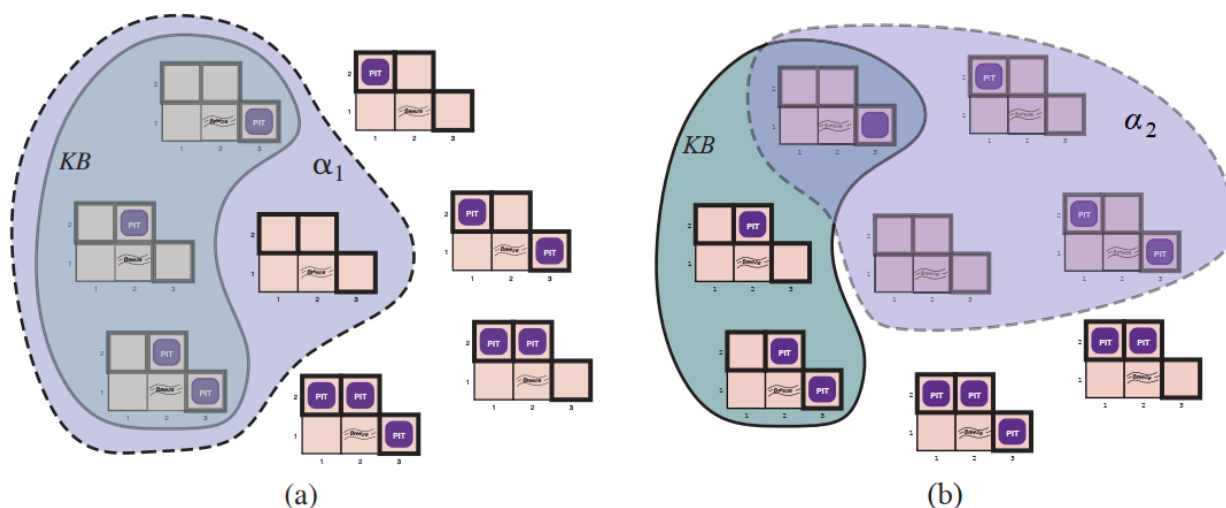
Bây giờ chúng ta đã có khái niệm về chân lý, chúng ta đã sẵn sàng để nói về suy luận logic. Điều này liên quan đến mối quan hệ **entailment** (kéo theo) logic giữa các câu—ý tưởng rằng một câu logic kéo theo một câu khác. Trong ký hiệu toán học, chúng ta viết:

$$\alpha \models \beta$$

để chỉ rằng câu α kéo theo câu β . Định nghĩa chính thức của entailment là: $\alpha \models \beta$ nếu và chỉ nếu, trong mọi mô hình mà α đúng, β cũng đúng. Sử dụng ký hiệu vừa giới thiệu, chúng ta có thể viết:

$$\alpha \models \beta \text{ nếu và chỉ nếu } M(\alpha) \subseteq M(\beta).$$

(Lưu ý hướng của $\subseteq$ ở đây: nếu $\alpha \models \beta$, thì α là một khẳng định mạnh hơn β : nó loại trừ nhiều thế giới có thể hơn.) Mối quan hệ entailment quen thuộc từ số học; chúng ta dễ dàng chấp nhận ý tưởng rằng câu $x = 0$ kéo theo câu $xy = 0$. Rõ ràng, trong bất kỳ mô hình nào mà x bằng không, thì xy cũng bằng không (bất kể giá trị của y là gì).



Hình 7.5 Các mô hình khả thi cho sự hiện diện của các hố trong các ô [1,2], [2,2] và [3,1]. KB tương ứng với các quan sát không có gì ở [1,1] và một làn gió ở [2,1] được thể hiện bằng đường liền. (a) Đường chấm thể hiện các mô hình của α_1 (không có hố ở [1,2]). (b) Đường chấm thể hiện các mô hình của α_2 (không có hố ở [2,2]).

Chúng ta có thể áp dụng cùng một loại phân tích cho ví dụ suy luận trong thế giới wumpus được đưa ra trong phần trước. Xem xét tình huống trong Hình 7.3(b): tác nhân không phát hiện gì ở [1,1]

và cảm nhận một làn gió ở [2,1]. Những nhận thức này, kết hợp với kiến thức của tác nhân về các quy tắc của thế giới wumpus, tạo thành KB. Tác nhân quan tâm đến việc liệu các ô liền kề [1,2], [2,2] và [3,1] có chứa hồ hay không. Mỗi trong ba ô này có thể chứa hoặc không chứa hồ, vì vậy (bỏ qua các khía cạnh khác của thế giới hiện tại) có $2^3 = 8$ mô hình có thể. Tám mô hình này được hiển thị trong Hình 7.5.³

KB có thể được coi là một tập hợp các câu hoặc một câu đơn lẻ khẳng định tất cả các câu riêng lẻ. KB sai trong các mô hình mâu thuẫn với những gì tác nhân biết—ví dụ, KB sai trong bất kỳ mô hình nào mà [1,2] chứa một hồ, bởi vì không có làn gió ở [1,1]. Trên thực tế, chỉ có ba mô hình mà KB đúng, và chúng được hiển thị với một đường viền đặc trong Hình 7.5. Bây giờ chúng ta hãy xem xét hai kết luận có thể:

$$\alpha_1 = \text{"Không có hồ ở [1,2]."} \quad \alpha_2 = \text{"Không có hồ ở [2,2]."}$$

Chúng ta đã bao quanh các mô hình của α_1 và α_2 bằng các đường chấm chấm trong Hình 7.5(a) và 7.5(b), tương ứng. Bằng cách kiểm tra, chúng ta thấy:

- Trong mọi mô hình mà KB đúng, α_1 cũng đúng.

Do đó, $KB \models \alpha_1$: không có hồ ở [1,2]. Chúng ta cũng có thể thấy rằng:

- Trong một số mô hình mà KB đúng, α_2 sai.

Do đó, KB không kéo theo α_2 : tác nhân không thể kết luận rằng không có hồ ở [2,2]. (Cũng không thể kết luận rằng có một hồ ở [2,2].)⁴

Ví dụ trước không chỉ minh họa entailment mà còn cho thấy cách định nghĩa của entailment có thể được áp dụng để rút ra kết luận—tức là thực hiện **suy luận logic**. Thuật toán suy luận được minh họa trong Hình 7.5 được gọi là **kiểm tra mô hình (model checking)**, vì nó liệt kê tất cả các mô hình có thể để kiểm tra rằng α đúng trong tất cả các mô hình mà KB đúng, tức là $M(KB) \subseteq M(\alpha)$.

³ Mặc dù hình vẽ cho thấy các mô hình như các phần của thế giới wumpus, chúng thực sự không gì khác hơn là các phép gán đúng và sai cho các câu “có một hồ ở [1,2]” v.v. Các mô hình, theo nghĩa toán học, không cần phải có những con wumpus khủng khiếp trong đó.

⁴ Tác nhân có thể tính toán xác suất có một hồ ở [2,2]; Chương 12 sẽ chỉ ra cách thực hiện điều này.

Suy luận logic

Trong việc hiểu entailment và suy luận, có thể hữu ích khi nghĩ về tập hợp tất cả các hệ quả của KB như một đồng cỏ khô và α như một cây kim. Entailment giống như việc cây kim nằm trong đồng cỏ khô; suy luận giống như việc tìm ra nó. Sự khác biệt này được thể hiện trong một số ký hiệu chính thức: nếu một thuật toán suy luận i có thể suy ra α từ KB, chúng ta viết:

$$KB \vdash_i \alpha,$$

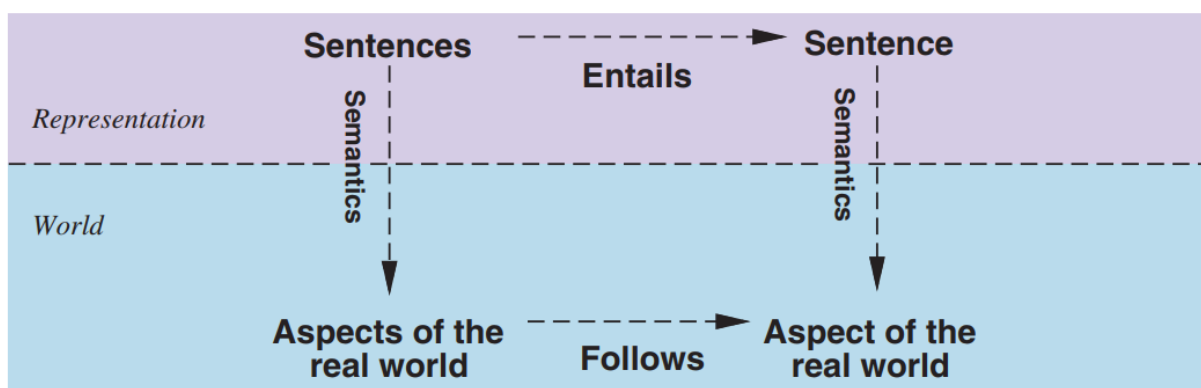
được phát âm là “ α được suy ra từ KB bởi i ” hoặc “ i suy ra α từ KB.”

Một thuật toán suy luận chỉ suy ra các câu được kéo theo được gọi là **sound** (đúng đắn) hoặc **bảo toàn chân lý (truth-preserving)**. Tính đúng đắn là một tính chất rất mong muốn. Một quy trình

suy luận không đúng dẫn về cơ bản tạo ra mọi thứ khi nó tiến hành—nó thông báo việc phát hiện ra những cây kim không tồn tại.

Dễ dàng thấy rằng kiểm tra mô hình, khi có thể áp dụng,⁵ là một quy trình đúng đắn.

Tính chất **đầy đủ (completeness)** cũng rất mong muốn: một thuật toán suy luận là đầy đủ nếu nó có thể suy ra bất kỳ câu nào được kéo theo. Đối với những đồng cỏ khô thực tế, có phạm vi hữu hạn, có vẻ hiển nhiên rằng một sự kiểm tra hệ thống luôn có thể quyết định liệu cây kim có trong đồng cỏ khô hay không. Tuy nhiên, đối với nhiều cơ sở tri thức, đồng cỏ khô của các hệ quả là vô hạn, và tính đầy đủ trở thành một vấn đề quan trọng.⁶ May mắn thay, có các quy trình suy luận đầy đủ cho các hệ thống logic đủ biểu đạt để xử lý nhiều cơ sở tri thức.



Hình 7.6 Các câu là các cấu hình vật lý của tác tử, và suy luận là một quá trình xây dựng các cấu hình vật lý mới từ các cấu hình cũ. Suy luận logic nên đảm bảo rằng các cấu hình mới đại diện cho các khía cạnh của thế giới thực sự đi theo các khía cạnh mà các cấu hình cũ đại diện.

Chúng ta đã mô tả một quá trình suy luận mà các kết luận của nó được đảm bảo đúng trong bất kỳ thế giới nào mà các tiền đề đúng; cụ thể, nếu KB đúng trong thế giới thực, thì bất kỳ câu α nào được suy ra từ KB bằng một quy trình suy luận đúng đắn cũng đúng trong thế giới thực. Vì vậy, trong khi một quá trình suy luận hoạt động trên “cú pháp”—các cấu hình vật lý bên trong như các bit trong thanh ghi hoặc các mẫu tín hiệu điện trong não bộ—quá trình này tương ứng với mối quan hệ thế giới thực, theo đó một khía cạnh nào đó của thế giới thực là đúng nhờ các khía cạnh khác của thế giới thực cũng đúng.⁷ Mối tương quan này giữa thế giới và biểu diễn được minh họa trong Hình 7.6.

Vấn đề cuối cùng cần xem xét là **grounding**—sự kết nối giữa các quá trình suy luận logic và môi trường thực tế mà tác nhân tồn tại. Cụ thể, làm thế nào chúng ta biết rằng KB đúng trong thế giới thực? (Xét cho cùng, KB chỉ là “cú pháp” bên trong tâm trí của tác nhân.) Đây là một câu hỏi triết học mà rất nhiều sách đã được viết về nó. (Xem Chương 28.) Một câu trả lời đơn giản là các cảm biến của tác nhân tạo ra kết nối. Ví dụ, tác nhân trong thế giới wumpus có một cảm biến mùi. Chương trình tác nhân tạo ra một câu phù hợp bất cứ khi nào có mùi. Sau đó, bất cứ khi nào câu đó có trong cơ sở tri thức, nó đúng trong thế giới thực. Do đó, ý nghĩa và chân lý của các câu nhận thức được xác định bởi các quá trình cảm nhận và xây dựng câu tạo ra chúng. Còn phần còn lại của kiến thức của tác nhân, chẳng hạn như niềm tin rằng wumpus gây ra mùi ở các ô liền kề, thì sao? Đây không phải là biểu diễn trực tiếp của một nhận thức đơn lẻ, mà là một quy tắc chung—có thể được suy ra từ kinh nghiệm nhận thức nhưng không giống với một tuyên bố về kinh nghiệm

đó. Các quy tắc chung như thế này được tạo ra bởi một quá trình xây dựng câu gọi là **học tập**, là chủ đề của Phần V. Học tập có thể sai lầm. Có thể xảy ra trường hợp wumpus gây ra mùi trừ ngày 29 tháng 2 trong năm nhuận, khi chúng đi tắm. Do đó, KB có thể không đúng trong thế giới thực, nhưng với các quy trình học tập tốt, chúng ta có lý do để lạc quan.

⁵ Kiểm tra mô hình hoạt động nếu không gian của các mô hình là hữu hạn—ví dụ, trong các thế giới wumpus có kích thước cố định. Đối với số học, mặt khác, không gian của các mô hình là vô hạn: ngay cả khi chúng ta giới hạn bản thân trong các số nguyên, có vô hạn cặp giá trị cho x và y trong câu $x + y = 4$.

⁶ So sánh với trường hợp không gian tìm kiếm vô hạn trong Chương 3, nơi tìm kiếm theo chiều sâu không đầy đủ.

⁷ Như Wittgenstein (1922) đã nói trong tác phẩm nổi tiếng *Tractatus* của mình: “Thế giới là tất cả những gì đang xảy ra.”

7.4 Logic Mệnh Đề: Một Logic Đơn Giản

Trong phần này, chúng tôi trình bày logic mệnh đề. Chúng tôi mô tả cú pháp (cấu trúc của các câu) và ngữ nghĩa (cách xác định tính đúng đắn của các câu). Từ đó, chúng tôi rút ra một thuật toán suy luận cú pháp đơn giản để triển khai khái niệm ngữ nghĩa của sự kéo theo. Tất nhiên, mọi thứ diễn ra trong thế giới wumpus.

7.4.1 Cú pháp

Cú pháp của logic mệnh đề định nghĩa các câu hợp lệ. Các câu nguyên tử bao gồm một ký hiệu mệnh đề đơn lẻ. Mỗi ký hiệu như vậy đại diện cho một mệnh đề có thể đúng hoặc sai. Chúng tôi sử dụng các ký hiệu bắt đầu bằng chữ in hoa và có thể chứa các chữ cái hoặc chỉ số khác, ví dụ: $P, Q, R, W_{1,3}$ và FacingEast. Các tên này là tùy ý nhưng thường được chọn để có giá trị gợi nhớ—chúng tôi sử dụng $W_{1,3}$ để đại diện cho mệnh đề rằng con wumpus ở vị trí [1,3]. (Hãy nhớ rằng các ký hiệu như $W_{1,3}$ là nguyên tử, tức là $W, 1$ và 3 không phải là các phần có ý nghĩa của ký hiệu.) Có hai ký hiệu mệnh đề với ý nghĩa cố định: True là mệnh đề luôn đúng và False là mệnh đề luôn sai.

Các câu phức tạp được xây dựng từ các câu đơn giản hơn, sử dụng dấu ngoặc đơn và các toán tử gọi là liên từ logic. Có năm liên từ thường được sử dụng:

- **¬ (phủ định)**: Một câu như $\neg W_{1,3}$ được gọi là phủ định của $W_{1,3}$. Một literal là một câu nguyên tử (literal dương) hoặc một câu nguyên tử bị phủ định (literal âm).
- **∧ (và)**: Một câu có liên từ chính là $\wedge$, chẳng hạn như $W_{1,3} \wedge P_{3,1}$, được gọi là một hội; các phần của nó là các hội. (Ký hiệu $\wedge$ trông giống chữ “A” trong “And”).
- **∨ (hoặc)**: Một câu có liên từ chính là $\vee$, chẳng hạn như $(W_{1,3} \wedge P_{3,1}) \vee W_{2,2}$, là một tuyển; các phần của nó là các tuyển—trong ví dụ này là $(W_{1,3} \wedge P_{3,1})$ và $W_{2,2}$.
- **⇒ (kéo theo)**: Một câu như $(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$ được gọi là một kéo theo (hoặc điều kiện). Tiền đề hoặc giả thiết của nó là $(W_{1,3} \wedge P_{3,1})$, và kết luận hoặc hệ quả của nó là

$\neg W_{2,2}$. Các kéo theo còn được gọi là các quy tắc hoặc các câu lệnh “nếu–thì”. Ký hiệu kéo theo đôi khi được viết trong các sách khác là $\supset$ hoặc $\rightarrow$.

- $\Leftrightarrow$ (khi và chỉ khi): Câu $W_{1,3} \Leftrightarrow \neg W_{2,2}$ là một song điều kiện.

$$\begin{aligned}
 \text{Sentence} &\rightarrow \text{AtomicSentence} \mid \text{ComplexSentence} \\
 \text{AtomicSentence} &\rightarrow \text{True} \mid \text{False} \mid P \mid Q \mid R \mid \dots \\
 \text{ComplexSentence} &\rightarrow (\text{Sentence}) \\
 &\mid \neg \text{Sentence} \\
 &\mid \text{Sentence} \wedge \text{Sentence} \\
 &\mid \text{Sentence} \vee \text{Sentence} \\
 &\mid \text{Sentence} \Rightarrow \text{Sentence} \\
 &\mid \text{Sentence} \Leftrightarrow \text{Sentence}
 \end{aligned}$$

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Hình 7.7 Ngữ pháp BNF (Dạng Backus–Naur) của các câu trong logic mệnh đề, cùng với thứ tự ưu tiên của toán tử, từ cao nhất đến thấp nhất.

Hình 7.7 trình bày một ngữ pháp BNF (Backus–Naur Form) chính thức của các câu trong logic mệnh đề, cùng với thứ tự ưu tiên của các toán tử, từ cao nhất đến thấp nhất. Ký hiệu BNF được giải thích ở trang 1081. Ngữ pháp BNF được bổ sung với một danh sách thứ tự ưu tiên toán tử để loại bỏ sự mơ hồ khi sử dụng nhiều toán tử. Toán tử “not” ($\neg$) có độ ưu tiên cao nhất, nghĩa là trong câu $\neg A \wedge B$, $\neg$ liên kết chặt chẽ nhất, cho chúng ta tương đương với $(\neg A) \wedge B$ chứ không phải $\neg(A \wedge B)$. (Ký hiệu trong số học thông thường cũng tương tự: $-2 + 4$ là 2, không phải -6.) Khi thích hợp, chúng tôi cũng sử dụng dấu ngoặc đơn và ngoặc vuông để làm rõ cấu trúc câu dự định và cải thiện khả năng đọc.

7.4.2 Ngữ nghĩa

Sau khi xác định cú pháp của logic mệnh đề, chúng tôi sẽ xác định ngữ nghĩa của nó. Ngữ nghĩa định nghĩa các quy tắc để xác định tính đúng đắn của một câu đối với một mô hình cụ thể. Trong logic mệnh đề, một mô hình đơn giản chỉ định giá trị chân lý—đúng hoặc sai—cho mọi ký hiệu mệnh đề. Ví dụ, nếu các câu trong cơ sở tri thức sử dụng các ký hiệu mệnh đề $P_{1,2}$, $P_{2,2}$ và $P_{3,1}$, thì một mô hình có thể là:

$$m_1 = \{P_{1,2} = \text{false}, P_{2,2} = \text{false}, P_{3,1} = \text{true}\}.$$

Với ba ký hiệu mệnh đề, có $2^3 = 8$ mô hình khả dĩ—chính xác là những mô hình được mô tả trong Hình 7.5. Tuy nhiên, lưu ý rằng các mô hình này chỉ là các đối tượng toán học mà không nhất thiết phải có kết nối với thế giới wumpus. $P_{1,2}$ chỉ là một ký hiệu; nó có thể có nghĩa là “có một hố ở [1,2]” hoặc “hôm nay và ngày mai tôi ở Paris.”

Ngữ nghĩa của logic mệnh đề phải chỉ định cách tính giá trị chân lý của bất kỳ câu nào, với một mô hình cho trước. Điều này được thực hiện một cách đệ quy. Tất cả các câu được xây dựng từ các câu nguyên tử và năm liên từ; do đó, chúng tôi cần chỉ định cách tính giá trị chân lý của các câu nguyên tử và cách tính giá trị chân lý của các câu được hình thành với mỗi liên từ trong năm liên từ. Các câu nguyên tử rất dễ xác định:

- **True** luôn đúng trong mọi mô hình và **False** luôn sai trong mọi mô hình.
- Giá trị chân lý của mọi ký hiệu mệnh đề khác phải được chỉ định trực tiếp trong mô hình. Ví dụ, trong mô hình m_1 đã cho, $P_{1,2}$ là sai.

Đối với các câu phức tạp, chúng tôi có năm quy tắc, áp dụng cho bất kỳ câu con P và Q nào (nguyên tử hoặc phức tạp) trong bất kỳ mô hình m nào (ở đây “iff” có nghĩa là “khi và chỉ khi”):

- $\neg P$ đúng iff P sai trong m .
- $P \wedge Q$ đúng iff cả P và Q đều đúng trong m .
- $P \vee Q$ đúng iff P hoặc Q đúng trong m .
- $P \Rightarrow Q$ đúng trừ khi P đúng và Q sai trong m .
- $P \Leftrightarrow Q$ đúng iff P và Q cùng đúng hoặc cùng sai trong m .

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Hình 7.8 Bảng chân lý cho năm liên từ logic. Để sử dụng bảng tính toán, ví dụ, giá trị của $P \vee Q$ khi P đúng và Q sai, trước tiên hãy nhìn vào bên trái để tìm hàng mà P đúng và Q sai (hàng thứ ba). Sau đó nhìn trong hàng đó dưới cột $P \vee Q$ để thấy kết quả: đúng.

Các quy tắc này cũng có thể được biểu diễn bằng bảng chân lý, chỉ định giá trị chân lý của một câu phức tạp cho mỗi phép gán giá trị chân lý có thể có cho các thành phần của nó. Bảng chân lý cho năm liên từ được trình bày trong Hình 7.8. Từ các bảng này, giá trị chân lý của bất kỳ câu s nào có thể được tính toán đối với bất kỳ mô hình m nào bằng cách đánh giá đệ quy đơn giản. Ví dụ, câu $\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$, được đánh giá trong m_1 , cho $true \wedge (false \vee true) = true \wedge true = true$. Bài tập 7.TRUV yêu cầu bạn viết thuật toán PL-True?(s, m), tính toán giá trị chân lý của một câu logic mệnh đề s trong một mô hình m .

Bảng chân lý cho “và”, “hoặc” và “không” phù hợp chặt chẽ với trực giác của chúng ta về các từ tiếng Anh tương ứng. Điểm có thể gây nhầm lẫn chính là $P \vee Q$ đúng khi P đúng hoặc Q đúng hoặc cả hai. Một liên từ khác, gọi là “hoặc loại trừ” (“xor” viết tắt), cho kết quả sai khi cả hai tuyển đều đúng. Không có sự đồng thuận về ký hiệu cho “hoặc loại trừ”; một số lựa chọn là $\vee$ hoặc $\oplus$.

Bảng chân lý cho $\Rightarrow$ có thể không hoàn toàn phù hợp với sự hiểu biết trực giác của một người về ” P kéo theo Q ” hoặc ”nếu P thì Q ”. Một mặt, logic mệnh đề không yêu cầu bất kỳ mối quan hệ nhân quả hoặc liên quan nào giữa P và Q . Câu ”5 là số lẻ kéo theo Tokyo là thủ đô của Nhật Bản” là một câu đúng trong logic mệnh đề (theo cách diễn giải thông thường), mặc dù đó là một câu tiếng Anh rất kỳ lạ. Một điểm gây nhầm lẫn khác là bất kỳ kéo theo nào cũng đúng khi tiền đề của nó sai. Ví dụ, ”5 là số chẵn kéo theo Sam thông minh” là đúng, bất kể Sam có thông minh hay không. Điều này có vẻ kỳ lạ, nhưng nó có ý nghĩa nếu bạn nghĩ về ” $P \Rightarrow Q$ ” như là nói rằng, ”Nếu P đúng, thì tôi khẳng định rằng Q đúng; nếu không, tôi không đưa ra bất kỳ khẳng định nào.” Cách duy nhất để câu này sai là khi P đúng nhưng Q sai.

Song điều kiện, $P \Leftrightarrow Q$, đúng bất cứ khi nào cả $P \Rightarrow Q$ và $Q \Rightarrow P$ đều đúng. Trong tiếng Anh, điều này thường được viết là ” P nếu và chỉ nếu Q “. Nhiều quy tắc của thế giới wumpus được viết tốt nhất bằng cách sử dụng $\Leftrightarrow$. Ví dụ, một ô vuông có gió nếu một ô vuông lân cận có hố, và một ô vuông có gió chỉ khi một ô vuông lân cận có hố. Vì vậy, chúng ta cần một song điều kiện:

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}),$$

trong đó $B_{1,1}$ có nghĩa là có gió ở $[1,1]$.

7.4.3 Một cơ sở tri thức đơn giản

Bây giờ chúng ta có thể xây dựng một cơ sở tri thức cho thế giới wumpus. Trước tiên, chúng tôi tập trung vào các khía cạnh bất biến của thế giới wumpus, để các khía cạnh thay đổi cho một phần sau. Hiện tại, chúng tôi cần các ký hiệu sau cho mỗi vị trí $[x,y]$:

- $P_{x,y}$ đúng nếu có một hố ở $[x,y]$.
- $W_{x,y}$ đúng nếu có một wumpus ở $[x,y]$, dù sống hay chết.
- $B_{x,y}$ đúng nếu có gió ở $[x,y]$.
- $S_{x,y}$ đúng nếu có mùi hôi ở $[x,y]$.
- $L_{x,y}$ đúng nếu tác nhân ở vị trí $[x,y]$.

Các câu chúng tôi viết sẽ đủ để suy ra $\neg P_{1,2}$ (không có hố ở $[1,2]$), như đã được thực hiện một cách không chính thức trong Phần 7.3. Chúng tôi gán nhãn mỗi câu là R_i để có thể tham chiếu chúng:

- Không có hố ở $[1,1]$:

$$R_1: \neg P_{1,1}.$$

- Một ô vuông có gió nếu và chỉ nếu có một hố trong một ô vuông lân cận. Điều này phải được nêu cho mỗi ô vuông; hiện tại, chúng tôi chỉ bao gồm các ô vuông liên quan:

$$R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}).$$

$$R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1}).$$

- Các câu trước đúng trong mọi thế giới wumpus. Bây giờ chúng tôi bao gồm các nhận thức về gió cho hai ô vuông đầu tiên mà tác nhân đã ghé thăm trong thế giới cụ thể mà tác nhân đang ở, dẫn đến tình huống trong Hình 7.3(b).

$$R_4: \neg B_{1,1}.$$

$$R_5: B_{2,1}.$$

7.4.4 Một quy trình suy luận đơn giản

Mục tiêu của chúng ta bây giờ là quyết định xem $KB \models \alpha$ cho một số câu α . Ví dụ, $\neg P_{1,2}$ có được kéo theo từ KB của chúng ta không? Thuật toán đầu tiên của chúng ta để suy luận là một cách tiếp cận kiểm tra mô hình, triển khai trực tiếp định nghĩa của sự kéo theo: liệt kê các mô hình và kiểm tra rằng α đúng trong mọi mô hình mà KB đúng. Các mô hình là các phép gán đúng hoặc sai cho mọi ký hiệu mệnh đề. Quay lại ví dụ thế giới wumpus của chúng ta, các ký hiệu mệnh đề liên quan là $B_{1,1}, B_{2,1}, P_{1,1}, P_{1,2}, P_{2,1}, P_{2,2}$ và $P_{3,1}$. Với bảy ký hiệu, có $2^7 = 128$ mô hình khả dĩ; trong ba mô hình này, KB đúng (Hình 7.9). Trong ba mô hình đó, $\neg P_{1,2}$ đúng, do đó không có hồ ở $[1,2]$. Mặt khác, $P_{2,2}$ đúng trong hai trong số ba mô hình và sai trong một, vì vậy chúng ta chưa thể biết liệu có một hồ ở $[2,2]$ hay không.

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	false	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	true	true	true	true	true	true	<u>true</u>
false	true	false	false	true	false	false	true	false	false	true	true	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
true	true	true	true	true	true	true	false	true	true	false	true	false

Hình 7.9 Một bảng chân lý được xây dựng cho cơ sở tri thức được đưa ra trong văn bản. KB đúng nếu R_1 đến R_5 đúng, điều này chỉ xảy ra trong 3 trong số 128 hàng (các hàng được gạch dưới ở cột ngoài cùng bên phải). Trong cả 3 hàng, $P_{1,2}$ sai, vì vậy không có hồ ở $[1,2]$. Mặt khác, có thể có (hoặc có thể không) có một cái hồ ở $[2,2]$.

Hình 7.9 tái tạo một cách chính xác hơn lập luận được minh họa trong Hình 7.5. Một thuật toán tổng quát để quyết định sự kéo theo trong logic mệnh đề được hiển thị trong Hình 7.10. Giống như thuật toán Backtracking-Search ở trang 176, TT-Entails? thực hiện một liệt kê đệ quy theo chiều sâu của một không gian hữu hạn các phép gán cho các ký hiệu. Thuật toán này đúng vì nó triển khai trực tiếp định nghĩa của sự kéo theo và đầy đủ vì nó hoạt động cho bất kỳ KB và α nào và luôn dừng—chỉ có hữu hạn mô hình để kiểm tra.


```

function TT-ENTAILS?(KB,  $\alpha$ ) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
            $\alpha$ , the query, a sentence in propositional logic

  symbols  $\leftarrow$  a list of the proposition symbols in KB and  $\alpha$ 
  return TT-CHECK-ALL(KB,  $\alpha$ , symbols, { })

function TT-CHECK-ALL(KB,  $\alpha$ , symbols, model) returns true or false
  if EMPTY?(symbols) then
    if PL-TRUE?(KB, model) then return PL-TRUE?( $\alpha$ , model)
    else return true      // when KB is false, always return true
  else
    P  $\leftarrow$  FIRST(symbols)
    rest  $\leftarrow$  REST(symbols)
    return (TT-CHECK-ALL(KB,  $\alpha$ , rest, model  $\cup$  {P = true})
           and
           TT-CHECK-ALL(KB,  $\alpha$ , rest, model  $\cup$  {P = false }))

```

Hình 7.10 Một thuật toán liệt kê bảng chân lý để quyết định sự bao hàm mệnh đề. (TT là viết tắt của bảng chân lý.) PL-TRUE? trả về giá trị true nếu một câu đúng trong một mô hình. Biến model đại diện cho một mô hình một phần—một sự gán giá trị cho một số ký hiệu. Từ khóa and ở đây là một ký hiệu hàm trung tố trong ngôn ngữ lập trình giả mã, không phải là một toán tử trong logic mệnh đề; nó nhận hai đối số và trả về true hoặc false.

Tất nhiên, “hữu hạn” không phải lúc nào cũng giống như “ít”. Nếu *KB* và α chứa tổng cộng *n* ký hiệu, thì có 2^n mô hình. Do đó, độ phức tạp thời gian của thuật toán là $O(2^n)$. (Độ phức tạp không gian chỉ là $O(n)$ vì việc liệt kê là theo chiều sâu.) Sau này trong chương này, chúng tôi sẽ trình bày các thuật toán hiệu quả hơn trong nhiều trường hợp. Thật không may, vấn đề kéo theo mệnh đề là co-NP-đầy đủ (tức là có lẽ không dễ hơn NP-đầy đủ—xem Phụ lục A), vì vậy mọi thuật toán suy luận đã biết cho logic mệnh đề đều có độ phức tạp trong trường hợp xấu nhất là hàm mũ theo kích thước đầu vào.

7.5 Định lý Chứng minh Mệnh đề

Trong phần trước, chúng ta đã chỉ ra cách xác định hệ quả logic bằng cách kiểm tra mô hình: liệt kê các mô hình và chứng minh rằng câu cần kiểm tra phải đúng trong tất cả các mô hình. Trong phần này, chúng ta sẽ trình bày cách thực hiện hệ quả logic bằng **định lý chứng minh**—áp dụng trực tiếp các quy tắc suy luận vào các câu trong cơ sở tri thức để xây dựng một chứng minh cho câu mong muốn mà không cần tham khảo các mô hình. Nếu số lượng mô hình lớn nhưng độ dài của chứng minh ngắn, thì định lý chứng minh có thể hiệu quả hơn kiểm tra mô hình.

$$\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}$$

Hình 7.11 Các tương đương logic chuẩn. Các ký hiệu α , β , và γ đại diện cho các câu tùy ý của logic mệnh đề.

Trước khi đi vào chi tiết các thuật toán định lý chứng minh, chúng ta cần một số khái niệm bổ sung liên quan đến hệ quả logic. Khái niệm đầu tiên là **tương đương logic**: hai câu α và β được gọi là tương đương logic nếu chúng đúng trong cùng một tập hợp các mô hình. Ký hiệu $\alpha \equiv \beta$. (Lưu ý rằng $\equiv$ dùng để khẳng định về các câu, trong khi $\Leftrightarrow$ là một phần của câu.) Chẳng hạn, ta có thể dễ dàng chứng minh (bằng bảng chân lý) rằng $P \wedge Q$ và $Q \wedge P$ là tương đương logic; các tương đương khác được trình bày trong Hình 7.11. Những tương đương này đóng vai trò tương tự như các đẳng thức số học trong toán học thông thường.

Một định nghĩa khác của tương đương là: hai câu α và β tương đương nếu và chỉ nếu mỗi câu là hệ quả của câu kia:

$$\alpha \equiv \beta \quad \text{nếu và chỉ nếu} \quad \alpha \models \beta \quad \text{và} \quad \beta \models \alpha.$$

Khái niệm thứ hai chúng ta cần là **tính hợp lệ (validity)**. Một câu được gọi là hợp lệ nếu nó đúng trong mọi mô hình. Ví dụ, câu $P \vee \neg P$ là hợp lệ. Các câu hợp lệ còn được gọi là **hằng đúng (tautologies)**—chúng luôn đúng. Vì câu **True** luôn đúng trong mọi mô hình, nên mọi câu hợp lệ đều tương đương logic với **True**. Các câu hợp lệ có ích như thế nào? Từ định nghĩa của hệ quả logic, ta có thể suy ra **định lý suy luận (deduction theorem)**, vốn đã được người Hy Lạp cổ đại biết đến:

$$\text{Với mọi câu } \alpha \text{ và } \beta, \alpha \models \beta \text{ nếu và chỉ nếu câu } (\alpha \Rightarrow \beta) \text{ là hợp lệ.}$$

(Phần Bài tập 7.DEDU yêu cầu chứng minh điều này.) Do đó, ta có thể kiểm tra $\alpha \models \beta$ bằng cách xác minh rằng $(\alpha \Rightarrow \beta)$ đúng trong mọi mô hình—về cơ bản là những gì thuật toán suy luận trong Hình 7.10 thực hiện—hoặc bằng cách chứng minh rằng $(\alpha \Rightarrow \beta)$ tương đương với **True**. Ngược lại, định lý suy luận khẳng định rằng mọi câu kéo theo hợp lệ đều mô tả một suy luận hợp lệ.

Khái niệm cuối cùng chúng ta cần là **tính thỏa được (satisfiability)**. Một câu được gọi là thỏa được nếu nó đúng trong một mô hình nào đó. Ví dụ, cơ sở tri thức đã cho trước đó, $(R_1 \wedge R_2 \wedge R_3 \wedge R_4 \wedge R_5)$, là thỏa được vì có ba mô hình mà nó đúng, như trong Hình 7.9. Tính thỏa được có

thể được kiểm tra bằng cách liệt kê các mô hình khả dĩ cho đến khi tìm thấy một mô hình thỏa mãn câu đó. Bài toán xác định tính thỏa được của các câu trong logic mệnh đề—**bài toán SAT**—là bài toán đầu tiên được chứng minh là **NP-đầy đủ**. Nhiều vấn đề trong khoa học máy tính thực chất là các bài toán thỏa mãn ràng buộc. Chẳng hạn, tất cả các bài toán thỏa mãn ràng buộc trong Chương 5 đều hỏi liệu các ràng buộc có thể được thỏa mãn bởi một phép gán nào đó hay không.

Tính hợp lệ và tính thỏa được có mối liên hệ chặt chẽ: α là hợp lệ khi và chỉ khi $\neg\alpha$ không thỏa được; ngược lại, α là thỏa được khi và chỉ khi $\neg\alpha$ không hợp lệ. Chúng ta cũng có kết quả hữu ích sau:

$$\alpha \models \beta \text{ nếu và chỉ nếu câu } (\alpha \wedge \neg\beta)$$

Việc chứng minh β từ α bằng cách kiểm tra tính không thỏa được của $(\alpha \wedge \neg\beta)$ tương ứng chính xác với kỹ thuật chứng minh toán học tiêu chuẩn **phản chứng (reductio ad absurdum)** (nghĩa là “dẫn đến điều vô lý”). Nó còn được gọi là **chứng minh bằng phủ định** hoặc **chứng minh bằng mâu thuẫn**. Người ta giả định câu β là sai và chỉ ra rằng điều này dẫn đến mâu thuẫn với các tiên đề đã biết α . Mâu thuẫn này chính là điều được hiểu khi nói rằng câu $(\alpha \wedge \neg\beta)$ không thỏa được.

7.5.1 Suy luận và Chứng minh

Phần này trình bày các quy tắc suy luận có thể được áp dụng để suy ra một **chứng minh**—một chuỗi kết luận dẫn đến mục tiêu mong muốn. Quy tắc nổi tiếng nhất là **Modus Ponens** (tiếng Latinh có nghĩa là “phương thức khẳng định”) và được viết như sau:

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}.$$

Ký hiệu này có nghĩa là, bất cứ khi nào có các câu dạng $\alpha \Rightarrow \beta$ và α , thì câu β có thể được suy ra. Ví dụ, nếu $(WumpusAhead \wedge WumpusAlive) \Rightarrow Shoot$ và $(WumpusAhead \wedge WumpusAlive)$ được cho, thì $Shoot$ có thể được suy ra.

Một quy tắc suy luận hữu ích khác là **Loại bỏ Và (And-Elimination)**, khẳng định rằng từ một phép hội, bất kỳ thành phần nào trong đó cũng có thể được suy ra:

$$\frac{\alpha \wedge \beta}{\alpha}.$$

Ví dụ, từ $(WumpusAhead \wedge WumpusAlive)$, có thể suy ra $WumpusAlive$.

Bằng cách xem xét các giá trị chân lý khả dĩ của α và β , ta có thể dễ dàng chứng minh một lần cho tất cả rằng Modus Ponens và Loại bỏ Và là **đúng đắn (sound)**. Những quy tắc này sau đó có thể được sử dụng trong bất kỳ trường hợp cụ thể nào mà chúng áp dụng, tạo ra các suy luận đúng đắn mà không cần liệt kê các mô hình.

Tất cả các tương đương logic trong Hình 7.11 đều có thể được sử dụng như các quy tắc suy luận. Ví dụ, tương đương cho phép loại bỏ kéo theo hai chiều tạo ra hai quy tắc suy luận:

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \quad \text{và} \quad \frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}.$$

Không phải tất cả các quy tắc suy luận đều hoạt động theo cả hai hướng như vậy. Ví dụ, ta không thể chạy Modus Ponens theo hướng ngược lại để thu được $\alpha \Rightarrow \beta$ và α từ β .

Hãy xem cách các quy tắc suy luận và tương đương này có thể được sử dụng trong thế giới wumpus. Chúng ta bắt đầu với cơ sở tri thức chứa R_1 đến R_5 và chỉ ra cách chứng minh $\neg P_{1,2}$, tức là không có hồ ở $[1,2]$:

3. Áp dụng phép loại bỏ kéo theo hai chiều vào R_2 để thu được:

$$R_6: (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}).$$

4. Áp dụng Loại bỏ Và vào R_6 để thu được:

$$R_7: ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}).$$

5. Tương đương logic cho phép phản đảo cho:

$$R_8: (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1})).$$

6. Áp dụng Modus Ponens với R_8 và tri giác R_4 (tức là $\neg B_{1,1}$), để thu được:

$$R_9: \neg(P_{1,2} \vee P_{2,1}).$$

7. Áp dụng quy tắc De Morgan, ta có kết luận:

$$R_{10}: \neg P_{1,2} \wedge \neg P_{2,1}.$$

Nghĩa là, không có hồ ở $[1,2]$ hoặc $[2,1]$.

Bất kỳ thuật toán tìm kiếm nào trong Chương 3 đều có thể được sử dụng để tìm một chuỗi các bước tạo thành một chứng minh như trên. Chúng ta chỉ cần định nghĩa một bài toán chứng minh như sau:

- **Trạng thái ban đầu:** cơ sở tri thức ban đầu.
- **Hành động:** tập hợp các hành động bao gồm tất cả các quy tắc suy luận áp dụng cho tất cả các câu khớp với nửa trên của quy tắc suy luận.
- **Kết quả:** kết quả của một hành động là thêm câu ở nửa dưới của quy tắc suy luận.
- **Mục tiêu:** mục tiêu là một trạng thái chứa câu mà chúng ta đang cố gắng chứng minh.

Do đó, tìm kiếm các chứng minh là một phương án thay thế cho việc liệt kê các mô hình. Trong nhiều trường hợp thực tế, việc tìm một chứng minh có thể hiệu quả hơn vì chứng minh có thể bỏ qua các mệnh đề không liên quan, bất kể có bao nhiêu mệnh đề như vậy. Ví dụ, chứng minh vừa rồi dẫn đến $\neg P_{1,2} \wedge \neg P_{2,1}$ không đề cập đến các mệnh đề $B_{2,1}, P_{1,1}, P_{2,2}$, hoặc $P_{3,1}$. Chúng có thể được bỏ qua vì mệnh đề mục tiêu, $P_{1,2}$, chỉ xuất hiện trong câu R_2 ; các mệnh đề khác trong R_2 chỉ xuất hiện trong R_4 và R_5 ; do đó, R_1, R_3 , và R_5 không liên quan đến chứng minh. Điều tương tự cũng đúng ngay cả khi chúng ta thêm hàng triệu câu vào cơ sở tri thức; trong khi đó, thuật toán bảng chân lý đơn giản sẽ bị quá tải bởi sự bùng nổ theo cấp số nhân của các mô hình.

Một tính chất cuối cùng của các hệ thống logic là **tính đơn điệu (monotonicity)**, khẳng định rằng tập hợp các câu được suy ra chỉ có thể tăng lên khi thêm thông tin vào cơ sở tri thức. Với mọi câu α và β :

$$\text{nếu } KB \models \alpha \text{ thì } KB \wedge \beta \models \alpha.$$

Ví dụ, giả sử cơ sở tri thức chứa một khẳng định bổ sung β nói rằng có đúng tám hổ trong thế giới. Tri thức này có thể giúp tác nhân rút ra các kết luận bổ sung, nhưng không thể làm mất hiệu lực bất kỳ kết luận α nào đã được suy ra trước đó—chẳng hạn như kết luận rằng không có hổ ở [1,2]. Tính đơn điệu có nghĩa là các quy tắc suy luận có thể được áp dụng bất cứ khi nào tìm thấy các tiền đề phù hợp trong cơ sở tri thức—kết luận của quy tắc phải được suy ra bất kể những gì khác có trong cơ sở tri thức.

7.5.2 Chứng minh bằng Giải tích

Chúng ta đã lập luận rằng các quy tắc suy luận được đề cập cho đến nay là đúng đắn, nhưng chúng ta chưa thảo luận về vấn đề **đầy đủ (completeness)** của các thuật toán suy luận sử dụng chúng. Các thuật toán tìm kiếm như tìm kiếm theo chiều sâu lặp (trang 99) là đầy đủ theo nghĩa chúng sẽ tìm thấy bất kỳ mục tiêu nào có thể đạt được, nhưng nếu các quy tắc suy luận hiện có không đủ, thì mục tiêu không thể đạt được—không tồn tại chứng minh chỉ sử dụng các quy tắc suy luận đó. Ví dụ, nếu chúng ta loại bỏ quy tắc loại bỏ kéo theo hai chiều, chứng minh trong phần trước sẽ không thực hiện được. Phần này giới thiệu một quy tắc suy luận duy nhất, **giải tích (resolution)**, tạo ra một thuật toán suy luận đầy đủ khi kết hợp với bất kỳ thuật toán tìm kiếm đầy đủ nào.

Trước tiên, chúng ta sử dụng một phiên bản đơn giản của quy tắc giải tích trong thế giới wumpus. Hãy xem xét các bước dẫn đến Hình 7.4(a): tác nhân quay trở lại từ [2,1] về [1,1] và sau đó đi đến [1,2], nơi nó nhận thấy mùi hôi nhưng không có gió. Chúng ta thêm các sự kiện sau vào cơ sở tri thức:

$$R_{11}: \neg B_{1,2}.$$

$$R_{12}: B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3}).$$

Bằng quy trình tương tự dẫn đến R_{10} trước đó, bây giờ chúng ta có thể suy ra sự vắng mặt của các hổ ở [2,2] và [1,3] (nhớ rằng [1,1] đã được biết là không có hổ):

$$R_{13}: \neg P_{2,2}.$$

$$R_{14}: \neg P_{1,3}.$$

Chúng ta cũng có thể áp dụng phép loại bỏ kéo theo hai chiều cho R_3 , sau đó là Modus Ponens với R_5 , để thu được sự kiện có một hổ ở [1,1], [2,2], hoặc [3,1]:

$$R_{15}: P_{1,1} \vee P_{2,2} \vee P_{3,1}.$$

Bây giờ là lần áp dụng đầu tiên của quy tắc giải tích: literal $\neg P_{2,2}$ trong R_{13} giải tích với literal $P_{2,2}$ trong R_{15} để cho **kết quả giải tích (resolvent)**:

$$R_{16}: P_{1,1} \vee P_{3,1}.$$

Nói cách khác: nếu có một hỏ ở một trong các vị trí [1,1], [2,2], và [3,1] và nó không ở [2,2], thì nó phải ở [1,1] hoặc [3,1]. Tương tự, literal $\neg P_{1,1}$ trong R_1 giải tích với literal $P_{1,1}$ trong R_{16} để cho:

$$R_{17}: P_{3,1}.$$

Nói cách khác: nếu có một hỏ ở [1,1] hoặc [3,1] và nó không ở [1,1], thì nó phải ở [3,1]. Hai bước suy luận cuối cùng này là ví dụ về quy tắc suy luận **giải tích đơn vị (unit resolution)**:

$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k},$$

trong đó ℓ_i và m là các literal bù nhau (tức là một cái là phủ định của cái kia). Do đó, quy tắc giải tích đơn vị lấy một mệnh đề—một phép tuyển của các literal—và một literal, rồi tạo ra một mệnh đề mới. Lưu ý rằng một literal đơn lẻ có thể được xem như một phép tuyển của một literal, còn được gọi là **mệnh đề đơn vị (unit clause)**.

Quy tắc giải tích đơn vị có thể được tổng quát hóa thành quy tắc giải tích đầy đủ:

$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m_1 \vee \dots \vee m_n}{\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n},$$

trong đó ℓ_i và m_j là các literal bù nhau. Điều này có nghĩa là giải tích lấy hai mệnh đề và tạo ra một mệnh đề mới chứa tất cả các literal của hai mệnh đề ban đầu ngoại trừ hai literal bù nhau. Ví dụ:

$$\frac{P_{1,1} \vee P_{3,1}, \quad \neg P_{1,1} \vee \neg P_{2,2}}{P_{3,1} \vee \neg P_{2,2}}.$$

Bạn chỉ có thể giải tích một cặp literal bù nhau tại một thời điểm. Ví dụ, bạn có thể giải tích P và $\neg P$ để suy ra:

$$\frac{P \vee \neg Q \vee R, \quad \neg P \vee Q}{\neg Q \vee Q \vee R},$$

nhưng bạn không thể giải tích đồng thời cả P và Q để suy ra R .

Một khía cạnh kỹ thuật nữa của quy tắc giải tích là: mệnh đề kết quả chỉ nên chứa một bản sao của mỗi literal. Việc loại bỏ các bản sao trùng lặp của các literal được gọi là **nhân tử hóa (factoring)**. Ví dụ, nếu chúng ta giải tích $(A \vee B)$ với $(A \vee \neg B)$, chúng ta thu được $(A \vee A)$, được rút gọn thành chỉ A bằng cách nhân tử hóa.

Tính đúng đắn của quy tắc giải tích có thể được thấy rõ bằng cách xem xét literal ℓ_i bù với literal m_j trong mệnh đề kia. Nếu ℓ_i đúng, thì m_j sai, và do đó $m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n$ phải đúng, vì $m_1 \vee \dots \vee m_n$ đã được cho. Nếu ℓ_i sai, thì $\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k$ phải đúng vì $\ell_1 \vee \dots \vee \ell_k$ đã được cho. Bây giờ, ℓ_i hoặc đúng hoặc sai, nên một trong hai kết luận này phải đúng—chính xác như quy tắc giải tích khẳng định.

Điều đáng ngạc nhiên hơn về quy tắc giải tích là nó tạo thành cơ sở cho một họ các thủ tục suy luận đầy đủ. Một chương trình chứng minh định lý dựa trên giải tích có thể, đối với bất kỳ câu α

và β nào trong logic mệnh đề, quyết định xem $\alpha \models \beta$ có đúng hay không. Hai phần tiếp theo sẽ giải thích cách giải tích đạt được điều này.

Dạng chuẩn hội (Conjunctive Normal Form)

Quy tắc giải tích chỉ áp dụng cho các mệnh đề (tức là các phép tuyển của các literal), vì vậy nó dường như chỉ liên quan đến các cơ sở tri thức và truy vấn bao gồm các mệnh đề. Vậy làm thế nào nó có thể dẫn đến một thủ tục suy luận đầy đủ cho tất cả các logic mệnh đề? Câu trả lời là mọi câu của logic mệnh đề đều tương đương logic với một phép hội của các mệnh đề.

$$\begin{aligned}
 CNFSentence &\rightarrow Clause_1 \wedge \dots \wedge Clause_n \\
 Clause &\rightarrow Literal_1 \vee \dots \vee Literal_m \\
 Fact &\rightarrow Symbol \\
 Literal &\rightarrow Symbol \mid \neg Symbol \\
 Symbol &\rightarrow P \mid Q \mid R \mid \dots \\
 HornClauseForm &\rightarrow DefiniteClauseForm \mid GoalClauseForm \\
 DefiniteClauseForm &\rightarrow Fact \mid (Symbol_1 \wedge \dots \wedge Symbol_l) \Rightarrow Symbol \\
 GoalClauseForm &\rightarrow (Symbol_1 \wedge \dots \wedge Symbol_l) \Rightarrow False
 \end{aligned}$$

Hình 7.12 Một ngữ pháp cho dạng chuẩn hội, mệnh đề Horn và mệnh đề xác định. Một mệnh đề CNF như $\neg A \vee \neg B \vee C$ có thể được viết ở dạng mệnh đề xác định là $(A \wedge B) \Rightarrow C$.

Một câu được biểu diễn dưới dạng phép hội của các mệnh đề được gọi là ở **dạng chuẩn hội (conjunctive normal form - CNF)** (xem Hình 7.12). Bây giờ chúng ta sẽ mô tả một quy trình chuyển đổi sang CNF. Chúng ta minh họa quy trình bằng cách chuyển đổi câu $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ thành CNF. Các bước như sau:

8. **Loại bỏ $\Leftrightarrow$** , thay thế $\alpha \Leftrightarrow \beta$ bằng $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$:

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}).$$

9. **Loại bỏ $\Rightarrow$** , thay thế $\alpha \Rightarrow \beta$ bằng $\neg\alpha \vee \beta$:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1}).$$

10. CNF yêu cầu $\neg$ chỉ xuất hiện trong các literal, vì vậy chúng ta “đưa $\neg$ vào trong” bằng cách áp dụng lặp lại các tương đương từ Hình 7.11:

- $\neg(\neg\alpha) \equiv \alpha$ (loại bỏ phủ định kép)
- $\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$ (De Morgan)
- $\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$ (De Morgan)

Trong ví dụ này, chúng ta chỉ cần áp dụng quy tắc cuối cùng:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1}).$$

11. Bây giờ chúng ta có một câu chứa các toán tử $\wedge$ và $\vee$ lồng nhau áp dụng cho các literal. Chúng ta áp dụng **luật phân phối** từ Hình 7.11, phân phối $\vee$ qua $\wedge$ ở mọi nơi có thể:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1}).$$

Câu ban đầu bây giờ đã ở dạng CNF, dưới dạng một phép hội của ba mệnh đề. Mặc dù khó đọc hơn, nhưng nó có thể được sử dụng làm đầu vào cho một thủ tục giải tích.

Một thuật toán giải tích

Các thủ tục suy luận dựa trên giải tích hoạt động bằng cách sử dụng nguyên lý **chứng minh bằng phản chứng** được giới thiệu ở trang 241. Tức là, để chứng minh $KB \models \alpha$, chúng ta chứng minh rằng $(KB \wedge \neg\alpha)$ là không thỏa được. Chúng ta thực hiện điều này bằng cách chứng minh một mâu thuẫn.

function PL-RESOLUTION(KB, α) **returns** *true or false*

inputs: KB , the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic

$clauses \leftarrow$ the set of clauses in the CNF representation of $KB \wedge \neg\alpha$

$new \leftarrow \{\}$

while true do

for each pair of clauses C_i, C_j **in** $clauses$ **do**

$resolvents \leftarrow$ PL-RESOLVE(C_i, C_j)

if $resolvents$ contains the empty clause **then return true**

$new \leftarrow new \cup resolvents$

if $new \subseteq clauses$ **then return false**

$clauses \leftarrow clauses \cup new$

Hình 7.13 Một thuật toán giải quyết đơn giản cho logic mệnh đề. PL-RESOLVE trả về tập hợp tất cả các mệnh đề khả thi thu được bằng cách giải quyết hai đầu vào của nó.

Một thuật toán giải tích được trình bày trong Hình 7.13. Đầu tiên, $(KB \wedge \neg\alpha)$ được chuyển đổi thành CNF. Sau đó, quy tắc giải tích được áp dụng cho các mệnh đề kết quả. Mỗi cặp chứa các literal bù nhau được giải tích để tạo ra một mệnh đề mới, được thêm vào tập hợp nếu nó chưa có. Quá trình tiếp tục cho đến khi một trong hai điều sau xảy ra:

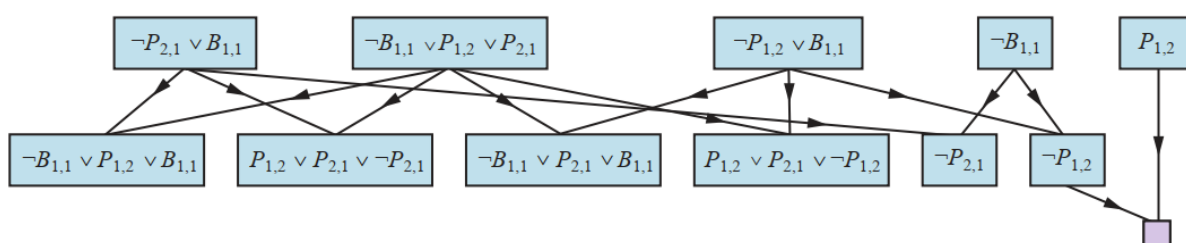
- Không có mệnh đề mới nào có thể được thêm vào, trong trường hợp này KB không suy ra α ; hoặc,
- Hai mệnh đề giải tích để tạo ra **mệnh đề rỗng (empty clause)**, trong trường hợp này KB suy ra α .

Mệnh đề rỗng—một phép tuyên không có mệnh đề nào—tương đương với **False** vì một phép tuyên chỉ đúng khi ít nhất một trong các mệnh đề của nó đúng. Hơn nữa, mệnh đề rỗng chỉ phát sinh từ việc giải tích hai mệnh đề đơn vị mâu thuẫn, chẳng hạn như P và $\neg P$.

Chúng ta có thể áp dụng thủ tục giải tích cho một suy luận rất đơn giản trong thế giới wumpus. Khi tác nhân ở [1,1], không có gió, vì vậy không thể có hổ ở các ô lân cận. Cơ sở tri thức liên quan là:

$$KB = R_2 \wedge R_4 = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1},$$

và chúng ta muốn chứng minh α , chẳng hạn là $\neg P_{1,2}$. Khi chúng ta chuyển đổi $(KB \wedge \neg \alpha)$ thành CNF, chúng ta thu được các mệnh đề được hiển thị ở đầu Hình 7.14. Hàng thứ hai của hình cho thấy các mệnh đề thu được bằng cách giải tích các cặp trong hàng đầu tiên. Sau đó, khi $P_{1,2}$ được giải tích với $\neg P_{1,2}$, chúng ta thu được mệnh đề rỗng, được hiển thị dưới dạng một hình vuông nhỏ. Kiểm tra Hình 7.14 cho thấy nhiều bước giải tích là vô nghĩa. Ví dụ, mệnh đề $B_{1,1} \vee \neg B_{1,1} \vee P_{1,2}$ tương đương với **True** $\vee P_{1,2}$, tương đương với **True**. Suy ra rằng **True** là đúng không hữu ích lắm. Do đó, bất kỳ mệnh đề nào chứa hai literal bù nhau đều có thể bị loại bỏ.



Hình 7.14 Ứng dụng một phần của **PL-RESOLUTION** cho một suy luận đơn giản trong thế giới wumpus để chứng minh truy vấn $\neg P_{1,2}$. Mỗi một trong bốn mệnh đề ngoài cùng bên trái ở hàng trên cùng được ghép nối với mỗi mệnh đề trong ba mệnh đề còn lại, và quy tắc giải quyết được áp dụng để tạo ra các mệnh đề ở hàng dưới cùng. Chúng ta thấy rằng mệnh đề thứ ba và thứ tư ở hàng trên cùng kết hợp để tạo ra mệnh đề $\neg P_{1,2}$, sau đó được giải quyết với $P_{1,2}$ để tạo ra mệnh đề rỗng, có nghĩa là truy vấn được chứng minh.

Tính đầy đủ của giải tích

Để kết luận phần thảo luận về giải tích, bây giờ chúng ta sẽ chỉ ra tại sao PL-Resolution là đầy đủ. Để làm điều này, chúng ta giới thiệu **bao đóng giải tích (resolution closure)** $RC(S)$ của một tập hợp các mệnh đề S , là tập hợp tất cả các mệnh đề có thể suy ra bằng cách áp dụng lặp lại quy tắc giải tích cho các mệnh đề trong S hoặc các dẫn xuất của chúng. Bao đóng giải tích là những gì PL-Resolution tính toán như giá trị cuối cùng của biến clauses. Dễ dàng thấy rằng $RC(S)$ phải là hữu hạn: nhờ bước nhân tử hóa, chỉ có hữu hạn các mệnh đề phân biệt có thể được xây dựng từ các ký hiệu $P_1, \dots, P_k$ xuất hiện trong S . Do đó, PL-Resolution luôn kết thúc.

Định lý đầy đủ cho giải tích trong logic mệnh đề được gọi là **định lý giải tích cơ bản (ground resolution theorem)**: Nếu một tập hợp các mệnh đề là không thỏa được, thì bao đóng giải tích của các mệnh đề đó chứa mệnh đề rỗng.

Định lý này được chứng minh bằng cách chứng minh phản đảo của nó: nếu bao đóng $RC(S)$ không chứa mệnh đề rỗng, thì S là thỏa được. Trên thực tế, chúng ta có thể xây dựng một mô hình cho S với các giá trị chân lý phù hợp cho $P_1, \dots, P_k$. Quy trình xây dựng như sau:

Với mỗi i từ 1 đến k ,

- Nếu một mệnh đề trong $RC(S)$ chứa literal $\neg P_i$ và tất cả các literal khác đều sai dưới phép gán đã chọn cho $P_1, \dots, P_{i-1}$, thì gán **false** cho P_i .
- Ngược lại, gán **true** cho P_i .

Phép gán này cho $P_1, \dots, P_k$ là một mô hình của S . Để thấy điều này, giả sử ngược lại—rằng tại một giai đoạn i nào đó trong chuỗi, việc gán ký hiệu P_i khiến một mệnh đề C nào đó trở thành sai. Để điều này xảy ra, tất cả các literal khác trong C phải đã bị phủ định bởi các phép gán cho $P_1, \dots, P_{i-1}$. Do đó, C lúc này phải có dạng như $(\text{false} \vee \text{false} \vee \dots \vee \text{false} \vee P_i)$ hoặc $(\text{false} \vee \text{false} \vee \dots \vee \text{false} \vee \neg P_i)$. Nếu chỉ một trong hai mệnh đề này nằm trong $RC(S)$, thì thuật toán sẽ gán giá trị chân lý phù hợp cho P_i để C trở thành đúng. Vì vậy, C chỉ có thể bị phủ định nếu cả hai mệnh đề này đều có trong $RC(S)$.

Bây giờ, vì $RC(S)$ đóng dưới phép giải, nó sẽ chứa kết quả giải của hai mệnh đề này, và kết quả giải đó sẽ có tất cả các literal của nó đã bị phủ định bởi các phép gán cho $P_1, \dots, P_{i-1}$. Điều này mâu thuẫn với giả định ban đầu rằng mệnh đề bị phủ định đầu tiên xuất hiện ở giai đoạn i . Do đó, chúng ta đã chứng minh rằng quá trình xây dựng này không bao giờ phủ định một mệnh đề trong $RC(S)$; nghĩa là, nó tạo ra một mô hình của $RC(S)$. Cuối cùng, vì S được chứa trong $RC(S)$, bất kỳ mô hình nào của $RC(S)$ cũng là một mô hình của S chính nó.

7.5.3 Mệnh đề Horn và mệnh đề xác định

Sự đầy đủ của luật phân giải làm cho nó trở thành một phương pháp suy luận quan trọng. Tuy nhiên, trong nhiều tình huống thực tế, sức mạnh đầy đủ của phân giải là không cần thiết. Một số cơ sở tri thức trong thực tế tuân theo các hạn chế nhất định về hình thức của các câu, điều này cho phép chúng sử dụng một thuật toán suy luận hạn chế hơn nhưng hiệu quả hơn.

Một dạng hạn chế như vậy là **mệnh đề xác định (definite clause)**, một phép tuyên của các literal trong đó chỉ có một literal là dương. Ví dụ, mệnh đề $(\neg L_{1,1} \vee \neg Breeze \vee B_{1,1})$ là một mệnh đề xác định, trong khi $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1})$ thì không, vì nó có hai literal dương.

Tổng quát hơn một chút là **mệnh đề Horn**, một phép tuyên của các literal trong đó có nhiều nhất một literal dương. Do đó, tất cả các mệnh đề xác định đều là mệnh đề Horn, cũng như các mệnh đề không có literal dương nào; những mệnh đề này được gọi là **mệnh đề mục tiêu (goal clauses)**. Mệnh đề Horn đóng dưới phép phân giải: nếu bạn phân giải hai mệnh đề Horn, bạn sẽ nhận được một mệnh đề Horn. Một lớp khác là câu **k-CNF**, một câu CNF trong đó mỗi mệnh đề có nhiều nhất k literal.

Cơ sở tri thức chỉ chứa các mệnh đề xác định thú vị vì ba lý do:

12. **Biểu diễn ngụ ý:** Mỗi mệnh đề xác định có thể được viết dưới dạng một phép kéo theo, trong đó tiền đề là một phép hội của các literal dương và kết luận là một literal dương duy nhất. Ví dụ, mệnh đề xác định $(\neg L_{1,1} \vee \neg Breeze \vee B_{1,1})$ có thể được viết thành phép kéo theo $(L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$. Ở dạng này, câu dễ hiểu hơn: nếu tác nhân ở $[1,1]$ và nhận được cảm giác có gió, thì $[1,1]$ có gió. Trong mệnh đề Horn, tiền đề được gọi là **thân (body)** và kết luận được gọi là **đầu (head)**. Một câu chỉ gồm một literal dương, chẳng hạn

$L_{1,1}$, được gọi là **sự kiện (fact)**. Nó cũng có thể được viết dưới dạng phép kéo theo $True \Rightarrow L_{1,1}$, nhưng viết đơn giản là $L_{1,1}$ sẽ ngắn gọn hơn.

13. **Suy luận hiệu quả:** Suy luận với mệnh đề Horn có thể được thực hiện thông qua các thuật toán **lan truyền tiến (forward chaining)** và **lan truyền ngược (backward chaining)**, chúng ta sẽ giải thích ngay sau đây. Cả hai thuật toán này đều tự nhiên, vì các bước suy luận rõ ràng và dễ theo dõi đối với con người. Kiểu suy luận này là nền tảng cho **lập trình logic**, sẽ được thảo luận trong Chương 9.
14. **Độ phức tạp tuyến tính:** Quyết định hệ quả logic với mệnh đề Horn có thể được thực hiện trong thời gian tuyến tính so với kích thước của cơ sở tri thức—một điều đáng ngạc nhiên thú vị.

7.5.4 Lan truyền tiến và lan truyền ngược

Thuật toán **lan truyền tiến** (PL-FC-ENTAILS?) xác định xem một ký hiệu mệnh đề đơn q —truy vấn—có được suy ra từ một cơ sở tri thức gồm các mệnh đề xác định hay không. Nó bắt đầu từ các sự kiện đã biết (các literal dương) trong cơ sở tri thức. Nếu tất cả các tiền đề của một phép kéo theo đã được biết, thì kết luận của nó sẽ được thêm vào tập các sự kiện đã biết. Ví dụ, nếu $L_{1,1}$ và $Breeze$ đã biết và $(L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$ nằm trong cơ sở tri thức, thì $B_{1,1}$ có thể được thêm vào. Quá trình này tiếp tục cho đến khi truy vấn q được thêm vào hoặc không thể suy luận thêm được nữa. Thuật toán được trình bày trong Hình 7.15; điểm chính cần nhớ là nó chạy trong thời gian tuyến tính.

function PL-FC-ENTAILS?(KB, q) **returns** *true or false*

inputs: KB , the knowledge base, a set of propositional definite clauses

q , the query, a proposition symbol

$count \leftarrow$ a table, where $count[c]$ is initially the number of symbols in clause c 's premise

$inferred \leftarrow$ a table, where $inferred[s]$ is initially *false* for all symbols

$queue \leftarrow$ a queue of symbols, initially symbols known to be true in KB

while $queue$ is not empty **do**

$p \leftarrow \text{POP}(queue)$

if $p = q$ **then return** *true*

if $inferred[p] = \text{false}$ **then**

$inferred[p] \leftarrow \text{true}$

for each clause c in KB where p is in c .PREMISE **do**

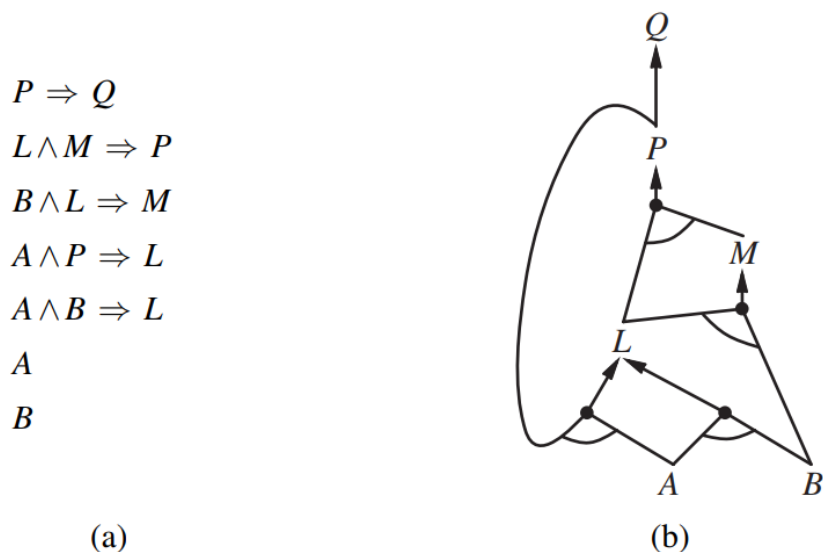
decrement $count[c]$

if $count[c] = 0$ **then** add c .CONCLUSION to $queue$

return *false*

Hình 7.15 Thuật toán suy diễn tiến trong logic mệnh đề. *Hàng đợi (queue)* theo dõi các ký hiệu đã biết là đúng nhưng chưa được “xử lý.” Bảng *count* ghi nhận số lượng tiền đề của mỗi suy diễn chưa được chứng minh. Mỗi khi một ký hiệu mới p từ hàng đợi được xử lý, giá trị *count* sẽ giảm đi một cho mỗi suy diễn mà tiền đề của nó chứa p (có thể xác định nhanh trong thời gian hằng số nhờ lập chỉ mục phù hợp). Nếu *count* đạt giá trị 0, tất cả tiền đề của suy diễn đã được biết đến, do

đó kết luận của nó có thể được thêm vào hàng đợi. Cuối cùng, cần theo dõi các ký hiệu đã được xử lý; một ký hiệu đã có trong tập ký hiệu suy ra không cần thêm vào hàng đợi lần nữa. Điều này tránh công việc trùng lặp và ngăn các vòng lặp vô hạn do các suy diễn như $P \Rightarrow Q$ và $Q \Rightarrow P$ gây ra.



Hình 7.16 (a) Một tập hợp các mệnh đề Horn. (b) Đồ thị AND–OR tương ứng.

Cách tốt nhất để hiểu thuật toán là thông qua một ví dụ và hình ảnh. Hình 7.16(a) cho thấy một cơ sở tri thức đơn giản gồm các mệnh đề Horn với A và B là các sự kiện đã biết. Hình 7.16(b) cho thấy cùng một cơ sở tri thức được vẽ dưới dạng đồ thị AND-OR (xem Chương 4). Trong đồ thị AND-OR, nhiều cạnh được nối bằng một cung biểu thị một phép hội—mọi cạnh phải được chứng minh—trong khi nhiều cạnh không có cung biểu thị một phép tuyển—bất kỳ cạnh nào cũng có thể được chứng minh. Có thể dễ dàng thấy cách lan truyền tiến hoạt động trong đồ thị. Các lá đã biết (ở đây là A và B) được thiết lập, và suy luận lan truyền lên đồ thị càng xa càng tốt. Bất cứ khi nào xuất hiện một phép hội, quá trình lan truyền sẽ chờ cho đến khi tất cả các thành phần của phép hội được biết trước khi tiếp tục. Bạn đọc được khuyến khích làm việc chi tiết qua ví dụ này.

Dễ dàng thấy rằng lan truyền tiến là **đúng đắn (sound)**: mỗi suy luận về cơ bản là một ứng dụng của Modus Ponens. Lan truyền tiến cũng **đầy đủ (complete)**: mọi câu nguyên tử được suy ra sẽ được dẫn xuất. Cách dễ nhất để thấy điều này là xem xét trạng thái cuối cùng của bảng suy luận (sau khi thuật toán đạt đến điểm cô định mà không có suy luận mới nào có thể thực hiện được). Bảng này chứa giá trị true cho mỗi ký hiệu được suy ra trong quá trình và false cho tất cả các ký hiệu khác. Chúng ta có thể xem bảng này như một mô hình logic; hơn nữa, mọi mệnh đề xác định trong cơ sở tri thức ban đầu đều đúng trong mô hình này.

Hơn nữa, bất kỳ câu nguyên tử q nào được suy ra bởi cơ sở tri thức phải đúng trong tất cả các mô hình của nó và trong mô hình này nói riêng. Do đó, mọi câu nguyên tử được suy ra q phải được suy ra bởi thuật toán.

Lan truyền tiến là một ví dụ về khái niệm chung của **lập luận hướng dữ liệu (data-driven reasoning)**—tức là lập luận trong đó trọng tâm chú ý bắt đầu với dữ liệu đã biết. Nó có thể được sử dụng trong một tác nhân để suy ra các kết luận từ các nhận thức đến, thường mà không có một truy vấn cụ thể trong tâm trí. Ví dụ, tác nhân trong thế giới wumpus có thể **nói (Tell)** các nhận thức của nó cho cơ sở tri thức bằng cách sử dụng một thuật toán lan truyền tiến gia tăng, trong đó các sự kiện mới có thể được thêm vào chương trình nghị sự để khởi tạo các suy luận mới. Ở con người, một lượng lập luận hướng dữ liệu nhất định xảy ra khi thông tin mới đến. Ví dụ, nếu tôi đang ở trong nhà và nghe thấy tiếng mưa bắt đầu rơi, tôi có thể nghĩ rằng buổi dã ngoại sẽ bị hủy. Tuy nhiên, có lẽ tôi sẽ không nghĩ rằng cánh hoa thứ mười bảy trên bông hồng lớn nhất trong vườn của hàng xóm sẽ bị ướt; con người kiểm soát chặt chẽ quá trình lan truyền tiến, để không bị choáng ngợp bởi những hậu quả không liên quan.

Thuật toán **lan truyền ngược (backward chaining)**, như tên gọi của nó, hoạt động ngược từ truy vấn. Nếu truy vấn q đã được biết là đúng, thì không cần thực hiện thêm công việc nào. Ngược lại, thuật toán tìm các phép kéo theo trong cơ sở tri thức có kết luận là q . Nếu tất cả các tiền đề của một trong những phép kéo theo đó có thể được chứng minh là đúng (bằng lan truyền ngược), thì q là đúng. Khi áp dụng cho truy vấn Q trong Hình 7.16, nó làm việc ngược xuống đồ thị cho đến khi gặp một tập các sự kiện đã biết, A và B , tạo thành cơ sở cho một bằng chứng. Thuật toán về cơ bản giống với thuật toán AND-OR-GRAPH-SEARCH trong Hình 4.11. Giống như lan truyền tiến, một triển khai hiệu quả chạy trong thời gian tuyến tính.

Lan truyền ngược là một dạng của **lập luận hướng mục tiêu (goal-directed reasoning)**. Nó hữu ích để trả lời các câu hỏi cụ thể như “Tôi nên làm gì bây giờ?” và “Chìa khóa của tôi ở đâu?” Thông thường, chi phí của lan truyền ngược ít hơn nhiều so với thời gian tuyến tính so với kích thước của cơ sở tri thức, vì quá trình chỉ chạm vào các sự kiện có liên quan.

7.6 Kiểm tra mô hình hiệu quả trong logic mệnh đề

Trong phần này, chúng ta sẽ mô tả hai họ thuật toán hiệu quả để suy luận tổng quát trong logic mệnh đề dựa trên kiểm tra mô hình: một phương pháp dựa trên tìm kiếm quay lui (backtracking search) và một phương pháp dựa trên tìm kiếm leo đồi cục bộ (local hill-climbing search). Những thuật toán này là một phần của “công nghệ” logic mệnh đề. Phần này có thể được đọc lướt qua trong lần đọc đầu tiên của chương.

Các thuật toán chúng ta mô tả dùng để kiểm tra tính thỏa mãn (satisfiability) — bài toán SAT. (Như đã đề cập trong Mục 7.5, kiểm tra hệ quả logic, $\alpha \models \beta$, có thể được thực hiện bằng cách kiểm tra tính không thỏa mãn của $\alpha \wedge \neg\beta$). Chúng ta đã đề cập trên trang 241 về mối liên hệ giữa việc tìm kiếm một mô hình thỏa mãn một câu logic và việc tìm kiếm lời giải cho một bài toán thỏa mãn ràng buộc (CSP). Do đó, hai họ thuật toán kiểm tra tính thỏa mãn trong logic mệnh đề rất giống với các thuật toán quay lui trong Mục 5.3 và các thuật toán tìm kiếm cục bộ trong Mục 5.4. Tuy nhiên, chúng cực kỳ quan trọng vì nhiều bài toán tổ hợp trong khoa học máy tính có thể được quy về việc kiểm tra tính thỏa mãn của một câu logic mệnh đề. Bất kỳ cải tiến nào trong các thuật toán kiểm tra tính thỏa mãn đều có tác động lớn đến khả năng xử lý độ phức tạp nói chung.

7.6.1 Thuật toán quay lui hoàn chỉnh

Thuật toán đầu tiên chúng ta xem xét thường được gọi là thuật toán Davis–Putnam, theo bài báo quan trọng của Martin Davis và Hilary Putnam (1960). Thuật toán này thực chất là phiên bản được mô tả bởi Davis, Logemann và Loveland (1962), nên chúng ta sẽ gọi nó là DPLL theo tên viết tắt của cả bốn tác giả. DPLL nhận đầu vào là một câu ở dạng chuẩn hội (CNF) — một tập các mệnh đề. Giống như Backtracking-Search và TT-ENTAILS?, nó thực chất là một phép liệt kê đệ quy, theo chiều sâu của các mô hình khả dĩ. Thuật toán này bao gồm ba cải tiến so với phương pháp đơn giản của TT-ENTAILS?:

- **Kết thúc sớm (Early termination):** Thuật toán phát hiện xem câu có phải là đúng hay sai ngay cả khi mô hình chưa hoàn chỉnh. Một mệnh đề là đúng nếu bất kỳ literal nào trong đó là đúng, ngay cả khi các literal khác chưa có giá trị chân lý; do đó, toàn bộ câu có thể được xác định là đúng ngay cả trước khi mô hình hoàn tất. Ví dụ, câu $(A \vee B) \wedge (A \vee C)$ là đúng nếu A đúng, bất kể giá trị của B và C . Tương tự, một câu là sai nếu bất kỳ mệnh đề nào trong đó là sai, điều này xảy ra khi tất cả các literal của mệnh đề đó đều sai. Việc kết thúc sớm tránh phải kiểm tra toàn bộ các cây con trong không gian tìm kiếm.
- **Heuristic ký hiệu thuần túy (Pure symbol heuristic):** Một ký hiệu thuần túy là một ký hiệu luôn xuất hiện với cùng một “dấu” trong tất cả các mệnh đề. Ví dụ, trong ba mệnh đề $(A \vee \neg B)$, $(\neg B \vee \neg C)$ và $(C \vee A)$, ký hiệu A là thuần túy vì chỉ xuất hiện dạng khẳng định, B là thuần túy vì chỉ xuất hiện dạng phủ định, và C là không thuần túy. Dễ thấy rằng nếu một câu có mô hình, thì nó cũng có một mô hình với các ký hiệu thuần túy được gán sao cho các literal của chúng là đúng, vì việc này không bao giờ làm cho một mệnh đề trở thành sai. Lưu ý rằng, khi xác định tính thuần túy của một ký hiệu, thuật toán có thể bỏ qua các mệnh đề đã được biết là đúng trong mô hình đã xây dựng đến thời điểm hiện tại. Ví dụ, nếu mô hình chứa $B = \text{false}$, thì mệnh đề $(\neg B \vee \neg C)$ đã đúng, và trong các mệnh đề còn lại, C chỉ xuất hiện dưới dạng khẳng định; do đó C trở thành thuần túy.
- **Heuristic mệnh đề đơn vị (Unit clause heuristic):** Một mệnh đề đơn vị được định nghĩa trước đó là một mệnh đề chỉ có một literal. Trong ngữ cảnh của DPLL, nó cũng có nghĩa là các mệnh đề trong đó tất cả các literal trừ một đã được gán giá trị sai bởi mô hình. Ví dụ, nếu mô hình chứa $B = \text{true}$, thì $(\neg B \vee \neg C)$ đơn giản hóa thành $\neg C$, là một mệnh đề đơn vị. Rõ ràng, để mệnh đề này đúng, C phải được gán là false. Heuristic mệnh đề đơn vị gán tất cả các ký hiệu như vậy trước khi phân nhánh phần còn lại. Một hệ quả quan trọng của heuristic này là mọi nỗ lực chứng minh (bằng phản chứng) một literal đã có trong cơ sở tri thức sẽ thành công ngay lập tức (Bài tập 7.KNOW). Lưu ý rằng việc gán một mệnh đề đơn vị có thể tạo ra một mệnh đề đơn vị khác — ví dụ, khi C được gán là false, $(C \vee A)$ trở thành một mệnh đề đơn vị, khiến A được gán là true. “Hiệu ứng dây chuyền” này của các phép gán bắt buộc được gọi là lan truyền đơn vị (unit propagation). Nó giống với quá trình suy diễn tiến với các mệnh đề xác định (definite clauses), và thực tế, nếu biểu thức CNF chỉ chứa các mệnh đề xác định thì DPLL về cơ bản sẽ sao chép quá trình suy diễn tiến. (Xem Bài tập 7.DPLL.)

function DPLL-SATISFIABLE?(*s*) **returns** *true* or *false*

inputs: *s*, a sentence in propositional logic

clauses $\leftarrow$ the set of clauses in the CNF representation of *s*

symbols $\leftarrow$ a list of the proposition symbols in *s*

return DPLL(*clauses*, *symbols*, { })

function DPLL(*clauses*, *symbols*, *model*) **returns** *true* or *false*

if every clause in *clauses* is true in *model* **then return** *true*

if some clause in *clauses* is false in *model* **then return** *false*

P, *value* $\leftarrow$ FIND-PURE-SYMBOL(*symbols*, *clauses*, *model*)

if *P* is non-null **then return** DPLL(*clauses*, *symbols* – *P*, *model* $\cup$ {*P*=*value*})

P, *value* $\leftarrow$ FIND-UNIT-CLAUSE(*clauses*, *model*)

if *P* is non-null **then return** DPLL(*clauses*, *symbols* – *P*, *model* $\cup$ {*P*=*value*})

P $\leftarrow$ FIRST(*symbols*); *rest* $\leftarrow$ REST(*symbols*)

return DPLL(*clauses*, *rest*, *model* $\cup$ {*P*=*true*}) **or**

DPLL(*clauses*, *rest*, *model* $\cup$ {*P*=*false*})

Hình 7.17 Thuật toán DPLL để kiểm tra tính thỏa mãn của một câu trong logic mệnh đề. Các ý tưởng đằng sau FIND-PURE-SYMBOL và FIND-UNIT-CLAUSE được mô tả trong văn bản; mỗi hàm trả về một ký hiệu (hoặc null) và giá trị chân lý để gán cho ký hiệu đó. Giống như TT-ENTAILS?, DPLL hoạt động trên các mô hình một phần.

Thuật toán DPLL được trình bày trong Hình 7.17, cung cấp bộ khung cơ bản của quá trình tìm kiếm mà không đi vào chi tiết triển khai.

Những gì Hình 7.17 không thể hiện là các thủ thuật giúp các bộ giải SAT có thể mở rộng để giải quyết các bài toán lớn. Thú vị là hầu hết các thủ thuật này thực chất khá tổng quát, và chúng ta đã gặp chúng trước đây dưới các hình thức khác:

15. **Phân tích thành phần (Component analysis)** (như đã thấy với Tasmania trong các bài toán CSP): Khi DPLL gán giá trị chân lý cho các biến, tập các mệnh đề có thể được tách thành các tập con rời rạc, gọi là các thành phần, không chia sẻ biến chưa được gán. Nếu có một cách hiệu quả để phát hiện khi nào điều này xảy ra, một bộ giải có thể đạt được tốc độ đáng kể bằng cách làm việc trên từng thành phần riêng biệt.
16. **Thứ tự biến và giá trị (Variable and value ordering)** (như đã thấy trong Mục 5.3.1 cho các bài toán CSP): Triển khai đơn giản của DPLL sử dụng một thứ tự biến tùy ý và luôn thử giá trị true trước false. Heuristic bậc (degree heuristic) (xem trang 177) gợi ý chọn biến xuất hiện thường xuyên nhất trong tất cả các mệnh đề còn lại.
17. **Quay lui thông minh (Intelligent backtracking)** (như đã thấy trong Mục 5.3.3 cho các bài toán CSP): Nhiều bài toán không thể giải được trong nhiều giờ chạy với quay lui theo thứ tự thời gian có thể được giải trong vài giây với quay lui thông minh, quay lại ngay điểm xung đột liên quan. Tất cả các bộ giải SAT thực hiện quay lui thông minh đều sử dụng một dạng nào đó của học mệnh đề xung đột (conflict clause learning) để ghi lại các xung đột, nhằm tránh lặp lại chúng sau này trong quá trình tìm kiếm. Thông thường, một tập hợp có

kích thước giới hạn các xung đột được giữ lại, và những xung đột hiếm khi sử dụng sẽ bị loại bỏ.

18. **Khởi động lại ngẫu nhiên (Random restarts)** (như đã thấy trên trang 131 cho leo đồi): Đôi khi một lần chạy tốc đường như không tiến triển. Trong trường hợp này, chúng ta có thể bắt đầu lại từ đầu cây tìm kiếm, thay vì cố gắng tiếp tục. Sau khi khởi động lại, các lựa chọn ngẫu nhiên khác nhau (trong việc chọn biến và giá trị) được thực hiện. Các mệnh đề đã học trong lần chạy đầu tiên được giữ lại sau khi khởi động lại và có thể giúp cắt tỉa không gian tìm kiếm. Việc khởi động lại không đảm bảo rằng một lời giải sẽ được tìm thấy nhanh hơn, nhưng nó làm giảm phương sai về thời gian tìm lời giải.

19. **Lập chỉ mục thông minh (Clever indexing)** (như đã thấy trong nhiều thuật toán): Các phương pháp tăng tốc được sử dụng trong chính DPLL, cũng như các thủ thuật được sử dụng trong các bộ giải hiện đại, yêu cầu lập chỉ mục nhanh cho các thứ như “tập các mệnh đề trong đó biến X_i xuất hiện dưới dạng literal khẳng định.” Nhiệm vụ này trở nên phức tạp do các thuật toán chỉ quan tâm đến các mệnh đề chưa được thỏa mãn bởi các phép gán trước đó cho biến, vì vậy các cấu trúc chỉ mục phải được cập nhật động khi quá trình tính toán tiến triển.

Với những cải tiến này, các bộ giải hiện đại có thể xử lý các bài toán với hàng chục triệu biến. Chúng đã cách mạng hóa các lĩnh vực như xác minh phần cứng và xác minh giao thức bảo mật, vốn trước đây đòi hỏi các chứng minh thủ công, mất nhiều công sức.

7.6.2 Các thuật toán tìm kiếm cục bộ

Chúng ta đã thấy một số thuật toán tìm kiếm cục bộ trong cuốn sách này, bao gồm **HILL-CLIMBING** (trang 129) và **SIMULATED-ANNEALING** (trang 133). Những thuật toán này có thể được áp dụng trực tiếp vào các bài toán thỏa mãn (SAT), miễn là chúng ta chọn đúng hàm đánh giá. Vì mục tiêu là tìm một phép gán thỏa mãn mọi mệnh đề, một hàm đánh giá đếm số mệnh đề không thỏa mãn sẽ đáp ứng yêu cầu này. Thực tế, đây chính là cách đo lường được sử dụng bởi thuật toán **MIN-CONFLICTS** cho các bài toán CSP (trang 182). Tất cả các thuật toán này thực hiện các bước trong không gian các phép gán hoàn chỉnh, thay đổi giá trị chân lý của một ký hiệu mỗi lần. Không gian này thường chứa nhiều cực tiểu cục bộ, và để thoát khỏi chúng, cần áp dụng các hình thức ngẫu nhiên khác nhau. Trong những năm gần đây, đã có rất nhiều thử nghiệm để tìm ra sự cân bằng tốt nhất giữa tính tham lam và tính ngẫu nhiên.

function WALKSAT(*clauses*, *p*, *max_flips*) **returns** a satisfying model or *failure*
inputs: *clauses*, a set of clauses in propositional logic
p, the probability of choosing to do a “random walk” move, typically around 0.5
max_flips, number of value flips allowed before giving up

model $\leftarrow$ a random assignment of *true/false* to the symbols in *clauses*
for each *i* = 1 **to** *max_flips* **do**
 if *model* satisfies *clauses* **then return** *model*
 clause $\leftarrow$ a randomly selected clause from *clauses* that is false in *model*
 if RANDOM(0, 1) $\leq p$ **then**
 flip the value in *model* of a randomly selected symbol from *clause*
 else flip whichever symbol in *clause* maximizes the number of satisfied clauses
return *failure*

Hình 7.18 Thuật toán WALKSAT để kiểm tra tính thỏa mãn bằng cách lật ngẫu nhiên các giá trị của các biến. Tồn tại nhiều phiên bản của thuật toán.

Một trong những thuật toán đơn giản và hiệu quả nhất xuất hiện từ những nghiên cứu này là **WALKSAT** (Hình 7.18). Trong mỗi lần lặp, thuật toán chọn một mệnh đề chưa được thỏa mãn và chọn một ký hiệu trong mệnh đề đó để đảo giá trị. Nó lựa chọn ngẫu nhiên giữa hai cách để chọn ký hiệu cần đảo: (1) bước “min-conflicts” nhằm tối thiểu hóa số mệnh đề không thỏa mãn trong trạng thái mới, và (2) bước “random walk” chọn ngẫu nhiên một ký hiệu trong mệnh đề.

Khi **WALKSAT** trả về một mô hình, câu đầu vào chắc chắn là thỏa mãn được. Tuy nhiên, khi nó trả về thất bại, có hai nguyên nhân có thể xảy ra: hoặc câu đó không thỏa mãn được, hoặc chúng ta cần cho thuật toán thêm thời gian. Nếu đặt $\text{max_flips} = \infty$ và $p > 0$, **WALKSAT** cuối cùng sẽ trả về một mô hình (nếu tồn tại), vì các bước “random walk” cuối cùng sẽ tìm ra lời giải. Tuy nhiên, nếu max_flips là vô hạn và câu không thỏa mãn được, thuật toán sẽ không bao giờ dừng!

Vì lý do này, **WALKSAT** hữu ích nhất khi chúng ta kỳ vọng tồn tại lời giải—ví dụ, các bài toán được thảo luận trong Chương 3 và 5 thường có lời giải. Mặt khác, **WALKSAT** không thể phát hiện tính không thỏa mãn, điều cần thiết để quyết định hệ quả logic. Ví dụ, một tác nhân không thể dùng **WALKSAT** để chứng minh một ô là an toàn trong thế giới wumpus. Thay vào đó, nó có thể nói, “Tôi đã suy nghĩ trong một giờ và không thể tìm ra một thế giới khả dĩ nào mà ô đó không an toàn.” Đây có thể là một chỉ số thực nghiệm tốt cho thấy ô đó an toàn, nhưng chắc chắn không phải là một chứng minh.

7.6.3 Bức tranh tổng quan về các bài toán SAT ngẫu nhiên

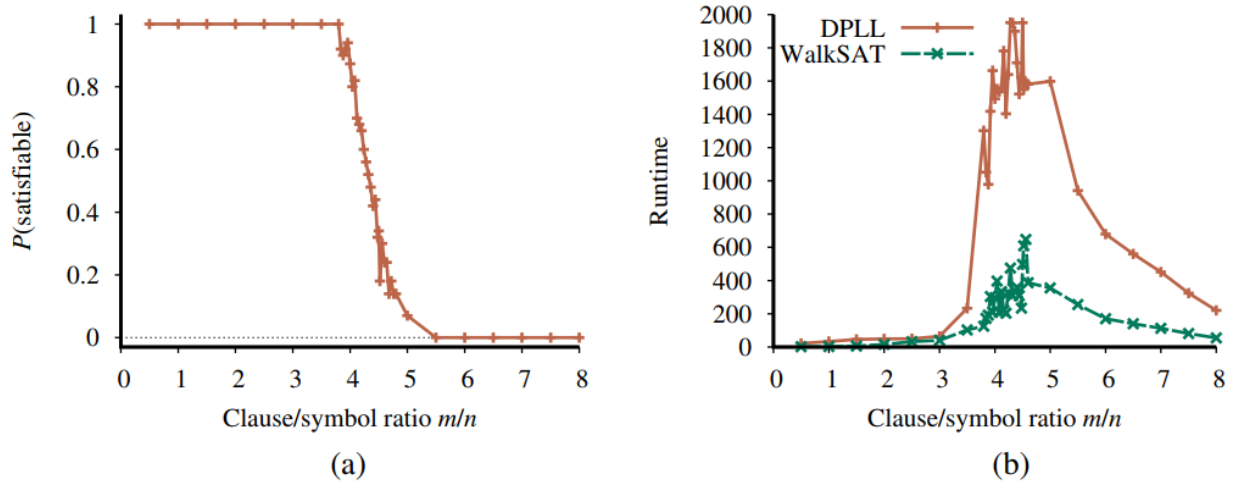
Một số bài toán SAT khó hơn những bài toán khác. Các bài toán dễ có thể được giải quyết bằng bất kỳ thuật toán cũ nào, nhưng vì chúng ta biết rằng SAT là NP-đầy đủ, ít nhất một số trường hợp bài toán phải yêu cầu thời gian chạy theo hàm mũ. Trong Chương 5, chúng ta đã thấy một số khám phá đáng ngạc nhiên về một số loại bài toán. Ví dụ, bài toán **n-queens**—tưởng chừng khá phức tạp đối với các thuật toán tìm kiếm quay lui—lại trở nên cực kỳ dễ dàng đối với các phương pháp tìm kiếm cục bộ. Điều này là do các lời giải được phân bố rất dày đặc trong không gian các phép gán, và bất kỳ phép gán ban đầu nào cũng đảm bảo có một lời giải gần đó. Do đó, **n-queens** là dễ vì nó bị thiếu ràng buộc.

Khi xem xét các bài toán thỏa mãn trong dạng chuẩn hội (CNF), một bài toán thiếu ràng buộc là bài toán có tương đối ít mệnh đề ràng buộc các biến. Ví dụ, đây là một câu 3-CNF ngẫu nhiên với năm ký hiệu và năm mệnh đề:

$$(-D \vee \neg B \vee C) \wedge (B \vee \neg A \vee \neg C) \wedge (\neg C \vee \neg B \vee E) \wedge (E \vee \neg D \vee B) \wedge (B \vee E \vee \neg C).$$

16 trong số 32 phép gán khả dĩ là mô hình của câu này, vì vậy, trung bình chỉ cần hai lần đoán ngẫu nhiên để tìm ra một mô hình. Đây là một bài toán thỏa mãn dễ dàng, giống như hầu hết các bài toán thiếu ràng buộc. Mặt khác, một bài toán thừa ràng buộc có nhiều mệnh đề so với số biến và có khả năng không có lời giải. Các bài toán thừa ràng buộc thường dễ giải, vì các ràng buộc nhanh chóng dẫn đến một lời giải hoặc một ngõ cụt không thể thoát ra.

Để vượt ra khỏi những trực giác cơ bản này, chúng ta phải định nghĩa chính xác cách các câu ngẫu nhiên được tạo ra. Ký hiệu $CNF_k(m, n)$ biểu thị một câu k-CNF với m mệnh đề và n ký hiệu, trong đó các mệnh đề được chọn đồng nhất, độc lập và không lặp lại từ tất cả các mệnh đề có k literal khác nhau, với các literal dương hoặc âm được chọn ngẫu nhiên. (Một ký hiệu không thể xuất hiện hai lần trong một mệnh đề, và một mệnh đề cũng không thể xuất hiện hai lần trong một câu.)



Hình 7.19 (a) Đồ thị cho thấy xác suất một câu 3-CNF ngẫu nhiên với $n=50$ ký hiệu là thỏa mãn, dưới dạng một hàm của tỉ lệ mệnh đề/ký hiệu m/n . (b) Đồ thị của thời gian chạy trung bình (được đo bằng số lần lặp) cho cả DPLL và WALKSAT trên các câu 3-CNF ngẫu nhiên. Các bài toán khó nhất có tỉ lệ mệnh đề/ký hiệu khoảng 4.3.

Với một nguồn tạo câu ngẫu nhiên, chúng ta có thể đo lường xác suất thỏa mãn. Hình 7.19(a) vẽ đồ thị xác suất cho $CNF_3(m, 50)$, tức là các câu với 50 biến và 3 literal mỗi mệnh đề, như một hàm của tỷ lệ mệnh đề/ký hiệu, m/n . Như chúng ta mong đợi, với m/n nhỏ, xác suất thỏa mãn gần bằng 1, và với m/n lớn, xác suất gần bằng 0. Xác suất giảm khá mạnh xung quanh $m/n = 4.3$. Thực nghiệm cho thấy “vách đá” này vẫn giữ nguyên vị trí (với $k = 3$) và ngày càng trở nên sắc nét hơn khi n tăng lên.

Về mặt lý thuyết, **giả thuyết ngưỡng thỏa mãn** nói rằng với mọi $k \geq 3$, tồn tại một ngưỡng tỷ lệ r_k sao cho khi n tiến đến vô cùng, xác suất để $CNF_k(n, rn)$ thỏa mãn trở thành 1 với mọi giá trị r

dưới ngưỡng, và 0 với mọi giá trị trên ngưỡng. Giả thuyết này vẫn chưa được chứng minh, ngay cả với các trường hợp đặc biệt như $k = 3$. Dù là một định lý hay không, hiệu ứng ngưỡng này thường xuất hiện trong các bài toán NP-khó.

Bây giờ chúng ta đã biết bài toán thỏa mãn và không thỏa mãn nằm ở đâu, câu hỏi tiếp theo là: đâu là những bài toán khó? Hóa ra chúng thường nằm ở giá trị ngưỡng. Hình 7.19(b) cho thấy các bài toán 50 ký hiệu ở giá trị ngưỡng 4.3 khó giải gấp khoảng 20 lần so với các bài toán có tỷ lệ 3.3. Các bài toán thiếu ràng buộc là dễ giải nhất (vì rất dễ đoán ra lời giải); các bài toán thừa ràng buộc không dễ bằng bài toán thiếu ràng buộc, nhưng vẫn dễ hơn nhiều so với các bài toán nằm ngay tại ngưỡng.

7.7 Tác nhân dựa trên Logic Mệnh đề

Trong phần này, chúng ta sẽ tổng hợp những kiến thức đã học để xây dựng các tác nhân trong thế giới Wumpus sử dụng logic mệnh đề. Bước đầu tiên là giúp tác nhân suy luận, ở mức độ có thể, trạng thái của thế giới dựa trên lịch sử nhận thức của nó. Điều này đòi hỏi phải viết ra một mô hình logic đầy đủ về các tác động của hành động. Sau đó, chúng ta sẽ chỉ ra cách tác nhân có thể sử dụng suy luận logic trong thế giới Wumpus. Chúng ta cũng sẽ trình bày cách tác nhân có thể theo dõi thế giới một cách hiệu quả mà không cần quay lại lịch sử nhận thức cho mỗi lần suy luận. Cuối cùng, chúng ta sẽ chỉ ra cách tác nhân có thể sử dụng suy luận logic để xây dựng các kế hoạch đảm bảo đạt được mục tiêu, với điều kiện cơ sở tri thức của nó đúng trong thế giới thực.

7.7.1 Trạng thái hiện tại của thế giới

Như đã nêu ở đầu chương, một tác nhân logic hoạt động bằng cách suy luận những gì cần làm từ một cơ sở tri thức chứa các câu về thế giới. Cơ sở tri thức bao gồm các tiên đề—kiến thức chung về cách thế giới vận hành—và các câu nhận thức thu được từ trải nghiệm của tác nhân trong một thế giới cụ thể. Trong phần này, chúng ta tập trung vào vấn đề suy luận trạng thái hiện tại của thế giới Wumpus—vị trí hiện tại, ô nào an toàn, v.v.

Chúng ta đã bắt đầu thu thập các tiên đề trong Mục 7.4.3. Tác nhân biết rằng ô bắt đầu không có hổ ($\neg P_{1,1}$) và không có Wumpus ($\neg W_{1,1}$). Hơn nữa, đối với mỗi ô, tác nhân biết rằng ô đó có gió nếu và chỉ nếu một ô lân cận có hổ; và một ô có mùi hôi nếu và chỉ nếu một ô lân cận có Wumpus. Do đó, chúng ta bao gồm một tập hợp lớn các câu có dạng sau:

$$\begin{aligned} B_{1,1} &\Leftrightarrow (P_{1,2} \vee P_{2,1}) \\ S_{1,1} &\Leftrightarrow (W_{1,2} \vee W_{2,1}) \\ &\dots \end{aligned}$$

Tác nhân cũng biết rằng chỉ có một Wumpus. Điều này được biểu diễn bằng hai phần. Đầu tiên, chúng ta phải nói rằng có ít nhất một Wumpus:

$$W_{1,1} \vee W_{1,2} \vee \dots \vee W_{4,3} \vee W_{4,4}.$$

Sau đó, chúng ta phải nói rằng có tối đa một Wumpus. Đối với mỗi cặp vị trí, chúng ta thêm một câu nói rằng ít nhất một trong hai ô đó không có Wumpus:

$$\begin{aligned}
& \neg W_{1,1} \vee \neg W_{1,2} \\
& \neg W_{1,1} \vee \neg W_{1,3} \\
& \dots \\
& \neg W_{4,3} \vee \neg W_{4,4}.
\end{aligned}$$

Cho đến nay, mọi thứ đều ổn. Bây giờ hãy xem xét các nhận thức của tác nhân. Chúng ta đang sử dụng $S_{1,1}$ để chỉ có mùi hôi ở $[1,1]$; liệu chúng ta có thể sử dụng một mệnh đề đơn giản, Stench, để chỉ rằng tác nhân nhận thấy mùi hôi không? Thật không may là không thể: nếu không có mùi hôi ở bước thời gian trước, $\neg$ Stench đã được khẳng định, và việc khẳng định mới sẽ dẫn đến mâu thuẫn. Vấn đề được giải quyết khi chúng ta nhận ra rằng một nhận thức chỉ khẳng định điều gì đó về thời điểm hiện tại. Do đó, nếu bước thời gian (được cung cấp cho MAKE-PERCEPT-SENTENCE trong Hình 7.1) là 4, chúng ta thêm Stench⁴ vào cơ sở tri thức, thay vì Stench—tránh được bất kỳ mâu thuẫn nào với $\neg$ Stench³. Điều tương tự áp dụng cho các nhận thức về gió, va chạm, ánh sáng lấp lánh và tiếng hét.

Ý tưởng gắn các mệnh đề với các bước thời gian mở rộng đến bất kỳ khía cạnh nào của thế giới thay đổi theo thời gian. Ví dụ, cơ sở tri thức ban đầu bao gồm $L_{1,1}^0$ —tác nhân ở ô $[1,1]$ tại thời điểm 0—cũng như FacingEast⁰, HaveArrow⁰ và WumpusAlive⁰. Chúng ta sử dụng thuật ngữ fluent (từ tiếng Latinh fluens, chảy) để chỉ một khía cạnh của thế giới thay đổi. “Fluent” là từ đồng nghĩa với “biến trạng thái”, theo nghĩa được mô tả trong phần thảo luận về biểu diễn phân tách trong Mục 2.4.7 (trang 76). Các ký hiệu liên quan đến các khía cạnh vĩnh viễn của thế giới không cần chỉ số thời gian và đôi khi được gọi là biến phi thời gian.

Chúng ta có thể kết nối trực tiếp các nhận thức về mùi hôi và gió với các thuộc tính của các ô mà chúng được trải nghiệm như sau. Đối với bất kỳ bước thời gian t và bất kỳ ô $[x,y]$ nào, chúng ta khẳng định:

$$\begin{aligned}
L_{x,y}^t & \Rightarrow (Breeze^t \Leftrightarrow B_{x,y}) \\
L_{x,y}^t & \Rightarrow (Stench^t \Leftrightarrow S_{x,y}).
\end{aligned}$$

Bây giờ, tất nhiên, chúng ta cần các tiên đề cho phép tác nhân theo dõi các fluent như $L_{x,y}^t$. Các fluent này thay đổi do kết quả của các hành động được thực hiện bởi tác nhân, vì vậy, theo thuật ngữ của Chương 3, chúng ta cần viết ra mô hình chuyển tiếp của thế giới Wumpus dưới dạng một tập hợp các câu logic.

Đầu tiên, chúng ta cần các ký hiệu mệnh đề cho các sự kiện hành động. Giống như nhận thức, các ký hiệu này được đánh chỉ số theo thời gian; do đó, Forward⁰ có nghĩa là tác nhân thực hiện hành động Forward tại thời điểm 0. Theo quy ước, nhận thức cho một bước thời gian nhất định xảy ra trước, sau đó là hành động cho bước thời gian đó, tiếp theo là chuyển tiếp sang bước thời gian tiếp theo.

Để mô tả cách thế giới thay đổi, chúng ta có thể thử viết các tiên đề hiệu ứng chỉ định kết quả của một hành động tại bước thời gian tiếp theo. Ví dụ, nếu tác nhân ở vị trí $[1,1]$ hướng về phía đông tại thời điểm 0 và di chuyển Forward, kết quả là tác nhân ở ô $[2,1]$ và không còn ở $[1,1]$:

$$L_{1,1}^0 \wedge \text{FacingEast}^0 \wedge \text{Forward}^0 \Rightarrow (L_{2,1}^1 \wedge \neg L_{1,1}^1).$$

Chúng ta sẽ cần một câu như vậy cho mỗi bước thời gian có thể, cho mỗi ô trong 16 ô, và mỗi hướng trong bốn hướng. Chúng ta cũng sẽ cần các câu tương tự cho các hành động khác: Grab, Shoot, Climb, TurnLeft và TurnRight.

Giả sử tác nhân quyết định di chuyển Forward tại thời điểm 0 và khẳng định sự kiện này vào cơ sở tri thức của nó. Với tiên đề hiệu ứng trong Phương trình (7.1), kết hợp với các khẳng định ban đầu về trạng thái tại thời điểm 0, tác nhân bây giờ có thể suy luận rằng nó đang ở [2,1]. Tức là, $ASK(KB, L_{2,1}^1) = \text{true}$. Cho đến nay, mọi thứ đều tốt. Thật không may, nếu chúng ta $ASK(KB, HaveArrow^1)$, câu trả lời là false, tức là tác nhân không thể chứng minh rằng nó vẫn có mũi tên; cũng không thể chứng minh rằng nó không có! Thông tin đã bị mất vì tiên đề hiệu ứng không nêu rõ những gì không thay đổi do kết quả của một hành động. Nhu cầu làm điều này dẫn đến vấn đề khung hình (frame problem). Một giải pháp khả thi cho vấn đề khung hình là thêm các tiên đề khung hình khẳng định rõ ràng tất cả các mệnh đề không thay đổi. Ví dụ, đối với mỗi thời điểm t , chúng ta sẽ có:

$$\begin{aligned} Forward^t &\Rightarrow (HaveArrow^t \Leftrightarrow HaveArrow^{t+1}) \\ Forward^t &\Rightarrow (WumpusAlive^t \Leftrightarrow WumpusAlive^{t+1}) \\ &\dots \end{aligned}$$

trong đó chúng ta liệt kê rõ ràng mọi mệnh đề không thay đổi từ thời điểm t đến thời điểm $t + 1$ dưới hành động Forward. Mặc dù tác nhân bây giờ biết rằng nó vẫn có mũi tên sau khi di chuyển về phía trước và Wumpus không chết hoặc sống lại, nhưng sự gia tăng của các tiên đề khung hình dường như không hiệu quả. Trong một thế giới với m hành động khác nhau và n fluent, tập hợp các tiên đề khung hình sẽ có kích thước $O(mn)$. Biểu hiện cụ thể này của vấn đề khung hình đôi khi được gọi là vấn đề khung hình biểu diễn. Vấn đề này đóng một vai trò quan trọng trong lịch sử AI; chúng ta sẽ khám phá thêm trong phần ghi chú ở cuối chương.

Vấn đề khung hình biểu diễn rất quan trọng vì thế giới thực có rất nhiều fluent, nói một cách nhẹ nhàng. May mắn thay cho con người, mỗi hành động thường chỉ thay đổi một số lượng nhỏ k trong số các fluent đó—thế giới thể hiện tính cục bộ. Giải quyết vấn đề khung hình biểu diễn đòi hỏi phải định nghĩa mô hình chuyển tiếp với một tập hợp các tiên đề có kích thước $O(mk)$ thay vì $O(mn)$. Ngoài ra còn có vấn đề khung hình suy luận: vấn đề dự đoán kết quả của một kế hoạch hành động t bước trong thời gian $O(kt)$ thay vì $O(nt)$.

Giải pháp cho vấn đề liên quan đến việc thay đổi trọng tâm từ viết các tiên đề về hành động sang viết các tiên đề về fluent. Do đó, đối với mỗi fluent F , chúng ta sẽ có một tiên đề định nghĩa giá trị chân lý của F^{t+1} dựa trên các fluent (bao gồm cả F chính nó) tại thời điểm t và các hành động có thể đã xảy ra tại thời điểm t . Bây giờ, giá trị chân lý của F^{t+1} có thể được đặt theo một trong hai cách: hoặc hành động tại thời điểm t khiến F trở thành true tại $t + 1$, hoặc F đã là true tại thời điểm t và hành động tại thời điểm t không khiến nó trở thành false. Một tiên đề có dạng này được gọi là tiên đề trạng thái kế tiếp và có dạng sau:

$$F^{t+1} \Leftrightarrow \text{ActionCauses}F^t \vee (F^t \wedge \neg \text{ActionCausesNot}F^t).$$

Một trong những tiên đề trạng thái kế tiếp đơn giản nhất là tiên đề cho HaveArrow. Vì không có hành động nạp lại, phần $\text{ActionCauses}F^t$ biến mất và chúng ta còn lại:

$$\text{HaveArrow}^{t+1} \Leftrightarrow (\text{HaveArrow}^t \wedge \neg \text{Shoot}^t). \quad (7.2)$$

Đối với vị trí của tác nhân, các tiên đề trạng thái kế tiếp phức tạp hơn. Ví dụ, $L_{1,1}^{t+1}$ là true nếu (a) tác nhân di chuyển Forward từ [1,2] khi hướng về phía nam, hoặc từ [2,1] khi hướng về phía tây; hoặc (b) $L_{1,1}^t$ đã là true và hành động không gây ra di chuyển (do hành động không phải Forward hoặc do hành động va vào tường). Viết ra bằng logic mệnh đề, điều này trở thành:

$$\begin{aligned} L_{1,1}^{t+1} \Leftrightarrow & (L_{1,1}^t \wedge (\neg \text{Forward}^t \vee \text{Bump}^{t+1})) \\ & \vee (L_{1,2}^t \wedge (\text{FacingSouth}^t \wedge \text{Forward}^t)) \\ & \vee (L_{2,1}^t \wedge (\text{FacingWest}^t \wedge \text{Forward}^t)). \end{aligned}$$

Bài tập 7.SSAX yêu cầu bạn viết các tiên đề cho các fluent còn lại trong thế giới Wumpus.

Với một tập hợp đầy đủ các tiên đề trạng thái kế tiếp và các tiên đề khác được liệt kê ở đầu phần này, tác nhân sẽ có thể HỎI và trả lời bất kỳ câu hỏi nào có thể trả lời về trạng thái hiện tại của thế giới. Ví dụ, trong Mục 7.2, chuỗi nhận thức và hành động ban đầu là:

$$\begin{aligned} & \neg \text{Stench}^0 \wedge \neg \text{Breeze}^0 \wedge \neg \text{Glitter}^0 \wedge \neg \text{Bump}^0 \wedge \neg \text{Scream}^0 ; \text{Forward}^0 \\ & \neg \text{Stench}^1 \wedge \text{Breeze}^1 \wedge \neg \text{Glitter}^1 \wedge \neg \text{Bump}^1 \wedge \neg \text{Scream}^1 ; \text{TurnRight}^1 \\ & \neg \text{Stench}^2 \wedge \text{Breeze}^2 \wedge \neg \text{Glitter}^2 \wedge \neg \text{Bump}^2 \wedge \neg \text{Scream}^2 ; \text{TurnRight}^2 \\ & \neg \text{Stench}^3 \wedge \text{Breeze}^3 \wedge \neg \text{Glitter}^3 \wedge \neg \text{Bump}^3 \wedge \neg \text{Scream}^3 ; \text{Forward}^3 \\ & \neg \text{Stench}^4 \wedge \neg \text{Breeze}^4 \wedge \neg \text{Glitter}^4 \wedge \neg \text{Bump}^4 \wedge \neg \text{Scream}^4 ; \text{TurnRight}^4 \\ & \neg \text{Stench}^5 \wedge \neg \text{Breeze}^5 \wedge \neg \text{Glitter}^5 \wedge \neg \text{Bump}^5 \wedge \neg \text{Scream}^5 ; \text{Forward}^5 \\ & \text{Stench}^6 \wedge \neg \text{Breeze}^6 \wedge \neg \text{Glitter}^6 \wedge \neg \text{Bump}^6 \wedge \neg \text{Scream}^6 \end{aligned}$$

Tại thời điểm này, chúng ta có $\text{ASK}(KB, L_{1,2}^6) = \text{true}$, vì vậy tác nhân biết nó đang ở đâu. Hơn nữa, $\text{ASK}(KB, W_{1,3}) = \text{true}$ và $\text{ASK}(KB, P_{3,1}) = \text{true}$, vì vậy tác nhân đã tìm thấy Wumpus và một trong các hố. Câu hỏi quan trọng nhất đối với tác nhân là liệu một ô có an toàn để di chuyển vào hay không—tức là ô đó không có hố hoặc Wumpus còn sống. Thật tiện lợi khi thêm các tiên đề cho điều này, có dạng:

$$\text{OK}_{x,y}^t \Leftrightarrow \neg P_{x,y} \wedge \neg (W_{x,y} \wedge \text{WumpusAlive}^t).$$

Cuối cùng, $\text{ASK}(KB, \text{OK}_{2,2}^6) = \text{true}$, vì vậy ô [2,2] an toàn để di chuyển vào. Trên thực tế, với một thuật toán suy luận đúng đắn và đầy đủ như DPLL, tác nhân có thể trả lời bất kỳ câu hỏi nào có thể trả lời về ô nào an toàn—và có thể làm như vậy chỉ trong vài mili giây đối với các thế giới Wumpus cỡ nhỏ đến trung bình.

7.7.2 Một tác nhân lai (hybrid agent)

Khả năng suy luận các khía cạnh khác nhau của thế giới có thể được kết hợp khá đơn giản với các quy tắc điều kiện-hành động (xem Phần 2.4.2) và các thuật toán giải quyết vấn đề từ Chương 3 và 4 để tạo ra một tác nhân lai cho thế giới wumpus. Hình 7.20 cho thấy một cách có thể thực hiện điều này. Chương trình tác nhân duy trì và cập nhật một cơ sở tri thức cũng như một kế hoạch hiện tại. Cơ sở tri thức ban đầu chứa các tiên đề phi thời gian—những tiên đề không phụ thuộc vào t , chẳng hạn như tiên đề liên quan đến sự thoáng mát của các ô vuông với sự hiện diện của hố. Tại

mỗi bước thời gian, câu nhận thức mới được thêm vào cùng với tất cả các tiên đề phụ thuộc vào t , chẳng hạn như các tiên đề trạng thái kế tiếp. (Phần tiếp theo sẽ giải thích tại sao tác nhân không cần các tiên đề cho các bước thời gian trong tương lai.) Sau đó, tác nhân sử dụng suy luận logic, bằng cách đặt câu hỏi cho cơ sở tri thức, để xác định ô vuông nào an toàn và ô nào chưa được khám phá.

```

function HYBRID-WUMPUS-AGENT(percept) returns an action
  inputs: percept, a list, [stench,breeze,glitter,bump,scream]
  persistent: KB, a knowledge base, initially the atemporal “wumpus physics”
               t, a counter, initially 0, indicating time
               plan, an action sequence, initially empty

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  TELL the KB the temporal “physics” sentences for time t
  safe  $\leftarrow \{[x,y] : \text{ASK}(\text{KB}, \text{OK}_{x,y}^t) = \text{true}\}$ 
  if ASK(KB, Glittert) = true then
    plan  $\leftarrow [\text{Grab}] + \text{PLAN-ROUTE}(\text{current}, \{[1,1]\}, \text{safe}) + [\text{Climb}]$ 
  if plan is empty then
    unvisited  $\leftarrow \{[x,y] : \text{ASK}(\text{KB}, \text{L}_{x,y}^{t'}) = \text{false for all } t' \leq t\}$ 
    plan  $\leftarrow \text{PLAN-ROUTE}(\text{current}, \text{unvisited} \cap \text{safe}, \text{safe})$ 
  if plan is empty and ASK(KB, HaveArrowt) = true then
    possible_wumpus  $\leftarrow \{[x,y] : \text{ASK}(\text{KB}, \neg \text{W}_{x,y}) = \text{false}\}$ 
    plan  $\leftarrow \text{PLAN-SHOT}(\text{current}, \text{possible\_wumpus}, \text{safe})$ 
  if plan is empty then // no choice but to take a risk
    not_unsafe  $\leftarrow \{[x,y] : \text{ASK}(\text{KB}, \neg \text{OK}_{x,y}^t) = \text{false}\}$ 
    plan  $\leftarrow \text{PLAN-ROUTE}(\text{current}, \text{unvisited} \cap \text{not\_unsafe}, \text{safe})$ 
  if plan is empty then
    plan  $\leftarrow \text{PLAN-ROUTE}(\text{current}, \{[1, 1]\}, \text{safe}) + [\text{Climb}]$ 
  action  $\leftarrow \text{POP}(\text{plan})$ 
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t  $\leftarrow t + 1$ 
  return action

function PLAN-ROUTE(current, goals, allowed) returns an action sequence
  inputs: current, the agent’s current position
           goals, a set of squares; try to plan a route to one of them
           allowed, a set of squares that can form part of the route

  problem  $\leftarrow \text{ROUTE-PROBLEM}(\text{current}, \text{goals}, \text{allowed})$ 
  return SEARCH(problem) // Any search algorithm from Chapter 3

```

Hình 7.20 Một chương trình tác tử lai (hybrid) cho thế giới wumpus. Nó sử dụng một cơ sở tri thức mệnh đề để suy luận trạng thái của thế giới, và sự kết hợp giữa tìm kiếm giải quyết vấn đề và mã chuyên biệt cho miền để chọn các hành động. Mỗi khi HYBRID-WUMPUS-AGENT được gọi, nó thêm percept vào cơ sở tri thức, và sau đó hoặc dựa vào một kế hoạch đã được xác định

trước hoặc tạo ra một kế hoạch mới, và loại bỏ bước đầu tiên của kế hoạch như hành động tiếp theo để thực hiện.

Phần chính của chương trình tác nhân xây dựng một kế hoạch dựa trên mức độ ưu tiên giảm dần của các mục tiêu. Đầu tiên, nếu có ánh sáng lấp lánh, chương trình sẽ xây dựng một kế hoạch để lấy vàng, đi theo một tuyến đường trở về vị trí ban đầu và leo ra khỏi hang. Ngược lại, nếu không có kế hoạch hiện tại, chương trình sẽ lập kế hoạch một tuyến đường đến ô vuông an toàn gần nhất mà nó chưa từng ghé thăm, đảm bảo tuyến đường chỉ đi qua các ô vuông an toàn.

Việc lập kế hoạch tuyến đường được thực hiện với thuật toán tìm kiếm A^* , không phải bằng cách sử dụng ASK. Nếu không có ô vuông an toàn nào để khám phá, bước tiếp theo—nếu tác nhân vẫn còn một mũi tên—là cố gắng tạo ra một ô vuông an toàn bằng cách bắn vào một trong các vị trí có thể có của wumpus. Những vị trí này được xác định bằng cách hỏi nơi mà $ASK(KB, -W_{x,y})$ trả về false—tức là nơi không biết chắc chắn rằng không có wumpus. Hàm PLAN-SHOT (không được hiển thị) sử dụng PLAN-ROUTE để lập kế hoạch một chuỗi hành động sẽ thực hiện cú bắn này. Nếu thất bại, chương trình sẽ tìm một ô vuông để khám phá mà không chắc chắn là không an toàn—tức là một ô vuông mà $ASK(KB, -OK_{x,y}^t)$ trả về false. Nếu không có ô vuông nào như vậy, nhiệm vụ là không thể thực hiện được và tác nhân sẽ rút về [1,1] và leo ra khỏi hang.

7.7.3 Ước lượng trạng thái logic

Chương trình tác nhân trong Hình 7.20 hoạt động khá tốt, nhưng có một điểm yếu lớn: khi thời gian trôi qua, chi phí tính toán liên quan đến các lệnh gọi ASK ngày càng tăng. Điều này xảy ra chủ yếu vì các suy luận cần thiết phải quay ngược lại xa hơn trong thời gian và liên quan đến ngày càng nhiều ký hiệu mệnh đề. Rõ ràng, điều này không bền vững—chúng ta không thể có một tác nhân mà thời gian xử lý mỗi nhận thức tăng tỷ lệ thuận với độ dài cuộc đời của nó! Điều chúng ta thực sự cần là thời gian cập nhật không đổi—tức là độc lập với t . Câu trả lời rõ ràng là lưu hoặc lưu trữ kết quả của suy luận, để quá trình suy luận ở bước thời gian tiếp theo có thể xây dựng dựa trên kết quả của các bước trước đó thay vì phải bắt đầu lại từ đầu.

Như đã thấy trong Phần 4.4, lịch sử nhận thức và tất cả các hệ quả của chúng có thể được thay thế bằng trạng thái niềm tin—tức là một số biểu diễn của tập hợp tất cả các trạng thái hiện tại có thể có của thế giới. Biểu diễn trạng thái niềm tin dưới dạng một câu logic là một phương pháp tự nhiên. Ví dụ, nếu cơ sở tri thức chứa $L_{1,1}^0$ và $-L_{1,2}^0$, thì trạng thái niềm tin ban đầu là $L_{1,1}^0 \wedge -L_{1,2}^0$. Khi tác nhân di chuyển về phía trước và không cảm nhận được va chạm, nó thêm $L_{2,1}^1$ vào cơ sở tri thức. Sau đó, nếu nó cảm nhận được một làn gió thoảng ở [2,1], nó thêm $B_{2,1}^1$, dẫn đến trạng thái niềm tin $L_{2,1}^1 \wedge B_{2,1}^1$. Tiếp tục như vậy, mỗi khi tác nhân thực hiện một hành động hoặc nhận được một nhận thức, trạng thái niềm tin được cập nhật.

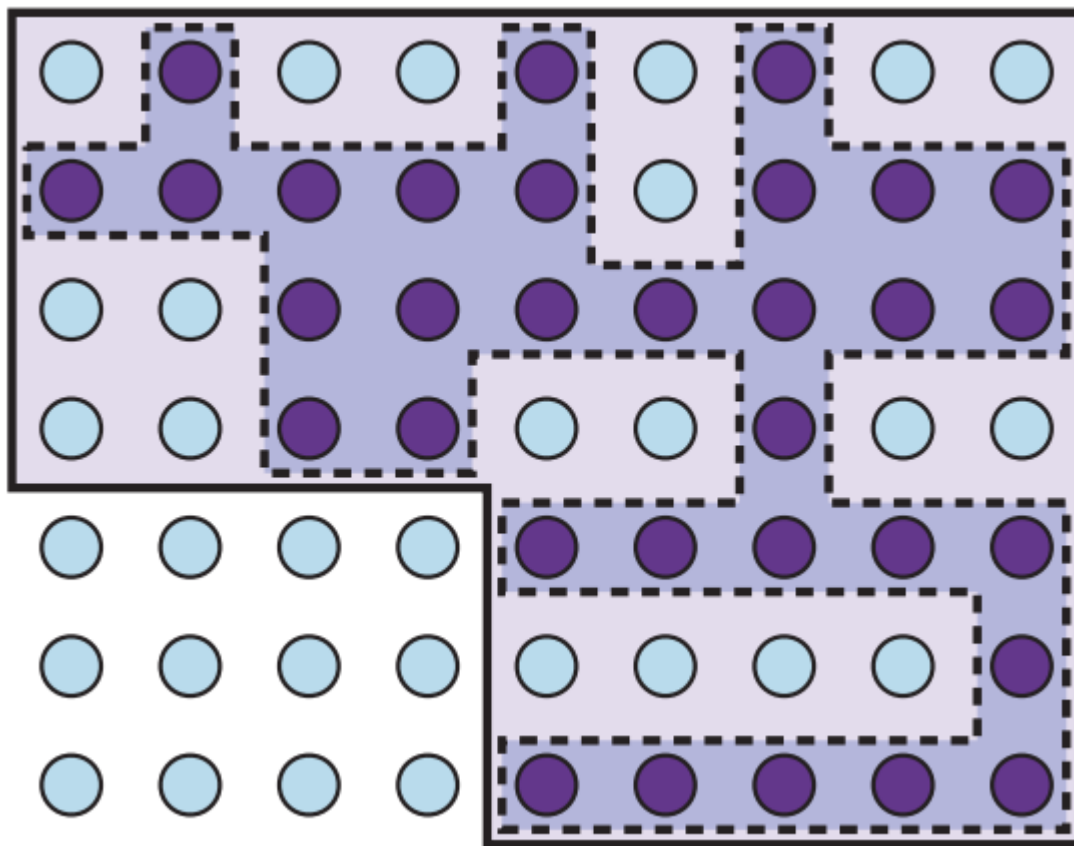
Quá trình cập nhật trạng thái niềm tin khi nhận thức mới đến được gọi là ước lượng trạng thái (xem trang 150). Trong khi ở Phần 4.4 trạng thái niềm tin là một danh sách rõ ràng các trạng thái, ở đây chúng ta có thể sử dụng một câu logic liên quan đến các ký hiệu mệnh đề gắn với bước thời gian hiện tại, cũng như các ký hiệu phi thời gian. Ví dụ, câu logic

$$WumpusAlive^1 \wedge L_{2,1}^1 \wedge B_{2,1}^1 \wedge (P_{3,1} \vee P_{2,2}) \quad (7.4)$$

biểu diễn tập hợp tất cả các trạng thái tại thời điểm 1 trong đó wumpus còn sống, tác nhân ở [2,1], ô vuông đó thoáng mát và có một hố ở [3,1] hoặc [2,2] hoặc cả hai.

Việc duy trì một trạng thái niềm tin chính xác dưới dạng một công thức logic hóa ra không dễ dàng. Nếu có n ký hiệu trạng thái cho thời gian t , thì có 2^n trạng thái có thể—tức là các phép gán giá trị chân lý cho các ký hiệu đó. Bây giờ, tập hợp các trạng thái niềm tin là tập lũy thừa (tập tất cả các tập con) của tập hợp các trạng thái vật lý. Có 2^n trạng thái vật lý, do đó có 2^{2^n} trạng thái niềm tin. Ngay cả khi chúng ta sử dụng mã hóa gọn nhất có thể cho các công thức logic, với mỗi trạng thái niềm tin được biểu diễn bằng một số nhị phân duy nhất, chúng ta sẽ cần các số có $\log_2(2^{2^n}) = 2^n$ bit để gán nhãn trạng thái niềm tin hiện tại. Tức là, ước lượng trạng thái chính xác có thể yêu cầu các công thức logic có kích thước hàm mũ theo số lượng ký hiệu.

Một phương pháp phổ biến và tự nhiên để ước lượng trạng thái gần đúng là biểu diễn các trạng thái niềm tin dưới dạng các liên kết của các literal, tức là các công thức 1-CNF. Để làm điều này, chương trình tác nhân đơn giản cố gắng chứng minh X^t và $\neg X^t$ cho mỗi ký hiệu X^t (cũng như mỗi ký hiệu phi thời gian mà giá trị chân lý chưa được biết), dựa trên trạng thái niềm tin tại $t - 1$. Liên kết của các literal có thể chứng minh được trở thành trạng thái niềm tin mới, và trạng thái niềm tin trước đó bị loại bỏ.



Hình 7.21 Mô tả trạng thái niềm tin 1-CNF (đường viền đậm) như một xấp xỉ bảo thủ, có thể biểu diễn đơn giản cho trạng thái niềm tin chính xác (đường gọn sáng) (vùng tô bóng với đường viền nét đứt). Mỗi thể giới khả thi được hiển thị dưới dạng một vòng tròn; những vòng tròn được tô bóng là nhất quán với tất cả các percept.

Điều quan trọng là phải hiểu rằng phương pháp này có thể làm mất một số thông tin khi thời gian trôi qua. Ví dụ, nếu câu trong Phương trình (7.4) là trạng thái niềm tin chính xác, thì cả $P_{3,1}$ và $P_{2,2}$ đều không thể chứng minh được riêng lẻ và không xuất hiện trong trạng thái niềm tin 1-CNF. (Bài tập 7.HYBR khám phá một giải pháp khả thi cho vấn đề này.) Mặt khác, vì mọi literal trong trạng thái niềm tin 1-CNF được chứng minh từ trạng thái niềm tin trước đó, và trạng thái niềm tin ban đầu là một khẳng định đúng, chúng ta biết rằng toàn bộ trạng thái niềm tin 1-CNF phải đúng. Do đó, tập hợp các trạng thái có thể được biểu diễn bởi trạng thái niềm tin 1-CNF bao gồm tất cả các trạng thái thực sự có thể xảy ra dựa trên toàn bộ lịch sử nhận thức. Như minh họa trong Hình 7.21, trạng thái niềm tin 1-CNF hoạt động như một phong bì đơn giản, hoặc một xấp xỉ bảo toàn, xung quanh trạng thái niềm tin chính xác. Chúng ta thấy ý tưởng về các xấp xỉ bảo toàn cho các tập hợp phức tạp như một chủ đề lặp đi lặp lại trong nhiều lĩnh vực của AI.

7.7.4 Tạo kế hoạch bằng suy luận mệnh đề

Tác nhân trong Hình 7.20 sử dụng suy luận logic để xác định ô vuông nào an toàn, nhưng sử dụng thuật toán tìm kiếm A^* để tạo kế hoạch. Trong phần này, chúng tôi chỉ ra cách tạo kế hoạch bằng suy luận logic. Ý tưởng cơ bản rất đơn giản:

20. Xây dựng một câu bao gồm:

- **Init**⁰, một tập hợp các khẳng định về trạng thái ban đầu;
- **Transition**¹, ..., **Transition**^t, các tiên đề trạng thái kế tiếp cho tất cả các hành động có thể tại mỗi thời điểm lên đến một thời điểm tối đa t ;
- Khẳng định rằng mục tiêu đạt được tại thời điểm t : **HaveGold**^t $\vee$ **ClimbedOut**^t.

21. Đưa toàn bộ câu cho một bộ giải SAT. Nếu bộ giải tìm thấy một mô hình thỏa mãn, thì mục tiêu có thể đạt được; nếu câu không thỏa mãn, thì vấn đề không giải quyết được.

22. Giả sử tìm thấy một mô hình, trích xuất từ mô hình các biến đại diện cho hành động và được gán giá trị true. Chúng cùng nhau đại diện cho một kế hoạch để đạt được mục tiêu.

function SATPLAN(*init*, *transition*, *goal*, $T_{\max}$) **returns** solution or *failure*

inputs: *init*, *transition*, *goal*, constitute a description of the problem

$T_{\max}$, an upper limit for plan length

for $t = 0$ **to** $T_{\max}$ **do**

$cnf \leftarrow \text{TRANSLATE-TO-SAT}(init, transition, goal, t)$

$model \leftarrow \text{SAT-SOLVER}(cnf)$

if *model* is not null **then**

return EXTRACT-SOLUTION(*model*)

return *failure*

Hình 7.22 Thuật toán SATPLAN. Bài toán lập kế hoạch được dịch sang một câu CNF trong đó mục tiêu được khẳng định là đúng tại một bước thời gian t cố định và các tiên đề được bao gồm cho mỗi bước thời gian lên đến t . Nếu thuật toán thỏa mãn tìm thấy một mô hình, thì một kế hoạch

được trích xuất bằng cách nhìn vào các ký hiệu mệnh đề đề cập đến các hành động và được gán giá trị đúng trong mô hình. Nếu không có mô hình nào tồn tại, thì quá trình được lặp lại với mục tiêu được dịch chuyển một bước sau đó.

Một quy trình lập kế hoạch mệnh đề, **SATPLAN**, được hiển thị trong Hình 7.22. Nó triển khai ý tưởng cơ bản vừa nêu, với một điểm khác biệt. Vì tác nhân không biết sẽ mất bao nhiêu bước để đạt được mục tiêu, thuật toán thử từng số bước t có thể, lên đến một độ dài kế hoạch tối đa có thể tương đương T_{max} . Bằng cách này, nó đảm bảo tìm được kế hoạch ngắn nhất nếu có tồn tại. Do cách SATPLAN tìm kiếm giải pháp, phương pháp này không thể sử dụng trong môi trường quan sát một phần; SATPLAN sẽ chỉ gán các biến không quan sát được với các giá trị cần thiết để tạo ra một giải pháp.

Bước quan trọng trong việc sử dụng SATPLAN là xây dựng cơ sở tri thức. Thoạt nhìn, có vẻ như các tiên đề về thể giới wumpus trong Phần 7.7.1 đủ cho các bước 1(a) và 1(b) ở trên. Tuy nhiên, có một sự khác biệt đáng kể giữa các yêu cầu cho quan hệ kéo theo (được kiểm tra bằng ASK) và các yêu cầu cho tính thỏa mãn.

Ví dụ, xét vị trí ban đầu của tác nhân là $[1,1]$, và giả sử mục tiêu khiêm tốn của tác nhân là ở $[2,1]$ tại thời điểm 1. Cơ sở tri thức ban đầu chứa $L_{1,1}^0$ và mục tiêu là $L_{2,1}^1$. Sử dụng ASK, chúng ta có thể chứng minh $L_{2,1}^1$ nếu **Forward**⁰ được khẳng định, và, một cách đáng yên tâm, chúng ta không thể chứng minh $L_{2,1}^1$ nếu, chẳng hạn, **Shoot**⁰ được khẳng định thay thế. Bây giờ, SATPLAN sẽ tìm thấy kế hoạch [**Forward**⁰]; cho đến nay, mọi thứ đều tốt.

Thật không may, SATPLAN cũng tìm thấy kế hoạch [**Shoot**⁰]. Làm thế nào điều này có thể xảy ra? Để tìm hiểu, chúng ta kiểm tra mô hình mà SATPLAN xây dựng: nó bao gồm phép gán $L_{2,1}^0$, tức là tác nhân có thể ở $[2,1]$ tại thời điểm 1 bằng cách ở đó tại thời điểm 0 và bắn. Người ta có thể hỏi, “Chẳng phải chúng ta đã nói tác nhân ở $[1,1]$ tại thời điểm 0 sao?” Đúng, nhưng chúng ta đã không nói với tác nhân rằng nó không thể ở hai nơi cùng một lúc! Đối với quan hệ kéo theo, $L_{2,1}^0$ không được biết và do đó không thể được sử dụng trong một chứng minh; đối với tính thỏa mãn, $L_{2,1}^0$ không được biết và có thể được gán bất kỳ giá trị nào giúp mục tiêu trở thành true.

SATPLAN là một công cụ gỡ lỗi tốt cho cơ sở tri thức vì nó tiết lộ những nơi thiếu kiến thức. Trong trường hợp cụ thể này, chúng ta có thể sửa cơ sở tri thức bằng cách khẳng định rằng, tại mỗi bước thời gian, tác nhân chỉ ở một vị trí, sử dụng một tập hợp các câu tương tự như những câu được sử dụng để khẳng định sự tồn tại của duy nhất một wumpus. Ngoài ra, chúng ta có thể khẳng định $\neg L_{x,y}^0$ cho tất cả các vị trí khác $[1,1]$; tiên đề trạng thái kế tiếp cho vị trí sẽ xử lý các bước thời gian tiếp theo. Các cách sửa chữa tương tự cũng áp dụng để đảm bảo tác nhân có một và chỉ một hướng tại một thời điểm.

SATPLAN còn có nhiều bất ngờ khác. Đầu tiên là nó tìm thấy các mô hình với các hành động không thể thực hiện, chẳng hạn như bắn mà không có mũi tên. Để hiểu tại sao, chúng ta cần xem xét kỹ hơn những gì các tiên đề trạng thái kế tiếp (như Phương trình (7.3)) nói về các hành động mà điều kiện tiên quyết không được thỏa mãn. Các tiên đề này dự đoán chính xác rằng không có gì xảy ra khi một hành động như vậy được thực hiện (xem Bài tập 7.SATP), nhưng chúng không nói rằng hành động đó không thể được thực hiện! Để tránh tạo ra các kế hoạch với hành động không hợp lệ, chúng ta phải thêm các tiên đề điều kiện tiên quyết khẳng định rằng một hành động yêu cầu các điều kiện tiên quyết phải được thỏa mãn. Ví dụ, chúng ta cần nói, cho mỗi thời điểm t , rằng

$$Shoot^t \Rightarrow HaveArrow^t.$$

Điều này đảm bảo rằng nếu một kế hoạch chọn hành động Shoot tại bất kỳ thời điểm nào, thì tác nhân phải có một mũi tên tại thời điểm đó.

Bắt ngờ thứ hai của SATPLAN là tạo ra các kế hoạch với nhiều hành động đồng thời. Ví dụ, nó có thể đưa ra một mô hình trong đó cả **Forward**⁰ và **Shoot**⁰ đều true, điều này không được phép. Để loại bỏ vấn đề này, chúng ta giới thiệu các tiên đề loại trừ hành động: cho mỗi cặp hành động A_i^t và A_j^t , chúng ta thêm tiên đề

$$-A_i^t \vee -A_j^t.$$

Có thể chỉ ra rằng việc đi về phía trước và bắn cùng một lúc không quá khó khăn, trong khi, chẳng hạn, bắn và nắm cùng một lúc là khá không thực tế. Bằng cách áp dụng các tiên đề loại trừ hành động chỉ cho các cặp hành động thực sự cản trở lẫn nhau, chúng ta có thể cho phép các kế hoạch bao gồm nhiều hành động đồng thời—và vì SATPLAN tìm thấy kế hoạch hợp lệ ngắn nhất, chúng ta có thể chắc chắn rằng nó sẽ tận dụng khả năng này.

Tóm lại, SATPLAN tìm các mô hình cho một câu chứa trạng thái ban đầu, mục tiêu, các tiên đề trạng thái kế tiếp, các tiên đề điều kiện tiên quyết và các tiên đề loại trừ hành động. Có thể chỉ ra rằng tập hợp các tiên đề này là đủ, theo nghĩa không còn bất kỳ “giải pháp” giả mạo nào. Bất kỳ mô hình nào thỏa mãn câu mệnh đề sẽ là một kế hoạch hợp lệ cho vấn đề ban đầu. Công nghệ giải SAT hiện đại làm cho phương pháp này khá thiết thực. Ví dụ, một bộ giải kiểu DPLL không gặp khó khăn trong việc tạo ra giải pháp cho trường hợp thế giới wumpus được hiển thị trong Hình 7.2.

Phần này đã mô tả một cách tiếp cận khai báo để xây dựng tác nhân: tác nhân hoạt động bằng cách kết hợp khẳng định các câu trong cơ sở tri thức và thực hiện suy luận logic. Cách tiếp cận này có một số điểm yếu ẩn trong các cụm từ như “cho mỗi thời điểm t ” và “cho mỗi ô vuông $[x, y]$ ”. Đối với bất kỳ tác nhân thực tế nào, các cụm từ này phải được triển khai bằng mã tạo ra các trường hợp của lược đồ câu chung một cách tự động để chèn vào cơ sở tri thức. Đối với một thế giới wumpus có kích thước hợp lý—tương đương với một trò chơi máy tính nhỏ—chúng ta có thể cần một bảng 100×100 và 1000 bước thời gian, dẫn đến cơ sở tri thức với hàng chục hoặc hàng trăm triệu câu.

Điều này không chỉ trở nên khá không thực tế mà còn minh họa một vấn đề sâu sắc hơn: chúng ta biết điều gì đó về thế giới wumpus—cụ thể là “vật lý” hoạt động giống nhau trên tất cả các ô vuông và tất cả các bước thời gian—mà chúng ta không thể diễn đạt trực tiếp bằng ngôn ngữ logic mệnh đề. Để giải quyết vấn đề này, chúng ta cần một ngôn ngữ biểu đạt hơn, trong đó các cụm từ như “cho mỗi thời điểm t ” và “cho mỗi ô vuông $[x, y]$ ” có thể được viết một cách tự nhiên. Logic bậc nhất, được mô tả trong Chương 8, là một ngôn ngữ như vậy; trong logic bậc nhất, một thế giới wumpus với bất kỳ kích thước và thời lượng nào có thể được mô tả trong khoảng mười câu logic thay vì mười triệu hoặc mười nghìn tỷ.

Tóm tắt

Chúng ta đã giới thiệu về các tác nhân dựa trên tri thức và cách định nghĩa một hệ thống logic để các tác nhân này có thể suy luận về thế giới. Những điểm chính bao gồm:

- **Tác nhân thông minh cần tri thức về thế giới** để đưa ra quyết định tốt.
- **Tri thức được chứa trong tác nhân** dưới dạng các câu trong ngôn ngữ biểu diễn tri thức, lưu trữ trong cơ sở tri thức.
- **Một tác nhân dựa trên tri thức** bao gồm cơ sở tri thức và cơ chế suy luận. Nó hoạt động bằng cách lưu trữ các câu về thế giới trong cơ sở tri thức, sử dụng cơ chế suy luận để suy ra các câu mới, và dùng những câu này để quyết định hành động.
- **Ngôn ngữ biểu diễn** được định nghĩa bởi cú pháp (xác định cấu trúc câu) và ngữ nghĩa (xác định tính đúng đắn của mỗi câu trong mỗi thế giới hoặc mô hình có thể).
- **Mối quan hệ kéo theo (entailment)** giữa các câu rất quan trọng để hiểu về suy luận. Một câu α kéo theo câu β nếu β đúng trong mọi thế giới mà α đúng. Các định nghĩa tương đương bao gồm tính hợp lệ của câu $\alpha \Rightarrow \beta$ và tính không thỏa mãn của câu $\alpha \wedge \neg\beta$.
- **Suy luận** là quá trình suy ra các câu mới từ các câu cũ. Các thuật toán suy luận đúng đắn (sound) chỉ suy ra các câu được kéo theo; các thuật toán đầy đủ (complete) suy ra tất cả các câu được kéo theo.
- **Logic mệnh đề** là một ngôn ngữ đơn giản gồm các ký hiệu mệnh đề và các phép logic. Nó có thể xử lý các mệnh đề được biết là đúng, sai hoặc hoàn toàn không xác định.
- **Tập hợp các mô hình khả dĩ**, với một bộ từ vựng mệnh đề cố định, là hữu hạn, do đó tính kéo theo có thể được kiểm tra bằng cách liệt kê các mô hình. Các thuật toán suy luận kiểm tra mô hình hiệu quả cho logic mệnh đề bao gồm phương pháp quay lui (backtracking) và tìm kiếm cục bộ (local search), thường giải quyết các bài toán lớn nhanh chóng.
- **Quy tắc suy luận** là các mẫu suy luận đúng đắn có thể được sử dụng để tìm chứng minh. Quy tắc phân giải (resolution) tạo ra một thuật toán suy luận đầy đủ cho các cơ sở tri thức được biểu diễn dưới dạng chuẩn hội (conjunctive normal form). Suy luận tiến (forward chaining) và suy luận lùi (backward chaining) là các thuật toán suy luận tự nhiên cho cơ sở tri thức dạng Horn.
- **Phương pháp tìm kiếm cục bộ** như WALKSAT có thể được sử dụng để tìm nghiệm. Các thuật toán này đúng đắn nhưng không đầy đủ.
- **Ước lượng trạng thái logic** liên quan đến việc duy trì một câu logic mô tả tập hợp các trạng thái khả dĩ phù hợp với lịch sử quan sát. Mỗi bước cập nhật đòi hỏi suy luận sử dụng mô hình chuyển tiếp của môi trường, được xây dựng từ các tiên đề trạng thái kế tiếp (successor-state axioms) mô tả cách mỗi fluent thay đổi.

- **Quyết định trong một tác nhân logic** có thể được thực hiện bằng cách giải SAT: tìm các mô hình khả dĩ chỉ định các chuỗi hành động tương lai đạt được mục tiêu. Cách tiếp cận này chỉ hoạt động cho các môi trường hoàn toàn quan sát được hoặc không có cảm biến.
- **Logic mệnh đề không mở rộng** được cho các môi trường có kích thước không giới hạn vì thiếu khả năng biểu diễn ngắn gọn về thời gian, không gian và các mẫu quan hệ phổ quát giữa các đối tượng.

Ghi chú lịch sử và thư mục

Bài báo “Programs with Common Sense” của John McCarthy (1958, 1968) truyền bá ý tưởng về các tác nhân sử dụng suy luận logic để điều hòa giữa tri giác và hành động. Nó cũng giương cao ngọn cờ của chủ nghĩa khai báo (declarativism), chỉ ra rằng việc nói cho tác nhân biết những gì nó cần biết là một cách thanh lịch để xây dựng phần mềm. Bài báo “The Knowledge Level” của Allen Newell (1982) lập luận rằng các tác nhân hợp lý có thể được mô tả và phân tích ở mức trừu tượng được định nghĩa bởi tri thức mà chúng sở hữu thay vì các chương trình chúng chạy.

Bản thân logic có nguồn gốc từ triết học và toán học Hy Lạp cổ đại. Plato thảo luận về cấu trúc cú pháp của câu, tính đúng sai của chúng, ý nghĩa và tính hợp lệ của các lập luận logic. Nghiên cứu hệ thống đầu tiên về logic là Organon của Aristotle. Các tam đoạn luận (syllogisms) của ông là những gì chúng ta ngày nay gọi là quy tắc suy luận, mặc dù chúng thiếu tính cấu thành (compositionality) so với các quy tắc hiện đại.

Trường phái Megarian và Stoic bắt đầu nghiên cứu hệ thống các phép logic cơ bản vào thế kỷ thứ 5 trước Công nguyên. Bảng chân lý (truth tables) là công trình của Philo of Megara. Các nhà Stoic coi năm quy tắc suy luận cơ bản là hợp lệ mà không cần chứng minh, bao gồm quy tắc mà ngày nay chúng ta gọi là Modus Ponens. Họ suy ra nhiều quy tắc khác từ năm quy tắc này, sử dụng, trong số các nguyên lý khác, định lý suy luận (deduction theorem, trang 240) và rõ ràng hơn Aristotle về chứng minh (Mates, 1953).

Ý tưởng rút gọn suy luận logic thành một quá trình thuần túy cơ học là của Wilhelm Leibniz (1646–1716). George Boole (1847) giới thiệu hệ thống logic hình thức toàn diện và khả thi đầu tiên trong cuốn sách The Mathematical Analysis of Logic. Logic của Boole mô phỏng chặt chẽ đại số thông thường của số thực và sử dụng phép thay thế các biểu thức tương đương logic làm phương pháp suy luận chính. Mặc dù nó không xử lý được toàn bộ logic mệnh đề, các nhà toán học khác đã sớm lấp đầy những phần còn thiếu. Schröder (1877) mô tả dạng chuẩn hội (conjunctive normal form), trong khi dạng Horn (Horn form) được Alfred Horn (1951) giới thiệu muộn hơn nhiều. Trình bày toàn diện đầu tiên về logic mệnh đề hiện đại (và logic bậc nhất) được tìm thấy trong Begriffsschrift (“Concept Writing” hoặc “Conceptual Notation”) của Gottlob Frege (1879).

Thiết bị cơ học đầu tiên thực hiện suy luận logic là Stanhope Demonstrator, được chế tạo bởi Bá tước Stanhope thứ ba (1753–1816). William Stanley Jevons, một trong những nhà toán học mở rộng công trình của Boole, đã chế tạo “piano logic” vào năm 1869 để thực hiện suy luận trong logic Boolean. Một lịch sử thú vị về các thiết bị suy luận cơ học đầu tiên này được Martin Gardner (1968) trình bày. Chương trình máy tính đầu tiên cho suy luận logic là chương trình chứng minh trong số học Presburger của Martin Davis (1954) (Davis, 1957), và Logic Theorist của Newell, Shaw, và Simon (1957). Emil Post (1921) và Ludwig Wittgenstein (1922) độc lập sử dụng bảng chân lý như một phương pháp kiểm tra tính hợp lệ của các câu logic mệnh đề.

Thuật toán Davis–Putnam (Davis và Putnam, 1960) là thuật toán đầu tiên cho phép phân giải (resolution) trong logic mệnh đề, và thuật toán quay lui DPLL cải tiến (Davis et al., 1962) tỏ ra hiệu quả hơn. Quy tắc phân giải và chứng minh tính đầy đủ của nó được phát triển một cách tổng quát cho logic bậc nhất bởi J. A. Robinson (1965). Stephen Cook (1971) chỉ ra rằng việc quyết định tính thỏa mãn của một câu trong logic mệnh đề (bài toán SAT) là NP-đầy đủ. Nhiều tập con của logic mệnh đề được biết đến mà bài toán thỏa mãn có thể giải được trong thời gian đa thức; mệnh đề Horn là một tập con như vậy.

Các nghiên cứu ban đầu cho thấy DPLL có độ phức tạp trung bình đa thức đối với một số phân bố bài toán tự nhiên. Thậm chí tốt hơn, Franco và Pauli (1983) chỉ ra rằng các bài toán tương tự có thể được giải trong thời gian hằng số bằng cách đoán ngẫu nhiên các phép gán. Được thúc đẩy bởi thành công thực nghiệm của tìm kiếm cục bộ, Koutsoupias và Papadimitriou (1992) chỉ ra rằng một thuật toán leo đồi đơn giản có thể giải quyết hầu hết các trường hợp bài toán thỏa mãn rất nhanh, gợi ý rằng các bài toán khó là hiếm. Schöning (1999) trình bày một thuật toán leo đồi ngẫu nhiên mà thời gian chạy dự kiến trong trường hợp xấu nhất đối với bài toán 3-SAT là $O(1.333^n)$ — vẫn là hàm mũ, nhưng nhanh hơn đáng kể so với các giới hạn trường hợp xấu nhất trước đó. Kỷ lục hiện tại là $O(1.32216^n)$ (Rolf, 2006).

Hiệu quả của các bộ giải mệnh đề đã tăng lên nhanh chóng. Với thời gian tính toán 10 phút, thuật toán DPLL ban đầu trên phần cứng năm 1962 chỉ có thể giải các bài toán với 10 hoặc 15 biến (trên máy tính xách tay năm 2019 sẽ là khoảng 30 biến). Đến năm 1995, bộ giải SATZ (Li và Anbulagan, 1997) có thể xử lý 1.000 biến, nhờ vào các cấu trúc dữ liệu tối ưu hóa để lập chỉ mục biến. Hai đóng góp quan trọng là kỹ thuật lập chỉ mục literal được theo dõi (watched literal indexing) của Zhang và Stickel (1996), giúp lan truyền đơn vị (unit propagation) rất hiệu quả, và việc giới thiệu các kỹ thuật học mệnh đề (tức ràng buộc) từ cộng đồng CSP bởi Bayardo và Schrag (1997). Sử dụng những ý tưởng này, và được thúc đẩy bởi triển vọng giải quyết các bài toán xác minh mạch quy mô công nghiệp, Moskewicz et al. (2001) đã phát triển bộ giải CHAFF, có thể xử lý các bài toán với hàng triệu biến. Bắt đầu từ năm 2002, các cuộc thi SAT hàng năm được tổ chức; hầu hết các giải thưởng thuộc về các biến thể của CHAFF. Bức tranh tổng quan về các bộ giải được Gomes et al. (2008) khảo sát.

Các thuật toán tìm kiếm cục bộ cho bài toán thỏa mãn đã được nhiều tác giả thử nghiệm trong suốt những năm 1980, dựa trên ý tưởng tối thiểu hóa số lượng mệnh đề không thỏa mãn (Hansen và Jaumard, 1990). Một thuật toán đặc biệt hiệu quả được phát triển bởi Gu (1989) và độc lập bởi Selman et al. (1992), họ gọi nó là GSAT và chỉ ra rằng nó có khả năng giải quyết nhiều bài toán rất khó một cách rất nhanh chóng. Thuật toán WALKSAT được mô tả trong chương này là của Selman et al. (1996).

“Chuyển pha” (phase transition) trong tính thỏa mãn của các bài toán 3-SAT ngẫu nhiên lần đầu tiên được quan sát bởi Simon và Dubois (1989) và đã tạo ra rất nhiều nghiên cứu lý thuyết và thực nghiệm — một phần do liên hệ với hiện tượng chuyển pha trong vật lý thống kê. Crawford và Auton (1993) xác định vị trí chuyển pha 3-SAT tại tỷ lệ mệnh đề/biến khoảng 4.26, lưu ý rằng điều này trùng với đỉnh nhọn trong thời gian chạy của bộ giải SAT của họ. Cook và Mitchell (1997) cung cấp một tổng quan tuyệt vời về tài liệu ban đầu về vấn đề này. Các thuật toán như lan truyền khảo sát (survey propagation) (Parisi và Zecchina, 2002; Maneva et al., 2007) tận dụng các tính chất đặc biệt của các trường hợp SAT ngẫu nhiên gần ngưỡng thỏa mãn và vượt trội hơn nhiều so với các bộ giải SAT tổng quát trên các trường hợp như vậy. Tình trạng hiện tại của sự hiểu biết lý thuyết được tổng hợp bởi Achlioptas (2009).

Các nguồn thông tin tốt về tính thỏa mãn, cả lý thuyết và thực tiễn, bao gồm Handbook of Satisfiability (Biere et al., 2009), chuyên khảo về tính thỏa mãn của Donald Knuth (2015), và các hội nghị quốc tế thường niên về Lý thuyết và Ứng dụng Kiểm tra Tính thỏa mãn, được gọi là SAT.

Ý tưởng xây dựng các tác nhân bằng logic mệnh đề có thể bắt nguồn từ bài báo mang tính bước ngoặt của McCulloch và Pitts (1943), nổi tiếng với việc khởi xướng lĩnh vực mạng nơ-ron, nhưng thực sự quan tâm đến việc triển khai thiết kế tác nhân dựa trên mạch Boolean trong não bộ. Stan Rosenschein (Rosenstein, 1985; Kaelbling và Rosenstein, 1990) đã phát triển các cách biên dịch các tác nhân dựa trên mạch từ các mô tả khai báo của môi trường tác vụ. Rod Brooks (1986, 1989) chứng minh hiệu quả của các thiết kế dựa trên mạch để điều khiển robot (xem Chương 26). Brooks (1991) lập luận rằng các thiết kế dựa trên mạch là tất cả những gì cần thiết cho AI — rằng biểu diễn và suy luận là công kênh, tốn kém và không cần thiết. Theo quan điểm của chúng tôi, cả suy luận và mạch đều cần thiết. Williams et al. (2003) mô tả một tác nhân lai — không quá khác biệt so với tác nhân wumpus của chúng tôi — điều khiển tàu vũ trụ NASA, lập kế hoạch chuỗi hành động và chẩn đoán, khôi phục sau lỗi.

Vấn đề chung theo dõi một môi trường quan sát một phần được giới thiệu cho các biểu diễn dựa trên trạng thái trong Chương 4. Phiên bản của nó cho các biểu diễn mệnh đề được nghiên cứu bởi Amir và Russell (2003), những người đã xác định một số lớp môi trường cho phép các thuật toán ước lượng trạng thái hiệu quả và chỉ ra rằng đối với một số lớp khác, vấn đề là không thể giải quyết được. Bài toán chiếu thời gian (temporal projection), liên quan đến việc xác định các mệnh đề nào đúng sau khi thực hiện một chuỗi hành động, có thể được coi là một trường hợp đặc biệt của ước lượng trạng thái với tri giác rỗng. Nhiều tác giả đã nghiên cứu bài toán này do tầm quan trọng của nó trong lập kế hoạch; một số kết quả khó quan trọng đã được thiết lập bởi Liberatore (1997). Ý tưởng biểu diễn một trạng thái niềm tin bằng các mệnh đề có thể bắt nguồn từ Wittgenstein (1922).

Cách tiếp cận ước lượng trạng thái logic sử dụng chỉ số thời gian trên các biến mệnh đề được đề xuất bởi Kautz và Selman (1992). Các thế hệ sau của SATPLAN có thể tận dụng những tiến bộ trong bộ giải SAT và vẫn là một trong những cách hiệu quả nhất để giải quyết các bài toán lập kế hoạch khó (Kautz, 2006).

Bài toán khung (frame problem) lần đầu tiên được nhận ra bởi McCarthy và Hayes (1969). Nhiều nhà nghiên cứu coi bài toán này không thể giải quyết được trong logic bậc nhất, và nó đã thúc đẩy rất nhiều nghiên cứu về logic không đơn điệu (nonmonotonic logics). Các triết gia từ Dreyfus (1972) đến Crockett (1994) đã coi bài toán khung như một triệu chứng của sự thất bại không thể tránh khỏi của toàn bộ ngành AI. Giải pháp cho bài toán khung bằng các tiên đề trạng thái kế tiếp (successor-state axioms) là của Ray Reiter (1991). Thielscher (1999) xác định bài toán khung suy luận (inferential frame problem) như một ý tưởng riêng biệt và cung cấp một giải pháp. Nhìn lại, người ta có thể thấy rằng các tác nhân của Rosenschein (1985) đã sử dụng các mạch triển khai các tiên đề trạng thái kế tiếp, nhưng Rosenschein không nhận ra rằng bài toán khung do đó đã được giải quyết phần lớn.

Các bộ giải mệnh đề hiện đại đã được áp dụng cho nhiều ứng dụng công nghiệp, chẳng hạn như tổng hợp phần cứng máy tính (Nowick et al., 1993). Bộ kiểm tra tính thỏa mãn SATMC đã được sử dụng để phát hiện một lỗ hổng chưa từng được biết đến trước đây trong giao thức đăng nhập trình duyệt Web (Armando et al., 2008).

Thế giới wumpus được phát minh như một trò chơi bởi Gregory Yob (1975). Trớ trêu thay, Yob phát triển nó vì chán các trò chơi trên lưới hình chữ nhật: ông đặt wumpus của mình trên một khối

mười hai mặt, và chúng tôi đặt nó trở lại trên lưới cũ nhằm chán. Michael Genesereth đề xuất sử dụng thế giới wumpus như một bộ thử nghiệm cho tác nhân.

CHƯƠNG 8: LOGIC BẬC NHẤT

Trong chương này, chúng ta nhận thấy rằng thế giới được ban tặng nhiều đối tượng, một số trong đó có liên quan đến các đối tượng khác, và chúng ta sẽ nỗ lực suy luận về chúng.

Logic mệnh đề đã đủ để minh họa các khái niệm cơ bản về logic, suy diễn và các tác tử dựa trên tri thức. Thật không may, logic mệnh đề bị giới hạn về những gì nó có thể diễn đạt. Trong chương này, chúng ta xem xét logic bậc nhất, một logic có thể biểu diễn nhiều thứ hơn một cách ngắn gọn. Chúng ta bắt đầu trong Mục 8.1 với một cuộc thảo luận chung về các ngôn ngữ biểu diễn; Mục 8.2 bao gồm cú pháp và ngữ nghĩa của logic bậc nhất; Mục 8.3 và 8.4 minh họa việc sử dụng logic bậc nhất cho các biểu diễn đơn giản.

8.1 Ôn lại về Biểu diễn

Trong phần này, chúng ta sẽ thảo luận về bản chất của các ngôn ngữ biểu diễn. Các ngôn ngữ lập trình (như C++, Java, hoặc Python) là lớp ngôn ngữ hình thức được sử dụng phổ biến nhất. Cấu trúc dữ liệu trong các chương trình có thể được sử dụng để biểu diễn các sự thật; ví dụ, một chương trình có thể sử dụng mảng 4×4 để biểu diễn nội dung của thế giới wumpus. Do đó, câu lệnh trong ngôn ngữ lập trình $\text{World}[2,2] \leftarrow \text{Pit}$ là một cách khá tự nhiên để khẳng định rằng có một cái hố (pit) ở ô $[2,2]$. Xây dựng một chuỗi các câu lệnh như vậy là đủ để chạy một mô phỏng thế giới wumpus.

Cái mà các ngôn ngữ lập trình thiếu là một cơ chế chung để suy ra các sự thật từ những sự thật khác. Mỗi lần cập nhật một cấu trúc dữ liệu đều được thực hiện bởi một thủ tục cụ thể theo miền, với các chi tiết được lập trình viên rút ra từ kiến thức của chính họ về miền đó. Cách tiếp cận theo thủ tục này có thể được đối lập với bản chất khai báo của logic mệnh đề, trong đó kiến thức và suy luận là riêng biệt, và suy luận hoàn toàn độc lập với miền. Các cơ sở dữ liệu SQL kết hợp cả kiến thức khai báo và thủ tục.

Một nhược điểm thứ hai của cấu trúc dữ liệu trong chương trình (và của cơ sở dữ liệu) là thiếu một cách dễ dàng để nói, ví dụ, “Có một cái hố ở $[2,2]$ hoặc $[3,1]$ ” hoặc “Nếu wumpus ở $[1,1]$ thì nó không ở $[2,2]$ ”. Các chương trình có thể lưu trữ một giá trị duy nhất cho mỗi biến, và một số hệ thống cho phép giá trị là “không xác định”, nhưng chúng thiếu khả năng diễn đạt cần thiết để xử lý trực tiếp thông tin không đầy đủ.

Logic mệnh đề là một ngôn ngữ khai báo vì ngữ nghĩa của nó dựa trên một quan hệ chân lý giữa các câu và các thế giới khả dĩ. Nó cũng có đủ sức biểu đạt để xử lý thông tin không đầy đủ, bằng cách sử dụng phép tuyển và phép phủ định. Logic mệnh đề có một thuộc tính thứ ba đáng mong đợi ở các ngôn ngữ biểu diễn, đó là **tính hợp thành**. Trong một ngôn ngữ có tính hợp thành, ý nghĩa của một câu là một hàm của ý nghĩa các bộ phận của nó. Ví dụ, ý nghĩa của “ $S_{1,2}$ ” liên quan đến ý nghĩa của “ $S_{1,4}$ ” và “ $S_{1,2}$ ”. “Sẽ rất lạ nếu” $S_{1,4}$ có nghĩa là có mùi hôi ở ô $[1,4]$ và “ $S_{1,2}$ ” có nghĩa là có mùi hôi ở ô $[1,2]$, nhưng “ $S_{1,4} \wedge S_{1,2}$ ” lại có nghĩa là Pháp và Ba Lan hòa 1-1 trong trận đấu vòng loại khúc côn cầu trên băng tuần trước.

Tuy nhiên, logic mệnh đề, với tư cách là một biểu diễn được phân tách (factored representation), thiếu sức mạnh diễn đạt để mô tả một môi trường có nhiều đối tượng một cách súc tích. Ví dụ, chúng ta buộc phải viết một quy tắc riêng về gió thoảng và hố cho mỗi ô, chẳng hạn như

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}).$$

Mặt khác, trong tiếng Anh, có vẻ khá dễ dàng để nói một lần và mãi mãi, “Các ô liền kề với hồ thì có gió thoảng”. Cú pháp và ngữ nghĩa của tiếng Anh giúp có thể mô tả môi trường một cách súc tích: tiếng Anh, giống như logic bậc nhất, là một biểu diễn có cấu trúc.

8.1.1 Ngôn ngữ của tư tưởng

Các ngôn ngữ tự nhiên (như tiếng Anh hoặc tiếng Tây Ban Nha) thực sự rất biểu cảm. Chúng ta đã xoay sở để viết gần như toàn bộ cuốn sách này bằng ngôn ngữ tự nhiên, chỉ thỉnh thoảng mới sử dụng các ngôn ngữ khác (chủ yếu là toán học và biểu đồ). Có một truyền thống lâu đời trong ngôn ngữ học và triết học ngôn ngữ cho rằng ngôn ngữ tự nhiên là một ngôn ngữ biểu diễn kiến thức khai báo. Nếu chúng ta có thể khám phá ra các quy tắc của ngôn ngữ tự nhiên, chúng ta có thể sử dụng chúng trong các hệ thống biểu diễn và suy luận và thu được lợi ích từ hàng tỷ trang đã được viết bằng ngôn ngữ tự nhiên.

Quan điểm hiện đại về ngôn ngữ tự nhiên là nó đóng vai trò như một phương tiện giao tiếp hơn là một biểu diễn thuần túy. Khi một người nói chỉ tay và nói, “Nhìn kia!”, người nghe biết rằng, chẳng hạn, Superman cuối cùng đã xuất hiện trên nóc nhà. Tuy nhiên, chúng ta sẽ không muốn nói rằng câu “Nhìn kia!” biểu diễn sự thật đó. Thay vào đó, ý nghĩa của câu phụ thuộc vào cả bản thân câu và ngữ cảnh mà câu đó được nói ra. Rõ ràng, người ta không thể lưu trữ một câu như “Nhìn kia!” trong một cơ sở kiến thức và mong đợi khôi phục được ý nghĩa của nó mà không lưu trữ một biểu diễn của ngữ cảnh - điều này lại đặt ra câu hỏi làm thế nào để biểu diễn chính ngữ cảnh đó.

Các ngôn ngữ tự nhiên cũng bị ảnh hưởng bởi tính mơ hồ, một vấn đề đối với một ngôn ngữ biểu diễn. Như Pinker (1995) đã nói: “Khi mọi người nghĩ về ‘spring’ (mùa xuân), chắc chắn họ không bối rối về việc liệu họ đang nghĩ về một mùa hay một thứ gì đó kêu ‘boing’ - và nếu một từ có thể tương ứng với hai ý nghĩ, thì ý nghĩ không thể là từ”.

Giả thuyết Sapir-Whorf nổi tiếng (Whorf, 1956) cho rằng sự hiểu biết của chúng ta về thế giới bị ảnh hưởng mạnh mẽ bởi ngôn ngữ chúng ta nói. Chắc chắn là các cộng đồng nói tiếng khác nhau chia thế giới theo những cách khác nhau. Người Pháp có hai từ “chaise” và “fauteuil” cho một khái niệm mà người nói tiếng Anh chỉ dùng một từ: “chair” (ghế). Nhưng người nói tiếng Anh có thể dễ dàng nhận ra danh mục

fauteuil và đặt cho nó một cái tên - đại khái là “ghế bành hờ tay vịn” - vậy ngôn ngữ có thực sự tạo ra sự khác biệt không? Whorf chủ yếu dựa vào trực giác và suy đoán, và những ý tưởng của ông đã bị bác bỏ phần lớn, nhưng trong những năm sau đó, chúng ta thực sự đã có dữ liệu thực tế từ các nghiên cứu nhân học, tâm lý học và thần kinh học.

Ví dụ, bạn có nhớ câu nào trong hai câu sau đây đã tạo thành phần mở đầu của Phần 8.1 không?

- “Trong phần này, chúng ta sẽ thảo luận về bản chất của các ngôn ngữ biểu diễn...”
- “Phần này bao gồm chủ đề về các ngôn ngữ biểu diễn kiến thức...”

Wanner (1974) đã làm một thí nghiệm tương tự và nhận thấy rằng các đối tượng chọn đúng ở mức độ ngẫu nhiên - khoảng 50% thời gian - nhưng lại nhớ nội dung họ đã đọc với độ chính xác hơn 90%. Điều này cho thấy rằng mọi người diễn giải các từ họ đọc và hình thành một biểu diễn phi ngôn ngữ bên trong, và rằng các từ chính xác không có ý nghĩa quan trọng.

Thú vị hơn là trường hợp một khái niệm hoàn toàn không có trong một ngôn ngữ. Người nói ngôn ngữ thổ dân Úc **Guugu Yimithirr** không có từ nào cho các hướng tương đối (hoặc vị trí trung

tâm), chẳng hạn như trước, sau, phải hoặc trái. Thay vào đó, họ sử dụng các hướng tuyệt đối, nói, ví dụ, tương đương với “Tôi bị đau ở cánh tay phía bắc của mình”. Sự khác biệt về ngôn ngữ này tạo ra sự khác biệt trong hành vi: người nói tiếng Guugu Yimithirr giỏi điều hướng hơn trên địa hình rộng mở, trong khi người nói tiếng Anh giỏi hơn trong việc đặt cái đĩa bên phải cái đĩa. Ngôn ngữ cũng dường như ảnh hưởng đến suy nghĩ thông qua các đặc điểm ngữ pháp có vẻ tùy tiện như giới tính của danh từ. Ví dụ, “cây cầu” (bridge) là giống đực trong tiếng Tây Ban Nha và giống cái trong tiếng Đức. Boroditsky (2003) đã yêu cầu các đối tượng chọn các tính từ tiếng Anh để mô tả một bức ảnh về một cây cầu cụ thể. Người nói tiếng Tây Ban Nha đã chọn *lớn*, *nguy hiểm*, *mạnh mẽ* và *cao vút*, trong khi người nói tiếng Đức đã chọn *đẹp*, *thanh lịch*, *mỏng manh* và *thon thả*.

Các từ có thể đóng vai trò là điểm neo ảnh hưởng đến cách chúng ta nhìn nhận thế giới. Loftus và Palmer (1974) đã cho các đối tượng thí nghiệm xem một bộ phim về một vụ tai nạn ô tô. Các đối tượng được hỏi “Các xe đã đi nhanh như thế nào khi chúng **tiếp xúc** với nhau?” đã báo cáo tốc độ trung bình là 32 mph, trong khi các đối tượng được hỏi câu hỏi với từ “**smashed**” (đập tan) thay vì “**contacted**” (tiếp xúc) đã báo cáo 41 mph cho cùng một chiếc xe trong cùng một bộ phim. Nhìn chung, có những khác biệt có thể đo lường được nhưng nhỏ trong quá trình nhận thức của người nói các ngôn ngữ khác nhau, nhưng không có bằng chứng thuyết phục nào cho thấy điều này dẫn đến sự khác biệt lớn trong thế giới quan.

Trong một hệ thống suy luận logic sử dụng dạng chuẩn hội (CNF), chúng ta có thể thấy rằng các dạng ngôn ngữ $\neg(A \vee B)$ và $\neg A \wedge \neg B$ là giống nhau bởi vì chúng ta có thể nhìn vào bên trong hệ thống và thấy rằng hai câu được lưu trữ dưới cùng một dạng CNF chuẩn. Việc làm điều gì đó tương tự với bộ não con người đang bắt đầu trở nên khả thi. Mitchell và cộng sự (2008) đã đặt các đối tượng vào một máy chụp cộng hưởng từ chức năng (fMRI), cho họ xem các từ như “cần tây” và chụp ảnh não của họ. Một chương trình học máy được huấn luyện trên các cặp (từ, hình ảnh) đã có thể dự đoán chính xác 77% thời gian trong các nhiệm vụ lựa chọn nhị phân (ví dụ: “cần tây” hoặc “máy bay”). Hệ thống thậm chí có thể dự đoán ở mức trên ngẫu nhiên cho những từ mà nó chưa bao giờ thấy hình ảnh fMRI trước đây (bằng cách xem xét hình ảnh của các từ liên quan) và cho những người mà nó chưa bao giờ thấy trước đây (chứng minh rằng fMRI tiết lộ một mức độ biểu diễn chung giữa mọi người). Loại công việc này vẫn còn ở giai đoạn sơ khai, nhưng fMRI (và các công nghệ hình ảnh khác như điện sinh lý nội sọ (Sahin et al., 2009)) hứa hẹn sẽ cung cấp cho chúng ta những ý tưởng cụ thể hơn nhiều về biểu diễn kiến thức của con người là như thế nào.

Từ quan điểm của logic hình thức, việc biểu diễn cùng một kiến thức theo hai cách khác nhau hoàn toàn không tạo ra sự khác biệt; các sự thật giống nhau sẽ có thể suy ra được từ cả hai biểu diễn. Tuy nhiên, trên thực tế, một biểu diễn có thể yêu cầu ít bước hơn để suy ra một kết luận, có nghĩa là một người suy luận có tài nguyên hạn chế có thể đi đến kết luận bằng cách sử dụng biểu diễn này nhưng không phải là biểu diễn kia. Đối với các nhiệm vụ không suy diễn như học hỏi từ kinh nghiệm, kết quả nhất thiết phải phụ thuộc vào dạng biểu diễn được sử dụng. Chúng ta sẽ chỉ ra trong Chương 19 rằng khi một chương trình học cân nhắc hai lý thuyết khả dĩ của thế giới, cả hai đều nhất quán với tất cả dữ liệu, cách phổ biến nhất để phá vỡ thế bế tắc là chọn lý thuyết súc tích nhất - và điều đó phụ thuộc vào ngôn ngữ được sử dụng để biểu diễn các lý thuyết. Do đó, ảnh hưởng của ngôn ngữ đối với suy nghĩ là không thể tránh khỏi đối với bất kỳ tác nhân nào thực hiện việc học.

8.1.2 Kết hợp những điểm tốt nhất của ngôn ngữ hình thức và ngôn ngữ tự nhiên

Chúng ta có thể áp dụng nền tảng của logic mệnh đề - một ngữ nghĩa khai báo, tổ hợp, độc lập với ngữ cảnh và không mơ hồ - và xây dựng một logic biểu đạt hơn trên nền tảng đó, vay mượn các ý tưởng biểu diễn từ ngôn ngữ tự nhiên trong khi tránh các nhược điểm của nó. Khi chúng ta nhìn vào cú pháp của ngôn ngữ tự nhiên, các yếu tố rõ ràng nhất là danh từ và cụm danh từ để chỉ **đối tượng** (ô vuông, hồ, wumpus) và động từ và cụm động từ cùng với tính từ và trạng từ để chỉ **quan hệ** giữa các đối tượng (có gió thoảng, liền kề với, bắn). Một số mối quan hệ này là **hàm** - các mối quan hệ trong đó chỉ có một “giá trị” cho một “đầu vào” nhất định. Rất dễ để bắt đầu liệt kê các ví dụ về đối tượng, quan hệ và hàm:

- **Đối tượng:** con người, ngôi nhà, con số, lý thuyết, Ronald McDonald, màu sắc, các trận đấu bóng chày, các cuộc chiến tranh, các thể kỷ...
- **Quan hệ:** có thể là các quan hệ một ngôi (unary) hoặc thuộc tính (property) như *đỏ*, *tròn*, *giả*, *nguyên tố*, *nhiều tầng*... hoặc các quan hệ n-ngôi tổng quát hơn như *anh/em của*, *lớn hơn*, *bên trong*, *là một phần của*, *có màu*, *xảy ra sau*, *sở hữu*, *nằm giữa*...
- **Hàm:** *cha của*, *bạn thân nhất*, *hiệp thứ ba của*, *nhiều hơn một*, *bắt đầu của*...

Thật vậy, gần như bất kỳ khẳng định nào cũng có thể được coi là đề cập đến các đối tượng và thuộc tính hoặc quan hệ. Một số ví dụ sau đây:

- “Một cộng hai bằng ba.” **Đối tượng:** *một, hai, ba, một cộng hai*; **Quan hệ:** *bằng*; **Hàm:** *cộng*. (“Một cộng hai” là một tên cho đối tượng được tạo ra bằng cách áp dụng hàm “cộng” cho các đối tượng “một” và “hai.” “Ba” là một tên khác cho đối tượng này.)
- “Các ô liền kề với wumpus thì có mùi hôi.” **Đối tượng:** *wumpus, các ô vuông*; **Thuộc tính:** *có mùi hôi*; **Quan hệ:** *liền kề*.
- “Vua John độc ác đã cai trị nước Anh vào năm 1200.” **Đối tượng:** *John, Anh, 1200*; **Quan hệ:** *cai trị trong*; **Thuộc tính:** *độc ác, vua*.

Ngôn ngữ của logic bậc nhất, mà cú pháp và ngữ nghĩa của nó chúng ta sẽ định nghĩa trong phần tiếp theo, được xây dựng xung quanh các đối tượng và quan hệ. Nó đã trở nên quan trọng đối với toán học, triết học và trí tuệ nhân tạo chính xác bởi vì những lĩnh vực đó - và thực sự, phần lớn sự tồn tại hàng ngày của con người - có thể được coi là hữu ích khi giải quyết các đối tượng và các mối quan hệ giữa chúng. Logic bậc nhất cũng có thể diễn đạt các sự thật về một số hoặc tất cả các đối tượng trong vũ trụ. Điều này cho phép người ta biểu diễn các quy luật hoặc quy tắc chung, chẳng hạn như câu “Các ô liền kề với wumpus thì có mùi hôi”.

Sự khác biệt cơ bản giữa logic mệnh đề và logic bậc nhất nằm ở **cam kết bản thể học** (ontological commitment) mà mỗi ngôn ngữ đưa ra - tức là, những gì nó giả định về bản chất của thực tại. Về mặt toán học, cam kết này được thể hiện thông qua bản chất của các mô hình hình thức mà theo đó chân lý của các câu được định nghĩa. Ví dụ, logic mệnh đề giả định rằng có những sự thật hoặc tồn tại hoặc không tồn tại trong thế giới. Mỗi sự thật có thể ở một trong hai trạng thái - **đúng** hoặc **sai** - và mỗi mô hình gán đúng hoặc sai cho mỗi ký hiệu mệnh đề (xem Phần 7.4.2). Logic bậc nhất giả định nhiều hơn; cụ thể là, thế giới bao gồm các đối tượng với các mối quan hệ nhất định giữa chúng, có hoặc không tồn tại. (Xem Hình 8.1.) Các mô hình hình thức tương ứng phức tạp hơn so với các mô hình của logic mệnh đề.

Ngôn ngữ	Cam kết bản thể học (Những gì tồn tại trong thế giới)	Cam kết nhận thức luận (Những gì một tác nhân tin về sự thật)
Logic mệnh đề	các sự thật	đúng/sai/không xác định
Logic bậc nhất	các sự thật, đối tượng, quan hệ	đúng/sai/không xác định
Logic thời gian	các sự thật, đối tượng, quan hệ, thời gian	đúng/sai/không xác định
Lý thuyết xác suất	các sự thật	mức độ tin tưởng $\in [0, 1]$
Logic mờ	các sự thật với mức độ chân lý $\in [0, 1]$	giá trị khoảng đã biết

Hình 8.1 Các ngôn ngữ hình thức và các cam kết bản thể học và nhận thức luận của chúng.

Cam kết bản thể học này là một thể mạnh lớn của logic (cả logic mệnh đề và logic bậc nhất), bởi vì nó cho phép chúng ta bắt đầu với các câu đúng và suy ra các câu đúng khác. Nó đặc biệt mạnh mẽ trong các miền mà mỗi mệnh đề có ranh giới rõ ràng, chẳng hạn như toán học hoặc thế giới wumpus, nơi một ô vuông hoặc có hoặc không có hồ; không có khả năng có một ô vuông với một vết lõm giống hồ một cách mơ hồ. Nhưng trong thế giới thực, nhiều mệnh đề có ranh giới mơ hồ: Vienna có phải là một thành phố lớn không? Nhà hàng này có phục vụ đồ ăn ngon không? Người đó có cao không? Nó phụ thuộc vào người bạn hỏi, và câu trả lời của họ có thể là “đại loại thế”.

Một phản ứng là tinh chỉnh biểu diễn: nếu một đường thô sơ chia các thành phố thành “lớn” và “không lớn” bỏ qua quá nhiều thông tin cho ứng dụng đang được xem xét, thì người ta có thể tăng số lượng các danh mục kích thước hoặc sử dụng một ký hiệu hàm

Dân số. Một giải pháp được đề xuất khác đến từ **Logic mờ** (Fuzzy logic), đưa ra cam kết bản thể học rằng các mệnh đề có một **mức độ chân lý** (degree of truth) từ 0 đến 1. Ví dụ, câu “Vienna là một thành phố lớn” có thể đúng ở mức độ 0.8 trong logic mờ, trong khi “Paris là một thành phố lớn” có thể đúng ở mức độ 0.9. Điều này tương ứng tốt hơn với quan niệm trực quan của chúng ta về thế giới, nhưng nó làm cho việc suy luận trở nên khó khăn hơn: thay vì một quy tắc để xác định chân lý của $A \wedge B$, logic mờ cần các quy tắc khác nhau tùy thuộc vào miền. Một khả năng khác, được đề cập trong Phần 25.1, là gán mỗi khái niệm cho một điểm trong không gian đa chiều, và sau đó đo khoảng cách giữa khái niệm “thành phố lớn” và khái niệm “Vienna” hoặc “Paris”.

Các logic chuyên dụng khác nhau đưa ra các cam kết bản thể học xa hơn nữa; ví dụ, **logic thời gian** (temporal logic) giả định rằng các sự thật tồn tại ở những thời điểm cụ thể và những thời điểm đó (có thể là điểm hoặc khoảng) được sắp xếp. Do đó, các logic chuyên dụng cung cấp cho các loại đối tượng nhất định (và các tiên đề về chúng) trạng thái “hạng nhất” trong logic, thay vì chỉ đơn giản là định nghĩa chúng trong cơ sở kiến thức.

Logic bậc cao (higher-order logic) xem các quan hệ và hàm được đề cập bởi logic bậc nhất như là các đối tượng trong chính chúng. Điều này cho phép người ta đưa ra các khẳng định về tất cả các quan hệ - ví dụ, người ta có thể muốn định nghĩa ý nghĩa của việc một quan hệ là bắc cầu. Không giống như hầu hết các logic chuyên dụng, logic bậc cao có tính biểu đạt nghiêm ngặt hơn logic bậc nhất, theo nghĩa là một số câu của logic bậc cao không thể được diễn đạt bằng bất kỳ số câu hữu hạn nào của logic bậc nhất.

Một logic cũng có thể được đặc trưng bởi **cam kết nhận thức luận** (epistemological commitments) của nó - các trạng thái kiến thức khả dĩ mà nó cho phép đối với mỗi sự thật. Trong cả logic mệnh đề và logic bậc nhất, một câu biểu diễn một sự thật và tác nhân hoặc tin rằng câu đó là đúng, tin rằng nó là sai, hoặc không có ý kiến. Do đó, các logic này có ba trạng thái kiến thức khả dĩ đối với bất kỳ câu nào.

Mặt khác, các hệ thống sử dụng lý thuyết xác suất có thể có bất kỳ mức độ tin tưởng nào, hoặc khả năng chủ quan, dao động từ 0 (hoàn toàn không tin) đến 1 (hoàn toàn tin). Điều quan trọng là không được nhầm lẫn mức độ tin tưởng trong lý thuyết xác suất với mức độ chân lý trong logic mờ. Thật vậy, một số hệ thống mờ cho phép sự không chắc chắn (mức độ tin tưởng) về mức độ chân lý. Ví dụ, một tác nhân thể giới wumpus dựa trên xác suất có thể tin rằng wumpus ở $[1,3]$ với xác suất 0.75 và ở $[2, 3]$ với xác suất 0.25 (mặc dù wumpus chắc chắn ở một ô vuông cụ thể).

8.2 Cú pháp và ngữ nghĩa của logic bậc nhất

Trong phần này, chúng ta sẽ bắt đầu bằng việc xác định một cách chính xác hơn cách mà các thể giới khả dĩ của logic bậc nhất phản ánh cam kết bản thể luận đối với các đối tượng và quan hệ. Sau đó, chúng ta sẽ giới thiệu các yếu tố khác nhau của ngôn ngữ, đồng thời giải thích ngữ nghĩa của chúng. Các điểm chính là cách ngôn ngữ này tạo điều kiện cho các biểu diễn ngắn gọn và cách ngữ nghĩa của nó dẫn đến các thủ tục suy luận hợp lý.

8.2.1 Các mô hình cho logic bậc nhất

Chương 7 đã nói rằng các mô hình của một ngôn ngữ logic là các cấu trúc hình thức tạo nên các thể giới khả dĩ được xem xét. Mỗi mô hình liên kết từ vựng của các câu logic với các yếu tố của thể giới khả dĩ, để có thể xác định tính đúng của bất kỳ câu nào. Do đó, các mô hình cho logic mệnh đề liên kết các ký hiệu mệnh đề với các giá trị chân lý được xác định trước.

Các mô hình cho logic bậc nhất thú vị hơn nhiều. Đầu tiên, chúng có các đối tượng! **Miền (domain)** của một mô hình là tập hợp các đối tượng hoặc các phần tử miền mà nó chứa. Miền được yêu cầu phải không rỗng—mỗi thể giới khả dĩ phải chứa ít nhất một đối tượng. (Xem Bài tập 8.EMPT để thảo luận về các thể giới rỗng.) Về mặt toán học, không quan trọng các đối tượng này là gì—tất cả những gì quan trọng là có bao nhiêu đối tượng trong mỗi mô hình cụ thể—nhưng vì mục đích sư phạm, chúng ta sẽ sử dụng một ví dụ cụ thể. Hình 8.2 cho thấy một mô hình với năm đối tượng: Richard Sư tử tâm, Vua Anh từ năm 1189 đến 1199; em trai ông, Vua John độc ác, người trị vì từ năm 1199 đến 1215; chân trái của Richard và John; và một chiếc vương miện.

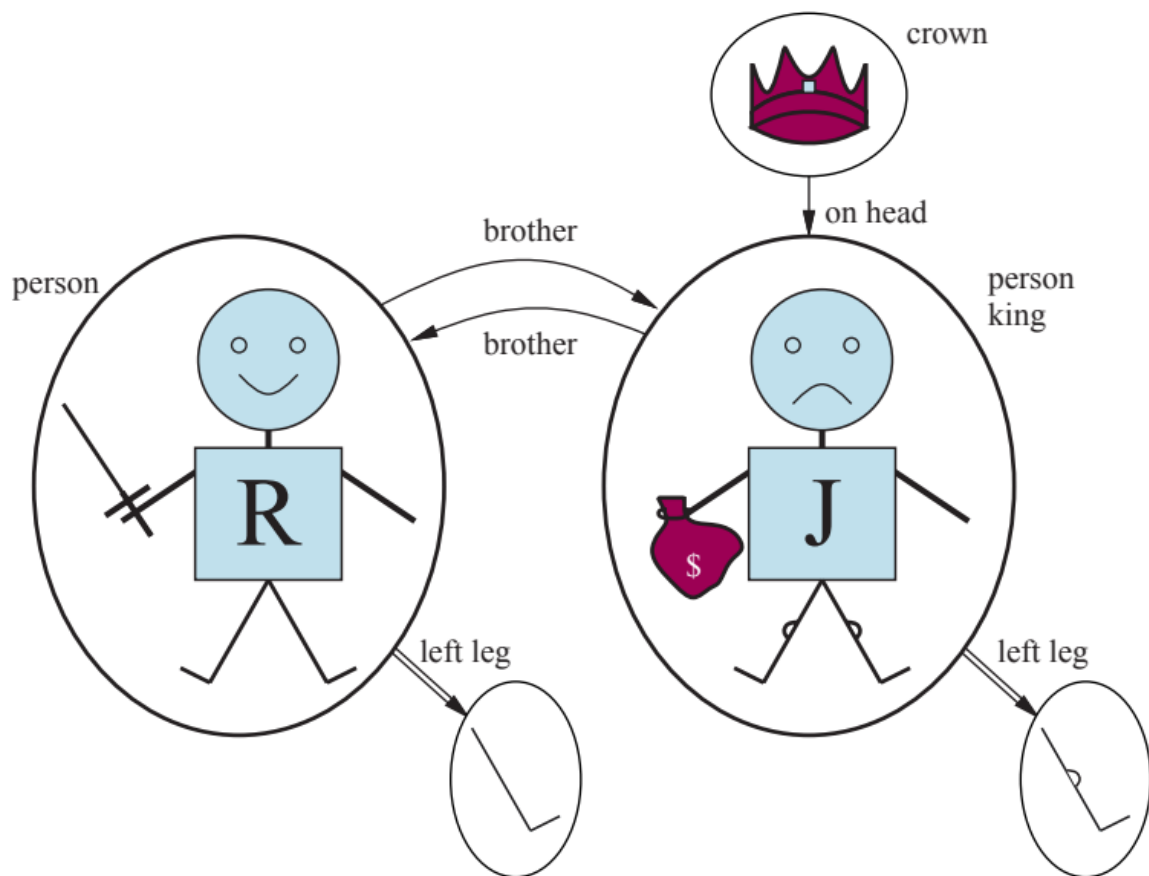
Các đối tượng trong mô hình có thể liên quan với nhau theo nhiều cách khác nhau. Trong hình, Richard và John là anh em. Về mặt hình thức, một **quan hệ** chỉ là tập hợp các bộ **tuple** các đối tượng có liên quan. (Một tuple là một tập hợp các đối tượng được sắp xếp theo một thứ tự cố định và được viết bằng các dấu ngoặc nhọn bao quanh các đối tượng.) Do đó, quan hệ anh em trong mô hình này là tập hợp $\{ \langle \text{Richard Sư tử tâm}, \text{Vua John} \rangle, \langle \text{Vua John}, \text{Richard Sư tử tâm} \rangle \}$. (8.1) (Ở đây chúng ta đã đặt tên các đối tượng bằng tiếng Anh, nhưng nếu muốn, bạn có thể thay thế tính thân các hình ảnh cho các tên.) Vương miện nằm trên đầu Vua John, vì vậy quan hệ “trên đầu” chỉ chứa một tuple, $\langle \text{vương miện}, \text{Vua John} \rangle$. Các quan hệ “anh em” và “trên đầu” là các **quan hệ nhị phân**—nghĩa là, chúng liên kết các cặp đối tượng. Mô hình cũng chứa các **quan hệ một ngôi**, hoặc thuộc tính: thuộc tính “người” đúng với cả Richard và John; thuộc tính “vua” chỉ đúng với

John (có lẽ vì Richard đã chết vào thời điểm này); và thuộc tính “vương miện” chỉ đúng với vương miện.

Một số loại quan hệ nhất định được coi là **hàm**, trong đó một đối tượng cho trước phải liên quan đến chính xác một đối tượng theo cách này. Ví dụ, mỗi người có một chân trái, vì vậy mô hình có một hàm “chân trái” một ngôi—một ánh xạ từ một tuple một phần tử đến một đối tượng—bao gồm các ánh xạ sau:

$\langle \text{Richard Sur tử tâm} \rangle \rightarrow \text{chân trái của Richard}$

$\langle \text{Vua John} \rangle \rightarrow \text{chân trái của John. (8.2)}$



Hình 8.2 Một mô hình chứa năm đối tượng, hai quan hệ hai ngôi (brother và on-head), ba quan hệ một ngôi (person, king, và crown), và một hàm một ngôi (left-leg).

Nói một cách chính xác, các mô hình trong logic bậc nhất yêu cầu các **hàm toàn phần**, tức là phải có một giá trị cho mọi tuple đầu vào. Do đó, chiếc vương miện phải có một chân trái và mỗi chiếc chân trái cũng vậy. Có một giải pháp kỹ thuật cho vấn đề khó xử này liên quan đến một đối tượng “vô hình” bổ sung là chân trái của mọi thứ không có chân trái, kể cả chính nó. May mắn thay, miễn là người ta không đưa ra các khẳng định về chân trái của những thứ không có chân trái, những chi tiết kỹ thuật này không quan trọng.

Cho đến nay, chúng ta đã mô tả các yếu tố tạo nên các mô hình cho logic bậc nhất. Phần thiết yếu khác của một mô hình là sự liên kết giữa các yếu tố đó và từ vựng của các câu logic, mà chúng ta sẽ giải thích tiếp theo.

8.2.2 Ký hiệu và diễn giải

Bây giờ chúng ta chuyển sang cú pháp của logic bậc nhất. Độc giả thiếu kiên nhẫn có thể lấy mô tả đầy đủ từ ngữ pháp hình thức trong Hình 8.3. Các yếu tố cú pháp cơ bản của logic bậc nhất là các ký hiệu đại diện cho các đối tượng, quan hệ và hàm. Do đó, các ký hiệu có ba loại: các **ký hiệu hằng**, đại diện cho các đối tượng; các **ký hiệu vị từ**, đại diện cho các quan hệ; và các **ký hiệu hàm**, đại diện cho các hàm. Chúng ta tuân theo quy ước rằng các ký hiệu này sẽ bắt đầu bằng các chữ cái viết hoa. Ví dụ, chúng ta có thể sử dụng các ký hiệu hằng Richard và John; các ký hiệu vị từ Brother, OnHead, Person, King và Crown; và ký hiệu hàm LeftLeg. Tương tự như các ký hiệu mệnh đề, việc lựa chọn tên hoàn toàn tùy thuộc vào người dùng. Mỗi ký hiệu vị từ và hàm đi kèm với một **arity** (số ngôi) cố định số lượng đối số.

<i>Sentence</i>	→	<i>AtomicSentence</i> <i>ComplexSentence</i>
<i>AtomicSentence</i>	→	<i>Predicate</i> <i>Predicate</i> (<i>Term</i> ,...) <i>Term</i> = <i>Term</i>
<i>ComplexSentence</i>	→	(<i>Sentence</i>)
		¬ <i>Sentence</i>
		<i>Sentence</i> ∧ <i>Sentence</i>
		<i>Sentence</i> ∨ <i>Sentence</i>
		<i>Sentence</i> ⇒ <i>Sentence</i>
		<i>Sentence</i> ⇔ <i>Sentence</i>
		<i>Quantifier Variable</i> ,... <i>Sentence</i>
<i>Term</i>	→	<i>Function</i> (<i>Term</i> ,...)
		<i>Constant</i>
		<i>Variable</i>
<i>Quantifier</i>	→	∀ ∃
<i>Constant</i>	→	<i>A</i> <i>X</i> ₁ <i>John</i> ...
<i>Variable</i>	→	<i>a</i> <i>x</i> <i>s</i> ...
<i>Predicate</i>	→	<i>True</i> <i>False</i> <i>After</i> <i>Loves</i> <i>Raining</i> ...
<i>Function</i>	→	<i>Mother</i> <i>LeftLeg</i> ...

OPERATOR PRECEDENCE : ¬, =, ∧, ∨, ⇒, ⇔

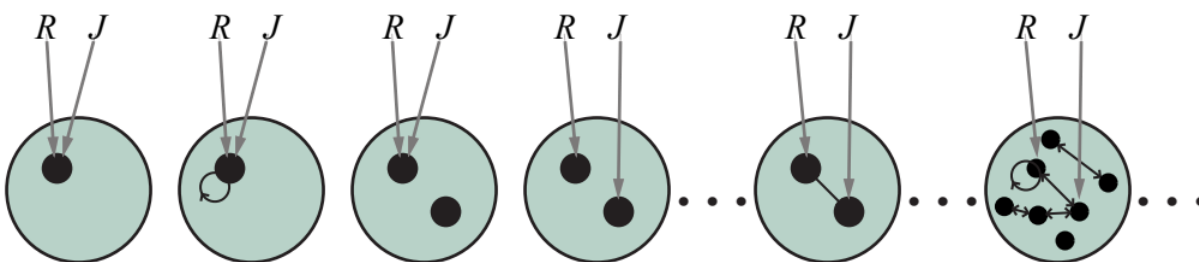
Hình 8.3. Cú pháp của **logic bậc nhất** với phép bằng, được chỉ định dưới dạng Backus-Naur form (BNF) (xem trang 1081 nếu bạn không quen thuộc với ký hiệu này). Độ ưu tiên của các toán tử được chỉ định, từ cao nhất đến thấp nhất. Độ ưu tiên của các lượng từ là một lượng từ có hiệu lực trên mọi thứ ở bên phải nó.

Mỗi mô hình phải cung cấp thông tin cần thiết để xác định xem bất kỳ câu nào có đúng hay sai hay không. Do đó, ngoài các đối tượng, quan hệ và hàm của nó, mỗi mô hình bao gồm một **diễn giải** chỉ định chính xác các đối tượng, quan hệ và hàm nào được đề cập bởi các ký hiệu hằng, vị từ

và hàm. Một diễn giải khả thi cho ví dụ của chúng ta—mà một nhà logic học sẽ gọi là **diễn giải có chủ ý**—như sau:

- Richard đề cập đến Richard Sư tử tâm và John đề cập đến Vua John độc ác.
- Brother đề cập đến quan hệ anh em—nghĩa là, tập hợp các tuple đối tượng được cho trong Phương trình (8.1); OnHead là một quan hệ đúng giữa vương miện và Vua John; Person, King và Crown là các quan hệ một ngôi xác định người, vua và vương miện.
- LeftLeg đề cập đến hàm “chân trái” như được định nghĩa trong Phương trình (8.2).

Tất nhiên, có nhiều diễn giải khả thi khác. Ví dụ, một diễn giải ánh xạ Richard đến vương miện và John đến chân trái của Vua John. Có năm đối tượng trong mô hình, vì vậy có 25 diễn giải khả thi chỉ cho các ký hiệu hằng Richard và John.



Hình 8.4 Một số thành viên của tập hợp tất cả các mô hình cho một ngôn ngữ có hai ký hiệu hằng, R và J, và một ký hiệu quan hệ hai ngôi. Sự diễn giải của mỗi ký hiệu hằng được thể hiện bằng một mũi tên màu xám. Trong mỗi mô hình, các đối tượng có liên quan được kết nối bằng các mũi tên.

Lưu ý rằng không phải tất cả các đối tượng đều cần có tên—ví dụ, diễn giải có chủ ý không đặt tên cho vương miện hoặc các chân. Một đối tượng cũng có thể có nhiều tên; có một diễn giải trong đó cả Richard và John đều đề cập đến vương miện. Nếu bạn thấy khả năng này gây nhầm lẫn, hãy nhớ rằng, trong logic mệnh đề, hoàn toàn có thể có một mô hình trong đó Cloudy và Sunny đều đúng; nhiệm vụ của cơ sở tri thức là loại trừ các mô hình không nhất quán với tri thức của chúng ta.

Tóm lại, một **mô hình** trong logic bậc nhất bao gồm một tập hợp các đối tượng và một diễn giải ánh xạ các ký hiệu hằng đến các đối tượng, các ký hiệu hàm đến các hàm trên các đối tượng đó và các ký hiệu vị từ đến các quan hệ. Cũng như với logic mệnh đề, sự kéo theo, tính hợp lệ, v.v., được định nghĩa theo tất cả các mô hình khả dĩ. Để có ý tưởng về tập hợp tất cả các mô hình khả dĩ trông như thế nào, hãy xem Hình 8.4. Nó cho thấy rằng các mô hình thay đổi về số lượng đối tượng mà chúng chứa—từ một đến vô cùng—và theo cách các ký hiệu hằng ánh xạ đến các đối tượng.

Bởi vì số lượng mô hình bậc nhất là vô hạn, chúng ta không thể kiểm tra sự kéo theo bằng cách liệt kê tất cả chúng (như chúng ta đã làm với logic mệnh đề). Ngay cả khi số lượng đối tượng bị hạn chế, số lượng kết hợp có thể rất lớn. (Xem Bài tập 8.MCNT.) Đối với ví dụ trong Hình 8.4, có 137.506.194.466 mô hình với sáu đối tượng trở xuống.

8.2.3 Thuật ngữ

Một **thuật ngữ** là một biểu thức logic đề cập đến một đối tượng. Các ký hiệu hằng là các thuật ngữ, nhưng không phải lúc nào cũng thuận tiện khi có một ký hiệu riêng biệt để đặt tên cho mọi đối tượng. Trong tiếng Anh, chúng ta có thể sử dụng biểu thức “chân trái của Vua John” thay vì đặt tên cho chân của ông. Đây là lý do tại sao các ký hiệu hàm tồn tại: thay vì sử dụng một ký hiệu hằng, chúng ta sử dụng $\text{LeftLeg}(\text{John})$.

Trong trường hợp tổng quát, một **thuật ngữ phức tạp** được hình thành bởi một ký hiệu hàm theo sau là một danh sách các thuật ngữ được đặt trong ngoặc đơn làm đối số cho ký hiệu hàm. Điều quan trọng cần nhớ là một thuật ngữ phức tạp chỉ là một loại tên phức tạp. Nó không phải là một “lời gọi chương trình con” để “trả về một giá trị.” Không có chương trình con LeftLeg nào lấy một người làm đầu vào và trả về một cái chân. Chúng ta có thể suy luận về chân trái (ví dụ, nêu ra quy tắc chung rằng mọi người đều có một chân và sau đó suy ra rằng John cũng phải có một chân) mà không cần phải cung cấp một định nghĩa về LeftLeg . Đây là điều không thể thực hiện được với các chương trình con trong các ngôn ngữ lập trình.

Ngữ nghĩa hình thức của các thuật ngữ rất đơn giản. Hãy xem xét một thuật ngữ $f(t_1, \dots, t_n)$. Ký hiệu hàm f đề cập đến một hàm nào đó trong mô hình (gọi nó là F); các thuật ngữ đối số đề cập đến các đối tượng trong miền (gọi chúng là $d_1, \dots, d_n$); và thuật ngữ nói chung đề cập đến đối tượng là giá trị của hàm F được áp dụng cho $d_1, \dots, d_n$. Ví dụ, giả sử ký hiệu hàm LeftLeg đề cập đến hàm được hiển thị trong Phương trình (8.2) và John đề cập đến Vua John, thì $\text{LeftLeg}(\text{John})$ đề cập đến chân trái của Vua John. Bằng cách này, diễn giải sửa lỗi tham chiếu của mọi thuật ngữ.

8.2.4 Câu nguyên tử

Bây giờ chúng ta đã có các thuật ngữ để đề cập đến các đối tượng và các ký hiệu vị từ để đề cập đến các quan hệ, chúng ta có thể kết hợp chúng để tạo ra các **câu nguyên tử** (hoặc gọi tắt là **nguyên tử**) để nêu các sự kiện. Một câu nguyên tử được hình thành từ một ký hiệu vị từ tùy chọn theo sau là một danh sách các thuật ngữ được đặt trong ngoặc đơn, chẳng hạn như $\text{Brother}(\text{Richard}, \text{John})$. Điều này nói rằng, theo diễn giải có chủ ý đã cho ở trên, Richard Sư tử tâm là anh trai của Vua John. Các câu nguyên tử có thể có các thuật ngữ phức tạp làm đối số. Do đó, $\text{Married}(\text{Father}(\text{Richard}), \text{Mother}(\text{John}))$ nói rằng cha của Richard Sư tử tâm đã kết hôn với mẹ của Vua John (một lần nữa, theo một diễn giải phù hợp).

Một câu nguyên tử là đúng trong một mô hình nhất định nếu quan hệ được đề cập bởi ký hiệu vị từ giữ giữa các đối tượng được đề cập bởi các đối số.

8.2.5 Câu phức tạp

Chúng ta có thể sử dụng các phép nối logic để xây dựng các câu phức tạp hơn, với cùng cú pháp và ngữ nghĩa như trong phép tính mệnh đề. Dưới đây là bốn câu đúng trong mô hình của Hình 8.2 theo diễn giải có chủ ý của chúng ta:

$\neg \text{Brother}(\text{LeftLeg}(\text{Richard}), \text{John})$

$\text{Brother}(\text{Richard}, \text{John}) \wedge \text{Brother}(\text{John}, \text{Richard})$

$\text{King}(\text{Richard}) \vee \text{King}(\text{John})$

$\neg \text{King}(\text{Richard}) \Rightarrow \text{King}(\text{John})$

8.2.6 Các lượng từ

Khi chúng ta có một logic cho phép các đối tượng, điều tự nhiên là muốn thể hiện các thuộc tính của toàn bộ các tập hợp đối tượng, thay vì liệt kê các đối tượng theo tên. Các **lượng từ** cho phép chúng ta làm điều này. Logic bậc nhất chứa hai lượng từ chuẩn, được gọi là phổ quát và tồn tại.

Lượng từ phổ quát ($\forall$)

Hãy nhớ lại khó khăn của chúng ta trong Chương 7 với việc biểu diễn các quy tắc chung trong logic mệnh đề. Các quy tắc như “Các ô vuông lân cận wumpus có mùi” và “Tất cả các vị vua đều là người” là những điều cơ bản của logic bậc nhất. Chúng ta sẽ giải quyết quy tắc đầu tiên trong số này trong Mục 8.3. Quy tắc thứ hai, “Tất cả các vị vua đều là người,” được viết trong logic bậc nhất là:

$$\forall x \text{King}(x) \Rightarrow \text{Person}(x)$$

Lượng từ phổ quát $\forall$ thường được phát âm là “Với mọi...”. (Hãy nhớ rằng chữ A lộn ngược có nghĩa là “tất cả.”) Do đó, câu nói, “Với mọi x , nếu x là một vị vua, thì x là một người.” Ký hiệu x được gọi là một **biến**. Theo quy ước, các biến là các chữ cái viết thường. Một biến là một thuật ngữ tự nó, và do đó cũng có thể đóng vai trò là đối số của một hàm—ví dụ, $\text{LeftLeg}(x)$. Một thuật ngữ không có biến được gọi là **thuật ngữ cơ bản**.

Một cách trực quan, câu $\forall xP$, trong đó P là bất kỳ câu logic nào, nói rằng P là đúng đối với mọi đối tượng x . Chính xác hơn, $\forall xP$ là đúng trong một mô hình nhất định nếu P là đúng trong tất cả các **diễn giải mở rộng** khả dĩ được xây dựng từ diễn giải được đưa ra trong mô hình, trong đó mỗi diễn giải mở rộng chỉ định một phần tử miền mà x đề cập đến.

Điều này nghe có vẻ phức tạp, nhưng nó thực sự chỉ là một cách cẩn thận để diễn đạt ý nghĩa trực quan của lượng từ phổ quát. Hãy xem xét mô hình được hiển thị trong Hình 8.2 và diễn giải có chủ ý đi kèm với nó. Chúng ta có thể mở rộng diễn giải theo năm cách:

$x \rightarrow \text{Richard Sur tử tâm}$

$x \rightarrow \text{Vua John}$

$x \rightarrow \text{chân trái của Richard}$

$x \rightarrow \text{chân trái của John}$

$x \rightarrow \text{vương miện}$

Câu được lượng từ phổ quát $\forall x \text{King}(x) \Rightarrow \text{Person}(x)$ là đúng trong mô hình gốc nếu câu $\text{King}(x) \Rightarrow \text{Person}(x)$ là đúng theo mỗi trong năm diễn giải mở rộng. Nghĩa là, câu được lượng từ phổ quát tương đương với việc khẳng định năm câu sau:

Richard Sur tử tâm là một vị vua $\Rightarrow$ Richard Sur tử tâm là một người. Vua John là một vị vua $\Rightarrow$ Vua John là một người. Chân trái của Richard là một vị vua $\Rightarrow$ chân trái của Richard là một người. Chân trái của John là một vị vua $\Rightarrow$ chân trái của John là một người. Vương miện là một vị vua $\Rightarrow$ vương miện là một người.

Hãy xem xét cẩn thận tập hợp các khẳng định này. Vì, trong mô hình của chúng ta, Vua John là vị vua duy nhất, câu thứ hai khẳng định rằng ông là một người, như chúng ta hy vọng. Nhưng còn bốn câu kia thì sao, dường như đưa ra các tuyên bố về chân và vương miện? Đó có phải là một phần ý nghĩa của “Tất cả các vị vua đều là người”? Trên thực tế, bốn khẳng định kia là đúng trong mô hình, nhưng không hề đưa ra bất kỳ tuyên bố nào về phẩm chất người của chân, vương miện, hay thậm chí là Richard. Điều này là do không có đối tượng nào trong số này là một vị vua. Nhìn vào bảng chân lý cho $\Rightarrow$ (Hình 7.8 trên trang 237), chúng ta thấy rằng phép kéo theo là đúng bất cứ khi nào tiền đề của nó sai—bất kể tính đúng của kết luận. Do đó, bằng cách khẳng định câu được lượng từ phổ quát, tương đương với việc khẳng định toàn bộ danh sách các phép kéo theo cá nhân, chúng ta kết thúc bằng việc khẳng định kết luận của quy tắc chỉ cho những đối tượng mà tiền đề là đúng và không nói gì cả về những đối tượng mà tiền đề là sai. Do đó, định nghĩa bảng chân lý của $\Rightarrow$ hóa ra là hoàn hảo để viết các quy tắc chung với các lượng từ phổ quát.

Một sai lầm phổ biến, thường xuyên được thực hiện ngay cả bởi những độc giả siêng năng đã đọc đoạn này vài lần, là sử dụng phép nối thay vì phép kéo theo. Câu

$$\forall x King(x) \wedge Person(x)$$

sẽ tương đương với việc khẳng định Richard Su tử tâm là một vị vua $\wedge$ Richard Su tử tâm là một người, Vua John là một vị vua $\wedge$ Vua John là một người, chân trái của Richard là một vị vua $\wedge$ chân trái của Richard là một người, v.v. Rõ ràng, điều này không nắm bắt được những gì chúng ta muốn.

Lượng từ tồn tại ($\exists$)

Lượng từ phổ quát đưa ra các tuyên bố về mọi đối tượng. Tương tự, chúng ta có thể đưa ra một tuyên bố về một số đối tượng mà không cần đặt tên cho nó, bằng cách sử dụng một **lượng từ tồn tại**. Để nói, ví dụ, rằng Vua John có một chiếc vương miện trên đầu, chúng ta viết

$$\exists x Crown(x) \wedge OnHead(x, John)$$

$\exists x$ được phát âm là “Tồn tại một x sao cho...” hoặc “Với một số x ...”

Một cách trực quan, câu $\exists xP$ nói rằng P là đúng đối với ít nhất một đối tượng x . Chính xác hơn, $\exists xP$ là đúng trong một mô hình nhất định nếu P là đúng trong ít nhất một diễn giải mở rộng gán x cho một phần tử miền. Nghĩa là, ít nhất một trong những điều sau đây là đúng:

Richard Su tử tâm là một vương miện $\wedge$ Richard Su tử tâm đang ở trên đầu John; Vua John là một vương miện $\wedge$ Vua John đang ở trên đầu John; chân trái của Richard là một vương miện $\wedge$ chân trái của Richard đang ở trên đầu John; chân trái của John là một vương miện $\wedge$ chân trái của John đang ở trên đầu John; Vương miện là một vương miện $\wedge$ vương miện đang ở trên đầu John.

Khẳng định thứ năm là đúng trong mô hình, vì vậy câu được lượng từ tồn tại ban đầu là đúng trong mô hình. Lưu ý rằng, theo định nghĩa của chúng ta, câu cũng sẽ đúng trong một mô hình trong đó Vua John đang đội hai chiếc vương miện. Điều này hoàn toàn nhất quán với câu gốc “Vua John có một chiếc vương miện trên đầu.”

Giống như $\Rightarrow$ dường như là phép nối tự nhiên để sử dụng với $\forall$, $\wedge$ là phép nối tự nhiên để sử dụng với $\exists$. Sử dụng $\wedge$ làm phép nối chính với $\forall$ đã dẫn đến một tuyên bố quá mạnh trong ví dụ ở phần trước; sử dụng $\Rightarrow$ với $\exists$ thường dẫn đến một tuyên bố rất yếu. Hãy xem xét câu sau:

$$\exists x Crown(x) \Rightarrow OnHead(x, John)$$

Bề ngoài, điều này có vẻ giống như một cách diễn đạt hợp lý cho câu của chúng ta. Áp dụng ngữ nghĩa, chúng ta thấy rằng câu nói rằng ít nhất một trong các khẳng định sau là đúng: Richard Sư tử tâm là một vương miện $\Rightarrow$ Richard Sư tử tâm đang ở trên đầu John; Vua John là một vương miện $\Rightarrow$ Vua John đang ở trên đầu John; chân trái của Richard là một vương miện $\Rightarrow$ chân trái của Richard đang ở trên đầu John; và cứ thế. Một phép kéo theo là đúng nếu cả tiền đề và kết luận đều đúng, hoặc nếu tiền đề của nó sai; vì vậy nếu Richard Sư tử tâm không phải là một vương miện, thì khẳng định đầu tiên là đúng và lượng từ tồn tại được thỏa mãn. Do đó, một câu kéo theo được lượng từ tồn tại là đúng bất cứ khi nào bất kỳ đối tượng nào không thỏa mãn tiền đề; do đó, những câu như vậy thực sự không nói lên nhiều điều.

Các lượng từ lồng nhau

Chúng ta sẽ thường muốn diễn đạt các câu phức tạp hơn bằng cách sử dụng nhiều lượng từ. Trường hợp đơn giản nhất là khi các lượng từ cùng loại. Ví dụ, “Anh em là anh chị em” có thể được viết là

$$\forall x \forall y Brother(x, y) \Rightarrow Sibling(x, y)$$

Các lượng từ liên tiếp cùng loại có thể được viết thành một lượng từ với nhiều biến. Ví dụ, để nói rằng quan hệ anh chị em là một quan hệ đối xứng, chúng ta có thể viết

$$\forall x, y Sibling(x, y) \Leftrightarrow Sibling(y, x)$$

Trong các trường hợp khác, chúng ta sẽ có sự pha trộn. “Mọi người yêu một ai đó” có nghĩa là với mỗi người, có một người mà người đó yêu:

$$\forall x \exists y Loves(x, y)$$

Mặt khác, để nói “Có một người được mọi người yêu”, chúng ta viết

$$\exists y \forall x Loves(x, y)$$

Do đó, thứ tự của lượng từ là rất quan trọng. Nó trở nên rõ ràng hơn nếu chúng ta chen dấu ngoặc đơn. $\forall x (\exists y Loves(x, y))$ nói rằng mọi người đều có một thuộc tính cụ thể, cụ thể là thuộc tính mà họ yêu một ai đó. Mặt khác, $\exists y (\forall x Loves(x, y))$ nói rằng một người nào đó trên thế giới có một thuộc tính cụ thể, cụ thể là thuộc tính được mọi người yêu.

Một số nhầm lẫn có thể phát sinh khi hai lượng từ được sử dụng với cùng một tên biến. Hãy xem xét câu

$$\forall x (Crown(x) \vee (\exists x Brother(Richard, x)))$$

Ở đây, x trong $Brother(Richard, x)$ được lượng hóa tồn tại. Quy tắc là biến thuộc về lượng từ trong cùng nhất đề cập đến nó; sau đó nó sẽ không bị ảnh hưởng bởi bất kỳ lượng từ nào khác. Một cách khác để nghĩ về nó là thế này: $\exists x Brother(Richard, x)$ là một câu về Richard (rằng anh ta có một người anh em), không phải về x ; vì vậy việc đặt một $\forall x$ bên ngoài nó không có tác dụng gì. Nó cũng có thể được viết là $\exists z Brother(Richard, z)$. Bởi vì điều này có thể là một nguồn gây nhầm lẫn, chúng ta sẽ luôn sử dụng các tên biến khác nhau với các lượng từ lồng nhau.

Mối liên hệ giữa $\forall$ và $\exists$

Hai lượng từ thực sự có mối liên hệ mật thiết với nhau, thông qua phủ định. Khẳng định rằng mọi người đều không thích củ cải là giống như khẳng định rằng không tồn tại ai thích chúng, và ngược lại:

$$\forall x \neg Likes(x, Parsnips) \text{ tương đương với } \neg \exists x Likes(x, Parsnips)$$

Chúng ta có thể đi xa hơn một bước: “Mọi người đều thích kem” có nghĩa là không có ai không thích kem:

$$\forall x Likes(x, IceCream) \text{ tương đương với } \neg \exists x \neg Likes(x, IceCream)$$

Bởi vì $\forall$ thực sự là một phép nối trên vũ trụ của các đối tượng và $\exists$ là một phép tuyển, nên không có gì đáng ngạc nhiên khi chúng tuân theo các quy tắc của De Morgan. Các quy tắc của De Morgan cho các câu được lượng hóa và không được lượng hóa như sau:

$$\begin{array}{ll} \neg \exists x P & \equiv \forall x \neg P \\ \neg \forall x P & \equiv \exists x \neg P \\ \forall x P & \equiv \neg \exists x \neg P \\ \exists x P & \equiv \neg \forall x \neg P \end{array} \qquad \begin{array}{ll} \neg(P \vee Q) & \equiv \neg P \wedge \neg Q \\ \neg(P \wedge Q) & \equiv \neg P \vee \neg Q \\ P \wedge Q & \equiv \neg(\neg P \vee \neg Q) \\ P \vee Q & \equiv \neg(\neg P \wedge \neg Q) \end{array}$$

Do đó, chúng ta không thực sự cần cả $\forall$ và $\exists$, giống như chúng ta không thực sự cần cả $\wedge$ và $\vee$. Tuy nhiên, tính dễ đọc quan trọng hơn sự tiết kiệm, vì vậy chúng ta sẽ giữ cả hai lượng từ.

8.2.7 Dấu bằng

Logic bậc nhất bao gồm một cách nữa để tạo các câu nguyên tử, ngoài việc sử dụng một vị từ và các thuật ngữ như đã mô tả trước đó. Chúng ta có thể sử dụng **dấu bằng** để biểu thị rằng hai thuật ngữ đề cập đến cùng một đối tượng. Ví dụ,

$$Father(John) = Henry$$

nói rằng đối tượng được đề cập bởi $Father(John)$ và đối tượng được đề cập bởi $Henry$ là một và cùng một. Bởi vì một diễn giải sửa lỗi tham chiếu của bất kỳ thuật ngữ nào, việc xác định tính đúng của một câu bằng chỉ đơn giản là xem các tham chiếu của hai thuật ngữ có phải là cùng một đối tượng hay không.

Dấu bằng có thể được sử dụng để nêu các sự kiện về một hàm nhất định, như chúng ta vừa làm với ký hiệu $Father$. Nó cũng có thể được sử dụng với phủ định để khẳng định rằng hai thuật ngữ không phải là cùng một đối tượng. Để nói rằng Richard có ít nhất hai anh em, chúng ta sẽ viết

$$\exists x, y Brother(x, Richard) \wedge Brother(y, Richard) \wedge \neg(x = y)$$

Câu

$$\exists x, y Brother(x, Richard) \wedge Brother(y, Richard)$$

không có ý nghĩa dự định. Cụ thể, nó là đúng trong mô hình của Hình 8.2, nơi Richard chỉ có một anh em. Để thấy điều này, hãy xem xét diễn giải mở rộng trong đó cả x và y đều được gán cho

Vua John. Việc thêm $\neg(x = y)$ loại trừ các mô hình như vậy. Ký hiệu $x \neq y$ đôi khi được sử dụng như một viết tắt cho $\neg(x = y)$.

8.2.8 Ngữ nghĩa cơ sở dữ liệu

Tiếp tục ví dụ từ phần trước, giả sử rằng chúng ta tin rằng Richard có hai anh em, John và Geoffrey. Chúng ta có thể viết

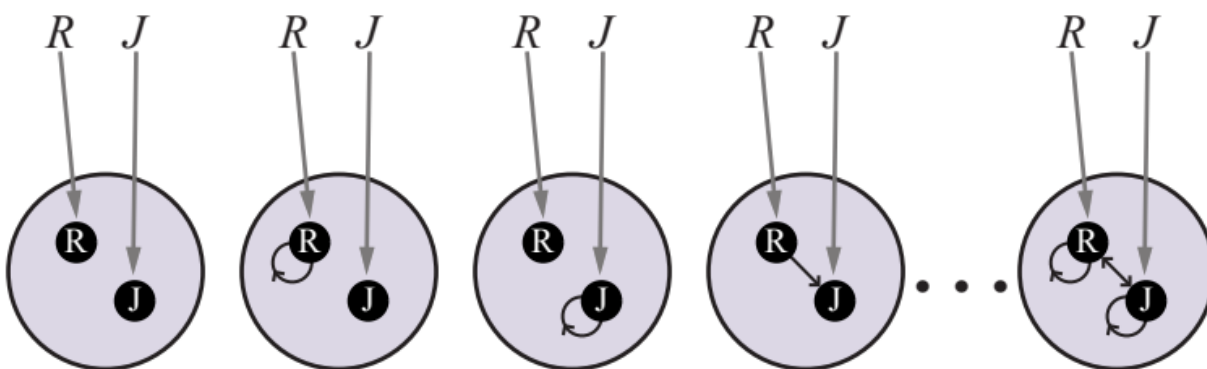
$$\text{Brother}(\text{John}, \text{Richard}) \wedge \text{Brother}(\text{Geoffrey}, \text{Richard})$$

(8.3) nhưng điều đó sẽ không nắm bắt hoàn toàn tình hình. Đầu tiên, khẳng định này là đúng trong một mô hình nơi Richard chỉ có một anh em—chúng ta cần thêm $\text{John} \neq \text{Geoffrey}$. Thứ hai, câu này không loại trừ các mô hình trong đó Richard có nhiều anh em hơn ngoài John và Geoffrey. Do đó, bản dịch chính xác của “Các anh em của Richard là John và Geoffrey” như sau:

$$\begin{aligned} &\text{Brother}(\text{John}, \text{Richard}) \wedge \text{Brother}(\text{Geoffrey}, \text{Richard}) \wedge \text{John} \\ &\neq \text{Geoffrey} \wedge \forall x \text{Brother}(x, \text{Richard}) \Rightarrow (x = \text{John} \vee x = \text{Geoffrey}) \end{aligned}$$

Câu logic này dường như cồng kềnh hơn nhiều so với câu tiếng Anh tương ứng. Nhưng nếu chúng ta không dịch tiếng Anh đúng cách, hệ thống suy luận logic của chúng ta sẽ mắc lỗi. Chúng ta có thể nghĩ ra một ngữ nghĩa cho phép một câu logic đơn giản hơn không?

Một đề xuất rất phổ biến trong các hệ thống cơ sở dữ liệu hoạt động như sau. Đầu tiên, chúng ta khẳng định rằng mọi ký hiệu hằng đều đề cập đến một đối tượng riêng biệt—**giả định tên duy nhất**. Thứ hai, chúng ta giả định rằng các câu nguyên tử không được biết là đúng thực sự là sai—**giả định thế giới đóng**. Cuối cùng, chúng ta gọi **đóng miền**, có nghĩa là mỗi mô hình không chứa nhiều phần tử miền hơn những phần tử được đặt tên bởi các ký hiệu hằng.



Hình 8.5 Một số thành viên của tập hợp tất cả các mô hình cho một ngôn ngữ có hai ký hiệu hằng, R và J, và một ký hiệu quan hệ hai ngôi, theo ngữ nghĩa cơ sở dữ liệu. Sự diễn giải của các ký hiệu hằng được cố định, và có một đối tượng riêng biệt cho mỗi ký hiệu hằng.

Theo ngữ nghĩa kết quả, Phương trình (8.3) thực sự nói rằng Richard có chính xác hai anh em, John và Geoffrey. Chúng ta gọi đây là **ngữ nghĩa cơ sở dữ liệu** để phân biệt nó với ngữ nghĩa chuẩn của logic bậc nhất. Ngữ nghĩa cơ sở dữ liệu cũng được sử dụng trong các hệ thống lập trình logic, như được giải thích trong Mục 9.4.4.

Thật hữu ích khi xem xét tập hợp tất cả các mô hình khả dĩ theo ngữ nghĩa cơ sở dữ liệu cho trường hợp tương tự như trong Hình 8.4 (trang 277). Hình 8.5 cho thấy một số mô hình, từ mô hình không có tuple nào thỏa mãn quan hệ đến mô hình với tất cả các tuple thỏa mãn quan hệ. Với hai đối tượng, có bốn tuple hai phần tử khả dĩ, vì vậy có $2^4 = 16$ tập con tuple khác nhau có thể thỏa mãn quan hệ. Do đó, có tất cả 16 mô hình khả dĩ—ít hơn nhiều so với vô số mô hình cho ngữ nghĩa bậc nhất chuẩn. Mặt khác, ngữ nghĩa cơ sở dữ liệu yêu cầu kiến thức xác định về những gì thế giới chứa.

Ví dụ này đưa ra một điểm quan trọng: không có một ngữ nghĩa “đúng” duy nhất cho logic. Tính hữu ích của bất kỳ ngữ nghĩa nào được đề xuất phụ thuộc vào việc nó làm cho việc diễn đạt các loại tri thức mà chúng ta muốn viết xuống trở nên ngắn gọn và trực quan như thế nào, và việc phát triển các quy tắc suy luận tương ứng dễ dàng và tự nhiên như thế nào. Ngữ nghĩa cơ sở dữ liệu hữu ích nhất khi chúng ta chắc chắn về danh tính của tất cả các đối tượng được mô tả trong cơ sở tri thức và khi chúng ta có tất cả các sự kiện trong tay; trong các trường hợp khác, nó khá khó xử. Đối với phần còn lại của chương này, chúng ta giả định ngữ nghĩa chuẩn trong khi lưu ý các trường hợp mà lựa chọn này dẫn đến các biểu thức công kênh.

8.3 Sử dụng Logic Bậc nhất

Sau khi đã định nghĩa một ngôn ngữ logic biểu cảm, chúng ta hãy học cách sử dụng nó. Trong mục này, chúng ta sẽ cung cấp các câu ví dụ trong một số miền đơn giản. Trong biểu diễn tri thức, một **miền** (domain) chỉ là một phần nào đó của thế giới mà chúng ta muốn biểu diễn một số tri thức về nó. Chúng ta bắt đầu với một mô tả ngắn gọn về giao diện TELL/ASK cho các cơ sở tri thức logic bậc nhất. Sau đó, chúng ta sẽ xem xét các miền về quan hệ họ hàng, số, tập hợp, danh sách và thế giới wumpus. Mục 8.4.2 chứa một ví dụ thực chất hơn (mạch điện tử) và Chương 10 bao quát mọi thứ trong vũ trụ.

8.3.1 Các khẳng định và truy vấn trong logic bậc nhất

Các câu được thêm vào một cơ sở tri thức bằng cách sử dụng TELL, giống hệt như trong logic mệnh đề. Những câu như vậy được gọi là các **khẳng định** (assertions). Ví dụ, chúng ta có thể khẳng định rằng John là vua, Richard là người, và tất cả các vị vua đều là người:

TELL(KB, King(John)).

TELL(KB, Person(Richard)).

TELL(KB, $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$).

Chúng ta có thể đặt câu hỏi cho cơ sở tri thức bằng cách sử dụng ASK. Ví dụ,

ASK(KB, King(John))

sẽ trả về true. Các câu hỏi được đặt với ASK được gọi là các **truy vấn** (queries) hoặc **mục tiêu** (goals). Nói chung, bất kỳ truy vấn nào được suy ra một cách logic từ cơ sở tri thức đều nên được trả lời là khẳng định. Ví dụ, với ba khẳng định trên, truy vấn

ASK(KB, Person(John))

cũng nên trả về true.

Chúng ta có thể hỏi các truy vấn có lượng từ, chẳng hạn như

ASK(KB, $\exists x$ Person(x)).

Câu trả lời là true, nhưng điều này có lẽ không hữu ích như chúng ta mong muốn. Nó giống như trả lời câu hỏi “Bạn có thể cho tôi biết mấy giờ rồi không?” bằng câu “Có”. Nếu chúng ta muốn biết giá trị nào của x làm cho câu đó đúng, chúng ta sẽ cần một hàm khác, mà chúng ta gọi là ASKVARs,

ASKVARs(KB, Person(x)) và nó sẽ trả về một luồng các câu trả lời. Trong trường hợp này sẽ có hai câu trả lời:

{x/John} và {x/Richard}. Một câu trả lời như vậy được gọi là một **phép thế** (substitution) hoặc **danh sách ràng buộc** (binding list).

ASKVARs thường được dành riêng cho các cơ sở tri thức chỉ bao gồm các mệnh đề Horn, bởi vì trong các cơ sở tri thức như vậy, mọi cách làm cho truy vấn đúng sẽ ràng buộc các biến với các giá trị cụ thể. Điều đó không đúng với logic bậc nhất; trong một KB chỉ được cho biết King(John) $\vee$ King(Richard), không có một ràng buộc duy nhất nào cho x làm cho truy vấn $\exists x$ King(x) đúng, mặc dù truy vấn đó thực sự là đúng.

8.3.2 Miền quan hệ họ hàng

Ví dụ đầu tiên chúng ta xem xét là miền các mối quan hệ gia đình, hay **họ hàng** (kinship). Miền này bao gồm các sự kiện như “Elizabeth là mẹ của Charles” và “Charles là cha của William” và các quy tắc như “Bà của một người là mẹ của cha hoặc mẹ của người đó”.

Rõ ràng, các đối tượng trong miền của chúng ta là người. Các vị từ một ngôi bao gồm Male (Nam) và Female (Nữ), cùng những vị từ khác. Các quan hệ họ hàng—quan hệ cha mẹ, anh chị em, hôn nhân, v.v.—được biểu diễn bằng các vị từ hai ngôi:

Parent (ChaMẹ), Sibling (AnhChịEm), Brother (AnhEmTrai), Sister (ChịEmGái), Child (Con), Daughter (ConGái), Son (ConTrai), Spouse (VợChồng), Wife (Vợ), Husband (Chồng), Grandparent (ÔngBà), Grandchild (Cháu), Cousin (AnhChịEmHọ), Aunt (CôDì), và Uncle (ChúCậu). Chúng ta sử dụng các hàm cho Mother (Mẹ) và Father (Cha), bởi vì mỗi người có chính xác một người mẹ và một người cha về mặt sinh học (mặc dù chúng ta có thể giới thiệu thêm các hàm cho mẹ nuôi, mẹ mang thai hộ, v.v.).

Chúng ta có thể đi qua từng hàm và vị từ, viết ra những gì chúng ta biết dưới dạng các ký hiệu khác. Ví dụ, mẹ của một người là cha mẹ của người đó và là phụ nữ:

$\forall m, c$ Mother(c)=m $\Leftrightarrow$ Female(m) $\wedge$ Parent(m, c).

Chồng của một người là người phối ngẫu nam của người đó:

$\forall w, h$ Husband(h,w) $\Leftrightarrow$ Male(h) $\wedge$ Spouse(h,w).

Quan hệ cha mẹ và con cái là nghịch đảo của nhau:

$$\forall p, c \text{ Parent}(p, c) \Leftrightarrow \text{Child}(c, p).$$

Ông bà là cha mẹ của cha mẹ mình:

$$\forall g, c \text{ Grandparent}(g, c) \Leftrightarrow \exists p \text{ Parent}(g, p) \wedge \text{Parent}(p, c).$$

Anh chị em là một người con khác của cha mẹ mình:

$$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow x \neq y \wedge \exists p \text{ Parent}(p, x) \wedge \text{Parent}(p, y).$$

Mỗi câu này có thể được xem như một **tiên đề** (axiom) của miền quan hệ họ hàng. Các tiên đề thường được liên kết với các miền toán học thuần túy—chúng ta sẽ sớm thấy một số tiên đề cho số—nhưng chúng cần thiết trong mọi miền. Chúng cung cấp thông tin thực tế cơ bản để từ đó có thể suy ra các kết luận hữu ích. Các tiên đề về họ hàng của chúng ta cũng là các **định nghĩa** (definitions); chúng có dạng $\forall x, y P(x, y) \Leftrightarrow \dots$

Không phải tất cả các câu logic về một miền đều là tiên đề. Một số là **định lý** (theorems)—tức là, chúng được suy ra từ các tiên đề. Ví dụ, hãy xem xét khẳng định rằng quan hệ anh chị em là đối xứng:

$$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow \text{Sibling}(y, x).$$

Đây là một tiên đề hay một định lý? Thực tế, nó là một định lý suy ra một cách logic từ tiên đề định nghĩa quan hệ anh chị em.

Không phải tất cả các tiên đề đều là định nghĩa. Một số cung cấp thông tin tổng quát hơn về các vị từ nhất định mà không cấu thành một định nghĩa. Ví dụ, không có cách hoàn chỉnh rõ ràng nào để hoàn thành câu:

$$\forall x \text{ Person}(x) \Leftrightarrow \dots$$

Thay vào đó, chúng ta có thể viết các đặc tả một phần về các thuộc tính mà mọi người đều có và các thuộc tính làm cho một thứ gì đó trở thành người.

Các tiên đề cũng có thể là “các sự thật đơn thuần”, chẳng hạn như $\text{Male}(\text{Jim})$ và $\text{Spouse}(\text{Jim}, \text{Laura})$. Những sự thật như vậy tạo thành mô tả về các trường hợp vấn đề cụ thể, cho phép trả lời các câu hỏi cụ thể.

8.3.3 Số, tập hợp, và danh sách

Số có lẽ là ví dụ sinh động nhất về cách một lý thuyết lớn có thể được xây dựng từ một hạt nhân tiên đề nhỏ bé. Chúng ta mô tả ở đây lý thuyết về **số tự nhiên** hay số nguyên không âm. Chúng ta cần một vị từ NatNum sẽ đúng với các số tự nhiên; một ký hiệu hằng, 0; và một ký hiệu hàm, S (hàm kế vị).

Các tiên đề Peano định nghĩa số tự nhiên và phép cộng. Các số tự nhiên được định nghĩa đệ quy:

$$\text{NatNum}(0).$$

$$\forall n \text{ NatNum}(n) \Rightarrow \text{NatNum}(S(n)).$$

Nghĩa là, 0 là một số tự nhiên, và với mọi đối tượng n , nếu n là số tự nhiên, thì $S(n)$ là số tự nhiên. Chúng ta cũng cần các tiên đề để ràng buộc hàm kế vị:

$$\forall n \ 0 \neq S(n).$$

$$\forall m, n \ m \neq n \Rightarrow S(m) \neq S(n).$$

Bây giờ chúng ta có thể định nghĩa phép cộng theo hàm kế vị:

$$\forall m \text{ NatNum}(m) \Rightarrow + (0, m) = m.$$

$$\forall m, n \text{ NatNum}(m) \wedge \text{NatNum}(n) \Rightarrow + (S(m), n) = S(+ (m, n)).$$

Việc sử dụng ký hiệu **trung tố** (infix), ví dụ viết $m+0$ thay vì $+(m,0)$ (ký hiệu **tiền tố** - prefix), là một ví dụ về **cú pháp rút gọn** (syntactic sugar), tức là một sự mở rộng hoặc viết tắt của cú pháp chuẩn mà không làm thay đổi ngữ nghĩa.

Miền **tập hợp** (set) cũng là nền tảng cho toán học cũng như cho lập luận thông thường. Chúng ta sẽ sử dụng từ vựng thông thường của lý thuyết tập hợp làm cú pháp rút gọn. Tập hợp rỗng là một hằng số được viết là $\{\}$. Có một vị từ một ngôi, Set , đúng với các tập hợp. Các vị từ hai ngôi là $x \in s$ (x là một phần tử của tập hợp s) và $s_1 \subseteq s_2$ (tập hợp s_1 là tập hợp con của s_2). Các hàm hai ngôi là $s_1 \cap s_2$ (phép giao), $s_1 \cup s_2$ (phép hợp), và $\text{Add}(x, s)$ (tập hợp thu được khi thêm phần tử x vào tập hợp s). Một bộ tiên đề khả dĩ như sau:

- Các tập hợp duy nhất là tập hợp rỗng và những tập hợp được tạo ra bằng cách thêm một cái gì đó vào một tập hợp:
 - $\forall s \text{ Set}(s) \Leftrightarrow (s = \{\}) \vee (\exists x, s_2 \text{ Set}(s_2) \wedge s = \text{Add}(x, s_2)).$
- Tập hợp rỗng không có phần tử nào được thêm vào:
 - $\neg \exists x, s \text{ Add}(x, s) = \{\}.$
- Thêm một phần tử đã có trong tập hợp không có tác dụng:
 - $\forall x, s \ x \in s \Leftrightarrow s = \text{Add}(x, s).$
- Các phần tử duy nhất của một tập hợp là các phần tử đã được thêm vào nó:
 - $\forall x, s \ x \in s \Leftrightarrow \exists y, s_2 (s = \text{Add}(y, s_2) \wedge (x = y \vee x \in s_2)).$
- Một tập hợp là tập con của tập hợp khác khi và chỉ khi tất cả các phần tử của tập hợp đầu tiên đều là phần tử của tập hợp thứ hai:
 - $\forall s_1, s_2 \ s_1 \subseteq s_2 \Leftrightarrow (\forall x \ x \in s_1 \Rightarrow x \in s_2).$
- Hai tập hợp bằng nhau khi và chỉ khi mỗi tập là tập con của tập kia:
 - $\forall s_1, s_2 (s_1 = s_2) \Leftrightarrow (s_1 \subseteq s_2 \wedge s_2 \subseteq s_1).$
- Một đối tượng nằm trong giao của hai tập hợp khi và chỉ khi nó là phần tử của cả hai tập hợp:
 - $\forall x, s_1, s_2 \ x \in (s_1 \cap s_2) \Leftrightarrow (x \in s_1 \wedge x \in s_2).$
- Một đối tượng nằm trong hợp của hai tập hợp khi và chỉ khi nó là phần tử của một trong hai tập hợp:
 - $\forall x, s_1, s_2 \ x \in (s_1 \cup s_2) \Leftrightarrow (x \in s_1 \vee x \in s_2).$

Danh sách (list) tương tự như tập hợp. Điểm khác biệt là danh sách có thứ tự và cùng một phần tử có thể xuất hiện nhiều lần trong danh sách. Chúng ta có thể sử dụng từ vựng của Lisp cho danh sách:

Nil là danh sách hằng không có phần tử;

Cons, Append, First, và Rest là các hàm; và Find là vị từ thực hiện chức năng tương tự Member cho tập hợp.

8.3.4 Thế giới wumpus

Các tiên đề logic bậc nhất trong mục này ngắn gọn hơn nhiều, nắm bắt một cách tự nhiên chính xác những gì chúng ta muốn nói.

Tác tử wumpus nhận một vector tri giác (percept) với năm phần tử. Câu logic bậc nhất tương ứng được lưu trong cơ sở tri thức phải bao gồm cả tri giác và thời điểm nó xảy ra. Chúng ta dùng số nguyên cho các bước thời gian. Một câu tri giác điển hình sẽ là:

Percept([Stench,Breeze,Glitter,None,None],5).

Ở đây, *Percept* là một vị từ nhị phân, và *Stench* cùng các hằng số khác được đặt trong một danh sách. Các hành động trong thế giới wumpus có thể được biểu diễn bằng các biểu thức logic:

Turn(Right), Turn(Left), Forward, Shoot, Grab, Climb.

Để xác định hành động nào là tốt nhất, chương trình tác nhân thực hiện truy vấn

AskVars(KB, BestAction(a,5)),

trả về một danh sách ràng buộc như {a/Grab}. Chương trình tác nhân sau đó có thể trả về *Grab* như hành động cần thực hiện. Dữ liệu tri giác thô ngụ ý một số sự thật về trạng thái hiện tại. Ví dụ:

$$\forall t, s, g, w, c \text{ Percept}([s, \text{Breeze}, g, w, c], t) \Rightarrow \text{Breeze}(t)$$

$$\forall t, s, g, w, c \text{ Percept}([s, \text{None}, g, w, c], t) \Rightarrow \neg \text{Breeze}(t)$$

$$\forall t, s, b, w, c \text{ Percept}([s, b, \text{Glitter}, w, c], t) \Rightarrow \text{Glitter}(t)$$

$$\forall t, s, b, w, c \text{ Percept}([s, b, \text{None}, w, c], t) \Rightarrow \neg \text{Glitter}(t)$$

và tiếp tục như vậy. Các quy tắc này thể hiện một dạng đơn giản của quá trình suy luận gọi là **tri giác**, mà chúng ta sẽ nghiên cứu sâu hơn trong Chương 27. Lưu ý phép lượng hóa theo thời gian t . Trong logic mệnh đề, chúng ta sẽ cần các bản sao của mỗi câu cho từng bước thời gian.

Hành vi “phản xạ” đơn giản cũng có thể được triển khai bằng các câu suy diễn lượng hóa. Ví dụ, chúng ta có

$$\forall t \text{ Glitter}(t) \Rightarrow \text{BestAction}(\text{Grab}, t).$$

Bây giờ là lúc biểu diễn môi trường. Các đối tượng rõ ràng là ô vuông, hổ, và wumpus. Tốt hơn là sử dụng một thuật ngữ phức tạp trong đó hàng và cột xuất hiện dưới dạng số nguyên; ví dụ, chúng ta có thể sử dụng thuật ngữ danh sách [1,2]. Sự kề nhau của hai ô vuông bất kỳ có thể được định nghĩa là:

$$\forall x, y, a, b \text{ Adjacent}([x, y], [a, b]) \Leftrightarrow (x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1)).$$

Đơn giản hơn là sử dụng một vị từ một ngôi Pit đúng với các ô vuông chứa hổ. Vị trí của tác tử thay đổi theo thời gian, vì vậy chúng ta viết $At(\text{Agent}, s, t)$ để có nghĩa là tác tử đang ở ô s tại thời điểm t .

Với vị trí hiện tại, tác tử có thể suy ra các thuộc tính của ô vuông từ các thuộc tính của tri giác hiện tại. Ví dụ:

$$\forall s, t \text{ At}(\text{Agent}, s, t) \wedge \text{Breeze}(t) \Rightarrow \text{Breezy}(s).$$

Biết nơi nào có gió (hoặc có mùi) và không có gió (hoặc không có mùi), tác tử có thể suy ra vị trí của các hổ. Trong khi logic mệnh đề đòi hỏi một tiên đề riêng cho mỗi ô, logic bậc nhất chỉ cần một tiên đề:

$$\forall s \text{ Breezy}(s) \Leftrightarrow \exists r \text{ Adjacent}(r, s) \wedge \text{Pit}(r).$$

Tương tự, trong logic bậc nhất, chúng ta có thể lượng từ hóa theo thời gian, vì vậy chúng ta chỉ cần một tiên đề trạng thái kế vị cho mỗi vị từ, thay vì một bản sao khác nhau cho mỗi bước thời gian. Ví dụ, tiên đề cho mũi tên trở thành:

$$\forall t \text{ HaveArrow}(t + 1) \Leftrightarrow (\text{HaveArrow}(t) \wedge \neg \text{Action}(\text{Shoot}, t)).$$

Từ hai câu ví dụ này, chúng ta có thể thấy rằng công thức logic bậc nhất không kém phần cô đọng so với mô tả bằng ngôn ngữ tiếng Anh ban đầu.

8.4 Kỹ thuật Tri thức trong Logic Bậc nhất

Phần trước đã minh họa việc sử dụng logic bậc nhất để biểu diễn tri thức trong ba miền đơn giản. Phần này mô tả quy trình chung để xây dựng cơ sở tri thức - một quy trình được gọi là

kỹ thuật tri thức (knowledge engineering). Một **kỹ sư tri thức** (knowledge engineer) là người điều tra một miền cụ thể, tìm hiểu những khái niệm quan trọng trong miền đó và tạo ra một biểu diễn hình thức của các đối tượng và quan hệ trong miền. Chúng ta sẽ minh họa quy trình kỹ thuật tri thức trong một miền mạch điện tử. Cách tiếp cận này phù hợp để phát triển các cơ sở tri thức chuyên dụng có miền được giới hạn cẩn thận và phạm vi các truy vấn được biết trước. Các cơ sở tri thức tổng quát, bao gồm một phạm vi rộng lớn kiến thức của con người và nhằm hỗ trợ các nhiệm vụ như hiểu ngôn ngữ tự nhiên, được thảo luận trong Chương 10.

8.4.1 Quy trình kỹ thuật tri thức

Các dự án kỹ thuật tri thức rất đa dạng về nội dung, phạm vi và độ khó, nhưng tất cả các dự án như vậy đều bao gồm các bước sau:

1. **Xác định các câu hỏi.** Kỹ sư tri thức phải xác định rõ phạm vi các câu hỏi mà cơ sở tri thức sẽ hỗ trợ và các loại sự thật sẽ có sẵn cho mỗi trường hợp bài toán cụ thể. Ví dụ, cơ sở tri thức wumpus có cần phải có khả năng chọn hành động hay chỉ cần trả lời các câu hỏi về nội dung của môi trường? Các sự thật cảm biến có bao gồm vị trí hiện tại không? Nhiệm vụ sẽ xác định kiến thức nào phải được biểu diễn để kết nối các trường hợp bài toán với các câu trả lời. Bước này tương tự như quy trình PEAS để thiết kế các tác nhân trong Chương 2.
2. **Thu thập kiến thức liên quan.** Kỹ sư tri thức có thể đã là một chuyên gia trong miền đó, hoặc có thể cần phải làm việc với các chuyên gia thực sự để trích xuất những gì họ biết - một quy trình được gọi là **thu thập tri thức** (knowledge acquisition). Ở giai đoạn này, kiến thức không được biểu diễn một cách hình thức. Ý tưởng là để hiểu phạm vi của cơ sở tri thức, được xác định bởi nhiệm vụ, và để hiểu cách miền đó thực sự hoạt động. Đối với thế giới wumpus, được xác định bởi một tập hợp các quy tắc nhân tạo, kiến thức liên quan rất dễ xác định. (Tuy nhiên, hãy lưu ý rằng định nghĩa về tính liên kết không được cung cấp rõ ràng trong các quy tắc của thế giới wumpus.) Đối với các miền thực tế, vấn đề về sự liên quan có thể khá khó khăn - ví dụ, một hệ thống để mô phỏng các thiết kế VLSI có thể cần hoặc không cần tính đến các điện dung ký sinh và hiệu ứng bề mặt.
3. **Quyết định từ vựng của các vị từ (predicates), hàm (functions) và hằng số (constants).** Tức là, dịch các khái niệm cấp miền quan trọng thành các tên cấp logic. Điều này liên quan đến nhiều câu hỏi về phong cách kỹ thuật tri thức. Giống như phong cách lập trình, điều này có thể có tác động đáng kể đến sự thành công cuối cùng của dự án. Ví dụ, các hồ có nên được biểu diễn bằng các đối tượng hay bằng một vị từ một ngôi trên các ô vuông? Hướng của tác nhân có nên là một hàm hay một vị từ? Vị trí của wumpus có phụ thuộc vào thời gian không? Một khi các lựa chọn đã được đưa ra, kết quả là một từ vựng được gọi là **bản thể luận** (ontology) của miền đó. Từ bản thể luận có nghĩa là một lý thuyết cụ thể về bản chất của sự tồn tại. Bản thể luận xác định những loại sự vật tồn tại, nhưng không xác định các thuộc tính cụ thể và mối quan hệ qua lại của chúng.
4. **Mã hóa kiến thức chung về miền.** Kỹ sư tri thức viết ra các tiên đề cho tất cả các thuật ngữ trong từ vựng. Điều này xác định (trong chừng mực có thể) ý nghĩa của các thuật ngữ, cho phép chuyên gia kiểm tra nội dung. Thông thường, bước này bộc lộ những quan niệm sai lầm hoặc khoảng trống trong từ vựng phải được khắc phục bằng cách quay lại bước 3 và lặp lại quy trình.
5. **Mã hóa mô tả về trường hợp bài toán.** Nếu bản thể luận được suy nghĩ kỹ, bước này sẽ dễ dàng. Nó liên quan đến việc viết các câu nguyên tử đơn giản về các trường hợp của các khái niệm đã là một phần của bản thể luận. Đối với một tác nhân logic, các trường hợp bài toán được cung cấp bởi các cảm biến, trong khi một cơ sở tri thức “phi thực thể” được cung cấp các câu theo cùng một cách mà các chương trình truyền thống được cung cấp dữ liệu đầu vào.
6. **Đưa ra các truy vấn cho thủ tục suy luận và nhận câu trả lời.** Đây là nơi có phần thưởng: chúng ta có thể để thủ tục suy luận hoạt động trên các tiên đề và các sự thật cụ thể của bài toán để suy ra các sự thật mà chúng ta quan tâm. Do đó, chúng ta tránh được nhu cầu viết một thuật toán giải pháp cụ thể cho ứng dụng.
7. **Gỡ lỗi và đánh giá cơ sở tri thức.** Than ôi, các câu trả lời cho các truy vấn hiếm khi đúng ngay từ lần thử đầu tiên. Chính xác hơn, các câu trả lời sẽ đúng với cơ sở tri thức như đã viết, giả sử rằng thủ tục suy luận là đúng đắn, nhưng chúng sẽ không phải là những câu mà người dùng mong đợi. Ví dụ, nếu một tiên đề bị thiếu, một số truy vấn sẽ không thể trả lời

được từ cơ sở tri thức. Một quy trình gỡ lỗi đáng kể có thể xảy ra. Các tiên đề bị thiếu hoặc các tiên đề quá yếu có thể dễ dàng được xác định bằng cách nhận thấy những nơi mà chuỗi suy luận dừng lại một cách bất ngờ. Ví dụ, nếu cơ sở tri thức bao gồm một quy tắc chẩn đoán (xem Bài tập 8.WUMD) để tìm wumpus,

$$\forall s \text{Smelly}(s) \Rightarrow \text{Adjacent}(\text{Home}(\text{Wumpus}), s)$$

thay vì câu điều kiện hai chiều, thì tác nhân sẽ không bao giờ có thể chứng minh sự vắng mặt của wumpus. Các tiên đề không chính xác có thể được xác định vì chúng là các câu sai về thế giới. Ví dụ, câu

$$\forall x \text{NumOfLegs}(x, 4) \Rightarrow \text{Mammal}(x)$$

là sai đối với bò sát, động vật lưỡng cư và bọ. Sự sai lầm của câu này có thể được xác định độc lập với phần còn lại của cơ sở tri thức. Ngược lại, một lỗi điển hình trong một chương trình trông như thế này:

offset = position + 1.

Không thể biết liệu offset nên là position hay position + 1 mà không hiểu ngữ cảnh xung quanh.

Khi bạn đạt đến điểm mà không có lỗi rõ ràng trong cơ sở tri thức của mình, thật hấp dẫn khi tuyên bố thành công. Nhưng trừ khi rõ ràng là không có lỗi, tốt hơn hết là nên đánh giá chính thức hệ thống của bạn bằng cách chạy nó trên một bộ truy vấn kiểm tra và đo lường xem bạn trả lời đúng bao nhiêu. Nếu không có phép đo lường khách quan, quá dễ để tự thuyết phục mình rằng công việc đã hoàn thành. Để hiểu rõ hơn quy trình bảy bước này, bây giờ chúng ta sẽ áp dụng nó vào một ví dụ mở rộng - miền mạch điện tử.

8.4.2 Miền mạch điện tử

Chúng ta sẽ phát triển một bản thể luận và cơ sở tri thức cho phép chúng ta suy luận về các mạch kỹ thuật số như được hiển thị trong Hình 8.6. Chúng ta tuân theo quy trình bảy bước cho kỹ thuật tri thức.

Xác định các câu hỏi

Có nhiều nhiệm vụ suy luận liên quan đến các mạch kỹ thuật số. Ở mức cao nhất, người ta phân tích chức năng của mạch. Ví dụ, mạch trong Hình 8.6 có thực sự cộng đúng cách không? Nếu tất cả các đầu vào đều cao, đầu ra của cổng A2 là gì? Các câu hỏi về cấu trúc của mạch cũng thú vị. Ví dụ, tất cả các cổng được kết nối với thiết bị đầu cuối đầu vào đầu tiên là gì? Mạch có chứa các vòng lặp phản hồi không? Đây sẽ là các nhiệm vụ của chúng ta trong phần này. Có các mức phân tích chi tiết hơn, bao gồm những mức liên quan đến độ trễ thời gian, diện tích mạch, tiêu thụ điện năng, chi phí sản xuất, v.v. Mỗi mức này sẽ yêu cầu kiến thức bổ sung.

Thu thập kiến thức liên quan

Chúng ta biết gì về các mạch kỹ thuật số? Đối với mục đích của chúng ta, chúng được cấu tạo bởi các dây dẫn và cổng. Các tín hiệu chạy dọc theo dây dẫn đến các thiết bị đầu cuối đầu vào của các cổng, và mỗi cổng tạo ra một tín hiệu trên thiết bị đầu cuối đầu ra chạy dọc theo một dây dẫn khác.

Để xác định các tín hiệu này sẽ là gì, chúng ta cần biết các cổng biến đổi tín hiệu đầu vào của chúng như thế nào. Có bốn loại cổng: cổng AND, OR, và XOR có hai thiết bị đầu cuối đầu vào, và cổng NOT có một. Tất cả các cổng đều có một thiết bị đầu cuối đầu ra. Các mạch, giống như các cổng, có các thiết bị đầu cuối đầu vào và đầu ra.

Để suy luận về chức năng và kết nối, chúng ta không cần nói về bản thân các dây dẫn, đường đi của chúng hay các điểm nối nơi chúng gặp nhau. Tất cả những gì quan trọng là các kết nối giữa các thiết bị đầu cuối - chúng ta có thể nói rằng một thiết bị đầu cuối đầu ra được kết nối với một thiết bị đầu cuối đầu vào khác mà không cần phải nói điều gì thực sự kết nối chúng. Các yếu tố khác như kích thước, hình dạng, màu sắc hoặc chi phí của các thành phần khác nhau không liên quan đến phân tích của chúng ta.

Nếu mục đích của chúng ta là thứ gì đó khác ngoài việc xác minh các thiết kế ở cấp độ cổng, bản thể luận sẽ khác. Ví dụ, nếu chúng ta quan tâm đến việc gỡ lỗi các mạch bị lỗi, thì có lẽ sẽ là một ý tưởng hay nếu bao gồm các dây dẫn trong bản thể luận, bởi vì một dây dẫn bị lỗi có thể làm hỏng tín hiệu chạy dọc theo nó. Để giải quyết các lỗi về thời gian, chúng ta sẽ cần bao gồm độ trễ của cổng. Nếu chúng ta quan tâm đến việc thiết kế một sản phẩm có lợi nhuận, thì chi phí của mạch và tốc độ của nó so với các sản phẩm khác trên thị trường sẽ rất quan trọng.

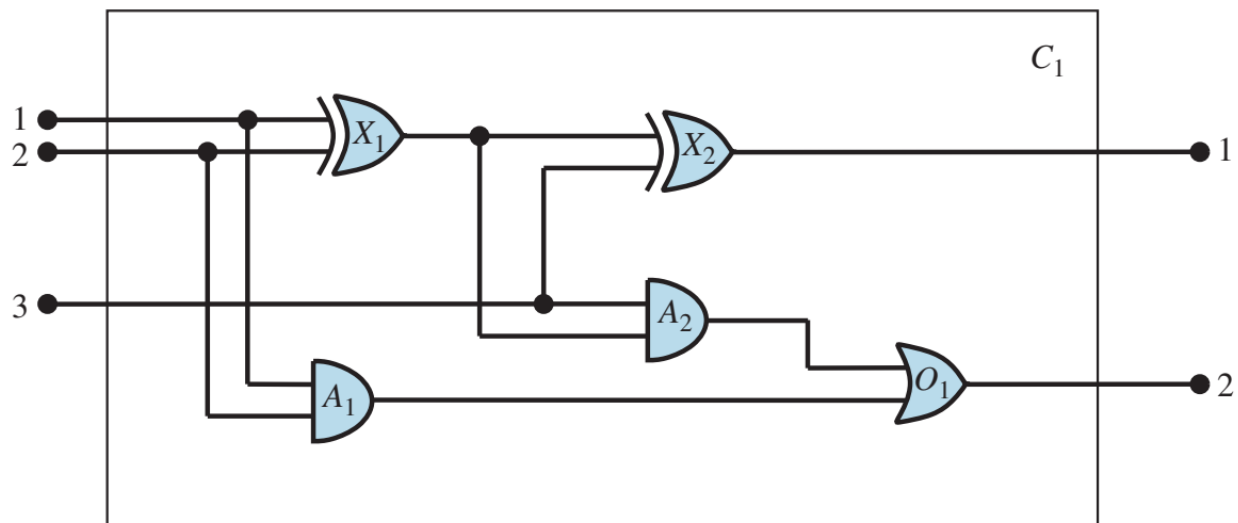
Quyết định một từ vựng

Bây giờ chúng ta biết rằng chúng ta muốn nói về các mạch, thiết bị đầu cuối, tín hiệu và cổng. Bước tiếp theo là chọn các hàm, vị từ và hằng số để biểu diễn chúng.

Đầu tiên, chúng ta cần có khả năng phân biệt các cổng với nhau và với các đối tượng khác. Mỗi cổng được biểu diễn như một đối tượng được đặt tên bởi một hằng số, mà chúng ta khẳng định rằng nó là một cổng với, chẳng hạn, $\text{Gate}(X1)$. Hành vi của mỗi cổng được xác định bởi loại của nó: một trong các hằng số AND, OR, XOR, hoặc NOT. Vì một cổng có chính xác một loại, một hàm là phù hợp:

$\text{Type}(X1) = \text{XOR}$. Các mạch, giống như các cổng, được xác định bởi một vị từ: $\text{Circuit}(C1)$.

Tiếp theo chúng ta xem xét các thiết bị đầu cuối, được xác định bởi vị từ $\text{Terminal}(x)$. Một mạch có thể có một hoặc nhiều thiết bị đầu cuối đầu vào và một hoặc nhiều thiết bị đầu cuối đầu ra. Chúng ta sử dụng hàm $\text{In}(1, X1)$ để chỉ thiết bị đầu cuối đầu vào đầu tiên cho mạch $X1$. Một hàm tương tự $\text{Out}(n, c)$ được sử dụng cho các thiết bị đầu cuối đầu ra. Vị từ $\text{Arity}(c, i, j)$ cho biết mạch c có i thiết bị đầu cuối đầu vào và j thiết bị đầu cuối đầu ra. Kết nối giữa các cổng có thể được biểu diễn bằng một vị từ, Connected , nhận hai thiết bị đầu cuối làm đối số, như trong $\text{Connected}(\text{Out}(1, X1), \text{In}(1, X2))$.



Hình 8.6 Một mạch kỹ thuật số C_1 , được cho là một bộ cộng đầy đủ một bit. Hai đầu vào đầu tiên là hai bit cần cộng và đầu vào thứ ba là bit nhớ. Đầu ra đầu tiên là tổng và đầu ra thứ hai là bit nhớ cho bộ cộng tiếp theo. Mạch chứa hai cổng XOR, hai cổng AND và một cổng OR.

Cuối cùng, chúng ta cần biết liệu một tín hiệu đang bật hay tắt. Một khả năng là sử dụng một vị từ một ngôi, $On(t)$, đúng khi tín hiệu tại một thiết bị đầu cuối đang bật. Tuy nhiên, điều này khiến cho việc đặt các câu hỏi như “Tất cả các giá trị khả dĩ của các tín hiệu tại các thiết bị đầu cuối đầu ra của mạch C_1 là gì?” trở nên khó khăn. Do đó, chúng ta giới thiệu hai giá trị tín hiệu, 1 và 0, là các đối tượng, tương ứng với “bật” và “tắt”, và một hàm $Signal(t)$ biểu thị giá trị tín hiệu cho thiết bị đầu cuối t .

Mã hóa kiến thức chung của miền

Một dấu hiệu cho thấy chúng ta có một bản thể luận tốt là chúng ta chỉ cần một vài quy tắc chung, có thể được trình bày rõ ràng và súc tích. Đây là tất cả các tiên đề mà chúng ta sẽ cần:

- Nếu hai thiết bị đầu cuối được kết nối, thì chúng có cùng tín hiệu: $\forall t_1, t_2 Terminal(t_1) \wedge Terminal(t_2) \wedge Connected(t_1, t_2) \Rightarrow Signal(t_1) = Signal(t_2)$
- Tín hiệu tại mỗi thiết bị đầu cuối hoặc là 1 hoặc 0:
 - $\forall t Terminal(t) \Rightarrow Signal(t) = 1 \vee Signal(t) = 0$
- Connected có tính giao hoán:
 - $\forall t_1, t_2 Connected(t_1, t_2) \Leftrightarrow Connected(t_2, t_1)$
- Có bốn loại cổng:
 - $\forall g Gate(g) \wedge k = Type(g) \Rightarrow k = AND \vee k = OR \vee k = XOR \vee k = NOT$
- Đầu ra của cổng AND là 0 khi và chỉ khi bất kỳ đầu vào nào của nó là 0:
 - $\forall g Gate(g) \wedge Type(g) = AND \Rightarrow Signal(Out(1, g)) = 0 \Leftrightarrow \exists n Signal(In(n, g)) = 0$
- Đầu ra của cổng OR là 1 khi và chỉ khi bất kỳ đầu vào nào của nó là 1:

- $\forall g \text{Gate}(g) \wedge \text{Type}(g) = OR \Rightarrow \text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \exists n \text{Signal}(\text{In}(n, g)) = 1$
- Đầu ra của cổng XOR là 1 khi và chỉ khi các đầu vào của nó khác nhau:
 - $\forall g \text{Gate}(g) \wedge \text{Type}(g) = XOR \Rightarrow \text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(2, g))$
- Đầu ra của cổng NOT khác với đầu vào của nó:
 - $\forall g \text{Gate}(g) \wedge \text{Type}(g) = NOT \Rightarrow \text{Signal}(\text{Out}(1, g)) \neq \text{Signal}(\text{In}(1, g))$
- Các cổng (ngoại trừ NOT) có hai đầu vào và một đầu ra.
 - $\forall g \text{Gate}(g) \wedge \text{Type}(g) = NOT \Rightarrow \text{Arity}(g, 1, 1) \quad \forall g \text{Gate}(g) \wedge k = \text{Type}(g) \wedge (k = AND \vee k = OR \vee k = XOR) \Rightarrow \text{Arity}(g, 2, 1)$
- Một mạch có các thiết bị đầu cuối, lên đến số lượng đầu vào và đầu ra của nó, và không có gì ngoài số lượng đó:
 - $\forall c, i, j \text{Circuit}(c) \wedge \text{Arity}(c, i, j) \Rightarrow \forall n (n \leq i \Rightarrow \text{Terminal}(\text{In}(n, c))) \wedge (n > i \Rightarrow \text{In}(n, c) = \text{Nothing}) \wedge \forall n (n \leq j \Rightarrow \text{Terminal}(\text{Out}(n, c))) \wedge (n > j \Rightarrow \text{Out}(n, c) = \text{Nothing})$
- Các cổng, thiết bị đầu cuối và tín hiệu đều khác nhau:
 - $\forall g, t, s \text{Gate}(g) \wedge \text{Terminal}(t) \wedge \text{Signal}(s) \Rightarrow g \neq t \wedge g \neq s \wedge t \neq s$
- Các cổng là các mạch:
 - $\forall g \text{Gate}(g) \Rightarrow \text{Circuit}(g)$

Mã hóa trường hợp bài toán cụ thể

Mạch được hiển thị trong Hình 8.6 được mã hóa dưới dạng mạch C1 với mô tả sau. Đầu tiên chúng ta phân loại mạch và các cổng thành phần của nó:

$$\begin{aligned}
 &\text{Circuit}(C_1) \wedge \text{Arity}(C_1, 3, 2) \\
 &\text{Gate}(X_1) \wedge \text{Type}(X_1) = XOR \\
 &\text{Gate}(X_2) \wedge \text{Type}(X_2) = XOR \\
 &\text{Gate}(A_1) \wedge \text{Type}(A_1) = AND \\
 &\text{Gate}(A_2) \wedge \text{Type}(A_2) = AND \\
 &\text{Gate}(O_1) \wedge \text{Type}(O_1) = OR.
 \end{aligned}$$

Sau đó, chúng ta hiển thị các kết nối giữa chúng:

$$\begin{aligned}
 &\text{Connected}(\text{Out}(1, X_1), \text{In}(1, X_2)) && \text{Connected}(\text{In}(1, C_1), \text{In}(1, X_1)) \\
 &\text{Connected}(\text{Out}(1, X_1), \text{In}(2, A_2)) && \text{Connected}(\text{In}(1, C_1), \text{In}(1, A_1)) \\
 &\text{Connected}(\text{Out}(1, A_2), \text{In}(1, O_1)) && \text{Connected}(\text{In}(2, C_1), \text{In}(2, X_1)) \\
 &\text{Connected}(\text{Out}(1, A_1), \text{In}(2, O_1)) && \text{Connected}(\text{In}(2, C_1), \text{In}(2, A_1)) \\
 &\text{Connected}(\text{Out}(1, X_2), \text{Out}(1, C_1)) && \text{Connected}(\text{In}(3, C_1), \text{In}(2, X_2)) \\
 &\text{Connected}(\text{Out}(1, O_1), \text{Out}(2, C_1)) && \text{Connected}(\text{In}(3, C_1), \text{In}(1, A_2)).
 \end{aligned}$$

Đưa ra các truy vấn cho thủ tục suy luận

Những tổ hợp đầu vào nào sẽ khiến đầu ra đầu tiên của C1 (bit tổng) là 0 và đầu ra thứ hai của C1 (bit nhớ) là 1

$$\exists i_1, i_2, i_3 \text{ } Signal(In(1, C_1)) = i_1 \wedge Signal(In(2, C_1)) = i_2 \wedge Signal(In(3, C_1)) = i_3 \\ \wedge Signal(Out(1, C_1)) = 0 \wedge Signal(Out(2, C_1)) = 1.$$

Các câu trả lời là các phép thay thế cho các biến i_1, i_2 , và i_3 sao cho câu lệnh kết quả được suy ra bởi cơ sở tri thức. ASKVARs sẽ cung cấp cho chúng ta ba phép thay thế như vậy:

$$\{i_1/1, i_2/1, i_3/0\} \quad \{i_1/1, i_2/0, i_3/1\} \quad \{i_1/0, i_2/1, i_3/1\}$$

Tập hợp các giá trị khả dĩ của tất cả các thiết bị đầu cuối cho mạch cộng là gì?

$$\exists i_1, i_2, i_3, o_1, o_2 \text{ } Signal(In(1, C_1)) = i_1 \wedge Signal(In(2, C_1)) = i_2 \\ \wedge Signal(In(3, C_1)) = i_3 \wedge Signal(Out(1, C_1)) = o_1 \wedge Signal(Out(2, C_1)) = o_2.$$

Truy vấn cuối cùng này sẽ trả về một bảng đầu vào-đầu ra hoàn chỉnh cho thiết bị, có thể được sử dụng để kiểm tra xem nó có thực sự cộng các đầu vào của nó một cách chính xác hay không. Đây là một ví dụ đơn giản về **xác minh mạch** (circuit verification). Chúng ta cũng có thể sử dụng định nghĩa của mạch để xây dựng các hệ thống kỹ thuật số lớn hơn, mà quy trình xác minh tương tự có thể được thực hiện. (Xem Bài tập 8.ADDR.) Nhiều miền có thể áp dụng cùng một loại phát triển cơ sở tri thức có cấu trúc, trong đó các khái niệm phức tạp hơn được định nghĩa trên nền tảng các khái niệm đơn giản hơn.

Gỡ lỗi cơ sở tri thức

Chúng ta có thể làm nhiều cơ sở tri thức theo nhiều cách khác nhau để xem những loại hành vi sai sót nào xuất hiện. Ví dụ, giả sử chúng ta không đọc Phần 8.2.8 và do đó quên khẳng định rằng $1 \neq 0$. Giả sử chúng ta thấy rằng hệ thống không thể chứng minh bất kỳ đầu ra nào cho mạch, ngoại trừ các trường hợp đầu vào 000 và 110. Chúng ta có thể xác định vấn đề bằng cách yêu cầu đầu ra của mỗi cổng. Ví dụ, chúng ta có thể hỏi $\exists i_1, i_2, o \text{ } Signal(In(1, C_1)) = i_1 \wedge Signal(In(2, C_1)) = i_2 \wedge Signal(Out(1, X_1)) = o$ điều này tiết lộ rằng không có đầu ra nào được biết tại X1 cho các trường hợp đầu vào 10 và 01. Sau đó, chúng ta xem xét tiên đề cho các cổng XOR, được áp dụng cho X1:

$$Signal(Out(1, X_1)) = 1 \Leftrightarrow Signal(In(1, X_1)) \neq Signal(In(2, X_1))$$

Nếu các đầu vào được biết là, chẳng hạn, 1 và 0, thì điều này rút gọn thành $Signal(Out(1, X_1)) = 1 \Leftrightarrow 1 \neq 0$

Bây giờ vấn đề đã rõ ràng: hệ thống không thể suy ra rằng $Signal(Out(1, X_1)) = 1$, vì vậy chúng ta cần nói cho nó biết rằng $1 \neq 0$.

Tóm tắt

Chương này đã giới thiệu về logic bậc nhất, một ngôn ngữ biểu diễn mạnh mẽ hơn nhiều so với logic mệnh đề. Các điểm quan trọng như sau:

- Các ngôn ngữ biểu diễn tri thức nên có tính **khai báo, hợp thành, biểu cảm, độc lập với ngữ cảnh, và không mơ hồ**.
- Các logic khác nhau ở **cam kết bản thể luận** và **cam kết nhận thức luận** của chúng. Trong khi logic mệnh đề chỉ cam kết về sự tồn tại của các sự kiện, logic bậc nhất cam kết về sự tồn tại của các **đối tượng** và **quan hệ**, và nhờ đó có được sức biểu đạt lớn hơn, phù hợp với các miền như thế giới wumpus và mạch điện tử.
- Cả logic mệnh đề và logic bậc nhất đều gặp khó khăn trong việc biểu diễn các mệnh đề **mơ hồ**. Khó khăn này hạn chế khả năng áp dụng của chúng trong các miền đòi hỏi sự phán xét cá nhân, như chính trị hay ẩm thực.
- Cú pháp của logic bậc nhất được xây dựng trên cú pháp của logic mệnh đề. Nó bổ sung các **thuật ngữ** (terms) để biểu diễn các đối tượng, và có các **lượng từ phổ dụng** và **lượng từ tồn tại** để xây dựng các khẳng định về tất cả hoặc một số giá trị khả dĩ của các biến được lượng từ hóa.
- Một thế giới khả dĩ, hay **mô hình**, cho logic bậc nhất bao gồm một tập hợp các đối tượng và một **diễn giải** (interpretation) ánh xạ các ký hiệu hằng đến các đối tượng, các ký hiệu vị từ đến các quan hệ giữa các đối tượng, và các ký hiệu hàm đến các hàm trên các đối tượng.
- Một câu nguyên tử là đúng chỉ khi quan hệ được đặt tên bởi vị từ đúng với các đối tượng được đặt tên bởi các thuật ngữ. Các **diễn giải mở rộng**, ánh xạ các biến của lượng từ đến các đối tượng trong mô hình, định nghĩa chân lý của các câu có lượng từ.
- Phát triển một cơ sở tri thức bằng logic bậc nhất đòi hỏi một quy trình cẩn thận gồm phân tích miền, chọn từ vựng, và mã hóa các tiên đề cần thiết để hỗ trợ các suy diễn mong muốn.

Ghi chú về Lịch sử và Thư mục

Mặc dù logic của Aristotle đã đề cập đến sự tổng quát hóa trên các đối tượng, nó còn thua xa sức biểu đạt của logic bậc nhất. Một rào cản lớn cho sự phát triển tiếp theo của nó là sự tập trung vào các vị từ một ngôi mà bỏ qua các vị từ quan hệ nhiều ngôi. Sự luận giải có hệ thống đầu tiên về các quan hệ được đưa ra bởi Augustus De Morgan (1864), người đã trích dẫn ví dụ sau để cho thấy các loại suy diễn mà logic của Aristotle không thể xử lý: “Tất cả ngựa là động vật; do đó, đầu của một con ngựa là đầu của một con vật.” Suy diễn này không thể thực hiện được đối với Aristotle bởi vì bất kỳ quy tắc hợp lệ nào có thể hỗ trợ suy diễn này trước tiên phải phân tích câu bằng cách sử dụng vị từ hai ngôi “x là đầu của y.” Logic của các quan hệ đã được Charles Sanders Peirce nghiên cứu sâu (Peirce, 1870; Misak, 2004).

Logic bậc nhất thực sự bắt nguồn từ việc giới thiệu các lượng từ trong tác phẩm

Begriffsschrift (“Hệ thống ký hiệu khái niệm” hay “Ký hiệu khái niệm”) của Gottlob Frege (1879). Peirce (1883) cũng đã phát triển logic bậc nhất một cách độc lập với Frege, mặc dù muộn hơn một chút. Khả năng lồng các lượng từ của Frege là một bước tiến lớn, nhưng ông đã sử dụng một hệ thống ký hiệu khó sử dụng. Ký hiệu hiện tại cho logic bậc nhất phần lớn là nhờ Giuseppe Peano (1889), nhưng ngữ nghĩa gần như giống hệt của Frege.

Leopold Löwenheim (1915) đã đưa ra một luận giải có hệ thống về lý thuyết mô hình cho logic bậc nhất, bao gồm cả cách xử lý đúng đắn đầu tiên về ký hiệu bằng. Các kết quả của Löwenheim đã được Thoralf Skolem (1920) mở rộng thêm. Alfred Tarski (1935, 1956) đã đưa ra một định

nghĩa rõ ràng về chân lý và sự thỏa mãn theo lý thuyết mô hình trong logic bậc nhất, bằng cách sử dụng lý thuyết tập hợp.

John McCarthy (1958) là người chịu trách nhiệm chính cho việc giới thiệu logic bậc nhất như một công cụ để xây dựng các hệ thống AI. Triền vọng cho AI dựa trên logic đã được thúc đẩy đáng kể bởi sự phát triển của Robinson (1965) về phép giải (resolution), một thủ tục hoàn chỉnh cho suy diễn bậc nhất. Cách tiếp cận logic đã bén rễ tại Đại học Stanford. Cordell Green (1969a, 1969b) đã phát triển một hệ thống suy luận bậc nhất, QA3, dẫn đến những nỗ lực đầu tiên để xây dựng một robot logic tại SRI (Fikes và Nilsson, 1971). Logic bậc nhất đã được Zohar Manna và Richard Waldinger (1971) áp dụng để suy luận về các chương trình và sau đó bởi Michael Genesereth (1984) để suy luận về các mạch. Ở châu Âu, lập trình logic (một dạng suy luận bậc nhất bị hạn chế) đã được phát triển để phân tích ngôn ngữ (Colmerauer và cộng sự, 1973) và cho các hệ thống khai báo tổng quát (Kowalski, 1974).

Các ứng dụng thực tế được xây dựng bằng logic bậc nhất bao gồm một hệ thống để đánh giá các yêu cầu sản xuất cho các sản phẩm điện tử (Mannion, 2002), một hệ thống để suy luận về các chính sách truy cập tệp và quản lý bản quyền kỹ thuật số (Halpern và Weissman, 2008), và một hệ thống để tự động kết hợp các dịch vụ Web (McIlraith và Zeng, 2001).

Các phản ứng đối với giả thuyết Whorf (Whorf, 1956) và vấn đề ngôn ngữ và tư duy nói chung, xuất hiện trong nhiều cuốn sách (Pullum, 1991; Pinker, 2003) bao gồm cả những tựa sách có vẻ đối lập

Why the World Looks Different in Other Languages (Deutscher, 2010) và *Why The World Looks the Same in Any Language* (McWhorter, 2014) (mặc dù cả hai tác giả đều đồng ý rằng có sự khác biệt và sự khác biệt đó là nhỏ). “Lý thuyết của lý thuyết” (“theory” theory) (Gopnik và Glymour, 2002; Tenenbaum và cộng sự, 2007) xem việc học của trẻ em về thế giới tương tự như việc xây dựng các lý thuyết khoa học.

Có một số sách giáo khoa nhập môn tốt về logic bậc nhất, bao gồm một số của các nhân vật hàng đầu trong lịch sử logic: Alfred Tarski (1941), Alonzo Church (1956), và W.V. Quine (1982) (một trong những cuốn dễ đọc nhất). Enderton (1972) đưa ra một góc nhìn thiên về toán học hơn. Một luận giải rất hình thức về logic bậc nhất, cùng với nhiều chủ đề nâng cao khác trong logic, được cung cấp bởi Bell và Machover (1977). Manna và Waldinger (1985) đưa ra một phần giới thiệu dễ đọc về logic từ góc độ khoa học máy tính, cũng như Huth và Ryan (2004), những người tập trung vào việc kiểm chứng chương trình. Barwise và Etchemendy (2002) có cách tiếp cận tương tự như cách được sử dụng ở đây. Smullyan (1995) trình bày các kết quả một cách ngắn gọn, sử dụng định dạng tableau. Gallier (1986) cung cấp một trình bày toán học cực kỳ chặt chẽ về logic bậc nhất, cùng với rất nhiều tài liệu về việc sử dụng nó trong suy luận tự động.

Logical Foundations of Artificial Intelligence (Genesereth và Nilsson, 1987) vừa là một phần giới thiệu vững chắc về logic vừa là luận giải có hệ thống đầu tiên về các tác tử logic với tri giác và hành động, và có hai cuốn cẩm nang tốt: van Bentham và ter Meulen (1997) và Robinson và Voronkov (2001). Tạp chí chính thức của ngành logic toán học thuần túy là

Journal of Symbolic Logic, trong khi *Journal of Applied Logic* giải quyết các vấn đề gần gũi hơn với trí tuệ nhân tạo.

CHƯƠNG 9 SUY LUẬN TRONG LOGIC BẬC NHẤT

Trong chương này, chúng ta định nghĩa các quy trình hiệu quả để trả lời các câu hỏi được đặt ra trong logic bậc nhất.

Trong chương này, chúng ta sẽ mô tả các thuật toán có thể trả lời bất kỳ câu hỏi logic bậc nhất nào có thể trả lời được. Mục 9.1 giới thiệu các quy tắc suy luận cho các lượng từ và chỉ ra cách để rút gọn suy luận bậc nhất thành suy luận mệnh đề, mặc dù có thể tốn kém. Mục 9.2 mô tả cách sử dụng phép hợp nhất (unification) để xây dựng các quy tắc suy luận hoạt động trực tiếp với các câu logic bậc nhất. Sau đó, chúng ta sẽ thảo luận về ba nhóm thuật toán suy luận bậc nhất chính: suy luận tiến (forward chaining) (Mục 9.3), suy luận lùi (backward chaining) (Mục 9.4), và chứng minh định lý dựa trên phép giải (resolution-based theorem proving) (Mục 9.5).

9.1 Suy luận Mệnh đề so với Suy luận Bậc nhất

Trong phần này, chúng ta sẽ giới thiệu các quy tắc suy luận cho lượng từ và chỉ ra cách giảm suy luận bậc nhất về suy luận mệnh đề, mặc dù có thể tốn kém nhiều chi phí.

Một cách để thực hiện suy luận bậc nhất là chuyển đổi cơ sở tri thức bậc nhất sang logic mệnh đề và sử dụng suy luận mệnh đề, vốn đã được biết đến cách thực hiện. Bước đầu tiên là loại bỏ các lượng từ phổ dụng. Ví dụ, giả sử cơ sở tri thức của chúng ta chứa tiên đề dân gian tiêu chuẩn rằng tất cả các vị vua tham lam đều độc ác:

$$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x).$$

Từ đó, chúng ta có thể suy ra bất kỳ câu nào sau đây:

$$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$$

$$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$$

$$\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John})).$$

Nói chung, quy tắc **Khởi tạo Phổ dụng (UI)** cho biết chúng ta có thể suy ra bất kỳ câu nào thu được bằng cách thay thế một **hạng tử nền** (một hạng tử không chứa biến) cho một biến được lượng từ phổ dụng.¹

¹ Không nên nhầm lẫn các phép thay thế này với các diễn giải mở rộng được sử dụng để định nghĩa ngữ nghĩa của lượng từ trong Mục 8.2.6. Phép thay thế thay một biến bằng một hạng tử (một phần của cú pháp) để tạo ra một câu mới, trong khi một diễn giải ánh xạ một biến thành một đối tượng trong miền.

Để viết ra quy tắc suy luận một cách chính thức, chúng ta sử dụng khái niệm **phép thay thế** đã được giới thiệu trong Mục 8.3. Ký hiệu là kết quả của việc áp dụng phép thay thế θ vào câu α . Khi đó, quy tắc được viết như sau:

cho bất kỳ biến v và hạng tử nền g . Ví dụ, ba câu được đưa ra trước đó thu được bằng các phép thay thế $\{x/\text{John}\}$, $\{x/\text{Richard}\}$, và $\{x/\text{Father}(\text{John})\}$.

Tương tự, quy tắc **Khởi tạo Tồn tại** thay thế một biến được lượng từ tồn tại bằng một ký hiệu hằng mới duy nhất. Phát biểu chính thức như sau: với bất kỳ câu α , biến v , và ký hiệu hằng k không xuất hiện ở nơi nào khác trong cơ sở tri thức,

Ví dụ, từ câu

$$\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

chúng ta có thể suy ra câu

$$\text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$$

miễn là C_1 không xuất hiện ở nơi nào khác trong cơ sở tri thức. Về cơ bản, câu tồn tại nói rằng có một số đối tượng thỏa mãn một điều kiện, và việc áp dụng quy tắc khởi tạo tồn tại chỉ đơn giản là đặt tên cho đối tượng đó. Tất nhiên, tên đó không được thuộc về một đối tượng khác. Toán học cung cấp một ví dụ hay: giả sử chúng ta phát hiện ra rằng có một số lớn hơn một chút so với 2.71828 và thỏa mãn phương trình $d(x^y)/dy = x^y$ cho x . Chúng ta có thể đặt tên cho số này là e , nhưng sẽ là sai lầm nếu đặt tên nó bằng một đối tượng đã tồn tại, chẳng hạn như π . Trong logic, tên mới này được gọi là **hằng Skolem**.

Trong khi Khởi tạo Phổ dụng có thể được áp dụng nhiều lần cho cùng một tiên đề để tạo ra nhiều hệ quả khác nhau, Khởi tạo Tồn tại chỉ cần được áp dụng một lần, và sau đó câu được lượng từ tồn tại có thể được loại bỏ. Ví dụ, chúng ta không cần $\exists x \text{Kill}(x, \text{Victim})$ một khi đã thêm câu $\text{Kill}(\text{Murderer}, \text{Victim})$.

9.1.1 Giảm về Suy luận Mệnh đề

Bây giờ chúng ta sẽ chỉ ra cách chuyển đổi bất kỳ cơ sở tri thức bậc nhất nào thành một cơ sở tri thức mệnh đề. Ý tưởng đầu tiên là, giống như một câu tồn tại có thể được thay thế bằng một khởi tạo, một câu phổ dụng có thể được thay thế bằng tập hợp tất cả các khởi tạo có thể. Ví dụ, giả sử cơ sở tri thức của chúng ta chỉ chứa các câu sau:

$$\forall x \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$$

$$\text{King}(\text{John})$$

$$\text{Greedy}(\text{John})$$

$$\text{Brother}(\text{Richard}, \text{John}).$$

và các đối tượng duy nhất là John và Richard. Chúng ta áp dụng UI cho câu đầu tiên bằng tất cả các phép thay thế có thể, $\{x/\text{John}\}$ và $\{x/\text{Richard}\}$. Chúng ta thu được:

$$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$$

$$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard}).$$

Tiếp theo, thay thế các câu nguyên tử nền, chẳng hạn như $\text{King}(\text{John})$, bằng các ký hiệu mệnh đề, chẳng hạn như JohnIsKing . Cuối cùng, áp dụng bất kỳ thuật toán mệnh đề hoàn chỉnh nào trong Chương 7 để thu được các kết luận như JohnIsEvil , tương đương với $\text{Evil}(\text{John})$.

Kỹ thuật **mệnh đề hóa** này có thể được tổng quát hóa hoàn toàn, như chúng ta sẽ chỉ ra trong Mục 9.5. Tuy nhiên, có một vấn đề: khi cơ sở tri thức bao gồm một ký hiệu hàm, tập hợp các phép thay thế hạng tử nền có thể là vô hạn! Ví dụ, nếu cơ sở tri thức đề cập đến ký hiệu *Father*, thì vô số các hạng tử lồng nhau như *Father(Father(Father(John)))* có thể được tạo ra.

May mắn thay, có một định lý nổi tiếng của Jacques Herbrand (1930) nói rằng nếu một câu được suy ra từ cơ sở tri thức bậc nhất ban đầu, thì có một chứng minh liên quan chỉ đến một tập hợp con hữu hạn của cơ sở tri thức đã được mệnh đề hóa. Vì bất kỳ tập hợp con nào như vậy đều có độ sâu lồng nhau tối đa trong số các hạng tử nền của nó, chúng ta có thể tìm thấy tập hợp con đó bằng cách đầu tiên tạo ra tất cả các khởi tạo với các ký hiệu hằng (Richard và John), sau đó tất cả các hạng tử có độ sâu 1 (*Father(Richard)* và *Father(John)*), tiếp theo là tất cả các hạng tử có độ sâu 2, và cứ tiếp tục như vậy, cho đến khi chúng ta có thể xây dựng một chứng minh mệnh đề của câu được suy ra.

Chúng ta đã phác thảo một cách tiếp cận để suy luận bậc nhất thông qua mệnh đề hóa, đây là một cách tiếp cận **hoàn chỉnh**—nghĩa là bất kỳ câu nào được suy ra đều có thể được chứng minh. Đây là một thành tựu lớn, vì không gian của các mô hình có thể là vô hạn. Mặt khác, chúng ta không biết liệu câu đó có được suy ra hay không cho đến khi chứng minh hoàn tất! Điều gì sẽ xảy ra khi câu đó không được suy ra? Chúng ta có thể biết được không? Đối với logic bậc nhất, hóa ra chúng ta không thể. Quy trình chứng minh của chúng ta có thể tiếp tục mãi, tạo ra ngày càng nhiều hạng tử lồng nhau sâu hơn, nhưng chúng ta sẽ không biết liệu nó có bị kẹt trong một vòng lặp vô vọng hay chứng minh sắp xuất hiện. Điều này rất giống với **bài toán dừng** đối với máy Turing. Alan Turing (1936) và Alonzo Church (1936) đều đã chứng minh, theo những cách khá khác nhau, sự không thể tránh khỏi của tình trạng này. Câu hỏi về sự suy ra đối với logic bậc nhất là **nửa quyết định được**—nghĩa là tồn tại các thuật toán có thể trả lời “có” cho mọi câu được suy ra, nhưng không tồn tại thuật toán nào có thể trả lời “không” cho mọi câu không được suy ra.

9.2 Phép Hợp Nhất và Suy Luận Bậc Nhất

Người đọc tinh ý sẽ nhận thấy rằng phương pháp mệnh đề hóa tạo ra nhiều thể hiện không cần thiết của các câu lượng từ hóa phổ quát. Chúng ta mong muốn một phương pháp chỉ sử dụng một quy tắc duy nhất, suy luận rằng phép thế $\{x/John\}$ giải quyết truy vấn $Evil(x)$ như sau: từ quy tắc cho rằng những vị vua tham lam là độc ác, hãy tìm một x sao cho x là vua và x tham lam, sau đó suy ra rằng x này độc ác. Tổng quát hơn, nếu có một phép thế θ nào đó khiến mỗi liên từ trong tiền đề của phép kéo theo trùng khớp với các câu đã có trong cơ sở tri thức, thì chúng ta có thể khẳng định kết luận của phép kéo theo sau khi áp dụng θ . Trong trường hợp này, phép thế $\theta = \{x/John\}$ đạt được mục đích đó.

Giả sử thay vì biết *Greedy(John)*, chúng ta biết rằng mọi người đều tham lam:

$$\forall y \text{ Greedy}(y).$$

Khi đó, chúng ta vẫn muốn kết luận rằng *Evil(John)*, bởi vì chúng ta biết John là vua (theo giả thiết) và John tham lam (vì mọi người đều tham lam). Để điều này hoạt động, chúng ta cần tìm một phép thế cho cả các biến trong câu kéo theo và các biến trong các câu thuộc cơ sở tri thức. Trong trường hợp này, áp dụng phép thế $\{x/John, y/John\}$ vào các tiền đề *King(x)* và *Greedy(x)* cùng các câu trong cơ sở tri thức *King(John)* và *Greedy(y)* sẽ khiến chúng trùng khớp. Do đó, chúng ta có thể suy ra hệ quả của phép kéo theo.

Quá trình suy luận này có thể được mô hình hóa như một quy tắc suy luận đơn lẻ mà chúng ta gọi là **Modus Ponens Tổng quát**. Đối với các câu nguyên tử $p_i, p_{i'}$ và q , nếu tồn tại một phép thế θ sao cho $\text{SUBST}(\theta, p_{i'}) = \text{SUBST}(\theta, p_i)$ với mọi i , thì quy tắc được viết như sau:

$$\frac{p_{1'}, p_{2'}, \dots, p_{n'}, (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}$$

Quy tắc này có $n + 1$ tiền đề: n câu nguyên tử $p_{i'}$ và một phép kéo theo. Kết luận là kết quả của việc áp dụng phép thế θ vào hệ quả q . Ví dụ:

- $p_{1'}$ là *King(John)*, p_1 là *King(x)*
- $p_{2'}$ là *Greedy(y)*, p_2 là *Greedy(x)*
- θ là $\{x/\text{John}, y/\text{John}\}$
- q là *Evil(x)*
- $\text{SUBST}(\theta, q)$ là *Evil(John)*.

Có thể dễ dàng chứng minh rằng Modus Ponens Tổng quát là một quy tắc suy luận đúng đắn. Đầu tiên, ta nhận thấy rằng với mọi câu p (các biến được giả định là lượng từ hóa phổ quát) và mọi phép thế θ , câu $p \models \text{SUBST}(\theta, p)$ luôn đúng theo Quy tắc Thể hiện Phổ quát. Điều này đặc biệt đúng với θ thỏa mãn các điều kiện của Modus Ponens Tổng quát. Do đó, từ $p_{1'}, \dots, p_{n'}$, ta có thể suy ra:

$$\text{SUBST}(\theta, p_{1'}) \wedge \dots \wedge \text{SUBST}(\theta, p_{n'})$$

Và từ phép kéo theo $p_1 \wedge \dots \wedge p_n \Rightarrow q$, ta có thể suy ra:

$$\text{SUBST}(\theta, p_1) \wedge \dots \wedge \text{SUBST}(\theta, p_n) \Rightarrow \text{SUBST}(\theta, q).$$

Vì θ trong Modus Ponens Tổng quát được định nghĩa sao cho $\text{SUBST}(\theta, p_{i'}) = \text{SUBST}(\theta, p_i)$ với mọi i , nên câu đầu tiên trong hai câu trên khớp chính xác với tiền đề của câu thứ hai. Do đó, $\text{SUBST}(\theta, q)$ được suy ra bằng Modus Ponens.

Modus Ponens Tổng quát là phiên bản “nâng cao” của Modus Ponens—nó nâng Modus Ponens từ logic mệnh đề không biến lên logic bậc nhất. Trong phần còn lại của chương này, chúng ta sẽ thấy rằng có thể phát triển các phiên bản nâng cao của các thuật toán suy diễn tiến, suy diễn lùi và phân giải đã được giới thiệu trong Chương 7. Ưu điểm chính của các quy tắc suy luận nâng cao so với mệnh đề hóa là chúng chỉ thực hiện các phép thế cần thiết để cho phép các suy luận cụ thể tiến hành.

9.2.1 Phép Hợp Nhất

Các quy tắc suy luận nâng cao yêu cầu tìm các phép thế khiến các biểu thức logic khác nhau trở nên giống nhau. Quá trình này được gọi là **phép hợp nhất** và là một thành phần quan trọng của tất cả các thuật toán suy luận bậc nhất. Thuật toán **Hợp nhất (UNIFY)** nhận hai câu và trả về một bộ hợp nhất (một phép thế) nếu tồn tại:

$$\text{UNIFY}(p, q) = \theta \text{ với } \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q).$$

Hãy xem một số ví dụ về cách UNIFY hoạt động. Giả sử chúng ta có truy vấn $\text{AskVars}(\text{Knows}(\text{John}, x))$: John biết ai? Câu trả lời cho truy vấn này có thể được tìm thấy bằng cách tìm tất cả các câu trong cơ sở tri thức hợp nhất với $\text{Knows}(\text{John}, x)$. Dưới đây là kết quả hợp nhất với bốn câu khác nhau có thể có trong cơ sở tri thức:

- $\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(\text{John}, \text{Jane})) = \{x/\text{Jane}\}$
- $\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{Bill})) = \{x/\text{Bill}, y/\text{John}\}$
- $\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{Mother}(y))) = \{y/\text{John}, x/\text{Mother}(\text{John})\}$
- $\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x, \text{Elizabeth})) = \text{thất bại}$.

Phép hợp nhất cuối cùng thất bại vì x không thể đồng thời nhận giá trị John và Elizabeth. Lưu ý rằng $\text{Knows}(x, \text{Elizabeth})$ có nghĩa là “Mọi người đều biết Elizabeth,” vì vậy lẽ ra chúng ta có thể suy ra rằng John biết Elizabeth. Vấn đề chỉ phát sinh vì hai câu trùng tên biến x . Vấn đề này có thể tránh được bằng cách **chuẩn hóa riêng biệt** một trong hai câu đang hợp nhất, nghĩa là đổi tên các biến để tránh trùng lặp. Ví dụ, chúng ta có thể đổi x trong $\text{Knows}(x, \text{Elizabeth})$ thành x_{17} (một tên biến mới) mà không làm thay đổi ý nghĩa. Khi đó, phép hợp nhất sẽ thành công:

$$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x_{17}, \text{Elizabeth})) = \{x/\text{Elizabeth}, x_{17}/\text{John}\}.$$

Có một vấn đề phức tạp khác: chúng ta đã nói rằng UNIFY nên trả về một phép thế khiến hai đối số trông giống nhau. Nhưng có thể có nhiều hơn một bộ hợp nhất như vậy. Ví dụ, $\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, z))$ có thể trả về $\{y/\text{John}, x/z\}$ hoặc $\{y/\text{John}, x/\text{John}, z/\text{John}\}$. Bộ hợp nhất đầu tiên cho kết quả hợp nhất là $\text{Knows}(\text{John}, z)$, trong khi bộ thứ hai cho $\text{Knows}(\text{John}, \text{John})$. Kết quả thứ hai có thể thu được từ kết quả đầu tiên bằng một phép thế bổ sung $\{z/\text{John}\}$; chúng ta nói rằng bộ hợp nhất đầu tiên **tổng quát hơn** bộ thứ hai, vì nó đặt ít ràng buộc hơn lên giá trị của các biến.

Mỗi cặp biểu thức hợp nhất được có một **bộ hợp nhất tổng quát nhất (MGU)** duy nhất, không phân biệt bởi việc đổi tên và thay thế biến. Ví dụ, $\{x/\text{John}\}$ và $\{y/\text{John}\}$ được coi là tương đương, tương tự như $\{x/\text{John}, y/\text{John}\}$ và $\{x/\text{John}, y/x\}$.

Một thuật toán tính toán các bộ hợp nhất tổng quát nhất được trình bày trong Hình 9.1. Quá trình này đơn giản: duyệt đệ quy hai biểu thức đồng thời “song song,” xây dựng bộ hợp nhất dần dần, nhưng thất bại nếu hai điểm tương ứng trong cấu trúc không khớp. Có một bước tốn kém: khi khớp một biến với một biểu thức phức tạp, cần kiểm tra xem bản thân biến có xuất hiện trong biểu thức hay không; nếu có, việc khớp sẽ thất bại vì không thể xây dựng một bộ hợp nhất nhất quán. Ví dụ, $S(x)$ không thể hợp nhất với $S(S(x))$. Kiểm tra này, gọi là **kiểm tra xuất hiện (occur check)**, khiến độ phức tạp của toàn bộ thuật toán là bậc hai theo kích thước của các biểu thức đang hợp nhất. Một số hệ thống, bao gồm nhiều hệ thống lập trình logic, đơn giản bỏ qua kiểm tra xuất hiện và đặt trách nhiệm lên người dùng để tránh các suy luận không chắc chắn. Các hệ thống khác sử dụng thuật toán hợp nhất phức tạp hơn với độ phức tạp tuyến tính.

9.2.2 Lưu trữ và Truy xuất

Các hàm **TELL**, **ASK** và **ASKVARS** dùng để thông báo và truy vấn cơ sở tri thức được xây dựng dựa trên các hàm nguyên thủy hơn là **STORE** và **FETCH**. **STORE(s)** lưu trữ một câu s vào cơ

sở tri thức, và **FETCH**(q) trả về tất cả các bộ hợp nhất sao cho truy vấn q hợp nhất với một câu nào đó trong cơ sở tri thức. Vấn đề chúng ta sử dụng để minh họa phép hợp nhất—tìm tất cả các sự kiện hợp nhất với $Knows(John, x)$ —là một ví dụ của **FETCH**.

Cách đơn giản nhất để triển khai **STORE** và **FETCH** là lưu tất cả các sự kiện vào một danh sách dài và thử hợp nhất mỗi truy vấn với từng phần tử trong danh sách. Quá trình này không hiệu quả nhưng vẫn hoạt động. Phần còn lại của mục này sẽ đề cập đến các cách để cải thiện hiệu quả truy xuất.

Chúng ta có thể làm cho **FETCH** hiệu quả hơn bằng cách đảm bảo rằng các phép hợp nhất chỉ được thử với các câu có khả năng hợp nhất. Ví dụ, không có lý do gì để thử hợp nhất $Knows(John, x)$ với $Brother(Richard, John)$. Chúng ta có thể tránh các phép hợp nhất như vậy bằng cách lập chỉ mục các sự kiện trong cơ sở tri thức. Một phương pháp đơn giản gọi là **lập chỉ mục vị từ** đặt tất cả các sự kiện $Knows$ vào một nhóm và tất cả các sự kiện $Brother$ vào một nhóm khác. Các nhóm này có thể được lưu trữ trong một bảng băm để truy cập nhanh.

```
function UNIFY( $x, y, \theta = \text{empty}$ ) returns a substitution to make  $x$  and  $y$  identical, or failure
  if  $\theta = \text{failure}$  then return failure
  else if  $x = y$  then return  $\theta$ 
  else if VARIABLE?( $x$ ) then return UNIFY-VAR( $x, y, \theta$ )
  else if VARIABLE?( $y$ ) then return UNIFY-VAR( $y, x, \theta$ )
  else if COMPOUND?( $x$ ) and COMPOUND?( $y$ ) then
    return UNIFY(ARGS( $x$ ), ARGS( $y$ ), UNIFY(OP( $x$ ), OP( $y$ ),  $\theta$ ))
  else if LIST?( $x$ ) and LIST?( $y$ ) then
    return UNIFY(REST( $x$ ), REST( $y$ ), UNIFY(FIRST( $x$ ), FIRST( $y$ ),  $\theta$ ))
  else return failure
```

```
function UNIFY-VAR( $var, x, \theta$ ) returns a substitution
  if  $\{var/val\} \in \theta$  for some  $val$  then return UNIFY( $val, x, \theta$ )
  else if  $\{x/val\} \in \theta$  for some  $val$  then return UNIFY( $var, val, \theta$ )
  else if OCCUR-CHECK?( $var, x$ ) then return failure
  else return add  $\{var/x\}$  to  $\theta$ 
```

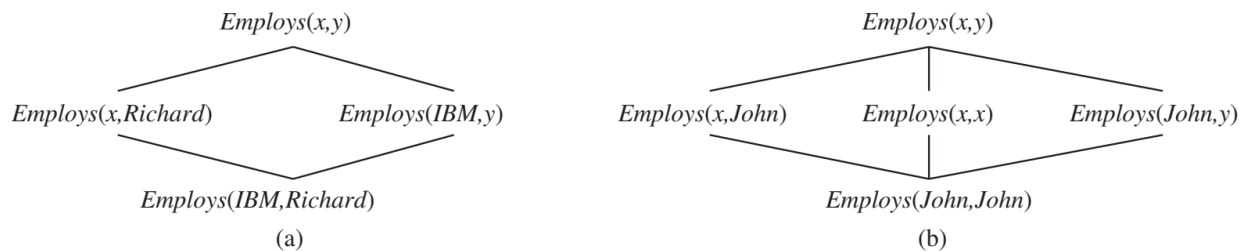
Hình 9.1 Thuật toán hợp nhất. Các đối số x và y có thể là bất kỳ biểu thức nào: một hằng số, một biến, hoặc một biểu thức phức hợp như một câu hoặc một thuật ngữ phức tạp, hoặc một danh sách các biểu thức. Đối số θ là một phép thế, ban đầu là phép thế rỗng, nhưng sau đó các cặp {biến/giá trị} được thêm vào khi chúng ta đệ quy qua các đầu vào, so sánh từng phần tử của các biểu thức. Trong một biểu thức phức hợp như $F(A, B)$, trường **OP**(x) chọn ra ký hiệu hàm F và trường **ARGS**(x) chọn ra danh sách đối số (A, B).

Lập chỉ mục vị từ hữu ích khi có nhiều ký hiệu vị từ nhưng chỉ một vài mệnh đề cho mỗi ký hiệu. Tuy nhiên, đôi khi một vị từ có nhiều mệnh đề. Ví dụ, giả sử cơ quan thuế muốn theo dõi ai thuê ai bằng vị từ $Employs(x, y)$. Đây sẽ là một nhóm rất lớn với hàng triệu nhà tuyển dụng và hàng chục triệu nhân viên. Trả lời một truy vấn như $Employs(x, Richard)$ bằng lập chỉ mục vị từ sẽ yêu cầu quét toàn bộ nhóm.

Đối với truy vấn cụ thể này, sẽ hữu ích nếu các sự kiện được lập chỉ mục theo cả vị từ và đối số thứ hai, có thể sử dụng một khóa kết hợp trong bảng băm. Khi đó, chúng ta chỉ cần tạo khóa từ truy vấn và truy xuất chính xác các sự kiện hợp nhất với truy vấn. Đối với các truy vấn khác, như $Employs(IBM, y)$, chúng ta cần lập chỉ mục các sự kiện bằng cách kết hợp vị từ với đối số đầu tiên. Do đó, các sự kiện có thể được lưu trữ dưới nhiều khóa chỉ mục, giúp chúng có thể truy cập ngay lập tức cho các truy vấn khác nhau mà chúng có thể hợp nhất.

Với một câu cần lưu trữ, có thể xây dựng các chỉ mục cho tất cả các truy vấn có thể hợp nhất với nó. Đối với sự kiện $Employs(IBM, Richard)$, các truy vấn là:

- $Employs(IBM, Richard)$: IBM có thuê Richard không?
- $Employs(x, Richard)$: Ai thuê Richard?
- $Employs(IBM, y)$: IBM thuê ai?
- $Employs(x, y)$: Ai thuê ai?



Hình 9.2 (a) Lưới bao hàm (subsumption lattice) có nút thấp nhất là $Employs(IBM, Richard)$. (b) Lưới bao hàm cho câu $Employs(John, John)$.

Các truy vấn này tạo thành một **lattice bao phủ**, như trong Hình 9.2(a). Lattice này có một số tính chất thú vị. Mỗi nút con trong lattice thu được từ nút cha bằng một phép thế duy nhất; và nút chung “cao nhất” của hai nút bất kỳ là kết quả của việc áp dụng bộ hợp nhất tổng quát nhất của chúng. Một câu với các hằng số lặp lại có một lattice hơi khác, như trong Hình 9.2(b). Mặc dù các ký hiệu hàm không được hiển thị trong hình, chúng cũng có thể được tích hợp vào cấu trúc lattice.

Đối với các vị từ có số lượng đối số nhỏ, việc tạo chỉ mục cho mọi điểm trong lattice bao phủ là một sự cân bằng tốt. Điều này yêu cầu nhiều công sức hơn khi lưu trữ, nhưng tăng tốc thời gian truy xuất. Tuy nhiên, đối với một vị từ có n đối số, lattice chứa $O(2^n)$ nút. Nếu cho phép các ký hiệu hàm, số lượng nút cũng tăng theo hàm mũ theo kích thước của các biểu thức trong câu cần lưu trữ. Điều này có thể dẫn đến một số lượng lớn chỉ mục.

Chúng ta phải bằng cách nào đó giới hạn các chỉ mục vào những chỉ mục có khả năng được sử dụng thường xuyên trong các truy vấn; nếu không, chúng ta sẽ tốn nhiều thời gian tạo chỉ mục hơn là tiết kiệm được từ việc có chúng. Chúng ta có thể áp dụng một chính sách cố định, như chỉ duy trì các chỉ mục trên các khóa bao gồm một vị từ cộng với một đối số duy nhất. Hoặc chúng ta có thể học một chính sách thích ứng tạo ra các chỉ mục đáp ứng nhu cầu của các loại truy vấn được đặt ra. Đối với các cơ sở dữ liệu thương mại với hàng tỷ sự kiện, vấn đề này đã được nghiên cứu chuyên sâu, phát triển công nghệ và tối ưu hóa liên tục.

9.3 Suy diễn tiến

Trong Mục 7.5, chúng ta đã trình bày một thuật toán suy diễn tiến cho các cơ sở tri thức chứa các mệnh đề xác định dạng mệnh đề. Ở đây, chúng ta mở rộng ý tưởng đó để áp dụng cho các mệnh đề xác định bậc nhất.

Tất nhiên, có một số câu logic không thể biểu diễn dưới dạng mệnh đề xác định và do đó không thể xử lý bằng phương pháp này. Tuy nhiên, các quy tắc có dạng **Tiền đề $\Rightarrow$ Kết luận** đủ để bao phủ nhiều hệ thống thực tế thú vị.

9.3.1 Mệnh đề xác định bậc nhất

Một **mệnh đề xác định bậc nhất** là một phép tuyển (disjunction) của các literal, trong đó chỉ có một literal là dương. Điều này có nghĩa một mệnh đề xác định có thể là một câu nguyên tử hoặc một phép kéo theo có tiền đề là một phép hội của các literal dương và kết luận là một literal dương duy nhất. Các lượng tử tồn tại không được phép, và các lượng tử phổ dụng được ngầm hiểu: nếu bạn thấy biến x trong một mệnh đề xác định, điều đó có nghĩa là có một lượng tử phổ dụng $\forall x$ ngầm định. Một mệnh đề xác định bậc nhất điển hình có dạng như sau:

$$\text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x),$$

nhưng các literal như $\text{King}(\text{John})$ và $\text{Greedy}(y)$ cũng được tính là mệnh đề xác định. Các literal bậc nhất có thể chứa biến, ví dụ $\text{Greedy}(y)$ được hiểu là “mọi người đều tham lam” (lượng tử phổ dụng được ngầm hiểu).

Hãy áp dụng các mệnh đề xác định để biểu diễn bài toán sau:

Luật pháp quy định rằng một người Mỹ bán vũ khí cho các quốc gia thù địch là phạm tội. Đất nước Nono, kẻ thù của Mỹ, có một số tên lửa, và tất cả tên lửa của họ đều được Đại tá West, một người Mỹ, bán cho họ.

Đầu tiên, chúng ta biểu diễn các sự kiện này dưới dạng các mệnh đề xác định bậc nhất:

- “...một người Mỹ bán vũ khí cho các quốc gia thù địch là phạm tội”:

$$\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x). \quad (9.3)$$

- “Nono... có một số tên lửa.” Câu $\exists x \text{Owns}(\text{Nono}, x) \wedge \text{Missile}(x)$ được chuyển thành hai mệnh đề xác định bằng cách áp dụng **Tồn tại cụ thể hóa**, giới thiệu một hằng số mới M_1 :

$$\text{Missile}(M_1) \quad (9.5)$$

- “Tất cả tên lửa của Nono đều được Đại tá West bán cho họ”:

$$\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono}). \quad (9.6)$$

- Chúng ta cũng cần biết rằng tên lửa là vũ khí:

$$\text{Missile}(x) \Rightarrow \text{Weapon}(x). \quad (9.7)$$

- Và phải biết rằng kẻ thù của Mỹ được coi là “thù địch”:

$$\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x). \quad (9.8)$$

- “West, một người Mỹ... ”:

$$\text{American}(\text{West}). \quad (9.9)$$

- “Đất nước Nono, kẻ thù của Mỹ... ”:

$$\text{Enemy}(\text{Nono}, \text{America}). \quad (9.10)$$

Cơ sở tri thức này là một **cơ sở tri thức Datalog**: Datalog là ngôn ngữ gồm các mệnh đề xác định bậc nhất không chứa ký hiệu hàm. Tên Datalog xuất phát từ khả năng biểu diễn các phát biểu điển hình trong cơ sở dữ liệu quan hệ. Việc không có ký hiệu hàm làm cho quá trình suy diễn trở nên dễ dàng hơn nhiều.

9.3.2 Thuật toán suy diễn tiến đơn giản

Hình 9.3 trình bày một thuật toán suy diễn tiến đơn giản. Bắt đầu từ các sự kiện đã biết, thuật toán kích hoạt tất cả các quy tắc có tiền đề được thỏa mãn, sau đó thêm kết luận của chúng vào tập sự kiện đã biết. Quá trình này lặp lại cho đến khi truy vấn được trả lời (giả sử chỉ cần một câu trả lời) hoặc không có sự kiện mới nào được thêm vào.

```
function FOL-FC-ASK(KB,  $\alpha$ ) returns a substitution or false
  inputs: KB, the knowledge base, a set of first-order definite clauses
            $\alpha$ , the query, an atomic sentence

  while true do
    new  $\leftarrow \{\}$  // The set of new sentences inferred on each iteration
    for each rule in KB do
       $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-VARIABLES}(\text{rule})$ 
      for each  $\theta$  such that  $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$ 
        for some  $p'_1, \dots, p'_n$  in KB
           $q' \leftarrow \text{SUBST}(\theta, q)$ 
          if  $q'$  does not unify with some sentence already in KB or new then
            add  $q'$  to new
             $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
            if  $\phi$  is not failure then return  $\phi$ 
    if new =  $\{\}$  then return false
    add new to KB
```

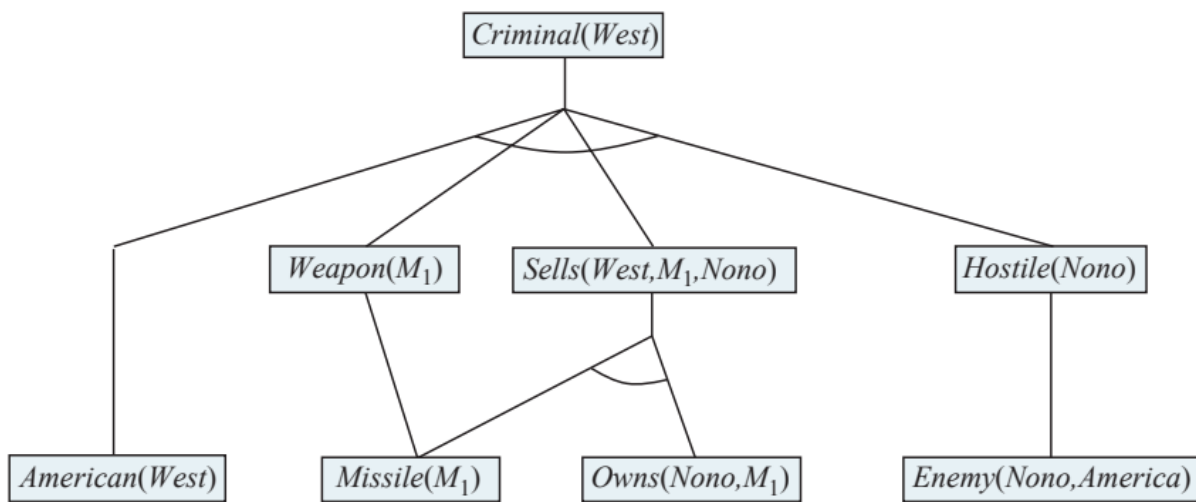
Hình 9.3 Một thuật toán suy diễn tiến (forward-chaining) đơn giản về mặt khái niệm, nhưng không hiệu quả. Ở mỗi vòng lặp, nó thêm vào KB (cơ sở tri thức) tất cả các câu nguyên tử có thể được suy ra trong một bước từ các câu kéo theo và các câu nguyên tử đã có trong KB. Hàm **STANDARDIZE-VARIABLES** thay thế tất cả các biến trong các đối số của nó bằng các biến mới chưa từng được sử dụng trước đây.

Lưu ý rằng một sự kiện không được coi là “mới” nếu nó chỉ là một cách đặt tên lại của một sự kiện đã biết—một câu được coi là đặt tên lại của câu khác nếu chúng giống hệt nhau ngoại trừ tên biến.

Ví dụ, $\text{Likes}(x, \text{IceCream})$ và $\text{Likes}(y, \text{IceCream})$ là các cách đặt tên lại của nhau. Cả hai đều có cùng ý nghĩa: “Mọi người đều thích kem.”

Chúng ta sử dụng bài toán tội phạm để minh họa cho FOL-FC-ASK. Các câu kéo theo có sẵn để áp dụng là (9.3), (9.6), (9.7) và (9.8). Cần hai lần lặp:

- **Lần lặp đầu tiên:** Quy tắc (9.3) có tiền đề chưa được thỏa mãn. Quy tắc (9.6) được thỏa mãn với $\{x/M_1\}$, và $\text{Sells}(\text{West}, M_1, \text{Nono})$ được thêm vào. Quy tắc (9.7) được thỏa mãn với $\{x/M_1\}$, và $\text{Weapon}(M_1)$ được thêm vào. Quy tắc (9.8) được thỏa mãn với $\{x/\text{Nono}\}$, và $\text{Hostile}(\text{Nono})$ được thêm vào.
- **Lần lặp thứ hai:** Quy tắc (9.3) được thỏa mãn với $\{x/\text{West}, y/M_1, z/\text{Nono}\}$, và suy luận $\text{Criminal}(\text{West})$ được thêm vào.



Hình 9.4 Cây chứng minh (proof tree) được tạo ra bởi suy diễn tiến (forward-chaining) trong ví dụ về tội phạm. Các sự thật ban đầu xuất hiện ở cấp dưới cùng, các sự thật được suy ra ở lần lặp đầu tiên ở cấp giữa, và các sự thật được suy ra ở lần lặp thứ hai ở cấp trên cùng.

Hình 9.4 cho thấy cây chứng minh được tạo ra. Lưu ý rằng không có suy luận mới nào có thể được thực hiện tại thời điểm này vì mọi câu có thể được suy ra bằng suy diễn tiến đã được chứa rõ ràng trong KB. Một cơ sở tri thức như vậy được gọi là **điểm bất động** của quá trình suy diễn.

FOL-FC-ASK dễ dàng được phân tích. Thứ nhất, nó **đúng đắn** vì mọi suy luận chỉ là áp dụng **Modus Ponens tổng quát**, vốn đã đúng đắn. Thứ hai, nó **đầy đủ** đối với các cơ sở tri thức chứa mệnh đề xác định; nghĩa là, nó trả lời mọi truy vấn mà câu trả lời được suy ra từ bất kỳ cơ sở tri thức mệnh đề xác định nào.

Đối với các cơ sở tri thức Datalog không chứa ký hiệu hàm, việc chứng minh tính đầy đủ khá đơn giản. Chúng ta bắt đầu bằng cách đếm số lượng sự kiện có thể được thêm vào, từ đó xác định số lần lặp tối đa. Gọi k là số lượng đối số tối đa của bất kỳ vị từ nào, p là số lượng vị từ, và n là số lượng ký hiệu hằng. Rõ ràng, không thể có nhiều hơn pn^k sự kiện nguyên tử khác nhau, do đó sau nhiều nhất pn^k lần lặp, thuật toán sẽ đạt đến điểm bất động. Sau đó, chúng ta có thể lập luận tương tự như chứng minh tính đầy đủ của suy diễn tiến mệnh đề (xem trang 249).

Đối với các mệnh đề xác định tổng quát có chứa ký hiệu hàm, FOL-FC-ASK có thể tạo ra vô hạn sự kiện mới, vì vậy chúng ta cần cẩn thận hơn. Đối với trường hợp câu truy vấn q được suy ra, chúng ta dựa vào **định lý Herbrand** (trang 300) để khẳng định rằng thuật toán sẽ tìm thấy chứng minh. Nếu truy vấn không có câu trả lời, thuật toán có thể không dừng trong một số trường hợp. Ví dụ, nếu cơ sở tri thức chứa các tiên đề Peano:

$$\begin{aligned} & \text{NatNum}(0) \\ & \forall n \text{ NatNum}(n) \Rightarrow \text{NatNum}(S(n)), \end{aligned}$$

thì suy diễn tiến sẽ thêm $\text{NatNum}(S(0))$, $\text{NatNum}(S(S(0)))$, $\text{NatNum}(S(S(S(0))))$, và cứ tiếp tục như vậy. Vấn đề này không thể tránh khỏi trong trường hợp tổng quát. Giống như logic bậc nhất tổng quát, bài toán suy ra với mệnh đề xác định là **nửa quyết định được**.

9.3.3 Suy diễn tiến hiệu quả

Thuật toán suy diễn tiến trong Hình 9.3 được thiết kế để dễ hiểu chứ không phải để hiệu quả. Có ba nguồn gây ra sự kém hiệu quả:

23. Vòng lặp bên trong cố gắng khớp mọi quy tắc với mọi sự kiện trong cơ sở tri thức.
24. Thuật toán kiểm tra lại mọi quy tắc trong mỗi lần lặp, ngay cả khi chỉ có rất ít sự kiện mới được thêm vào cơ sở tri thức.
25. Thuật toán có thể tạo ra nhiều sự kiện không liên quan đến mục tiêu.

Chúng ta sẽ giải quyết từng vấn đề này lần lượt.

Khớp quy tắc với sự kiện đã biết

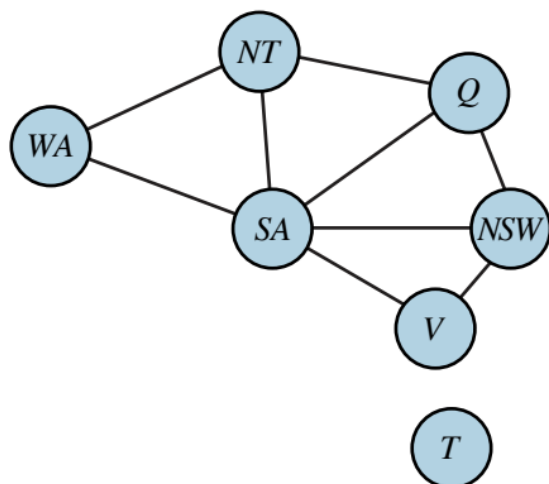
Vấn đề khớp tiên đề của một quy tắc với các sự kiện trong cơ sở tri thức có vẻ đơn giản. Ví dụ, giả sử chúng ta muốn áp dụng quy tắc:

$$\text{Missile}(x) \Rightarrow \text{Weapon}(x).$$

Chúng ta cần tìm tất cả các sự kiện khớp với $\text{Missile}(x)$; trong một cơ sở tri thức được lập chỉ mục phù hợp, điều này có thể được thực hiện trong thời gian không đổi cho mỗi sự kiện. Bây giờ, hãy xem xét một quy tắc như:

$$\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono}).$$

Một lần nữa, chúng ta có thể tìm tất cả các đối tượng thuộc sở hữu của Nono trong thời gian không đổi cho mỗi đối tượng; sau đó, với mỗi đối tượng, chúng ta kiểm tra xem nó có phải là tên lửa hay không. Tuy nhiên, nếu cơ sở tri thức chứa nhiều đối tượng thuộc sở hữu của Nono và rất ít tên lửa, thì tốt hơn là nên tìm tất cả các tên lửa trước, sau đó kiểm tra xem chúng có thuộc sở hữu của Nono hay không. Đây là **vấn đề thứ tự kết hợp**: tìm một thứ tự để giải quyết các kết hợp của tiên đề quy tắc sao cho tổng chi phí là tối thiểu. Hóa ra, việc tìm thứ tự tối ưu là **NP-khó**, nhưng có những heuristic tốt có thể áp dụng. Ví dụ, heuristic **giá trị còn lại tối thiểu (MRV)** được sử dụng cho CSP trong Chương 5 sẽ gợi ý sắp xếp các kết hợp để tìm tên lửa trước nếu có ít tên lửa hơn số đối tượng thuộc sở hữu của Nono.



(a)

$$\begin{aligned} &Diff(wa, nt) \wedge Diff(wa, sa) \wedge \\ &Diff(nt, q) \wedge Diff(nt, sa) \wedge \\ &Diff(q, nsw) \wedge Diff(q, sa) \wedge \\ &Diff(nsw, v) \wedge Diff(nsw, sa) \wedge \\ &Diff(v, sa) \Rightarrow Colorable() \end{aligned}$$

$$\begin{aligned} &Diff(Red, Blue) \quad Diff(Red, Green) \\ &Diff(Green, Red) \quad Diff(Green, Blue) \\ &Diff(Blue, Red) \quad Diff(Blue, Green) \end{aligned}$$

(b)

Hình 9.5 (a) Đồ thị ràng buộc (constraint graph) để tô màu bản đồ Úc. (b) Bài toán CSP (Constraint Satisfaction Problem - Bài toán thỏa mãn ràng buộc) tô màu bản đồ được biểu diễn dưới dạng một mệnh đề xác định (definite clause) duy nhất. Mỗi vùng trên bản đồ được biểu diễn dưới dạng một biến có giá trị có thể là một trong các hằng số Red (Đỏ), Green (Xanh lá) hoặc Blue (Xanh dương) (được khai báo là Diff - Khác nhau).

Mối liên hệ giữa **khớp mẫu** và **thỏa mãn ràng buộc** thực sự rất gần. Chúng ta có thể xem mỗi kết hợp như một ràng buộc trên các biến mà nó chứa—ví dụ, $Missile(x)$ là một ràng buộc một ngôi trên x . Mở rộng ý tưởng này, chúng ta có thể biểu diễn mọi CSP miền hữu hạn dưới dạng một mệnh đề xác định duy nhất cùng với một số sự kiện nền. Xem xét bài toán tô màu bản đồ từ Hình 5.1, được hiển thị lại trong Hình 9.5(a). Một công thức tương đương dưới dạng một mệnh đề xác định duy nhất được đưa ra trong Hình 9.5(b). Rõ ràng, kết luận $Colorable()$ chỉ có thể được suy ra nếu CSP có nghiệm.

Vì các CSP nói chung bao gồm các bài toán 3-SAT như các trường hợp đặc biệt, chúng ta có thể kết luận rằng việc khớp một mệnh đề xác định với một tập hợp sự kiện là **NP-khó**. Điều này có vẻ đáng ngại, nhưng có ba cách để lạc quan hơn:

- **Giới hạn kích thước quy tắc:** Trong các cơ sở tri thức thực tế, hầu hết các quy tắc đều nhỏ và đơn giản (như các quy tắc trong ví dụ tội phạm) thay vì lớn và phức tạp (như công thức CSP trong Hình 9.5). Trong thế giới cơ sở dữ liệu, người ta thường giả định rằng cả kích thước quy tắc và số lượng đối số của vị từ đều bị giới hạn bởi một hằng số, và chỉ quan tâm đến **độ phức tạp dữ liệu**—nghĩa là độ phức tạp của suy diễn như một hàm của số lượng sự kiện nền trong cơ sở tri thức. Dễ dàng chứng minh rằng độ phức tạp dữ liệu của suy diễn tiến là đa thức, không phải hàm mũ.
- **Lớp con của quy tắc:** Xem xét các lớp con của quy tắc mà việc khớp là hiệu quả. Về cơ bản, mọi mệnh đề Datalog có thể được xem như định nghĩa một CSP, do đó việc khớp sẽ khả thi khi CSP tương ứng khả thi. Chương 5 mô tả một số họ CSP khả thi. Ví dụ, nếu đồ thị ràng buộc (đồ thị có các nút là biến và các cạnh là ràng buộc) tạo thành một cây, thì

CSP có thể được giải trong thời gian tuyến tính. Kết quả tương tự cũng áp dụng cho khớp quy tắc.

- **Giảm thiểu khớp thừa:** Cố gắng loại bỏ các lần thử khớp quy tắc thừa trong thuật toán suy diễn tiến, như sẽ được mô tả tiếp theo.

Suy diễn tiến tăng cường

Khi chúng tôi trình bày cách suy diễn tiến hoạt động trên ví dụ tội phạm, chúng tôi đã “gian lận” một chút. Cụ thể, chúng tôi đã bỏ qua một số khớp quy tắc được thực hiện bởi thuật toán trong Hình 9.3. Ví dụ, trong lần lặp thứ hai, quy tắc:

$$\text{Missile}(x) \Rightarrow \text{Weapon}(x)$$

khớp với $\text{Missile}(M_1)$ (một lần nữa), và tất nhiên kết luận $\text{Weapon}(M_1)$ đã được biết nên không có gì xảy ra. Những lần khớp quy tắc thừa như vậy có thể tránh được nếu chúng ta nhận thấy rằng:

Mọi sự kiện mới được suy ra trong lần lặp t phải được suy ra từ ít nhất một sự kiện mới được suy ra trong lần lặp $t - 1$.

Điều này đúng vì bất kỳ suy luận nào không yêu cầu sự kiện mới từ lần lặp $t - 1$ đã có thể được thực hiện ở lần lặp $t - 1$.

Nhận xét này dẫn đến một thuật toán suy diễn tiến tăng cường, trong đó ở lần lặp t , chúng ta chỉ kiểm tra một quy tắc nếu tiền đề của nó chứa một kết hợp p_i khớp với một sự kiện $p_{i'}$ mới được suy ra ở lần lặp $t - 1$. Bước khớp quy tắc sau đó cố định p_i để khớp với $p_{i'}$, nhưng cho phép các kết hợp khác của quy tắc khớp với các sự kiện từ bất kỳ lần lặp nào trước đó. Thuật toán này tạo ra chính xác các sự kiện giống nhau ở mỗi lần lặp như thuật toán trong Hình 9.3, nhưng hiệu quả hơn nhiều.

Với việc lập chỉ mục phù hợp, dễ dàng xác định tất cả các quy tắc có thể được kích hoạt bởi bất kỳ sự kiện nào, và nhiều hệ thống thực tế hoạt động ở chế độ “cập nhật”, trong đó suy diễn tiến được thực hiện để phản ứng với mọi lệnh TELL. Các suy luận sẽ lan truyền qua tập hợp các quy tắc cho đến khi đạt đến điểm bất động, sau đó quá trình bắt đầu lại với sự kiện mới tiếp theo.

Thông thường, chỉ một phần nhỏ các quy tắc trong cơ sở tri thức thực sự được kích hoạt khi thêm một sự kiện cụ thể. Điều này có nghĩa là một lượng lớn công việc thừa được thực hiện khi liên tục xây dựng các khớp một phần có một số tiền đề chưa được thỏa mãn. Ví dụ tội phạm của chúng ta quá nhỏ để minh họa hiệu quả điều này, nhưng hãy lưu ý rằng một khớp một phần được xây dựng trong lần lặp đầu tiên giữa quy tắc:

$$\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$$

và sự kiện $\text{American}(\text{West})$. Khớp một phần này sau đó bị loại bỏ và xây dựng lại trong lần lặp thứ hai (khi quy tắc thành công). Sẽ tốt hơn nếu giữ lại và dần dần hoàn thiện các khớp một phần khi các sự kiện mới xuất hiện, thay vì loại bỏ chúng.

Thuật toán **Rete** là thuật toán đầu tiên giải quyết vấn đề này. Thuật toán tiền xử lý tập hợp các quy tắc trong cơ sở tri thức để xây dựng một mạng luồng dữ liệu, trong đó mỗi nút là một literal từ tiền đề của một quy tắc. Các phép gán biến chảy qua mạng và bị lọc ra khi chúng không khớp với một

literal. Nếu hai literal trong một quy tắc chia sẻ cùng một biến—ví dụ, $Sells(x, y, z)$ và $Hostile(z)$ trong ví dụ tội phạm—thì các phép gán từ mỗi literal được lọc qua một nút bình đẳng. Một phép gán biến đến một nút cho một literal n -ngôi như $Sells(x, y, z)$ có thể phải chờ các phép gán cho các biến khác được thiết lập trước khi quá trình có thể tiếp tục. Tại bất kỳ thời điểm nào, trạng thái của mạng Rete nắm bắt tất cả các khớp một phần của các quy tắc, tránh được một lượng lớn tính toán lại.

Mạng Rete và nhiều cải tiến khác đã trở thành thành phần chính của các hệ thống **sản xuất**, một trong những hệ thống suy diễn tiên đầu tiên được sử dụng rộng rãi. Hệ thống XCON (ban đầu gọi là R1; McDermott, 1982) được xây dựng với kiến trúc hệ thống sản xuất. XCON chứa hàng ngàn quy tắc để thiết kế cấu hình các thành phần máy tính cho khách hàng của Tập đoàn Thiết bị Kỹ thuật số. Đây là một trong những thành công thương mại đầu tiên trong lĩnh vực mới nổi của hệ chuyên gia. Nhiều hệ thống tương tự khác đã được xây dựng với cùng công nghệ cơ bản, được triển khai trong ngôn ngữ đa năng Ops-5.

Các hệ thống sản xuất cũng phổ biến trong các kiến trúc nhận thức—tức là các mô hình lập luận của con người—như ACT (Anderson, 1983) và SOAR (Laird et al., 1987). Trong các hệ thống như vậy, “bộ nhớ làm việc” của hệ thống mô hình hóa bộ nhớ ngắn hạn của con người, và các sản xuất là một phần của bộ nhớ dài hạn. Trong mỗi chu kỳ hoạt động, các sản xuất được khớp với bộ nhớ làm việc chứa các sự kiện. Một sản xuất có điều kiện được thỏa mãn có thể thêm hoặc xóa sự kiện trong bộ nhớ làm việc. Ngược lại với tình huống điển hình trong cơ sở dữ liệu, các hệ thống sản xuất thường có nhiều quy tắc và tương đối ít sự kiện. Với công nghệ khớp được tối ưu hóa phù hợp, các hệ thống có thể hoạt động trong thời gian thực với hàng chục triệu quy tắc.

Các sự kiện không liên quan

Một nguồn kém hiệu quả khác là suy diễn tiến thực hiện tất cả các suy luận cho phép dựa trên các sự kiện đã biết, ngay cả khi chúng không liên quan đến mục tiêu. Trong ví dụ tội phạm của chúng ta, không có quy tắc nào có thể rút ra các kết luận không liên quan. Nhưng nếu có nhiều quy tắc mô tả thói quen ăn uống của người Mỹ hoặc các thành phần và giá cả của tên lửa, thì FOL-FC-ASK sẽ tạo ra các kết luận không liên quan.

Một cách để tránh rút ra các kết luận không liên quan là sử dụng **suy diễn lùi**, như được mô tả trong Mục 9.4. Một cách khác là giới hạn suy diễn tiến cho một tập hợp con được chọn lọc của các quy tắc, như trong PL-FC-ENTAILS? (trang 249). Một cách tiếp cận thứ ba đã xuất hiện trong lĩnh vực **cơ sở dữ liệu suy diễn**, là các cơ sở dữ liệu quy mô lớn, giống như cơ sở dữ liệu quan hệ, nhưng sử dụng suy diễn tiến làm công cụ suy luận tiêu chuẩn thay vì các truy vấn SQL. Ý tưởng là viết lại tập hợp quy tắc, sử dụng thông tin từ mục tiêu, sao cho chỉ các phép gán biến liên quan—thuộc về một tập hợp gọi là **tập ma thuật**—được xem xét trong quá trình suy luận tiến. Ví dụ, nếu mục tiêu là $Criminal(West)$, quy tắc kết luận $Criminal(x)$ sẽ được viết lại để bao gồm một kết hợp bổ sung ràng buộc giá trị của x :

$$Magic(x) \wedge American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x).$$

Sự kiện $Magic(West)$ cũng được thêm vào KB. Bằng cách này, ngay cả khi cơ sở tri thức chứa thông tin về hàng triệu người Mỹ, chỉ có Đại tá West sẽ được xem xét trong quá trình suy luận tiến. Quá trình hoàn chỉnh để định nghĩa các tập ma thuật và viết lại cơ sở tri thức quá phức tạp để trình bày ở đây, nhưng ý tưởng cơ bản là thực hiện một loại suy luận lùi “tổng quát” từ mục tiêu

để xác định các phép gán biến cần được ràng buộc. Do đó, cách tiếp cận tập ma thuật có thể được coi là một dạng lai giữa suy luận tiến và tiến xử lý lùi.

This response is AI-generated, for reference only.

9.4 Suy luận lùi (Backward Chaining)

Họ thuật toán suy luận logic thứ hai sử dụng **suy luận lùi** trên các mệnh đề xác định (definite clauses). Các thuật toán này làm việc ngược từ mục tiêu, thông qua các luật để tìm các sự kiện đã biết hỗ trợ cho chứng minh.

9.4.1 Thuật toán suy luận lùi

Hình 9.6 trình bày một thuật toán suy luận lùi cho các mệnh đề xác định. FOL-BC-ASK(*KB*, *goal*) sẽ được chứng minh nếu cơ sở tri thức chứa một luật có dạng $lhs \Rightarrow goal$, trong đó *lhs* (vế trái) là một danh sách các mệnh đề kết hợp. Một sự kiện nguyên tử như *American(West)* được coi là một mệnh đề có *lhs* là danh sách rỗng. Một truy vấn chứa biến có thể được chứng minh bằng nhiều cách. Ví dụ, truy vấn *Person(x)* có thể được chứng minh với phép thế $\{x/John\}$ hoặc $\{x/Richard\}$. Do đó, chúng ta triển khai FOL-BC-ASK như một **generator**—một hàm trả về nhiều lần, mỗi lần đưa ra một kết quả khả dĩ (xem Phụ lục B).

```
function FOL-BC-ASK(KB, query) returns a generator of substitutions  
return FOL-BC-OR(KB, query, { })
```

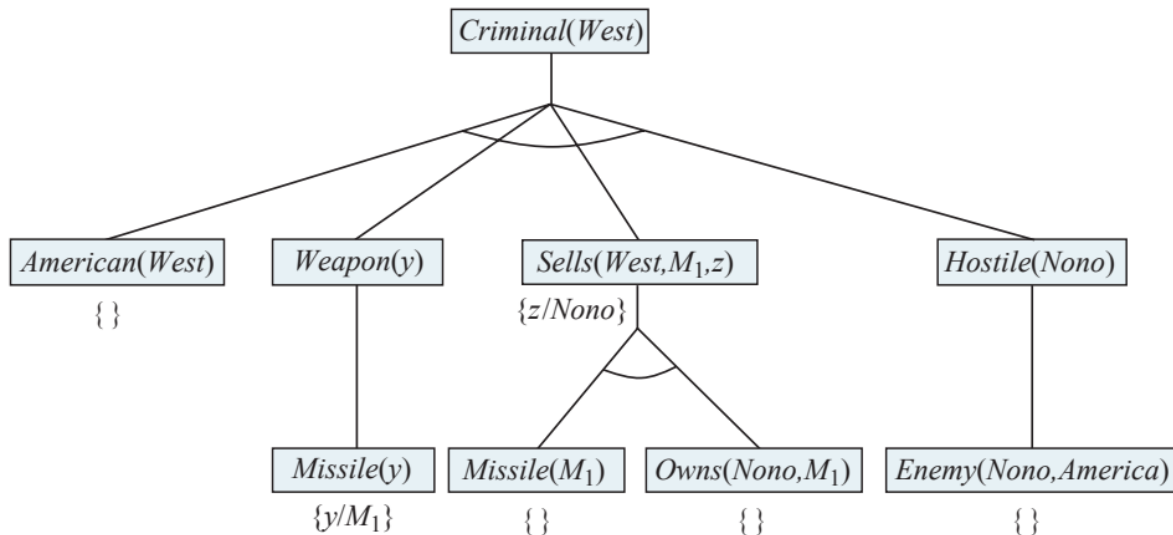
```
function FOL-BC-OR(KB, goal,  $\theta$ ) returns a substitution  
for each rule in FETCH-RULES-FOR-GOAL(KB, goal) do  
  (lhs  $\Rightarrow$  rhs)  $\leftarrow$  STANDARDIZE-VARIABLES(rule)  
  for each  $\theta'$  in FOL-BC-AND(KB, lhs, UNIFY(rhs, goal,  $\theta$ )) do  
    yield  $\theta'$ 
```

```
function FOL-BC-AND(KB, goals,  $\theta$ ) returns a substitution  
if  $\theta = failure$  then return  
else if LENGTH(goals) = 0 then yield  $\theta$   
else  
  first, rest  $\leftarrow$  FIRST(goals), REST(goals)  
  for each  $\theta'$  in FOL-BC-OR(KB, SUBST( $\theta$ , first),  $\theta$ ) do  
    for each  $\theta''$  in FOL-BC-AND(KB, rest,  $\theta'$ ) do  
      yield  $\theta''$ 
```

Hình 9.6 Một thuật toán suy diễn lùi (backward-chaining) đơn giản cho các cơ sở tri thức bậc nhất (first-order knowledge bases).

Suy luận lùi là một dạng tìm kiếm **AND/OR**—phần **OR** vì mục tiêu truy vấn có thể được chứng minh bằng bất kỳ luật nào trong cơ sở tri thức, và phần **AND** vì tất cả các mệnh đề kết hợp trong *lhs* của một mệnh đề phải được chứng minh. FOL-BC-OR hoạt động bằng cách lấy tất cả các mệnh đề có thể kết hợp với mục tiêu, chuẩn hóa các biến trong mệnh đề thành các biến mới, và sau đó, nếu *rhs* của mệnh đề thực sự kết hợp được với mục tiêu, chứng minh từng mệnh đề trong *lhs* bằng FOL-BC-AND. Hàm này làm việc bằng cách chứng minh lần lượt từng mệnh đề, theo dõi phép thế

tích lũy trong quá trình thực hiện. Hình 9.7 là cây chứng minh để suy ra $Criminal(West)$ từ các mệnh đề (9.3) đến (9.10).



Hình 9.7 Cây chứng minh (proof tree) được xây dựng bằng suy diễn lùi (backward chaining) để chứng minh rằng West là tội phạm. Cây này nên được đọc theo chiều sâu, từ trái sang phải. Để chứng minh $Criminal(West)$, chúng ta phải chứng minh bốn mệnh đề liên kết (conjuncts) bên dưới nó. Một số trong số này đã có trong cơ sở tri thức (knowledge base), và những mệnh đề khác yêu cầu suy diễn lùi tiếp. Các phép gán (bindings) cho mỗi lần hợp nhất (unification) thành công được hiển thị bên cạnh mục tiêu con (subgoal) tương ứng. Lưu ý rằng khi một mục tiêu con trong một mệnh đề liên kết thành công, phép thế (substitution) của nó sẽ được áp dụng cho các mục tiêu con tiếp theo. Do đó, khi thuật toán **FOL-BC-ASK** đến mệnh đề liên kết cuối cùng, ban đầu là $Hostile(z)$, thì z đã được gán với **Nono**.

Suy luận lùi, như chúng ta đã viết, rõ ràng là một thuật toán tìm kiếm theo chiều sâu. Điều này có nghĩa là yêu cầu không gian của nó tỷ lệ tuyến tính với kích thước của chứng minh. Nó cũng có nghĩa là suy luận lùi (không như suy luận tiến) gặp vấn đề với các trạng thái lặp và tính không đầy đủ. Mặc dù có những hạn chế này, suy luận lùi đã trở nên phổ biến và hiệu quả trong các ngôn ngữ lập trình logic.

9.4.2 Lập trình logic

Lập trình logic là một công nghệ tiệm cận lý tưởng khai báo được mô tả trong Chương 7: các hệ thống nên được xây dựng bằng cách biểu diễn tri thức trong một ngôn ngữ hình thức và các vấn đề nên được giải quyết bằng cách chạy các quá trình suy luận trên tri thức đó. Lý tưởng này được tóm tắt trong phương trình của Robert Kowalski:

Algorithm = Logic + Control.

Prolog là ngôn ngữ lập trình logic được sử dụng rộng rãi nhất. Nó chủ yếu được sử dụng như một ngôn ngữ tạo mẫu nhanh và cho các nhiệm vụ thao tác ký hiệu như viết trình biên dịch (Van Roy, 1990) và phân tích ngôn ngữ tự nhiên (Pereira và Warren, 1980). Nhiều hệ chuyên gia đã được viết bằng Prolog trong các lĩnh vực pháp lý, y tế, tài chính và các lĩnh vực khác.

Các chương trình Prolog là tập hợp các mệnh đề xác định được viết bằng một ký hiệu hơi khác so với logic bậc nhất tiêu chuẩn. Prolog sử dụng chữ in hoa cho biến và chữ thường cho hằng—ngược lại với quy ước của chúng ta về logic. Dấu phẩy phân tách các mệnh đề kết hợp trong một mệnh đề, và mệnh đề được viết “ngược” so với những gì chúng ta quen thuộc; thay vì $(A \ B \ C)$ trong Prolog chúng ta có $(C := A, B)$. Dưới đây là một ví dụ điển hình:

```
criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).
```

Trong Prolog, ký hiệu $[E|L]$ biểu thị một danh sách có phần tử đầu tiên là E và phần còn lại là L. Dưới đây là một chương trình Prolog cho $\text{append}(X,Y,Z)$, thành công nếu danh sách Z là kết quả của việc nối danh sách X và Y:

```
append([], Y, Y).  
append([A|X], Y, [A|Z]) :- append(X, Y, Z).
```

Bằng tiếng Anh, chúng ta có thể đọc các mệnh đề này như sau: (1) nối danh sách rỗng và danh sách Y tạo ra cùng danh sách Y, và (2) $[A|Z]$ là kết quả của việc nối $[A|X]$ và Y, với điều kiện Z là kết quả của việc nối X và Y. Trong hầu hết các ngôn ngữ cấp cao, chúng ta có thể viết một hàm đệ quy tương tự mô tả cách nối hai danh sách. Tuy nhiên, định nghĩa Prolog thực sự mạnh mẽ hơn, vì nó mô tả một quan hệ giữa ba đối số, thay vì một hàm được tính từ hai đối số. Ví dụ, chúng ta có thể hỏi truy vấn $\text{append}(X, Y, [1,2,3])$: hai danh sách nào có thể được nối để tạo ra $[1,2,3]$? Prolog trả về các giải pháp:

```
X = []    Y = [1,2,3];  
X = [1]   Y = [2,3];  
X = [1,2] Y = [3];  
X = [1,2,3] Y = [].
```

Việc thực thi các chương trình Prolog được thực hiện thông qua suy luận lùi theo chiều sâu, trong đó các mệnh đề được thử theo thứ tự chúng được viết trong cơ sở tri thức. Thiết kế của Prolog thể hiện sự thỏa hiệp giữa tính khai báo và hiệu quả thực thi. Một số khía cạnh của Prolog nằm ngoài suy luận logic tiêu chuẩn:

- Prolog sử dụng ngữ nghĩa cơ sở dữ liệu (Section 8.2.8) thay vì ngữ nghĩa bậc nhất, và điều này thể hiện trong cách xử lý đẳng thức và phủ định (xem Section 9.4.4).
- Có một tập hợp các hàm tích hợp sẵn cho số học. Các ký hiệu nguyên tử sử dụng các ký hiệu hàm này được “chứng minh” bằng cách thực thi mã thay vì suy luận thêm. Ví dụ, mục tiêu “X is 4+3” thành công với X được gán là 7. Mặt khác, mục tiêu “5 is X+Y” thất bại, vì các hàm tích hợp không giải được phương trình tùy ý.
- Có các vị từ tích hợp có tác dụng phụ khi thực thi. Chúng bao gồm các vị từ nhập/xuất và các vị từ `assert/retract` để sửa đổi cơ sở tri thức. Các vị từ như vậy không có trong logic và có thể tạo ra kết quả khó hiểu—ví dụ, nếu các sự kiện được khẳng định trong một nhánh của cây chứng minh cuối cùng thất bại.
- Kiểm tra xuất hiện (occur check) bị bỏ qua trong thuật toán hợp nhất của Prolog. Điều này có nghĩa là một số suy luận không chính xác có thể được thực hiện; những điều này hầu như không bao giờ là vấn đề trong thực tế.
- Prolog sử dụng tìm kiếm suy luận lùi theo chiều sâu mà không kiểm tra đệ quy vô hạn. Điều này tạo ra một ngôn ngữ lập trình dễ sử dụng và rất nhanh khi được sử dụng đúng.

cách, nhưng nó có nghĩa là một số chương trình trông giống như logic hợp lệ sẽ không bao giờ dừng.

9.4.3 Suy luận dư thừa và vòng lặp vô hạn

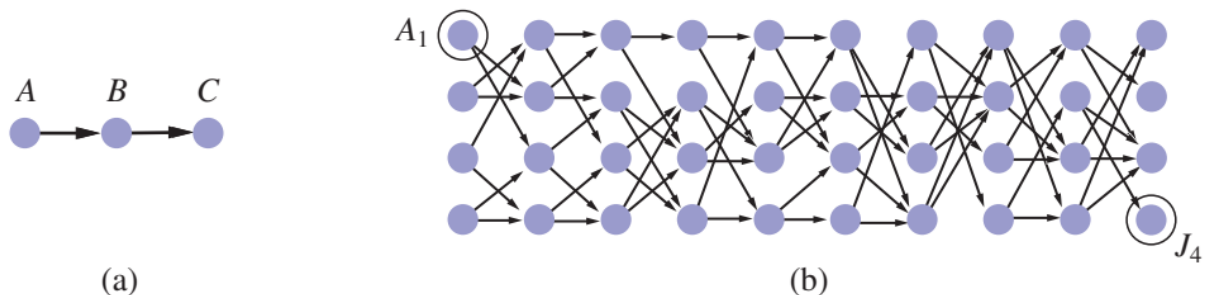
Giờ chúng ta chuyển sang điểm yếu của Prolog: sự không phù hợp giữa tìm kiếm theo chiều sâu và các cây tìm kiếm bao gồm các trạng thái lặp lại và các đường dẫn vô hạn. Xét chương trình logic sau đây quyết định xem có tồn tại đường đi giữa hai điểm trên đồ thị có hướng hay không:

```
path(X, Z) :- link(X, Z).  
path(X, Z) :- path(X, Y), link(Y, Z).
```

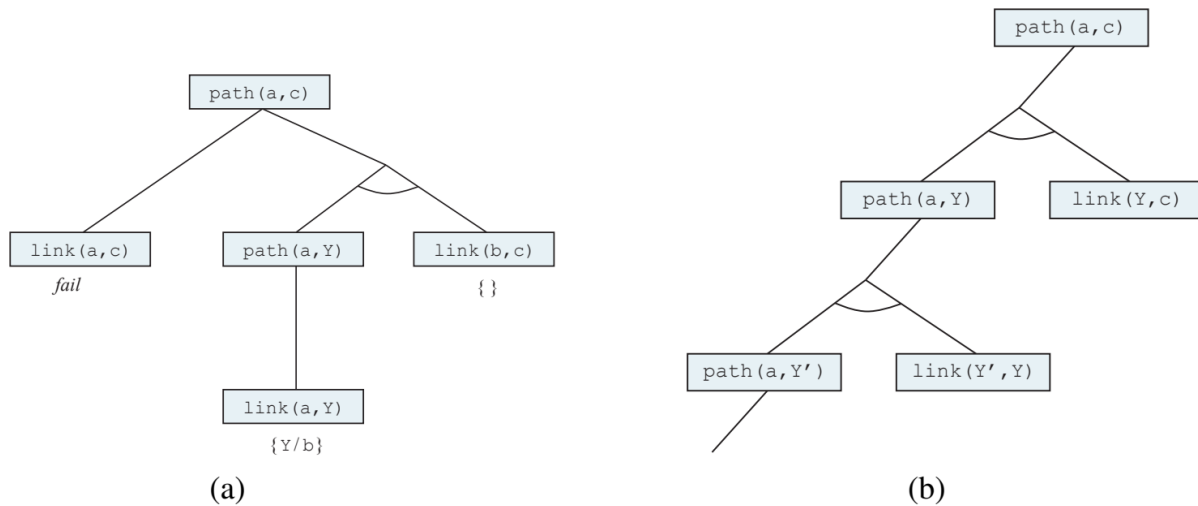
Một đồ thị đơn giản ba nút, được mô tả bởi các sự kiện $\text{link}(a, b)$ và $\text{link}(b, c)$, được hiển thị trong Hình 9.8(a). Với chương trình này, truy vấn $\text{path}(a, c)$ tạo ra cây chứng minh như trong Hình 9.9(a). Mặt khác, nếu chúng ta đặt hai mệnh đề theo thứ tự:

```
path(X, Z) :- path(X, Y), link(Y, Z).  
path(X, Z) :- link(X, Z).
```

thì Prolog sẽ đi theo đường dẫn vô hạn như trong Hình 9.9(b). Do đó, Prolog không đầy đủ như một bộ chứng minh định lý cho các mệnh đề xác định—ngay cả đối với các chương trình Datalog, như ví dụ này cho thấy—vì, đối với một số cơ sở tri thức, nó không thể chứng minh các câu được suy ra. Lưu ý rằng suy luận tiến không gặp vấn đề này: một khi $\text{path}(a, b)$, $\text{path}(b, c)$ và $\text{path}(a, c)$ được suy ra, suy luận tiến sẽ dừng.



Hình 9.8 (a) Việc tìm một đường đi từ A đến C có thể khiến Prolog rơi vào vòng lặp vô hạn. (b) Một đồ thị trong đó mỗi nút được kết nối với hai nút con ngẫu nhiên ở lớp tiếp theo. Việc tìm một đường đi từ A_1 đến J_4 đòi hỏi 877 suy luận.



Hình 9.9 (a) Bằng chứng rằng tồn tại một đường đi từ A đến C. (b) Cây chứng minh vô hạn được tạo ra khi các mệnh đề được sắp xếp theo thứ tự “sai”.

Suy luận lùi theo chiều sâu cũng có vấn đề với các tính toán dư thừa. Ví dụ, khi tìm đường đi từ A1 đến J4 trong Hình 9.8(b), Prolog thực hiện 877 suy luận, hầu hết trong số đó liên quan đến việc tìm tất cả các đường đi có thể đến các nút mà từ đó mục tiêu không thể đạt được. Điều này tương tự như vấn đề trạng thái lặp được thảo luận trong Chương 3. Tổng số suy luận có thể tăng theo cấp số nhân so với số sự kiện cơ bản được tạo ra. Nếu chúng ta áp dụng suy luận tiến thay thế, tối đa (n^2) sự kiện `path(X, Y)` có thể được tạo ra liên kết n nút. Đối với vấn đề trong Hình 9.8(b), chỉ cần 62 suy luận.

Suy luận tiến trên các bài toán tìm kiếm đồ thị là một ví dụ về **quy hoạch động**, trong đó các giải pháp cho các bài toán con được xây dựng dần từ các bài toán con nhỏ hơn và được lưu vào bộ nhớ đệm để tránh tính toán lại. Chúng ta có thể đạt được hiệu ứng tương tự trong một hệ thống suy luận lùi, ngoại trừ việc ở đây chúng ta chia nhỏ các mục tiêu lớn thành các mục tiêu nhỏ hơn, thay vì xây dựng chúng.

Dù bằng cách nào, việc lưu trữ các kết quả trung gian để tránh trùng lặp là chìa khóa. Đây là cách tiếp cận của các hệ thống lập trình logic có bảng (**tabled logic programming**), sử dụng các cơ chế lưu trữ và truy xuất hiệu quả. Lập trình logic có bảng kết hợp tính định hướng mục tiêu của suy luận lùi với hiệu quả quy hoạch động của suy luận tiến. Nó cũng đầy đủ cho các cơ sở tri thức Datalog, có nghĩa là người lập trình không cần lo lắng nhiều về các vòng lặp vô hạn. (Tuy nhiên, vẫn có thể gặp vòng lặp vô hạn với các vị từ như `father(X, Y)` tham chiếu đến một số lượng đối tượng không giới hạn.)

9.4.4 Ngữ nghĩa cơ sở dữ liệu của Prolog

Prolog sử dụng ngữ nghĩa cơ sở dữ liệu, như đã thảo luận trong Mục 8.2.8. Giả định tên duy nhất nói rằng mọi hằng số Prolog và mọi hạng tử cơ bản đều tham chiếu đến một đối tượng riêng biệt, và giả định thế giới đóng nói rằng chỉ những câu được suy ra từ cơ sở tri thức mới là đúng. Không có cách nào để khẳng định một câu là sai trong Prolog. Điều này làm cho Prolog kém biểu cảm hơn logic bậc nhất, nhưng nó là một phần giúp Prolog hiệu quả và ngắn gọn hơn. Xét các khẳng định sau về một số khóa học:

$\text{Course}(\text{CS}, 101), \text{Course}(\text{CS}, 102), \text{Course}(\text{CS}, 106), \text{Course}(\text{EE}, 101).$ (9.11)

Theo giả định tên duy nhất, CS và EE là khác nhau (cũng như 101, 102 và 106), vì vậy điều này có nghĩa là có bốn khóa học riêng biệt. Theo giả định thế giới đóng, không có khóa học nào khác, vì vậy chính xác có bốn khóa học. Nhưng nếu đây là các khẳng định trong FOL thay vì ngữ nghĩa cơ sở dữ liệu, thì tất cả những gì chúng ta có thể nói là có từ một đến vô hạn khóa học. Đó là vì các khẳng định (trong FOL) không phủ nhận khả năng tồn tại các khóa học khác không được đề cập, cũng như không nói rằng các khóa học được đề cập là khác nhau. Nếu chúng ta muốn dịch Phương trình (9.11) sang FOL, chúng ta sẽ nhận được câu sau:

$\text{Course}(d, n) \Leftrightarrow (d = \text{CS} \wedge n = 101) \vee (d = \text{CS} \wedge n = 102) \vee (d = \text{CS} \wedge n = 106) \vee (d = \text{EE} \wedge n = 101).$ (9.12)

Đây được gọi là **phần bù (completion)** của Phương trình (9.11). Nó biểu diễn trong FOL ý tưởng rằng có tối đa bốn khóa học. Để biểu diễn trong FOL ý tưởng rằng có ít nhất bốn khóa học, chúng ta cần viết phần bù của vị từ đẳng thức:

$x = y \Leftrightarrow (x = \text{CS} \wedge y = \text{CS}) \vee (x = \text{EE} \wedge y = \text{EE}) \vee (x = 101 \wedge y = 101) \vee (x = 102 \wedge y = 102) \vee (x = 106 \wedge y = 106).$

Phần bù hữu ích để hiểu ngữ nghĩa cơ sở dữ liệu, nhưng với mục đích thực tế, nếu vấn đề của bạn có thể được mô tả bằng ngữ nghĩa cơ sở dữ liệu, hiệu quả hơn là suy luận bằng Prolog hoặc một số hệ thống ngữ nghĩa cơ sở dữ liệu khác, thay vì dịch sang FOL và suy luận bằng một bộ chứng minh định lý FOL đầy đủ.

9.4.5 Lập trình logic ràng buộc

Trong phần thảo luận về suy luận tiến (Mục 9.3), chúng ta đã chỉ ra cách các bài toán thỏa mãn ràng buộc (CSP) có thể được mã hóa thành các mệnh đề xác định. Prolog tiêu chuẩn giải quyết các bài toán như vậy giống hệt như thuật toán quay lui được đưa ra trong Hình 5.5.

Vì quay lui liệt kê các miền giá trị của biến, nó chỉ hoạt động cho các CSP có miền hữu hạn. Theo thuật ngữ Prolog, phải có một số lượng hữu hạn các giải pháp cho bất kỳ mục tiêu nào với các biến chưa ràng buộc. (Ví dụ: bài toán tô màu bản đồ trong đó mỗi biến có thể nhận một trong bốn màu khác nhau.) Các CSP có miền vô hạn—ví dụ: với các biến giá trị nguyên hoặc thực—yêu cầu các thuật toán hoàn toàn khác, chẳng hạn như lan truyền ràng buộc hoặc quy hoạch tuyến tính.

Xét ví dụ sau. Chúng ta định nghĩa $\text{triangle}(X, Y, Z)$ như một vị từ đúng nếu ba đối số là các số thỏa mãn bất đẳng thức tam giác:

$\text{triangle}(X, Y, Z) :-$
 $X > 0, Y > 0, Z > 0, X + Y > Z, Y + Z > X, X + Z > Y.$

Nếu chúng ta hỏi Prolog truy vấn $\text{triangle}(3, 4, 5)$, nó sẽ thành công. Mặt khác, nếu chúng ta hỏi $\text{triangle}(3, 4, Z)$, sẽ không tìm thấy giải pháp nào, vì mục tiêu con $Z > 0$ không thể được xử lý bởi Prolog; chúng ta không thể so sánh một giá trị chưa ràng buộc với 0.

Lập trình logic ràng buộc (Constraint Logic Programming - CLP) cho phép các biến bị ràng buộc thay vì bị buộc. Một giải pháp CLP là tập hợp các ràng buộc cụ thể nhất trên các biến truy vấn có thể được suy ra từ cơ sở tri thức. Ví dụ, giải pháp cho truy vấn $\text{triangle}(3, 4, Z)$ là ràng buộc

($7 > Z > 1$). Các chương trình logic tiêu chuẩn chỉ là một trường hợp đặc biệt của CLP trong đó các ràng buộc giải pháp phải là các ràng buộc đẳng thức—tức là các phép gán.

Các hệ thống CLP kết hợp nhiều thuật toán giải ràng buộc khác nhau cho các ràng buộc được phép trong ngôn ngữ. Ví dụ, một hệ thống cho phép các bất đẳng thức tuyến tính trên các biến giá trị thực có thể bao gồm một thuật toán quy hoạch tuyến tính để giải các ràng buộc đó. Các hệ thống CLP cũng áp dụng cách tiếp cận linh hoạt hơn để giải các truy vấn logic tiêu chuẩn. Ví dụ, thay vì quay lui theo chiều sâu, từ trái sang phải, chúng có thể sử dụng bất kỳ thuật toán hiệu quả nào được thảo luận trong Chương 5, bao gồm sắp xếp mệnh đề kết hợp theo heuristic, quay lui có nhảy (backjumping), điều kiện cắt (cutset conditioning), v.v. Do đó, các hệ thống CLP kết hợp các yếu tố của thuật toán thỏa mãn ràng buộc, lập trình logic và cơ sở dữ liệu suy diễn.

Một số hệ thống cho phép lập trình viên kiểm soát nhiều hơn thứ tự tìm kiếm để suy luận đã được định nghĩa. Ngôn ngữ MRS (Genesereth và Smith, 1981; Russell, 1985) cho phép lập trình viên viết các siêu luật (metarules) để xác định mệnh đề kết hợp nào được thử trước. Người dùng có thể viết một luật nói rằng mục tiêu có ít biến nhất nên được thử trước hoặc có thể viết các luật dành riêng cho miền cho các vị từ cụ thể.

9.5 Phương pháp Phân giải

Phương pháp phân giải là họ thứ ba trong ba họ hệ thống logic mà chúng ta xem xét, và là phương pháp duy nhất có thể áp dụng cho bất kỳ cơ sở tri thức nào, không chỉ các mệnh đề xác định. Trong mục 7.5.2, chúng ta đã thấy rằng phương pháp phân giải trong logic mệnh đề là một thủ tục suy diễn đầy đủ cho logic mệnh đề; trong phần này, chúng ta mở rộng nó sang logic bậc nhất.

9.5.1 Dạng chuẩn hội (CNF) trong logic bậc nhất

Bước đầu tiên là chuyển đổi các câu sang dạng chuẩn hội (CNF)—nghĩa là một hội các mệnh đề, trong đó mỗi mệnh đề là một tuyển các literal. Trong CNF, các literal có thể chứa các biến, được giả định là được lượng tử hóa phổ dụng. Ví dụ, câu:

$$\forall x, y, z \text{ American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$$

sẽ trở thành, trong CNF:

$$\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x).$$

Điểm quan trọng là mọi câu trong logic bậc nhất đều có thể được chuyển đổi thành một câu CNF tương đương về mặt suy diễn.

Quy trình chuyển đổi sang CNF tương tự như trường hợp logic mệnh đề, đã được trình bày trong mục 7.5.2. Sự khác biệt chính nằm ở việc cần loại bỏ các lượng tử tồn tại. Chúng ta minh họa quy trình bằng cách dịch câu: "Mọi người yêu tất cả động vật sẽ được ai đó yêu", hay:

$$\forall x[\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)].$$

Các bước như sau:

- **Loại bỏ phép kéo theo:** Thay $P \Rightarrow Q$ bằng $\neg P \vee Q$. Trong câu mẫu, điều này cần được thực hiện hai lần:

$$\forall x \neg [\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists y \text{Loves}(y, x)].$$

- **Đưa phủ định vào trong:** Ngoài các quy tắc thông thường cho các phép kết nối bị phủ định, chúng ta cần các quy tắc cho các lượng tử bị phủ định:

$$\neg \forall x p \quad \text{trở thành} \quad \exists x \neg p \quad \neg \exists x p \quad \text{trở thành} \quad \forall x \neg p.$$

Câu của chúng ta trải qua các biến đổi sau:

$$\forall x [\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists y \text{Loves}(y, x)].$$

- **Chuẩn hóa biến:** Đối với các câu sử dụng cùng tên biến nhiều lần, chúng ta đổi tên một trong các biến. Ví dụ:

$$\forall x [\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists z \text{Loves}(z, x)].$$

- **Skolem hóa:** Quá trình loại bỏ các lượng tử tồn tại bằng cách thay thế chúng bằng các hằng số hoặc hàm Skolem. Ví dụ:

$$\forall x [\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee \text{Loves}(G(x), x).$$

Ở đây, F và G là các hàm Skolem.

- **Loại bỏ lượng tử phổ dụng:** Tất cả các biến còn lại đều được lượng tử hóa phổ dụng, nên chúng ta có thể bỏ các lượng tử này:

$$[\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee \text{Loves}(G(x), x).$$

- **Phân phối phép tuyển ($\vee$) trên phép hội ($\wedge$):**

$$[\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)] \wedge [\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)].$$

Câu này giờ đã ở dạng CNF và gồm hai mệnh đề. Mặc dù khó đọc hơn câu gốc, nhưng quá trình chuyển đổi có thể được tự động hóa dễ dàng.

9.5.2 Quy tắc suy diễn phân giải

Quy tắc phân giải cho các mệnh đề bậc nhất là phiên bản nâng cao của quy tắc phân giải trong logic mệnh đề. Hai mệnh đề, được giả định là đã được chuẩn hóa để không có biến chung, có thể được phân giải nếu chúng chứa các literal bù nhau. Các literal trong logic mệnh đề là bù nhau nếu một cái là phủ định của cái kia; trong logic bậc nhất, các literal là bù nhau nếu chúng hợp nhất được với phủ định của nhau. Do đó, chúng ta có:

$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m_1 \vee \dots \vee m_n}{\text{SUBST}(\theta, \ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)}$$

trong đó $\text{UNIFY}(\ell_i, \neg m_j) = \theta$.

Ví dụ, chúng ta có thể phân giải hai mệnh đề:

$$[\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)] \quad \text{và} \quad [\neg \text{Loves}(u, v) \vee \neg \text{Kills}(u, v)]$$

bằng cách loại bỏ các literal bù nhau $\text{Loves}(G(x), x)$ và $\neg \text{Loves}(u, v)$, với bộ hợp nhất $\theta = \{u/G(x), v/x\}$, để tạo ra mệnh đề phân giải:

$$[\text{Animal}(F(x)) \vee \neg \text{Kills}(G(x), x)].$$

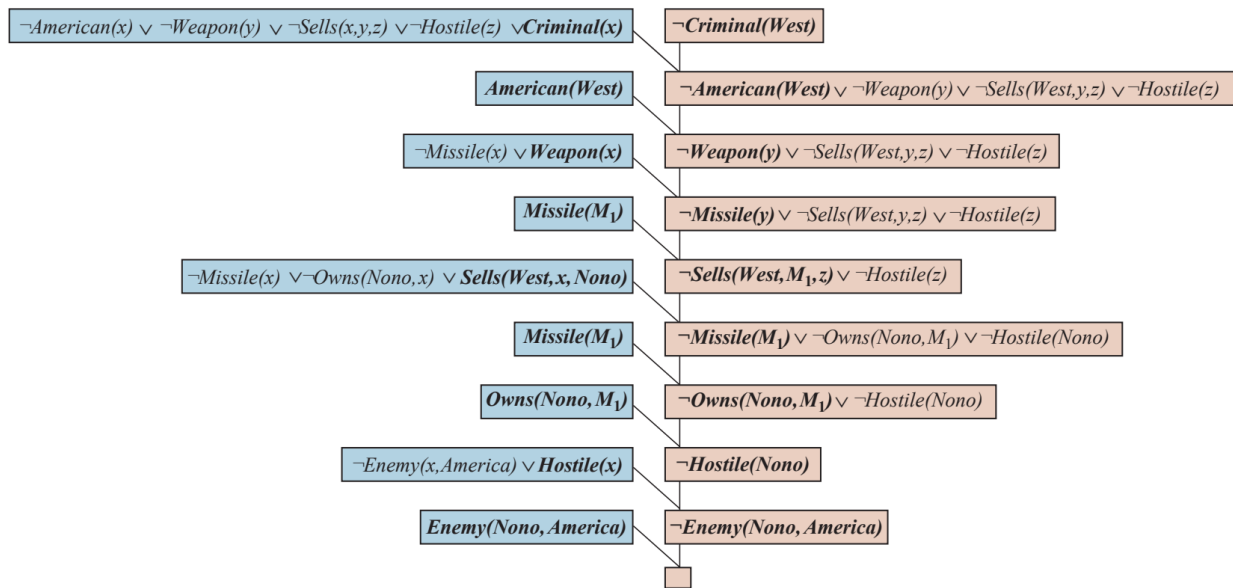
Quy tắc này được gọi là quy tắc phân giải nhị phân vì nó phân giải chính xác hai literal. Tuy nhiên, quy tắc phân giải nhị phân tự nó không tạo ra một thủ tục suy diễn đầy đủ. Quy tắc phân giải đầy đủ phân giải các tập con của các literal trong mỗi mệnh đề có thể hợp nhất được. Một cách tiếp cận khác là mở rộng phép nhân tử (factoring)—loại bỏ các literal dư thừa—sang trường hợp logic bậc nhất. Phép nhân tử trong logic mệnh đề giảm hai literal giống nhau thành một; trong logic bậc nhất, nó giảm hai literal thành một nếu chúng có thể hợp nhất. Bộ hợp nhất phải được áp dụng cho toàn bộ mệnh đề. Sự kết hợp giữa phân giải nhị phân và phép nhân tử là đầy đủ.

9.5.3 Ví dụ minh họa

Phương pháp phân giải chứng minh rằng $KB \models \alpha$ bằng cách chứng minh rằng $KB \wedge \neg \alpha$ không thỏa mãn được—nghĩa là, bằng cách suy ra mệnh đề rỗng. Cách tiếp cận thuật toán giống hệt với trường hợp mệnh đề, được mô tả trong Hình 7.13, vì vậy chúng ta không cần lặp lại ở đây. Thay vào đó, chúng tôi đưa ra hai ví dụ chứng minh. Ví dụ đầu tiên là vụ án từ Mục 9.3. Các câu ở dạng CNF là:

1. $\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x)$
2. $\neg \text{Missile}(x) \vee \neg \text{Owns}(\text{Nono}, x) \vee \text{Sells}(\text{West}, x, \text{Nono})$
3. $\neg \text{Enemy}(x, \text{America}) \vee \text{Hostile}(x)$
4. $\neg \text{Missile}(x) \vee \text{Weapon}(x)$
5. $\text{Owns}(\text{Nono}, M_1)$
6. $\text{Missile}(M_1)$
7. $\text{American}(\text{West})$
8. $\text{Enemy}(\text{Nono}, \text{America})$.

Chúng tôi cũng bao gồm mục tiêu phủ định $\neg \text{Criminal}(\text{West})$. Chứng minh bằng phương pháp phân giải được thể hiện trong Hình 9.10. Lưu ý cấu trúc: một "xương sống" duy nhất bắt đầu bằng mệnh đề mục tiêu, phân giải với các mệnh đề từ cơ sở tri thức cho đến khi mệnh đề rỗng được sinh ra. Đây là đặc trưng của phương pháp phân giải trên các cơ sở tri thức mệnh đề Horn. Trên thực tế, các mệnh đề dọc theo xương sống chính tương ứng chính xác với các giá trị liên tiếp của biến mục tiêu trong thuật toán suy diễn lùi của Hình 9.6. Điều này là do chúng tôi luôn chọn phân giải với một mệnh đề mà literal dương hợp nhất được với literal ngoài cùng bên trái của mệnh đề "hiện tại" trên xương sống; đây chính xác là những gì xảy ra trong suy diễn lùi. Do đó, suy diễn lùi chỉ là một trường hợp đặc biệt của phương pháp phân giải với một chiến lược kiểm soát cụ thể để quyết định bước phân giải nào sẽ thực hiện tiếp theo.



Hình 9.10 Một chứng minh bằng phép giải (resolution proof) rằng West là tội phạm. Tại mỗi bước giải, các vị từ (literals) hợp nhất được in đậm và mệnh đề với vị từ dương được tô màu xanh lam.

Ví dụ thứ hai của chúng tôi sử dụng phép Skolem hóa và liên quan đến các mệnh đề không phải là mệnh đề xác định. Điều này dẫn đến một cấu trúc chứng minh phức tạp hơn. Trong tiếng Anh:

1. Mọi người yêu tất cả động vật sẽ được ai đó yêu.
2. Bất kỳ ai giết một con vật sẽ không được ai yêu.
3. Jack yêu tất cả động vật.
4. Jack hoặc Curiosity đã giết con mèo tên Tuna.
5. Có phải Curiosity đã giết con mèo?

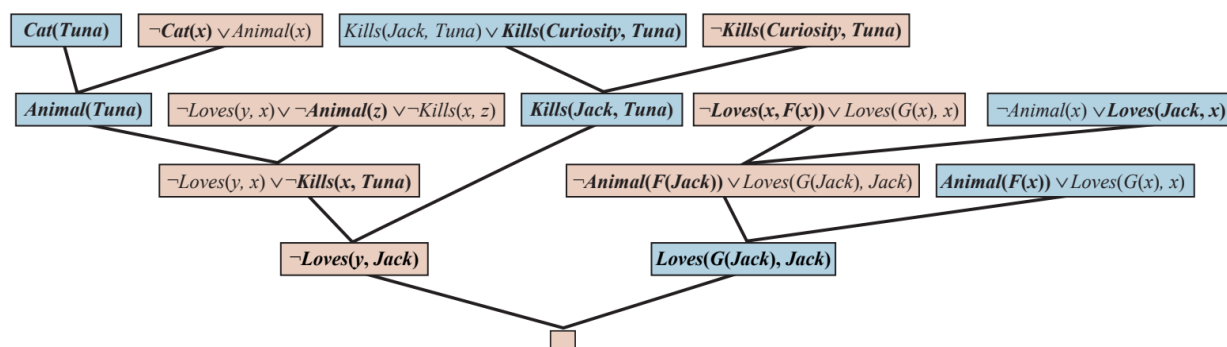
Đầu tiên, chúng tôi biểu diễn các câu gốc, một số kiến thức nền và mục tiêu phủ định G trong logic bậc nhất:

- A. $\forall x[\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x,y)] \Rightarrow [\exists y \text{ Loves}(y,x)]$
- B. $\forall x[\exists z \text{ Animal}(z) \wedge \text{Kills}(x,z)] \Rightarrow [\forall y \neg \text{Loves}(y,x)]$
- C. $\forall x \text{ Animal}(x) \Rightarrow \text{Loves}(\text{Jack},x)$
- D. $\text{Kills}(\text{Jack},\text{Tuna}) \vee \text{Kills}(\text{Curiosity},\text{Tuna})$
- E. $\text{Cat}(\text{Tuna})$
- F. $\forall x \text{ Cat}(x) \Rightarrow \text{Animal}(x)$
- $\neg G. \neg \text{Kills}(\text{Curiosity},\text{Tuna})$

Bây giờ chúng tôi áp dụng quy trình chuyển đổi để chuyển mỗi câu sang dạng CNF:

- **A1.** $Animal(F(x)) \vee Loves(G(x), x)$
- **A2.** $\neg Loves(x, F(x)) \vee Loves(G(x), x)$
- **B.** $\neg Loves(y, x) \vee \neg Animal(z) \vee \neg Kills(x, z)$
- **C.** $\neg Animal(x) \vee Loves(Jack, x)$
- **D.** $Kills(Jack, Tuna) \vee Kills(Curiosity, Tuna)$
- **E.** $Cat(Tuna)$
- **F.** $\neg Cat(x) \vee Animal(x)$
- **$\neg G.$** $\neg Kills(Curiosity, Tuna)$

Chứng minh bằng phương pháp phân giải rằng Curiosity đã giết con mèo được trình bày trong Hình 9.11. Bằng tiếng Anh, chứng minh có thể được diễn giải như sau:



Hình 9.11 Một chứng minh bằng phép giải (resolution proof) rằng "Sự tò mò đã giết chết con mèo". Lưu ý việc sử dụng **factoring** (nhân tử hóa) trong quá trình suy ra mệnh đề $Loves(G(Jack), Jack)$. Cũng cần lưu ý ở phía trên bên phải, phép hợp nhất (unification) của $Loves(x, F(x))$ và $Loves(Jack, x)$ chỉ có thể thành công sau khi các biến đã được **standardized apart** (chuẩn hóa riêng biệt).

Giả sử Curiosity không giết Tuna. Chúng ta biết rằng hoặc Jack hoặc Curiosity đã làm điều đó; do đó, Jack phải là thủ phạm. Bây giờ, Tuna là một con mèo và mèo là động vật, vì vậy Tuna là một con vật. Vì bất kỳ ai giết một con vật đều không được ai yêu, chúng ta biết rằng không ai yêu Jack. Mặt khác, Jack yêu tất cả động vật, vì vậy ai đó yêu anh ta; do đó, chúng ta có một mâu thuẫn. Vì vậy, Curiosity đã giết con mèo.

Chứng minh trả lời câu hỏi "Có phải Curiosity đã giết con mèo?" nhưng thường chúng ta muốn đặt các câu hỏi tổng quát hơn, chẳng hạn như "Ai đã giết con mèo?" Phương pháp phân giải có thể làm điều này, nhưng cần thêm một chút công việc để có được câu trả lời. Mục tiêu là $\exists w Kills(w, Tuna)$, khi bị phủ định, trở thành $\neg Kills(w, Tuna)$ trong CNF. Lập lại chứng minh trong Hình 9.11 với mục tiêu phủ định mới, chúng ta thu được một cây chứng minh tương tự, nhưng với phép thay thế $\{w/Curiosity\}$ trong một trong các bước. Vì vậy, trong trường hợp này,

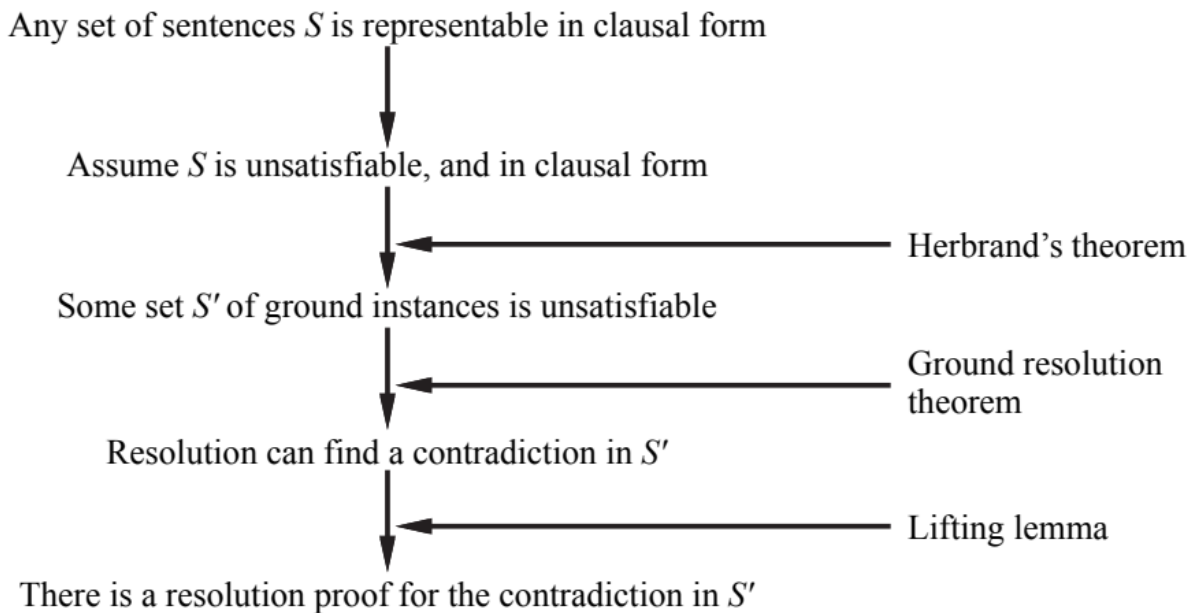
việc tìm ra ai đã giết con mèo chỉ là vấn đề theo dõi các ràng buộc cho các biến truy vấn trong chứng minh. Thật không may, phương pháp phân giải đôi khi có thể tạo ra các chứng minh không mang tính xây dựng cho các mục tiêu tồn tại, nơi chúng ta biết một truy vấn là đúng, nhưng không có một ràng buộc duy nhất cho biến.

9.5.4 Tính đầy đủ của phương pháp phân giải

Phần này trình bày một chứng minh về tính đầy đủ của phương pháp phân giải. Những ai sẵn lòng chấp nhận điều này có thể bỏ qua phần này một cách an toàn.

Chúng tôi sẽ chứng minh rằng phương pháp phân giải có tính chất **đầy đủ trong việc phủ định** (refutation-complete), nghĩa là nếu một tập hợp các câu không thỏa mãn được, thì phương pháp phân giải luôn có thể suy ra một mâu thuẫn. Phương pháp phân giải không thể được sử dụng để sinh ra tất cả các hệ quả logic của một tập hợp các câu, nhưng nó có thể được sử dụng để chứng minh rằng một câu nhất định được suy ra từ tập hợp các câu đó. Do đó, nó có thể được sử dụng để tìm tất cả các câu trả lời cho một câu hỏi $Q(x)$ bằng cách chứng minh rằng $KB \wedge \neg Q(x)$ không thỏa mãn được.

Chúng tôi coi như đã biết rằng bất kỳ câu nào trong logic bậc nhất (không bao gồm đẳng thức) đều có thể được viết lại thành một tập hợp các mệnh đề ở dạng CNF. Điều này có thể được chứng minh bằng quy nạp trên dạng của câu, sử dụng các câu nguyên tố làm trường hợp cơ sở (Davis và Putnam, 1960). Mục tiêu của chúng tôi do đó là chứng minh điều sau: nếu S là một tập hợp các mệnh đề không thỏa mãn được, thì việc áp dụng một số hữu hạn các bước phân giải cho S sẽ dẫn đến một mâu thuẫn.



Hình 9.12 Cấu trúc của một chứng minh tính đầy đủ (completeness proof) cho phép giải (resolution).

Phác thảo chứng minh của chúng tôi dựa trên chứng minh gốc của Robinson với một số đơn giản hóa từ Genesereth và Nilsson (1987). Cấu trúc cơ bản của chứng minh (Hình 9.12) như sau:

1. Đầu tiên, chúng tôi quan sát thấy rằng nếu S không thỏa mãn được, thì tồn tại một tập hợp cụ thể các trường hợp nền (ground instances) của các mệnh đề trong S sao cho tập hợp này cũng không thỏa mãn được (định lý Herbrand).
2. Sau đó, chúng tôi viện dẫn định lý phân giải nền (ground resolution theorem) đã được trình bày trong Chương 7, trong đó nói rằng phương pháp phân giải mệnh đề là đầy đủ đối với các câu nền.
3. Tiếp theo, chúng tôi sử dụng một bổ đề nâng (lifting lemma) để chỉ ra rằng, với bất kỳ chứng minh phân giải mệnh đề nào sử dụng tập hợp các câu nền, đều tồn tại một chứng minh phân giải logic bậc nhất tương ứng sử dụng các câu logic bậc nhất mà từ đó các câu nền được sinh ra.

Để thực hiện bước đầu tiên, chúng ta cần ba khái niệm mới:

- **Vũ trụ Herbrand (Herbrand universe):** Nếu S là một tập hợp các mệnh đề, thì H_S , vũ trụ Herbrand của S , là tập hợp tất cả các hạng tử nền có thể xây dựng được từ:
 - a. Các ký hiệu hàm trong S , nếu có.
 - b. Các ký hiệu hằng trong S , nếu có; nếu không có, thì sử dụng một ký hiệu hằng mặc định, S .

Ví dụ, nếu S chỉ chứa mệnh đề $\neg P(x, F(x, A)) \vee \neg Q(x, A) \vee R(x, B)$, thì H_S là tập hợp vô hạn các hạng tử nền sau:

$$\{A, B, F(A, A), F(A, B), F(B, A), F(B, B), F(A, F(A, A)), \dots\}.$$

- **Sự bão hòa (Saturation):** Nếu S là một tập hợp các mệnh đề và P là một tập hợp các hạng tử nền, thì $P(S)$, sự bão hòa của S đối với P , là tập hợp tất cả các mệnh đề nền thu được bằng cách áp dụng tất cả các phép thế nhất quán có thể có của các hạng tử nền trong P cho các biến trong S .
- **Cơ sở Herbrand (Herbrand base):** Sự bão hòa của một tập hợp S các mệnh đề đối với vũ trụ Herbrand của nó được gọi là cơ sở Herbrand của S , ký hiệu là $H_S(S)$. Ví dụ, nếu S chỉ chứa mệnh đề nêu trên, thì $H_S(S)$ là tập hợp vô hạn các mệnh đề sau:

$$\begin{aligned} &\neg P(A, F(A, A)) \vee \neg Q(A, A) \vee R(A, B) \\ &\neg P(B, F(B, A)) \vee \neg Q(B, A) \vee R(B, B), \neg P(F(A, A), F(F(A, A), A)) \vee \neg Q(F(A, A), A) \\ &\vee R(F(A, A), B), \neg P(F(A, B), F(F(A, B), A)) \vee \neg Q(F(A, B), A) \\ &\vee R(F(A, B), B), \dots \end{aligned}$$

Các định nghĩa này cho phép chúng ta phát biểu một dạng của định lý Herbrand (Herbrand, 1930): Nếu một tập hợp S các mệnh đề không thỏa mãn được, thì tồn tại một tập hợp con hữu hạn của $H_S(S)$ cũng không thỏa mãn được.

Gọi S' là tập hợp con hữu hạn này của các câu nền. Bây giờ, chúng ta có thể viện dẫn định lý phân giải nền (trang 246) để chỉ ra rằng bao đóng phân giải $RC(S')$ chứa mệnh đề rỗng. Nghĩa là, việc chạy phương pháp phân giải mệnh đề đến khi hoàn thành trên S' sẽ suy ra một mâu thuẫn.

Bây giờ chúng ta đã thiết lập rằng luôn tồn tại một chứng minh phân giải liên quan đến một tập hợp con hữu hạn của cơ sở Herbrand của S , bước tiếp theo là chỉ ra rằng có một chứng minh phân giải sử dụng các mệnh đề của chính S , vốn không nhất thiết là các mệnh đề nền. Chúng ta bắt đầu bằng cách xem xét một ứng dụng đơn lẻ của quy tắc phân giải. Robinson đã phát biểu bổ đề sau:

Cho C_1 và C_2 là hai mệnh đề không có biến chung, và cho C_1' và C_2' là các trường hợp nền của C_1 và C_2 . Nếu C' là một phân giải thức của C_1' và C_2' , thì tồn tại một mệnh đề C sao cho (1) C là một phân giải thức của C_1 và C_2 và (2) C' là một trường hợp nền của C .

Định lý Bất toàn của Gödel

Bằng cách mở rộng ngôn ngữ logic bậc nhất một chút để cho phép **sơ đồ quy nạp toán học** trong số học, Kurt Gödel đã chứng minh được trong **định lý bất toàn** của mình rằng tồn tại những câu số học đúng nhưng không thể chứng minh được.

Việc chứng minh định lý bất toàn có phần vượt ra ngoài phạm vi của cuốn sách này, vì nó chiếm ít nhất 30 trang, nhưng chúng ta có thể đưa ra một gợi ý ở đây. Chúng ta bắt đầu với lý thuyết logic về các con số. Trong lý thuyết này, có một hằng số duy nhất, 0, và một hàm duy nhất, S (hàm kế thừa). Trong mô hình dự định, $S(0)$ biểu thị 1, $S(S(0))$ biểu thị 2, và cứ thế; do đó, ngôn ngữ này có tên cho tất cả các số tự nhiên. Từ vựng cũng bao gồm các ký hiệu hàm $+$, $\times$, và $Expt$ (hàm mũ) cùng với các liên từ logic và lượng tử thông thường.

Bước đầu tiên là nhận thấy rằng tập hợp các câu mà chúng ta có thể viết trong ngôn ngữ này có thể được liệt kê. (Hãy tưởng tượng định nghĩa một thứ tự bằng chữ cái trên các ký hiệu và sau đó sắp xếp, theo thứ tự bảng chữ cái, mỗi tập hợp các câu có độ dài 1, 2, và cứ thế.) Sau đó, chúng ta có thể đánh số mỗi câu α bằng một số tự nhiên duy nhất $\# \alpha$ (**số Gödel**). Điều này rất quan trọng: lý thuyết số học chứa tên cho mỗi câu của chính nó. Tương tự, chúng ta có thể đánh số mỗi chứng minh P bằng một số Gödel $G(P)$, vì một chứng minh đơn giản chỉ là một chuỗi hữu hạn các câu.

Bây giờ, giả sử chúng ta có một tập hợp A các câu có thể liệt kê đệ quy, là những phát biểu đúng về các số tự nhiên. Nhớ lại rằng A có thể được đặt tên bằng một tập hợp các số nguyên nhất định, chúng ta có thể tưởng tượng viết trong ngôn ngữ của mình một câu $\alpha(j, A)$ có dạng như sau:

Vi không phải là số Gödel của một chứng minh cho câu có số Gödel j ,
trong đó chứng minh chỉ sử dụng các tiền đề trong A .

Sau đó, đặt σ là câu $\alpha(\# \sigma, A)$, tức là một câu khẳng định tính không thể chứng minh được của chính nó từ A . (Việc câu này luôn tồn tại là đúng nhưng không hoàn toàn hiển nhiên.)

Bây giờ, chúng ta đưa ra lập luận khéo léo sau: Giả sử rằng σ có thể được chứng minh từ A ; khi đó σ là sai (vì σ nói rằng nó không thể được chứng minh). Nhưng như vậy, chúng ta có một câu sai có thể được chứng minh từ A , điều này có nghĩa là A không thể chỉ bao gồm các câu đúng—một sự vi phạm giả thiết của chúng ta. Do đó, σ không thể được chứng minh từ A . Nhưng đây chính là điều mà bản thân σ khẳng định; vì vậy, σ là một câu đúng.

Như vậy, chúng ta đã chỉ ra (ngoại trừ 29 trang rưỡi) rằng với bất kỳ tập hợp nào các câu đúng trong lý thuyết số học, và đặc biệt là bất kỳ tập hợp tiên đề cơ bản nào, đều tồn tại những câu đúng khác không thể được chứng minh từ những tiên đề đó. Điều này thiết lập, trong số những thứ khác, rằng chúng ta không bao giờ có thể chứng minh tất cả các định lý toán học trong bất kỳ hệ tiên đề nào. Rõ ràng, đây là một phát hiện quan trọng đối với toán học. Ý nghĩa của nó đối với A đã được tranh luận rộng rãi, bắt đầu từ những suy đoán của chính Gödel. Chúng ta sẽ tiếp tục cuộc tranh luận này trong Chương 28.

Bổ đề này được gọi là **bổ đề nâng (lifting lemma)**, vì nó nâng một bước chứng minh từ các mệnh đề nền lên các mệnh đề logic bậc nhất tổng quát. Để chứng minh bổ đề nâng cơ bản này, Robinson đã phải phát minh ra phép hợp nhất và suy ra tất cả các tính chất của bộ hợp nhất tổng quát nhất (MGU). Thay vì lặp lại chứng minh ở đây, chúng tôi chỉ minh họa bổ đề:

$$\begin{aligned} C_1 &= \neg P(x, F(x, A)) \vee \neg Q(x, A) \vee R(x, B) \\ C_2 &= \neg N(G(y), z) \vee P(H(y), z) \\ C_{1'} &= \neg P(H(B), F(H(B), A)) \vee \neg Q(H(B), A) \vee R(H(B), B) \\ C_{2'} &= \neg N(G(B), F(H(B), A)) \vee P(H(B), F(H(B), A)) \\ C' &= \neg N(G(B), F(H(B), A)) \vee \neg Q(H(B), A) \vee R(H(B), B) \\ C &= \neg N(G(y), F(H(y), A)) \vee \neg Q(H(y), A) \vee R(H(y), B). \end{aligned}$$

Chúng ta thấy rằng C' thực sự là một trường hợp nền của C . Nói chung, để $C_{1'}$ và $C_{2'}$ có bất kỳ phân giải thức nào, chúng phải được xây dựng bằng cách trước tiên áp dụng cho C_1 và C_2 bộ hợp nhất tổng quát nhất của một cặp literal bù nhau trong C_1 và C_2 .

Từ bổ đề nâng, dễ dàng suy ra một phát biểu tương tự về bất kỳ chuỗi ứng dụng nào của quy tắc phân giải:

Với bất kỳ mệnh đề C' nào trong bao đóng phân giải của S' , tồn tại một mệnh đề C trong bao đóng phân giải của S sao cho C' là một trường hợp nền của C và việc suy dẫn C có cùng độ dài với việc suy dẫn C' .

Từ sự kiện này, suy ra rằng nếu mệnh đề rỗng xuất hiện trong bao đóng phân giải của S' , thì nó cũng phải xuất hiện trong bao đóng phân giải của S . Điều này là do mệnh đề rỗng không thể là một trường hợp nền của bất kỳ mệnh đề nào khác. Tóm lại: chúng tôi đã chỉ ra rằng nếu S không thỏa mãn được, thì có một suy dẫn hữu hạn của mệnh đề rỗng bằng cách sử dụng quy tắc phân giải.

Việc nâng việc chứng minh định lý từ các mệnh đề nền lên các mệnh đề logic bậc nhất mang lại một sự gia tăng đáng kể về sức mạnh. Sự gia tăng này đến từ việc chứng minh logic bậc nhất chỉ cần khởi tạo các biến ở mức cần thiết cho chứng minh, trong khi các phương pháp mệnh đề nền yêu cầu kiểm tra một số lượng lớn các khởi tạo tùy ý.

9.5.5 Bình đẳng (Equality)

Không có phương pháp suy luận nào được mô tả trước đó trong chương này có thể xử lý một khẳng định dạng $x = y$ mà không cần thêm một số công việc bổ sung. Có ba cách tiếp cận riêng biệt có thể được áp dụng:

1. **Tiên đề hóa bình đẳng (Axiomatizing equality):** Viết ra các câu về quan hệ bình đẳng trong cơ sở tri thức. Chúng ta cần nói rằng bình đẳng có tính phản xạ, đối xứng và bắc cầu, đồng thời cũng phải nói rằng chúng ta có thể thay thế các giá trị bằng nhau trong bất kỳ vị từ hoặc hàm nào. Vì vậy, chúng ta cần ba tiên đề cơ bản, sau đó là một tiên đề cho mỗi vị từ và hàm:

$$\begin{aligned}
& \forall x \ x = x \\
& \forall x, y \ x = y \Rightarrow y = x \\
& \forall x, y, z \ x = y \wedge y = z \Rightarrow x = z \\
& \forall x, y \ x = y \Rightarrow (P_1(x) \Leftrightarrow P_1(y)) \\
& \forall x, y \ x = y \Rightarrow (P_2(x) \Leftrightarrow P_2(y)) \\
& \vdots \\
& \forall w, x, y, z \ w = y \wedge x = z \Rightarrow (F_1(w, x) = F_1(y, z)) \\
& \forall w, x, y, z \ w = y \wedge x = z \Rightarrow (F_2(w, x) = F_2(y, z)) \\
& \vdots
\end{aligned}$$

Với các câu này, một quy trình suy luận tiêu chuẩn như phân giải có thể thực hiện các nhiệm vụ đòi hỏi lập luận về bình đẳng, chẳng hạn như giải các phương trình toán học. Tuy nhiên, những tiên đề này sẽ tạo ra rất nhiều kết luận, hầu hết trong số chúng không hữu ích cho việc chứng minh.

2. **Thêm quy tắc suy luận thay vì tiên đề (Adding inference rules rather than axioms):** Quy tắc đơn giản nhất, **demodulation**, lấy một mệnh đề đơn vị $x = y$ và một mệnh đề α chứa thuật ngữ x , sau đó tạo ra một mệnh đề mới bằng cách thay thế y cho x trong α . Nó hoạt động nếu thuật ngữ trong α hợp nhất được với x ; không cần phải chính xác bằng x . Lưu ý rằng demodulation có hướng; với $x = y$, x luôn được thay thế bằng y , không bao giờ ngược lại. Điều này có nghĩa là demodulation có thể được sử dụng để đơn giản hóa các biểu thức bằng cách sử dụng các demodulator như $z + 0 = z$ hoặc $z^1 = z$.

Ví dụ khác, với:

Birthdate(Father(Father(Bella)),1926)

chúng ta có thể kết luận bằng demodulation:

Birthdate(PaternalGrandfather(Bella),1926).

Một cách chính thức hơn, chúng ta có:

- **Demodulation:** Với bất kỳ thuật ngữ x, y và z , trong đó z xuất hiện ở đâu đó trong literal m_i và $UNIFY(x, z) = \theta \neq \text{failure}$,

$$\frac{x=y, \ m_1 \vee \dots \vee m_n}{\text{SUB}(\text{SUBST}(\theta, x), \text{SUBST}(\theta, y), m_1 \vee \dots \vee m_n)}$$

trong đó SUBST là phép thay thế thông thường của một danh sách ràng buộc, và SUB(x, y, m) có nghĩa là thay thế x bằng y ở đâu đó trong m .

Quy tắc này cũng có thể được mở rộng để xử lý các mệnh đề không phải đơn vị trong đó có dấu bằng:

- **Paramodulation:** Với bất kỳ thuật ngữ x, y và z , trong đó z xuất hiện ở đâu đó trong literal m_i và $UNIFY(x, z) = \theta$,

$$\frac{\ell_1 \vee \dots \vee \ell_k \vee x=y, \quad m_1 \vee \dots \vee m_n}{SUB(SUBST(\theta, x), SUBST(\theta, y), SUBST(\theta, \ell_1 \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_n))}$$

Ví dụ, từ:

$$P(F(x, B), x) \vee Q(x) \quad \text{và} \quad F(A, y) = y \vee R(y)$$

chúng ta có $\theta = UNIFY(F(A, y), F(x, B)) = \{x/A, y/B\}$, và có thể kết luận bằng paramodulation mệnh đề:

$$P(B, A) \vee Q(A) \vee R(B).$$

Paramodulation tạo ra một quy trình suy luận hoàn chỉnh cho logic bậc nhất với bình đẳng.

3. **Xử lý lập luận bình đẳng hoàn toàn trong một thuật toán hợp nhất mở rộng (Handling equality reasoning entirely within an extended unification algorithm):** Nghĩa là, các thuật ngữ có thể hợp nhất nếu chúng có thể chứng minh được là bằng nhau dưới một phép thay thế nào đó, trong đó "có thể chứng minh" cho phép lập luận về bình đẳng. Ví dụ, các thuật ngữ $1 + 2$ và $2 + 1$ thông thường không thể hợp nhất, nhưng một thuật toán hợp nhất biết rằng $x + y = y + x$ có thể hợp nhất chúng với phép thay thế rỗng. **Hợp nhất phương trình (Equational unification)** loại này có thể được thực hiện bằng các thuật toán hiệu quả được thiết kế cho các tiên đề cụ thể (giao hoán, kết hợp, v.v.) thay vì thông qua suy luận rõ ràng với các tiên đề đó. Các trình chứng minh định lý sử dụng kỹ thuật này có liên quan chặt chẽ với các hệ thống CLP được mô tả trong Mục 9.4.

9.5.6 Các chiến lược phân giải

Chúng ta biết rằng việc áp dụng lặp lại quy tắc suy luận phân giải cuối cùng sẽ tìm ra một chứng minh nếu chứng minh đó tồn tại. Trong phần này, chúng ta sẽ xem xét các chiến lược giúp tìm chứng minh một cách hiệu quả.

Ưu tiên đơn vị (Unit preference): Chiến lược này ưu tiên thực hiện các phép phân giải trong đó một trong các câu là một ký tự đơn (còn được gọi là mệnh đề đơn vị). Ý tưởng đằng sau chiến lược này là chúng ta đang cố gắng tạo ra một mệnh đề rỗng, vì vậy có thể là một ý tưởng tốt để ưu tiên các suy luận tạo ra các mệnh đề ngắn hơn. Việc phân giải một câu đơn vị (chẳng hạn như P) với bất kỳ câu nào khác (chẳng hạn như $\neg P \vee \neg Q \vee R$) luôn tạo ra một mệnh đề (trong trường hợp này là $\neg Q \vee R$) ngắn hơn mệnh đề kia. Khi chiến lược ưu tiên đơn vị lần đầu tiên được thử nghiệm cho suy luận mệnh đề vào năm 1964, nó đã dẫn đến một sự cải thiện đáng kể về tốc độ, giúp việc chứng minh các định lý mà trước đây không thể xử lý trở nên khả thi. **Phân giải đơn vị (Unit resolution)** là một dạng hạn chế của phân giải trong đó mỗi bước phân giải phải liên quan đến một mệnh đề đơn vị. Phân giải đơn vị không đầy đủ trong trường hợp tổng quát, nhưng đầy đủ cho các mệnh đề Horn. Các chứng minh phân giải đơn vị trên các mệnh đề Horn tương tự như suy luận tiến.

Bộ chứng minh định lý **OTTER** (McCune, 1990) sử dụng một dạng tìm kiếm ưu tiên tốt nhất. Hàm heuristic của nó đo lường "trọng số" của mỗi mệnh đề, trong đó các mệnh đề nhẹ hơn được ưu tiên. Việc lựa chọn chính xác heuristic phụ thuộc vào người dùng, nhưng nhìn chung, trọng số

của một mệnh đề nên tương quan với kích thước hoặc độ phức tạp của nó. Các mệnh đề đơn vị được coi là nhẹ; do đó, việc tìm kiếm có thể được xem như một sự tổng quát hóa của chiến lược ưu tiên đơn vị.

Tập hỗ trợ (Set of support): Các ưu tiên cố gắng thực hiện một số phép phân giải trước tiên là hữu ích, nhưng nhìn chung, việc cố gắng loại bỏ hoàn toàn một số phép phân giải tiềm năng sẽ hiệu quả hơn. Ví dụ, chúng ta có thể yêu cầu rằng mỗi bước phân giải phải liên quan đến ít nhất một phần tử của một tập hợp đặc biệt các mệnh đề—**tập hỗ trợ**. Mệnh đề phân giải sau đó được thêm vào tập hỗ trợ. Nếu tập hỗ trợ nhỏ so với toàn bộ cơ sở tri thức, không gian tìm kiếm sẽ giảm đáng kể.

Để đảm bảo tính đầy đủ của chiến lược này, chúng ta có thể chọn tập hỗ trợ S sao cho phần còn lại của các câu có thể thỏa mãn đồng thời. Ví dụ, người ta có thể sử dụng phủ định của truy vấn làm tập hỗ trợ, với giả định rằng cơ sở tri thức ban đầu là nhất quán. (Xét cho cùng, nếu nó không nhất quán, thì việc truy vấn suy ra từ nó là vô nghĩa.) Chiến lược tập hỗ trợ có thêm lợi thế là tạo ra các cây chứng minh hướng mục tiêu, thường dễ hiểu đối với con người.

Phân giải đầu vào (Input resolution): Trong chiến lược này, mỗi phép phân giải kết hợp một trong các câu đầu vào (từ cơ sở tri thức hoặc truy vấn) với một số câu khác. Chứng minh trong Hình 9.10 (trang 319) chỉ sử dụng các phép phân giải đầu vào và có hình dạng đặc trưng của một "xương sống" đơn với các câu đơn kết hợp vào xương sống. Rõ ràng, không gian của các cây chứng minh có hình dạng này nhỏ hơn không gian của tất cả các đồ thị chứng minh. Trong các cơ sở tri thức Horn, **Modus Ponens** là một dạng chiến lược phân giải đầu vào, vì nó kết hợp một phép suy diễn từ cơ sở tri thức ban đầu với một số câu khác. Do đó, không có gì ngạc nhiên khi phân giải đầu vào là đầy đủ cho các cơ sở tri thức ở dạng Horn, nhưng không đầy đủ trong trường hợp tổng quát. Chiến lược **phân giải tuyến tính (linear resolution)** là một sự tổng quát hóa nhẹ cho phép P và Q được phân giải cùng nhau nếu P nằm trong cơ sở tri thức ban đầu hoặc nếu P là một tổ tiên của Q trong cây chứng minh. Phân giải tuyến tính là đầy đủ.

Bao hàm (Subsumption): Phương pháp bao hàm loại bỏ tất cả các câu bị bao hàm bởi (nghĩa là cụ thể hơn) một câu hiện có trong cơ sở tri thức. Ví dụ, nếu $P(x)$ có trong cơ sở tri thức, thì không có lý do gì để thêm $P(A)$ và thậm chí ít lý do hơn để thêm $P(A) \vee Q(B)$. Bao hàm giúp giữ cho cơ sở tri thức nhỏ và do đó giúp giảm không gian tìm kiếm.

Học hỏi (Learning): Chúng ta có thể cải thiện một bộ chứng minh định lý bằng cách học hỏi từ kinh nghiệm. Với một tập hợp các định lý đã được chứng minh trước đó, huấn luyện một hệ thống học máy để trả lời câu hỏi: với một tập tiền đề và một mục tiêu cần chứng minh, các bước chứng minh nào tương tự như các bước đã thành công trong quá khứ? Hệ thống **DeepHOL** (Bansal et al., 2019) thực hiện chính xác điều này, sử dụng mạng nơ-ron sâu (xem Chương 22) để xây dựng các mô hình (gọi là **embeddings**) của các mục tiêu và tiền đề, và sử dụng chúng để đưa ra các lựa chọn. Việc huấn luyện có thể sử dụng cả các chứng minh do con người và máy tính tạo ra làm ví dụ, bắt đầu từ một tập hợp 10.000 chứng minh.

Ứng dụng thực tế của các bộ chứng minh định lý phân giải

Chúng ta đã chỉ ra cách logic bậc nhất có thể biểu diễn một kịch bản thế giới thực đơn giản liên quan đến các khái niệm như bán hàng, vũ khí và quốc tịch. Nhưng các kịch bản thế giới thực phức tạp có quá nhiều sự không chắc chắn và quá nhiều ẩn số. Logic đã chứng minh thành công hơn đối với các kịch bản liên quan đến các khái niệm chính thức, được định nghĩa chặt chẽ, chẳng hạn như

tổng hợp và **xác minh** cả phần cứng và phần mềm. Nghiên cứu về chứng minh định lý được tiến hành trong các lĩnh vực thiết kế phần cứng, ngôn ngữ lập trình và kỹ thuật phần mềm—không chỉ trong AI.

Trong trường hợp phần cứng, các tiên đề mô tả sự tương tác giữa các tín hiệu và các phần tử mạch. (Xem Phần 8.4.2, trang 291 để biết ví dụ.) Các bộ suy luận logic được thiết kế đặc biệt cho việc xác minh đã có thể xác minh toàn bộ CPU, bao gồm cả các thuộc tính thời gian (Srivasa và Bickford, 1990). Bộ chứng minh định lý **Aura** đã được áp dụng để thiết kế các mạch nhỏ gọn hơn bất kỳ thiết kế nào trước đây (Wojciechowski và Wojcik, 1983).

Trong trường hợp phần mềm, việc lập luận về các chương trình khá giống với lập luận về các hành động, như trong Chương 7: các tiên đề mô tả các điều kiện tiên quyết và hiệu ứng của mỗi câu lệnh. Việc tổng hợp thuật toán một cách chính thức là một trong những ứng dụng đầu tiên của các bộ chứng minh định lý, như được phác thảo bởi Cordell Green (1969a), dựa trên các ý tưởng trước đó của Herbert Simon (1963). Ý tưởng là chứng minh một định lý một cách xây dựng rằng "tồn tại một chương trình p thỏa mãn một đặc tả nhất định." Mặc dù **tổng hợp suy diễn (deductive synthesis)** hoàn toàn tự động vẫn chưa khả thi đối với lập trình đa năng, việc tổng hợp suy diễn có hướng dẫn bằng tay đã thành công trong việc thiết kế một số thuật toán mới và tinh vi. Việc tổng hợp các chương trình đặc biệt, chẳng hạn như mã tính toán khoa học, cũng là một lĩnh vực nghiên cứu tích cực.

Các kỹ thuật tương tự hiện đang được áp dụng để xác minh phần mềm bởi các hệ thống như bộ kiểm tra mô hình **Spin** (Holzmann, 1997). Ví dụ, chương trình điều khiển tàu vũ trụ **Remote Agent** đã được xác minh trước và sau chuyến bay (Havelund et al., 2000). Thuật toán mã hóa khóa công khai **RSA** và thuật toán khớp chuỗi **Boyer-Moore** cũng đã được xác minh theo cách này (Boyer và Moore, 1984).

Tóm tắt

Chúng tôi đã trình bày một phân tích về suy luận logic trong logic bậc nhất và một số thuật toán để thực hiện nó.

- Một cách tiếp cận đầu tiên sử dụng các quy tắc suy luận (**phép khái quát hóa phổ quát** và **phép khái quát hóa tồn tại**) để **mệnh đề hóa** bài toán suy luận. Thông thường, cách tiếp cận này chậm, trừ khi miền giá trị nhỏ.
- Việc sử dụng **phép hợp nhất** để xác định các phép thế thích hợp cho các biến loại bỏ bước khái quát hóa trong các chứng minh bậc nhất, làm cho quá trình hiệu quả hơn trong nhiều trường hợp.
- Một phiên bản nâng cao của **Modus Ponens** sử dụng phép hợp nhất để cung cấp một quy tắc suy luận tự nhiên và mạnh mẽ, **Modus Ponens tổng quát**. Các thuật toán **suy diễn tiến** và **suy diễn lùi** áp dụng quy tắc này cho các tập hợp mệnh đề xác định.
- **Modus Ponens tổng quát** là đầy đủ cho các mệnh đề xác định, mặc dù bài toán suy diễn là bán quyết định. Đối với các cơ sở tri thức **Datalog** bao gồm các mệnh đề xác định không chứa hàm, bài toán suy diễn là quyết định được.

- **Suy diễn tiến** được sử dụng trong các cơ sở dữ liệu suy diễn, nơi nó có thể kết hợp với các thao tác cơ sở dữ liệu quan hệ. Nó cũng được sử dụng trong các hệ thống sản xuất, thực hiện các cập nhật hiệu quả với các bộ quy tắc rất lớn. Suy diễn tiến là đầy đủ cho Datalog và chạy trong thời gian đa thức.
- **Suy diễn lùi** được sử dụng trong các hệ thống lập trình logic, sử dụng công nghệ trình biên dịch tĩnh vì để cung cấp suy luận cực nhanh. Suy diễn lùi gặp phải vấn đề về các suy luận dư thừa và vòng lặp vô hạn; những vấn đề này có thể được giảm bớt bằng cách ghi nhớ.
- **Prolog**, không giống như logic bậc nhất, sử dụng một thể giới đóng với giả định tên duy nhất và phủ định như thất bại. Những điều này làm cho Prolog trở thành một ngôn ngữ lập trình thực tế hơn, nhưng cũng làm nó xa rời logic thuần túy.
- Quy tắc suy luận **phân giải tổng quát** cung cấp một hệ thống chứng minh đầy đủ cho logic bậc nhất, sử dụng các cơ sở tri thức ở dạng chuẩn hội.
- Một số chiến lược tồn tại để giảm không gian tìm kiếm của một hệ thống phân giải mà không làm mất tính đầy đủ. Một trong những vấn đề quan trọng nhất là xử lý đẳng thức; chúng tôi đã chỉ ra cách **khử điều chế (demodulation)** và **điều chế tham số (paramodulation)** có thể được sử dụng.
- Các bộ chứng minh định lý dựa trên phân giải hiệu quả đã được sử dụng để chứng minh các định lý toán học thú vị và để xác minh cũng như tổng hợp phần mềm và phần cứng.

Ghi chú lịch sử và thư mục

Gottlob Frege, người đã phát triển logic bậc nhất đầy đủ vào năm 1879, dựa trên hệ thống suy luận của mình bằng một tập hợp các lược đồ hợp lệ cộng với một quy tắc suy luận duy nhất, **Modus Ponens**. Whitehead và Russell (1910) đã giải thích các quy tắc chuyển đổi (thuật ngữ thực tế là từ Herbrand (1930)) được sử dụng để di chuyển các lượng tử lên đầu các công thức. Các hằng số **Skolem** và hàm **Skolem** được giới thiệu một cách thích hợp bởi Thoralf Skolem (1920). Điều thú vị là chính Skolem đã giới thiệu vũ trụ **Herbrand** (Skolem, 1928).

Định lý **Herbrand** (Herbrand, 1930) đã đóng một vai trò quan trọng trong sự phát triển của lập luận tự động. Herbrand cũng là người phát minh ra phép hợp nhất. Gödel (1930) đã xây dựng dựa trên các ý tưởng của Skolem và Herbrand để chỉ ra rằng logic bậc nhất có một thủ tục chứng minh đầy đủ. Alan Turing (1936) và Alonzo Church (1936) đồng thời chỉ ra, bằng các chứng minh rất khác nhau, rằng tính hợp lệ trong logic bậc nhất là không quyết định được. Văn bản xuất sắc của Enderton (1972) giải thích tất cả các kết quả này một cách chặt chẽ nhưng dễ hiểu.

Abraham Robinson đề xuất rằng một bộ suy luận tự động có thể được xây dựng bằng cách sử dụng mệnh đề hóa và định lý Herbrand, và Paul Gilmore (1960) đã viết chương trình đầu tiên. Davis và Putnam (1960) giới thiệu phương pháp mệnh đề hóa trong Phần 9.1. Prawitz (1960) phát triển ý tưởng chính là để việc tìm kiếm sự không nhất quán mệnh đề điều khiển quá trình tìm kiếm, và chỉ tạo ra các hạng tử từ vũ trụ Herbrand khi chúng cần thiết để thiết lập sự không nhất quán mệnh đề. Ý tưởng này dẫn John Alan Robinson (không liên quan) phát triển **phân giải** (Robinson, 1965). Phân giải được áp dụng cho các hệ thống trả lời câu hỏi bởi Cordell Green và Bertram Raphael (1968). Các triển khai AI ban đầu đã dành nhiều nỗ lực vào các cấu trúc dữ liệu cho phép truy xuất hiệu quả các sự kiện; công việc này được đề cập trong các văn bản lập trình AI (Charniak et al.,

1987; Norvig, 1992; Forbus và de Kleer, 1993). Đến đầu những năm 1970, **suy diễn tiến** đã được thiết lập tốt trong AI như một lựa chọn thay thế dễ hiểu cho phân giải. Các ứng dụng AI thường liên quan đến một số lượng lớn các quy tắc, do đó, việc phát triển công nghệ khớp quy tắc hiệu quả, đặc biệt là cho các cập nhật gia tăng, là rất quan trọng. Công nghệ cho các hệ thống sản xuất được phát triển để hỗ trợ các ứng dụng như vậy. Ngôn ngữ hệ thống sản xuất **Ops-5** (Forgy, 1981; Brownston et al., 1985), kết hợp quá trình khớp **Rete** hiệu quả (Forgy, 1982), được sử dụng cho các ứng dụng như hệ thống chuyên gia **RI** để cấu hình máy tính mini (McDermott, 1982). Kraska et al. (2017) mô tả cách các mạng nơ-ron có thể học một sơ đồ lập chỉ mục hiệu quả cho các tập dữ liệu cụ thể. Kiến trúc nhận thức **Soar** (Laird et al., 1987; Laird, 2008) được thiết kế để xử lý các bộ quy tắc rất lớn—lên đến một triệu quy tắc (Doorenbos, 1994). Các ứng dụng ví dụ của Soar bao gồm điều khiển máy bay chiến đấu mô phỏng (Jones et al., 1998), quản lý không phận (Taylor et al., 2007), nhân vật AI cho trò chơi máy tính (Wintermute et al., 2007) và công cụ huấn luyện cho binh lính (Wray và Jones, 2005).

Lĩnh vực cơ sở dữ liệu suy diễn bắt đầu với một hội thảo ở Toulouse năm 1977 với sự tham dự của các chuyên gia về suy luận logic và cơ sở dữ liệu (Gallaire và Minker, 1978). Công trình có ảnh hưởng của Chandra và Harel (1980) và Ullman (1985) đã dẫn đến việc chấp nhận **Datalog** như một ngôn ngữ tiêu chuẩn cho cơ sở dữ liệu suy diễn. Sự phát triển của kỹ thuật **tập ma thuật (magic sets)** để viết lại quy tắc bởi Bancilhon et al. (1986) cho phép suy diễn tiến mượn lợi thế định hướng mục tiêu từ suy diễn lùi. Sự trỗi dậy của Internet đã dẫn đến việc tăng tính sẵn có của các cơ sở dữ liệu trực tuyến khổng lồ. Điều này thúc đẩy sự quan tâm ngày càng tăng trong việc tích hợp nhiều cơ sở dữ liệu thành một không gian dữ liệu nhất quán (Halevy, 2007). Kraska et al. (2017) đã chỉ ra tốc độ tăng lên đến 70% bằng cách sử dụng học máy để tạo ra các cấu trúc chỉ mục học được để tra cứu dữ liệu hiệu quả.

Suy diễn lùi cho suy luận logic bắt nguồn từ ngôn ngữ **PLANNER** (Hewitt, 1969). Trong khi đó, vào năm 1972, Alain Colmerauer đã phát triển và triển khai **Prolog** với mục đích phân tích ngôn ngữ tự nhiên—các mệnh đề Prolog ban đầu được dự định như các quy tắc ngữ pháp phi ngữ cảnh (Roussel, 1975; Colmerauer et al., 1973). Nhiều nền tảng lý thuyết cho lập trình logic được phát triển bởi Robert Kowalski tại Imperial College London, làm việc với Colmerauer; xem Kowalski (1988) và Colmerauer và Roussel (1993) để biết tổng quan lịch sử. Các trình biên dịch Prolog hiệu quả thường dựa trên mô hình máy tính **Warren Abstract Machine (WAM)** được phát triển bởi David H. D. Warren (1983). Van Roy (1990) đã chỉ ra rằng các chương trình Prolog có thể cạnh tranh với các chương trình C về tốc độ. Các phương pháp tránh lặp không cần thiết trong các chương trình logic đệ quy được phát triển độc lập bởi Smith et al. (1986) và Tamaki và Sato (1986). Bài báo sau cũng bao gồm việc ghi nhớ cho các chương trình logic, một phương pháp được phát triển rộng rãi như lập trình logic có bảng bởi David S. Warren. Swift và Warren (1994) chỉ ra cách mở rộng WAM để xử lý bảng, cho phép các chương trình Datalog thực thi nhanh hơn một bậc độ lớn so với các hệ thống cơ sở dữ liệu suy diễn tiến.

Công trình ban đầu về lập trình logic ràng buộc được thực hiện bởi Jaffar và Lassez (1987). Jaffar et al. (1992) đã phát triển hệ thống **CLP(R)** để xử lý các ràng buộc giá trị thực. Hiện nay có các sản phẩm thương mại để giải quyết các vấn đề cấu hình và tối ưu hóa quy mô lớn với lập trình ràng buộc; một trong những sản phẩm nổi tiếng nhất là **ILOG** (Junker, 2003). **Lập trình tập đáp án (Answer set programming)** (Gelfond, 2008) mở rộng Prolog, cho phép phép hội và phủ định.

Các văn bản về lập trình logic và Prolog bao gồm Shoham (1994), Bratko (2009), Clocksin (2003) và Clocksin và Mellish (2003). Trước năm 2000, **Tạp chí Lập trình Logic** là tạp chí chính thức;

nó đã được thay thế bằng **Lý thuyết và Thực hành Lập trình Logic**. Các hội nghị về lập trình logic bao gồm **Hội nghị Quốc tế về Lập trình Logic (ICLP)** và **Hội nghị Chuyên đề Lập trình Logic Quốc tế (ILPS)**.

Nghiên cứu về chứng minh định lý toán học bắt đầu ngay cả trước khi các hệ thống bậc nhất đầy đủ đầu tiên được phát triển. **Trình chứng minh Định lý Hình học** của Herbert Gelernter (Gelernter, 1959) sử dụng các phương pháp tìm kiếm heuristic kết hợp với sơ đồ để loại bỏ các mục tiêu phụ sai và có thể chứng minh một số kết quả khá phức tạp trong hình học Euclid. Các quy tắc **khử điều chế (demodulation)** và **điều chế tham số (paramodulation)** cho lập luận đẳng thức được giới thiệu bởi Wos et al. (1967) và Wos và Robinson (1968), tương ứng. Các quy tắc này cũng được phát triển độc lập trong bối cảnh các hệ thống viết lại hạng tử (Knuth và Bendix, 1970). Việc kết hợp lập luận đẳng thức vào thuật toán hợp nhất là do Gordon Plotkin (1972). Jouannaud và Kirchner (1991) khảo sát phép hợp nhất đẳng thức từ góc độ viết lại hạng tử. Tổng quan về phép hợp nhất được cung cấp bởi Baader và Snyder (2001).

Một số chiến lược điều khiển đã được đề xuất cho phân giải, bắt đầu với chiến lược **ưu tiên đơn vị** (Wos et al., 1964). Chiến lược **tập hỗ trợ** được đề xuất bởi Wos et al. (1965) để cung cấp một mức độ định hướng mục tiêu trong phân giải. **Phân giải tuyến tính** xuất hiện lần đầu tiên trong Loveland (1970). Genesereth và Nilsson (1987, Chương 5) cung cấp một phân tích về nhiều chiến lược điều khiển khác nhau. Alemi et al. (2017) chỉ ra cách hệ thống **DEEPMATH** sử dụng mạng nơ-ron sâu để chọn các tiên đề có khả năng dẫn đến chứng minh cao nhất khi được chuyển cho một bộ chứng minh định lý truyền thống. Theo một nghĩa nào đó, mạng nơ-ron đóng vai trò như trực giác của nhà toán học, và bộ chứng minh định lý đóng vai trò như chuyên môn kỹ thuật của nhà toán học. (Loos et al., 2017) chỉ ra rằng cách tiếp cận này có thể được mở rộng để giúp định hướng tìm kiếm, cho phép chứng minh nhiều định lý hơn.

A Computational Logic (Boyer và Moore, 1979) là tài liệu tham khảo cơ bản về bộ chứng minh định lý **Boyer-Moore**. Stickel (1992) mô tả **Bộ chứng minh Định lý Công nghệ Prolog (PTTP)**, kết hợp biên dịch Prolog và loại bỏ mô hình. **Setheo** (Letz et al., 1992) là một bộ chứng minh định lý khác được sử dụng rộng rãi dựa trên cách tiếp cận này. **LeanTAP** (Beckert và Posegga, 1995) là một bộ chứng minh định lý hiệu quả được triển khai chỉ trong 25 dòng Prolog. Weidenbach (2001) mô tả **SPASS**, một trong những bộ chứng minh định lý mạnh nhất hiện nay. Bộ chứng minh định lý thành công nhất trong các cuộc thi hàng năm gần đây là **VAMPIRE** (Riazanov và Voronkov, 2002). Hệ thống **Coq** (Bertot et al., 2004) và bộ giải đẳng thức **E** (Schulz, 2004) cũng đã chứng minh là những công cụ giá trị để chứng minh tính chính xác.

Các bộ chứng minh định lý đã được sử dụng để tự động tổng hợp và xác minh phần mềm. Ví dụ bao gồm phần mềm điều khiển cho khoang **Orion** của NASA (Lowry, 2008) và các tàu vũ trụ khác (Denney et al., 2006). Thiết kế của bộ vi xử lý 32-bit **FM9001** đã được chứng minh là chính xác bởi hệ thống chứng minh định lý **NQTHM** (Hunt và Brock, 1992).

Hội nghị về Suy diễn Tự động (CADE) tổ chức một cuộc thi hàng năm cho các bộ chứng minh định lý tự động. Sutcliffe (2016) mô tả cuộc thi năm 2016; các hệ thống đạt điểm cao bao gồm **VAMPIRE** (Riazanov và Voronkov, 2002), **PROVER9** (Sabri, 2015) và một phiên bản cập nhật của **E** (Schulz, 2013). Wiedijk (2003) so sánh sức mạnh của 15 bộ chứng minh toán học. **TPTP (Thousands of Problems for Theorem Provers)** là một thư viện các bài toán chứng minh định lý, hữu ích để so sánh hiệu suất của các hệ thống (Sutcliffe và Suttner, 1998; Sutcliffe et al., 2006).

Các bộ chứng minh định lý đã đưa ra các kết quả toán học mới mà các nhà toán học con người không thể tìm ra trong nhiều thập kỷ, như được trình bày chi tiết trong cuốn sách **Automated Reasoning and the Discovery of Missing Elegant Proofs** (Wos và Pieper, 2003). Chương trình **SAM (Semi-Automated Mathematics)** là chương trình đầu tiên, chứng minh một bổ đề trong lý thuyết mạng (Guard et al., 1969). Chương trình **AURA** cũng đã trả lời các câu hỏi mở trong một số lĩnh vực toán học (Wos và Winker, 1983). Bộ chứng minh định lý **Boyer-Moore** (Boyer và Moore, 1979) đã được Natarajan Shankar sử dụng để xây dựng một chứng minh chính thức về Định lý Không đầy đủ của Gödel (Shankar, 1986). Hệ thống **NUPRL** đã chứng minh nghịch lý Girard (Howe, 1987) và Bổ đề Higman (Murthy và Russell, 1990).

Năm 1933, Herbert Robbins đã đề xuất một hệ tiên đề đơn giản - đại số Robbins - tương chừng như định nghĩa được đại số Boolean, nhưng không ai tìm ra cách chứng minh (dù nhiều nhà toán học như Alfred Tarski đã nghiên cứu nghiêm túc) cho đến khi chương trình EQP (một phiên bản của OTTER) tính toán được chứng minh này (McCune, 1997). Benzmüller và Paleo (2013) đã sử dụng một trình chứng minh định lý bậc cao để xác minh chứng minh của Gödel về sự tồn tại của "Thượng đế". Định lý xếp hình cầu Kepler được Thomas Hales chứng minh năm 2005 với sự trợ giúp của các tính toán máy tính phức tạp, nhưng phải đến khi một chứng minh hình thức được tạo ra nhờ các công cụ hỗ trợ chứng minh HOL Light và Isabelle (Hales và cộng sự, 2017) thì nó mới được chấp nhận hoàn toàn.

Nhiều bài báo tiên phong về logic toán học được tập hợp trong cuốn "From Frege to Gödel: A Source Book in Mathematical Logic" (van Heijenoort, 1967). Các giáo trình về suy diễn tự động bao gồm tác phẩm kinh điển "Symbolic Logic and Mechanical Theorem Proving" (Chang và Lee, 1973), cùng các công trình gần đây hơn của Duffy (1991), Wos và cộng sự (1992), Bibel (1993), và Kaufmann cùng cộng sự (2000). Tạp chí chính về chứng minh định lý tự động là Journal of Automated Reasoning; các hội nghị chính gồm Hội nghị thường niên về Suy diễn Tự động (CADE) và Hội nghị Liên hợp Quốc tế về Suy diễn Tự động (IJCAR). Cuốn "Handbook of Automated Reasoning" (Robinson và Voronkov, 2001) tập hợp các bài báo trong lĩnh vực này. Tác phẩm "Mechanizing Proof" của MacKenzie (2004) giới thiệu lịch sử và công nghệ chứng minh định lý tự động cho độc giả phổ thông.

CHƯƠNG 10 BIỂU DIỄN TRI THỨC

Trong chương này, chúng tôi trình bày cách biểu diễn các sự kiện đa dạng về thế giới thực dưới dạng có thể sử dụng để suy luận và giải quyết vấn đề.

- **Kỹ thuật bản thể luận**
- **Bản thể luận cấp cao**

Các chương trước đã cho thấy một tác nhân (agent) với cơ sở tri thức có thể đưa ra các suy luận giúp nó hành động phù hợp. Trong chương này, chúng tôi giải quyết câu hỏi về nội dung cần đưa vào cơ sở tri thức của tác nhân đó—cách biểu diễn các sự kiện về thế giới. Chúng tôi sử dụng logic bậc nhất làm ngôn ngữ biểu diễn, nhưng các chương sau sẽ giới thiệu các hình thức biểu diễn khác như:

- **Mạng lưới nhiệm vụ phân cấp** để suy luận về kế hoạch (Chương 11),
- **Mạng Bayesian** để suy luận trong điều kiện không chắc chắn (Chương 13),
- **Mô hình Markov** để suy luận theo thời gian (Chương 16),
- **Mạng nơ-ron sâu** để suy luận về hình ảnh, âm thanh và dữ liệu khác (Chương 22).

Dù bạn sử dụng hình thức biểu diễn nào, các sự kiện về thế giới vẫn cần được xử lý, và chương này sẽ giúp bạn hiểu rõ hơn về vấn đề này.

- **Mục 10.1** giới thiệu ý tưởng về bản thể luận tổng quát, phân loại mọi thứ trong thế giới thành một hệ thống phân cấp.
- **Mục 10.2** trình bày các phạm trù cơ bản như đối tượng, chất liệu và đơn vị đo lường.
- **Mục 10.3** tập trung vào sự kiện.
- **Mục 10.4** thảo luận về tri thức liên quan đến niềm tin.

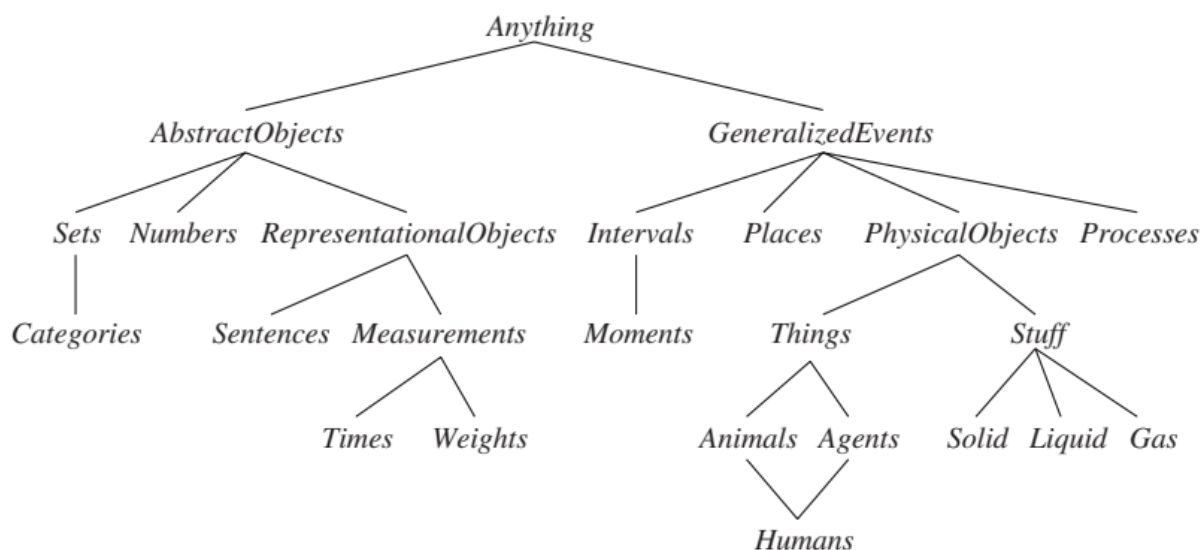
Sau đó, chúng tôi quay lại xem xét công nghệ suy luận với nội dung này:

- **Mục 10.5** thảo luận về các hệ thống suy luận được thiết kế để suy diễn hiệu quả với các phạm trù.
- **Mục 10.6** trình bày cách suy luận với thông tin mặc định.

10.1 Kỹ thuật Bản thể luận (Ontological Engineering)

Trong các lĩnh vực "đồ chơi" (toy domains), việc lựa chọn cách biểu diễn không quá quan trọng; nhiều lựa chọn đều có thể hoạt động. Tuy nhiên, các lĩnh vực phức tạp như mua sắm trên Internet hay lái xe trong điều kiện giao thông đòi hỏi những cách biểu diễn tổng quát và linh hoạt hơn. Chương này sẽ trình bày cách tạo ra những biểu diễn như vậy, tập trung vào các khái niệm chung—chẳng hạn như *Sự kiện (Events)*, *Thời gian (Time)*, *Đối tượng vật lý (Physical Objects)* và *Niềm tin (Beliefs)*—xuất hiện trong nhiều lĩnh vực khác nhau. Việc biểu diễn các khái niệm trừu tượng này đôi khi được gọi là **kỹ thuật bản thể luận (ontological engineering)**.

Chúng ta không thể hy vọng biểu diễn *mọi thứ* trên thế giới, ngay cả trong một cuốn sách giáo khoa dày 1000 trang, nhưng chúng ta có thể để lại các "chỗ giữ chỗ" (placeholders) để tri thức mới trong bất kỳ lĩnh vực nào có thể được thêm vào sau này. Ví dụ, chúng ta sẽ định nghĩa thế nào là một đối tượng vật lý, và các chi tiết về các loại đối tượng khác nhau—robot, tivi, sách, hoặc bất cứ thứ gì—có thể được điền vào sau. Điều này tương tự như cách các nhà thiết kế một framework lập trình hướng đối tượng (chẳng hạn như framework đồ họa Java Swing) định nghĩa các khái niệm chung như *Cửa sổ* (*Window*), với kỳ vọng người dùng sẽ sử dụng chúng để định nghĩa các khái niệm cụ thể hơn như *Cửa sổ bảng tính* (*SpreadsheetWindow*). Khung khái niệm tổng quát này được gọi là **bản thể luận cấp cao (upper ontology)** do quy ước vẽ đồ thị với các khái niệm chung ở trên cùng và các khái niệm cụ thể hơn ở phía dưới, như trong Hình 10.1.



Hình 10.1 Ontology tổng quát của thế giới, thể hiện các chủ đề sẽ được đề cập sau trong chương. Mỗi liên kết chỉ ra rằng khái niệm phía dưới là một trường hợp đặc biệt của khái niệm phía trên. Các trường hợp đặc biệt không nhất thiết phải rời rạc - con người vừa là động vật vừa là tác nhân. Chúng ta sẽ thấy trong Mục 10.3.2 lý do tại sao đối tượng vật lý lại thuộc về sự kiện tổng quát.

Trước khi xem xét sâu hơn về bản thể luận, chúng ta cần nêu rõ một lưu ý quan trọng. Chúng ta đã chọn sử dụng logic bậc nhất (first-order logic) để thảo luận về nội dung và tổ chức tri thức, mặc dù một số khía cạnh của thế giới thực rất khó nắm bắt trong logic bậc nhất. Khó khăn chính là hầu hết các khái niệm hóa đều có ngoại lệ hoặc chỉ đúng ở một mức độ nào đó. Ví dụ, mặc dù quy tắc "cà chua có màu đỏ" rất hữu ích, nhưng một số quả cà chua lại có màu xanh, vàng hoặc cam. Các ngoại lệ tương tự có thể được tìm thấy đối với hầu hết các quy tắc trong chương này. Khả năng xử lý ngoại lệ và sự không chắc chắn là cực kỳ quan trọng, nhưng nó lại độc lập với nhiệm vụ hiểu bản thể luận tổng quát. Vì lý do này, chúng ta sẽ hoãn thảo luận về ngoại lệ đến Mục 10.5 của chương này, và chủ đề tổng quát hơn về suy luận với sự không chắc chắn đến Chương 12.

Vậy bản thể luận cấp cao có tác dụng gì? Hãy xem xét bản thể luận về mạch điện trong Mục 8.4.2. Nó đưa ra nhiều giả định đơn giản hóa: thời gian hoàn toàn bị bỏ qua; tín hiệu được cố định và không lan truyền; cấu trúc mạch không thay đổi. Một bản thể luận tổng quát hơn sẽ xem xét các tín hiệu tại những thời điểm cụ thể, bao gồm cả chiều dài dây dẫn và độ trễ lan truyền. Điều này

cho phép chúng ta mô phỏng các đặc tính thời gian của mạch, và thực tế, các kỹ sư mạch thường xuyên thực hiện những mô phỏng như vậy.

Chúng ta cũng có thể giới thiệu các loại cổng logic thú vị hơn, ví dụ, bằng cách mô tả công nghệ (TTL, CMOS, v.v.) cũng như đặc tả đầu vào–đầu ra. Nếu muốn thảo luận về độ tin cậy hoặc chẩn đoán, chúng ta sẽ bao gồm khả năng cấu trúc mạch hoặc tính chất của các cổng có thể thay đổi tự phát. Để giải thích các điện dung ký sinh, chúng ta cần biểu diễn vị trí của các dây dẫn trên bảng mạch.

Nếu nhìn vào thế giới Wumpus, các cân nhắc tương tự cũng áp dụng. Mặc dù chúng ta có biểu diễn thời gian, nhưng nó có cấu trúc đơn giản: Không có gì xảy ra ngoại trừ khi tác nhân hành động, và mọi thay đổi đều tức thời. Một bản thể luận tổng quát hơn, phù hợp hơn với thế giới thực, sẽ cho phép các thay đổi đồng thời và kéo dài theo thời gian. Chúng ta cũng đã sử dụng vị từ *Pit* để chỉ những ô nào có hố. Chúng ta có thể cho phép nhiều loại hố khác nhau bằng cách có nhiều cá thể thuộc lớp hố, mỗi cá thể có các thuộc tính khác nhau. Tương tự, chúng ta có thể muốn bao gồm các loài động vật khác ngoài Wumpus. Có thể không thể xác định chính xác loài nào dựa trên các dấu hiệu cảm nhận có sẵn, vì vậy chúng ta cần xây dựng một hệ thống phân loại sinh học để giúp tác nhân dự đoán hành vi của những sinh vật sống trong hang động từ những manh mối ít ỏi.

Đối với bất kỳ bản thể luận chuyên dụng nào, chúng ta có thể thực hiện những thay đổi như trên để tiến gần hơn đến tính tổng quát. Một câu hỏi hiển nhiên đặt ra là: liệu tất cả các bản thể luận này có hội tụ về một bản thể luận đa dụng chung không? Sau nhiều thế kỷ nghiên cứu triết học và tính toán, câu trả lời là "Có thể". Trong mục này, chúng tôi trình bày một bản thể luận đa dụng tổng hợp các ý tưởng từ những nghiên cứu đó. Hai đặc điểm chính của các bản thể luận đa dụng phân biệt chúng với tập hợp các bản thể luận chuyên dụng:

- Một bản thể luận đa dụng phải có thể áp dụng trong hầu hết mọi lĩnh vực chuyên dụng (với việc bổ sung các tiên đề cụ thể của lĩnh vực đó). Điều này có nghĩa là không có vấn đề biểu diễn nào có thể bị bỏ qua hoặc che giấu.
- Trong bất kỳ lĩnh vực đủ phức tạp nào, các lĩnh vực tri thức khác nhau phải được thống nhất, vì việc suy luận và giải quyết vấn đề có thể liên quan đến nhiều lĩnh vực cùng một lúc. Ví dụ, một hệ thống robot sửa chữa mạch điện cần suy luận về mạch điện dựa trên kết nối điện và bố cục vật lý, đồng thời cũng cần suy luận về thời gian, cả để phân tích thời gian mạch và ước tính chi phí nhân công. Do đó, các câu mô tả thời gian phải có thể kết hợp với những câu mô tả bố cục không gian và phải hoạt động hiệu quả cho cả đơn vị nanogiây lẫn phút, angstrom lẫn mét.

Chúng ta phải thừa nhận ngay từ đầu rằng nỗ lực xây dựng bản thể luận đa dụng cho đến nay chỉ đạt được thành công hạn chế. Không có ứng dụng AI hàng đầu nào (như liệt kê trong Chương 1) sử dụng bản thể luận đa dụng—tất cả đều dựa vào kỹ thuật tri thức chuyên dụng và học máy. Các yếu tố xã hội/chính trị có thể khiến các bên cạnh tranh khó đồng thuận về một bản thể luận chung. Như Tom Gruber (2004) đã nói: "Mọi bản thể luận đều là một hiệp ước—một thỏa thuận xã hội—giữa những người có động cơ chung trong việc chia sẻ." Khi các mối quan tâm cạnh tranh lấn át động cơ chia sẻ, sẽ không thể có bản thể luận chung. Số lượng các bên liên quan càng ít thì việc tạo ra bản thể luận càng dễ dàng, và do đó, việc tạo ra một bản thể luận đa dụng khó hơn nhiều so với một bản thể luận hạn chế mục đích, chẳng hạn như Open Biomedical Ontology (Smith et al., 2007). Những bản thể luận hiện có được xây dựng theo bốn hướng chính:

4. **Bởi một nhóm các chuyên gia bản thể luận hoặc logic học**, những người thiết kế bản thể luận và viết các tiên đề. Hệ thống CYC chủ yếu được xây dựng theo cách này (Lenat và Guha, 1990).
5. **Bằng cách nhập khẩu các danh mục, thuộc tính và giá trị từ một hoặc nhiều cơ sở dữ liệu có sẵn**. DBpedia được xây dựng bằng cách nhập các dữ kiện có cấu trúc từ Wikipedia (Bizer et al., 2007).
6. **Bằng cách phân tích các tài liệu văn bản và trích xuất thông tin từ chúng**. TextRunner được xây dựng bằng cách đọc một kho ngữ liệu lớn các trang web (Banko và Etzioni, 2008).
7. **Bằng cách khuyến khích những người không chuyên nhập tri thức thông thường**. Hệ thống OpenMind được xây dựng bởi các tình nguyện viên đề xuất các dữ kiện bằng tiếng Anh (Singh et al., 2002; Chklovski và Gil, 2005).

Ví dụ, Google Knowledge Graph sử dụng nội dung bán cấu trúc từ Wikipedia, kết hợp với các nội dung khác được thu thập từ khắp nơi trên web dưới sự kiểm duyệt của con người. Nó chứa hơn 70 tỷ dữ kiện và cung cấp câu trả lời cho khoảng một phần ba các lượt tìm kiếm trên Google (Dong et al., 2014).

10.2 Danh mục và Đối tượng

Việc tổ chức các đối tượng thành các danh mục là một phần quan trọng của biểu diễn tri thức. Mặc dù tương tác với thế giới diễn ra ở cấp độ các đối tượng riêng lẻ, phần lớn lập luận lại diễn ra ở cấp độ danh mục. Ví dụ, một người mua sắm thường có mục tiêu mua một quả bóng rổ nói chung, chứ không phải một quả bóng rổ cụ thể như BB_9 . Các danh mục cũng giúp đưa ra dự đoán về các đối tượng sau khi chúng được phân loại. Người ta suy ra sự hiện diện của các đối tượng từ đầu vào tri giác, suy ra tư cách thành viên danh mục từ các thuộc tính được nhận thức của đối tượng, và sau đó sử dụng thông tin danh mục để dự đoán về đối tượng. Ví dụ, từ lớp da xanh và vàng loang lổ, đường kính một foot, hình bầu dục, thịt đỏ, hạt đen, và sự hiện diện ở khu trái cây, người ta có thể suy ra rằng một đối tượng là quả dưa hấu; từ đó, người ta suy ra rằng nó sẽ hữu ích cho món salad trái cây.

Có hai lựa chọn để biểu diễn các danh mục trong logic bậc nhất: vị từ và đối tượng. Nghĩa là, chúng ta có thể sử dụng vị từ $Basketball(b)$, hoặc chúng ta có thể vật chất hóa danh mục thành một đối tượng, $Basketballs$. Sau đó, chúng ta có thể nói $Member(b, Basketballs)$, viết tắt là $b \in Basketballs$, để chỉ rằng b là một thành viên của danh mục các quả bóng rổ. Chúng ta nói $Subset(Basketballs, Balls)$, viết tắt là $Basketballs \subset Balls$, để chỉ rằng $Basketballs$ là một danh mục con của $Balls$. Chúng ta sẽ sử dụng các thuật ngữ danh mục con, lớp con và tập hợp con thay thế cho nhau.

Các danh mục tổ chức tri thức thông qua kế thừa. Nếu chúng ta nói rằng tất cả các thể hiện của danh mục $Food$ đều có thể ăn được, và nếu chúng ta khẳng định rằng $Fruit$ là một lớp con của $Food$ và $Apples$ là một lớp con của $Fruit$, thì chúng ta có thể suy ra rằng mọi quả táo đều có thể ăn được. Chúng ta nói rằng các quả táo riêng lẻ kế thừa thuộc tính có thể ăn được, trong trường hợp này từ tư cách thành viên của chúng trong danh mục $Food$.

Các quan hệ lớp con tổ chức các danh mục thành một hệ thống phân cấp phân loại hoặc phân loại học. Các hệ thống phân loại đã được sử dụng một cách rõ ràng trong nhiều thế kỷ trong các lĩnh vực kỹ thuật. Hệ thống phân loại lớn nhất tổ chức khoảng 10 triệu loài còn sống và đã tuyệt chủng, nhiều trong số đó là bộ cánh cứng, thành một hệ thống phân cấp duy nhất; khoa học thư viện đã phát triển một hệ thống phân loại tất cả các lĩnh vực tri thức, được mã hóa như hệ thống thập phân Dewey; và các cơ quan thuế cùng các bộ phận chính phủ khác đã phát triển các hệ thống phân loại rộng lớn về nghề nghiệp và sản phẩm thương mại.

Logic bậc nhất giúp dễ dàng phát biểu các sự kiện về danh mục, bằng cách liên hệ các đối tượng với danh mục hoặc bằng cách lượng hóa các thành viên của chúng. Dưới đây là một số ví dụ:

- Một đối tượng là thành viên của một danh mục.
 $BB_9 \in Basketballs$
- Một danh mục là lớp con của một danh mục khác.
 $Basketballs \subset Balls$
- Tất cả các thành viên của một danh mục có một số thuộc tính.
 $(x \in Basketballs) \Rightarrow Spherical(x)$
- Các thành viên của một danh mục có thể được nhận biết bởi một số thuộc tính.
 $Orange(x) \wedge Round(x) \wedge Diameter(x) = 9.5'' \wedge x \in Balls \Rightarrow x \in Basketballs$
- Một danh mục nói chung có một số thuộc tính.
 $Dogs \in DomesticatedSpecies$

Lưu ý rằng vì *Dogs* là một danh mục và là thành viên của *DomesticatedSpecies*, nên *DomesticatedSpecies* phải là một danh mục của các danh mục. Tất nhiên, có nhiều ngoại lệ đối với các quy tắc trên (ví dụ, các quả bóng rổ bị thủng không phải là hình cầu); chúng ta sẽ giải quyết các ngoại lệ này sau.

Mặc dù quan hệ lớp con và thành viên là quan trọng nhất đối với các danh mục, chúng ta cũng muốn có thể phát biểu các quan hệ giữa các danh mục không phải là lớp con của nhau. Ví dụ, nếu chúng ta chỉ nói rằng *Undergraduates* và *GraduateStudents* là các lớp con của *Students*, thì chúng ta chưa nói rằng một sinh viên đại học không thể đồng thời là một sinh viên sau đại học. Chúng ta nói rằng hai hoặc nhiều danh mục là rời rạc nếu chúng không có thành viên chung. Chúng ta cũng có thể muốn nói rằng các lớp *Undergraduates* và *GraduateStudents* tạo thành một phân rã đầy đủ của các sinh viên đại học. Một phân rã đầy đủ của các tập hợp rời rạc được gọi là một phân hoạch. Dưới đây là một số ví dụ khác về ba khái niệm này:

- $Disjoint(\{Animals, Vegetables\})$
- $ExhaustiveDecomposition(\{Americans, Canadians, Mexicans\}, NorthAmericans)$
- $Partition(\{Animals, Plants, Fungi, Protista, Monera\}, LivingThings)$.

(Lưu ý rằng *ExhaustiveDecomposition* của *NorthAmericans* không phải là một *Partition*, vì một số người có hai quốc tịch.) Ba vị từ này được định nghĩa như sau:

- $Disjoint(s) \Leftrightarrow (\forall c_1, c_2 \in s \wedge c_1 \neq c_2 \Rightarrow Intersection(c_1, c_2) = \{\})$

- $ExhaustiveDecomposition(s, c) \Leftrightarrow (\forall i \in c \Leftrightarrow \exists c_2 \in s \wedge i \in c_2)$
- $Partition(s, c) \Leftrightarrow Disjoint(s) \wedge ExhaustiveDecomposition(s, c).$

Các danh mục cũng có thể được định nghĩa bằng cách cung cấp các điều kiện cần và đủ cho tư cách thành viên. Ví dụ, một người độc thân là một nam giới trưởng thành chưa kết hôn:

$$x \in Bachelors \Leftrightarrow Unmarried(x) \wedge x \in Adults \wedge x \in Males.$$

Như chúng ta thảo luận trong phần bên lề về các loại tự nhiên ở trang 338, các định nghĩa logic chặt chẽ cho các danh mục thường chỉ khả thi đối với các thuật ngữ hình thức nhân tạo, chứ không phải cho các đối tượng thông thường. Nhưng các định nghĩa không phải lúc nào cũng cần thiết.

10.2.1 Cấu trúc vật lý

Ý tưởng rằng một đối tượng có thể là một phần của một đối tượng khác là một ý tưởng quen thuộc. Mũi của một người là một phần của đầu người đó, Romania là một phần của châu Âu, và chương này là một phần của cuốn sách này. Chúng ta sử dụng quan hệ *PartOf* chung để nói rằng một thứ là một phần của một thứ khác. Các đối tượng có thể được nhóm thành các hệ thống phân cấp *PartOf*, gọi nhớ đến hệ thống phân cấp *Subset*:

- $PartOf(Bucharest, Romania)$
- $PartOf(Romania, EasternEurope)$
- $PartOf(EasternEurope, Europe)$
- $PartOf(Europe, Earth).$

Quan hệ *PartOf* có tính bắc cầu và phản xạ; nghĩa là:

- $PartOf(x, y) \wedge PartOf(y, z) \Rightarrow PartOf(x, z)$
- $PartOf(x, x).$

Do đó, chúng ta có thể kết luận $PartOf(Bucharest, Earth)$. Các danh mục của các đối tượng tổng hợp thường được đặc trưng bởi các quan hệ cấu trúc giữa các phần. Ví dụ, một sinh vật hai chân là một đối tượng có chính xác hai chân gắn vào một thân:

$$\begin{aligned} Biped(a) \Rightarrow \exists l_1, l_2, b \text{ } & Leg(l_1) \wedge Leg(l_2) \wedge Body(b) \wedge PartOf(l_1, a) \wedge PartOf(l_2, a) \\ & \wedge PartOf(b, a) \wedge Attached(l_1, b) \wedge Attached(l_2, b) \wedge l_1 \\ & \neq l_2 \wedge [\forall l_3 \text{ } Leg(l_3) \wedge PartOf(l_3, a) \Rightarrow (l_3 = l_1 \vee l_3 = l_2)]. \end{aligned}$$

Ký hiệu cho "chính xác hai" hơi phức tạp; chúng ta buộc phải nói rằng có hai chân, chúng không giống nhau, và nếu ai đó đề xuất một chân thứ ba, thì nó phải giống một trong hai chân kia. Trong Mục 10.5.2, chúng ta sẽ mô tả một hình thức gọi là logic mô tả giúp dễ dàng biểu diễn các ràng buộc như "chính xác hai."

Chúng ta có thể định nghĩa một quan hệ *PartPartition* tương tự như quan hệ *Partition* cho các danh mục. (Xem Bài tập 10.DECM.) Một đối tượng được cấu thành từ các phần trong *PartPartition* của nó và có thể được xem như có được một số thuộc tính từ các phần đó. Ví dụ,

khối lượng của một đối tượng tổng hợp là tổng khối lượng của các phần. Lưu ý rằng điều này không áp dụng cho các danh mục, vốn không có khối lượng, mặc dù các phần tử của chúng có thể có.

Cũng hữu ích khi định nghĩa các đối tượng tổng hợp với các phần xác định nhưng không có cấu trúc cụ thể. Ví dụ, chúng ta có thể muốn nói "Những quả táo trong túi này nặng hai pound." Cầm đồ sẽ là gán trọng lượng này cho tập hợp các quả táo trong túi, nhưng đây sẽ là một sai lầm vì tập hợp là một khái niệm toán học trừu tượng có các phần tử nhưng không có trọng lượng. Thay vào đó, chúng ta cần một khái niệm mới, mà chúng ta sẽ gọi là một nhóm. Ví dụ, nếu các quả táo là $Apple_1, Apple_2$ và $Apple_3$, thì:

$$BunchOf(\{Apple_1, Apple_2, Apple_3\})$$

biểu thị đối tượng tổng hợp với ba quả táo như các phần (không phải là các phần tử). Sau đó, chúng ta có thể sử dụng nhóm như một đối tượng bình thường, dù không có cấu trúc. Lưu ý rằng $BunchOf(\{x\}) = x$. Hơn nữa, $BunchOf(Apples)$ là đối tượng tổng hợp bao gồm tất cả các quả táo—không nên nhầm lẫn với $Apples$, danh mục hoặc tập hợp tất cả các quả táo.

Chúng ta có thể định nghĩa $BunchOf$ theo quan hệ $PartOf$. Rõ ràng, mỗi phần tử của s là một phần của $BunchOf(s)$:

$$\forall x \, x \in s \Rightarrow PartOf(x, BunchOf(s)).$$

Hơn nữa, $BunchOf(s)$ là đối tượng nhỏ nhất thỏa mãn điều kiện này. Nói cách khác, $BunchOf(s)$ phải là một phần của bất kỳ đối tượng nào có tất cả các phần tử của s như các phần:

$$\forall y \, [\forall x \, x \in s \Rightarrow PartOf(x, y)] \Rightarrow PartOf(BunchOf(s), y).$$

Các tiên đề này là một ví dụ về kỹ thuật tối thiểu hóa logic, nghĩa là định nghĩa một đối tượng như là đối tượng nhỏ nhất thỏa mãn các điều kiện nhất định.

10.2.2 Đo lường

Trong cả các lý thuyết khoa học và thông thường về thế giới, các đối tượng có chiều cao, khối lượng, chi phí, v.v. Các giá trị mà chúng ta gán cho các thuộc tính này được gọi là các đại lượng đo. Các đại lượng đo định lượng thông thường khá dễ biểu diễn. Chúng ta tưởng tượng rằng vũ trụ bao gồm các "đối tượng đo" trừu tượng, chẳng hạn như độ dài là độ dài của đoạn thẳng này: $|\text{---}|$. Chúng ta có thể gọi độ dài này là 1.5 inch hoặc 3.81 cm. Do đó, cùng một độ dài có các tên khác nhau trong ngôn ngữ của chúng ta. Chúng ta biểu diễn độ dài bằng một hàm đơn vị nhận một số làm đối số. (Một cách tiếp cận khác được khám phá trong Bài tập 10.ALTM.) Hàm đơn vị

Nếu đoạn thẳng được gọi là L_1 , chúng ta có thể viết:

$$Length(L_1) = Inches(1.5) = Centimeters(3.81).$$

Việc chuyển đổi giữa các đơn vị được thực hiện bằng cách đặt bội số của một đơn vị bằng một đơn vị khác:

$$Centimeters(2.54 \times d) = Inches(d).$$

Các tiên đề tương tự có thể được viết cho pound và kilogram, giây và ngày, đô la và cent. Các đại lượng đo có thể được sử dụng để mô tả các đối tượng như sau:

- $Diameter(Basketball_{12}) = Inches(9.5)$
- $ListPrice(Basketball_{12}) = \(19)
- $Weight(BunchOf(\{Apple_1, Apple_2, Apple_3\})) = Pounds(2)$
- $d \in Days \Rightarrow Duration(d) = Hours(24).$

Lưu ý rằng $\$(1)$ không phải là một tờ đô la—nó là một mức giá. Người ta có thể có hai tờ đô la, nhưng chỉ có một đối tượng duy nhất được gọi là $\$(1)$. Cũng lưu ý rằng, mặc dù $Inches(0)$ và $Centimeters(0)$ đều chỉ cùng một độ dài bằng không, chúng không giống với các đại lượng đo bằng không khác, chẳng hạn như $Seconds(0)$.

Các Loại Tự Nhiên

Một số danh mục có định nghĩa chặt chẽ: một vật thể là hình tam giác nếu và chỉ nếu nó là đa giác có ba cạnh. Mặt khác, hầu hết các danh mục trong thế giới thực không có định nghĩa rõ ràng; những danh mục này được gọi là **loại tự nhiên**. Ví dụ, cà chua thường có màu đỏ xỉn, hình cầu, có vết lõm ở phía trên nơi cuống đã gắn vào, đường kính khoảng hai đến bốn inch, với lớp vỏ mỏng nhưng dai, và bên trong có thịt, hạt và nước. Tuy nhiên, có sự biến đổi: một số quả cà chua có màu vàng hoặc cam, cà chua chưa chín có màu xanh lá cây, một số nhỏ hơn hoặc lớn hơn mức trung bình, và cà chua bi thì đồng loạt nhỏ. Thay vì có một định nghĩa hoàn chỉnh về cà chua, chúng ta có một tập hợp các đặc điểm giúp xác định những vật thể rõ ràng là cà chua điển hình, nhưng có thể không xác định dứt khoát những vật thể khác. (Liệu có thể tồn tại một quả cà chua có lông như quả đào không?)

Điều này đặt ra vấn đề cho một tác nhân logic. Tác nhân không thể chắc chắn rằng một vật thể mà nó nhận thức được là cà chua, và ngay cả khi chắc chắn, nó cũng không thể xác định được vật thể đó có những thuộc tính nào của cà chua điển hình. Vấn đề này là hệ quả tất yếu của việc hoạt động trong môi trường chỉ quan sát được một phần.

Một cách tiếp cận hữu ích là tách biệt điều gì đúng với tất cả các thể hiện của một danh mục và điều gì chỉ đúng với các thể hiện điển hình. Vì vậy, ngoài danh mục *Cà chua*, chúng ta cũng sẽ có danh mục *ĐiểnHình(Cà chua)*. Ở đây, hàm *ĐiểnHình* ánh xạ một danh mục đến phân lớp chỉ chứa các thể hiện điển hình:

$$\text{ĐiểnHình}(c) \subseteq c.$$

Hầu hết kiến thức về các loại tự nhiên thực sự sẽ là về các thể hiện điển hình của chúng:

$$x \in \text{ĐiểnHình}(Càchua) \Rightarrow \text{Đỏ}(x) \wedge \text{Tròn}(x).$$

Như vậy, chúng ta có thể ghi lại những sự thật hữu ích về các danh mục mà không cần định nghĩa chính xác. Khó khăn trong việc đưa ra định nghĩa chính xác cho hầu hết các danh mục tự nhiên đã được Wittgenstein (1953) giải thích sâu sắc. Ông sử dụng ví dụ về *trò chơi* để chỉ ra rằng các thành viên của một danh mục chia sẻ "những nét tương đồng gia đình" thay vì các đặc điểm cần và đủ: định nghĩa chặt chẽ nào bao gồm cờ vua, trò đuổi bắt, bài solitaire và bóng ném?

Tính hữu ích của khái niệm định nghĩa chặt chẽ cũng bị Quine (1953) thách thức. Ông chỉ ra rằng ngay cả định nghĩa "người đàn ông độc thân" là nam giới trưởng thành chưa lập gia đình cũng đáng ngờ; người ta có thể, ví dụ, đặt câu hỏi về một tuyên bố như "Giáo hoàng là người đàn ông độc thân." Mặc dù không hoàn toàn *sai*, cách dùng này chắc chắn *không phù hợp* vì nó gây ra những suy luận không mong muốn từ phía người nghe. Căng thẳng này có lẽ có thể được giải quyết bằng cách phân biệt giữa các định nghĩa logic phù hợp cho biểu diễn tri thức nội bộ và các tiêu chí tinh tế hơn cho cách sử dụng ngôn ngữ phù hợp. Cách thứ hai có thể đạt được bằng cách "lọc" các khẳng định được suy ra từ cách thứ nhất. Cũng có thể những thất bại trong cách sử dụng ngôn ngữ đóng vai trò là phản hồi để sửa đổi các định nghĩa nội bộ, sao cho việc lọc trở nên không cần thiết.

Các đại lượng đo định lượng đơn giản rất dễ biểu diễn. Các đại lượng đo khác đặt ra nhiều vấn đề hơn, vì chúng không có thang đo giá trị được thống nhất. Các bài tập có độ khó, các món tráng miệng có độ ngon, và các bài thơ có vẻ đẹp, nhưng không thể gán số cho những phẩm chất này. Người ta có thể, trong một khoảnh khắc thuần túy của kế toán, bỏ qua các thuộc tính như vậy như là vô ích cho mục đích lập luận logic; hoặc, tệ hơn, cố gắng áp đặt một thang đo số cho vẻ đẹp. Đây sẽ là một sai lầm nghiêm trọng, vì nó không cần thiết. Khía cạnh quan trọng nhất của các đại lượng đo không phải là các giá trị số cụ thể, mà là thực tế rằng các đại lượng đo có thể được sắp xếp.

Mặc dù các đại lượng đo không phải là số, chúng ta vẫn có thể so sánh chúng, sử dụng một ký hiệu sắp xếp như $>$. Ví dụ, chúng ta có thể tin rằng các bài tập của Norvig khó hơn của Russell, và người ta đạt điểm thấp hơn trong các bài tập khó hơn:

$$\begin{aligned} e_1 \in \text{Exercises} \wedge e_2 \in \text{Exercises} \wedge \text{Wrote}(\text{Norvig}, e_1) \wedge \text{Wrote}(\text{Russell}, e_2) \\ \Rightarrow \text{Difficulty}(e_1) > \text{Difficulty}(e_2). e_1 \in \text{Exercises} \wedge e_2 \\ \in \text{Exercises} \wedge \text{Difficulty}(e_1) > \text{Difficulty}(e_2) \Rightarrow \text{ExpectedScore}(e_1) \\ < \text{ExpectedScore}(e_2). \end{aligned}$$

Điều này đủ để cho phép một người quyết định nên làm bài tập nào, ngay cả khi không có giá trị số nào cho độ khó được sử dụng. (Tuy nhiên, người ta phải khám phá xem ai đã viết bài tập nào.) Các mối quan hệ đơn điệu như vậy giữa các đại lượng đo tạo thành cơ sở cho lĩnh vực vật lý định tính, một lĩnh vực con của AI nghiên cứu cách lập luận về các hệ thống vật lý mà không cần đi sâu vào các phương trình chi tiết và mô phỏng số. Vật lý định tính được thảo luận trong phần ghi chú lịch sử.

10.2.3 Đối tượng: Vật thể và chất liệu

Thế giới thực có thể được xem như bao gồm các đối tượng nguyên thủy (ví dụ, các hạt nguyên tử) và các đối tượng tổng hợp được xây dựng từ chúng. Bằng cách lập luận ở cấp độ các đối tượng lớn như táo và ô tô, chúng ta có thể vượt qua sự phức tạp khi phải xử lý một số lượng lớn các đối tượng nguyên thủy riêng lẻ. Tuy nhiên, có một phần đáng kể của thực tế dường như chống lại bất kỳ sự cá thể hóa rõ ràng nào—sự phân chia thành các đối tượng riêng biệt. Chúng ta gọi phần này bằng cái tên chung là chất liệu. Ví dụ, giả sử tôi có một ít bơ và một con lợn đất trước mặt. Tôi có thể nói có một con lợn đất, nhưng không có số lượng rõ ràng các "đối tượng bơ," vì bất kỳ phần nào của một đối tượng bơ cũng là một đối tượng bơ, ít nhất cho đến khi chúng ta đi đến các phần rất nhỏ. Đây là sự khác biệt chính giữa chất liệu và vật thể. Nếu chúng ta cắt đôi một con lợn đất,

chúng ta không nhận được hai con lợn đất (thật không may). Ngôn ngữ tiếng Anh phân biệt rõ ràng giữa chất liệu và vật thể. Chúng ta nói "a aardvark," nhưng, ngoại trừ trong các nhà hàng California sang trọng, người ta không thể nói "a butter." Các nhà ngôn ngữ học phân biệt giữa danh từ đếm được, như aardvarks, lỗ hồng và định lý, và danh từ không đếm được, như bơ, nước và năng lượng.

Có một số bản thể học cạnh tranh tuyên bố xử lý sự phân biệt này. Ở đây, chúng tôi mô tả chỉ một; những bản thể học khác được đề cập trong phần ghi chú lịch sử. Để biểu diễn chất liệu một cách chính xác, chúng ta bắt đầu với điều hiển nhiên. Chúng ta cần có ít nhất các "khối" chất liệu thô mà chúng ta tương tác như các đối tượng trong bản thể học của chúng ta. Ví dụ, chúng ta có thể nhận ra một khối bơ là khối được để lại trên bàn đêm hôm trước; chúng ta có thể nhặt nó lên, cân nó, bán nó, hoặc bất cứ điều gì. Theo nghĩa này, nó là một đối tượng giống như con lợn đất. Hãy gọi nó là *Butters*. Chúng ta cũng định nghĩa danh mục *Butter*. Một cách không chính thức, các phần tử của nó sẽ là tất cả những thứ mà người ta có thể nói "Nó là bơ," bao gồm cả *Butters*. Với một số lưu ý về các phần rất nhỏ mà chúng tôi sẽ bỏ qua ở đây, bất kỳ phần nào của một đối tượng bơ cũng là một đối tượng bơ:

$$b \in Butter \wedge PartOf(p, b) \Rightarrow p \in Butter.$$

Bây giờ chúng ta có thể nói rằng bơ tan chảy ở khoảng 30 độ C:

$$b \in Butter \Rightarrow MeltingPoint(b, Centigrade(30)).$$

Chúng ta có thể tiếp tục nói rằng bơ có màu vàng, ít đặc hơn nước, mềm ở nhiệt độ phòng, có hàm lượng chất béo cao, v.v. Mặt khác, bơ không có kích thước, hình dạng hoặc trọng lượng cụ thể. Chúng ta có thể định nghĩa các danh mục bơ chuyên biệt hơn như *UnsaltedButter*, cũng là một loại chất liệu. Lưu ý rằng danh mục *PoundOfButter*, bao gồm tất cả các đối tượng bơ nặng một pound, không phải là một loại chất liệu. Nếu chúng ta cắt một pound bơ làm đôi, chúng ta sẽ không, tiếc thay, nhận được hai pound bơ.

Điều gì thực sự đang diễn ra là thế này: một số thuộc tính là nội tại: chúng thuộc về bản chất của đối tượng, chứ không phải đối tượng như một tổng thể. Khi bạn cắt một thể hiện của chất liệu làm đôi, hai phần vẫn giữ các thuộc tính nội tại—những thứ như mật độ, điểm sôi, hương vị, màu sắc, quyền sở hữu, v.v. Mặt khác, các thuộc tính ngoại lai của chúng—trọng lượng, chiều dài, hình dạng, v.v.—không được giữ lại khi phân chia. Một danh mục các đối tượng mà trong định nghĩa của nó chỉ bao gồm các thuộc tính nội tại sau đó là một chất liệu, hoặc danh từ không đếm được; một lớp bao gồm bất kỳ thuộc tính ngoại lai nào trong định nghĩa của nó là một danh từ đếm được. *Stuff* và *Thing* là các danh mục chất liệu và đối tượng tổng quát nhất, tương ứng.

10.3 Sự kiện

Trong Mục 7.7.1, chúng ta đã thảo luận về hành động: những điều xảy ra, như *Shoot_i*; và trạng thái động (fluents): các khía cạnh của thế giới thay đổi, như *HaveArrow_i*. Cả hai đều được biểu diễn dưới dạng mệnh đề, và chúng ta sử dụng các tiên đề trạng thái kế tiếp để nói rằng một trạng thái động sẽ đúng tại thời điểm $t + 1$ nếu hành động tại thời điểm t khiến nó trở thành đúng, hoặc nếu nó đã đúng tại thời điểm t và hành động không khiến nó trở thành sai. Điều này áp dụng cho một thế giới trong đó các hành động là rời rạc, tức thời, xảy ra một lần và không có biến thể trong cách thực hiện (nghĩa là chỉ có một loại hành động *Shoot*, không có sự phân biệt giữa bắn nhanh,

chậm, lo lắng, v.v.). Nhưng khi chuyển từ các miền đơn giản sang thế giới thực, chúng ta phải đối mặt với một phạm vi hành động hoặc sự kiện phong phú hơn nhiều.

Hãy xem xét một hành động liên tục, như việc đổ đầy bồn tắm. Một tiên đề trạng thái kế tiếp có thể nói rằng bồn tắm trống trước khi hành động và đầy khi hành động kết thúc, nhưng nó không thể mô tả điều gì xảy ra trong quá trình hành động. Nó cũng không thể dễ dàng mô tả hai hành động xảy ra đồng thời—chẳng hạn như đánh răng trong khi chờ bồn tắm đầy. Để xử lý các trường hợp như vậy, chúng ta giới thiệu một phương pháp tiếp cận được gọi là **phép tính sự kiện** (event calculus).

Các đối tượng của phép tính sự kiện là sự kiện, trạng thái động và điểm thời gian. $At(Shankar, Berkeley)$ là một trạng thái động: một đối tượng đề cập đến sự kiện Shankar đang ở Berkeley. Sự kiện E_1 của Shankar bay từ San Francisco đến Washington, D.C., được mô tả như sau:

$$E_1 \in Flying \wedge Flyer(E_1, Shankar) \wedge Origin(E_1, SF) \wedge Destination(E_1, DC).$$

Ở đây, *Flying* là danh mục của tất cả các sự kiện bay. Bằng cách vật chất hóa sự kiện, chúng ta có thể thêm bất kỳ lượng thông tin tùy ý nào về chúng. Ví dụ, chúng ta có thể nói rằng chuyến bay của Shankar bị xóc với $Bumpy(E_1)$. Trong một bản thể học mà sự kiện là các vị từ n -ngôi, sẽ không có cách nào để thêm thông tin bổ sung như vậy; việc chuyển sang vị từ $n + 1$ -ngôi không phải là giải pháp có thể mở rộng.

Để khẳng định rằng một trạng thái động thực sự đúng bắt đầu từ một thời điểm t_1 và tiếp tục đến thời điểm t_2 , chúng ta sử dụng vị từ T , như trong $T(At(Shankar, Berkeley), t_1, t_2)$. Tương tự, chúng ta sử dụng $Happens(E_1, t_1, t_2)$ để nói rằng sự kiện E_1 thực sự đã xảy ra, bắt đầu từ thời điểm t_1 và kết thúc tại thời điểm t_2 . Tập hợp đầy đủ các vị từ cho một phiên bản của phép tính sự kiện là:

- $T(f, t_1, t_2)$: Trạng thái động f đúng cho mọi thời điểm giữa t_1 và t_2 .
- $Happens(e, t_1, t_2)$: Sự kiện e bắt đầu tại thời điểm t_1 và kết thúc tại t_2 .
- $Initiates(e, f, t)$: Sự kiện e khiến trạng thái động f trở thành đúng tại thời điểm t .
- $Terminates(e, f, t)$: Sự kiện e khiến trạng thái động f không còn đúng tại thời điểm t .
- $Initiated(f, t_1, t_2)$: Trạng thái động f trở thành đúng tại một thời điểm nào đó giữa t_1 và t_2 .
- $Terminated(f, t_1, t_2)$: Trạng thái động f không còn đúng tại một thời điểm nào đó giữa t_1 và t_2 .
- $t_1 < t_2$: Điểm thời gian t_1 xảy ra trước thời gian t_2 .

Chúng ta có thể mô tả tác động của một sự kiện bay:

$$\begin{aligned} E &= Flying(a, here, there) \wedge Happens(E, t_1, t_2) \\ \Rightarrow &Terminates(E, At(a, here), t_1) \wedge Initiates(E, At(a, there), t_2). \end{aligned}$$

Chúng ta giả định một sự kiện đặc biệt, *Start*, mô tả trạng thái ban đầu bằng cách nói trạng thái động nào là đúng (sử dụng *Initiates*) hoặc sai (sử dụng *Terminated*) tại thời điểm bắt đầu. Sau đó, chúng ta có thể mô tả trạng thái động nào đúng tại thời điểm nào bằng một cặp tiên đề cho T và $\neg T$ tuân theo định dạng chung tương tự như các tiên đề trạng thái kế tiếp: Giả sử một sự kiện xảy ra giữa thời điểm t_1 và t_3 , và tại t_2 ở đâu đó trong khoảng thời gian đó, sự kiện thay đổi giá trị của trạng thái động f , hoặc khởi tạo nó (làm cho nó đúng) hoặc kết thúc nó (làm cho nó sai). Sau đó, tại thời điểm t_4 trong tương lai, nếu không có sự kiện can thiệp nào khác thay đổi trạng thái động (đã kết thúc hoặc khởi tạo nó, tương ứng), thì trạng thái động sẽ duy trì giá trị của nó. Một cách hình thức, các tiên đề là:

$$\begin{aligned} &Happens(e, t_1, t_3) \wedge Initiates(e, f, t_2) \wedge \neg Terminated(f, t_2, t_4) \wedge t_1 \leq t_2 \leq t_3 \leq t_4 \\ &\Rightarrow T(f, t_2, t_4) Happens(e, t_1, t_3) \wedge Terminates(e, f, t_2) \wedge \neg Initiated(f, t_2, t_4) \\ &\wedge t_1 \leq t_2 \leq t_3 \leq t_4 \Rightarrow \neg T(f, t_2, t_4) \end{aligned}$$

trong đó *Terminated* và *Initiated* được định nghĩa như sau:

$$\begin{aligned} Terminated(f, t_1, t_5) &\Leftrightarrow \exists e, t_2, t_3, t_4 Happens(e, t_2, t_4) \wedge Terminates(e, f, t_3) \wedge t_1 \leq t_2 \\ &\leq t_3 \leq t_4 \leq t_5 Initiated(f, t_1, t_5) \\ &\Leftrightarrow \exists e, t_2, t_3, t_4 Happens(e, t_2, t_4) \wedge Initiates(e, f, t_3) \wedge t_1 \leq t_2 \leq t_3 \leq t_4 \leq t_5 \end{aligned}$$

Chúng ta có thể mở rộng phép tính sự kiện để biểu diễn các sự kiện đồng thời (như hai người cần thiết để chơi bập bênh), sự kiện ngoại sinh (như gió di chuyển một vật thể), sự kiện liên tục (như thủy triều dâng), sự kiện không xác định (như tung đồng xu và nhận được mặt ngựa hoặc mặt sấp), và các phức tạp khác.

10.3.1 Thời gian

Phép tính sự kiện mở ra khả năng nói về điểm thời gian và khoảng thời gian. Chúng ta sẽ xem xét hai loại khoảng thời gian: khoảnh khắc và khoảng thời gian mở rộng. Sự khác biệt là chỉ khoảnh khắc có thời lượng bằng không:

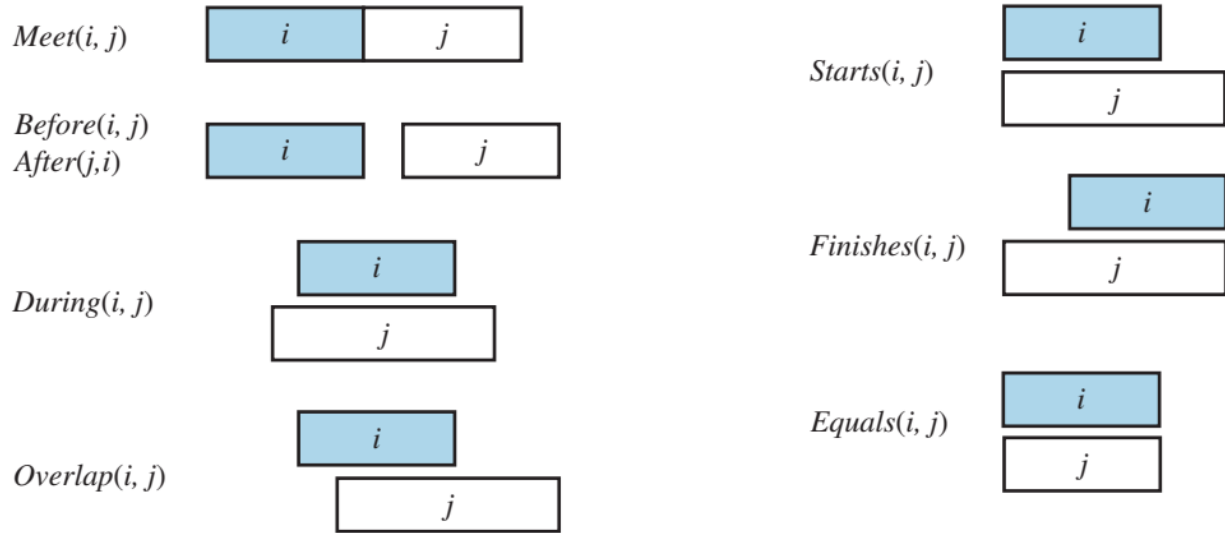
$$\begin{aligned} Partition(\{Moments, ExtendedIntervals\}, Intervals) &i \in Moments \Leftrightarrow Duration(i) \\ &= Seconds(0). \end{aligned}$$

Tiếp theo, chúng ta tạo ra một thang thời gian và liên kết các điểm trên thang đó với các khoảnh khắc, cho chúng ta thời gian tuyệt đối. Thang thời gian là tùy ý; chúng ta sẽ đo nó bằng giây và nói rằng khoảnh khắc lúc nửa đêm (GMT) ngày 1 tháng 1 năm 1900 có thời gian 0. Các hàm *Begin* và *End* chỉ ra khoảnh khắc sớm nhất và muộn nhất trong một khoảng thời gian, và hàm *Time* trả về điểm trên thang thời gian cho một khoảnh khắc. Hàm *Duration* cho biết sự khác biệt giữa thời gian kết thúc và thời gian bắt đầu.

$$\begin{aligned} Interval(i) &\Rightarrow Duration(i) = (Time(End(i)) - Time(Begin(i))).Time(Begin(AD1900)) \\ &= Seconds(0).Time(Begin(AD2001)) \\ &= Seconds(3187324800).Time(End(AD2001)) \\ &= Seconds(3218860800).Duration(AD2001) = Seconds(31536000). \end{aligned}$$

Để làm cho các con số này dễ đọc hơn, chúng ta cũng giới thiệu một hàm *Date*, nhận sáu đối số (giờ, phút, giây, ngày, tháng và năm) và trả về một điểm thời gian:

$$\begin{aligned} \text{Time}(\text{Begin}(\text{AD2001})) &= \text{Date}(0,0,0,1,\text{Jan},2001) - \text{Date}(0,20,21,24,1,1995) \\ &= \text{Seconds}(3000000000). \end{aligned}$$



Hình 10.2 Các vị từ trên khoảng thời gian.

Hai khoảng thời gian *Meet* nếu thời gian kết thúc của khoảng thứ nhất bằng thời gian bắt đầu của khoảng thứ hai. Tập hợp đầy đủ các quan hệ khoảng thời gian (Allen, 1983) được hiển thị dưới đây và trong Hình 10.2:

- $\text{Meet}(i, j) \Leftrightarrow \text{End}(i) = \text{Begin}(j)$
- $\text{Before}(i, j) \Leftrightarrow \text{End}(i) < \text{Begin}(j)$
- $\text{After}(j, i) \Leftrightarrow \text{Before}(i, j)$
- $\text{During}(i, j) \Leftrightarrow \text{Begin}(j) < \text{Begin}(i) < \text{End}(i) < \text{End}(j)$
- $\text{Overlap}(i, j) \Leftrightarrow \text{Begin}(i) < \text{Begin}(j) < \text{End}(i) < \text{End}(j)$
- $\text{Starts}(i, j) \Leftrightarrow \text{Begin}(i) = \text{Begin}(j)$
- $\text{Finishes}(i, j) \Leftrightarrow \text{End}(i) = \text{End}(j)$
- $\text{Equals}(i, j) \Leftrightarrow \text{Begin}(i) = \text{Begin}(j) \wedge \text{End}(i) = \text{End}(j)$

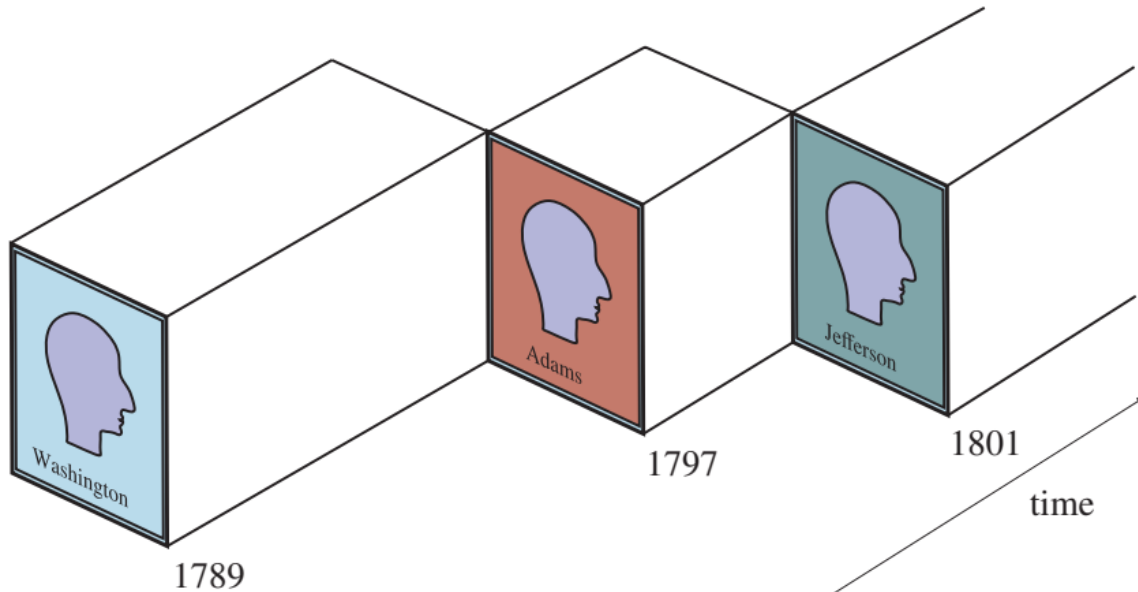
Tất cả đều có ý nghĩa trực quan, ngoại trừ *Overlap*: chúng ta thường nghĩ về sự chồng chéo là đối xứng (nếu i chồng lên j thì j chồng lên i), nhưng trong định nghĩa này, $\text{Overlap}(i, j)$ chỉ đúng nếu i bắt đầu trước j . Kinh nghiệm cho thấy định nghĩa này hữu ích hơn khi viết các tiên đề.

Để nói rằng triều đại của Elizabeth II ngay lập tức sau triều đại của George VI, và triều đại của Elvis chồng lấn với thập niên 1950, chúng ta có thể viết như sau:

$$\begin{aligned} &\text{Meets}(\text{ReignOf}(\text{GeorgeVI}), \text{ReignOf}(\text{ElizabethII})). \text{Overlap}(\text{Fifties}, \text{ReignOf}(\text{Elvis})). \text{Begin}(\text{Fifties}) \\ &= \text{Begin}(\text{AD1950}). \text{End}(\text{Fifties}) = \text{End}(\text{AD1959}). \end{aligned}$$

10.3.2 Trạng thái động và đối tượng

Các đối tượng vật lý có thể được xem như các sự kiện tổng quát, theo nghĩa là một đối tượng vật lý là một phần của không-thời gian. Ví dụ, Hoa Kỳ có thể được coi là một sự kiện bắt đầu vào năm 1776 như một liên minh của 13 tiểu bang và vẫn đang tiếp diễn cho đến ngày nay như một liên minh của 50 tiểu bang. Chúng ta có thể mô tả các thuộc tính thay đổi của Hoa Kỳ bằng các trạng thái động, như *Population(USA)*. Một thuộc tính của Hoa Kỳ thay đổi bốn hoặc tám năm một lần, trừ những sự cố, là tổng thống của nó. Người ta có thể đề xuất rằng *President(USA)* là một thuật ngữ logic biểu thị một đối tượng khác nhau tại các thời điểm khác nhau.



Hình 10.3 Góc nhìn sơ đồ về đối tượng *President(USA)* trong những năm đầu.

Thật không may, điều này là không thể, vì một thuật ngữ biểu thị chính xác một đối tượng trong một cấu trúc mô hình nhất định. (Thuật ngữ *President(USA, t)* có thể biểu thị các đối tượng khác nhau, tùy thuộc vào giá trị của t , nhưng bản thể học của chúng ta tách biệt chỉ số thời gian khỏi trạng thái động.) Khả năng duy nhất là *President(USA)* biểu thị một đối tượng duy nhất bao gồm những người khác nhau tại các thời điểm khác nhau. Đó là đối tượng mà từ năm 1789 đến 1797 là George Washington, từ năm 1797 đến 1801 là John Adams, và cứ tiếp tục, như trong Hình 10.3. Để nói rằng George Washington là tổng thống trong suốt năm 1790, chúng ta có thể viết:

$$T(\text{Equals}(\text{President(USA)}, \text{GeorgeWashington}), \text{Begin(AD1790)}, \text{End(AD1790)}).$$

Chúng ta sử dụng ký hiệu hàm *Equals* thay vì vị từ logic tiêu chuẩn $=$, vì chúng ta không thể có một vị từ làm đối số cho T , và vì cách diễn giải không phải là *GeorgeWashington* và *President(USA)* giống hệt nhau về mặt logic vào năm 1790; sự giống hệt về logic không phải là thứ có thể thay đổi theo thời gian. Sự giống hệt là giữa các sự kiện con của các đối tượng *President(USA)* và *GeorgeWashington* được xác định bởi khoảng thời gian 1790.

10.4 Đối Tượng Tinh Thần và Logic Hình Thái

Các tác nhân mà chúng ta đã xây dựng cho đến nay đều có niềm tin và có thể suy luận ra các niềm tin mới. Tuy nhiên, không có tác nhân nào trong số đó có kiến thức về niềm tin hoặc về quá trình suy luận. Kiến thức về kiến thức và quá trình lập luận của bản thân rất hữu ích để kiểm soát suy luận. Ví dụ, giả sử Alice hỏi "căn bậc hai của 1764 là bao nhiêu" và Bob trả lời "Tôi không biết." Nếu Alice nói "hãy nghĩ kỹ hơn," Bob nên nhận ra rằng với một chút suy nghĩ thêm, câu hỏi này thực sự có thể được trả lời. Mặt khác, nếu câu hỏi là "Hiện tại tổng thống có đang ngồi không?" thì Bob nên nhận ra rằng việc suy nghĩ thêm khó có thể giúp ích. Kiến thức về kiến thức của các tác nhân khác cũng quan trọng; Bob nên nhận ra rằng tổng thống biết điều đó.

Chúng ta cần một mô hình về các đối tượng tinh thần tồn tại trong đầu ai đó (hoặc trong cơ sở tri thức của một thực thể) và về các quá trình tinh thần thao tác các đối tượng tinh thần đó. Mô hình không cần phải chi tiết. Chúng ta không cần dự đoán chính xác một tác nhân cụ thể sẽ mất bao nhiêu mili giây để đưa ra một suy luận. Chúng ta sẽ hài lòng nếu có thể kết luận rằng người mẹ biết liệu bà ấy có đang ngồi hay không.

Chúng ta bắt đầu với các thái độ mệnh đề mà một tác nhân có thể có đối với các đối tượng tinh thần: các thái độ như *Tin*, *Biết*, *Muốn*, và *Thông báo*. Khó khăn là các thái độ này không hoạt động như các vị từ "bình thường". Ví dụ, giả sử chúng ta cố gắng khẳng định rằng Lois biết rằng Superman có thể bay:

$\text{Biết}(\text{Lois}, \text{CóThếBay}(\text{Superman}))$.

Một vấn đề nhỏ ở đây là chúng ta thường nghĩ về $\text{CóThếBay}(\text{Superman})$ như một câu, nhưng ở đây nó xuất hiện như một hạng. Vấn đề này có thể được khắc phục bằng cách tái hiện $\text{CóThếBay}(\text{Superman})$; biến nó thành một fluent. Một vấn đề nghiêm trọng hơn là, nếu đúng là Superman là Clark Kent, thì chúng ta phải kết luận rằng Lois biết Clark có thể bay, điều này là sai vì (trong hầu hết các phiên bản của câu chuyện) Lois không biết Clark là Superman.

$(\text{Superman} = \text{Clark}) \wedge \text{Biết}(\text{Lois}, \text{CóThếBay}(\text{Superman})) \models \text{Biết}(\text{Lois}, \text{CóThếBay}(\text{Clark}))$

Đây là hệ quả của việc lập luận về sự bằng nhau được tích hợp sẵn trong logic. Thông thường, đó là một điều tốt; nếu tác nhân của chúng ta biết rằng $2 + 2 = 4$ và $4 < 5$, thì chúng ta muốn tác nhân biết rằng $2 + 2 < 5$. Tính chất này được gọi là **tính minh bạch tham chiếu**—không quan trọng logic sử dụng hạng nào để tham chiếu đến một đối tượng, điều quan trọng là đối tượng mà hạng đó đặt tên. Nhưng đối với các thái độ mệnh đề như tin và biết, chúng ta muốn có tính **mờ đục tham chiếu**—các hạng được sử dụng có quan trọng, vì không phải tất cả các tác nhân đều biết các hạng nào cùng tham chiếu.

Chúng ta có thể khắc phục điều này bằng cách tái hiện nhiều hơn: chúng ta có thể có một đối tượng để đại diện cho Clark/Superman, một đối tượng khác để đại diện cho người mà Lois biết là Clark, và một đối tượng khác nữa cho người Lois biết là Superman. Tuy nhiên, sự gia tăng các đối tượng này khiến các câu chúng ta muốn viết nhanh chóng trở nên dài dòng và vụng về.

Logic hình thái được thiết kế để giải quyết vấn đề này. Logic thông thường chỉ quan tâm đến một hình thái duy nhất, hình thái của sự thật, cho phép chúng ta biểu diễn "P đúng" hoặc "P sai". Logic hình thái bao gồm các **toán tử hình thái** đặc biệt nhận các câu (thay vì các hạng) làm đối số. Ví dụ, "A biết P" được biểu diễn bằng ký hiệu $\mathbf{K}_A P$, trong đó $\mathbf{K}$ là toán tử hình thái cho tri thức. Nó

nhận hai đối số, một tác nhân (được viết dưới dạng chỉ số) và một câu. Cú pháp của logic hình thái giống với logic vị từ cấp một, ngoại trừ việc các câu cũng có thể được hình thành bằng các toán tử hình thái.

Ngữ nghĩa của logic hình thái phức tạp hơn. Trong logic vị từ cấp một, một **mô hình** chứa một tập hợp các đối tượng và một phép diễn giải ánh xạ mỗi tên đến đối tượng, quan hệ hoặc hàm thích hợp. Trong logic hình thái, chúng ta muốn có thể xem xét cả khả năng bí mật danh tính của Superman là Clark và khả năng nó không phải.

Do đó, chúng ta sẽ cần một mô hình phức tạp hơn, một mô hình bao gồm một tập hợp các **thế giới khả dĩ** thay vì chỉ một thế giới đúng duy nhất. Các thế giới được kết nối trong một đồ thị bằng các **quan hệ khả truy cập**, một quan hệ cho mỗi toán tử hình thái. Chúng ta nói rằng thế giới w_1 có thể truy cập từ thế giới w_0 đối với toán tử hình thái K_A nếu mọi thứ trong w_1 phù hợp với những gì A biết trong w_0 . Ví dụ, trong thế giới thực, Bucharest là thủ đô của Romania, nhưng đối với một tác nhân không biết điều đó, một thế giới mà thủ đô của Romania là, chẳng hạn, Sofia, là có thể truy cập. Hy vọng rằng một thế giới nơi $2 + 2 = 5$ sẽ không thể truy cập đối với bất kỳ tác nhân nào.

Nói chung, một nguyên tử tri thức K_AP đúng trong thế giới w nếu và chỉ nếu P đúng trong mọi thế giới có thể truy cập từ w . Chân lý của các câu phức tạp hơn được suy ra bằng cách áp dụng đệ quy quy tắc này và các quy tắc thông thường của logic vị từ cấp một. Điều đó có nghĩa là logic hình thái có thể được sử dụng để lập luận về các câu tri thức lồng nhau: những gì một tác nhân biết về tri thức của một tác nhân khác. Ví dụ, chúng ta có thể nói rằng mặc dù Lois không biết liệu bí mật danh tính của Superman có phải là Clark Kent hay không, cô ấy biết rằng Clark biết:

$$K_{Lois}[K_{Clark}DanhTinh(Superman, Clark) \vee K_{Clark}\neg DanhTinh(Superman, Clark)]$$

Logic hình thái giải quyết một số vấn đề phức tạp với sự tương tác giữa các lượng từ và tri thức. Câu tiếng Anh "Bond biết rằng ai đó là gián điệp" có thể hiểu theo hai cách. Cách hiểu thứ nhất là có một ai đó cụ thể mà Bond biết là gián điệp; chúng ta có thể viết điều này như sau:

$$\exists x K_{Bond} GiánĐiệp(x),$$

trong logic hình thái có nghĩa là tồn tại một x mà, trong tất cả các thế giới có thể truy cập, Bond biết là gián điệp. Cách hiểu thứ hai là Bond chỉ biết rằng có ít nhất một gián điệp:

$$K_{Bond} \exists x GiánĐiệp(x).$$

Cách diễn giải trong logic hình thái là trong mỗi thế giới có thể truy cập, tồn tại một x là gián điệp, nhưng x đó không cần phải giống nhau trong mỗi thế giới.

Bây giờ chúng ta đã có một toán tử hình thái cho tri thức, chúng ta có thể viết các tiên đề cho nó. Đầu tiên, chúng ta có thể nói rằng các tác nhân có thể rút ra các kết luận; nếu một tác nhân biết P và biết rằng P kéo theo Q , thì tác nhân đó biết Q :

$$(K_a P \wedge K_a (P \Rightarrow Q)) \Rightarrow K_a Q.$$

Từ đây (và một số quy tắc khác về các đồng nhất thức logic), chúng ta có thể thiết lập rằng $K_A(P \vee \neg P)$ là một hằng đúng; mọi tác nhân đều biết mọi mệnh đề P hoặc đúng hoặc sai. Mặt khác,

$(K_A P) \vee (K_A \neg P)$ không phải là một hằng đúng; nói chung, sẽ có rất nhiều mệnh đề mà một tác nhân không biết là đúng và cũng không biết là sai.

Người ta nói (bắt nguồn từ Plato) rằng tri thức là niềm tin đúng đắn có căn cứ. Nghĩa là, nếu một điều gì đó là đúng, nếu bạn tin vào nó, và nếu bạn có một lý do vững chắc không thể bác bỏ, thì bạn biết điều đó. Điều đó có nghĩa là nếu bạn biết một điều gì đó, nó phải là sự thật, và chúng ta có tiên đề sau:

$$K_a P \Rightarrow P.$$

Hơn nữa, các tác nhân logic (nhưng không phải tất cả mọi người) có thể tự suy ngẫm về tri thức của chính mình. Nếu họ biết một điều gì đó, thì họ biết rằng họ biết điều đó:

$$K_a P \Rightarrow K_a (K_a P).$$

Chúng ta có thể định nghĩa các tiên đề tương tự cho niềm tin (thường được ký hiệu bằng B) và các hình thái khác. Tuy nhiên, một vấn đề với phương pháp logic hình thái là nó giả định sự toàn tri logic ở các tác nhân. Nghĩa là, nếu một tác nhân biết một tập hợp các tiên đề, thì nó biết tất cả các hệ quả của các tiên đề đó. Điều này khá mong manh ngay cả đối với khái niệm trừu tượng về tri thức, nhưng có vẻ còn tệ hơn đối với niềm tin, vì niềm tin có nhiều hàm ý hơn về những thứ được biểu diễn vật lý trong tác nhân, chứ không chỉ là những thứ có thể suy ra được.

Đã có những nỗ lực để định nghĩa một dạng lý trí hạn chế cho các tác nhân—để nói rằng các tác nhân chỉ tin vào những khẳng định có thể được suy ra với không quá k bước lập luận, hoặc không quá s giây tính toán. Những nỗ lực này nhìn chung không đạt yêu cầu.

10.4.1 Các logic hình thái khác

Nhiều logic hình thái đã được đề xuất, cho các hình thái khác ngoài tri thức. Một đề xuất là thêm các toán tử hình thái cho khả năng và sự cần thiết: có thể đúng là một trong những tác giả của cuốn sách này đang ngồi ngay lúc này, và chắc chắn đúng là $2 + 2 = 4$.

Như đã đề cập trong Mục 8.1.2, một số nhà logic học ủng hộ các hình thái liên quan đến thời gian. Trong logic thời gian tuyến tính, chúng ta thêm các toán tử hình thái sau:

- **X P**: "P sẽ đúng trong bước thời gian tiếp theo"
- **F P**: "P cuối cùng sẽ đúng trong một bước thời gian tương lai nào đó"
- **G P**: "P luôn (toàn cục) đúng"
- **P U Q**: "P vẫn đúng cho đến khi Q xảy ra"

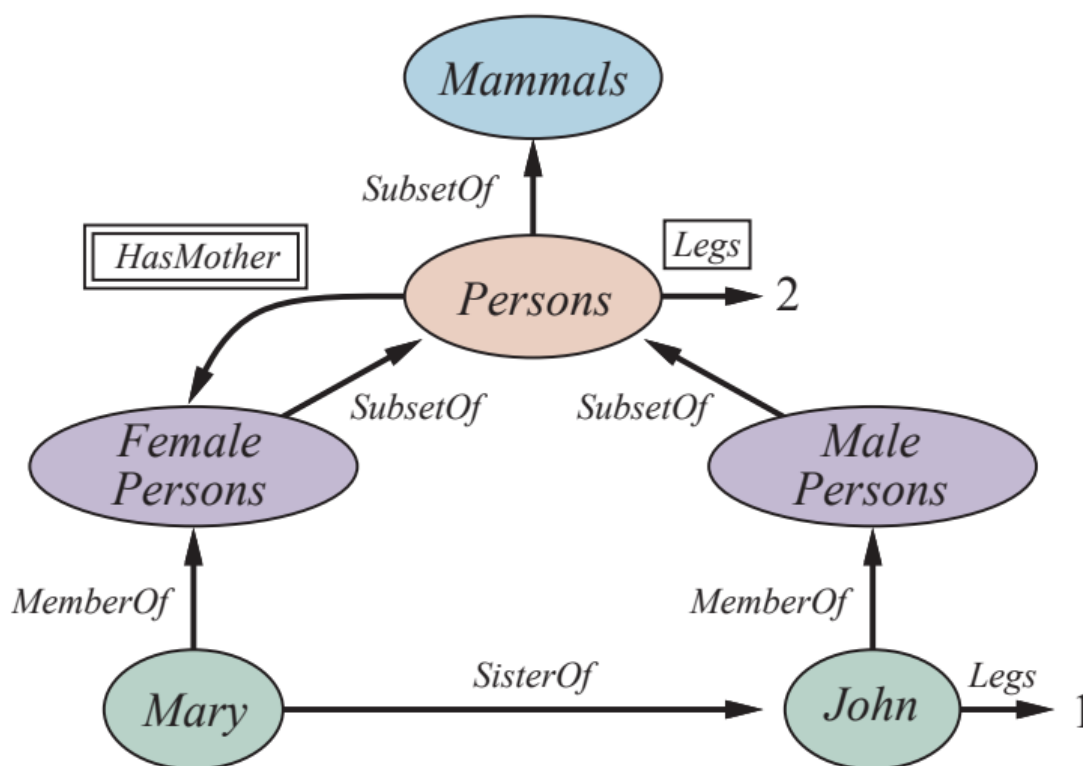
Đôi khi có các toán tử bổ sung có thể được suy ra từ những toán tử này. Việc thêm các toán tử hình thái này làm cho bản thân logic phức tạp hơn (và do đó khiến thuật toán suy luận logic khó tìm ra chứng minh hơn). Nhưng các toán tử này cũng cho phép chúng ta phát biểu một số sự thật một cách ngắn gọn hơn (giúp tăng tốc suy luận logic). Việc lựa chọn logic nào để sử dụng tương tự như việc lựa chọn ngôn ngữ lập trình nào: hãy chọn một ngôn ngữ phù hợp với nhiệm vụ của bạn, quen thuộc với bạn và những người khác sẽ chia sẻ công việc của bạn, và đủ hiệu quả cho mục đích của bạn.

10.5 Hệ thống lập luận cho các danh mục

Các danh mục là những khối xây dựng chính của các hệ thống biểu diễn tri thức quy mô lớn. Phần này mô tả các hệ thống được thiết kế đặc biệt để tổ chức và lập luận với các danh mục. Có hai họ hệ thống liên quan chặt chẽ với nhau: mạng ngữ nghĩa cung cấp các công cụ trực quan hóa để hình dung cơ sở tri thức và các thuật toán hiệu quả để suy ra các thuộc tính của một đối tượng dựa trên việc nó thuộc về danh mục nào; và logic mô tả cung cấp một ngôn ngữ hình thức để xây dựng và kết hợp các định nghĩa danh mục cùng với các thuật toán hiệu quả để quyết định mối quan hệ tập hợp con và tập hợp cha giữa các danh mục.

10.5.1 Mạng ngữ nghĩa

Năm 1909, Charles S. Peirce đề xuất một ký hiệu đồ họa gồm các nút và cạnh gọi là đồ thị hiện hữu, mà ông gọi là "logic của tương lai." Từ đó, một cuộc tranh luận kéo dài đã nổ ra giữa những người ủng hộ "logic" và những người ủng hộ "mạng ngữ nghĩa." Thật không may, cuộc tranh luận này đã che khuất thực tế rằng mạng ngữ nghĩa chính là một dạng của logic. Ký hiệu mà mạng ngữ nghĩa cung cấp cho một số loại câu nhất định thường tiện lợi hơn, nhưng nếu gạt bỏ các vấn đề về "giao diện người dùng," các khái niệm cơ bản—đối tượng, quan hệ, lượng hóa, v.v.—vẫn giống nhau.



Hình 10.4 Mạng ngữ nghĩa với bốn đối tượng (John, Mary, 1 và 2) và bốn phạm trù. Các quan hệ được biểu diễn bằng các liên kết có nhãn.

Có nhiều biến thể của mạng ngữ nghĩa, nhưng tất cả đều có khả năng biểu diễn các đối tượng riêng lẻ, các danh mục đối tượng và các quan hệ giữa các đối tượng. Một ký hiệu đồ họa điển hình hiển

thị tên đối tượng hoặc danh mục trong hình oval hoặc hộp, và kết nối chúng bằng các liên kết có nhãn. Ví dụ, Hình 10.4 có liên kết MemberOf giữa Mary và FemalePersons, tương ứng với khẳng định logic $Mary \in FemalePersons$; tương tự, liên kết SisterOf giữa Mary và John tương ứng với khẳng định $SisterOf(Mary, John)$. Chúng ta có thể kết nối các danh mục bằng các liên kết SubsetOf, v.v. Việc vẽ các bong bóng và mũi tên rất thú vị đến mức người ta có thể bị cuốn theo. Ví dụ, chúng ta biết rằng con người có mẹ là phụ nữ, vậy chúng ta có thể vẽ liên kết HasMother từ Persons đến FemalePersons không? Câu trả lời là không, vì HasMother là một quan hệ giữa một người và mẹ của họ, và các danh mục không có mẹ.

Vì lý do này, chúng tôi đã sử dụng một ký hiệu đặc biệt—liên kết hộp đôi—trong Hình 10.4. Liên kết này khẳng định rằng:

$$\forall x \, x \in Persons \Rightarrow [\forall y \, HasMother(x, y) \Rightarrow y \in FemalePersons].$$

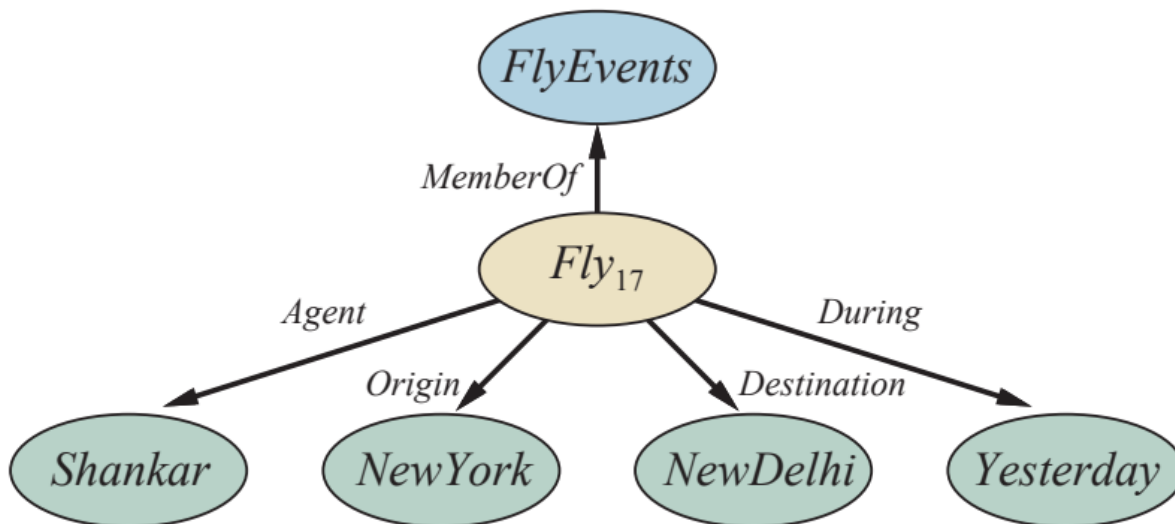
Chúng ta cũng có thể muốn khẳng định rằng con người có hai chân—tức là:

$$\forall x \, x \in Persons \Rightarrow Legs(x, 2).$$

Như trước đây, chúng ta cần cẩn thận để không khẳng định rằng một danh mục có chân; liên kết hộp đơn trong Hình 10.4 được sử dụng để khẳng định thuộc tính của mọi thành viên trong một danh mục.

Ký hiệu mạng ngữ nghĩa giúp việc thực hiện lập luận kế thừa (như đã giới thiệu trong Mục 10.2) trở nên thuận tiện. Ví dụ, do là một con người, Mary kế thừa thuộc tính có hai chân. Do đó, để tìm hiểu xem Mary có bao nhiêu chân, thuật toán kế thừa sẽ theo liên kết MemberOf từ Mary đến danh mục mà cô ấy thuộc về, sau đó theo các liên kết SubsetOf lên hệ thống phân cấp cho đến khi tìm thấy một danh mục có liên kết Legs được đóng hộp—trong trường hợp này là danh mục Persons. Sự đơn giản và hiệu quả của cơ chế suy luận này, so với việc chứng minh định lý bán khả quyết trong logic, là một trong những điểm hấp dẫn chính của mạng ngữ nghĩa.

Lập luận kế thừa trở nên phức tạp khi một đối tượng có thể thuộc về nhiều danh mục hoặc khi một danh mục có thể là tập hợp con của nhiều danh mục khác; điều này được gọi là kế thừa đa cấp. Trong những trường hợp như vậy, thuật toán kế thừa có thể tìm thấy hai hoặc nhiều giá trị xung đột khi trả lời truy vấn. Vì lý do này, kế thừa đa cấp bị cấm trong một số ngôn ngữ lập trình hướng đối tượng (OOP) như Java, nơi sử dụng kế thừa trong hệ thống phân cấp lớp. Thông thường, kế thừa đa cấp được cho phép trong mạng ngữ nghĩa, nhưng chúng tôi sẽ thảo luận chi tiết về vấn đề này trong Mục 10.6.



Hình 10.5 Một phần của mạng ngữ nghĩa thể hiện biểu diễn khẳng định logic Fly(Shankar, NewYork, NewDelhi, Yesterday).

Người đọc có thể nhận thấy một nhược điểm rõ ràng của ký hiệu mạng ngữ nghĩa so với logic bậc nhất: thực tế là các liên kết giữa các bong bóng chỉ biểu diễn các quan hệ nhị phân. Ví dụ, câu Fly(Shankar, NewYork, NewDelhi, Yesterday) không thể được khẳng định trực tiếp trong mạng ngữ nghĩa. Tuy nhiên, chúng ta có thể đạt được hiệu quả của các khẳng định n-ngôi bằng cách vật chất hóa bản thân mệnh đề như một sự kiện thuộc về một danh mục sự kiện phù hợp. Hình 10.5 cho thấy cấu trúc mạng ngữ nghĩa cho sự kiện cụ thể này. Lưu ý rằng việc giới hạn ở các quan hệ nhị phân buộc phải tạo ra một bản thể học phong phú về các khái niệm được vật chất hóa.

Việc vật chất hóa các mệnh đề giúp biểu diễn mọi câu nguyên tử không hàm trong logic bậc nhất bằng ký hiệu mạng ngữ nghĩa. Một số loại câu được lượng hóa phổ quát có thể được khẳng định bằng cách sử dụng các liên kết ngược và các mũi tên hộp đơn hoặc hộp đôi áp dụng cho các danh mục, nhưng điều đó vẫn còn cách xa so với logic bậc nhất đầy đủ. Phủ định, phép tuyển, các hàm lồng nhau và lượng hóa tồn tại đều bị thiếu. Giờ đây, có thể mở rộng ký hiệu để làm cho nó tương đương với logic bậc nhất—như trong đồ thị hiện hữu của Peirce—nhưng làm như vậy sẽ phủ nhận một trong những lợi thế chính của mạng ngữ nghĩa, đó là sự đơn giản và minh bạch của các quá trình suy luận. Các nhà thiết kế có thể xây dựng một mạng lớn và vẫn có ý tưởng tốt về các truy vấn nào sẽ hiệu quả, bởi vì (a) dễ dàng hình dung các bước mà quy trình suy luận sẽ trải qua và (b) trong một số trường hợp, ngôn ngữ truy vấn quá đơn giản đến mức không thể đặt ra các truy vấn phức tạp.

Trong các trường hợp sức mạnh biểu đạt bị hạn chế, nhiều hệ thống mạng ngữ nghĩa cung cấp khả năng đánh kèm thủ tục để lấp đầy khoảng trống. Đánh kèm thủ tục là một kỹ thuật trong đó một truy vấn về (hoặc đôi khi là một khẳng định của) một quan hệ nhất định dẫn đến việc gọi một thủ tục đặc biệt được thiết kế cho quan hệ đó thay vì một thuật toán suy luận chung.

Một trong những khía cạnh quan trọng nhất của mạng ngữ nghĩa là khả năng biểu diễn các giá trị mặc định cho các danh mục. Khi kiểm tra kỹ Hình 10.4, người ta nhận thấy rằng John có một chân, mặc dù thực tế anh ta là một con người và tất cả con người đều có hai chân. Trong một cơ sở tri

thức logic nghiêm ngặt, đây sẽ là một mâu thuẫn, nhưng trong mạng ngữ nghĩa, khẳng định rằng tất cả con người có hai chân chỉ có trạng thái mặc định; nghĩa là, một người được giả định có hai chân trừ khi điều này bị mâu thuẫn bởi thông tin cụ thể hơn. Ngữ nghĩa mặc định được thực thi một cách tự nhiên bởi thuật toán kế thừa, bởi vì nó theo các liên kết đi lên từ chính đối tượng (John trong trường hợp này) và dừng lại ngay khi tìm thấy một giá trị. Chúng ta nói rằng giá trị mặc định bị ghi đè bởi giá trị cụ thể hơn. Lưu ý rằng chúng ta cũng có thể ghi đè số chân mặc định bằng cách tạo một danh mục `OneLeggedPersons`, một tập hợp con của `Persons` mà John là thành viên.

Chúng ta có thể giữ lại ngữ nghĩa logic nghiêm ngặt cho mạng nếu chúng ta nói rằng khẳng định `Legs` cho `Persons` bao gồm một ngoại lệ cho John:

$$\forall x \, x \in \text{Persons} \wedge x \neq \text{John} \Rightarrow \text{Legs}(x, 2).$$

Đối với một mạng cố định, điều này là đủ về mặt ngữ nghĩa nhưng sẽ kém súc tích hơn nhiều so với chính ký hiệu mạng nếu có nhiều ngoại lệ. Tuy nhiên, đối với một mạng sẽ được cập nhật với nhiều khẳng định hơn, cách tiếp cận như vậy sẽ thất bại—chúng ta thực sự muốn nói rằng bất kỳ người nào chưa được biết đến với một chân cũng là ngoại lệ. Mục 10.6 sẽ đi sâu hơn vào vấn đề này và vào lập luận mặc định nói chung.

10.5.2 Logic mô tả

Cú pháp của logic bậc nhất được thiết kế để giúp dễ dàng nói về các đối tượng. Logic mô tả là các ký hiệu được thiết kế để giúp dễ dàng mô tả các định nghĩa và thuộc tính của các danh mục. Các hệ thống logic mô tả phát triển từ mạng ngữ nghĩa để đáp ứng áp lực chính thức hóa ý nghĩa của các mạng trong khi vẫn duy trì trọng tâm vào cấu trúc phân loại như một nguyên tắc tổ chức.

Các nhiệm vụ suy luận chính cho logic mô tả là bao hàm (kiểm tra xem một danh mục có phải là tập hợp con của danh mục khác bằng cách so sánh các định nghĩa của chúng) và phân loại (kiểm tra xem một đối tượng có thuộc về một danh mục hay không). Một số hệ thống cũng bao gồm tính nhất quán của định nghĩa danh mục—liệu các tiêu chí thành viên có thể thỏa mãn logic hay không.

<i>Concept</i>	→	Thing <i>ConceptName</i>
		And (<i>Concept</i> ,...)
		All (<i>RoleName</i> , <i>Concept</i>)
		AtLeast (<i>Integer</i> , <i>RoleName</i>)
		AtMost (<i>Integer</i> , <i>RoleName</i>)
		Fills (<i>RoleName</i> , <i>IndividualName</i> ,...)
		SameAs (<i>Path</i> , <i>Path</i>)
		OneOf (<i>IndividualName</i> ,...)
<i>Path</i>	→	[<i>RoleName</i> ,...]
<i>ConceptName</i>	→	<i>Adult</i> <i>Female</i> <i>Male</i> ...
<i>RoleName</i>	→	<i>Spouse</i> <i>Daughter</i> <i>Son</i> ...

Hình 10.6 Cú pháp mô tả trong một tập hợp con của ngôn ngữ CLASSIC.

Ngôn ngữ CLASSIC (Borgida et al., 1989) là một logic mô tả điển hình. Cú pháp của các mô tả trong CLASSIC được hiển thị trong Hình 10.6. Ví dụ, để nói rằng những người độc thân là nam giới trưởng thành chưa kết hôn, chúng ta sẽ viết:

```
Bachelor = And(Unmarried, Adult, Male).
```

Tương đương trong logic bậc nhất sẽ là:

```
Bachelor(x)  $\Leftrightarrow$  Unmarried(x)  $\wedge$  Adult(x)  $\wedge$  Male(x).
```

Lưu ý rằng logic mô tả có một đại số các phép toán trên vị từ, điều mà chúng ta không thể làm trong logic bậc nhất. Bất kỳ mô tả nào trong CLASSIC đều có thể được dịch sang một câu logic bậc nhất tương đương, nhưng một số mô tả trong CLASSIC đơn giản hơn. Ví dụ, để mô tả tập hợp những người đàn ông có ít nhất ba con trai đều thất nghiệp và kết hôn với bác sĩ, và nhiều nhất hai con gái đều là giáo sư trong các khoa vật lý hoặc toán, chúng ta sẽ sử dụng:

```
And(Man, AtLeast(3, Son), AtMost(2, Daughter),  
    All(Son, And(Unemployed, Married, All(Spouse, Doctor))),  
    All(Daughter, And(Professor, Fills(Department, Physics, Math)))).
```

Chúng tôi để bài tập này cho người đọc dịch sang logic bậc nhất.

Có lẽ khía cạnh quan trọng nhất của logic mô tả là sự nhấn mạnh vào khả năng xử lý của suy luận. Một trường hợp vấn đề được giải quyết bằng cách mô tả nó và sau đó hỏi xem nó có được bao hàm bởi một trong số các danh mục giải pháp khả thi hay không. Trong các hệ thống logic bậc nhất tiêu chuẩn, việc dự đoán thời gian giải pháp thường là không thể. Người dùng thường phải tự điều chỉnh biểu diễn để tránh các tập hợp câu dường như khiến hệ thống mất nhiều tuần để giải quyết. Ngược lại, logic mô tả tập trung vào việc đảm bảo rằng việc kiểm tra bao hàm có thể được giải quyết trong thời gian đa thức so với kích thước của các mô tả.

Điều này nghe có vẻ tuyệt vời về nguyên tắc, cho đến khi người ta nhận ra rằng nó chỉ có thể dẫn đến một trong hai hậu quả: hoặc các vấn đề khó không thể được phát biểu chút nào, hoặc chúng yêu cầu các mô tả có kích thước theo cấp số nhân! Tuy nhiên, các kết quả về khả năng xử lý giúp làm sáng tỏ những loại cấu trúc nào gây ra vấn đề và do đó giúp người dùng hiểu cách các biểu diễn khác nhau hoạt động. Ví dụ, logic mô tả thường thiếu phủ định và phép tuyển. Mỗi thứ này buộc các hệ thống logic bậc nhất phải trải qua một phân tích trường hợp có khả năng theo cấp số nhân để đảm bảo tính đầy đủ. CLASSIC chỉ cho phép một dạng phép tuyển hạn chế trong các cấu trúc Fills và OneOf, cho phép phép tuyển trên các cá thể được liệt kê rõ ràng nhưng không phải trên các mô tả. Với các mô tả tuyển tính, các định nghĩa lồng nhau có thể dễ dàng dẫn đến một số lượng theo cấp số nhân các tuyến đường thay thế mà một danh mục có thể bao hàm danh mục khác.

10.6 Lập luận với thông tin mặc định

Trong phần trước, chúng ta đã thấy một ví dụ đơn giản về một khẳng định có trạng thái mặc định: con người có hai chân. Mặc định này có thể bị ghi đè bởi thông tin cụ thể hơn, chẳng hạn như Long John Silver chỉ có một chân. Chúng ta đã thấy rằng cơ chế kế thừa trong mạng ngữ nghĩa triển khai việc ghi đè các mặc định một cách đơn giản và tự nhiên. Trong phần này, chúng ta sẽ nghiên cứu các mặc định một cách tổng quát hơn, với mục tiêu hiểu rõ ngữ nghĩa của chúng thay vì chỉ đưa ra một cơ chế thủ tục.

10.6.1 Giới hạn hóa và logic mặc định

Chúng ta đã thấy hai ví dụ về quá trình suy luận vi phạm tính chất đơn điệu của logic, vốn đã được chứng minh trong Chương 7. Trong chương này, chúng ta thấy rằng một thuộc tính được kế thừa bởi tất cả các thành viên của một danh mục trong mạng ngữ nghĩa có thể bị ghi đè bởi thông tin cụ thể hơn cho một danh mục con. Trong Mục 9.4.4, chúng ta thấy rằng dưới giả định thế giới đóng, nếu một mệnh đề α không được đề cập trong KB thì $KB \models \neg\alpha$, nhưng $KB \wedge \alpha \models \alpha$.

Suy ngầm đơn giản cho thấy rằng những thất bại này của tính đơn điệu rất phổ biến trong lập luận thông thường. Dường như con người thường "vội kết luận". Ví dụ, khi nhìn thấy một chiếc xe đậu trên đường, người ta thường sẵn sàng tin rằng nó có bốn bánh mặc dù chỉ nhìn thấy ba. Lý thuyết xác suất chắc chắn có thể đưa ra kết luận rằng bánh thứ tư tồn tại với xác suất cao; tuy nhiên, đối với hầu hết mọi người, khả năng chiếc xe không có bốn bánh sẽ không nảy ra trừ khi có thêm bằng chứng mới. Do đó, có vẻ như kết luận về bốn bánh được đưa ra theo mặc định, trong trường hợp không có lý do để nghi ngờ nó. Nếu có bằng chứng mới xuất hiện—ví dụ, nếu thấy chủ xe mang theo một bánh và nhận thấy chiếc xe được nâng lên—thì kết luận có thể bị rút lại. Loại lập luận này được cho là thể hiện tính không đơn điệu, bởi vì tập hợp các niềm tin không phát triển một cách đơn điệu theo thời gian khi có thêm bằng chứng mới. Các logic không đơn điệu đã được phát triển với các khái niệm sửa đổi về chân lý và hệ quả để nắm bắt hành vi như vậy. Chúng ta sẽ xem xét hai logic như vậy đã được nghiên cứu kỹ lưỡng: giới hạn hóa và logic mặc định.

Giới hạn hóa có thể được coi là một phiên bản mạnh mẽ và chính xác hơn của giả định thế giới đóng. Ý tưởng là chỉ định các vị từ cụ thể được giả định là "sai nhất có thể"—tức là sai cho mọi đối tượng trừ những đối tượng mà chúng được biết là đúng. Ví dụ, giả sử chúng ta muốn khẳng định quy tắc mặc định rằng chim biết bay. Chúng ta sẽ giới thiệu một vị từ, chẳng hạn như $\text{Abnormal}_1(x)$, và viết:

$$\text{Bird}(x) \wedge \neg\text{Abnormal}_1(x) \Rightarrow \text{Flies}(x).$$

Nếu chúng ta nói rằng Abnormal_1 cần được giới hạn hóa, một bộ suy luận giới hạn hóa có quyền giả định $\neg\text{Abnormal}_1(x)$ trừ khi $\text{Abnormal}_1(x)$ được biết là đúng. Điều này cho phép rút ra kết luận $\text{Flies}(\text{Tweety})$ từ tiền đề $\text{Bird}(\text{Tweety})$, nhưng kết luận này không còn đúng nếu $\text{Abnormal}_1(\text{Tweety})$ được khẳng định.

Giới hạn hóa có thể được xem như một ví dụ về logic ưu tiên mô hình. Trong các logic như vậy, một câu được suy ra (với trạng thái mặc định) nếu nó đúng trong tất cả các mô hình ưu tiên của KB, thay vì yêu cầu đúng trong tất cả các mô hình như trong logic cổ điển. Đối với giới hạn hóa, một mô hình được ưu tiên hơn mô hình khác nếu nó có ít đối tượng bất thường hơn. Hãy xem cách ý tưởng này hoạt động trong bối cảnh kế thừa đa cấp trong mạng ngữ nghĩa. Ví dụ tiêu chuẩn mà kế thừa đa cấp gây ra vấn đề được gọi là "kim cương Nixon". Nó xuất phát từ quan sát rằng Richard Nixon vừa là người Quaker (và do đó theo mặc định là người theo chủ nghĩa hòa bình) vừa là người Cộng hòa (và do đó theo mặc định không phải là người theo chủ nghĩa hòa bình). Chúng ta có thể viết điều này như sau:

$$\begin{aligned} &\text{Republican}(\text{Nixon}) \wedge \text{Quaker}(\text{Nixon}). \text{Republican}(x) \wedge \neg\text{Abnormal}_2(x) \\ &\Rightarrow \neg\text{Pacifist}(x). \text{Quaker}(x) \wedge \neg\text{Abnormal}_3(x) \Rightarrow \text{Pacifist}(x). \end{aligned}$$

Nếu chúng ta giới hạn hóa Abnormal_2 và Abnormal_3 , sẽ có hai mô hình ưu tiên: một trong đó $\text{Abnormal}_2(\text{Nixon})$ và $\text{Pacifist}(\text{Nixon})$ là đúng, và một trong đó $\text{Abnormal}_3(\text{Nixon})$ và

$\neg \text{Pacifist}(\text{Nixon})$ là đúng. Do đó, bộ suy luận giới hạn hóa vẫn giữ thái độ trung lập thích hợp về việc liệu Nixon có phải là người theo chủ nghĩa hòa bình hay không. Nếu chúng ta muốn, ngoài ra, khẳng định rằng niềm tin tôn giáo được ưu tiên hơn niềm tin chính trị, chúng ta có thể sử dụng một hình thức gọi là giới hạn hóa ưu tiên để ưu tiên các mô hình trong đó Abnormal_3 được tối thiểu hóa.

Logic mặc định là một hình thức trong đó các quy tắc mặc định có thể được viết để tạo ra các kết luận tùy ý, không đơn điệu. Một quy tắc mặc định trông như thế này:

$$\text{Bird}(x):\text{Flies}(x)/\text{Flies}(x).$$

Quy tắc này có nghĩa là nếu $\text{Bird}(x)$ đúng, và nếu $\text{Flies}(x)$ phù hợp với cơ sở tri thức, thì $\text{Flies}(x)$ có thể được kết luận theo mặc định. Nói chung, một quy tắc mặc định có dạng:

$$P:J_1, \dots, J_n/C$$

trong đó P được gọi là điều kiện tiên quyết, C là kết luận, và J_i là các biện minh—nếu bất kỳ biện minh nào trong số chúng có thể được chứng minh là sai, thì kết luận không thể được rút ra. Bất kỳ biến nào xuất hiện trong J_i hoặc C cũng phải xuất hiện trong P . Ví dụ kim cương Nixon có thể được biểu diễn trong logic mặc định với một sự kiện và hai quy tắc mặc định:

$$\begin{aligned} & \text{Republican}(\text{Nixon}) \\ & \wedge \text{Quaker}(\text{Nixon}).\text{Republican}(x): \neg \text{Pacifist}(x) \\ & / \neg \text{Pacifist}(x).\text{Quaker}(x):\text{Pacifist}(x)/\text{Pacifist}(x). \end{aligned}$$

Để giải thích ý nghĩa của các quy tắc mặc định, chúng ta định nghĩa khái niệm mở rộng của một lý thuyết mặc định là một tập hợp tối đa các hệ quả của lý thuyết đó. Nghĩa là, một mở rộng S bao gồm các sự kiện đã biết ban đầu và một tập hợp các kết luận từ các quy tắc mặc định, sao cho không có kết luận bổ sung nào có thể được rút ra từ S , và các biện minh của mọi kết luận mặc định trong S đều phù hợp với S . Giống như trường hợp các mô hình ưu tiên trong giới hạn hóa, chúng ta có hai mở rộng khả dĩ cho kim cương Nixon: một trong đó Nixon là người theo chủ nghĩa hòa bình và một trong đó ông không phải là. Các sơ đồ ưu tiên tồn tại trong đó một số quy tắc mặc định có thể được ưu tiên hơn những quy tắc khác, cho phép giải quyết một số sự mơ hồ.

Kể từ năm 1980, khi các logic không đơn điệu lần đầu tiên được đề xuất, rất nhiều tiến bộ đã đạt được trong việc hiểu các thuộc tính toán học của chúng. Tuy nhiên, vẫn còn những câu hỏi chưa được giải quyết. Ví dụ, nếu "Xe hơi có bốn bánh" là sai, thì việc có nó trong cơ sở tri thức có ý nghĩa gì? Một tập hợp các quy tắc mặc định tốt để có là gì? Nếu chúng ta không thể quyết định, đối với từng quy tắc riêng biệt, liệu nó có thuộc về cơ sở tri thức của chúng ta hay không, thì chúng ta sẽ gặp phải vấn đề nghiêm trọng về tính không mô-đun. Cuối cùng, làm thế nào các niềm tin có trạng thái mặc định có thể được sử dụng để đưa ra quyết định? Đây có lẽ là vấn đề khó khăn nhất đối với lập luận mặc định.

Các quyết định thường liên quan đến sự đánh đổi, và do đó người ta cần so sánh sức mạnh của niềm tin vào kết quả của các hành động khác nhau, cũng như chi phí của việc đưa ra quyết định sai. Trong các trường hợp cùng loại quyết định được đưa ra lặp đi lặp lại, có thể giải thích các quy tắc mặc định như các phát biểu xác suất ngưỡng. Ví dụ, quy tắc mặc định "Phanh của tôi luôn ổn" thực sự có nghĩa là "Xác suất phanh của tôi ổn, với điều kiện không có thông tin khác, đủ cao để quyết định tối ưu là tôi lái xe mà không cần kiểm tra chúng." Khi bối cảnh quyết định thay đổi—

ví dụ, khi lái một chiếc xe tải nặng xuống một con đường dốc—quy tắc mặc định đột nhiên trở nên không phù hợp, ngay cả khi không có bằng chứng mới nào về phanh bị lỗi. Những cân nhắc này đã khiến các nhà nghiên cứu xem xét cách những lập luận mặc định trong lý thuyết xác suất hoặc lý thuyết tiện ích.

10.6.2 Hệ thống duy trì chân lý

Chúng ta đã thấy rằng nhiều suy luận được rút ra bởi một hệ thống biểu diễn tri thức sẽ chỉ có trạng thái mặc định, thay vì chắc chắn tuyệt đối. Không thể tránh khỏi, một số sự kiện được suy luận này sẽ hóa ra là sai và sẽ phải được rút lại khi có thông tin mới. Quá trình này được gọi là sửa đổi niềm tin. Giả sử rằng một cơ sở tri thức KB chứa một câu P —có thể là một kết luận mặc định được ghi lại bởi một thuật toán suy luận tiến, hoặc có thể chỉ là một khẳng định sai—và chúng ta muốn thực hiện $\text{TELL}(\text{KB}, \neg P)$. Để tránh tạo ra mâu thuẫn, trước tiên chúng ta phải thực hiện $\text{RETRACT}(\text{KB}, P)$. Nghe có vẻ đủ đơn giản. Tuy nhiên, vấn đề phát sinh nếu bất kỳ câu nào khác được suy luận từ P và được khẳng định trong KB. Ví dụ, hàm ý $P \Rightarrow Q$ có thể đã được sử dụng để thêm Q . "Giải pháp" hiển nhiên—rút lại tất cả các câu được suy luận từ P —thất bại vì những câu như vậy có thể có các biện minh khác ngoài P . Ví dụ, nếu R và $R \Rightarrow Q$ cũng có trong KB, thì Q không cần phải bị xóa. Các hệ thống duy trì chân lý, hoặc TMS, được thiết kế để xử lý chính xác những phức tạp như vậy.

Một cách tiếp cận đơn giản để duy trì chân lý là theo dõi thứ tự các câu được thêm vào cơ sở tri thức bằng cách đánh số chúng từ P_1 đến P_n . Khi lệnh $\text{RETRACT}(\text{KB}, P_i)$ được gọi, hệ thống sẽ quay trở lại trạng thái ngay trước khi P_i được thêm vào, do đó loại bỏ cả P_i và bất kỳ suy luận nào được rút ra từ P_i . Các câu từ P_{i+1} đến P_n sau đó có thể được thêm lại. Điều này rất đơn giản và đảm bảo rằng cơ sở tri thức sẽ nhất quán, nhưng việc rút lại P_i yêu cầu rút lại và khẳng định lại $n - i$ câu cũng như hoàn tác và làm lại tất cả các suy luận được rút ra từ những câu đó. Đối với các hệ thống mà nhiều sự kiện đang được thêm vào—chẳng hạn như các cơ sở dữ liệu thương mại lớn—điều này là không thực tế.

Một cách tiếp cận hiệu quả hơn là hệ thống duy trì chân lý dựa trên biện minh, hoặc JTMS. Trong JTMS, mỗi câu trong cơ sở tri thức được chú thích bằng một biện minh bao gồm tập hợp các câu mà từ đó nó được suy luận. Ví dụ, nếu cơ sở tri thức đã chứa $P \Rightarrow Q$, thì $\text{TELL}(P)$ sẽ khiến Q được thêm vào với biện minh $\{P, P \Rightarrow Q\}$. Nói chung, một câu có thể có bất kỳ số lượng biện minh nào. Các biện minh làm cho việc rút lại hiệu quả. Với lệnh $\text{RETRACT}(P)$, JTMS sẽ xóa chính xác những câu mà P là thành viên của mọi biện minh. Vì vậy, nếu một câu Q có biện minh duy nhất là $\{P, P \Rightarrow Q\}$, nó sẽ bị xóa; nếu nó có thêm biện minh $\{P, P \vee R \Rightarrow Q\}$, nó vẫn sẽ bị xóa; nhưng nếu nó cũng có biện minh $\{R, P \vee R \Rightarrow Q\}$, thì nó sẽ được giữ lại. Bằng cách này, thời gian cần thiết để rút lại P chỉ phụ thuộc vào số lượng câu được suy luận từ P chứ không phải vào số lượng câu được thêm sau P .

JTMS giả định rằng các câu đã được xem xét một lần có thể sẽ được xem xét lại, vì vậy thay vì xóa hoàn toàn một câu khỏi cơ sở tri thức khi nó mất tất cả các biện minh, chúng ta chỉ đánh dấu câu đó là không còn trong cơ sở tri thức. Nếu một khẳng định sau đó khôi phục một trong các biện minh, thì chúng ta đánh dấu câu đó là trở lại. Bằng cách này, JTMS giữ lại tất cả các chuỗi suy luận mà nó sử dụng và không cần phải suy luận lại các câu khi một biện minh trở nên hợp lệ trở lại.

Ngoài việc xử lý việc rút lại thông tin sai, TMS có thể được sử dụng để tăng tốc độ phân tích nhiều tình huống giả định. Ví dụ, giả sử Ủy ban Olympic Romania đang chọn địa điểm cho các nội dung bơi lội, điền kinh và cưỡi ngựa tại Thế vận hội năm 2048 được tổ chức tại Romania. Ví dụ, giả thuyết đầu tiên là Site(Swimming,Pitesti), Site(Athletics,Bucharest), và Site(Equestrian,Arad). Sau đó, cần phải thực hiện rất nhiều lập luận để đưa ra các hệ quả hậu cần và do đó là tính khả thi của lựa chọn này. Nếu chúng ta muốn xem xét Site(Athletics,Sibiu) thay thế, TMS tránh được việc phải bắt đầu lại từ đầu. Thay vào đó, chúng ta chỉ cần rút lại Site(Athletics,Bucharest) và khẳng định Site(Athletics,Sibiu), và TMS sẽ đảm nhiệm việc sửa đổi cần thiết. Các chuỗi suy luận được tạo ra từ việc lựa chọn Bucharest có thể được tái sử dụng với Sibiu, với điều kiện các kết luận là giống nhau.

Một hệ thống duy trì chân lý dựa trên giả định, hoặc ATMS, làm cho việc chuyển đổi ngữ cảnh giữa các thế giới giả định đặc biệt hiệu quả. Trong JTMS, việc duy trì các biện minh cho phép bạn di chuyển nhanh chóng từ trạng thái này sang trạng thái khác bằng cách thực hiện một vài lần rút lại và khẳng định, nhưng tại bất kỳ thời điểm nào chỉ có một trạng thái được biểu diễn. ATMS biểu diễn tất cả các trạng thái đã từng được xem xét cùng một lúc. Trong khi JTMS chỉ gắn nhãn mỗi câu là trong hay ngoài, ATMS theo dõi, đối với mỗi câu, những giả định nào sẽ khiến câu đó trở thành đúng. Nói cách khác, mỗi câu có một nhãn bao gồm một tập hợp các tập hợp giả định. Câu đó đúng trong những trường hợp mà tất cả các giả định trong một trong các tập hợp giả định là đúng.

Các hệ thống duy trì chân lý cũng cung cấp một cơ chế để tạo ra các giải thích. Về mặt kỹ thuật, một giải thích cho một câu P là một tập hợp các câu E sao cho E kéo theo P . Nếu các câu trong E đã được biết là đúng, thì E chỉ đơn giản cung cấp một cơ sở đủ để chứng minh rằng P phải đúng. Nhưng các giải thích cũng có thể bao gồm các giả định—các câu không được biết là đúng, nhưng sẽ đủ để chứng minh P nếu chúng đúng. Ví dụ, nếu xe của bạn không nổ máy, bạn có lẽ không có đủ thông tin để chứng minh dứt khoát lý do của vấn đề. Nhưng một giải thích hợp lý có thể bao gồm giả định rằng pin đã hết. Điều này, kết hợp với kiến thức về cách xe hoạt động, giải thích cho hành vi không hoạt động được quan sát. Trong hầu hết các trường hợp, chúng ta sẽ ưu tiên một giải thích E tối giản, nghĩa là không có tập hợp con thực sự nào của E cũng là một giải thích. Một ATMS có thể tạo ra các giải thích cho vấn đề "xe không nổ máy" bằng cách đưa ra các giả định (chẳng hạn như "không có xăng trong xe" hoặc "pin chết") theo bất kỳ thứ tự nào chúng ta thích, ngay cả khi một số giả định mâu thuẫn. Sau đó, chúng ta nhìn vào nhãn của câu "xe không nổ máy" để đọc các tập hợp giả định sẽ biện minh cho câu đó.

Các thuật toán chính xác được sử dụng để triển khai các hệ thống duy trì chân lý hơi phức tạp, và chúng ta không đề cập đến chúng ở đây. Độ phức tạp tính toán của vấn đề duy trì chân lý ít nhất cũng lớn như của suy luận mệnh đề—tức là NP-khó. Do đó, bạn không nên mong đợi duy trì chân lý là một giải pháp toàn năng. Tuy nhiên, khi được sử dụng cẩn thận, TMS có thể cung cấp một sự gia tăng đáng kể khả năng của một hệ thống logic để xử lý các môi trường và giả thuyết phức tạp.

Tóm tắt

Bằng cách đi sâu vào chi tiết cách biểu diễn nhiều loại tri thức khác nhau, chúng tôi hy vọng đã mang đến cho độc giả cái nhìn sâu sắc về cách xây dựng các cơ sở tri thức thực tế và cảm nhận về những vấn đề triết học thú vị nảy sinh. Những điểm chính bao gồm:

- Biểu diễn tri thức quy mô lớn đòi hỏi một bản thể luận tổng quát để tổ chức và liên kết các lĩnh vực tri thức cụ thể.
- Một bản thể luận tổng quát cần bao quát nhiều loại tri thức và về nguyên tắc, phải có khả năng xử lý bất kỳ lĩnh vực nào.
- Xây dựng một bản thể luận tổng quát lớn là một thách thức đáng kể và chưa được hoàn thiện đầy đủ, mặc dù các khuôn khổ hiện tại có vẻ khá vững chắc.
- Chúng tôi trình bày một bản thể luận cấp cao dựa trên các phạm trù và phép tính sự kiện. Chúng tôi đề cập đến các phạm trù, phạm trù con, bộ phận, đối tượng có cấu trúc, phép đo, chất liệu, sự kiện, thời gian và không gian, sự thay đổi, và niềm tin.
- Các loại tự nhiên không thể được định nghĩa hoàn toàn bằng logic, nhưng các thuộc tính của chúng có thể được biểu diễn.
- Hành động, sự kiện và thời gian có thể được biểu diễn bằng phép tính sự kiện. Cách biểu diễn này cho phép một tác nhân xây dựng chuỗi hành động và suy luận logic về những gì sẽ đúng khi các hành động đó xảy ra.
- Các hệ thống biểu diễn chuyên biệt, như mạng ngữ nghĩa và logic mô tả, được thiết kế để hỗ trợ tổ chức hệ thống phân cấp các phạm trù. Kế thừa là một hình thức suy luận quan trọng, cho phép suy ra thuộc tính của đối tượng từ việc chúng thuộc về các phạm trù nào đó.
- Giả định thế giới đóng, được triển khai trong các chương trình logic, cung cấp một cách đơn giản để tránh phải chỉ định nhiều thông tin phủ định. Nó được hiểu tốt nhất như một mặc định có thể bị ghi đè bởi thông tin bổ sung.
- Các logic không đơn điệu, như giới hạn hóa và logic mặc định, nhằm nắm bắt suy luận mặc định một cách tổng quát.
- Hệ thống duy trì sự thật xử lý các cập nhật và sửa đổi tri thức một cách hiệu quả.
- Việc xây dựng các bản thể luận lớn bằng tay rất khó khăn; trích xuất tri thức từ văn bản giúp công việc này dễ dàng hơn.

Ghi chú Tiểu sử và Lịch sử

Briggs (1985) cho rằng nghiên cứu biểu diễn tri thức bắt đầu từ lý thuyết ngữ pháp tiếng Phạn Shastric của Ấn Độ vào thiên niên kỷ thứ nhất trước Công nguyên. Các triết gia phương Tây truy nguyên công trình của họ về chủ đề này từ khoảng năm 300 trước Công nguyên trong tác phẩm *Siêu hình học* (nghĩa đen là những gì đến sau cuốn sách về vật lý) của Aristotle. Sự phát triển của thuật ngữ kỹ thuật trong bất kỳ lĩnh vực nào cũng có thể được coi là một hình thức biểu diễn tri thức.

Các cuộc thảo luận ban đầu về biểu diễn trong AI có xu hướng tập trung vào "biểu diễn vấn đề" hơn là "biểu diễn tri thức". (Xem ví dụ cuộc thảo luận của Amarel (1968) về vấn đề "Missionaries

and Cannibals".) Vào những năm 1970, AI nhấn mạnh vào việc phát triển các "hệ chuyên gia" (còn gọi là "hệ thống dựa trên tri thức") có thể, nếu được cung cấp tri thức miền phù hợp, sánh ngang hoặc vượt trội hiệu suất của các chuyên gia con người trong các nhiệm vụ được xác định hẹp. Ví dụ, hệ chuyên gia đầu tiên, DENDRAL (Feigenbaum et al., 1971; Lindsay et al., 1980), diễn giải đầu ra của máy quang phổ khối lượng (một loại dụng cụ dùng để phân tích cấu trúc các hợp chất hóa học hữu cơ) với độ chính xác ngang bằng các nhà hóa học chuyên nghiệp. Mặc dù thành công của DENDRAL đã góp phần thuyết phục cộng đồng nghiên cứu AI về tầm quan trọng của biểu diễn tri thức, các hình thức biểu diễn được sử dụng trong DENDRAL lại rất đặc thù cho lĩnh vực hóa học.

Theo thời gian, các nhà nghiên cứu bắt đầu quan tâm đến các hình thức biểu diễn tri thức tiêu chuẩn hóa và các bản thể luận có thể hỗ trợ việc tạo ra các hệ chuyên gia mới. Điều này đưa họ vào lãnh thổ trước đây đã được các triết gia khoa học và ngôn ngữ khám phá. Kỷ luật được áp đặt trong AI bởi nhu cầu các lý thuyết phải "hoạt động" đã dẫn đến tiến bộ nhanh chóng và sâu sắc hơn so với khi những vấn đề này chỉ thuộc phạm vi triết học (mặc dù đôi khi cũng dẫn đến việc tái phát minh ra bánh xe).

Nhưng chúng ta có thể tin tưởng vào tri thức chuyên gia đến mức nào? Từ năm 1955, Paul Meehl (xem thêm Grove and Meehl, 1996) đã nghiên cứu quá trình ra quyết định của các chuyên gia được đào tạo trong các nhiệm vụ chủ quan như dự đoán thành công của một sinh viên trong chương trình đào tạo hoặc khả năng tái phạm của một tội phạm. Trong 19 trong số 20 nghiên cứu mà ông xem xét, Meehl phát hiện ra rằng các thuật toán học thống kê đơn giản (như hồi quy tuyến tính hoặc naive Bayes) dự đoán tốt hơn các chuyên gia. Tetlock (2017) cũng nghiên cứu tri thức chuyên gia và nhận thấy nó thiếu sót trong các trường hợp khó. Dịch vụ Khảo thí Giáo dục (ETS) đã sử dụng một chương trình tự động để chấm điểm hàng triệu câu hỏi luận trong kỳ thi GMAT từ năm 1999. Chương trình này đồng ý với người chấm điểm 97% thời gian, tương đương với mức độ đồng thuận giữa hai người chấm điểm (Burstein et al., 2001). (Điều này không có nghĩa là chương trình hiểu các bài luận, chỉ là nó có thể phân biệt bài tốt và bài kém tốt như người chấm điểm.)

Việc tạo ra các phân loại hoặc hệ thống phân loại toàn diện có từ thời cổ đại. Aristotle (384–322 trước Công nguyên) nhấn mạnh mẽ vào các sơ đồ phân loại và phân nhóm. Tác phẩm *Organon* của ông, một tập hợp các công trình về logic do học trò biên soạn sau khi ông qua đời, bao gồm một chuyên luận có tên *Categories*, trong đó ông cố gắng xây dựng cái mà ngày nay chúng ta gọi là bản thể luận cấp cao. Ông cũng giới thiệu khái niệm chi và loài để phân loại ở cấp độ thấp hơn. Hệ thống phân loại sinh học hiện tại của chúng ta, bao gồm việc sử dụng "danh pháp nhị thức" (phân loại qua chi và loài theo nghĩa kỹ thuật), được phát minh bởi nhà sinh học người Thụy Điển Carolus Linnaeus, hay Carl von Linné (1707–1778). Các vấn đề liên quan đến loại tự nhiên và ranh giới không rõ ràng của phạm trù đã được giải quyết bởi Wittgenstein (1953), Quine (1953), Lakoff (1987) và Schwartz (1977), cùng nhiều người khác.

Xem Chương 25 để thảo luận về biểu diễn từ và khái niệm bằng mạng nơ-ron sâu, vốn tránh được một số vấn đề của bản thể luận nghiêm ngặt, nhưng cũng hy sinh một phần độ chính xác. Chúng ta vẫn chưa biết cách tốt nhất để kết hợp ưu điểm của mạng nơ-ron và ngữ nghĩa logic cho biểu diễn.

Mọi quan tâm đến các bản thể luận lớn hơn đang gia tăng, như được ghi nhận trong *Handbook on Ontologies* (Staab, 2004). Dự án OPENCYC (Lenat and Guha, 1990; Matuszek et al., 2006) đã phát hành một bản thể luận với 150.000 khái niệm, bao gồm một bản thể luận cấp cao tương tự

như trong Hình 10.1 cũng như các khái niệm cụ thể như "Màn hình OLED" và "iPhone", một loại "điện thoại di động", lần lượt là một loại "thiết bị điện tử tiêu dùng", "điện thoại", "thiết bị truyền thông không dây" và các khái niệm khác. Dự án NEXTKB mở rộng CYC và các tài nguyên khác bao gồm FrameNet và WordNet thành một cơ sở tri thức với gần 3 triệu sự kiện và cung cấp một công cụ suy luận, FIRE, đi kèm (Forbus et al., 2010).

Dự án DBpedia trích xuất dữ liệu có cấu trúc từ Wikipedia, cụ thể là từ các Infoboxes: các cặp thuộc tính/giá trị đi kèm với nhiều bài viết Wikipedia (Wu and Weld, 2008; Bizer et al., 2007). Tính đến năm 2015, DBpedia chứa 400 triệu sự kiện về 4 triệu đối tượng chỉ trong phiên bản tiếng Anh; tính tất cả 110 ngôn ngữ sẽ thu được 1,5 tỷ sự kiện (Lehmann et al., 2015).

Nhóm làm việc IEEE P1600.1 đã tạo ra SUMO, Bản thể luận cấp cao được đề xuất hợp nhất (Niles and Pease, 2001; Pease and Niles, 2002), với khoảng 1000 thuật ngữ trong bản thể luận cấp cao và liên kết đến hơn 20.000 thuật ngữ chuyên ngành. Stoffel et al. (1997) mô tả các thuật toán để quản lý hiệu quả một bản thể luận rất lớn. Một khảo sát về các kỹ thuật trích xuất tri thức từ trang web được trình bày bởi Etzioni et al. (2008).

Trên web, các ngôn ngữ biểu diễn đang xuất hiện. RDF (Brickley and Guha, 2004) cho phép đưa ra các khẳng định dưới dạng bộ ba quan hệ và cung cấp một số phương tiện để phát triển ý nghĩa của tên theo thời gian. OWL (Smith et al., 2004) là một logic mô tả hỗ trợ suy luận trên các bộ ba này. Cho đến nay, việc sử dụng dường như tỷ lệ nghịch với độ phức tạp của biểu diễn: các định dạng HTML và CSS truyền thống chiếm hơn 99% nội dung web, tiếp theo là các sơ đồ biểu diễn đơn giản nhất, như RDFa (Adida and Birbeck, 2008) và microformats (Khare, 2006; Patel-Schneider, 2014), sử dụng đánh dấu HTML và XHTML để thêm thuộc tính vào văn bản trên trang web. Việc sử dụng các bản thể luận RDF và OWL phức tạp vẫn chưa phổ biến, và tầm nhìn đầy đủ của *Semantic Web* (Berners-Lee et al., 2001) vẫn chưa được hiện thực hóa. Các hội nghị về *Formal Ontology in Information Systems* (FOIS) bao gồm cả bản thể luận tổng quát và chuyên ngành.

Bản thể luận được sử dụng trong chương này được phát triển bởi các tác giả và dựa một phần trên kinh nghiệm của họ trong dự án CYC và một phần trên công trình của Hwang and Schubert (1993) và Davis (1990, 2005). Một cuộc thảo luận truyền cảm hứng về dự án tổng thể của biểu diễn tri thức thông thường xuất hiện trong "Tuyên ngôn Vật lý Ngây thơ" của Hayes (1978, 1985b).

Các bản thể luận sâu thành công trong một lĩnh vực cụ thể bao gồm dự án *Gene Ontology* (Gene Ontology Consortium, 2008) và *Chemical Markup Language* (Murray-Rust et al., 2003). Những nghi ngờ về tính khả thi của một bản thể luận duy nhất cho mọi tri thức được bày tỏ bởi Doctorow (2001), Gruber (2004), Halevy et al. (2009) và Smith (2004).

Phép tính sự kiện được giới thiệu bởi Kowalski and Sergot (1986) để xử lý thời gian liên tục, và đã có nhiều biến thể (Sadri and Kowalski, 1995; Shanahan, 1997) và tổng quan (Shanahan, 1999; Mueller, 2006). James Allen giới thiệu khoảng thời gian vì lý do tương tự (Allen, 1984), lập luận rằng khoảng thời gian tự nhiên hơn nhiều so với tình huống để suy luận về các sự kiện kéo dài và đồng thời. Trong van Lambalgen and Hamm (2005), chúng ta thấy cách logic của sự kiện ánh xạ lên ngôn ngữ chúng ta sử dụng để nói về sự kiện. Một lựa chọn thay thế cho phép tính sự kiện và tình huống là phép tính fluent (Thielscher, 1999), tái hiện các sự kiện tạo nên trạng thái.

Peter Ladkin (1986a, 1986b) giới thiệu khoảng thời gian "lỗm" (khoảng thời gian có khoảng trống—về cơ bản là hợp của các khoảng thời gian "lồi" thông thường) và áp dụng các kỹ thuật của

đại số trừu tượng toán học để biểu diễn thời gian. Allen (1991) nghiên cứu có hệ thống nhiều kỹ thuật khác nhau có sẵn để biểu diễn thời gian; van Beek and Manchak (1996) phân tích các thuật toán cho suy luận thời gian. Có nhiều điểm tương đồng quan trọng giữa bản thể luận dựa trên sự kiện trong chương này và phân tích sự kiện của triết gia Donald Davidson (1980). Các lịch sử trong bản thể luận chất lỏng của Pat Hayes (1985a) và biên niên sử trong lý thuyết kế hoạch của McDermott (1985) cũng có ảnh hưởng quan trọng đến lĩnh vực và chương này.

Câu hỏi về trạng thái bản thể luận của chất liệu có một lịch sử lâu đời. Plato đề xuất rằng chất liệu là các thực thể trừu tượng hoàn toàn khác biệt với các đối tượng vật lý; ông sẽ nói *MadeOf(Butter₃, Butter)* thay vì *Butter₃ ∈ Butter*. Điều này dẫn đến một hệ thống phân cấp chất liệu trong đó, ví dụ, *UnsaltedButter* là một chất liệu cụ thể hơn *Butter*. Quan điểm được chấp nhận trong chương này, trong đó chất liệu là các phạm trù của đối tượng, được ủng hộ bởi Richard Montague (1973). Nó cũng được áp dụng trong dự án CYC. Copeland (1993) đưa ra một cuộc tấn công nghiêm túc, nhưng không phải là không thể vượt qua.

Cách tiếp cận thay thế được đề cập trong chương, trong đó bơ là một đối tượng bao gồm tất cả các đối tượng bơ trong vũ trụ, được đề xuất ban đầu bởi nhà logic học người Ba Lan Leśniewski (1916). *Mereology* của ông (tên bắt nguồn từ tiếng Hy Lạp có nghĩa là "phần") sử dụng quan hệ phần-toàn thể như một thay thế cho lý thuyết tập hợp toán học, với mục tiêu loại bỏ các thực thể trừu tượng như tập hợp. Một trình bày dễ hiểu hơn về các ý tưởng này được đưa ra bởi Leonard and Goodman (1940), và *The Structure of Appearance* của Goodman (1977) áp dụng các ý tưởng vào nhiều vấn đề trong biểu diễn tri thức.

Mặc dù một số khía cạnh của cách tiếp cận mereology là khó xử—ví dụ, nhu cầu về một cơ chế kế thừa riêng biệt dựa trên quan hệ phần-toàn thể—cách tiếp cận này đã nhận được sự ủng hộ của Quine (1960). Harry Bunt (1985) đã cung cấp một phân tích sâu rộng về việc sử dụng nó trong biểu diễn tri thức. Casati and Varzi (1999) bao gồm các phần, toàn thể và một lý thuyết tổng quát về vị trí không gian.

Có ba cách tiếp cận chính để nghiên cứu đối tượng tinh thần. Cách tiếp cận được thực hiện trong chương này, dựa trên logic hình thức và các thể giới có thể, là cách tiếp cận cổ điển từ triết học (Hintikka, 1962; Kripke, 1963; Hughes and Cresswell, 1996). Cuốn sách *Reasoning about Knowledge* (Fagin et al., 1995) cung cấp một giới thiệu kỹ lưỡng, và Gordon and Hobbs (2017) cung cấp *A Formal Theory of Commonsense Psychology*.

Cách tiếp cận thứ hai là một lý thuyết bậc nhất trong đó các đối tượng tinh thần là fluent. Davis (2005) và Davis and Morgenstern (2005) mô tả cách tiếp cận này. Nó dựa trên hình thức thể giới có thể và xây dựng dựa trên công trình của Robert Moore (1980, 1985).

Cách tiếp cận thứ ba là một lý thuyết cú pháp, trong đó các đối tượng tinh thần được biểu diễn bằng chuỗi ký tự. Một chuỗi chỉ là một thuật ngữ phức tạp biểu thị một danh sách các ký hiệu, vì vậy *CanFly(Clark)* có thể được biểu diễn bằng danh sách ký hiệu [C, a, n, F, l, y, (, C, l, a, r, k,)]. Lý thuyết cú pháp về đối tượng tinh thần lần đầu tiên được nghiên cứu sâu bởi Kaplan and Montague (1960), những người chỉ ra rằng nó dẫn đến nghịch lý nếu không được xử lý cẩn thận. Ernie Davis (1990) cung cấp một so sánh tuyệt vời giữa lý thuyết cú pháp và hình thức về tri thức. Pnueli (1977) mô tả một logic thời gian được sử dụng để suy luận về chương trình, công trình đã giúp ông giành giải Turing và được mở rộng bởi Vardi (1996). Littman et al. (2017) chỉ ra rằng logic thời gian có thể là một ngôn ngữ tốt để chỉ định mục tiêu cho robot học tăng cường theo cách dễ dàng cho con người chỉ định và tổng quát hóa tốt cho các môi trường khác nhau.

Triết gia Hy Lạp Porphyry (khoảng 234–305 sau Công nguyên), khi bình luận về *Categories* của Aristotle, đã vẽ cái có thể coi là mạng ngữ nghĩa đầu tiên. Charles S. Peirce (1909) phát triển đồ thị tồn tại như hình thức mạng ngữ nghĩa đầu tiên sử dụng logic hiện đại. Ross Quillian (1961), được thúc đẩy bởi sự quan tâm đến trí nhớ và xử lý ngôn ngữ của con người, đã khởi xướng công trình về mạng ngữ nghĩa trong AI. Một bài báo có ảnh hưởng của Marvin Minsky (1975) trình bày một phiên bản mạng ngữ nghĩa gọi là khung; một khung là biểu diễn của một đối tượng hoặc phạm trù, với các thuộc tính và quan hệ với các đối tượng hoặc phạm trù khác.

Vấn đề ngữ nghĩa nảy sinh khá gay gắt với mạng ngữ nghĩa của Quillian (và những người khác theo cách tiếp cận của ông), với các "liên kết IS-A" phổ biến và rất mơ hồ. Bài báo nổi tiếng của Bill Woods (1975) "What's In a Link?" đã thu hút sự chú ý của các nhà nghiên cứu AI về nhu cầu ngữ nghĩa chính xác trong các hình thức biểu diễn tri thức. Ron Brachman (1979) đã phân tích sâu hơn về điểm này và đề xuất các giải pháp. "The Logic of Frames" của Patrick Hayes (1979) đi sâu hơn nữa, tuyên bố rằng "Hầu hết 'khung' chỉ là một cú pháp mới cho các phần của logic bậc nhất." "Tarskian Semantics, or, No Notation without Denotation!" của Drew McDermott (1978b) lập luận rằng cách tiếp cận ngữ nghĩa lý thuyết mô hình được sử dụng trong logic bậc nhất nên được áp dụng cho tất cả các hình thức biểu diễn tri thức. Đây vẫn là một ý tưởng gây tranh cãi; đáng chú ý, chính McDermott đã đảo ngược lập trường của mình trong "A Critique of Pure Reason" (McDermott, 1987). Selman and Levesque (1993) thảo luận về độ phức tạp của kế thừa với ngoại lệ, chỉ ra rằng trong hầu hết các công thức, nó là NP-đầy đủ.

Logic mô tả được phát triển như một tập hợp con hữu ích của logic bậc nhất mà suy luận có thể tính toán được. Hector Levesque và Ron Brachman (1987) chỉ ra rằng một số cách sử dụng phép hoặc và phủ định chủ yếu chịu trách nhiệm cho tính không khả thi của suy luận logic. Điều này dẫn đến hiểu biết tốt hơn về sự tương tác giữa độ phức tạp và khả năng diễn đạt trong các hệ thống suy luận. Calvanese et al. (1999) tóm tắt tình hình hiện tại, và Baader et al. (2007) trình bày một cảm nang toàn diện về logic mô tả.

Ba hình thức chính để xử lý suy luận không đơn điệu—giới hạn hóa (McCarthy, 1980), logic mặc định (Reiter, 1980) và logic hình thức không đơn điệu (McDermott and Doyle, 1980)—đều được giới thiệu trong một số đặc biệt của *AI Journal*. Delgrande and Schaub (2003) thảo luận về ưu điểm của các biến thể, sau 25 năm nhìn lại. Lập trình tập hợp đáp ứng có thể được coi là một phần mở rộng của phủ định như thất bại hoặc một cải tiến của giới hạn hóa; lý thuyết cơ bản về ngữ nghĩa mô hình ổn định được giới thiệu bởi Gelfond and Lifschitz (1988), và các hệ thống lập trình tập hợp đáp ứng hàng đầu là DLV (Eiter et al., 1998) và SMOBELS (Niemelä et al., 2000). Brewka et al. (1997) cung cấp một cái nhìn tổng quan về các cách tiếp cận khác nhau đối với logic không đơn điệu. Clark (1978) bao gồm cách tiếp cận phủ định như thất bại đối với lập trình logic và hoàn thiện Clark. Lifschitz (2001) thảo luận về ứng dụng của lập trình tập hợp đáp ứng vào lập kế hoạch. Nhiều hệ thống suy luận không đơn điệu dựa trên lập trình logic được ghi lại trong các hội nghị về *Logic Programming and Nonmonotonic Reasoning* (LPNMR).

Nghiên cứu về hệ thống duy trì sự thật bắt đầu với các hệ thống TMS (Doyle, 1979) và RUP (McAllester, 1980), cả hai về cơ bản đều là JTMS. Forbus and de Kleer (1993) giải thích sâu về cách TMS có thể được sử dụng trong các ứng dụng AI. Nayak and Williams (1997) chỉ ra cách một TMS tăng hiệu quả gọi là ITMS giúp lập kế hoạch hoạt động của tàu vũ trụ NASA trong thời gian thực.

Chương này không thể bao quát mọi lĩnh vực của biểu diễn tri thức một cách sâu sắc. Ba chủ đề chính bị bỏ qua là:

Vật lý định tính: Vật lý định tính là một lĩnh vực con của biểu diễn tri thức tập trung cụ thể vào việc xây dựng một lý thuyết logic, không số về các đối tượng và quá trình vật lý. Thuật ngữ này được đặt ra bởi Johan de Kleer (1975), mặc dù công việc này có thể nói là bắt đầu từ BUILD của Fahlman (1974), một trình lập kế hoạch phức tạp để xây dựng các tòa tháp khối phức tạp. Fahlman phát hiện ra trong quá trình thiết kế rằng phần lớn nỗ lực (80%, theo ước tính của ông) dành cho việc mô hình hóa vật lý của thế giới khối để tính toán độ ổn định của các cụm khối khác nhau, thay vì cho lập kế hoạch. Ông phác thảo một quá trình giống vật lý ngây thơ để giải thích tại sao trẻ nhỏ có thể giải quyết các vấn đề giống BUILD mà không cần truy cập vào số học dấu phẩy động tốc độ cao được sử dụng trong mô hình vật lý của BUILD. Hayes (1985a) sử dụng "lịch sử"—các lát cắt bốn chiều của không-thời gian tương tự như sự kiện của Davidson—để xây dựng một vật lý ngây thơ khá phức tạp về chất lỏng. Davis (2008) cung cấp một cập nhật cho bản thể luận chất lỏng mô tả việc đổ chất lỏng vào bình chứa.

De Kleer and Brown (1985), Ken Forbus (1985) và Benjamin Kuipers (1985) độc lập và gần như đồng thời phát triển các hệ thống có thể suy luận về một hệ thống vật lý dựa trên các trừu tượng định tính của các phương trình cơ bản. Vật lý định tính sớm phát triển đến mức có thể phân tích nhiều hệ thống vật lý phức tạp (Yip, 1991). Các kỹ thuật định tính đã được sử dụng để thiết kế đồng hồ, cần gạt nước và robot sáu chân (Subramanian and Wang, 1994). Tuyển tập *Readings in Qualitative Reasoning about Physical Systems* (Weld and de Kleer, 1990), một bài bách khoa toàn thư của Kuipers (2001) và một bài viết trong sổ tay của Davis (2007) cung cấp giới thiệu tốt về lĩnh vực này.

Suy luận không gian: Suy luận cần thiết để điều hướng trong thế giới wumpus là tầm thường so với cấu trúc không gian phong phú của thế giới thực. Nỗ lực nghiêm túc đầu tiên để nắm bắt suy luận thông thường về không gian xuất hiện trong công trình của Ernest Davis (1986, 1990). Phép tính kết nối vùng của Cohn et al. (1997) hỗ trợ một dạng suy luận không gian định tính và đã dẫn đến các loại hệ thống thông tin địa lý mới; xem thêm (Davis, 2006). Như với vật lý định tính, một tác nhân có thể đi một chặng đường dài mà không cần dùng đến biểu diễn số liệu đầy đủ.

Lập luận tâm lý: Lập luận tâm lý liên quan đến việc phát triển một mô hình tâm lý học ứng dụng để các tác nhân nhân tạo sử dụng khi lập luận về bản thân và các tác nhân khác. Điều này thường dựa trên cái gọi là tâm lý học dân gian (folk psychology) – lý thuyết mà con người nói chung được cho là sử dụng khi suy luận về bản thân và người khác. Khi các nhà nghiên cứu AI trang bị cho tác nhân nhân tạo của họ các lý thuyết tâm lý để lập luận về các tác nhân khác, những lý thuyết này thường dựa trên mô tả của chính nhà nghiên cứu về thiết kế của các tác nhân logic. Hiện nay, lập luận tâm lý hữu ích nhất trong bối cảnh hiểu ngôn ngữ tự nhiên, nơi việc suy đoán ý định của người nói có tầm quan trọng hàng đầu.

Minker (2001) tập hợp các bài nghiên cứu của các học giả hàng đầu trong lĩnh vực biểu diễn tri thức, tổng kết 40 năm làm việc trong ngành. Các kỹ yếu của hội nghị quốc tế về Nguyên tắc Biểu diễn Tri thức và Lập luận (Principles of Knowledge Representation and Reasoning) là nguồn tài liệu cập nhật nhất về các công trình trong lĩnh vực này. Hai tuyển tập xuất sắc về biểu diễn tri thức là *Readings in Knowledge Representation* (Brachman và Levesque, 1985) và *Formal Theories of the Commonsense World* (Hobbs và Moore, 1985); cuốn đầu tập trung nhiều hơn vào các bài báo quan trọng mang tính lịch sử về ngôn ngữ và hình thức biểu diễn, trong khi cuốn sau chú trọng

vào việc tích lũy bản thân tri thức. Davis (1990), Stefik (1995) và Sowa (1999) cung cấp các giáo trình nhập môn về biểu diễn tri thức, van Harmelen và cộng sự (2007) đóng góp một cuốn sổ tay, còn Davis và Morgenstern (2004) biên tập một số chuyên đề của Tạp chí AI về chủ đề này. Davis (2017) tổng quan về logic cho lập luận thông thường. Hội nghị song niên *Theoretical Aspects of Reasoning About Knowledge* (TARK) bao gồm các ứng dụng lý thuyết tri thức trong AI, kinh tế và hệ thống phân tán.